
Fundamentals of C++ Programming

DRAFT

Richard L. Halterman
School of Computing
Southern Adventist University

January 26, 2014

Copyright © 2008–2014 Richard L. Halterman. All rights reserved.

Contents

1	The Context of Software Development	1
1.1	Software	2
1.2	Development Tools	2
1.3	Learning Programming with C++	5
1.4	Summary	6
1.5	Exercises	6
2	Writing a C++ Program	7
2.1	General Structure of a Simple C++ Program	7
2.2	Editing, Compiling, and Running the Program	9
2.3	Variations of our simple program	9
2.4	Template for simple C++ programs	12
2.5	Summary	13
2.6	Exercises	14
3	Values and Variables	15
3.1	Integer Values	15
3.2	Variables and Assignment	17
3.3	Identifiers	20
3.4	Floating-point Types	23
3.5	Constants	25
3.6	Other Numeric Types	25
3.7	Characters	26
3.8	Enumerated Types	28
3.9	Type Inference with auto	29
3.10	Summary	30

3.11 Exercises	32
4 Expressions and Arithmetic	35
4.1 Expressions	35
4.2 Mixed Type Expressions	41
4.3 Operator Precedence and Associativity	44
4.4 Comments	46
4.5 Formatting	47
4.6 Errors and Warnings	50
4.6.1 Compile-time Errors	50
4.6.2 Run-time Errors	51
4.6.3 Logic Errors	53
4.6.4 Compiler Warnings	53
4.7 Arithmetic Examples	55
4.8 More Arithmetic Operators	58
4.9 Algorithms	61
4.10 Summary	62
4.11 Exercises	64
5 Conditional Execution	71
5.1 Type bool	71
5.2 Boolean Expressions	72
5.3 The Simple if Statement	74
5.4 Compound Statements	77
5.5 The if/else Statement	78
5.6 Compound Boolean Expressions	82
5.7 Nested Conditionals	86
5.8 Multi-way if/else Statements	97
5.9 Errors in Conditional Statements	101
5.10 Summary	103
5.11 Exercises	104
6 Iteration	109
6.1 The while Statement	109
6.2 Nested Loops	119

6.3	Abnormal Loop Termination	127
6.3.1	The break statement	127
6.3.2	The goto Statement	128
6.3.3	The continue Statement	130
6.4	Infinite Loops	132
6.5	Iteration Examples	135
6.5.1	Drawing a Tree	136
6.5.2	Printing Prime Numbers	138
6.6	Summary	141
6.7	Exercises	142
7	Other Conditional and Iterative Statements	147
7.1	The switch Statement	147
7.2	The Conditional Operator	152
7.3	The do/while Statement	153
7.4	The for Statement	155
7.5	Summary	162
7.6	Exercises	163
8	Using Functions	167
8.1	Introduction to Using Functions	169
8.2	Standard Math Functions	174
8.3	Maximum and Minimum	178
8.4	clock Function	178
8.5	Character Functions	180
8.6	Random Numbers	182
8.7	Summary	185
8.8	Exercises	186
9	Writing Functions	189
9.1	Function Basics	190
9.2	Using Functions	198
9.3	Pass by Value	203
9.4	Function Examples	204
9.4.1	Better Organized Prime Generator	204

9.4.2	Command Interpreter	206
9.4.3	Restricted Input	207
9.4.4	Better Die Rolling Simulator	209
9.4.5	Tree Drawing Function	211
9.4.6	Floating-point Equality	212
9.4.7	Multiplication Table with Functions	214
9.5	Commenting Functions	217
9.6	Custom Functions vs. Standard Functions	219
9.7	Summary	221
9.8	Exercises	222
10	Managing Functions and Data	227
10.1	Global Variables	227
10.2	Static Variables	235
10.3	Overloaded Functions	238
10.4	Recursion	238
10.5	Making Functions Reusable	244
10.6	Pointers	249
10.7	Reference Variables	254
10.8	Pass by Reference	256
10.8.1	Pass by Reference via Pointers	257
10.8.2	Pass by Reference via References	259
10.9	Higher-order Functions	260
10.10	Summary	262
10.11	Exercises	264
11	Aggregate Data	271
11.1	Vectors	273
11.1.1	Declaring and Using Vectors	273
11.1.2	Traversing a Vector	277
11.1.3	Vector Methods	281
11.1.4	Vectors and Functions	284
11.1.5	Multidimensional Vectors	289
11.2	Arrays	292
11.3	Prime Generation with a Vector	299

11.4 Summary	302
11.5 Exercises	304
12 Sorting and Searching	307
12.1 Sorting	307
12.2 Flexible Sorting	310
12.3 Search	312
12.3.1 Linear Search	312
12.3.2 Binary Search	315
12.4 Vector Permutations	325
12.5 Randomly Permuting a Vector	331
12.6 Summary	337
12.7 Exercises	338
13 Standard C++ Classes	341
13.1 String Objects	342
13.2 Input/Output Streams	346
13.3 File Streams	349
13.4 Complex Numbers	353
13.5 Better Pseudorandom Number Generation	354
13.6 Summary	362
13.7 Exercises	362
14 Custom Objects	363
14.1 Object Basics	363
14.2 Fields	365
14.3 Methods	370
14.4 Constructors	377
14.5 Destructors	380
14.6 Defining a New Numeric Type	381
14.7 Encapsulation	383
14.8 Summary	386
14.9 Exercises	387
15 Fine Tuning Objects	393
15.1 Pointers to Objects and Object Arrays	393

15.2	The <code>this</code> Pointer	396
15.3	<code>const</code> Methods	398
15.4	Separating Method Declarations and Definitions	400
15.5	Preventing Multiple Inclusion	406
15.6	Overloaded Operators	410
15.6.1	Operator Functions	410
15.6.2	Operator Methods	413
15.7	<code>static</code> Members	414
15.8	Classes vs. structs	418
15.9	Summary	419
15.10	Exercises	419
16	Building some Useful Classes	421
16.1	A Better Rational Number Class	421
16.2	Stopwatch	423
16.3	Sorting with Logging	429
16.4	Linked Lists	433
16.5	Automating Testing	443
16.6	Convenient High-quality Pseudorandom Numbers	448
16.7	Summary	450
16.8	Exercises	450
17	Inheritance and Polymorphism	453
17.1	I/O Stream Inheritance	453
17.2	Inheritance Mechanics	455
17.3	Uses of Inheritance	458
17.4	Polymorphism	465
17.5	Protected Members	471
17.6	Fine Tuning Inheritance	479
17.7	Summary	489
17.8	Exercises	490
18	Generic Programming	493
18.1	Function Templates	493
18.2	Class Templates	504

18.3 Iterators	514
18.4 Iterator Ranges	518
18.5 The Standard Template Library	526
18.6 Summary	526
18.7 Exercises	526
19 Handling Exceptions	527
19.1 Motivation	527
19.2 Exception Examples	528
19.3 Custom Exceptions	536
19.4 Catching Multiple Exceptions	537
19.5 Exception Mechanics	541
19.6 Using Exceptions	544
19.7 Summary	545
19.8 Exercises	546
20 Concurrent Programming	547
20.1 Motivation	547
20.2 Summary	547
20.3 Exercises	547
Appendices	551
A Using Visual Studio 2013 to Develop C++ Programs	551
B Visual C++ Command Line Development	557
C Developing C++ Programs with the GNU Tools	561
D Introduction to Binary Numbers and Computer Arithmetic	565
D.1 The Integer Types	565
D.2 The Floating-Point Types	570
E Bitwise Operators	575
F Arrays	581
F.1 Declaring and Using Arrays	581
F.2 Arrays and Functions	588

F.3	Prime Generation with an Array	591
F.4	Multidimensional Arrays	592
F.5	Pointers and Arrays	595
F.6	Array Ranges	599
F.7	C Strings	600
F.8	Dynamic Arrays	604
F.9	The sizeof Operator	608
F.10	Copying an Array	610
F.11	Memory Management	612
F.12	Summary	616
F.13	Exercises	617
Bibliography		620
Index		621

Preface

Legal Notices and Information

Permission is hereby granted to make hardcopies and freely distribute the material herein under the following conditions:

- The copyright and this legal notice must appear in any copies of this document made in whole or in part.
- None of material herein can be sold or otherwise distributed for commercial purposes without written permission of the copyright holder.
- Instructors at any educational institution may freely use this document in their classes as a primary or optional textbook under the conditions specified above.

A local electronic copy of this document may be made under the terms specified for hard copies:

- The copyright and these terms of use must appear in any electronic representation of this document made in whole or in part.
- None of material herein can be sold or otherwise distributed in an electronic form for commercial purposes without written permission of the copyright holder.
- Instructors at any educational institution may freely store this document in electronic form on a local server as a primary or optional textbook under the conditions specified above.

Additionally, a hardcopy or a local electronic copy must contain the uniform resource locator (URL) providing a link to the original content so the reader can check for updated and corrected content. The current standard URL is <http://python.cs.southern.edu/cppbook/progcpp.pdf>.

Chapter 1

The Context of Software Development

A computer program, from one perspective, is a sequence of instructions that dictate the flow of electrical impulses within a computer system. These impulses affect the computer's memory and interact with the display screen, keyboard, mouse, and perhaps even other computers across a network in such a way as to produce the “magic” that permits humans to perform useful tasks, solve high-level problems, and play games. One program allows a computer to assume the role of a financial calculator, while another transforms the machine into a worthy chess opponent. Note the two extremes here:

- at the lower, more concrete level electrical impulses alter the internal state of the computer, while
- at the higher, more abstract level computer users accomplish real-world work or derive actual pleasure.

So well is the higher-level illusion achieved that most computer users are oblivious to the lower-level activity (the machinery under the hood, so to speak). Surprisingly, perhaps, most programmers today write software at this higher, more abstract level also. An accomplished computer programmer can develop sophisticated software with little or no interest or knowledge of the actual computer system upon which it runs. Powerful software construction tools hide the lower-level details from programmers, allowing them to solve problems in higher-level terms.

The concepts of computer programming are logical and mathematical in nature. In theory, computer programs can be developed without the use of a computer. Programmers can discuss the viability of a program and reason about its correctness and efficiency by examining abstract symbols that correspond to the features of real-world programming languages but appear in no real-world programming language. While such exercises can be very valuable, in practice computer programmers are not isolated from their machines. Software is written to be used on real computer systems. Computing professionals known as *software engineers* develop software to drive particular systems. These systems are defined by their underlying hardware and operating system. Developers use concrete tools like compilers, debuggers, and profilers. This chapter examines the context of software development, including computer systems and tools.

1.1 Software

A computer program is an example of computer *software*. Software makes a computer a truly universal machine transforming it into the proper tool for the task at hand. One can refer to a program as a *piece* of software as if it were a tangible object, but software is actually quite intangible. It is stored on a *medium*. A hard drive, a CD, a DVD, and a USB pen drive are all examples of media upon which software can reside. The CD is not the software; the software is a pattern on the CD. In order to be used, software must be stored in the computer's memory. Typically computer programs are loaded into memory from a medium like the computer's hard disk. An electromagnetic pattern representing the program is stored on the computer's hard drive. This pattern of electronic symbols must be transferred to the computer's memory before the program can be executed. The program may have been installed on the hard disk from a CD or from the Internet. In any case, the essence that was transferred from medium to medium was a pattern of electronic symbols that direct the work of the computer system.

These patterns of electronic symbols are best represented as a sequence of zeroes and ones, digits from the binary (base 2) number system. An example of a binary program sequence is

```
10001011011000010001000001001110
```

To the underlying computer hardware, specifically the processor, a zero here and three ones there might mean that certain electrical signals should be sent to the graphics device so that it makes a certain part of the display screen red. Unfortunately, only a minuscule number of people in the world would be able to produce, by hand, the complete sequence of zeroes and ones that represent the program Microsoft Word for an Intel-based computer running the Windows 8 operating system. Further, almost none of those who could produce the binary sequence would claim to enjoy the task.

The Word program for older Mac OS X computers using a PowerPC processor works similarly to the Windows version and indeed is produced by the same company, but the program is expressed in a completely different sequence of zeroes and ones! The Intel Core i7 processor in the Windows machine accepts a completely different binary language than the PowerPC processor in the Mac. We say the processors have their own *machine language*.

1.2 Development Tools

If very few humans can (or want) to speak the machine language of the computers' processors and software is expressed in this language, how has so much software been developed over the years?

Software can be represented by printed words and symbols that are easier for humans to manage than binary sequences. Tools exist that automatically convert a higher-level description of what is to be done into the required lower-level code. Higher-level programming languages like C++ allow programmers to express solutions to programming problems in terms that are much closer to a natural language like English. Some examples of the more popular of the hundreds of higher-level programming languages that have been devised over the past 60 years include FORTRAN, COBOL, Lisp, Haskell, C, Perl, Python, Java, and C#. Most programmers today, especially those concerned with high-level applications, usually do not worry about the details of underlying hardware platform and its machine language.

One might think that ideally such a conversion tool would accept a description in a natural language, such as English, and produce the desired executable code. This is not possible today because natural languages are quite complex compared to computer programming languages. Programs called *compilers* that translate one computer language into another have been around for over 60 years, but natural language

processing is still an active area of artificial intelligence research. Natural languages, as they are used by most humans, are inherently ambiguous. To understand properly all but a very limited subset of a natural language, a human (or artificially intelligent computer system) requires a vast amount of background knowledge that is beyond the capabilities of today's software. Fortunately, programming languages provide a relatively simple structure with very strict rules for forming statements that can express a solution to any problem that can be solved by a computer.

Consider the following program fragment written in the C++ programming language:

```
subtotal = 25;
tax = 3;
total = subtotal + tax;
```

These three lines do not make up a complete C++ program; they are merely a piece of a program. The statements in this program fragment look similar to expressions in algebra. We see no sequence of binary digits. Three words, **subtotal**, **tax**, and **total**, called *variables*, are used to hold information. Mathematicians have used variables for hundreds of years before the first digital computer was built. In programming, a variable represents a value stored in the computer's memory. Familiar operators (= and +) are used instead of some cryptic binary digit sequence that instructs the processor to perform the operation. Since this program is expressed in the C++ language, not machine language, it cannot be executed directly on any processor. A C++ compiler is used to translate the C++ code into machine code.

The higher-level language code is called *source code*. The compiled machine language code is called the *target code*. The compiler translates the source code into the target machine language.

The beauty of higher-level languages is this: the same C++ source code can be compiled to different target platforms. The target platform must have a C++ compiler available. Minor changes in the source code may be required because of architectural differences in the platforms, but the work to move the program from one platform to another is far less than would be necessary if the program for the new platform had to be rewritten by hand in the new machine language. Just as importantly, when writing the program the human programmer is free to think about writing the solution to the problem in C++, not in a specific machine language.

Programmers have a variety of tools available to enhance the software development process. Some common tools include:

- **Editors.** An *editor* allows the user to enter the program source code and save it to files. Most programming editors increase programmer productivity by using colors to highlight language features. The *syntax* of a language refers to the way pieces of the language are arranged to make well-formed sentences. To illustrate, the sentence

The tall boy runs quickly to the door.

uses proper English syntax. By comparison, the sentence

Boy the tall runs door to quickly the.

is not correct syntactically. It uses the same words as the original sentence, but their arrangement does not follow the rules of English.

Similarly, programmers must follow strict syntax rules to create well-formed computer programs. Only well-formed programs are acceptable and can be compiled and executed. Some syntax-aware editors can use colors or other special annotations to alert programmers of syntax errors before the program is compiled.

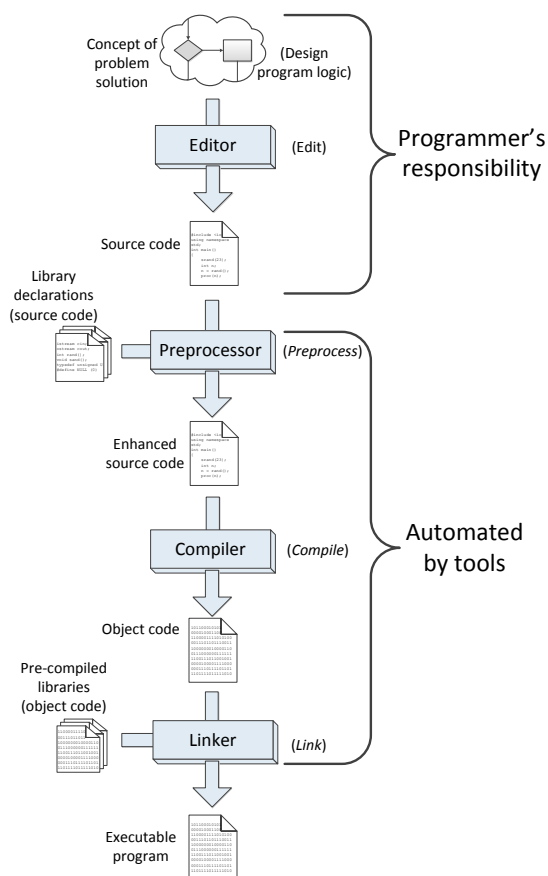


Figure 1.1: Source code to target code sequence

- **Compilers.** A *compiler* translates the source code to target code. The target code may be the machine language for a particular platform or embedded device. The target code could be another source language; for example, the earliest C++ compiler translated C++ into C, another higher-level language. The resulting C code was then processed by a C compiler to produce an executable program. C++ compilers today translate C++ directly into machine language.

The complete set of build tools for C++ includes a preprocessor, compiler, and linker:

- **Preprocessor**—adds to or modifies the contents of the source file before the compiler begins processing the code. We use the services of the preprocessor mainly to **#include** information about library routines our programs use.
- **Compiler**—translates C++ source code to machine code.
- **Linker**—combines the compiler-generated machine code with precompiled library code or compiled code from other sources to make a complete executable program. Most compiled C++ code is incapable of running by itself and needs some additional machine code to make a complete executable program. The missing machine code has been precompiled and stored in a repository of code called a *library*. A program called a *linker* combines the programmer's compiled code and the library code to make a complete program.

We generally do not think about the preprocessor, compiler, and linker working as three separate programs (although they do); the tools we use make it appear as only one process is taking place: translating our source code to an executable program.

- **Debuggers.** A *debugger* allows a programmer to more easily trace a program's execution in order to locate and correct errors in the program's implementation. With a debugger, a developer can simultaneously run a program and see which line in the source code is responsible for the program's current actions. The programmer can watch the values of variables and other program elements to see if their values change as expected. Debuggers are valuable for locating errors (also called *bugs*) and repairing programs that contain errors. (See Section 4.6 for more information about programming errors.)
- **Profilers.** A *profiler* collects statistics about a program's execution allowing developers to tune appropriate parts of the program to improve its overall performance. A profiler indicates how many times a portion of a program is executed during a particular run, and how long that portion takes to execute. Profilers also can be used for testing purposes to ensure all the code in a program is actually being used somewhere during testing. This is known as *coverage*. It is common for software to fail after its release because users exercise some part of the program that was not executed anytime during testing. The main purpose of profiling is to find the parts of a program that can be improved to make the program run faster.

The programming components of the development process are illustrated in Figure 1.1.

Many developers use integrated development environments (IDEs). An IDE includes editors, debuggers, and other programming aids in one comprehensive program. Examples of IDEs for C++ include Microsoft's Visual Studio 2013, the Eclipse Foundation's Eclipse CDT, and Apple's XCode.

Despite the plethora of tools (and tool vendors' claims), the programming process for all but trivial programs is not automatic. Good tools are valuable and certainly increase the productivity of developers, but they cannot write software. There are no substitutes for sound logical thinking, creativity, common sense, and, of course, programming experience.

1.3 Learning Programming with C++

Bjarne Stroustrup of AT&T Bell Labs created C++ in the mid 1980s. C++ is an extension of the programming language C, a product of AT&T Bell Labs from the early 1970s. C was developed to write the Unix operating system, and C is widely used for systems-level software and embedded systems development. C++ initially provided object-oriented programming features (see Chapter 13 and Chapter 14) and later added generic programming capabilities. C++'s close relationship to C allows C++ programs to utilize a large collection of code developed in C.

C++ is widely used in industry for commercial software development. It is an industrial strength programming language used for developing complex systems in business, science, and engineering. Examples of software written in C++ include Microsoft Windows 8, Microsoft Office, Mac OS X, and Adobe Creative Suite.

In order to meet the needs of commercial software development and accomplish all that it does, C++ itself is complex. While experienced programmers can accomplish great things with C++, beginners sometimes have a difficult time with it. Professional software developers enjoy the flexible design options that C++ permits, but beginners need more structure and fewer options so they can master simpler concepts before moving on to more complex ones.

This book does not attempt to cover all the facets of the C++ programming language. Experienced programmers should look elsewhere for books that cover C++ in much more detail. The focus here is on introducing programming techniques and developing good habits. To that end, our approach avoids some of the more esoteric features of C++ and concentrates on the programming basics that transfer directly to other imperative programming languages such as Java, C#, and Python. We stick with the basics and explore more advanced features of C++ only when necessary to handle the problem at hand.

1.4 Summary

- Computers require both hardware and software to operate. Software consists of instructions that control the hardware.
- At the lowest level, the instructions for a computer program can be represented as a sequence of zeros and ones. The pattern of zeros and ones determine the instructions performed by the processor.
- Two different kinds of processors can have different machine languages.
- Application software can be written largely without regard to the underlying hardware. A tool called a compiler translates the higher-level, abstract language into the machine language required by the hardware.
- Programmers develop software using tools such as editors, compilers, debuggers, and profilers.
- C++ is a higher-level programming language.
- An IDE is an integrated development environment—one program that provides all the tools that developers need to write software.

1.5 Exercises

1. What is a compiler?
2. How is compiled code different from source code?
3. What tool does a programmer use to produce C++ source code?
4. What tool(s) does a programmer use to convert C++ source code into executable machine code?
5. What does the linker do?
6. Does the linker deal with files containing source code or machine language code?
7. What does the preprocessor do to source code?
8. List several advantages developing software in a higher-level language has over developing software in machine language.
9. How can an IDE improve a programmer's productivity?
10. Name a popular C++ IDE is used by programmers developing for Microsoft Windows.
11. Name a popular C++ IDE is used by programmers developing for Apple Mac OS X.

Chapter 2

Writing a C++ Program

Properly written C++ programs have a particular structure. The syntax must be correct, or the compiler will generate error messages and not produce executable machine language. This chapter introduces C++ by providing some simple example programs and associated fundamental concepts. Most of the concepts presented in this chapter are valid in many other programming languages as well. While other languages may implement the concepts using slightly different syntax, the ideas are directly transferable to other languages like C, Java, C#, and Ada.

2.1 General Structure of a Simple C++ Program

Listing 2.1 (simple.cpp) is one of the simplest C++ programs that does something:

Listing 2.1: simple.cpp

```
#include <iostream>

using namespace std;

int main() {
    cout << "This is a simple C++ program!" << endl;
}
```

You can type the text as shown in Listing 2.1 (simple.cpp) into an editor and save it to a file named simple.cpp. The actual name of the file is irrelevant, but the name “simple” accurately describes the nature of this program. The extension .cpp is a common extension used for C++ source code.

After creating this file with a text editor and compiling it, you can run the program. The program prints the message

```
This is a simple C++ program!
```

Listing 2.1 (simple.cpp) contains five non-blank lines of code:

- **#include <iostream>**

This line is a preprocessing directive. All preprocessing directives within C++ source code begin with a **#** symbol. This one directs the preprocessor to add some predefined source code to our existing source code before the compiler begins to process it. This process is done automatically and is invisible to us.

Here we want to use some parts of the **iostream** library, a collection precompiled C++ code that C++ programs (like ours) can use. The **iostream** library contains routines that handle input and output (I/O) that include functions such as printing to the display, getting user input from the keyboard, and dealing with files.

Two items used in Listing 2.1 (simple.cpp), **cout** and **endl**, are not part of the C++ language itself. These items, along with many other things related to input and output, were developed in C++, compiled, and stored in the **iostream** library. The compiler needs to be aware of these **iostream** items so it can compile our program. The **#include** directive specifies a file, called a *header*, that contains the specifications for the library code. The compiler checks how we use **cout** and **endl** within our code against the specifications in the **<iostream>** header to ensure that we are using the library code correctly.

Most of the programs we write use this **#include <iostream>** directive, and some programs we will write in the future will **#include** other headers as well.

- **using namespace std;**

The two items our program needs to display a message on the screen, **cout** and **endl**, have longer names: **std::cout** and **std::endl**. This **using namespace std** directive allows us to omit the **std::** prefix and use their shorter names. This directive is optional, but if we omit it, we must use the longer names. Listing 2.2 (simple2.cpp) shows how the longer names are used. The name **std** stands for “standard,” and the **using namespace std** line indicates that some of the names we use in our program are part of the so-called “standard namespace.”

- **int main() {**

This specifies the real beginning of our program. Here we are declaring a function named **main**. All C++ programs must contain this function to be executable. Details about the meaning of **int** and the parentheses will appear in later chapters. More general information about functions appear in Chapter 8 and Chapter 9.

The opening curly brace at the end of the line marks the beginning of the body of a function. The body of a function contains the statements the function is to execute.

- **cout << "This is a simple C++ program!" << endl;**

The body of our **main** function contains only one statement. This statement directs the executing program to print the message *This is a simple C++ program!* on the screen. A statement is the fundamental unit of execution in a C++ program. Functions contain statements that the compiler translates into executable machine language instructions. C++ has a variety of different kinds of statements, and the chapters that follow explore these various kinds of statements. All statements in C++ end with a semicolon (;). A more detailed explanation of this statement appears below.

- **}**

The closing curly brace marks the end of the body of a function. Both the open curly brace and close curly brace are required for every function definition.



Note which lines in the program end with a semicolon (;) and which do not. Do not put a semicolon after the `#include` preprocessor directive. Do not put a semicolon on the line containing `main`, and do not put semicolons after the curly braces.

2.2 Editing, Compiling, and Running the Program

C++ programmers have two options for C++ development environments. One option involves a command-line environment with a collection of independent tools. The other option is to use an IDE (see Section 1.2) which combines all the tools into a convenient package. Visual Studio is the dominant IDE on the Microsoft Windows platform, and Apple Mac developers often use the XCode IDE. Appendix A provides an overview of how to use the Visual Studio 2013 IDE to develop a simple C++ program.

The myriad of features and configuration options in these powerful IDEs can be bewildering to those learning how to program. In a command-line environment the programmer needs only type a few simple commands into a console window to edit, compile, and execute programs. Some developers prefer the simplicity and flexibility of command-line build environments, especially for less complex projects.

One prominent command-line build system is the GNU Compiler Collection (<http://gcc.gnu.org>), or GCC for short. The GCC C++ compiler, called `g++`, is one of most C++ standards conforming compilers available. The GCC C++ compiler toolset is available for the Microsoft Windows, Apple Mac, and Linux platforms, and it is a free, open-source software project with a world-wide development team. Appendix C provides an overview of how to use the GCC C++ compiler.

Visual Studio and XCode offer *command line* development options as well. Appendix B provides an overview of the Visual Studio command line development process.

2.3 Variations of our simple program

The two items Listing 2.1 (`simple.cpp`) needed to display a message on the screen, `cout` and `endl`, have longer names: `std::cout` and `std::endl`. The `using namespace std` directive allows us to omit the `std::` prefixes and use their shorter names. This directive is optional, but if it is omitted, the longer names are required. For example, Listing 2.2 (`simple2.cpp`) shows an alternative way of writing Listing 2.1 (`simple.cpp`).

Listing 2.2: `simple2.cpp`

```
#include <iostream>

int main() {
    std::cout << "This is a simple C++ program!" << std::endl;
}
```

Listing 2.3 (`simple3.cpp`) shows another way to use the shorter names for `cout` and `endl` within a C++ program.

Listing 2.3: `simple3.cpp`

```
#include <iostream>

using std::cout;
using std::endl;

int main() {
    cout << "This is a simple C++ program!" << endl;
}
```

We generally will not use the third approach, although you will encounter it in published C++ code. The compiler will generate the same machine language code for all three versions. We generally will write programs that use the `using namespace` directive and, therefore, use the shorter names.

The statement in the `main` function in any of the three versions of our program uses the services of an object called `std::cout`. The `std::cout` object prints text on the computer's screen. The text of the message as it appears in the C++ source code is called a *string*, for *string of characters*. Strings are enclosed within quotation marks (`"`). The symbols `<<` make up the *insertion operator*. You can think of the message to be printed as being “inserted” into the `cout` object. The `cout` object represents the output stream; that is, text that the program prints to the console window. The `endl` word means “the end of line of printed text,” and it causes the cursor to move down to the next line so any subsequent output will appear on the next line. If you read the statement from left to right, the `cout` object, which is responsible for displaying text on the screen, first receives the text to print and then receives the end-of-line directive to move to the next line.

For simplicity, we'll refer to this type of statement as a *print statement*, even though the word *print* does not appear anywhere in the statement.

With minor exceptions, any statement in C++ must appear within a function definition. Our single print statement appears within the function named `main`.

Any function, including `main`, may contain multiple statements. In Listing 2.4 (`arrow.cpp`), six print statements draw an arrow on the screen:

Listing 2.4: `arrow.cpp`

```
#include <iostream>

using namespace std;

int main() {
    cout << "  *  " << endl;
    cout << " *** " << endl;
    cout << " ***** " << endl;
    cout << "  *  " << endl;
    cout << "  *  " << endl;
    cout << "  *  " << endl;
}
```

The output of Listing 2.4 (`arrow.cpp`) is



Each print statement “draws” a horizontal slice of the arrow. The six statements

```
cout << "    *    " << endl;
cout << "   ***   " << endl;
cout << "  ***** " << endl;
cout << "    *    " << endl;
cout << "     *    " << endl;
cout << "     *    " << endl;
```

constitute the *body* of the **main** function. The body consists of all the statements between the open curly brace ({) and close curly brace (}). We say that the curly braces *delimit* the body of the function. The word *delimit* means to determine the boundaries or limits of something. The { symbol determines the beginning of the function’s body, and the } symbol specifies the end of the function’s body.

We can rewrite Listing 2.4 (arrow.cpp) to achieve the same effect with only one long print statement as Listing 2.5 (arrow2.cpp) shows.

Listing 2.5: arrow2.cpp

```
#include <iostream>

using namespace std;

int main() {
    cout << "    *    " << endl
         << "   ***   " << endl
         << "  ***** " << endl
         << "    *    " << endl
         << "     *    " << endl
         << "     *    " << endl;
}
```

At first, Listing 2.4 (arrow.cpp) and Listing 2.5 (arrow2.cpp) may appear to be identical, but upon closer inspection of this new program we see that **cout** appears only once within **main**, and only one semicolon (;) appears within **main**. Since semicolons in C++ terminate statements, there really is only one statement. Notice that a single statement can be spread out over several lines. The statement within **main** appearing as

```
cout << "    *    " << endl
     << "   ***   " << endl
     << "  ***** " << endl
     << "    *    " << endl
     << "     *    " << endl
     << "     *    " << endl;
```

could have just as easily been written as

```
cout << "    *    " << endl << "    ***    "
    << endl << "    *****    " << endl
    << "    *    " << endl
    << "    *    " << endl << "    *    " << endl;
```

but the first way of expressing it better portrays how the output will appear. Read this second version carefully to convince yourself that the printed pieces will indeed flow to the `cout` printing object in the proper sequence to produce the same picture of the arrow.

Consider the mistake of putting semicolons at the end of each of the lines in the “one statement” version:

```
cout << "    *    " << endl;
    << "    ***    " << endl;
    << "    *****    " << endl;
    << "    *    " << endl;
    << "    *    " << endl;
    << "    *    " << endl;
```



If we put this code fragment in `main`, the program will not compile. The reason is simple—the semicolon at the end of the first line terminates the statement on that line. The compiler expects a new statement on the next line, but

```
<< "    ***    " << endl;
```

is not a complete legal C++ statement since the `<<` operator is missing the `cout` object. The string `" *** "` and the end-of-line marker has nothing to “flow into.”

Listing 2.6 (`empty.cpp`) is even simpler than Listing 2.1 (`simple.cpp`).

Listing 2.6: `empty.cpp`

```
int main() {
}
```

Since Listing 2.6 (`empty.cpp`) uses neither the `cout` object nor `endl`, it does not need the `#include` and `using` directives. While it is legal and sometimes even useful in C++ to write functions with empty bodies, such functions will do nothing when they execute. Listing 2.6 (`empty.cpp`) with its empty `main` function is, therefore, truly the simplest executable C++ program we can write, but it does nothing when we run it!

In general, a C++ program may contain multiple functions, but we defer such generality until Chapter 9. For now, we will restrict our attention to programs with only a `main` function.

2.4 Template for simple C++ programs

For our immediate purposes all the programs we write will have the form shown in Figure 2.1.

```
#include <iostream>

using namespace std;

int main() {
    program statements
}
```

Figure 2.1: The general structure of a very simple C++ program.

Our programs generally will print something, so we need the `#include` directive. Since we will use the short names for the printing objects, we also include the `using namespace` directive. The `main` function definition is required for an executable program, and we will fill its body with statements that make our program do as we wish. Later, our programs will become more sophisticated, and we will need to augment this simple template.

2.5 Summary

- Any programs that print to the screen must use the `#include` directive so that the program can make use of the precompiled `iostream` library code to do I/O (input and output).
- All preprocessor directives begin with the `#` symbol.
- Messages can be printed on the screen by sending them to the `std::cout` object using the `<<` operator.
- The `<<` operator used in conjunction with the `cout` stream object is known as the insertion operator.
- The end of a printed line is signified by sending `std::endl` to the `std::cout` object.
- The shorter names `cout` and `endl` can be used if the `using namespace std` appears at the top of the program's source code.
- An executable C++ program must contain a function named `main`.
- Functions contain statements that are executed by the computer when the program runs. (The program must be compiled for the computer to be able to execute the statements.)
- A function's body consists of all the statements inside the delimiting curly braces (`{}`).
- A function's body may be empty; however, such a function performs no useful activity when it executes.
- Statements are terminated with semicolons (`;`).
- Within the Visual Studio IDE a program is developed within a project.

- Visual Studio's command prompt environment can be used to develop simple C++ programs without the overhead of creating a project.
- In a printing statement the characters within the quotation marks (") are printed literally on the screen.

2.6 Exercises

1. What preprocessor directive is necessary to use statements with the `std::cout` printing stream object?
2. What statement allows the short names `cout` and `endl` to be used instead of `std::cout` and `std::endl`?
3. What does the name `std` stand for?
4. All C++ programs must have a function named what?
5. The body of `main` is enclosed within what symbols?
6. What operator directs information to the `std::cout` output stream?
7. Write a C++ program that prints your name in the console window.
8. Write a C++ program that prints your first and last name in the console window. Your first name should appear on one line, and your last name appear on the next line.
9. What other files must you distribute with your executable file so that your program will run on a Windows PC without Visual Studio installed?
10. Can a single statement in C++ span multiple lines in the source code?

Chapter 3

Values and Variables

In this chapter we explore some building blocks that are used to develop C++ programs. We experiment with the following concepts:

- numeric values
- variables
- declarations
- assignment
- identifiers
- reserved words

In the next chapter we will revisit some of these concepts in the context of other data types.

3.1 Integer Values

The number four (4) is an example of a *numeric* value. In mathematics, 4 is an *integer* value. Integers are whole numbers, which means they have no fractional parts, and an integer can be positive, negative, or zero. Examples of integers include 4, −19, 0, and −1005. In contrast, 4.5 is not an integer, since it is not a whole number.

C++ supports a number of numeric and non-numeric values. In particular, C++ programs can use integer values. It is easy to write a C++ program that prints the number four, as Listing 3.1 (number4.cpp) shows.

Listing 3.1: number4.cpp

```
#include <iostream>

using namespace std;

int main() {
    cout << 4 << endl;
}
```

Notice that unlike the programs we saw earlier, Listing 3.1 (`number4.cpp`) does not use quotation marks (`"`). Compare Listing 3.1 (`number4.cpp`) to Listing 3.2 (`number4-alt.cpp`).

Listing 3.2: `number4-alt.cpp`

```
#include <iostream>

using namespace std;

int main() {
    cout << "4" << endl;
}
```

Both programs behave identically, but Listing 3.1 (`number4.cpp`) prints the value of the number four, while Listing 3.2 (`number4-alt.cpp`) prints a message containing the digit four. The distinction here seems unimportant, but we will see in Section 3.2 that the presence or absence of the quotes can make a big difference in the output.

In C++ source code, integers may not contain commas. This means we must write the number two thousand, four hundred sixty-eight as **2468**, not **2,468**. In mathematics, integers are unbounded; said another way, the set of mathematical integers is infinite. In C++ the range of integers is limited because all computers have a finite amount of memory. The exact range of integers supported depends on the computer system and particular C++ compiler. C++ on most 32-bit computer systems can represent integers in the range $-2,147,483,648$ to $+2,147,483,647$.

What happens if you exceed the range of C++ integers? Try Listing 3.3 (`exceed.cpp`) on your system.

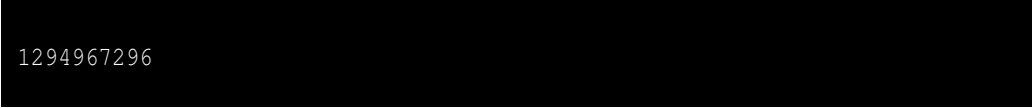
Listing 3.3: `exceed.cpp`

```
#include <iostream>

using namespace std;

int main() {
    cout << -3000000000 << endl;
}
```

Negative three billion is too large for 32-bit integers, however, and the program's output is obviously wrong:



1294967296

The number printed was not even negative! Most C++ compilers will issue a warning about this statement. `secrefsec:expressionsarithmetic.errors` explores errors vs. warnings in more detail. If the compiler finds an error in the source, it will not generate the executable code. A warning indicates a potential problem and does not stop the compiler from producing an executable program. Here we see that the programmer should heed this warning because the program's execution produces meaningless output.

This limited range of values is common among programming languages since each number is stored in a fixed amount of memory. Larger numbers require more storage in memory. In order to model the infinite set of mathematical integers an infinite amount of memory would be needed! As we will see later, C++ supports an integer type with a greater range.

Appendix D.1 provides some details about the implementation of C++ integers.

3.2 Variables and Assignment

In algebra, variables are used to represent numbers. The same is true in C++, except C++ variables also can represent values other than numbers. Listing 3.4 (`variable.cpp`) uses a variable to store an integer value and then prints the value of the variable.

Listing 3.4: `variable.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int x;
    x = 10;
    cout << x << endl;
}
```

The `main` function in Listing 3.4 (`variable.cpp`) contains three statements:

- `int x;`

This is a *declaration* statement. All variables in a C++ program must be declared. A declaration specifies the type of a variable. The word `int` indicates that the variable is an integer. The name of the integer variable is `x`. We say that variable `x` has type `int`. C++ supports types other than integers, and some types require more or less space in the computer's memory. The compiler uses the declaration to reserve the proper amount of memory to store the variable's value. Due to the declaration the compiler also can check to see if the variable is being used properly; for example, we will see that integers can be added together just like in mathematics. For some other data types, however, addition is not possible and so is not allowed. The compiler can ensure that a variable involved in an addition is compatible with addition. It can report an error if it is not.

The compiler will issue an error if a programmer attempts to use an undeclared variable. The compiler cannot deduce the storage requirements and cannot verify the variable's proper usage if it not declared. Once declared, a particular variable cannot be redeclared in the same context. A variable may not change its type during its lifetime.

- `x = 10;`

This is an *assignment* statement. An assignment statement associates a value with a variable. The key to an assignment statement is the symbol `=` which is known as the *assignment operator*. Here the value 10 is being assigned to the variable `x`. This means the value 10 will be stored in the memory location the compiler has reserved for the variable named `x`. We need not be concerned about where the variable is stored in memory; the compiler takes care of that detail.

After we declare a variable we may assign and reassign it as often as necessary.

- `cout << x << endl;`

This statement prints the variable `x`'s current value.



Note that the lack of quotation marks here is very important. If **x** has the value 10, the statement

```
cout << x << endl;
```

prints **10**, the value of the variable **x**, but the statement

```
cout << "x" << endl;
```

prints **x**, the message containing the single letter *x*.

The meaning of the assignment operator (=) is different from equality in mathematics. In mathematics, = asserts that the expression on its left is equal to the expression on its right. In C++, = makes the variable on its left take on the value of the expression on its right. It is best to read **x = 5** as “**x** is assigned the value 5,” or “**x** gets the value 5.” This distinction is important since in mathematics equality is symmetric: if $x = 5$, we know $5 = x$. In C++, this symmetry does not exist; the statement

```
5 = x;
```

attempts to reassign the value of the literal integer value 5, but this cannot be done, because 5 is always 5 and cannot be changed. Such a statement will produce a compiler error:

error C2106: '=' : left operand must be l-value

Variables can be reassigned different values as needed, as Listing 3.5 (multipleassignment.cpp) shows.

Listing 3.5: multipleassignment.cpp

```
#include <iostream>

using namespace std;

int main() {
    int x;
    x = 10;
    cout << x << endl;
    x = 20;
    cout << x << endl;
    x = 30;
    cout << x << endl;
}
```

Observe the each print statement in Listing 3.5 (multipleassignment.cpp) is identical, but when the program runs the print statements produce different results.

A variable may be given a value at the time of its declaration; for example, Listing 3.6 (variable-init.cpp) is a variation of Listing 3.4 (variable.cpp).

Listing 3.6: variable-init.cpp

```
#include <iostream>

using namespace std;
```

```
int main() {  
    int x = 10;  
    cout << x << endl;  
}
```

Notice that in Listing 3.6 (`variable-init.cpp`) the declaration and assignment of the variable `x` is performed in one statement instead of two. This combined declaration and immediate assignment is called *initialization*.

C++ supports another syntax for initializing variables as shown in Listing 3.7 (`alt-variable-init.cpp`).

Listing 3.7: `alt-variable-init.cpp`

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    int x{10};  
    cout << x << endl;  
}
```

This alternate form is not commonly used for simple variables, but it necessary for initializing more complicated kinds of variables called *objects*. We introduce objects in Chapter 13 and Chapter 14.

Multiple variables of the same type can be declared and, if desired, initialized in a single statement. The following statements declare three variables in one declaration statement:

```
int x, y, z;
```

The following statement declares three integer variables and initializes two of them:

```
int x = 0, y, z = 5;
```

Here `y`'s value is undefined. The declarations may be split up into multiple declaration statements:

```
int x = 0;  
int y;  
int z = 5;
```

In the case of multiple declaration statements the type name (here `int`) must appear in each statement.

The compiler maps a variable to a location in the computer's memory. We can visualize a variable and its corresponding memory location as a box as shown in Figure 3.1.



Figure 3.1: Representing a variable and its memory location as a box

We name the box with the variable's name. Figure 3.2 shows how the following sequence of C++ code affects memory.

```
int a, b;
a = 2;
b = 5;
a = b;
b = 4;
```

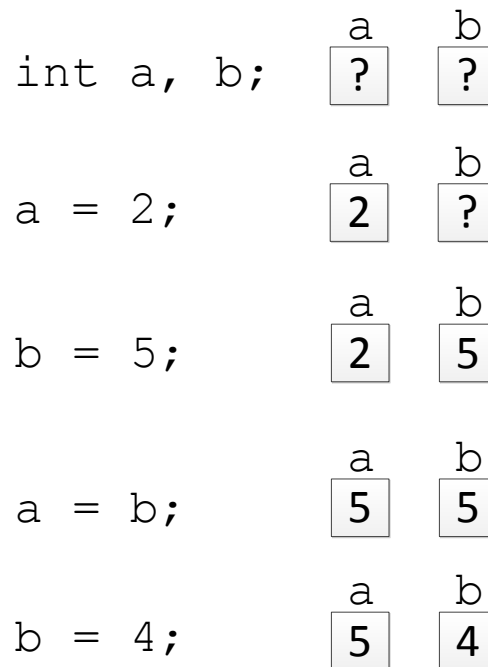


Figure 3.2: How memory changes during variable assignment

Importantly, the statement

```
a = b;
```

does not mean **a** and **b** refer to the same box (memory location). After this statement **a** and **b** still refer to separate boxes (memory locations). It simply means the value stored in **b**'s box (memory location) has been copied to **a**'s box (memory location). **a** and **b** remain distinct boxes (memory locations). The original value found in **a**'s box is overwritten when the contents of **b**'s box are copied into **a**. After the assignment of **b** to **a**, the reassignment of **b** to 4 does not affect **a**.

3.3 Identifiers

While mathematicians are content with giving their variables one-letter names like **x**, programmers should use longer, more descriptive variable names. Names such as **altitude**, **sum**, and **user_name** are much

better than the equally permissible **a**, **s**, and **u**. A variable's name should be related to its purpose within the program. Good variable names make programs more readable by humans. Since programs often contain many variables, well-chosen variable names can render an otherwise obscure collection of symbols more understandable.

C++ has strict rules for variable names. A variable name is one example of an *identifier*. An identifier is a word used to name things. One of the things an identifier can name is a variable. We will see in later chapters that identifiers name other things such as functions and classes. Identifiers have the following form:

- Identifiers must contain at least one character.
- The first character must be an alphabetic letter (upper or lower case) or the underscore

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz_

- The remaining characters (if any) may be alphabetic characters (upper or lower case), the underscore, or a digit

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz_0123456789

- No other characters (including spaces) are permitted in identifiers.
- A reserved word cannot be used as an identifier (see Table 3.1).

Here are some examples of valid and invalid identifiers:

- All of the following words are valid identifiers and so qualify as variable names: **x**, **x2**, **total**, **port_22**, and **FLAG**.
- None of the following words are valid identifiers: **sub-total** (dash is not a legal symbol in an identifier), **first entry** (space is not a legal symbol in an identifier), **4all** (begins with a digit), **#2** (pound sign is not a legal symbol in an identifier), and **class** (**class** is a reserved word).

C++ reserves a number of words for special use that could otherwise be used as identifiers. Called *reserved words* or *keywords*, these words are special and are used to define the structure of C++ programs and statements. Table 3.1 lists all the C++ reserved words.

The purposes of many of these reserved words are revealed throughout this book.

You may not use any of the reserved words in Table 3.1 as identifiers. Fortunately, if you accidentally attempt to use one of the reserved words in a program as a variable name, the compiler will issue an error (see Section 4.6 for more on compiler errors).

In Listing 2.1 (simple.cpp) we used several reserved words: **using**, **namespace**, and **int**. Notice that **include**, **cout**, **endl**, and **main** are not reserved words.

Some programming languages do not require programmers to declare variables before they are used; the type of a variable is determined by how the variable is used. Some languages allow the same variable to assume different types as its use differs in different parts of a program. Such languages are known as *dynamically-typed languages*. C++ is a *statically-typed language*. In a statically-typed language, the type of a variable must be explicitly specified before it is used by statements in a program. While the requirement to declare all variables may initially seem like a minor annoyance, it offers several advantages:

- When variables must be declared, the compiler can catch typographical errors that dynamically-typed languages cannot detect. For example, consider the following section of code:

<code>alignas</code>	<code>decltype</code>	<code>namespace</code>	<code>struct</code>
<code>alignof</code>	<code>default</code>	<code>new</code>	<code>switch</code>
<code>and</code>	<code>delete</code>	<code>noexcept</code>	<code>template</code>
<code>and_eq</code>	<code>double</code>	<code>not</code>	<code>this</code>
<code>asm</code>	<code>do</code>	<code>not_eq</code>	<code>thread_local</code>
<code>auto</code>	<code>dynamic_cast</code>	<code>nullptr</code>	<code>throw</code>
<code>bitand</code>	<code>else</code>	<code>operator</code>	<code>true</code>
<code>bitor</code>	<code>enum</code>	<code>or</code>	<code>try</code>
<code>bool</code>	<code>explicit</code>	<code>or_eq</code>	<code>typedef</code>
<code>break</code>	<code>export</code>	<code>private</code>	<code>typeid</code>
<code>case</code>	<code>extern</code>	<code>protected</code>	<code>typename</code>
<code>catch</code>	<code>false</code>	<code>public</code>	<code>union</code>
<code>char</code>	<code>float</code>	<code>register</code>	<code>unsigned</code>
<code>char16_t</code>	<code>for</code>	<code>reinterpret_cast</code>	<code>using</code>
<code>char32_t</code>	<code>friend</code>	<code>return</code>	<code>virtual</code>
<code>class</code>	<code>goto</code>	<code>short</code>	<code>void</code>
<code>compl</code>	<code>if</code>	<code>signed</code>	<code>volatile</code>
<code>const</code>	<code>inline</code>	<code>sizeof</code>	<code>wchar_t</code>
<code>constexpr</code>	<code>int</code>	<code>static</code>	<code>while</code>
<code>const_cast</code>	<code>long</code>	<code>static_assert</code>	<code>xor</code>
<code>continue</code>	<code>mutable</code>	<code>static_cast</code>	<code>xor_eq</code>

Table 3.1: C++ reserved words. C++ reserves these words for specific purposes in program construction. None of the words in this list may be used as an identifier; thus, you may not use any of these words to name a variable.

```
int ZERO;
ZERO = 1;
```

The identifier in the first line ends with a capital “Oh.” In the second line, the identifier ends with the digit zero. The distinction may be difficult or impossible to see in a particular editor or printout of the code. A C++ compiler would immediately detect the typo in the second statement, since **ZERO** (last letter a zero) has not been declared. A dynamically-typed language would create two variables: **ZERO** and **ZERO**.

- When variables must be declared, the compiler can catch invalid operations. For example, a variable may be declared to be of type `int`, but the programmer may accidentally assign a non-numeric value to the variable. In a dynamically-typed language, the variable would silently change its type introducing an error into the program. In C++, the compiler would report the improper assignment as error, since once declared a C++ variable cannot change its type.
- Ideally, requiring the programmer to declare variables forces the programmer to plan ahead and think more carefully about the variables a program might require. The purpose of a variable is tied to its type, so the programmer must have a clear notion of the variable’s purpose before declaring it. When variable declarations are not required, a programmer can “make up” variables as needed as the code is written. The programmer need not do the simple double check of the variable’s purpose that writing the variable’s declaration requires. While declaring the type of a variable specifies its purpose in only a very limited way, any opportunity to catch such errors is beneficial.
- Statically-typed languages are generally more efficient than dynamically-typed languages. The compiler knows how much storage a variable requires based on its type. The space for that variable’s value will not change over the life of the variable, since its type cannot change. In a dynamically

Type	Storage	Smallest Magnitude	Largest Magnitude	Minimum Precision
float	4 bytes	1.17549×10^{-38}	$3.40282 \times 10^{+38}$	6 digits
double	8 bytes	2.22507×10^{-308}	$1.79769 \times 10^{+308}$	15 digits

Table 3.2: Characteristics of Floating-point Numbers on 32-bit Computer Systems

typed language that allows a variable to change its type, if a variable's type changes during program execution, the storage it requires may change also, so memory for that variable must be allocated elsewhere to hold the different type. This memory reallocation at run time slows down the program's execution.

C++ is a case-sensitive language. This means that capitalization matters. **if** is a reserved word, but none of **If**, **IF**, or **iF** are reserved words. Identifiers are case sensitive also; the variable called **Name** is different from the variable called **name**.

Since it can be confusing to human readers, you should not distinguish variables merely by names that differ in capitalization. For the same reason, it is considered poor practice to give a variable the same name as a reserved word with one or more of its letters capitalized.

3.4 Floating-point Types

Many computational tasks require numbers that have fractional parts. For example, to compute the area of a circle given the circle's radius, the value π , or approximately 3.14159 is used. C++ supports such non-integer numbers, and they are called *floating-point* numbers. The name implies that during mathematical calculations the decimal point can move or "float" to various positions within the number to maintain the proper number of significant digits. The types **float** and **double** represent different types of floating-point numbers. The type **double** is used more often, since it stands for "double-precision floating-point," and it can represent a wider range of values with more digits of precision. The **float** type represents single-precision floating-point values that are less precise. Table 3.2 provides some information about floating-point values as commonly implemented on 32-bit computer systems. Floating point numbers can be both positive and negative.

As you can see from Table 3.2, **doubles** provide more precision at the cost of using more memory.

Listing 3.8 (pi-print.cpp) prints an approximation of the mathematical value π .

Listing 3.8: pi-print.cpp

```
#include <iostream>

using namespace std;

int main() {
    double pi = 3.14159;
    cout << "Pi = " << pi << endl;
    cout << "or " << 3.14 << " for short" << endl;
}
```

The first line in Listing 3.8 (pi-print.cpp) declares a variable named **pi** and assigns it a value. The second line in Listing 3.8 (pi-print.cpp) prints the value of the variable **pi**, and the third line prints a literal value. Any literal numeric value with a decimal point in a C++ program automatically has the type **double**, so

3.14

has type `double`. To make a literal floating-point value a `float`, you must append an `f` or `F` to the number, as in

3.14f

(The `f` or `F` suffix is used with literal values only; you cannot change a `double` variable into a `float` variable by appending an `f`. Attempting to do so would change the name of the variable!)



All floating-point literals are `double` values unless an `f` or `F` is appended to the end of the number; for example, `2.0` is a `double` literal, while `2.0f` is a `float` literal.

Floating-point numbers are an approximation of mathematical real numbers. As in the case of the `int` data type, the range of floating-point numbers is limited, since each value requires a fixed amount of memory. In some ways, though, `ints` are very different from `doubles`. Any integer within the range of the `int` data type can be represented exactly. This is not true for the floating-point types. Consider the real number π . Since π contains an infinite number of digits, its value only can be approximated. Since the number of digits available is limited, some numbers with a finite number of digits can be only approximated; for example, the number 23.3123400654033989 contains too many digits for the `double` type and must be approximated as 23.3023498654034. An example of the problems that can arise due to the inexact nature of floating-point numbers is demonstrated later in Listing 4.2 (`imprecise.cpp`).

We can express floating-point numbers in scientific notation. Since most programming editors do not provide superscripting and special symbols like $\times$, C++ slightly alters the normal scientific notation. The number 6.022×10^{23} is written `6.022e23`. The number to the left of the `e` (we can use capital `E` as well) is the mantissa, and the number to the right of the `e` is the exponent of 10. As another example, -5.1×10^4 is expressed in C++ as `-5.1e-4`. Listing 3.9 (`scientificnotation.cpp`) prints some scientific constants using scientific notation.

Listing 3.9: `scientificnotation.cpp`

```
#include <iostream>

using namespace std;

int main() {
    double avogadros_number = 6.022e23, c = 2.998e8;
    cout << "Avogadro's number = " << avogadros_number << endl;
    cout << "Speed of light = " << c << endl;
}
```

Appendix D.2 provides some insight into the implementation of C++ floating-point values and explains how internally all floating-point numbers are stored in exponential notation with a mantissa and exponent.

3.5 Constants

In Listing 3.9 (scientificnotation.cpp), Avogadro's number and the speed of light are scientific constants; that is, to the degree of precision to which they have been measured and/or calculated, they do not vary. C++ supports named constants. Constants are declared like variables with the addition of the `const` keyword:

```
const double PI = 3.14159;
```

Once declared and initialized, a constant can be used like a variable in all but one way—a constant may not be reassigned. It is illegal for a constant to appear on the left side of the assignment operator (=) outside its declaration statement. A subsequent statement like

```
PI = 2.5;
```

would cause the compiler to issue an error message:

```
error C3892: 'PI' : you cannot assign to a variable that is const
```

and fail to compile the program. Since the scientific constants do not change, Listing 3.10 (const.cpp) is a better version of Listing 3.9 (scientificnotation.cpp).

Listing 3.10: const.cpp

```
#include <iostream>

using namespace std;

int main() {
    const double avogadros_number = 6.022e23, c = 2.998e8;
    cout << "Avogadro's number = " << avogadros_number << endl;
    cout << "Speed of light = " << c << endl;
}
```

Since it is illegal to assign a constant outside of its declaration statement, all constants **must** be initialized where they are declared.

By convention, C++ programmers generally express constant names in all capital letters; in this way, within the source code a human reader can distinguish a constant quickly from a variable.

3.6 Other Numeric Types

C++ supports several other numeric data types:

- `long int`—typically provides integers with a greater range than the `int` type; its abbreviated name is `long`. It is guaranteed to provide a range of integer values at least as large as the `int` type.

An integer literal with a `L` suffix, as in `19L`, has type `long`. A lower case `l` (`1`) is allowed as a suffix as well, but you should not use it because it is difficult for human readers to distinguish between `l` (lower case *elle*) and `1` (digit *one*). (The `L` suffix is used with literal values only; you cannot change an `int` variable into a `long` by appending an `L`. Attempting to do so would change the name of the variable!)

0	'\0'	16	—	32	' '	48	'0'	64	'@'	80	'P'	96	'`'	112	'p'
1	—	17	—	33	'!'	49	'1'	65	'A'	81	'Q'	97	'a'	113	'q'
2	—	18	—	34	'"'	50	'2'	66	'B'	82	'R'	98	'b'	114	'r'
3	—	19	—	35	'#'	51	'3'	67	'C'	83	'S'	99	'c'	115	's'
4	—	20	—	36	'\$'	52	'4'	68	'D'	84	'T'	100	'd'	116	't'
5	—	21	—	37	'%'	53	'5'	69	'E'	85	'U'	101	'e'	117	'u'
6	—	22	—	38	'&'	54	'6'	70	'F'	86	'V'	102	'f'	118	'v'
7	'\a'	23	—	39	'\''	55	'7'	71	'G'	87	'W'	103	'g'	119	'w'
8	'\b'	24	—	40	'('	56	'8'	72	'H'	88	'X'	104	'h'	120	'x'
9	'\t'	25	—	41	')'	57	'9'	73	'I'	89	'Y'	105	'i'	121	'y'
10	'\n'	26	—	42	'*'	58	':'	74	'J'	90	'Z'	106	'j'	122	'z'
11	—	27	—	43	'+'	59	','	75	'K'	91	'['	107	'k'	123	'{'
12	'\f'	28	—	44	','	60	'<'	76	'L'	92	'\''	108	'l'	124	' '
13	'\r'	29	—	45	'-'	61	'='	77	'M'	93	']'	109	'm'	125	'}'
14	—	30	—	46	'.'	62	'>'	78	'N'	94	'^'	110	'n'	126	'~'
15	—	31	—	47	'/'	63	'?'	79	'O'	95	'_'	111	'o'	127	—

Table 3.3: ASCII codes for characters

- **short int**—typically provides integers with a smaller range than the **int** type; its abbreviated name is **short**. It is guaranteed that the range of **ints** is at least as big as the range of **shorts**.
- **unsigned int**—is restricted to non-negative integers; its abbreviated name is **unsigned**. While the **unsigned** type is limited in non-negative values, it can represent twice as many positive values as the **int** type. (The name **int** is actually the short name for **signed int** and **int** can be written as **signed**.)
- **long double**—can extend the range and precision of the **double** type.

While the C++ language standard specifies minimum ranges and precision for all the numeric data types, a particular C++ compiler may exceed the specified minimums.

C++ provides such a variety of numeric types for specialized purposes usually related to building highly efficient programs. We will have little need to use many of these types. Our examples will use mainly the numeric types **int** for integers, **double** for an approximation of real numbers, and, less frequently, **unsigned** when non-negative integral values are needed.

3.7 Characters

The **char** data type is used to represent single characters: letters of the alphabet (both upper and lower case), digits, punctuation, and control characters (like newline and tab characters). Most systems support the American Standard Code for Information Interchange (ASCII) character set. Standard ASCII can represent 128 different characters. Table 3.3 lists the ASCII codes for various characters.

In C++ source code, characters are enclosed by single quotes ('), as in

```
char ch = 'A';
```

Standard (double) quotes (") are reserved for strings, which are composed of characters, but strings and **chars** are very different. C++ strings are covered in Section F.7. The following statement would produce a compiler error message:

```
ch = "A";
```

since a string cannot be assigned to a character variable.

Internally, **chars** are stored as integer values, and C++ permits assigning numeric values to **char** variables and assigning characters to numeric variables. The statement

```
ch = 65;
```

assigns a number to a **char** variable to show that this is perfectly legal. The value 65 is the ASCII code for the character A. If **ch** is printed, as in

```
ch = 65;  
cout << ch;
```

the corresponding character, A, would be printed because **ch**'s declared type is **char**, not **int** or some other numeric type.

Listing 3.11 (charexample.cpp) shows how characters can be used within a program.

Listing 3.11: charexample.cpp

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    char ch1, ch2;  
    ch1 = 65;  
    ch2 = 'A';  
    cout << ch1 << ", " << ch2 << ", " << 'A' << endl;  
}
```

The program displays

A, A, A

The first A is printed because the statement

```
ch1 = 65;
```

assigns the ASCII code for A to **ch1**. The second A is printed because the statement

```
ch2 = 'A';
```

assigns the literal character A to **ch2**. The third A is printed because the literal character **'A'** is sent directly to the output stream.

Integers and characters can be freely assigned to each other, but the range of **chars** is much smaller than the range of **ints**, so care must be taken when assigning an **int** value to a **char** variable.

Some characters are *non-printable* characters. The ASCII chart lists several common non-printable characters:

- **'\n'**—the newline character
- **'\r'**—the carriage return character
- **'\b'**—the backspace character

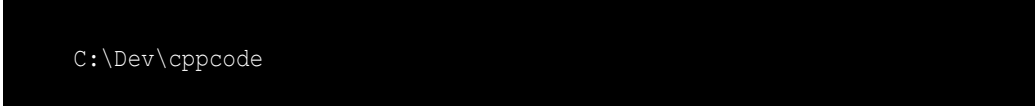
- `'\a'`—the “alert” character (causes a “beep” sound on many systems)
- `'\t'`—the tab character
- `'\f'`—the formfeed character
- `'\0'`—the *null* character (used in C strings, see Section F.7)

These special non-printable characters begin with a backslash (`\`) symbol. The backslash is called an *escape* symbol, and it signifies that the symbol that follows has a special meaning and should not be interpreted literally. This means the literal backslash character must be represented as two backslashes: `'\\'`.

These special non-printable character codes can be embedded within strings. To embed a backslash within a string, you must escape it; for example, the statement

```
cout << "C:\\Dev\\cppcode" << endl;
```

would print



```
C:\Dev\cppcode
```

See what this statement prints:

```
cout << "AB\bCD\aEF" << endl;
```

The following two statements behave identically on most computer systems:

```
cout << "End of line" << endl;
cout << "End of line\n";
```

On the Microsoft Windows platform, the character sequence `"\r\n"` (carriage return, line feed) appears at the end of lines in text files. Under Unix and Linux, lines in text files end with `'\n'` (line feed). On Apple Macintosh systems, text file lines end with the `'\r'` (carriage return) character. The compilers that adhere to the C++ standard will ensure that the `'\n'` character in a C++ program when sent to the output stream will produce the same results as the `endl` manipulator.

3.8 Enumerated Types

C++ allows a programmer to create a new, very simple type and list all the possible values of that type. Such a type is called an *enumerated type*, or an *enumeration type*. The `enum` keyword introduces an enumerated type:

```
enum Color { Red, Orange, Yellow, Green, Blue, Violet };
```

Here, the new type is named `Color`, and a variable of type `Color` may assume one of the values that appears in the list of values within the curly braces. The semicolon following the close curly brace is required. Sometimes the enumerated type definition is formatted as

```
enum Color {  
    Red,  
    Orange,  
    Yellow,  
    Green,  
    Blue,  
    Violet  
};
```

but the formatting makes no difference to the compiler.

The values listed with the curly braces constitute *all* the values that a variable of the enumerated type can attain. The name of each value of an enumerated type must be a valid C++ identifier (see Section 3.3).

Given the `Color` type defined as above, we can declare and use variables of the `enum` type as shown by the following code fragment:

```
Color myColor;  
myColor = Orange;
```

Here the variable `myColor` has our custom type `Color`, and its value is `Orange`.

It is illegal to reuse an enumerated value name within another enumerated type within the same program. In the following code, the enumerated value `Light` is used in both `Shade` and `Weight`:

```
enum Shade { Dark, Dim, Light, Bright };  
enum Weight { Light, Medium, Heavy };
```

These two enumerated types are incompatible because they share the value `Light`, and so the compiler will issue an error.

The value names within an `enum` type must be unique. The convention in C++ is to capitalize the first letter of an `enum` type and its associated values, although the language does not enforce this convention.

An `enum` type is handy for representing a small number of discrete, non-numeric options. For example, consider a program that controls the movements made by a small robot. The allowed orientations are forward, backward, left, right, up, and down. The program could encode these movements as integers, where 0 means left, 1 means backward, etc. While that implementation will work, it is not ideal. Integers may assume many more values than just the six values expected. The compiler cannot ensure that an integer variable representing a robot move will stay in the range 0...5. What if the programmer makes a mistake and under certain rare circumstances assigns a value outside of the range 0...5? The program then will contain an error that may result in erratic behavior by the robot. With `enum` types, if the programmer uses only the named values of the `enum` type, the compiler will ensure that such a mistake cannot happen.

A particular enumerated type necessarily has far fewer values than a type such as `int`. Imagine making an integer `enum` type and having to list all of its values! (The standard 32-bit `int` type represents over four billion values.) Enumerated types, therefore, are practical only for types that have a relatively small range of values.

3.9 Type Inference with auto

C++ requires that a variable be declared before it is used. Ordinarily this means specifying the variable's type, as in


```
int count;  
char ch;  
double limit;
```

A variable may be initialized when it is declared:

```
int count = 0;  
char ch = 'Z';  
double limit = 100.0;
```

Each of the values has a type: `0` is an `int`, `'Z'` is a `char`, and `0.0` is a `double`. The `auto` keyword allows the compiler to automatically deduce the type of a variable if it is initialized when it is declared:

```
auto count = 0;  
auto ch = 'Z';  
auto limit = 100.0;
```

The `auto` keyword may **not** be used without an accompanying initialization; for example, the following declaration is illegal:

```
auto x;
```

because the compiler cannot deduce `x`'s type.



Automatic type inference is supported only by compilers that comply with the latest C++11 standard. Programmers using older compilers must specify a variable's exact type during the variable's declaration.

Automatic type deduction with `auto` is not useful to beginning C++ programmers. It is just as easy to specify the variable's type. The value of `auto` will become clearer when we consider some of the more advanced features of C++ (see Section 18.3).

3.10 Summary

- C++ supports several kinds of numeric values and variables.
- In C++ commas cannot be used when expressing numeric literals.
- Numbers represented on a computer have limitations based on the finite nature of computer systems.
- All numeric types in C++ have limited ranges. This is because they must be stored in the computer's memory and be manipulated by the processor. The memory and processor deal with fixed-size quantities, and in order to handle an unlimited range of values memory would have to be infinite.
- Variables are used to store values.
- All variables in C++ must be declared before they are used within the program.
- How can you declare a variable named `total` to be of type integer?
- The `=` operator means *assignment*, not mathematical *equality*.

- A variable can be reassigned at any time.
- A variable can be given an initial value at the time of its declaration.
- Multiple variables can be declared in one declaration statement.
- A variable represents a location in memory capable of storing a value.
- The statement `a = b;` copies the value stored in variable `b` into variable `a`. `a` and `b` still refer to their own memory locations.
- A variable name is an example of an identifier.
- The name of a variable must follow the identifier naming rules.
- All identifiers must consist of at least one character. The first symbol must be an alphabetic letter or the underscore. Remaining symbols (if any) must be alphabetic letters, the underscore, or digits.
- Reserved words have special meaning within a C++ program and cannot be used as identifiers.
- Descriptive variable names are preferred over one-letter names.
- C++ is case sensitive; the name `X` is not the same as the name `x`.
- Floating numbers approximate mathematical real numbers.
- The `float` type represents single-precision floating-point numbers, and the `double` type represents double-precision floating-point numbers.
- There are many values that floating-point numbers cannot represent exactly.
- The symbol `f` or `F` can be appended to a floating-point literal to signify that it is a `float` value instead of a `double` value.
- In C++ we express scientific notation literals of the form 1.0×10^1 as `1.0e1.0`.
- The qualifier `const` renders a “variable” unchangeable; it signifies a constant.
- On some systems `longs` may have extended range compared to normal `ints`.
- On some systems `shorts` may have reduced range compared to normal `ints`.
- The `unsigned` type represents non-negative integers.
- On some systems `long doubles` may have extended precision compared to `doubles`.
- The `char` type represents characters.
- Characters are stored internally as 8-bit integers. Their values correspond to the ASCII code.
- A `char` value can be assigned to an `int` variable, and an `int` value can be assigned to a `char` variable. `char`s and `ints` are readily interchangeable in C++.
- Character literals appear within single quote marks (`'`), unlike string literals that appear within double quote marks (`"`).
- Special non-printable control codes like newline and tab are prefixed with the backslash escape character (`\`).
- The `'\n'` character is an alternative to using `endl` to end a printed line.

- The C++ enumerated type feature allows programmers to design their own custom type and specify the values that make up that type.
- A value of an enumerated type must be an identifier.
- An `enum` type is handy for representing a small number of discrete, non-numeric options.
- The `auto` keyword used during a variable's declaration allows the compiler to deduce the variable's type based on its initial value.

3.11 Exercises

1. Will the following lines of code print the same thing? Explain why or why not.

```
cout << 6 << endl;  
cout << "6" << endl;
```

2. Will the following lines of code print the same thing? Explain why or why not.

```
cout << x << endl;  
cout << "x" << endl;
```

3. What is the largest `int` available on your system?
4. What is the smallest `int` available on your system?
5. What is the largest `double` available on your system?
6. What is the smallest `double` available on your system?
7. What C++ data type represents non-negative integers?
8. What happens if you attempt to use a variable within a program, and that variable is not declared?
9. What is wrong with the following statement that attempts to assign the value ten to variable `x`?

```
10 = x;
```

10. Once a variable has been properly declared and initialized can its value be changed?
11. What is another way to write the following declaration and initialization?

```
int x = 10;
```

12. In C++ can you declare more than variable in the same declaration statement? If so, how?
13. In the declaration

```
int a;  
int b;
```

do `a` and `b` represent the same memory location?

14. Classify each of the following as either a *legal* or *illegal* C++ identifier:

(a) `fred`

- (b) `if`
 - (c) `2x`
 - (d) `-4`
 - (e) `sum_total`
 - (f) `sumTotal`
 - (g) `sum-total`
 - (h) `sum total`
 - (i) `sumtotal`
 - (j) `While`
 - (k) `x2`
 - (l) `Private`
 - (m) `public`
 - (n) `$16`
 - (o) `xTwo`
 - (p) `_static`
 - (q) `_4`
 - (r) `_____`
 - (s) `10%`
 - (t) `a27834`
 - (u) `wilma's`
15. What can you do if a variable name you would like to use is the same as a reserved word?
16. Why does C++ require programmers to declare a variable before using it? What are the advantages of declaring variables?
17. What is the difference between `float` and `double`?
18. How can a programmer force a floating-point literal to be a `float` instead of a `double`?
19. How is the value 2.45×10^{-5} expressed as a C++ literal?
20. How can you ensure that a variable's value can never be changed after its initialization?
21. How can you extend the range of `int` on some systems?
22. How can you extend the range and precision of `double` on some systems?
23. Write a program that prints the ASCII chart for all the values from 0 to 127.
24. Is `"i"` a string literal or character literal?
25. Is `'i'` a string literal or character literal?
26. Is it legal to assign a `char` value to an `int` variable?
27. Is it legal to assign an `int` value to a `char` variable?
28. What is printed by the following code fragment?

```
int x;  
x = 'A';  
cout << x << endl;
```

29. What is the difference between the character '**n**' and the character '**\n**'?
30. Write a C++ program that simply emits a beep sound when run.
31. Rewrite the following code fragment so that the code behaves exactly the same but does not use **endl**.

```
cout << endl << "Aye!" << endl << "Bye!" << endl;
```

32. Create an enumerated type that represents the days of the week.
33. Create an enumerated type that represents the months of the year.
34. Determine the exact type of each of the following variables:

- (a) **auto** a = 5;
- (b) **auto** b = **false**;
- (c) **auto** c = 9.3;
- (d) **auto** d = 5.1f;
- (e) **auto** e = 5L;

Chapter 4

Expressions and Arithmetic

This chapter uses the C++ numeric types introduced in Chapter 3 to build expressions and perform arithmetic. Some other important concepts are covered—user input, source formatting, comments, and dealing with errors.

4.1 Expressions

A literal value like 34 and a properly declared variable like **x** are examples of simple *expressions*. We can use operators to combine values and variables and form more complex expressions. Listing 4.1 (adder.cpp) shows how the addition operator (+) is used to add two integers.

Listing 4.1: adder.cpp

```
#include <iostream>

using namespace std;

int main() {
    int value1, value2, sum;
    cout << "Please enter two integer values: ";
    cin >> value1 >> value2;
    sum = value1 + value2;
    cout << value1 << " + " << value2 << " = " << sum << endl;
}
```

In Listing 4.1 (adder.cpp):

- **int value1, value2, sum;**

This statement declares three integer variables, but it does not initialize them. As we examine the rest of the program we will see that it would be superfluous to assign values to the variables here.

- **cout << "Please enter two integer values: ";**

This statement prompts the user to enter some information. This statement is our usual print statement, but it is not terminated with the end-of-line marker **endl**. This is because we want the cursor

to remain at the end of the printed line so when the user types in values they appear on the same line as the message prompting for the values. When the user presses the enter key to complete the input, the cursor will automatically move down to the next line.

- **`cin >> value1 >> value2;`**

This statement causes the program's execution to stop until the user types two numbers on the keyboard and then presses enter. The first number entered will be assigned to **`value1`**, and the second number entered will be assigned to **`value2`**. Once the user presses the enter key, the value entered is assigned to the variable. The user may choose to type one number, press enter, type the second number, and press enter again. Instead, the user may enter both numbers separated by one of more spaces and then press enter only once. The program will not proceed until the user enters two numbers.

The **`cin`** input stream object can assign values to multiple variables in one statement, as shown here:

```
int num1, num2, num3;
cin >> num1 >> num2 >> num3;
```

A common beginner's mistake is use commas to separate the variables, as in



```
int num1, num2, num3;
cin >> num1, num2, num3;
```

The compiler will not generate an error message, because it is legal C++ code. The statement, however, will not assign the three variables from user input as desired. The comma operator in C++ has different meanings in different contexts, and here it is treated like a statement separator; thus, the variables **`num2`** and **`num3`** are not involved with the **`cin`** input stream object. We will have no need to use the comma operator in this way, but you should be aware of this potential pitfall.

`cin` is a object that can be used to read input from the user. The **`>>`** operator—as used here in the context of the **`cin`** object—is known as the *extraction operator*. Notice that it is “backwards” from the **`<<`** operator used with the **`cout`** object. The **`cin`** object represents the input stream—information flowing into the program from user input from the keyboard. The **`>>`** operator extracts the data from the input stream **`cin`** and assigns the pieces of the data, in order, to the various variables on its right.

- **`sum = value1 + value2;`**

This is an assignment statement because it contains the assignment operator (**`=`**). The variable **`sum`** appears to the left of the assignment operator, so **`sum`** will receive a value when this statement executes. To the right of the assignment operator is an arithmetic expression involving two variables and the addition operator. The expression is *evaluated* by adding together the values of the two variables. Once the expression's value has been determined, that value can be assigned to the **`sum`** variable.

All expressions have a value. The process of determining the expression's value is called *evaluation*. Evaluating simple expressions is easy. The literal value 54 evaluates to 54. The value of a variable named **`x`** is the value stored in the memory location reserved for **`x`**. The value of a more complex expression is found by evaluating the smaller expressions that make it up and combining them with operators to form potentially new values.

Table 4.1 lists the main C++ arithmetic operators. Table 4.1. The common arithmetic operations, addition, subtraction, and multiplication, behave in the expected way. All these operators are classified as *binary* operators because they operate on two operands. In the statement

Operator	Meaning
+	addition
-	subtraction
*	multiplication
/	division
%	modulus

Table 4.1: The simple C++ arithmetic operators

```
x = y + z;
```

the right side is an addition expression **y + z**. The two operands of the **+** operator are **y** and **z**.

Two of the operators above, **+** and **-**, serve also as *unary* operators. A unary operator has only one operand. The **-** unary operator expects a single numeric expression (literal number, variable, or complex numeric expression within parentheses) immediately to its right; it computes the *additive inverse* of its operand. If the operand is positive (greater than zero), the result is a negative value of the same magnitude; if the operand is negative (less than zero), the result is a positive value of the same magnitude. Zero is unaffected. For example, the following code sequence

```
int x = 3;  
int y = -4;  
int z = 0;  
cout << -x << " " << -y << " " << -z << endl;
```

within a program would print

```
-3 4 0
```

The following statement

```
cout << -(4 - 5) << endl;
```

within a program would print

```
1
```

The unary **+** operator is present only for completeness; when applied to a numeric value, variable, or expression, the resulting value is no different from the original value of its operand. Omitting the unary **+** operator from the following statement

```
x = +y;
```

does not change the statement's behavior.

All the arithmetic operators are subject to the limitations of the data types on which they operate; for example, on a system in which the largest **int** is 2,147,483,647, the expression

```
2147483647 + 1
```


will not evaluate to the correct answer since the correct answer falls outside the range of `ints`.

If you add, subtract, multiply, or divide two `ints`, the result is an integer. As long as the operation does not exceed the range of `ints`, the arithmetic works as expected. Division, however, is another matter. The statement

```
cout << 10/3 << " " << 3/10 << endl;
```

prints

```
3 0
```

because in the first case 10 divided by 3 is 3 with a remainder of 1, and in the second case 3 divided by 10 is 0 with a remainder of 3. Since integers are whole numbers, any fractional part of the answer must be discarded. The process of discarding the fractional part leaving only the whole number part is called *truncation*. 10 divided by 3 should be 3.3333..., but that value is truncated to 3. Truncation is not rounding; for example, 11 divided by 3 is 3.6666..., but it also truncates to 3.



Truncation simply removes any fractional part of the value. It does not round. Both 10.01 and 10.999 truncate to 10.

The modulus operator (%) computes the remainder of integer division; thus,

```
cout << 10%3 << " " << 3%10 << endl;
```

prints

```
1 3
```

since 10 divided by 3 is 3 with a remainder of 1, and 3 divided by 10 is 0 with a remainder of 3. Figure 4.1 uses long division for a more hands on illustration of how the integer division and modulus operators work.

The modulus operator is more useful than it may first appear. Listing 4.11 (`timeconv.cpp`) shows how we can use it to convert a given number of seconds to hours, minutes, and seconds.

In contrast to integer arithmetic, floating-point arithmetic with `doubles` behaves as expected:

```
cout << 10.0/3.0 << " " << 3.0/10.0 << endl;
```

prints

```
3.33333 0.3
```

$$\begin{array}{r} 8 \\ 3 \overline{) 25} \\ \underline{-24} \\ 1 \end{array}$$

Figure 4.1: Integer division vs. integer modulus. Integer division produces the quotient, and modulus produces the remainder. In this example, $25/3$ is 8, and $25\%3$ is 1.

Recall from Section 3.4 that integers can be represented exactly, but floating-point numbers are imprecise approximations of real numbers. Listing 4.2 (`imprecise.cpp`) clearly demonstrates their weaknesses.

Listing 4.2: `imprecise.cpp`

```
#include <iostream>

using namespace std;

int main() {
    double one = 1.0,
           one_third = 1.0/3.0,
           zero = one - one_third - one_third - one_third;

    cout << "one = " << one << ", one_third = " << one_third
          << ", zero = " << zero << endl;
}
```

```
one = 1, one_third = 0.333333, zero = 1.11022e-016
```

The reported result is 1.11022×10^{-16} , or 0.000000000000000111022. While this number is very small, with mathematical real numbers we get

$$1 - \frac{1}{3} - \frac{1}{3} - \frac{1}{3} = 0$$

Floating-point numbers are not real numbers, so the result of `1.0/3.0` cannot be represented exactly without infinite precision. In the decimal (base 10) number system, one-third is a repeating fraction, so it has an infinite number of digits. Even simple non-repeating decimal numbers can be a problem. One-tenth (0.1) obviously is non-repeating, so it can be expressed exactly with a finite number of digits. As it turns out, since numbers within computers are stored in binary (base 2) form, even double-precision floating-point numbers cannot represent one-tenth exactly. Listing 4.3 (`imprecise10.cpp`) illustrates this fact.

Listing 4.3: imprecise10.cpp

```
#include <iostream>

using namespace std;

int main() {
    double one = 1.0,
           one_tenth = 1.0/10.0,
           zero = one - one_tenth - one_tenth - one_tenth
                  - one_tenth - one_tenth - one_tenth
                  - one_tenth - one_tenth - one_tenth
                  - one_tenth;

    cout << "one = " << one << ", one_tenth = " << one_tenth
          << ", zero = " << zero << endl;
}
```

The program's output is

```
one = 1, one_tenth = 0.1, zero = 1.38778e-016
```

Surely the reported answer (1.38778×10^{-16}) is close to the correct answer (zero). If you round it to the one-hundred trillionth place (15 places behind the decimal point), it is correct. Listing 5.5 (samedifferent.cpp) in Chapter 5 further emphasizes the point that programmers must use care when dealing with the imprecision of floating-point quantities.

Despite their inexactness, double-precision floating-point numbers are used every day throughout the world to solve sophisticated scientific and engineering problems. The limitations of floating-point numbers are unavoidable since computers have finite resources. Compromise is inevitable when we do our best to represent values with infinite characteristics in a finite way. Floating-point numbers provide a good trade-off of precision for practicality.

Since a `char` is stored internally as a number (see Section 3.7), we can perform arithmetic on characters. We will have little need to apply mathematics to characters, but sometimes it is useful. As an example, the lower-case letters of the alphabet a–z occupy ASCII values 97–123, with a = 97, b = 98, etc. The upper-case letters A–Z are coded as 65–91, with A = 65, B = 66, etc. To capitalize any lower-case letter, you need only subtract 32, as in

```
char lower = 'd', upper = lower - 32;
cout << upper << endl;
```

This section of code would print **D**. If you do not remember the offset of 32 between upper- and lower-case letter, you can compute it with the letters themselves:

```
upper = lower - ('a' - 'A');
```

In this case, if `lower` has been assigned any value in the range 'a' to 'z', the statement will assign to `upper` the capitalized version of `lower`. On the other hand, if `lower`'s value is outside of that range, `upper` will not receive a meaningful value.

4.2 Mixed Type Expressions

Expressions may contain mixed elements; for example, the following program fragment

```
int x = 4;
double y = 10.2, sum;
sum = x + y;
```

adds an `int` to a `double`, and the result is being assigned to a `double`. How is the arithmetic performed?

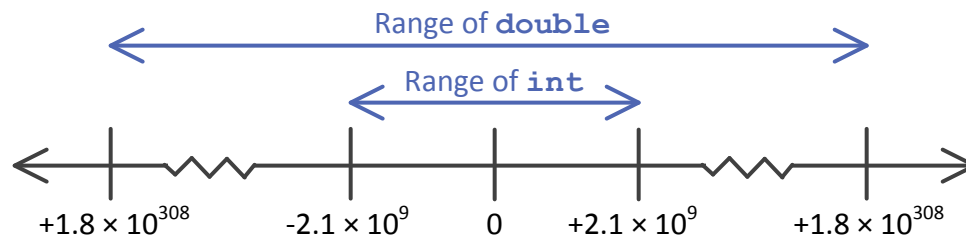


Figure 4.2: Range of ints vs. range of doubles

As shown in Figure 4.2, the range of `ints` falls completely within the range of `doubles`; thus, any `int` value can be represented by a `double`. The `int` 4 also can be expressed as the `double` 4.0. In fact, since the largest `int` on most systems is 2,147,483,647, the minimum 15 digits of `double` precision are more than adequate to represent all integers exactly. This means that any `int` value can be represented by a `double`. The converse is not true, however. 2,200,000,000 can be represented by a `double` but it is too big for the `int` type. We say that the `double` type is *wider* than the `int` type and that the `int` type is *narrower* than the `double` type.

It would be reasonable, then, to be able to assign `int` values to `double` variables. The process is called *widening*, and it is always safe to widen an `int` to a `double`. The following code fragment

```
double d1;
int i1 = 500;
d1 = i1;
cout << "d1 = " << d1 << endl;
```

is legal C++ code, and when part of a complete program it would display

```
d1 = 500
```

Assigning a `double` to an `int` variable is not always possible, however, since the `double` value may not be in the range of `ints`. Furthermore, if the `double` variable falls within the range of `ints` but is not a whole number, the `int` variable is unable to manage fractional part. Consider the following code fragment:

```
double d = 1.6;
int i = d;
```

The second line assigns 1 to `i`. Truncation loses the 0.6 fractional part (see Section 4.1). Note that proper rounding is not done. The Visual C++ compiler will warn us of a potential problem:

warning C4244: '=' : conversion from 'double' to 'int', possible loss of data

This warning reminds us that some information may be lost in the assignment. While the compiler and linker will generate an executable program when warnings are present, you should carefully scrutinize all warnings. This warning is particularly useful, since it is easy for errors due to the truncation of floating-point numbers to creep into calculations.

Converting from a wider type to a narrower type (like `double` to `int`) is called *narrowing*. It often is necessary to assign a floating-point value to an integer variable. If we know the value to assign is within the range of `ints`, and the value has no fractional parts or its truncation would do no harm, the assignment is safe. To perform the assignment without a warning from the compiler, we use a procedure called a *cast*, also called a *type cast*. The cast forces the compiler to accept the assignment without a warning. The following statement convinces the compiler to accept the `double`-to-`int` assignment without a warning:

```
i = static_cast<int>(d);
```

The reserved word `static_cast` performs the narrowing conversion and silences the compiler warning. The item to convert (in this case the variable `d`) is placed in the parentheses, and the desired type (in this case the type `int`) appears in the angle brackets. The statement

```
i = static_cast<int>(d);
```

does not change the type of the variable `d`; `d` is declared to be an `int` and so must remain an `int`. The statement makes a copy `d`'s value in a temporary memory location, converting it to its integer representation during the process.

We also can cast literal values and expressions:

```
i = static_cast<int>(1.6);  
i = static_cast<int>(x + 2.1);
```



Narrowing a floating-point value to an integer discards any fractional part. Narrowing truncates; it does not round. For example, the `double` value 1.7 narrows to the `int` value 1.

The widening conversion is always safe, so a type cast is not required. Narrowing is a potentially dangerous operation, and using an explicit cast does not remove the danger—it simply silences the compiler. For example, consider Listing 4.4 (`badnarrow.cpp`).

Listing 4.4: `badnarrow.cpp`

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    double d = 2200000000.0;
```

```
int i = d;  
cout << "d = " << d << ", i = " << i << endl;  
}
```

The Visual C++ compiler issues a warning about the possible loss of precision when assigning **d** to **i**. Silencing the warning with a type cast in this case is a bad idea; the program's output indicates that the warning should be heeded:

```
d = 2.2e+009, i = -2147483648
```

The printed values of **i** and **d** are not even close, nor can they be because it is impossible to represent the value 2,200,000,000 as an **int** on a system that uses 32-bit integers. When assigning a value of a wider type to a variable of a narrower type, the programmer must assume the responsibility to ensure that the actual value to be narrowed is indeed within the range of the narrower type. The compiler cannot ensure the safety of the assignment.

Casts should be used sparingly and with great care because a cast creates a spot in the program that is immune to the compiler's type checking. A careless assignment can produce a garbage result introducing an error into the program.

When we must perform mixed arithmetic—such as adding an **int** to a **double**—the compiler automatically produces machine language code that copies the **int** value to a temporary memory location and transforms it into its **double** equivalent. It then performs double-precision floating-point arithmetic to compute the result.

Integer arithmetic occurs only when both operands are **ints**. **1/3** thus evaluates to 0, but **1.0/3.0**, **1/3.0**, and **1.0/3** all evaluate to 0.33333.

Since **double** is wider than **int**, we say that **double** *dominates* **int**. In a mixed type arithmetic expression, the less dominant type is coerced into the more dominant type in order to perform the arithmetic operation.

Section 3.8 introduced enumerated types. Behind the scenes, the compiler translates enumerated values into integers. The first value in the enumeration is 0, the second value is 1, etc. The following code demonstrates the relationship between enumerated types and integers:

```
enum Color { Red, Orange, Yellow, Green, Blue, Violet };  
cout << Orange << " " << Green << endl;
```

The output of this code fragment is

```
1 3
```

C++ allows a programmer to assign an **enum** type to an integer variable; for example, the following statement is legal:

```
int num = Orange;
```

Here, the variable **num** assumes the value 1. A programmer, however, may not directly assign an integer value to a variable declared to be of an **enum** type; for example, the following statement:

```
Color col = 1;
```

is illegal. Sometimes it is convenient to convert an integer to an enumerated type, and a type cast enables the assignment:

```
enum Color { Red, Orange, Yellow, Green, Blue, Violet };  
Color myColor = static_cast<Color>(2);
```

Since the compiler encodes enumerated values with the integers 0, 1, 2, ... in the order the values are specified in the enumeration list, the assignment statement above assigns the value **Yellow** to the variable **myColor**. Casting, of course, can be dangerous; consider:

```
Color badColor = static_cast<Color>(45);
```

This statement assigns a meaningless value to the variable **badColor**, and the cast renders the compiler powerless to help.

Even though enumerated types are encoded as integers internally, programmers may not perform arithmetic on enumerated types without involving casts. Such opportunities should be very rare; if you need to perform arithmetic on a variable, it most likely should be a numerical type, not an enumerated type.

4.3 Operator Precedence and Associativity

When different operators are used in the same expression, the normal rules of arithmetic apply. All C++ operators have a *precedence* and *associativity*:

- **Precedence**—when an expression contains two different kinds of operators, which should be applied first?
- **Associativity**—when an expression contains two operators with the same precedence, which should be applied first?

To see how precedence works, consider the expression

```
2 + 3 * 4
```

Should it be interpreted as

```
(2 + 3) * 4
```

(that is, 20), or rather is

```
2 + (3 * 4)
```

(that is, 14) the correct interpretation? As in normal arithmetic, in C++ multiplication and division have equal importance and are performed before addition and subtraction. We say multiplication and division have precedence over addition and subtraction. In the expression

```
2 + 3 * 4
```

the multiplication is performed before addition, since multiplication has precedence over addition. The result is 14. The multiplicative operators (*****, **/**, and **%**) have equal precedence with each other, and the

additive operators (binary $+$ and $-$) have equal precedence with each other. The multiplicative operators have precedence over the additive operators.

As in standard arithmetic, in C++ if the addition is to be performed first, parentheses can override the precedence rules. The expression

$$(2 + 3) * 4$$

evaluates to 20. Multiple sets of parentheses can be arranged and nested in any ways that are acceptable in standard arithmetic.

To see how associativity works, consider the expression

$$2 - 3 - 4$$

The two operators are the same, so they have equal precedence. Should the first subtraction operator be applied before the second, as in

$$(2 - 3) - 4$$

(that is, -5), or rather is

$$2 - (3 - 4)$$

(that is, 3) the correct interpretation? The former (-5) is the correct interpretation. We say that the subtraction operator is *left associative*, and the evaluation is left to right. This interpretation agrees with standard arithmetic rules. All binary operators except assignment are left associative. Assignment is an exception; it is *right associative*. To see why associativity is an issue with assignment, consider the statement

$$\mathbf{w} = \mathbf{x} = \mathbf{y} = \mathbf{z};$$

This is legal C++ and is called *chained assignment*. Assignment can be used as both a statement and an expression. The *statement*

$$\mathbf{x} = 2;$$

assigns the value 2 to the variable $\mathbf{x}$. The *expression*

$$\mathbf{x} = 2$$

assigns the value 2 to the variable $\mathbf{x}$ and evaluates to the value that was assigned; that is, 2. Since assignment is right associative, the chained assignment example should be interpreted as

$$\mathbf{w} = (\mathbf{x} = (\mathbf{y} = \mathbf{z}));$$

which behaves as follows:

- The expression $\mathbf{y} = \mathbf{z}$ is evaluated first. $\mathbf{z}$'s value is assigned to $\mathbf{y}$, and the value of the expression $\mathbf{y} = \mathbf{z}$ is $\mathbf{z}$'s value.
- The expression $\mathbf{x} = (\mathbf{y} = \mathbf{z})$ is evaluated. The value of $\mathbf{y} = \mathbf{z}$, that is $\mathbf{z}$, is assigned to $\mathbf{x}$. The overall value of the expression $\mathbf{x} = \mathbf{y} = \mathbf{z}$ is thus the value of $\mathbf{z}$. Now the values of $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ are all equal (to $\mathbf{z}$).
- The expression $\mathbf{w} = (\mathbf{x} = \mathbf{y} = \mathbf{z})$ is evaluated. The value of the expression $\mathbf{x} = \mathbf{y} = \mathbf{z}$ is equal to $\mathbf{z}$'s value, so $\mathbf{z}$'s value is assigned to $\mathbf{w}$. The overall value of the expression $\mathbf{w} = \mathbf{x} = \mathbf{y} = \mathbf{z}$ is equal to $\mathbf{z}$, and the variables $\mathbf{w}$, $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ are all equal (to $\mathbf{z}$).

Arity	Operators	Associativity
Unary	<code>+</code> , <code>-</code>	
Binary	<code>*</code> , <code>/</code> , <code>%</code>	Left
Binary	<code>+</code> , <code>-</code>	Left
Binary	<code>=</code>	Right

Table 4.2: Operator precedence and associativity. The operators in each row have a higher precedence than the operators below it. Operators within a row have the same precedence.

As in the case of precedence, we can use parentheses to override the natural associativity within an expression.

The unary operators have a higher precedence than the binary operators, and the unary operators are right associative. This means the statements

```
cout << -3 + 2 << endl;
cout << -(3 + 2) << endl;
```

which display

```
-1
-5
```

behave as expected.

Table 4.2 shows the precedence and associativity rules for some C++ operators. The `*` operator also has a unary form that has nothing to do with mathematics; it is covered in Section 10.6.

4.4 Comments

Good programmers annotate their code by inserting remarks that explain the purpose of a section of code or why they chose to write a section of code the way they did. These notes are meant for human readers, not the compiler. It is common in industry for programs to be reviewed for correctness by other programmers or technical managers. Well-chosen identifiers (see Section 3.3) and comments can aid this assessment process. Also, in practice, teams of programmers develop software. A different programmer may be required to finish or fix a part of the program written by someone else. Well-written comments can help others understand new code quicker and increase their productivity modifying old or unfinished code. While it may seem difficult to believe, even the same programmer working on her own code months later can have a difficult time remembering what various parts do. Comments can help greatly.

Any text contained within comments is ignored by the compiler. C++ supports two types of comments: *single line comments* and *block comments*:

- **Single line comment**—the first type of comment is useful for writing a single line remark:

```
// Compute the average of the values
avg = sum / number;
```

The first line here is a comment that explains what the statement that follows it is supposed to do. The comment begins with the double forward slash symbols (`//`) and continues until the end

of that line. The compiler will ignore the `//` symbols and the contents of the rest of the line. This type of comment is also useful for appending a short comment to the end of a statement:

```
avg = sum / number;  // Compute the average of the values
```

Here, an executable statement and the comment appear on the same line. The compiler will read the assignment statement here, but it will ignore the comment. The compiler generates the same machine code for this example as it does for the preceding example, but this example uses one line of source code instead of two.

- **Block comment**—the second type of comment begins with the symbols `/*` and is in effect until the `*/` symbols are encountered. The `/* . . . */` symbols delimit the comment like parentheses delimit a parenthetical expression. Unlike parentheses, however, these block comments cannot be nested within other block comments.

The block comment is handy for multi-line comments:

```
/* After the computation is completed
   the result is displayed. */
cout << result << endl;
```

What should be commented? Avoid making a remark about the obvious; for example:

```
result = 0;  // Assign the value zero to the variable named result
```

The effect of this statement is clear to anyone with even minimal C++ programming experience. Thus, the audience of the comments should be taken into account; generally, “routine” activities require no remarks. Even though the *effect* of the above statement is clear, its *purpose* may need a comment. For example:

```
result = 0;  // Ensures 'result' has a well-defined minimum value
```

This remark may be crucial for readers to completely understand how a particular part of a program works. In general, programmers are not prone to providing too many comments. When in doubt, add a remark. The extra time it takes to write good comments is well worth the effort.

4.5 Formatting

Program comments are helpful to human readers but ignored by the compiler. Another aspect of source code that is largely irrelevant to the compiler but that people find valuable is its formatting. Imagine the difficulty of reading a book in which its text has no indentation or spacing to separate one paragraph from another. In comparison to the source code for a computer program, a book’s organization is quite simple. Over decades of software construction programmers have established a small collection of source code formatting styles that the industry finds acceptable.

The compiler allows a lot of leeway for source code formatting. Consider Listing 4.5 (reformattedvariable.cpp) which is a reformatted version of Listing 3.4 (variable.cpp).

Listing 4.5: reformattedvariable.cpp

```
#include <iostream>
using
namespace
std
```

```
;
int
main
(
)
{
int
x
;
x
=
10
;
cout
<<
x
<<
endl
;
}
```

Listing 4.6 (reformattedvariable2.cpp) is another reformatted version of Listing 3.4 (variable.cpp).

Listing 4.6: reformattedvariable2.cpp

```
#include <iostream>
using namespace std;int main(){int x;x=10;cout<<x<<endl;}
```

Both reformatted programs are valid C++ and compile to the same machine language code as the original version. Most would argue that the original version is easier to read and understand more quickly than either of the reformatted versions. The elements in Listing 3.4 (variable.cpp) are organized better. Experienced C++ programmers would find both Listing 4.5 (reformattedvariable.cpp) and Listing 4.6 (reformattedvariable2.cpp) visually painful.

What are some distinguishing characteristics of Listing 3.4 (variable.cpp)?

- Each statement appears on its own line. A statement is not unnecessarily split between two lines of text. Visually, one line of text implies one action (statement) to perform.
- The close curly brace aligns vertically with the line above that contains the corresponding open curly brace. This makes it easier to determine if the curly braces match and nest properly. It also better portrays the logical structure of the program. The ability to accurately communicate the logical structure of a program becomes very important as write more complex programs. Programs with complex logic frequently use multiple nested curly braces (for example, see Listing 5.11 (troubleshoot.cpp)). Without a consistent, organized arrangement of curly braces it can difficult to determine which opening brace goes with a particular closing brace.
- The statements that constitute the body of **main** are indented several spaces. This visually emphasizes the fact that the elements are indeed logically enclosed. As with curly brace alignment, indentation to emphasize logical enclosure becomes more important as more complex programs are considered.
- Spaces are used to spread out statements and group pieces of the statement. Space around the operators (=) makes it easier to visually separate the operands from the operators and comprehend the details of the expression. Most people find the statement

```
total_sale = subtotal + tax;
```

much easier to read than

```
total_sale=subtotal+tax;
```

since the lack of space in the second version makes it more difficult to pick out the individual pieces of the statement. In the first version with extra space, it is clearer where operators and variable names begin and end.

In a natural language like English, a book is divided into distinct chapters, and chapters are composed of paragraphs. One paragraph can be distinguished from another because the first line is indented or an extra space appears between two paragraphs. Space is used to separate words in each sentence. Consider how hard it would be to read a book if all the sentences were printed like this one:

Theboyranquicklytothetreetoseethestrandedcat.

Judiciously placed open space in a C++ program can greatly enhance its readability.



C++ gives the programmer a large amount of freedom in formatting source code. The compiler reads the characters that make up the source code one symbol at a time left to right within a line before moving to the next line. While extra space helps readability, spaces are not allowed in some places:

- Variable names and reserved words must appear as unbroken units.
- Multi-symbol operators like << cannot be separated (< < is illegal).

One common coding convention that is universal in C++ programming is demonstrated in Listing 3.10 (const.cpp). While programmers usually use lower-case letters in variable names, they usually express constant names with all capital letters; for example, **PI** is used for the mathematical constant π instead of **pi**. C++ does not require constants to be capitalized, but capitalizing them aids humans reading the source code so they can quickly distinguish between variables and constants.

Figure 4.3 shows the three most common ways programmers use indentation and place curly braces in C++ source code. The K&R and ANSI styles are the most popular in published C++ source code. The

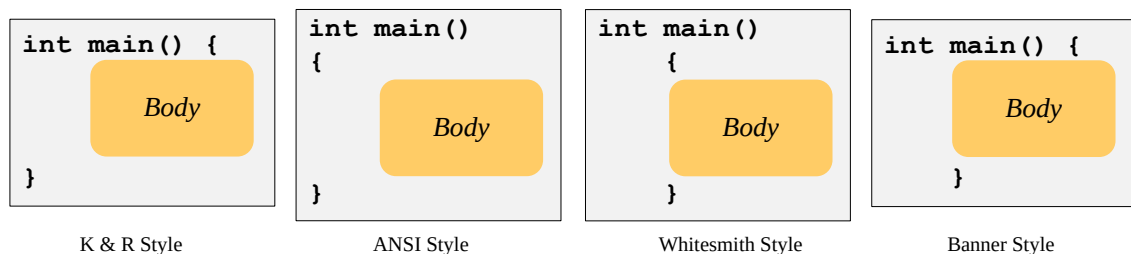


Figure 4.3: The most common C++ coding styles.

Whitesmith and Banner styles appear much less frequently. http://en.wikipedia.org/wiki/Indent_

style reviews the various ways to format C++ code. Observe that all the accepted formatting styles indent the block of statements contained in the `main` function.

Most software development organizations adopt a set of *style guidelines*, sometimes called *code conventions*. These guidelines dictate where to indent and by how many spaces, where to place curly braces, how to assign names to identifiers, etc. Programmers working for the organization are required to follow these style guidelines for the code they produce. This better enables any member of the development team to read and understand more quickly code written by someone else. This is necessary when code is reviewed for correctness or when code must be repaired or extended, and the original programmer is no longer with the development team.

Even if you are not forced to use a particular style, it is important to use a consistent style throughout the code you write. As our programs become more complex we will need to use additional curly braces and various levels of indentation to organize the code we write. A consistent style (especially one of the standard styles shown in Figure 4.3) makes it easier to read and verify that the code actually expresses our intent. It also makes it easier to find and fix errors. Said another way, haphazard formatting increases the time it takes to develop correct software because programmer's mistakes hide better in poorly formatted code.

Good software development tools can boost programmer productivity, and many programming editors have the ability to automatically format source code according to a standard style. Some of these editors can correct the code's style as the programmer types in the text. A standalone program known as a *pretty printer* can transform an arbitrarily formatted C++ source file into a properly formatted one.

4.6 Errors and Warnings

Beginning programmers make mistakes writing programs because of inexperience in programming in general or because of unfamiliarity with a programming language. Seasoned programmers make mistakes due to carelessness or because the proposed solution to a problem is faulty and the correct implementation of an incorrect solution will not produce a correct program. Regardless of the reason, a programming error falls under one of three categories:

- compile-time error
- run-time error
- logic error

4.6.1 Compile-time Errors

A *compile-time error* results from the programmer's misuse of the language. A *syntax error* is a common compile-time error. For example, in English one can say

The boy walks quickly.

This sentence uses correct syntax. However, the sentence

The boy walk quickly.

is not correct syntactically: the number of the subject (singular form) disagrees with the number of the verb (plural form). It contains a syntax error. It violates a grammatical rule of the English language. Similarly, the C++ statement

```
x = y + 2;
```

is syntactically correct because it obeys the rules for the structure of an assignment statement described in Section 3.2. However, consider replacing this assignment statement with a slightly modified version:

```
y + 2 = x;
```

If a statement like this one appears in a program and the variables **x** and **y** have been properly declared, the compiler will issue an error message; for example, the Visual C++ compiler reports (among other things):

error C2106: '=' : left operand must be l-value

The syntax of C++ does not allow an expression like **y** + 2 to appear on the left side of the assignment operator.

(The term *l-value* in the error message refers to the left side of the assignment operator; the **l** is an “elle,” not a “one.”)

The compiler may generate an error for a syntactically correct statement like

```
x = y + 2;
```

if either of the variables **x** or **y** has not been declared; for example, if **y** has not been declared, Visual C++ reports:

error C2065: 'y' : undeclared identifier

Other common compile-time errors include missing semicolons at the end of statements, mismatched curly braces and parentheses, and simple typographical errors.

Compile-time errors usually are the easiest to repair. The compiler pinpoints the exact location of the problem, and the error does not depend on the circumstances under which the program executes. The exact error can be reproduced by simply recompiling the same source code.

Compilers have the reputation for generating cryptic error messages. They seem to provide little help as far as novice programmers are concerned. Sometimes a combination of errors can lead to messages that indicate errors on lines that follow the line that contains the actual error. Once you encounter the same error several times and the compiler messages become more familiar, you become better able to deduce the actual problem from the reported message. Unfortunately C++ is such a complex language that sometimes a simple compile-time error can result in a message that is incomprehensible to beginning C++ programmers.

4.6.2 Run-time Errors

The compiler ensures that the structural rules of the C++ language are not violated. It can detect, for example, the malformed assignment statement and the use of a variable before its declaration. Some violations of the language cannot be detected at compile time, however. A program may not run to completion but instead terminate with an error. We commonly say the program “crashed.” Consider Listing 4.7 (dividedanger.cpp) which under certain circumstances will crash.

Listing 4.7: dividedanger.cpp

```
// File dividedanger.cpp

#include <iostream>

using namespace std;

int main() {
    int dividend, divisor;

    // Get two integers from the user
    cout << "Please enter two integers to divide:";
    cin >> dividend >> divisor;
    // Divide them and report the result
    cout << dividend << "/" << divisor << " = "
         << dividend/divisor << endl;
}
```

The expression

dividend/divisor

is potentially dangerous. If the user enters, for example, 32 and 4, the program works nicely

```
Please enter two integers to divide: 32 4
32/4 = 8
```

and displays the answer of 8. If the user instead types the numbers 32 and 0, the program reports an error and terminates. Division by zero is undefined in mathematics, and integer division by zero in C++ is illegal. When the program attempts the division at run time, the system detects the attempt and terminates the program.

This particular program can fail in other ways as well; for example, outside of the C++ world, 32.0 looks like a respectable integer. If the user types in 32.0 and 8, however, the program crashes because 32.0 is not a valid way to represent an integer in C++. When the compiler compiles the source line

```
cin >> dividend >> divisor;
```

given that **dividend** has been declared to be an **int**, it generates slightly different machine language code than it would if **dividend** has been declared to be a **double** instead. The compiled code expects the text entered by the user to be digits with no extra decoration. Any deviation from this expectation results in a run-time error. Similar results occur if the user enters text that does not represent an integer, like *fred*.

Observe that in either case—entry of a valid but inappropriate integer (zero) or entry of a non-integer (32.0 or *fred*)—it is impossible for the compiler to check for these problems at compile time. The compiler cannot predict what the user will enter when the program is run. This means it is up to the programmer to write code that can handle bad input that the user may provide. As we continue our exploration of programming in C++, we will discover ways to make our programs more robust against user input (see Listing 5.2 (betterdivision.cpp) in Chapter 5, for example). The solution involves changing the way the program runs depending on the actual input provided by the user.

4.6.3 Logic Errors

Consider the effects of replacing the expression

```
dividend/divisor;
```

in Listing 4.7 (dividedanger.cpp) with the expression:

```
divisor/dividend;
```

The program compiles with no errors. It runs, and unless a value of zero is entered for the dividend, no run-time errors arise. However, the answer it computes is not correct in general. The only time the correct answer is printed is when **dividend = divisor**. The program contains an error, but neither the compiler nor the run-time system is able to detect the problem. An error of this type is known as a *logic error*.

Listing 4.14 (faultytempconv.cpp) is an example of a program that contains a logic error. Listing 4.14 (faultytempconv.cpp) compiles and does not generate any run-time errors, but it produces incorrect results.

Beginning programmers tend to struggle early on with compile-time errors due to their unfamiliarity with the language. The compiler and its error messages are actually the programmer's best friend. As the programmer gains experience with the language and the programs written become more complicated, the number of compile-time errors decrease or are trivially fixed and the number of logic errors increase. Unfortunately, both the compiler and run-time environment are powerless to provide any insight into the nature and sometimes location of logic errors. Logic errors, therefore, tend to be the most difficult to find and repair. Tools such as debuggers are frequently used to help locate and fix logic errors, but these tools are far from automatic in their operation.

Errors that escape compiler detection (run-time errors and logic errors) are commonly called *bugs*. Since the compiler is unable to detect these problems, such bugs are the major source of frustration for developers. The frustration often arises because in complex programs the bugs sometimes only reveal themselves in certain situations that are difficult to reproduce exactly during testing. You will discover this frustration as your programs become more complicated. The good news is that programming experience and the disciplined application of good programming techniques can help reduce the number of logic errors. The bad news is that since software development is an inherently human intellectual pursuit, logic errors are inevitable. Accidentally introducing and later finding and eliminating logic errors is an integral part of the programming process.

4.6.4 Compiler Warnings

A warning issued by the compiler does mark a violation of the rules in the C++ language, but it is an notification to the programmer that the program contains a construct that is a potential problem. In Listing 4.10 (tempconv.cpp) the programmer is attempting to print the value of a variable before it has been given a known value.

Listing 4.8: uninitialized.cpp

```
// uninitialized.cpp

#include <iostream>

using namespace std;

int main() {
    int n;
```



```
    cout << n << endl;
}
```

An attempt to build Listing 4.8 (uninitialized.cpp) yields the following message from the Visual C++ compiler:

warning C4700: uninitialized local variable 'n' used

The compiler issued a warning but still generated the executable file. When run, the program produces a random result because it prints the value in memory associated with the variable, but the program does not initialize that memory location.

Listing 4.9 (narrow.cpp) assigns a `double` value to an `int` variable, which we know from Section 4.1 truncates the result.

Listing 4.9: narrow.cpp

```
#include <iostream>

using namespace std;

int main() {
    int n;
    double d = 1.6;
    n = d;
    cout << n << endl;
}
```

When compiled we see

warning C4244: '=' : conversion from 'double' to 'int', possible loss of data

Since it is a warning and not an error, the compiler generates the executable, but the warning should prompt us to stop and reflect about the correctness of the code. The enhanced warning level prevents the programmer from being oblivious to the situation.

The default Visual C++ warning level is 3 when compiling in the IDE and level 1 on the command line (that is why we use the `/W3` option on the command line); the highest warning level is 4. You can reduce the level to 1 or 2 or disable warnings altogether, but that is not recommended. The only reason you might want to reduce the warning level is to compile older existing C++ source code that does meet newer C++ standards. When developing new code, higher warning levels are preferred since they provide more help to the programmer. Unless otherwise noted, all the complete program examples in this book compile cleanly under Visual C++ set at warning level 3. Level 3 is helpful for detecting many common logic errors.

We can avoid most warnings by a simple addition to the code. Section 4.2 showed how we can use `static_cast` to coerce a wider type to a narrower type. At Visual C++ warning Level 3, the compiler issues a warning if the cast is not used. The little code that must be added should cause the programmer to stop and reflect about the correctness of the construct. The enhanced warning level prevents the programmer from being oblivious to the situation.



Use the strongest level of warnings available to your compiler. Treat all warnings as problems that must be corrected. Do not accept as completed a program that compiles with warnings.

4.7 Arithmetic Examples

The kind of arithmetic to perform in a complex expression is determined on an operator by operator basis. For example, consider Listing 4.10 (tempconv.cpp) that attempts to convert a temperature from degrees Fahrenheit to degrees Celsius using the formula

$$^{\circ}\text{C} = \frac{5}{9} \times (^{\circ}\text{F} - 32)$$

Listing 4.10: tempconv.cpp

```
// File tempconv.cpp

#include <iostream>

using namespace std;

int main() {
    double degreesF, degreesC;
    // Prompt user for temperature to convert
    cout << "Enter the temperature in degrees F: ";
    // Read in the user's input
    cin >> degreesF;
    // Perform the conversion
    degreesC = 5/9*(degreesF - 32);
    // Report the result
    cout << degreesC << endl;
}
```

Listing 4.10 (tempconv.cpp) contains comments that document each step explaining the code's purpose. An initial test is promising:

```
Enter the temperature in degrees F: 32
Degrees C = 0
```

Water freezes at 32 degrees Fahrenheit and 0 degrees Celsius, so the program's behavior is correct for this test. Several other attempts are less favorable—consider

```
Enter the temperature in degrees F: 212
Degrees C = 0
```

Water boils at 212 degrees Fahrenheit which is 100 degrees Celsius, so this answer is not correct.

```
Enter the temperature in degrees F: -40
Degrees C = 0
```

The value -40 is the point where the Fahrenheit and Celsius curves cross, so the result should be -40 , not zero. The first test was only *coincidentally correct*.

Unfortunately, the printed result is always zero regardless of the input. The problem is the division $5/9$ in the statement

```
degreesC = 5/9*(degreesF - 32);
```

Division and multiplication have equal precedence, and both are left associative; therefore, the division is performed first. Since both operands are integers, integer division is performed and the quotient is zero (5 divided by 9 is 0, remainder 5). Of course zero times any number is zero, thus the result. The fact that a floating-point value is involved in the expression (**degreesF**) and the overall result is being assigned to a floating-point variable, is irrelevant. The decision about the exact type of operation to perform is made on an operator-by-operator basis, not globally over the entire expression. Since the division is performed first and it involves two integer values, integer division is used before the other floating-point pieces become involved.

One solution simply uses a floating-point literal for either the five or the nine, as in

```
degreesC = 5.0/9*(degreesF - 32);
```

This forces a double-precision floating-point division (recall that the literal **5.0** is a **double**). The correct result, subject to rounding instead of truncation, is finally computed.

Listing 4.11 (timeconv.cpp) uses integer division and modulus to split up a given number of seconds to hours, minutes, and seconds.

Listing 4.11: timeconv.cpp

```
// File timeconv.cpp

#include <iostream>

using namespace std;

int main() {
    int hours, minutes, seconds;
    cout << "Please enter the number of seconds:";
    cin >> seconds;
    // First, compute the number of hours in the given number
    // of seconds
    hours = seconds / 3600; // 3600 seconds = 1 hours
    // Compute the remaining seconds after the hours are
    // accounted for
    seconds = seconds % 3600;
    // Next, compute the number of minutes in the remaining
    // number of seconds
    minutes = seconds / 60; // 60 seconds = 1 minute
```

```

// Compute the remaining seconds after the minutes are
// accounted for
seconds = seconds % 60;
// Report the results
cout << hours << " hr, " << minutes << " min, "
    << seconds << " sec" << endl;
}

```

If the user enters 10000, the program prints **2 hr, 46 min, 40 sec**. Notice the assignments to the **seconds** variable, such as

```
seconds = seconds % 3600
```

The right side of the assignment operator (=) is first evaluated. The remainder of **seconds** divided by 3,600 is assigned back to **seconds**. This statement can alter the value of **seconds** if the current value of **seconds** is greater than 3,600. A similar statement that occurs frequently in programs is one like

```
x = x + 1;
```

This statement increments the variable **x** to make it one bigger. A statement like this one provides further evidence that the C++ assignment operator does not mean mathematical equality. The following statement from mathematics

$$x = x + 1$$

is surely never true; a number cannot be equal to one more than itself. If that were the case, I would deposit one dollar in the bank and then insist that I really had two dollars in the bank, since a number is equal to one more than itself. That two dollars would become 3.00, then 4.00, etc., and soon I would be rich. In C++, however, this statement simply means “add one to **x** and assign the result back to **x**.”

A variation on Listing 4.11 (timeconv.cpp), Listing 4.12 (enhancedtimeconv.cpp) performs the same logic to compute the time pieces (hours, minutes, and seconds), but it uses more simple arithmetic to produce a slightly different output—instead of printing 11,045 seconds as **3 hr, 4 min, 5 sec**, Listing 4.12 (enhancedtimeconv.cpp) displays it as **3 : 04 : 05**. It is trivial to modify Listing 4.11 (timeconv.cpp) so that it would print **3 : 4 : 5**, but Listing 4.12 (enhancedtimeconv.cpp) includes some extra arithmetic to put leading zeroes in front of single-digit values for minutes and seconds as is done on digital clock displays.

Listing 4.12: enhancedtimeconv.cpp

```

// File enhancedtimeconv.cpp

#include <iostream>

using namespace std;

int main() {
    int hours, minutes, seconds;
    cout << "Please enter the number of seconds:";
    cin >> seconds;
    // First, compute the number of hours in the given number
    // of seconds
    hours = seconds / 3600; // 3600 seconds = 1 hours
    // Compute the remaining seconds after the hours are
    // accounted for
    seconds = seconds % 3600;
    // Next, compute the number of minutes in the remaining

```

```

// number of seconds
minutes = seconds / 60; // 60 seconds = 1 minute
// Compute the remaining seconds after the minutes are
// accounted for
seconds = seconds % 60;
// Report the results
cout << hours << ":";
// Compute tens digit of minutes
int tens = minutes / 10;
cout << tens;
// Compute ones digit of minutes
int ones = minutes % 10;
cout << ones << ":";
// Compute tens digit of seconds
tens = seconds / 10;
cout << tens;
// Compute ones digit of seconds
ones = seconds % 10;
cout << ones << endl;
}

```

Listing 4.12 (enhancedtimeconv.cpp) uses the fact that if x is a one- or two-digit number, $x \% 10$ is the tens digit of x . If $x \% 10$ is zero, x is necessarily a one-digit number.

4.8 More Arithmetic Operators

As Listing 4.12 (enhancedtimeconv.cpp) demonstrates, an executing program can alter a variable's value by performing some arithmetic on its current value. A variable may increase by one or decrease by five. The statement

```
x = x + 1;
```

increments x by one, making it one bigger than it was before this statement was executed. C++ has a shorter statement that accomplishes the same effect:

```
x++;
```

This is the *increment* statement. A similar *decrement* statement is available:

```
x--;    // Same as x = x - 1;
```

These statements are more precisely *post-increment* and *post-decrement* operators. There are also *pre-increment* and *pre-decrement* forms, as in

```
--x;    // Same as x = x - 1;
++y;    // Same as y = y + 1;
```

When they appear alone in a statement, the pre- and post- versions of the increment and decrement operators work identically. Their behavior is different when they are embedded within a more complex statement. Listing 4.13 (prevspost.cpp) demonstrates how the pre- and post- increment operators work slightly differently.

Listing 4.13: prevspost.cpp

```

#include <iostream>

using namespace std;

int main() {
    int x1 = 1, y1 = 10, x2 = 100, y2 = 1000;
    cout << "x1=" << x1 << ", y1=" << y1
         << ", x2=" << x2 << ", y2=" << y2 << endl;
    y1 = x1++;
    cout << "x1=" << x1 << ", y1=" << y1
         << ", x2=" << x2 << ", y2=" << y2 << endl;
    y2 = ++x2;
    cout << "x1=" << x1 << ", y1=" << y1
         << ", x2=" << x2 << ", y2=" << y2 << endl;
}

```

Listing 4.13 (prevspost.cpp) prints

```

x1=1, y1=10, x2=100, y2=1000
x1=2, y1=1, x2=100, y2=1000
x1=2, y1=1, x2=101, y2=101

```

If **x1** has the value 1 just before the statement

```
y1 = x1++;
```

then immediately after the statement executes **x1** is 2 and **y1** is 1.

If **x1** has the value 1 just before the statement

```
y1 = ++x1;
```

then immediately after the statement executes **x1** is 2 and **y1** is also 2.

As you can see, the pre-increment operator uses the new value of the incremented variable when evaluating the overall expression. In contrast, the post-increment operator uses the original value of the incremented variable when evaluating the overall expression. The pre- and post-decrement operator behaves similarly.

For beginning programmers it is best to avoid using the increment and decrement operators within more complex expressions. We will use them frequently as standalone statements since there is no danger of misinterpreting their behavior when they are not part of a more complex expression.

C++ provides a more general way of simplifying a statement that modifies a variable through simple arithmetic. For example, the statement

```
x = x + 5;
```

can be shorted to

```
x += 5;
```

This statement means “increase **x** by five.” Any statement of the form

$$\mathbf{x\ op=\ exp;}$$

where

- $\mathbf{x}$ is a variable.
- $\mathbf{op=}$ is an arithmetic operator combined with the assignment operator; for our purposes, the ones most useful to us are $\mathbf{+=}$, $\mathbf{-=}$, $\mathbf{*=}$, $\mathbf{/=}$, and $\mathbf{\%=}$.
- $\mathbf{exp}$ is an expression compatible with the variable $\mathbf{x}$.

Arithmetic reassignment statements of this form are equivalent to

$$\mathbf{x = x\ op\ exp;}$$

This means the statement

$$\mathbf{x\ *=\ y\ +\ z;}$$

is equivalent to

$$\mathbf{x = x\ *\ (y\ +\ z);}$$

The version using the arithmetic assignment does not require parentheses. The arithmetic assignment is especially handy if a variable with a long name is to be modified; consider

$$\mathbf{temporary_filename_length = temporary_filename_length / (y + z);}$$

versus

$$\mathbf{temporary_filename_length /= y + z;}$$

Do not accidentally reverse the order of the symbols for the arithmetic assignment operators, like in the statement

$$\mathbf{x\ =+ 5;}$$

Notice that the $\mathbf{+}$ and $\mathbf{=}$ symbols have been reversed. The compiler interprets this statement as if it had been written


$$\mathbf{x = +5;}$$

that is, assignment and the unary operator. This assigns $\mathbf{x}$ to exactly five instead of increasing it by five.

Similarly,

$$\mathbf{x\ -=\ 3;}$$

would assign $\mathbf{-3}$ to $\mathbf{x}$ instead of decreasing $\mathbf{x}$ by three.

Appendix E examines some additional arithmetic operators available in C++.

4.9 Algorithms

An *algorithm* is a finite sequence of steps, each step taking a finite length of time, that solves a problem or computes a result. A computer program is one example of an algorithm, as is a recipe to make lasagna. In both of these examples, the order of the steps matter. In the case of lasagna, the noodles must be cooked in boiling water before they are layered into the filling to be baked. It would be inappropriate to place the raw noodles into the pan with all the other ingredients, bake it, and then later remove the already baked noodles to cook them in boiling water separately. In the same way, the ordering of steps is very important in a computer program. While this point may be obvious, consider the following sound argument:

1. The relationship between degrees Celsius and degrees Fahrenheit can be expressed as

$$^{\circ}\text{C} = \frac{5}{9} \times (^{\circ}\text{F} - 32)$$

2. Given a temperature in degrees Fahrenheit, the corresponding temperature in degrees Celsius can be computed.

Armed with this knowledge, Listing 4.14 (faultytempconv.cpp) follows directly.

Listing 4.14: faultytempconv.cpp

```
// File faultytempconv.cpp

#include <iostream>

using namespace std;

int main() {
    double degreesF = 0, degreesC = 0;
    // Define the relationship between F and C
    degreesC = 5.0/9*(degreesF - 32);
    // Prompt user for degrees F
    cout << "Enter the temperature in degrees F: ";
    // Read in the user's input
    cin >> degreesF;
    // Report the result
    cout << degreesC << endl;
}
```

Unfortunately, the executing program always displays

```
-17.7778
```

regardless of the input provided. The English description provided above is correct. No integer division problems lurk, as in Listing 4.10 (tempconv.cpp). The problem lies simply in statement ordering. The statement

```
degreesC = 5.0/9*(degreesF - 32);
```


is an *assignment* statement, not a definition of a relationship that exists throughout the program. At the point of the assignment, **degreesF** has the value of zero. The executing program computes and assigns the **degreesC** variable *before* receiving **degreesF**'s value from the user.

As another example, suppose **x** and **y** are two integer variables in some program. How would we interchange the values of the two variables? We want **x** to have **y**'s original value and **y** to have **x**'s original value. This code may seem reasonable:

```
x = y;  
y = x;
```

The problem with this section of code is that after the first statement is executed, **x** and **y** both have the same value (**y**'s original value). The second assignment is superfluous and does nothing to change the values of **x** or **y**. The solution requires a third variable to remember the original value of one of the variables before it is reassigned. The correct code to swap the values is

```
temp = x;  
x = y;  
y = temp;
```

This small example emphasizes the fact that algorithms must be specified precisely. Informal notions about how to solve a problem can be valuable in the early stages of program design, but the coded program requires a correct detailed description of the solution.

The algorithms we have seen so far have been simple. Statement 1, followed by Statement 2, etc. until every statement in the program has been executed. Chapter 5 and Chapter 6 introduce some language constructs that permit optional and repetitive execution of some statements. These constructs allow us to build programs that do much more interesting things, but more complex algorithms are required to make it happen. We must not lose sight of the fact that a complicated algorithm that is 99% correct is *not* correct. An algorithm's design and implementation can be derailed by inattention to the smallest of details.

4.10 Summary

- The literal value **4** and integer **sum** are examples of simple C++ numeric expressions.
- **2*x + 4** is an example of a more complex C++ numeric expression.
- Expressions can be printed via the **cout** output stream object and assigned to variables.
- A binary operator performs an operation using two operands.
- With regard to binary operators: **+** represents arithmetic addition; **-** represents arithmetic subtraction; ***** represents arithmetic multiplication; **/** represents arithmetic division; **%** represents arithmetic modulus, or integer remainder after division.
- The **std::cin** object can be used to assign user keyboard input to variables when the program is executing.
- The **>>** operator is called the extraction operator. It assigns to variables data extracted from the input stream object **cin**.
- A unary operator performs an operation using one operand.
- The **-** unary operator represents the additive inverse of its operand.

- The `+` unary operator has no effect on its operand.
- Arithmetic applied to integer operands yields integer results.
- With a binary operation, double-precision floating-point arithmetic is performed if at least one of its operands is a floating-point number.
- Floating-point arithmetic is inexact and subject to rounding errors because all floating-point values have finite precision.
- When a floating-point value is assigned to an integer variable, the value is truncated, not properly rounded.
- A mixed expression is an expression that contains values and/or variables of differing types.
- Generally speaking, a type is narrower than another type if its range of values is smaller than the other type; a type is wider than another type if its range of values is larger than the other type.
- A wider type dominates a narrower type.
- In mixed arithmetic with a binary operator, the type of arithmetic performed is determined by the more dominant operand.
- Floating point values can be assigned to integer variables, but any fractional part will be truncated. Programmers must ensure that the value assigned falls within the range of integers.
- In C++ a wider type may be assigned to a narrower type, but a floating-point to integer conversion truncates, and a value outside the range of the narrower type results in a bogus value being assigned. Programmers should use caution when assigning a wider type to a narrower type.
- In C++, operators have both a precedence and an associativity.
- With regard to the arithmetic operators, C++ uses the same precedence rules as standard arithmetic: multiplication and division are applied before addition and subtraction unless parentheses dictate otherwise.
- The arithmetic operators associate left to right; assignment associates right to left.
- Chained assignment can be used to assign the same value to multiple variables within one statement.
- The unary operators `+` and `-` have precedence over the binary arithmetic operators `*`, `/`, and `%`, which have precedence over the binary arithmetic operators `+` and `-`, which have precedence over the assignment operator.
- Comments are notes within the source code. The compiler ignores comments when compiling the source code.
- Comments inform human readers about the code.
- Comments should not state the obvious, but it is better to provide too many comments rather than too few.
- A single line comment begins with the symbols `//` and continues until the end of the line.
- A block comment begins with the symbols `/*` and continues in the source code until the symbols `*/` terminate it.
- Source code should be formatted so that it is more easily read and understood by humans.

- C++ has some general formatting guidelines that should be followed:
 - Each statement should appear on its own line.
 - Curly braces should be aligned in order to be matched visually more quickly.
 - The statements that comprise the body of a function should be indented; four spaces are generally accepted as ideal.
 - Spaces should be used within statements separating the logical pieces to make the statements easier to read.
- Rewrite Listing 4.1 (`adder.cpp`) in four different ways:
 1. Use the K & R coding style
 2. Use the ANSI coding style
 3. Use the Whitesmith coding style
 4. Use the Banner coding style
- Compile-time errors are caused by the programmer's misuse of the C++ language. Compile-time errors are detected and reported by the compiler (and sometimes the linker).
- Runtime errors are errors that are detected when the program is executing. When a run-time error arises, the program terminates with an error message.
- Logic errors elude detection by the compiler and run-time environment. A logic error is indicated when the the program does not behave as expected.
- The compiler should be set at the highest warning level to check for as many problems as possible. Warnings should be taken seriously, and a program should not be considered finished while warnings remain.
- In complicated arithmetic expressions involving many operators and operands, the rules pertaining to mixed arithmetic are applied on an operator-by-operator basis, following the precedence and associativity laws, not globally over the entire expression.
- The `++` and `--` operators increment and decrement variables.
- The family of `op=` operators (`+=`, `-=`, `*=`, `/=`, and `%=`) allow variables to be changed by a given amount using a particular arithmetic operator.
- C++ programs implement algorithms; as such, C++ statements do not declare statements of fact or define relationships that hold throughout the program's execution; rather they indicate how the values of variables change as the execution of the program progresses.

4.11 Exercises

1. Is the literal `4` a valid C++ expression?
2. Is the variable `x` a valid C++ expression?
3. Is `x + 4` a valid C++ expression?
4. What affect does the unary `+` operator have when applied to a numeric expression?

5. Sort the following binary operators in order of high to low precedence: $+$, $-$, $*$, $/$, $\%$, $=$.
6. Write a C++ program that receives two integer values from the user. The program then should print the sum (addition), difference (subtraction), product (multiplication), quotient (division), and remainder after division (modulus). Your program must use only integers.

A sample program run would look like (the user enters the 10 and the 2 after the colons, and the program prints the rest):

```
Please enter the first number: 10
Please enter the second number: 2
10 + 2 = 12
10 - 2 = 8
10 * 2 = 20
10 / 2 = 5
10 % 2 = 0
```

Can you explain the results it produces for all of these operations?

7. Write a C++ program that receives two double-precision floating-point values from the user. The program then should print the sum (addition), difference (subtraction), product (multiplication), and quotient (division). Your program should use only integers.

A sample program run would look like (the user enters the 10 and the 2.5 after the colons, and the program prints the rest):

```
Please enter the first number: 10
Please enter the second number: 2.5
10 + 2.5 = 12.5
10 - 2.5 = 7.5
10 * 2.5 = 25
10 / 2.5 = 4
```

Can you explain the results it produces for all these operations? What happens if you attempt to compute the remainder after division (modulus) with double-precision floating-point values?

8. Given the following declaration:

```
int x = 2;
```

Indicate what each of the following C++ statements would print.

- (a) `cout << "x" << endl;`
 - (b) `cout << 'x' << endl;`
 - (c) `cout << x << endl;`
 - (d) `cout << "x + 1" << endl;`
 - (e) `cout << 'x' + 1 << endl;`
 - (f) `cout << x + 1 << endl;`
9. Sort the following types in order from narrowest to widest: `int`, `double`, `float`, `long`, `char`.

10. Given the following declarations:

```
int i1 = 2, i2 = 5, i3 = -3;
double d1 = 2.0, d2 = 5.0, d3 = -0.5;
```

Evaluate each of the following C++ expressions.

- (a) `i1 + i2`
- (b) `i1 / i2`
- (c) `i2 / i1`
- (d) `i1 * i3`
- (e) `d1 + d2`
- (f) `d1 / d2`
- (g) `d2 / d1`
- (h) `d3 * d1`
- (i) `d1 + i2`
- (j) `i1 / d2`
- (k) `d2 / i1`
- (l) `i2 / d1`
- (m) `i1/i2*d1`
- (n) `d1*i1/i2`
- (o) `d1/d2*i1`
- (p) `i1*d1/d2`
- (q) `i2/i1*d1`
- (r) `d1*i2/i1`
- (s) `d2/d1*i1`
- (t) `i1*d2/d1`

11. What is printed by the following statement:

```
cout << /* 5 */ 3 << endl;
```

12. Given the following declarations:

```
int i1 = 2, i2 = 5, i3 = -3;
double d1 = 2.0, d2 = 5.0, d3 = -0.5;
```

Evaluate each of the following C++ expressions.

- (a) `i1 + (i2 * i3)`
- (b) `i1 * (i2 + i3)`
- (c) `i1 / (i2 + i3)`
- (d) `i1 / i2 + i3`
- (e) `3 + 4 + 5 / 3`
- (f) `(3 + 4 + 5) / 3`
- (g) `d1 + (d2 * d3)`

- (h) $d1 + d2 * d3$
- (i) $d1 / d2 - d3$
- (j) $d1 / (d2 - d3)$
- (k) $d1 + d2 + d3 / 3$
- (l) $(d1 + d2 + d3) / 3$
- (m) $d1 + d2 + (d3 / 3)$
- (n) $3 * (d1 + d2) * (d1 - d3)$

13. How are single-line comments different from block comments?
14. Can block comments be nested?
15. Which is better, too many comments or too few comments?
16. What is the purpose of comments?
17. The programs in Listing 3.4 (variable.cpp), Listing 4.5 (reformattedvariable.cpp), and Listing 4.6 (reformattedvariable2.cpp) compile to the same machine code and behave exactly the same. What makes one of the programs clearly better than the others?
18. Why is human readability such an important consideration?
19. Consider the following program which contains some errors. You may assume that the comments within the program accurately describe the program's intended behavior.

```
#include <iostream>

using namespace std;

int main() {
    int n1, n2, d1;           // 1
    // Get two numbers from the user
    cin << n1 << n2;          // 2
    // Compute sum of the two numbers
    cout << n1 + n2 << endl;   // 3
    // Compute average of the two numbers
    cout << n1+n2/2 << endl;   // 4
    // Assign some variables
    d1 = d2 = 0;              // 5
    // Compute a quotient
    cout << n1/d1 << endl;     // 6
    // Compute a product
    n1*n2 = d1;               // 7
    // Print result
    cout << d1 << endl;       // 8
}
```

For each line listed in the comments, indicate whether or not a compile-time, run-time, or logic error is present. Not all lines contain an error.

20. What distinguishes a compiler warning from a compiler error? Should you be concerned about warnings? Why or why not?

21. What are the advantages to enhancing the warning reporting capabilities of the compiler?

22. Write the shortest way to express each of the following statements.

- (a) `x = x + 1;`
- (b) `x = x / 2;`
- (c) `x = x - 1;`
- (d) `x = x + y;`
- (e) `x = x - (y + 7);`
- (f) `x = 2*x;`
- (g) `number_of_closed_cases = number_of_closed_cases + 2*ncc;`

23. What is printed by the following code fragment?

```
int x1 = 2, y1, x2 = 2, y2;
y1 = ++x1;
y2 = x2++;
cout << x1 << " " << x2 << endl;
cout << y1 << " " << y2 << endl;
```

Why does the output appear as it does?

24. Consider the following program that attempts to compute the circumference of a circle given the radius entered by the user. Given a circle's radius, r , the circle's circumference, C is given by the formula:

$$C = 2\pi r$$

```
#include <iostream>

using namespace std;

int main() {
    double C, r;
    const double PI = 3.14159;
    // Formula for the area of a circle given its radius
    C = 2*PI*r;
    // Get the radius from the user
    cout >> "Please enter the circle's radius: ";
    cin << r;
    // Print the circumference
    cout << "Circumference is " << C << endl;
}
```

- (a) The compiler issues a warning. What is the warning?
- (b) The program does not produce the intended result. Why?
- (c) How can it be repaired so that it not only eliminates the warning but also removes the logic error?

Food	Calories
Bean burrito	357
Salad w/dressing	185
Milkshake	388

Table 4.3: Calorie content of several fast food items

25. In mathematics, the midpoint between the two points (x_1, y_1) and (x_2, y_2) is computed by the formula

$$\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$$

Write a C++ program that receives two mathematical points from the user and computes and prints their midpoint.

A sample run of the program produces

```
Please enter the first point: (0,0)
Please enter the second point: (1,1)
The midpoint of (0,0) and (1,1) is (0.5,0.5)
```

The user literally enters "(0,0)" and "(1,1)" with the parentheses and commas as shown. To see how to do this, suppose you want to allow a user to enter the point (2.3,9), assigning the x component of the point to a variable named **x** and the y component to a variable named **y**. You can add the following code fragment to your program to achieve the desired effect:

```
double x, y;
char left_paren, comma, right_paren;
cin >> left_paren >> x >> comma >> y >> right_paren;
```

If the user literally types (2.3,9), the **cin** statement will assign the (character to the variable **left_paren**. It next will assign 2.3 to the variable **x**. It assigns the , character to the variable named **comma**, the value 9 to the **y** variable, and the) character to the **right_paren** variable. The **left_paren**, **comma**, and **right_paren** variables are just placeholders for the user's input and are not used elsewhere within the program. In reality, the user can type in other characters in place of the parentheses and comma as long as the numbers are in the proper location relative to the characters; for example, the user can type *2.3:9#, and the program will interpret the input as the point (2.3,9).

26. Table 4.3 lists the Calorie contents of several foods. Running or walking burns off about 100 Calories per mile. Write a C++ program that requests three values from the user: the number of bean burritos, salads, and shakes consumed (in that order). The program should then display the number of miles that must be run or walked to burn off the Calories represented in that food. The program should run as follows (the user types in the 3 2 1):

```
How many bean burritos, bowls of salad, and milkshakes did you consume?

You ingested 1829 Calories
You will have to run 18.29 miles to expend that much energy
```

Observe that the result is a floating-point value, so you should use floating-point arithmetic to compute the answers for this problem.

Chapter 5

Conditional Execution

All the programs in the preceding chapters execute exactly the same statements regardless of the input, if any, provided to them. They follow a linear sequence: *Statement 1*, *Statement 2*, etc. until the last statement is executed and the program terminates. Linear programs like these are very limited in the problems they can solve. This chapter introduces constructs that allow program statements to be optionally executed, depending on the context (input) of the program's execution.

5.1 Type `bool`

Arithmetic expressions evaluate to numeric values; a *Boolean* expression, sometimes called a *predicate*, evaluates to `true` or `false`. While Boolean expressions may appear very limited on the surface, they are essential for building more interesting and useful programs.

C++ supports the non-numeric data type `bool`, which stands for Boolean. The term Boolean comes from the name of the British mathematician George Boole. A branch of discrete mathematics called Boolean algebra is dedicated to the study of the properties and the manipulation of logical expressions. Compared to the numeric types, the `bool` type is very simple in that it can represent only two values: `true` or `false`. Listing 5.1 (`boolvars.cpp`) is a simple program demonstrating the use of Boolean variables.

Listing 5.1: `boolvars.cpp`

```
#include <iostream>

using namespace std;

int main() {
    // Declare some Boolean variables
    bool a = true, b = false;
    cout << "a = " << a << ", b = " << b << endl;
    // Reassign a
    a = false;
    cout << "a = " << a << ", b = " << b << endl;
    // Mix integers and Booleans
    a = 1;
    b = 1;
```

Operator	Meaning
==	Equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
!=	Not equal to

Table 5.1: C++ Relational operators

```

cout << "a = " << a << ", b = " << b << endl;
// Assign Boolean value to an integer
int x = a, y = true;
cout << "a = " << a << ", b = " << b
    << ", x = " << x << ", y = " << y << endl;
// More mixing
a = 1725;    // Warning issued
b = -19;    // Warning issued
cout << "a = " << a << ", b = " << b << endl;
}

```

As you can see from running Listing 5.1 (`boolvars.cpp`), the Boolean values `false` and `true` are represented as integer 0 and integer 1. More precisely, zero represents the `bool` value `false`, and any non-zero integer (positive or negative) means `true`. The direct assignment to a `bool` variable of an integer other than 0 or 1 may result in a warning (Visual C++ reports truncation of 'int' to 'bool'), but the variable is still interpreted as `true`. The data type `bool` is basically a convenience for programmers; any C++ program that uses `bool` variables can be rewritten using integers instead to achieve the same results. While Boolean values and variables are freely compatible and interchangeable with integers, the `bool` type is convenient and should be used when the context involves truth values instead of numbers.

It is important to note that the Visual C++ compiler issues warnings for the last two assignment statements in Listing 5.1 (`boolvars.cpp`). Even though any non-zero value is considered `true`, 1 is the preferred integer equivalent to `true` (as you can see when you attempt to print the literal value `true`). Since the need to assign to a Boolean variable a value other than `true` or `false` or the equivalent 1 or 0 should be extremely rare, the compiler's message alerts the programmer to check to make sure the assignment is not a mistake.

5.2 Boolean Expressions

The simplest Boolean expressions are `false` and `true`, the Boolean literals. A Boolean variable is also a Boolean expression. An expression comparing numeric expressions for equality or inequality is also a Boolean expression. The simplest kinds of Boolean expressions use *relational operators* to compare two expressions. Table 5.1 lists the relational operators available in C++.

Table 5.2 shows some simple Boolean expressions with their associated values. An expression like `10 < 20` is legal but of little use, since the expression `true` is equivalent, simpler, and less likely to confuse human readers. Boolean expressions are extremely useful when their truth values depend on the values of one or more variables.

The relational operators are binary operators and are all left associative. They all have a lower prece-

Expression	Value
<code>10 < 20</code>	always true
<code>10 >= 20</code>	always false
<code>x == 10</code>	true only if <code>x</code> has the value 10
<code>x != y</code>	true unless <code>x</code> and <code>y</code> have the same values

Table 5.2: Relational operator examples

dence than any of the arithmetic operators; therefore, the expression

$$x + 2 < y / 10$$

is evaluated as if parentheses were placed as so:

$$(x + 2) < (y / 10)$$

C++ allows statements to be simple expressions; for example, the statement

```
x == 15;
```

may look like an attempt to assign the value 15 to the variable `x`, but it is not. The `=` operator performs assignment, but the `==` operator checks for relational equality. If you make a mistake and use `==` as shown here, Visual C++ will issue a warning that includes the message

warning C4553: '==' : operator has no effect; did you intend '='?

Recall from Section 4.6.4 that a compiler warning does not indicate a violation of the rules of the language; rather it alerts the programmer to a possible trouble spot in the code.

Another example of an expression used as a statement is

```
x + 15;
```



This statement is a legal (but useless) C++ statement, and the compiler notifies us accordingly:

warning C4552: '+' : operator has no effect; expected operator with side-effect

Why are expressions allowed as statements? Some simple expressions have side effects that do alter the behavior of the program. One example of such an expression is `x++`. Listing 4.13 (prevspost.cpp) showed how `x++` behaves both as a standalone statement and as an expression within a larger statement. A more common example is the use of a function call (which is an expression) as standalone a statement. (We introduce functions in Chapter 8.) In order to keep the structure of the language as uniform as possible, C++ tolerates useless expressions as statements to enable programmers to use the more useful expression-statements. Fortunately, most compilers issue informative warnings about the useless expression-statements to keep developers on track.

5.3 The Simple if Statement

The Boolean expressions described in Section 5.2 at first may seem arcane and of little use in practical programs. In reality, Boolean expressions are essential for a program to be able to adapt its behavior at run time. Most truly useful and practical programs would be impossible without the availability of Boolean expressions.

The run-time exceptions mentioned in Section 4.6 arise from logic errors. One way that Listing 4.7 (`dividedanger.cpp`) can fail is when the user enters a zero for the divisor. Fortunately, programmers can take steps to ensure that division by zero does not occur. Listing 5.2 (`betterdivision.cpp`) shows how it might be done.

Listing 5.2: `betterdivision.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int dividend, divisor;

    // Get two integers from the user
    cout << "Please enter two integers to divide:";
    cin >> dividend >> divisor;
    // If possible, divide them and report the result
    if (divisor != 0)
        cout << dividend << "/" << divisor << " = "
            << dividend/divisor << endl;
}
```

The second `cout` statement may not always be executed. In the following run

```
Please enter two integers to divide: 32 8
32/8 = 4
```

it is executed, but if the user enters a zero as the second number:

```
Please enter two integers to divide: 32 0
```

the program prints nothing after the user enters the values.

The last statement in Listing 5.2 (`betterdivision.cpp`) begins with the reserved word `if`. The `if` statement allows code to be optionally executed. In this case, the printing statement is executed only if the variable `divisor`'s value is not zero.

The Boolean expression

```
divisor != 0
```

determines if the single statement that follows the right parenthesis is executed. If **divisor** is not zero, the message is printed; otherwise, the program prints nothing.

Figure 5.1 shows how program execution flows through the **if** statement. of Listing 5.2 (betterdivision.cpp).

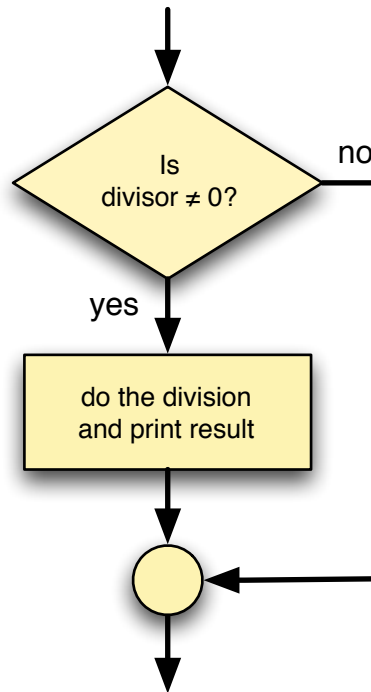


Figure 5.1: if flowchart

The general form of a simple **if** statement is

```
if ( condition )  
    statement
```

- The reserved word **if** begins the **if** statement.
- The Boolean expression *condition* determines whether or not the body will be executed. The Boolean expression *must* be enclosed within parentheses as shown.
- The *statement* is the statement to be executed if the Boolean expression is true. The statement makes up the *body* of the **if** statement. Section 5.4 shows how the body can be composed of multiple statements.

Good coding style dictates the body should be indented to emphasize the optional execution and improve the program's readability. The indenting is not required by the compiler. Sometimes programmers will place a short body on the same time as the `if`; for example, the following `if` statement optionally assigns `y`:

```
if (x < 10)
    y = x;
```

and could be written as

```
if (x < 10) y = x;
```

but should *not* be written as

```
if (x < 10)
y = x;
```

because the lack of indentation hides the fact that the program optionally executes the assignment statement. The compiler will accept it, but it is misleading to human readers accustomed to the indentation convention. The compiler, of course, will accept the code written as

```
if(x<10)y=x;
```

but the lack of spaces makes it difficult for humans to read.

When the `if` statement is written the preferred way using two lines of source code, it is important **not** to put a semicolon at the end of the first line:

```
if (x < 10); // No! Don't do this!
    y = x;
```

Here, the semicolon terminates the `if` statement, but the indentation implies that the second line is intended to be the body of the `if` statement. The compiler, however, interprets the badly formatted `if` statement as if it were written as

```
if (x < 10)
; // This is what is really going on.
y = x;
```

This is legal in C++; it means the `if` statement has an *empty* body. In which case the assignment is not part of the body. The assignment statement is after the body and always will be executed regardless of the truth value of the Boolean expression.



When checking for equality, as in

```
if (x == 10)
    cout << "ten";
```



be sure to use the relational equality operator (`==`), not the assignment operator (`=`). Since an assignment statement has a value (the value that is assigned, see Section 4.3), C++ allows `=` within the conditional expression. It is, however, almost always a mistake when beginning programmers use `=` in this context. Visual C++ at warning Level 4 checks for the use of assignment within a conditional expression; the default Level 3 does not.

5.4 Compound Statements

Sometimes you need to optionally execute more than one statement based on a particular condition. Listing 5.3 (`alternatedivision.cpp`) shows how you must use curly braces to group multiple statements together into one *compound statement*.

Listing 5.3: `alternatedivision.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int dividend, divisor, quotient;

    // Get two integers from the user
    cout << "Please enter two integers to divide:";
    cin >> dividend >> divisor;
    // If possible, divide them and report the result
    if (divisor != 0) {
        quotient = dividend / divisor;
        cout << dividend << " divided by " << divisor << " is "
             << quotient << endl;
    }
}
```

The assignment statement and printing statement are both a part of the body of the `if` statement. Given the truth value of the Boolean expression `divisor != 0` during a particular program run, either both statements will be executed or neither statement will be executed.

A compound statement consists of zero or more statements grouped within curly braces. We say the curly braces define a *block* of statements. As a matter of style many programmers always use curly braces to delimit the body of an `if` statement even if the body contains only one statement:

```
if (x < 10) {
    y = x;
}
```


They do this because it is easy to introduce a logic error if additional statements are added to the body later and the programmer forgets to add then required curly braces.

The format of the following code

```
if (x < 10)
    y = x;
    z = x + 5;
```

implies that both assignments are part of the body of the `if` statement. Since multiple statements making up the body must be in a compound statement within curly braces, the compiler interprets the code fragment as if it had been written

```
if (x < 10)
    y = x;
    z = x + 5;
```



Such code will optionally execute the first assignment statement and *always* execute the second assignment statement.

The programmer probably meant to write it as

```
if (x < 10) {
    y = x;
    z = x + 5;
}
```

The curly braces are optional if the body consists of a single statement. If the body consists of only one statement and curly braces are not used, then the semicolon that terminates the statement in the body also terminates the `if` statement. If curly braces are used to delimit the body, a semicolon is not required after the body's close curly brace.

An empty pair of curly braces represents an empty block. An empty block is a valid compound statement.

5.5 The if/else Statement

One undesirable aspect of Listing 5.2 (`betterdivision.cpp`) is if the user enters a zero divisor, the program prints nothing. It may be better to provide some feedback to the user to indicate that the divisor provided cannot be used. The `if` statement has an optional `else` clause that is executed only if the Boolean expression is false. Listing 5.4 (`betterfeedback.cpp`) uses the `if/else` statement to provide the desired effect.

Listing 5.4: `betterfeedback.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int dividend, divisor;
```

```
// Get two integers from the user
cout << "Please enter two integers to divide:";
cin >> dividend >> divisor;
// If possible, divide them and report the result
if (divisor != 0)
    cout << dividend << "/" << divisor << " = "
        << dividend/divisor << endl;
else
    cout << "Division by zero is not allowed" << endl;
}
```

A given program run will execute exactly one of either the `if` body or the `else` body. Unlike in Listing 5.2 (`betterdivision.cpp`), a message is always displayed.

```
Please enter two integers to divide: 32 0
Division by zero is not allowed
```

The `else` clause contains an alternate body that is executed if the condition is false. The program's flow of execution is shown in Figure 5.2.

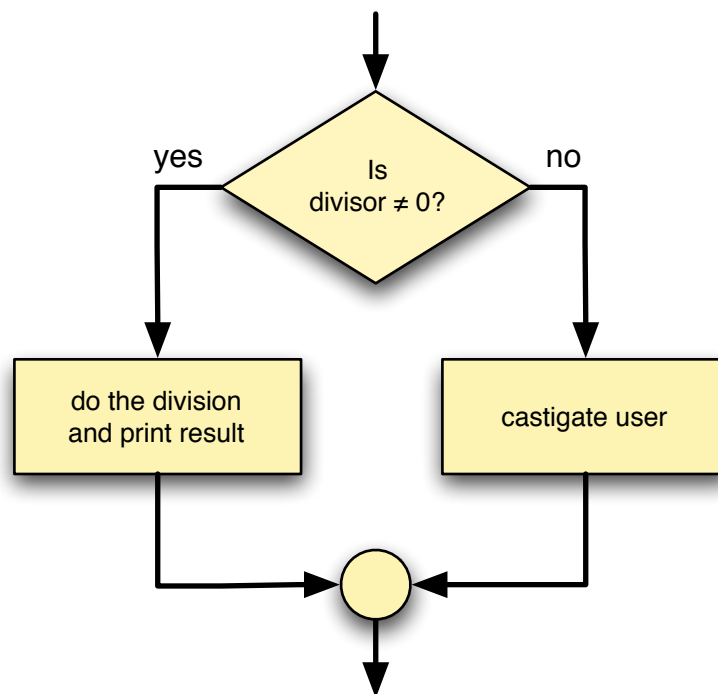


Figure 5.2: if/else flowchart

Listing 5.4 (`betterfeedback.cpp`) avoids the division by zero run-time error that causes the program to terminate prematurely, but it still alerts the user that there is a problem. Another application may handle the situation in a different way; for example, it may substitute some default value for **divisor** instead of zero.

The general form of an **if/else** statement is

```
if ( condition )
    statement 1
else
    statement 2
```

- The reserved word **if** begins the **if/else** statement.
- The *condition* is a Boolean expression that determines whether the running program will execute *statement 1* or *statement 2*. As with the simple **if** statement, the condition must appear within parentheses.
- The program executes *statement 1* if the condition is true. To make the **if/else** statement more readable, indent *statement 1* more spaces than the **if** line. This part of the **if** statement is sometimes called the body of the **if**.
- The reserved word **else** begins the second part of the **if/else** statement.
- The program executes *statement 2* if the condition is false. To make the **if/else** statement more readable, indent *statement 2* more spaces than the **else** line. This part of the **if/else** statement is sometimes called the body of the **else**.

The body of the **else** clause of an **if/else** statement may be a compound statement:

```
if (x == y)
    cout << x;
else {
    x = 0;
    cout << y;
}
```

or the **if** body alone may be a compound statement:

```
if (x == y) {
    cout << x;
    x = 0;
}
else
    cout << y;
```

or both parts may be compound:

```
if (x == y) {
    cout << x;
    x = 0;
}
else {
    cout << y;
    y = 0;
}
```

or, as in Listing 5.4 (betterfeedback.cpp), both the `if` body and the `else` body can be simple statements.



Remember, if you wish to associate more than one statement with the body of the `if` or `else`, you must use a compound statement. Compound statements are enclosed within curly braces (`{ }`).

If you ever attempt to use an `if/else` statement and discover that you need to leave the `else` clause empty, as in

```
if (x == 2)
    cout << "x = " << x << endl;
else
    ;    // Nothing to do otherwise
```

or, using a slightly different syntax, as

```
if (x == 2)
    cout << "x = " << x << endl;
else {
}    // Nothing to do otherwise
```

you instead should use a simple `if` statement:

```
if (x == 2)
    cout << "x = " << x << endl;
```

The empty `else` clauses shown above do work, but they complicate the code and make it more difficult for humans to read.

Due to the imprecise representation of floating-point numbers (see Listing 4.3 (imprecise10.cpp) in Section 4.1), programmers must use caution when using the equality operator (`==`) by itself to compare floating-point expressions. Listing 5.5 (samedifferent.cpp) uses an `if/else` statement to demonstrate the perils of using the equality operator with floating-point quantities.

Listing 5.5: samedifferent.cpp

```
#include <iostream>
#include <iomanip>

using namespace std;
```

```
int main() {
    double d1 = 1.11 - 1.10,
           d2 = 2.11 - 2.10;
    cout << "d1 = " << d1 << endl;
    cout << "d2 = " << d2 << endl;
    if (d1 == d2)
        cout << "Same" << endl;
    else
        cout << "Different" << endl;
    cout << "d1 = " << setprecision(20) << d1 << endl;
    cout << "d2 = " << setprecision(20) << d2 << endl;
}
```

In Listing 5.5 (samedifferent.cpp) the displayed values of **d1** and **d2** are rounded so they appear equivalent, but internally the exact representations are slightly different. By including the header **iomanip** we can use the **setprecision** stream manipulator to force **cout** to display more decimal places in the floating-point number it prints. Observe from the output of Listing 5.5 (samedifferent.cpp) that the two quantities that should be identically 0.01 are actually slightly different.

```
d1 = 0.01
d2 = 0.01
Different
d1 = 0.010000000000000009
d2 = 0.0099999999999997868
```

This result should not discourage you from using floating-point numbers where they truly are needed. In Section 9.4.6 we will see how to handle floating-point comparisons properly.

5.6 Compound Boolean Expressions

Simple Boolean expressions, each involving one relational operator, can be combined into more complex Boolean expressions using the logical operators **&&** (and), **||** (or), and **!** (not). A combination of two or more Boolean expressions using logical operators is called a *compound Boolean expression*.

To introduce compound Boolean expressions, consider a computer science degree that requires, among other computing courses, *Operating Systems* and *Programming Languages*. If we isolate those two courses, we can say a student must successfully complete both *Operating Systems* and *Programming Languages* to qualify for the degree. A student that passes *Operating Systems* but not *Programming Languages* will not have met the requirements. Similarly, *Programming Languages* without *Operating Systems* is insufficient, and a student completing neither *Operating Systems* nor *Programming Languages* surely does not qualify.

Logical **AND** works in exactly the same way. If e_1 and e_2 are two Boolean expressions, $e_1 \ \&\& \ e_2$ is true only if e_1 and e_2 are both true; if either one is false or both are false, the compound expression is false.

To illustrate logical **OR**, consider two mathematics courses, *Differential Equations* and *Linear Algebra*. A computer science degree requires one of those two courses. A student who successfully completes *Differential Equations* but does not take *Linear Algebra* meets the requirement. Similarly, a student may take *Linear Algebra* but not *Differential Equations*. It is important to note the a student may elect to take

e_1	e_2	$e_1 \ \&\& \ e_2$	$e_1 \ \ e_2$	$!e_1$
false	false	false	false	true
false	true	false	true	true
true	false	false	true	false
true	true	true	true	false

Table 5.3: Logical operators— e_1 and e_2 are Boolean expressions

both *Differential Equations* and *Linear Algebra* (perhaps on the way to a mathematics minor), but the requirement is no less fulfilled.

Logical **OR** works in a similar fashion. Given our Boolean expressions e_1 and e_2 , the compound expression $e_1 \ || \ e_2$ is false only if e_1 and e_2 are both false; if either one is true or both are true, the compound expression is true. Note that logical **OR** is an *inclusive or*, not an *exclusive or*. In informal conversation we often imply *exclusive or* in a statement like “Would you like cake **or** ice cream for dessert?” The implication is one or the other, not both. In computer programming the *or* is inclusive; if both subexpressions in an *or* expression are true, the *or* expression is true.

Logical **NOT** simply reverses the truth value of the expression to which it is applied. If e is a true Boolean expression, $!e$ is false; if e is false, $!e$ is true.

Table 5.3 is called a *truth table*. It shows all the combinations of truth values for two simple expressions and the values of compound Boolean expressions built from applying the **&&**, **||**, and **!** C++ logical operators.

Both **&&** and **||** are binary operators; that is, they require two operands, both of which must be Boolean expressions. Logical *not* (**!**) is a unary operator (see Section 4.1); it requires a single Boolean operand immediately to its right.

Operator **!** has higher precedence than both **&&** and **||**. **&&** has higher precedence than **||**. **&&** and **||** are left associative; **!** is right associative. **&&** and **||** have lower precedence than any other binary operator except assignment. This means the expression

```
x <= y && x <= z
```

is evaluated

```
(x <= y) && (x <= z)
```

Some programmers prefer to use the parentheses as shown here even though they are not required. The parentheses improve the readability of complex expressions, and the compiled code is no less efficient.

The relational operators such as `<` compare two operands. The result of the comparison is a Boolean value, which is freely convertible to an integer. The misapplication of relational operators can lead to surprising results; consider, for example, the expression

```
1 <= x <= 10
```

This expression is always true, regardless of the value of `x`! If the programmer's intent is to represent the mathematical notion of `x` falling within the range 1...10 inclusive, as in $1 \leq x \leq 10$, the above C++ expression is not equivalent.

The expression

```
1 <= x <= 10
```

is evaluated as



```
(1 <= x) <= 10
```

If `x` is greater than or equal to one, the subexpression `1 <= x` evaluates to true, or integer 1. Integer 1, however, is always less than 10, so the overall expression is true. If instead `x` is less than one, the subexpression `1 <= x` evaluates to false, or integer 0. Integer 0 is always less than 10, so the overall expression is true. The problem is due to the fact that C++ does not strictly distinguish between Boolean and integer values.

A correct way to represent the mathematical notion of $1 \leq x \leq 10$ is

```
1 <= x && x <= 10
```

In this case `x` must simultaneously be greater than or equal to 1 **and** less than or equal to 10. The revised Boolean expression is a little more verbose than the mathematical representation, but it is the correct formulation for C++.

The following section of code assigns the indicated values to a `bool`:

```
bool b;
int x = 10;
int y = 20;
b = (x == 10); // assigns true to b
b = (x != 10); // assigns false to b
b = (x == 10 && y == 20); // assigns true to b
b = (x != 10 && y == 20); // assigns false to b
b = (x == 10 && y != 20); // assigns false to b
b = (x != 10 && y != 20); // assigns false to b
b = (x == 10 || y == 20); // assigns true to b
b = (x != 10 || y == 20); // assigns true to b
b = (x == 10 || y != 20); // assigns true to b
b = (x != 10 || y != 20); // assigns false to b
```

Convince yourself that the following expressions are equivalent:

```
(x != y)
```

```
! (x == y)
(x < y || x > y)
```

In the expression $e_1 \ \&\& \ e_2$ both subexpressions e_1 and e_2 must be true for the overall expression to be true. Since the $\&\&$ operator evaluates left to right, this means that if e_1 is false, there is no need to evaluate e_2 . If e_1 is false, no value of e_2 can make the expression $e_1 \ \&\& \ e_2$ true. The logical *and* operator first tests the expression to its left. If it finds the expression to be false, it does not bother to check the right expression. This approach is called *short-circuit evaluation*. In a similar fashion, in the expression $e_1 \ || \ e_2$, if e_1 is true, then it does not matter what value e_2 has—a logical *or* expression is true unless both subexpressions are false. The $||$ operator uses short-circuit evaluation also.

Why is short-circuit evaluation important? Two situations show why it is important to consider:

- The order of the subexpressions can affect performance. When a program is running, complex expressions require more time for the computer to evaluate than simpler expressions. We classify an expression that takes a relatively long time to evaluate as an *expensive* expression. If a compound Boolean expression is made up of an expensive Boolean subexpression and an less expensive Boolean subexpression, and the order of evaluation of the two expressions does not affect the behavior of the program, then place the more expensive Boolean expression second. If the first subexpression is false and $\&\&$ is being used, then the expensive second subexpression is not evaluated; if the first subexpression is true and $||$ is being used, then, again, the expensive second subexpression is avoided.
- Subexpressions can be ordered to prevent run-time errors. This is especially true when one of the subexpressions depends on the other in some way. Consider the following expression:

```
(x != 0) && (z/x > 1)
```

Here, if x is zero, the division by zero is avoided. If the subexpressions were switched, a run-time error would result if x is zero.

Arity	Operators	Associativity
unary	(post) ++, (post) --, <code>static_cast</code>	
unary	(pre) ++, (pre) --, !, +, -	
binary	*, /, %	left
binary	+, -	left
binary	<<, >>	left
binary	>, <, >=, <=	left
binary	==, !=	left
binary	&&	left
binary		left
binary	=, +=, -=, *=, /=, %=	right

Table 5.4: Precedence of C++ Operators (High to Low)

Suppose you wish to print the word “OK” if a variable `x` is 1, 2, or 3. An informal translation from English might yield:

```
if (x == 1 || 2 || 3)
    cout << "OK" << endl;
```

Unfortunately, `x`’s value is irrelevant; the code always prints the word “OK.” Since the `==` operator has lower precedence than `||`, the expression

```
x == 1 || 2 || 3
```

is interpreted as

```
(x == 1) || 2 || 3
```

The expression `x == 1` is either true or false, but integer 2 is always interpreted as true, and integer 3 is interpreted as true as well.

The correct statement would be

```
if (x == 1 || x == 2 || x == 3)
    cout << "OK" << endl;
```

The revised Boolean expression is more verbose and less similar to the English rendition, but it is the correct formulation for C++.

Our current list of C++ operators is shown in Table 5.4.

5.7 Nested Conditionals

The statements in the body of the `if` or the `else` may be any C++ statements, including other `if/else` statements. We can use nested `if` statements to build arbitrarily complex control flow logic. Consider Listing 5.6 (`checkrange.cpp`) that determines if a number is between 0 and 10, inclusive.

Listing 5.6: `checkrange.cpp`

```
#include <iostream>
```

```
using namespace std;

int main() {
    int value;
    cout << "Please enter an integer value in the range 0...10: ";
    cin >> value;
    if (value >= 0)    // First check
        if (value <= 10) // Second check
            cout << "In range";
    cout << "Done" << endl;
}
```

Listing 5.6 (checkrange.cpp) behaves as follows:

- The program checks the **value** **>= 0** condition first. If **value** is less than zero, the executing program does not evaluate the second condition and does not print *In range*, but it immediately executes the print statement following the outer **if** statement which prints *Done*.
- If the executing program finds **value** to be greater than or equal to zero, it checks the second condition. If the second condition is met, it displays the *In range* message; otherwise, it is not. Regardless, the program prints *Done* before it terminates.

For the program to display the message *In range* both conditions of this nested **if** must be met. Said another way, the first condition *and* the second condition must be met for the *In range* message to be printed. From this perspective, we can rewrite the program to behave the same way with only *one* **if** statement, as Listing 5.7 (newcheckrange.cpp) shows.

Listing 5.7: newcheckrange.cpp

```
#include <iostream>

using namespace std;

int main() {
    int value;
    cout << "Please enter an integer value in the range 0...10: ";
    cin >> value;
    if (value >= 0 && value <= 10)
        cout << "In range";
    cout << endl;
}
```

Listing 5.7 (newcheckrange.cpp) uses a logical **&&** to check both conditions at the same time. Its logic is simpler, using only one **if** statement, at the expense of a slightly more complex Boolean expression in its condition. The second version is preferable here because simpler logic is usually a desirable goal.

Sometimes a program's logic cannot be simplified as in Listing 5.7 (newcheckrange.cpp). In Listing 5.8 (enhancedcheckrange.cpp) one **if** statement alone is insufficient to implement the necessary behavior.

Listing 5.8: enhancedcheckrange.cpp

```
#include <iostream>

using namespace std;
```

```
int main() {
    int value;
    cout << "Please enter an integer value in the range 0...10: ";
    cin >> value;
    if (value >= 0)    // First check
        if (value <= 10)    // Second check
            cout << value << " is acceptable";
        else
            cout << value << " is too large";
    else
        cout << value << " is too small";
    cout << endl;
}
```

Listing 5.8 (`enhancedcheckrange.cpp`) provides a more specific message instead of a simple notification of acceptance. The program prints exactly one of three messages based on the value of the variable. A single `if` or `if/else` statement cannot choose from among more than two different execution paths.

Listing 5.9 (`binaryconversion.cpp`) uses a series of `if` statements to print a 10-bit binary string representing the binary equivalent of a decimal integer supplied by the user. (Chapter D provides some background information about the binary number system.) We use `if/else` statements to print the individual digits left to right, essentially assembling the sequence of bits that represents the binary number.

Listing 5.9: `binaryconversion.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int value;
    // Get number from the user
    cout << "Please enter an integer value in the range 0...1023: ";
    cin >> value;
    // Integer must be less than 1024
    if (0 <= value && value < 1024) {
        if (value >= 512) {
            cout << 1;
            value %= 512;
        }
        else
            cout << 0;
        if (value >= 256) {
            cout << 1;
            value %= 256;
        }
        else
            cout << 0;
        if (value >= 128) {
            cout << 1;
            value %= 128;
        }
        else
            cout << 0;
    }
```

```
    if (value >= 64) {
        cout << 1;
        value %= 64;
    }
    else
        cout << 0;
    if (value >= 32) {
        cout << 1;
        value %= 32;
    } else
        cout << 0;
    if (value >= 16) {
        cout << 1;
        value %= 16;
    }
    else
        cout << 0;
    if (value >= 8) {
        cout << 1;
        value %= 8;
    }
    else
        cout << 0;
    if (value >= 4) {
        cout << 1;
        value %= 4;
    }
    else
        cout << 0;
    if (value >= 2) {
        cout << 1;
        value %= 2;
    }
    else
        cout << 0;
    cout << value << endl;
}
}
```

In Listing 5.9 (binaryconversion.cpp):

- The outer **if** checks to see if the value the user provides is in the proper range. The program works only for non-negative integer values less than 1,024, so the range is 0-1023.
- Each inner **if** compares the user-supplied entered integer against decreasing powers of two. If the number is large enough, the program:
 - prints the digit 1 to the console, and
 - removes via the remainder operator that power of two's contribution to the value.

If the number is not at least as big as the given power of two, the program prints a 0 instead and moves on without modifying the input value.

- For the ones place at the end no check is necessary—the remaining value will be 0 or 1 and so the program prints whatever remains.

The following shows a sample run of Listing 5.9 (binaryconversion.cpp):

```
Please enter an integer value in the range 0...1023: 805
1100100101
```

Figure 5.3 illustrates the execution of Listing 5.9 (binaryconversion.cpp) when the user enters 805.

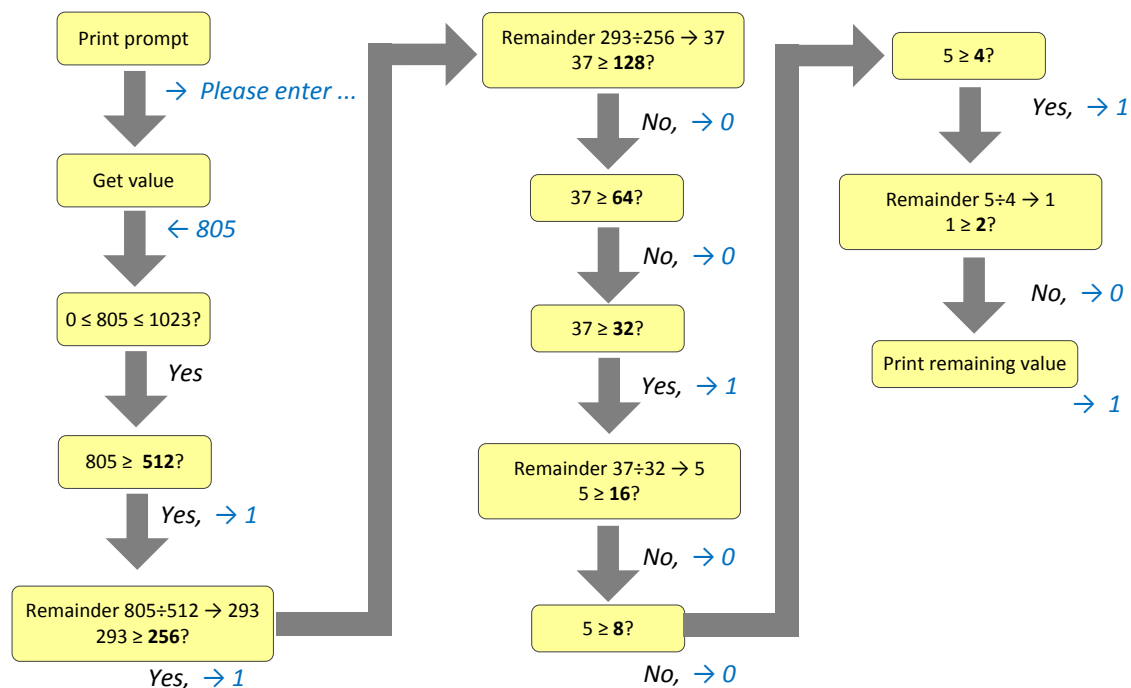


Figure 5.3: The process of the binary number conversion program when the user supplies 805 as the input value.

Listing 5.10 (simplerbinaryconversion.cpp) simplifies the logic of Listing 5.9 (binaryconversion.cpp) at the expense of some additional arithmetic. It uses only one **if** statement.

Listing 5.10: simplerbinaryconversion.cpp

```
#include <iostream>

using namespace std;

int main() {
    int value;
    // Get number from the user
    cout << "Please enter an integer value in the range 0...1023: ";
    cin >> value;
    // Integer must be less than 1024
```

```

    if (0 <= value && value < 1024) {
        cout << value/512;
        value %= 512;
        cout << value/256;
        value %= 256;
        cout << value/128;
        value %= 128;
        cout << value/64;
        value %= 64;
        cout << value/32;
        value %= 32;
        cout << value/16;
        value %= 16;
        cout << value/8;
        value %= 8;
        cout << value/4;
        value %= 4;
        cout << value/2;
        value %= 2;
        cout << value << endl;
    }
}

```

The sole `if` statement in Listing 5.10 (`simplerbinaryconversion.cpp`) ensures that the user provides an integer in the proper range. The other `if` statements that originally appeared in Listing 5.9 (`binaryconversion.cpp`) are gone. A clever sequence of integer arithmetic operations replace the original conditional logic. The two programs—`binaryconversion.cpp` and `simplerbinaryconversion.cpp`—behave identically but `simplerbinaryconversion.cpp`'s logic is simpler.

Listing 5.11 (`troubleshoot.cpp`) implements a very simple troubleshooting program that (an equally simple) computer technician might use to diagnose an ailing computer.

Listing 5.11: `troubleshoot.cpp`

```

#include <iostream>

using namespace std;

int main() {
    cout << "Help! My computer doesn't work!" << endl;
    char choice;
    cout << "Does the computer make any sounds "
        << "(fans, etc.) or show any lights? (y/n):";
    cin >> choice;
    // The troubleshooting control logic
    if (choice == 'n') { // The computer does not have power
        cout << "Is it plugged in? (y/n):";
        cin >> choice;
        if (choice == 'n') { // It is not plugged in, plug it in
            cout << "Plug it in. If the problem persists, "
                << "please run this program again." << endl;
        }
        else { // It is plugged in
            cout << "Is the switch in the \"on\" position? (y/n):";
            cin >> choice;
        }
    }
}

```

```

    if (choice == 'n') { // The switch is off, turn it on!
        cout << "Turn it on. If the problem persists, "
              << "please run this program again." << endl;
    }
    else { // The switch is on
        cout << "Does the computer have a fuse? (y/n):";
        cin >> choice;
        if (choice == 'n') { // No fuse
            cout << "Is the outlet OK? (y/n):";
            if (choice == 'n') { // Fix outlet
                cout << "Check the outlet's circuit "
                      << "breaker or fuse. Move to a "
                      << "new outlet, if necessary. "
                      << "If the problem persists, "
                      << "please run this program "
                      << "again." << endl;
            }
            else { // Beats me!
                cout << "Please consult a service "
                      << "technician." << endl;
            }
        }
        else { // Check fuse
            cout << "Check the fuse. Replace if "
                  << "necessary. If the problem "
                  << "persists, then "
                  << "please run this program again."
                  << endl;
        }
    }
}
}
}
else { // The computer has power
    cout << "Please consult a service technician." << endl;
}
}

```

This very simple troubleshooting program attempts to diagnose why a computer does not work. The potential for enhancement is unlimited, but this version only deals with power issues that have simple fixes. Notice that if the computer has power (fan or disk drive makes sounds or lights are visible), the program directs the user to seek help elsewhere! The decision tree capturing the basic logic of the program is shown in Figure 5.4. The steps performed are:

1. Is it plugged in? This simple fix is sometimes overlooked.
2. Is the switch in the *on* position? This is another simple fix.
3. If applicable, is the fuse blown? Some computer systems have a user-serviceable fuse that can blow out during a power surge. (Most newer computers have power supplies that can handle power surges and have no user-serviceable fuses.)
4. Is there power at the receptacle? Perhaps the outlet's circuit breaker or fuse has a problem.

The program directs the user to make the easier checks first. It progressively introduces more difficult checks as it continues. Based on your experience with troubleshooting computers that do not run properly,

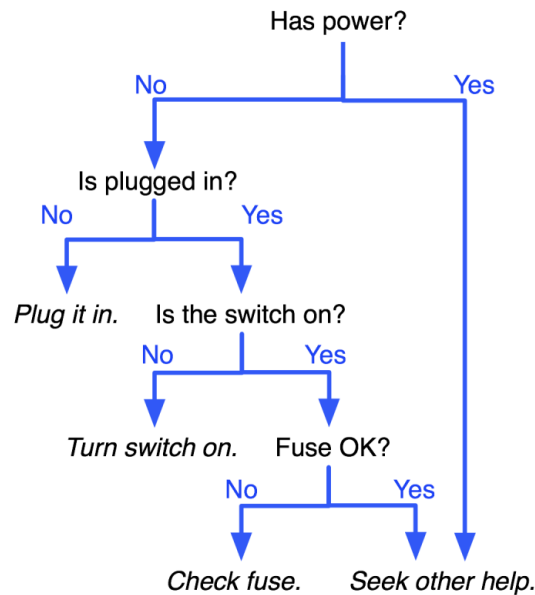


Figure 5.4: Decision tree for troubleshooting a computer system

you may be able to think of many enhancements to this simple program.

Note that in Listing 5.11 (`troubleshoot.cpp`) curly braces are used in many places where they strictly are not necessary. Their inclusion in Listing 5.11 (`troubleshoot.cpp`) improves the readability of the program and makes the logic easier to understand. Even if you do not subscribe to the philosophy of using curly braces for every `if/else` body, it is a good idea to use them in situations that improve the code's readability.

Recall the time conversion program in Listing 4.11 (`timeconv.cpp`). If the user enters **10000**, the program runs as follows:

```
Please enter the number of seconds:10000
2 hours 46 minutes 40 seconds
```

and if the user enters **9961**, the program prints:

```
Please enter the number of seconds:9961
2 hours 46 minutes 1 second
```

Suppose we wish to improve the English presentation by not using abbreviations. If we spell out *hours*, *minutes*, and *seconds*, we must be careful to use the singular form *hour*, *minute*, or *second* when the

corresponding value is one. Listing 5.12 (timeconvcond1.cpp) uses **if/else** statements to express to time units with the correct number.

Listing 5.12: timeconvcond1.cpp

```
// File timeconvcond1.cpp

#include <iostream>

using namespace std;

int main() {
    // Some useful conversion constants
    const int SECONDS_PER_MINUTE = 60,
              SECONDS_PER_HOUR   = 60*SECONDS_PER_MINUTE; // 3600
    int hours, minutes, seconds;
    cout << "Please enter the number of seconds:";
    cin >> seconds;
    // First, compute the number of hours in the given number
    // of seconds
    hours = seconds / SECONDS_PER_HOUR; // 3600 seconds = 1 hours
    // Compute the remaining seconds after the hours are
    // accounted for
    seconds = seconds % SECONDS_PER_HOUR;
    // Next, compute the number of minutes in the remaining
    // number of seconds
    minutes = seconds / SECONDS_PER_MINUTE; // 60 seconds = 1 minute
    // Compute the remaining seconds after the minutes are
    // accounted for
    seconds = seconds % SECONDS_PER_MINUTE;
    // Report the results
    cout << hours;
    // Decide between singular and plural form of hours
    if (hours == 1)
        cout << " hour ";
    else
        cout << " hours ";
    cout << minutes;
    // Decide between singular and plural form of minutes
    if (minutes == 1)
        cout << " minute ";
    else
        cout << " minutes ";
    cout << seconds;
    // Decide between singular and plural form of seconds
    if (seconds == 1)
        cout << " second";
    else
        cout << " seconds";
    cout << endl;
}
```

The **if/else** statements within Listing 5.12 (timeconvcond1.cpp) are responsible for printing the correct version—singular or plural—for each time unit. One run of Listing 5.12 (timeconvcond1.cpp) produces

```
Please enter the number of seconds:10000
2 hours 46 minutes 40 seconds
```

All the words are plural since all the value are greater than one. Another run produces

```
Please enter the number of seconds:9961
2 hours 46 minutes 1 second
```

Note the word *second* is singular as it should be.

```
Please enter the number of seconds:3601
1 hour 0 minutes 1 second
```

Here again the printed words agree with the number of the value they represent.

An improvement to Listing 5.12 (`timeconvcond1.cpp`) would not print a value and its associated time unit if the value is zero. Listing 5.13 (`timeconvcond2.cpp`) adds this feature.

Listing 5.13: `timeconvcond2.cpp`

```
// File timeconvcond1.cpp

#include <iostream>

using namespace std;

int main() {
    // Some useful conversion constants
    const int SECONDS_PER_MINUTE = 60,
              SECONDS_PER_HOUR   = 60*SECONDS_PER_MINUTE;    // 3600
    int hours, minutes, seconds;
    cout << "Please enter the number of seconds:";
    cin >> seconds;
    // First, compute the number of hours in the given number
    // of seconds
    hours = seconds / SECONDS_PER_HOUR;    // 3600 seconds = 1 hours
    // Compute the remaining seconds after the hours are
    // accounted for
    seconds = seconds % SECONDS_PER_HOUR;
    // Next, compute the number of minutes in the remaining
    // number of seconds
    minutes = seconds / SECONDS_PER_MINUTE;    // 60 seconds = 1 minute
    // Compute the remaining seconds after the minutes are
    // accounted for
    seconds = seconds % SECONDS_PER_MINUTE;
    // Report the results
    if (hours > 0) {    // Print hours at all?
```

```
    cout << hours;
    // Decide between singular and plural form of hours
    if (hours == 1)
        cout << " hour ";
    else
        cout << " hours ";
}
if (minutes > 0) {    // Print minutes at all?
    cout << minutes;
    // Decide between singular and plural form of minutes
    if (minutes == 1)
        cout << " minute ";
    else
        cout << " minutes ";
}
// Print seconds at all?
if (seconds > 0 || (hours == 0 && minutes == 0 && seconds == 0)) {
    cout << seconds;
    // Decide between singular and plural form of seconds
    if (seconds == 1)
        cout << " second";
    else
        cout << " seconds";
}
cout << endl;
}
```

In Listing 5.13 (timeconvcond2.cpp) each code segment responsible for printing a time value and its English word unit is protected by an **if** statement that only allows the code to execute if the time value is greater than zero. The exception is in the processing of seconds: if all time values are zero, the program should print *0 seconds*. Note that each of the **if/else** statements responsible for determining the singular or plural form is nested within the **if** statement that determines whether or not the value will be printed at all.

One run of Listing 5.13 (timeconvcond2.cpp) produces

```
Please enter the number of seconds:10000
2 hours 46 minutes 40 seconds
```

All the words are plural since all the value are greater than one. Another run produces

```
Please enter the number of seconds:9961
2 hours 46 minutes 1 second
```

Note the word *second* is singular as it should be.

```
Please enter the number of seconds:3601
1 hour 1 second
```

Here again the printed words agree with the number of the value they represent.

```
Please enter the number of seconds:7200
2 hours
```

Another run produces:

```
Please enter the number of seconds:60
1 minute
```

Finally, the following run shows that the program handles zero seconds properly:

```
Please enter the number of seconds:0
0 seconds
```

5.8 Multi-way if/else Statements

A simple **if/else** statement can select from between two execution paths. Suppose we wish to choose one execution path from among several possible paths, as shown in Figure 5.5? Listing 5.8 (enhancedcheckrange.cpp)

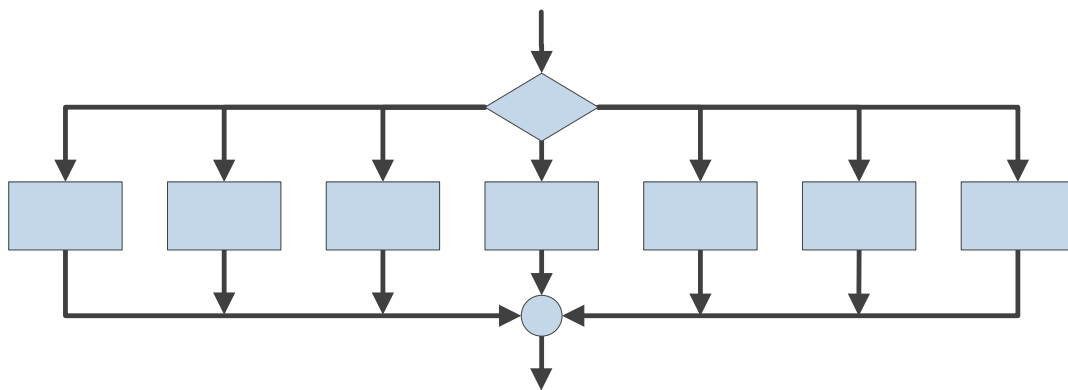


Figure 5.5: Flowchart with multiple optional execution pathways

showed how to select from among three options. What if exactly one of many actions should be taken?

Nested **if/else** statements are required, and the form of these nested **if/else** statements is shown in Listing 5.14 (digittoword.cpp).

Listing 5.14: digittoword.cpp

```
#include <iostream>

using namespace std;

int main() {
    int value;
    cout << "Please enter an integer in the range 0...5: ";
    cin >> value;
    if (value < 0)
        cout << "Too small";
    else
        if (value == 0)
            cout << "zero";
        else
            if (value == 1)
                cout << "one";
            else
                if (value == 2)
                    cout << "two";
                else
                    if (value == 3)
                        cout << "three";
                    else
                        if (value == 4)
                            cout << "four";
                        else
                            if (value == 5)
                                cout << "five";
                            else
                                cout << "Too large";

    cout << endl;
}
```

Observe the following about Listing 5.14 (digittoword.cpp):

- It prints exactly one of eight messages depending on the user's input.
- Notice that each **if** body contains a single printing statement and each **else** body, except the last one, contains an **if** statement. The control logic forces the program execution to check each condition in turn. The first condition that matches wins, and its corresponding **if** body will be executed. If none of the conditions are true, the last **else**'s *Too large* message will be printed.
- No curly braces are necessary to delimit the **if** or **else** bodies since each body contains only a single statement (although a single deeply nested **if/else** statement is a mighty big statement).

Listing 5.14 (digittoword.cpp) is formatted according to the conventions used in earlier examples. As a consequence, the mass of text drifts to the right as more conditions are checked. A commonly used alternative style, shown in Listing 5.15 (restyleddigittoword.cpp), avoids this rightward drift.

Listing 5.15: restyleddigittoword.cpp

```
#include <iostream>

using namespace std;

int main() {
    int value;
    cout << "Please enter an integer in the range 0...5: ";
    cin >> value;
    if (value < 0)
        cout << "Too small";
    else if (value == 0)
        cout << "zero";
    else if (value == 1)
        cout << "one";
    else if (value == 2)
        cout << "two";
    else if (value == 3)
        cout << "three";
    else if (value == 4)
        cout << "four";
    else if (value == 5)
        cout << "five";
    else
        cout << "Too large";
    cout << endl;
}
```

Based on our experience so far, the formatting of Listing 5.15 (restyleddigittoword.cpp) somewhat hides the true structure of the program's logic, but this style of formatting multi-way **if/else** statements is so common that it is regarded as acceptable by most programmers. The sequence of **else if** lines all indented to the same level identifies this construct as a multi-way **if/else** statement.

Listing 5.16 (datetransformer.cpp) uses a multi-way **if/else** to transform a numeric date in month/-day format to an expanded US English form and an international Spanish form; for example, **2/14** would be converted to **February 14** and **14 febrero**.

Listing 5.16: datetransformer.cpp

```
#include <iostream>

using namespace std;

int main() {
    cout << "Please enter the month and day as numbers: ";
    int month, day;
    cin >> month >> day;
    // Translate month into English
    if (month == 1)
        cout << "January";
    else if (month == 2)
        cout << "February";
    else if (month == 3)
        cout << "March";
    else if (month == 4)
```

```
    cout << "April";
else if (month == 5)
    cout << "May";
else if (month == 6)
    cout << "June";
else if (month == 7)
    cout << "July";
else if (month == 8)
    cout << "August";
else if (month == 9)
    cout << "September";
else if (month == 10)
    cout << "October";
else if (month == 11)
    cout << "November";
else
    cout << "December";
// Add the day
cout << " " << day << " or " << day << " de ";
// Translate month into Spanish
if (month == 1)
    cout << "enero";
else if (month == 2)
    cout << "febrero";
else if (month == 3)
    cout << "marzo";
else if (month == 4)
    cout << "abril";
else if (month == 5)
    cout << "mayo";
else if (month == 6)
    cout << "junio";
else if (month == 7)
    cout << "julio";
else if (month == 8)
    cout << "agosto";
else if (month == 9)
    cout << "septiembre";
else if (month == 10)
    cout << "octubre";
else if (month == 11)
    cout << "noviembre";
else
    cout << "diciembre";
cout << endl;
}
```

A sample run of Listing 5.16 (datetransformer.cpp) is shown here:

```
Please enter the month and day as numbers: 5 20
May 20 or 20 de mayo
```

Figure 5.6 compares the structure of the **if/else** statements in a program such as Listing 5.15 (restyleddigittoword.cpp) to those in a program like Listing 5.9 (binaryconversion.cpp). In a program like

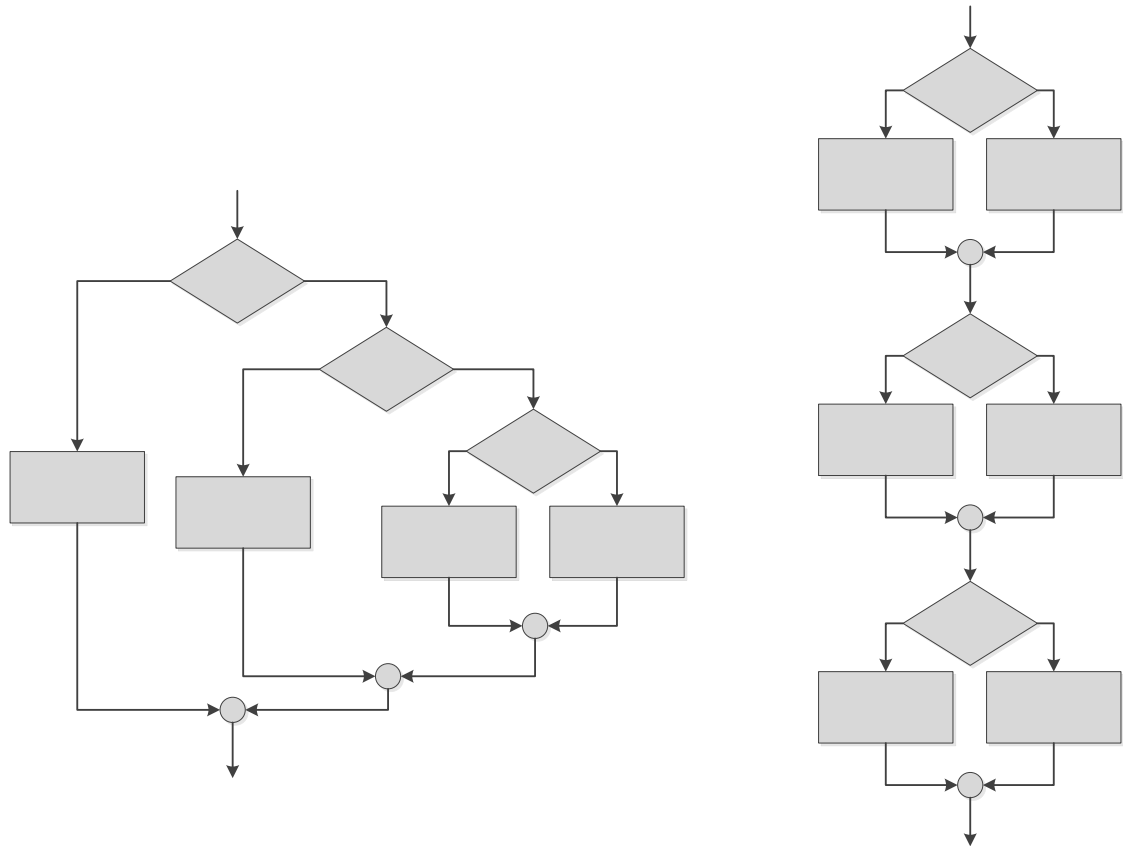


Figure 5.6: The structure of the **if** statements in a program such as Listing 5.15 (restyleddigittoword.cpp) (left) vs. those in a program like Listing 5.9 (binaryconversion.cpp) (right)

Listing 5.15 (restyleddigittoword.cpp), the **if/else** statements are nested, while in a program like Listing 5.9 (binaryconversion.cpp) the **if/else** statements are sequential.

5.9 Errors in Conditional Statements

Consider Listing 5.17 (badequality.cpp).

Listing 5.17: badequality.cpp

```
#include <iostream>

using namespace std;
```



```

int main() {
    int input;
    cout << "Please enter an integer:";
    cin >> input;
    if (input = 2)
        cout << "two" << endl;
    cout << "You entered " << input << endl;
}

```

Listing 5.17 (badequality.cpp) demonstrates a common mistake—using the assignment operator where the equality operator is intended. This program, when run, always prints the message “two” and insists the user entered 2 regardless of the actual input. Recall from Section 4.3 that the assignment expression has a value. The value of an assignment expression is same as the value that is assigned; thus, the expression

```
input = 2
```

has the value 2. When you consider also that every integer can be treated as a Boolean value (see Section 5.1) and any non-zero value is interpreted as **true**, you can see that the condition of **if** statement

```

if (input = 2)
    cout << "two" << endl;

```

is always true. Additionally, the variable **input** is always assigned the value 2.

Since it is such a common coding error, most C++ compilers can check for such misuse of assignment. At warning Level 4, for example, Visual C++ will issue a warning when assignment appears where a conditional expression is expected:

warning C4706: assignment within conditional expression

Occasionally the use of assignment within a conditional expression is warranted, so the compiler does not perform this check by default. For our purposes it is good idea to direct the compiler to perform this extra check.

Carefully consider each compound conditional used, such as

```
value > 0 && value <= 10
```

found in Listing 5.7 (newcheckrange.cpp). Confusing logical *and* and logical *or* is a common programming error. If you substitute **||** for **&&**, the expression

```
x > 0 || x <= 10
```

is always true, no matter what value is assigned to the variable **x**. A Boolean expression that is always true is known as a *tautology*. Think about it. If **x** is an **int**, what value could the variable **x** assume that would make

```
x > 0 || x <= 10
```

false? Regardless of its value, one or both of the subexpressions will be true, so this compound logical *or* expression is always true. This particular *or* expression is just a complicated way of expressing the value true.

Another common error is contriving compound Boolean expressions that are always false, known as *contradictions*. Suppose you wish to *exclude* values from a given range; for example, reject values in the

range 0...10 and accept all other numbers. Is the Boolean expression in the following code fragment up to the task?

```
// I want to use all but 0, 1, 2, ..., 10
if (value < 0 && value > 10)
    /* Code to execute goes here . . . */
```

A closer look at the condition reveals it can *never* be true. What number can be both less than zero and greater than ten *at the same time*? None can, of course, so the expression is a contradiction and a complicated way of expressing *false*. To correct this code fragment, replace the `&&` operator with `||`.

5.10 Summary

- The `bool` data type represents the values true and false.
- The name `bool` comes from Boolean algebra, the mathematical study of operations on truth values.
- In C++ the value `true` is represented by the integer one, and `false` is represented by zero.
- Any integer value except zero is treated as true.
- Integers and `bool`s are interchangeable and can be assigned to each other.
- Expressions involving the relational operators (`==`, `!=`, `<`, `>`, `<=`, and `>=`) evaluate to Boolean values.
- `!` is the unary *not* operator.
- Boolean expressions can be combined via `&&` (logical *AND*) and `||` (logical *OR*).
- The `if` statement can be used to optionally execute statements.
- A compound statement is a sequence of statements within a pair of curly braces.
- The `if` statement has an optional `else` clause to require the selection between two alternate paths of execution.
- The `if/else` statements can be nested to achieve arbitrary complexity.
- The bodies of `if/else` statements should be indented to aid human readers. Indentation does not affect the logic of the program; when multiple statements are to be part of the body of an `if` or `else`, the statements must be part of a compound statement.
- Beware placing a semicolon immediately after the close parenthesis of an `if` statement's condition.
- Complex Boolean expressions require special attention, as they are easy to get wrong.
-
-
-

5.11 Exercises

1. What values can a variable of type `bool` assume?
2. Where does the term `bool` originate?
3. What is the integer equivalent to `true` in C++?
4. What is the integer equivalent to `false` in C++?
5. Is the value `-16` interpreted as true or false?
6. May an integer value be assigned to a `bool` variable?
7. Can `true` be assigned to an `int` variable?
8. Given the following declarations:

```
int x = 3, y = 5, z = 7;  
bool b1 = true, b2 = false, b3 = x == 3, b4 = y < 3;
```

evaluate the following Boolean expressions:

- (a) `x == 3`
 - (b) `x < y`
 - (c) `x >= y`
 - (d) `x <= y`
 - (e) `x != y - 2`
 - (f) `x < 10`
 - (g) `x >= 0 && x < 10`
 - (h) `x < 0 && x < 10`
 - (i) `x >= 0 && x < 2`
 - (j) `x < 0 || x < 10`
 - (k) `x > 0 || x < 10`
 - (l) `x < 0 || x > 10`
 - (m) `b1`
 - (n) `!b1`
 - (o) `!b2`
 - (p) `b1 && b2`
9. Express the following Boolean expressions in simpler form; that is, use fewer operators. `x` is an `int`.
 - (a) `!(x == 2)`
 - (b) `x < 2 || x == 2`
 - (c) `!(x < y)`
 - (d) `!(x <= y)`
 - (e) `x < 10 && x > 20`
 - (f) `x > 10 || x < 20`

(g) `x != 0`

(h) `x == 0`

10. What is the simplest tautology?
11. What is the simplest contradiction?
12. Write a C++ program that requests an integer value from the user. If the value is between 1 and 100 inclusive, print "OK;" otherwise, do not print anything.
13. Write a C++ program that requests an integer value from the user. If the value is between 1 and 100 inclusive, print "OK;" otherwise, print "Out of range."
14. Write a C++ program that requests an integer value from the user. If the value is between 1 and 100 inclusive,
15. The following program attempts to print a message containing the the English word corresponding to a given integer input. For example, if the user enters the value 3, the program should print **"You entered a three"**. In its current state, the program contains logic errors. Locate the problems and repair them so the program will work as expected.

```
#include <iostream>

using namespace std;

int main() {
    cout << "Please in value in the range 1...5: ";
    int value;
    cin >> value;
    // Translate number into its English word
    if (month == 1)
        cout << "You entered a";
        cout << "one";
        cout << endl;
    else if (month == 2)
        cout << "You entered a";
        cout << "two";
        cout << endl;
    else if (month == 3)
        cout << "You entered a";
        cout << "three";
        cout << endl;
    else if (month == 4)
        cout << "You entered a";
        cout << "four";
        cout << endl;
    else if (month == 5)
        cout << "You entered a";
        cout << "five";
        cout << endl;
    else // Value out of range
        cout << "You entered a";
        cout << "value out of range";
}
```

```
        cout << endl;
    }
```

16. Consider the following section of C++ code:

```
// i, j, and k are ints
if (i < j) {
    if (j < k)
        i = j;
    else
        j = k;
}
else {
    if (j > k)
        j = i;
    else
        i = k;
}
cout << "i = " << i << " j = " << j << " k = " << k << endl;
```

What will the code print if the variables *i*, *j*, and *k* have the following values?

- (a) *i* is 3, *j* is 5, and *k* is 7
- (b) *i* is 3, *j* is 7, and *k* is 5
- (c) *i* is 5, *j* is 3, and *k* is 7
- (d) *i* is 5, *j* is 7, and *k* is 3
- (e) *i* is 7, *j* is 3, and *k* is 5
- (f) *i* is 7, *j* is 5, and *k* is 3

17. Consider the following C++ program that prints one line of text:

```
#include <iostream>
using namespace std;

int main() {
    int input;
    cin >> input;
    if (input < 10) {
        if (input != 5)
            cout << "wow ";
        else
            input++;
    }
    else {
        if (input == 17)
            input += 10;
        else
            cout << "whoa ";
    }
    cout << input << endl;
}
```

What will the program print if the user provides the following input?

- (a) 3
- (b) 21
- (c) 5
- (d) 17
- (e) -5

18. Why does the following section of code always print **"ByeHi"**?

```
int x;  
cin >> x;  
if (x < 0);  
    cout << "Bye";  
cout << "Hi" << endl;
```

19. Write a C++ program that requests five integer values from the user. It then prints the maximum and minimum values entered. If the user enters the values 3, 2, 5, 0, and 1, the program would indicate that 5 is the maximum and 0 is the minimum. Your program should handle ties properly; for example, if the user enters 2, 4 2, 3 and 3, the program should report 2 as the minimum and 4 as maximum.
20. Write a C++ program that requests five integer values from the user. It then prints one of two things: if any of the values entered are duplicates, it prints **"DUPLICATES"**; otherwise, it prints **"ALL UNIQUE"**.
21. Write a C++ program that ...

Chapter 6

Iteration

Iteration repeats the execution of a sequence of code. Iteration is useful for solving many programming problems. Iteration and conditional execution are key components of algorithm construction.

6.1 The while Statement

Listing 6.1 (counttofive.cpp) counts to five by printing a number on each output line.

Listing 6.1: counttofive.cpp

```
#include <iostream>

using namespace std;

int main() {
    cout << 1 << endl;
    cout << 2 << endl;
    cout << 3 << endl;
    cout << 4 << endl;
    cout << 5 << endl;
}
```

When compiled and run, this program displays



```
1
2
3
4
5
```

How would you write the code to count to 10,000? Would you copy, paste, and modify 10,000 printing statements? You could, but that would be impractical! Counting is such a common activity, and computers

routinely count up to very large values, so there must be a better way. What we really would like to do is print the value of a variable (call it `count`), then increment the variable (`count++`), and repeat this process until the variable is large enough (`count == 5` or perhaps `count == 10000`). This process of executing the same section of code over and over is known as *iteration*, or *looping*, and in C++ we can implement loops in several different ways.

Listing 6.2 (`iterativecounttofive.cpp`) uses a `while` statement to count to five:

Listing 6.2: `iterativecounttofive.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int count = 1;           // Initialize counter
    while (count <= 5) {
        cout << count << endl; // Display counter, then
        count++;               // Increment counter
    }
}
```

Listing 6.2 (`iterativecounttofive.cpp`) uses a `while` statement to display a variable that is counting up to five. Unlike the approach taken in Listing 6.1 (`counttofive.cpp`), it is trivial to modify Listing 6.2 (`iterativecounttofive.cpp`) to count up to 10,000—just change the literal value 5 to 10000.

The line

```
while (count <= 5)
```

begins the `while` statement. The expression within the parentheses must be a Boolean expression. If the Boolean expression is true when the the program's execution reaches the `while` statement, the program executes the body of the `while` statement and then checks the condition again. The program repeatedly executes the statement(s) within the body of the `while` as long as the Boolean expression remains true.

If the Boolean expression is true when the `while` statement is executed, the body of the `while` statement is executed, and the body is executed repeatedly as long as the Boolean expression remains true.

The statements

```
cout << count << endl;
count++;
```

constitute the body of the `while` statement. The curly braces are necessary since more than one statement makes up the body.

The `while` statement has the general form:

```
while ( condition )
    statement
```

- The reserved word **while** begins the **while** statement.
- The Boolean expression *condition* determines whether the body will be (or will continue to be) executed. The expression *must* be enclosed within parentheses as shown.
- The *statement* is the statement to be executed while the Boolean expression is true. The statement makes up the body of the **while** statement. The statement may be a compound statement (multiple statements enclosed within curly braces, see Section 5.4).

Except for using the reserved word **while** instead of **if**, a **while** statement looks identical to an **if** statement. Sometimes beginning programmers confuse the two or accidentally type **if** when they mean **while** or vice-versa. Usually the very different behavior of the two statements reveals the problem immediately; however, sometimes, especially in nested complex logic, this mistake can be hard to detect.

Figure 6.1 shows how program execution flows through Listing 6.2 (iterativecounttofive.cpp).

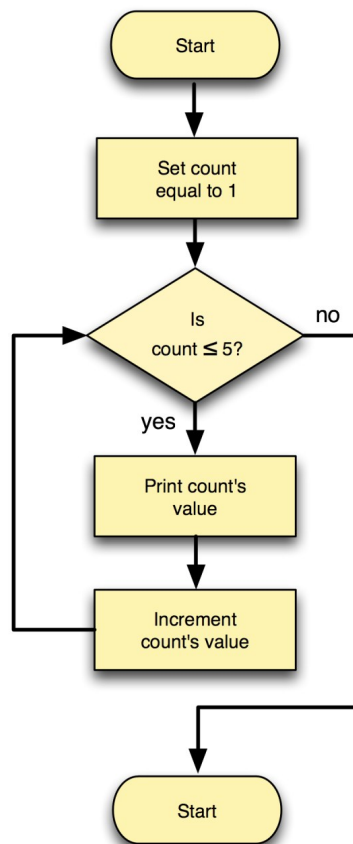


Figure 6.1: while flowchart for Listing 6.2 (iterativecounttofive.cpp)

The program checks the **while**'s condition before executing the body, and then re-checks the condition each time after it executes the body. If the condition is initially false the program's execution skips the

body completely and continues executing the statements that follow the **while**'s body. If the condition is initially true, the program repeatedly executes the body until the condition becomes false, at which point the loop terminates. Program execution then continues with the statements that follow the loop's body, if any. Observe that the body may never be executed if the Boolean expression in the condition is initially false.

Listing 6.3 (countup.cpp) counts up from zero as long as the user wishes to do so.

Listing 6.3: countup.cpp

```
/*
 * Counts up from zero. The user continues the count by entering
 * 'Y'. The user discontinues the count by entering 'N'.
 */

#include <iostream>

using namespace std;

int main() {
    char input;          // The users choice
    int count = 0;       // The current count
    bool done = false;   // We are not done

    while (!done) {

        // Print the current value of count
        cout << count << endl;
        cout << "Please enter \"Y\" to continue or \"N\" to quit: ";
        cin >> input;
        // Check for "bad" input
        if (input != 'Y' && input != 'y' && input != 'N' && input != 'n')
            cout << "\"" << input << "\""
                << " is not a valid choice" << endl;
        else if (input == 'Y' || input == 'y')
            count++;      // Keep counting
        else if (input == 'N' || input == 'n')
            done = true;  // Quit the loop
    }
}
```

A sample run of Listing 6.3 (countup.cpp) produces

```

0
Please enter "Y" to continue or "N" to quit: y
1
Please enter "Y" to continue or "N" to quit: y
2
Please enter "Y" to continue or "N" to quit: y
3
Please enter "Y" to continue or "N" to quit: q
"q" is not a valid choice
3
Please enter "Y" to continue or "N" to quit: r
"r" is not a valid choice
3
Please enter "Y" to continue or "N" to quit: W
"W" is not a valid choice
3
Please enter "Y" to continue or "N" to quit: Y
4
Please enter "Y" to continue or "N" to quit: y
5
Please enter "Y" to continue or "N" to quit: n

```

In Listing 6.3 (countup.cpp) the expression

```
input != 'Y' && input != 'y' && input != 'N' && input != 'n'
```

is true if the character variable **input** is not equal to one of the listed character literals. The Boolean variable **done** controls the loop's execution. It is important to note that the expression

```
!done
```

inside the **while**'s condition evaluates to the opposite truth value of the variable **done**; the expression does *not* affect the value of **done**. In other words, the **!** operator applied to a variable does not modify the variable's value. In order to actually change the variable **done**, you would need to reassign it, as in

```
done = !done; // Invert the truth value
```

For Listing 6.3 (countup.cpp) we have no need to invert its value. We ensure that its value is **false** initially and then make it **true** when the user enters a capital or lower-case *N*.

Listing 6.4 (addnonnegatives.cpp) is a program that allows a user to enter any number of non-negative integers. When the user enters a negative value, the program no longer accepts input, and it displays the sum of all the non-negative values. If a negative number is the first entry, the sum is zero.

Listing 6.4: addnonnegatives.cpp

```

/*
 * Allow the user to enter a sequence of non-negative
 * integers. The user ends the list with a negative
 * integer. At the end the sum of the non-negative
 * integers entered is displayed. The program prints
 * zero if the users no non-negative integers.
 */

```

```

*/

#include <iostream>

using namespace std;

int main() {
    int input = 0,    // Ensure the loop is entered
        sum = 0;     // Initialize sum

    // Request input from the user
    cout << "Enter numbers to sum, negative number ends list:";

    while (input >= 0) {    // A negative number exits the loop
        cin >> input;      // Get the value
        if (input >= 0)
            sum += input;  // Only add it if it is non-negative
    }
    cout << "Sum = " << sum << endl; // Display the sum
}

```

The initialization of `input` to zero coupled with the condition `input >= 0` of the `while` guarantees that program will execute the body of the `while` loop at least once. The `if` statement ensures that a negative entry will not be added to `sum`. (Could the condition have used `>` instead of `>=` and achieved the same results?) When the user enters a negative integer the program will not update `sum`, and the condition of the while will no longer be true. The program's execution then leaves the loop and executes the print statement at the end.

Listing 6.4 (`addnonnegatives.cpp`) shows that a `while` loop can be used for more than simple counting. The program does not keep track of the number of values entered. The program simply accumulates the entered values in the variable named `sum`.

It is a little awkward in Listing 6.4 (`addnonnegatives.cpp`) that the same condition appears twice, once in the `while` and again in the `if`. Furthermore, what if the user wishes to enter negative values along with non-negative values? The code can be simplified with a common C++ idiom that uses `cin` and the extraction operator as a condition within an `while` statement.

If `x` is an integer, the expression

```
cin >> x
```

evaluates to `false` if the user does not enter a valid integer literal. Armed with this knowledge we can simplify and enhance Listing 6.4 (`addnonnegatives.cpp`) as shown in Listing 6.5 (`addnumbers.cpp`).

Listing 6.5: `addnumbers.cpp`

```

#include <iostream>

using namespace std;

int main() {
    int input, sum = 0;
    cout << "Enter numbers to sum, type 'q' to end the list:";
    while (cin >> input)
        sum += input;
    cout << "Sum = " << sum << endl;
}

```

```
}

```

The condition reads a value from the input stream and, if it is successful, it is interpreted as **true**. When the user enters '**q**', the loop is terminated. If the user types '**q**' at the beginning, the loop is not entered. The **if** statement is no longer necessary, since the statement

```
sum += input;
```

can be executed only if **input** has been legitimately assigned. Also, the variable **input** no longer needs to be initialized with a value simply so the loop is entered the first time; now it is assigned and then checked within the condition of the **while**.

In Listing 6.5 (addnumbers.cpp), the program's execution will terminate with any letter the user types; an entry of '**x**' or Ctrl-Z will terminate the sequence just as well as '**q**'.

If you use the

```
while (cin >> x) {
    // Do something . . .
}
```



idiom, be aware that when the loop is exited due to the input stream being unable to read a value of the type of variable **x**, the **cin** input stream object is left in an error state and cannot be used for the rest of the program's execution. If you wish to use this technique and reuse **cin** later, you must reset **cin** and extract and discard keystrokes entered since the last valid use of the extractor operator. This recovery process is covered in Section 13.2, but, for now, use this idiom to control a loop only if the program does not require additional user input later during its execution.

Listing 6.6 (powersof10.cpp) prints the powers of 10 from 1 to 1,000,000,000 (the next power of ten, 10,000,000,000, is outside the range of the **int** type).

Listing 6.6: powersof10.cpp

```
#include <iostream>

using namespace std;

int main() {
    int power = 1;
    while (power <= 1000000000) {
        cout << power << endl;
        power *= 10;
    }
}
```

Listing 6.6 (powersof10.cpp) produces

```
1
10
100
1000
10000
100000
1000000
10000000
100000000
1000000000
```

It is customary to right justify a column of numbers, but Listing 6.6 (`powersof10.cpp`) prints the powers of ten with their most-significant digit left aligned. We can right align the numbers using a stream object called a *stream manipulator*. The specific stream manipulator we need is named **setw**. **setw** means “set width.” It can be used as

```
cout << setw(3) << x << endl;
```

This statement prints the value of **x** right justified within a three character horizontal space on the screen. Listing 6.7 (`powersof10justified.cpp`) shows the affects of **setw**.

Listing 6.7: `powersof10justified.cpp`

```
#include <iostream>
#include <iomanip>

using namespace std;

// Print the powers of 10 from 1 to 1,000,000,000
int main() {
    int power = 1;
    while (power <= 1000000000) {
        // Right justify each number in a field 10 wide
        cout << setw(10) << power << endl;
        power *= 10;
    }
}
```

Listing 6.7 (`powersof10justified.cpp`) prints

```

1
10
100
1000
10000
100000
1000000
10000000
100000000
1000000000

```

Observe that in order to use **setw** the compiler needs to be made aware of it. The needed information about **setw** is not found in the **iomanip** header file, so an additional preprocessor include directive is required:

```
#include <iomanip>
```

The **setw** manipulator “conditions” the output stream for the next item to be printed. The values passed to the “conditioned” stream are all right justified within the number of spaces specified by **setw**.

We can use a **while** statement to make Listing 5.11 (troubleshoot.cpp) more convenient for the user. Recall that the computer troubleshooting program forces the user to rerun the program once a potential program has been detected (for example, turn on the power switch, then run the program again to see what else might be wrong). A more desirable decision logic is shown in Figure 6.2.

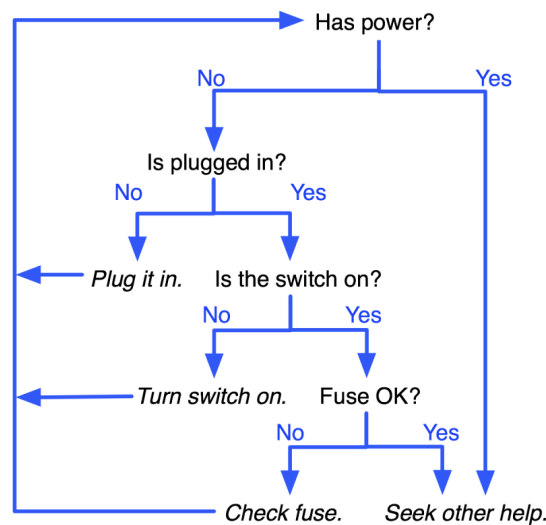


Figure 6.2: Decision tree for troubleshooting a computer system Listing 6.8 (troubleshootloop.cpp)

Listing 6.8 (troubleshootloop.cpp) incorporates a **while** statement so that the program’s execution continues until the problem is resolved or its resolution is beyond the capabilities of the program.

Listing 6.8: troubleshootloop.cpp

```

#include <iostream>

using namespace std;

int main() {
    cout << "Help! My computer doesn't work!" << endl;
    char choice;
    bool done = false; // Initially, we are not done

    while (!done) { // Continue until we are done
        cout << "Does the computer make any sounds "
            << "(fans, etc.) or show any lights? (y/n):";
        cin >> choice;
        // The troubleshooting control logic
        if (choice == 'n') { // The computer does not have power
            cout << "Is it plugged in? (y/n):";
            cin >> choice;
            if (choice == 'n') { // It is not plugged in, plug it in
                cout << "Plug it in." << endl;
            }
            else { // It is plugged in
                cout << "Is the switch in the \"on\" position? (y/n):";
                cin >> choice;
                if (choice == 'n') { // The switch is off, turn it on!
                    cout << "Turn it on." << endl;
                }
                else { // The switch is on
                    cout << "Does the computer have a fuse? (y/n):";
                    cin >> choice;
                    if (choice == 'n') { // No fuse
                        cout << "Is the outlet OK? (y/n):";
                        if (choice == 'n') { // Fix outlet
                            cout << "Check the outlet's circuit "
                                << "breaker or fuse. Move to a "
                                << "new outlet, if necessary. "
                                << endl;
                        }
                        else { // Beats me!
                            cout << "Please consult a service "
                                << "technician." << endl;
                            done = true; // Exhausted simple fixes
                        }
                    }
                    else { // Check fuse
                        cout << "Check the fuse. Replace if "
                            << "necessary." << endl;
                    }
                }
            }
        }
        else { // The computer has power
            cout << "Please consult a service technician." << endl;
            done = true; // Only troubleshoots power issues
        }
    }
}

```

```

    }
}

```

The bulk of the body of the Listing 6.8 (troubleshootloop.cpp) is wrapped by a **while** statement. The Boolean variable **done** is often called a *flag*. You can think of the flag being down when the value is false and raised when it is true. In this case, when the flag is raised, it is a signal that the program should terminate.

Notice the last 11 lines of Listing 6.8 (troubleshootloop.cpp):

```

    }
    }
}
else { // The computer has power
    cout << "Please consult a service technician." << endl;
    done = true; // Only troubleshoots power issues
}
}
}

```

In the way this code is organized, the matching opening curly brace of a particular closing curly brace can be found by scanning upward in the source code until the closest opening curly brace at the same indentation level is found. Our programming logic is now getting complex enough that the proper placement of curly braces is crucial for human readers to more quickly decipher how the program should work. See Section 4.5 for guidelines on indentation and curly brace placement to improve code readability.

6.2 Nested Loops

Just like in **if** statements, **while** bodies can contain arbitrary C++ statements, including other **while** statements. A loop can therefore be nested within another loop. To see how nested loops work, consider a program that prints out a multiplication table. Elementary school students use multiplication tables, or times tables, as they learn the products of integers up to 10 or even 12. Figure 6.3 shows a 10×10 multiplication table. We want our multiplication table program to be flexible and allow the user to specify the table's size. We will begin our development work with a simple program and add features as we go. First, we will not worry about printing the table's row and column titles, nor will we print the lines separating the titles from the contents of the table. Initially we will print only the contents of the table. We will see we need a nested loop to print the table's contents, but that still is too much to manage in our first attempt. In our first attempt we will print the rows of the table in a very rudimentary manner. Once we are satisfied that our simple program works we can add more features. Listing 6.9 (timestable-1st-try.cpp) shows our first attempt at a multiplication table.

Listing 6.9: timestable-1st-try.cpp

```

#include <iostream>

using namespace std;

int main() {
    int size; // The number of rows and columns in the table

```

×	1	2	3	4	5	6	7	8	9	10
1	1	2	3	4	5	6	7	8	9	10
2	2	4	6	8	10	12	14	16	18	20
3	3	6	9	12	15	18	21	24	27	30
4	4	8	12	16	20	24	28	32	36	40
5	5	10	15	20	25	30	35	40	45	50
6	6	12	18	24	30	36	42	48	54	60
7	7	14	21	28	35	42	49	56	63	70
8	8	16	24	32	40	48	56	64	72	80
9	9	18	27	36	45	54	63	72	81	90
10	10	20	30	40	50	60	70	80	90	100

Figure 6.3: A 10×10 multiplication table

```

cout << "Please enter the table size: ";
cin >> size;
// Print a size x size multiplication table
int row = 1;
while (row <= size) {           // Table has 10 rows.
    cout << "Row #" << row << endl;
    row++;                     // Next row
}
}

```

The output of Listing 6.9 (timestable-1st-try.cpp) is somewhat underwhelming:

```

Please enter the table size: 10
Row #1
Row #2
Row #3
Row #4
Row #5
Row #6
Row #7
Row #8
Row #9
Row #10

```

Listing 6.9 (timestable-1st-try.cpp) does indeed print each row in its proper place—it just does not supply the needed detail for each row. Our next step is to refine the way the program prints each row. Each row should contain **size** numbers. Each number within each row represents the product of the current row and current column; for example, the number in row 2, column 5 should be $2 \times 5 = 10$. In each row, therefore, we must vary the column number from 1 to **size**. Listing 6.10 (timestable-2nd-try.cpp) contains the needed refinement.

Listing 6.10: timestable-2nd-try.cpp

```
#include <iostream>

using namespace std;

int main() {
    int size; // The number of rows and columns in the table
    cout << "Please enter the table size: ";
    cin >> size;
    // Print a size x size multiplication table
    int row = 1;
    while (row <= size) { // Table has size rows.
        int column = 1; // Reset column for each row.
        while (column <= size) { // Table has size columns.
            int product = row*column; // Compute product
            cout << product << " "; // Display product
            column++; // Next element
        }
        cout << endl; // Move cursor to next row
        row++; // Next row
    }
}
```

We use a loop to print the contents of each row. The outer loop controls how many total rows the program prints, and the inner loop, executed in its entirety each time the program prints a row, prints the individual elements that make up a row.

The result of Listing 6.10 (timestable-2nd-try.cpp) is

```
Please enter the table size: 10
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
```

The numbers within each column are not lined up nicely, but the numbers are in their correct positions relative to each other. We can use the `setw` stream manipulator introduced in Listing 6.7 (powersof10justified.cpp) to right justify the numbers within a four-digit area. Listing 6.11 (timestable-3rd-try.cpp) contains this alignment adjustment.

Listing 6.11: timestable-3rd-try.cpp

```
#include <iostream>
#include <iomanip>
```

```

using namespace std;

int main() {
    int size; // The number of rows and columns in the table
    cout << "Please enter the table size: ";
    cin >> size;
    // Print a size x size multiplication table
    int row = 1;
    while (row <= size) { // Table has size rows.
        int column = 1; // Reset column for each row.
        while (column <= size) { // Table has size columns.
            int product = row*column; // Compute product
            cout << setw(4) << product; // Display product
            column++; // Next element
        }
        cout << endl; // Move cursor to next row
        row++; // Next row
    }
}

```

Listing 6.11 (timestable-3rd-try.cpp) produces the table's contents in an attractive form:

```

Please enter the table size: 10
 1  2  3  4  5  6  7  8  9 10
 2  4  6  8 10 12 14 16 18 20
 3  6  9 12 15 18 21 24 27 30
 4  8 12 16 20 24 28 32 36 40
 5 10 15 20 25 30 35 40 45 50
 6 12 18 24 30 36 42 48 54 60
 7 14 21 28 35 42 49 56 63 70
 8 16 24 32 40 48 56 64 72 80
 9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100

```

Input values of 5:

```

Please enter the table size: 5
 1  2  3  4  5
 2  4  6  8 10
 3  6  9 12 15
 4  8 12 16 20
 5 10 15 20 25

```

and 15:

```

Please enter the table size: 15
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
 2  4  6  8 10 12 14 16 18 20 22 24 26 28 30
 3  6  9 12 15 18 21 24 27 30 33 36 39 42 45
 4  8 12 16 20 24 28 32 36 40 44 48 52 56 60
 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
 6 12 18 24 30 36 42 48 54 60 66 72 78 84 90
 7 14 21 28 35 42 49 56 63 70 77 84 91 98 105
 8 16 24 32 40 48 56 64 72 80 88 96 104 112 120
 9 18 27 36 45 54 63 72 81 90 99 108 117 126 135
10 20 30 40 50 60 70 80 90 100 110 120 130 140 150
11 22 33 44 55 66 77 88 99 110 121 132 143 154 165
12 24 36 48 60 72 84 96 108 120 132 144 156 168 180
13 26 39 52 65 78 91 104 117 130 143 156 169 182 195
14 28 42 56 70 84 98 112 126 140 154 168 182 196 210
15 30 45 60 75 90 105 120 135 150 165 180 195 210 225

```

also give good results.

All that is left is to add the row and column titles and the lines that bound the edges of the table. Listing 6.12 (timestable.cpp) adds the necessary code.

Listing 6.12: timestable.cpp

```

#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int size; // The number of rows and columns in the table
    cout << "Please enter the table size: ";
    cin >> size;
    // Print a size x size multiplication table

    // First, print heading: 1 2 3 4 5 etc.
    cout << " ";
    // Print column heading
    int column = 1;
    while (column <= size) {
        cout << setw(4) << column; // Print heading for this column.
        column++;
    }
    cout << endl;

    // Print line separator: +-----+
    cout << "  +";
    column = 1;
    while (column <= size) {
        cout << "----"; // Print line for this column.
        column++;
    }
}

```

```

cout << endl;

// Print table contents
int row = 1;
while (row <= size) {           // Table has size rows.
    cout << setw(2) << row << " |"; // Print heading for this row.
    int column = 1;             // Reset column for each row.
    while (column <= size) {    // Table has size columns.
        int product = row*column; // Compute product
        cout << setw(4) << product; // Display product
        column++;               // Next element
    }
    row++;                     // Next row
    cout << endl;              // Move cursor to next row
}
}

```

When the user supplies the value 10, Listing 6.12 (timestable.cpp) produces

```

Please enter the table size: 10
      1  2  3  4  5  6  7  8  9 10
+-----+
1 | 1  2  3  4  5  6  7  8  9 10
2 | 2  4  6  8 10 12 14 16 18 20
3 | 3  6  9 12 15 18 21 24 27 30
4 | 4  8 12 16 20 24 28 32 36 40
5 | 5 10 15 20 25 30 35 40 45 50
6 | 6 12 18 24 30 36 42 48 54 60
7 | 7 14 21 28 35 42 49 56 63 70
8 | 8 16 24 32 40 48 56 64 72 80
9 | 9 18 27 36 45 54 63 72 81 90
10 | 10 20 30 40 50 60 70 80 90 100

```

An input of 15 yields

```

Please enter the table size: 15
      1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
+-----+
1 | 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
2 | 2  4  6  8 10 12 14 16 18 20 22 24 26 28 30
3 | 3  6  9 12 15 18 21 24 27 30 33 36 39 42 45
4 | 4  8 12 16 20 24 28 32 36 40 44 48 52 56 60
5 | 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
6 | 6 12 18 24 30 36 42 48 54 60 66 72 78 84 90
7 | 7 14 21 28 35 42 49 56 63 70 77 84 91 98 105
8 | 8 16 24 32 40 48 56 64 72 80 88 96 104 112 120
9 | 9 18 27 36 45 54 63 72 81 90 99 108 117 126 135
10 | 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150
11 | 11 22 33 44 55 66 77 88 99 110 121 132 143 154 165
12 | 12 24 36 48 60 72 84 96 108 120 132 144 156 168 180
13 | 13 26 39 52 65 78 91 104 117 130 143 156 169 182 195
14 | 14 28 42 56 70 84 98 112 126 140 154 168 182 196 210
15 | 15 30 45 60 75 90 105 120 135 150 165 180 195 210 225

```

If the user enters 7, the program prints

```

Please enter the table size: 7
      1  2  3  4  5  6  7
+-----+
1 | 1  2  3  4  5  6  7
2 | 2  4  6  8 10 12 14
3 | 3  6  9 12 15 18 21
4 | 4  8 12 16 20 24 28
5 | 5 10 15 20 25 30 35
6 | 6 12 18 24 30 36 42
7 | 7 14 21 28 35 42 49

```

The user even can enter a 1:

```

Please enter the table size: 1
      1
+----+
1 | 1

```

As we can see, the table automatically adjusts to the size and spacing required by the user's input.

This is how Listing 6.12 (timestable.cpp) works:

- It is important to distinguish what is done only once (outside all loops) from that which is done

repeatedly. The column heading across the top of the table is outside of all the loops; therefore, it is printed all at once.

- The work to print the heading for the rows is distributed throughout the execution of the outer loop. This is because the heading for a given row cannot be printed until all the results for the previous row have been printed.
- A code fragment like

```
if (x < 10)
    cout << " ";
cout << x;
```

prints **x** in one of two ways: if **x** is a one-digit number, it prints a space before it; otherwise, it does not print the extra space. The net effect is to right justify one and two digit numbers within a two character space printing area. This technique allows the columns within the times table to be properly right aligned.

- In the nested loop, **row** is the control variable for the outer loop; **column** controls the inner loop.
- The inner loop executes **size** times on every single iteration of the outer loop. How many times is the statement

```
cout << product << " ";    // Display product
```

executed? **size** × **size** times, one time for every product in the table.

- A newline is printed after the contents of each row is displayed; thus, all the values printed in the inner (**column**) loop appear on the same line.

Nested loops are used when an iterative process itself must be repeated. In our times table example, a **while** loop is used to print the contents of each row, but multiple rows must be printed. The inner loop prints the contents of each row, while the outer is responsible for printing all the rows.

Listing 6.13 (permuteabc.cpp) uses a triply-nested loop to print all the different arrangements of the letters A, B, and C. Each string printed is a *permutation* of **ABC**.

Listing 6.13: permuteabc.cpp

```
// File permuteabc.cpp

#include <iostream>

using namespace std;

int main() {
    char first = 'A';           // The first letter varies from A to C
    while (first <= 'C') {
        char second = 'A';
        while (second <= 'C') { // The second varies from A to C
            if (second != first) { // No duplicate letters
                char third = 'A';
                while (third <= 'C') { // The third varies from A to C
                    // Don't duplicate first or second letter
                    if (third != first && third != second)
                        cout << first << second << third << endl;
                }
            }
            second++;
        }
        first++;
    }
}
```

```
        third++;
    }
    second++;
}
first++;
}
```

Notice how the **if** statements are used to prevent duplicate letters within a given string. The output of Listing 6.13 (permuteabc.cpp) is all six permutations of **ABC**:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

6.3 Abnormal Loop Termination

By default, a **while** statement executes until its condition becomes false. The executing program checks this condition only at the “top” of the loop. This means that even if the Boolean expression that makes up the condition becomes false before the program completes executing all the statements within the body of the loop, all the remaining statements in the loop’s body must complete before the loop can once again check its condition. In other words, the **while** statement in and of itself cannot exit its loop somewhere in the middle of its body.

Ordinarily this behavior is not a problem. Usually the intention is to execute all the statements within the body as an indivisible unit. Sometimes, however, it is desirable to immediately exit the body or recheck the condition from the middle of the loop instead. C++ provides the **break** and **continue** statements to give programmers more flexibility designing the control logic of loops.

6.3.1 The break statement

C++ provides the **break** statement to implement middle-exiting control logic. The **break** statement causes the immediate exit from the body of the loop. Listing 6.14 (addmiddleexit.cpp) is a variation of Listing 6.4 (addnonnegatives.cpp) that illustrates the use of **break**.

Listing 6.14: addmiddleexit.cpp

```
#include <iostream>

using namespace std;

int main() {
    int input, sum = 0;
    cout << "Enter numbers to sum, negative number ends list:";
```

```
while (true) {
    cin >> input;
    if (input < 0)
        break;           // Exit loop immediately
    sum += input;
}
cout << "Sum = " << sum << endl;
}
```

The condition of the **while** in Listing 6.14 (addmiddleexit.cpp) is a tautology. This means the condition is true and can never be false. When the program's execution reaches the **while** statement it is guaranteed to enter the loop's body and the **while** loop itself does not provide a way of escape. The **if** statement in the loop's body:

```
if (input < 0) // Is input negative
    break;    // If so, exit the loop immediately
```

provides the necessary exit. In this case the **break** statement, executed conditionally based on the value of the variable **input**, exits the loop. In Listing 6.14 (addmiddleexit.cpp) the **break** statement executes only when the user enters a negative number. When the program's execution encounters the **break** statement, it immediately jumps out of the loop. It skips any statements following the **break** within the loop's body. Since the statement

```
sum += input; // Accumulate user input
```

appears after the **break**, it is not possible for the program to add a negative number to the **sum** variable.

Some software designers believe that programmers should use the **break** statement sparingly because it deviates from the normal loop control logic. Ideally, every loop should have a single entry point and single exit point. While Listing 6.14 (addmiddleexit.cpp) has a single exit point (the **break** statement), some programmers commonly use **break** statements within **while** statements in the which the condition for the while is not a tautology. Adding a **break** statement to such a loop adds an extra exit point (the top of the loop where the condition is checked is one point, and the **break** statement is another). Using multiple **break** statements within a single loop is particularly dubious and you should avoid that practice.

Why have the **break** statement at all if its use is questionable and it is dispensable? The logic in Listing 6.4 (addnonnegatives.cpp) is fairly simple, so the restructuring of Listing 6.14 (addmiddleexit.cpp) is straightforward; in general, the effort to restructure code to avoid a **break** statement may complicate the logic a bit and require the introduction of an additional Boolean variable. As shown in Figure 6.4, any program that uses a **break** statement can be rewritten so that the **break** statement is not used. The no-**break** version introduces a Boolean variable, and the loop control logic is a little more complicated. The no-**break** version uses more memory (an extra variable) and more time to execute (requires an extra check in the loop condition during every iteration of the loop). This extra memory is insignificant, and except for rare, specialized applications, the extra execution time is imperceptible. In most cases, the more important issue is that the more complicated the control logic for a given section of code, the more difficult the code is to write correctly. In some situations, even though it violates the “single entry point, single exit point” principle, a simple **break** statement is an acceptable loop control option.

6.3.2 The goto Statement

The **break** statement exits the single loop in which it is located. A **break** statement is insufficient to jump completely out of the middle of a nested loop. The **goto** statement allows the program's execution flow

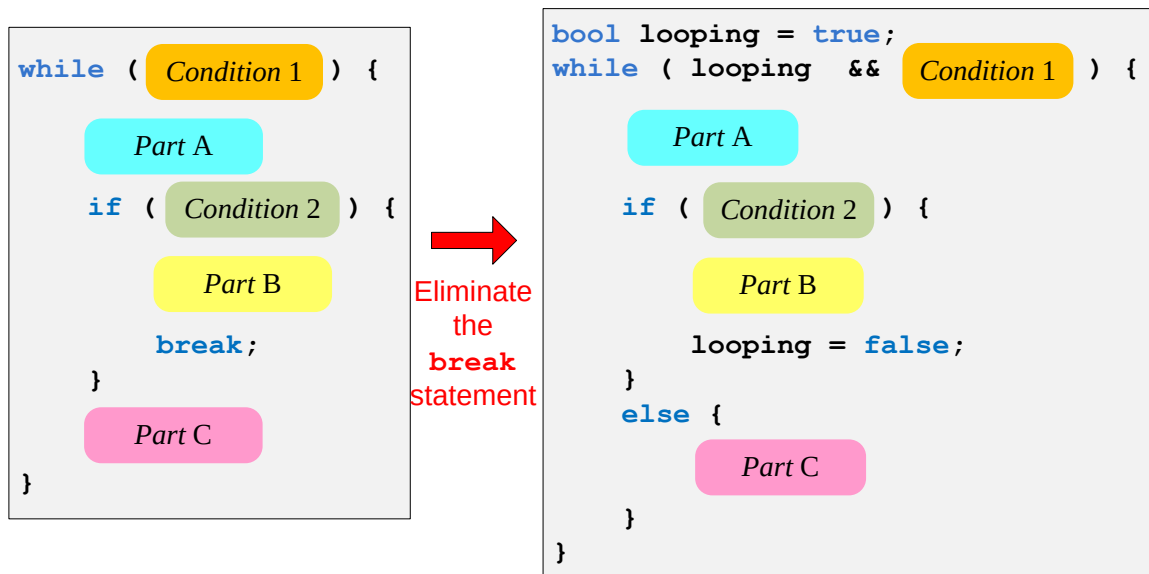


Figure 6.4: The code on the left generically represents any loop that uses a `break` statement. It is possible to transform the code on the left to eliminate the `break` statement, as the code on the right shows.

to jump to a specified location within the function. Listing 6.15 (`exitnested.cpp`) uses a `goto` statement to jump out from the middle of a nested loop.

Listing 6.15: `exitnested.cpp`

```
#include <iostream>

using namespace std;

int main() {
    // Compute some products
    int op1 = 2;
    while (op1 < 100) {
        int op2 = 2;
        while (op2 < 100) {
            if (op1 * op2 == 3731)
                goto end;
            cout << "Product is " << (op1 * op2) << endl;
            op2++;
        }
        op1++;
    }
end:
    cout << "The end" << endl;
}
```

When `op1 * op2` is 3731, program flow will jump to the specified label within the program. In this example, the label is named `end`, but this name is arbitrary. Like variable names, label names should be

chosen to indicate their intended purpose. The label here named **end** comes after and outside the nested **while** loops.

A label's name is an identifier (see Section 3.3), and a label is distinguished by the colon that immediately follows its name. A label represents a target to which a **goto** can jump. A **goto** label must appear before a statement within a function.

With the **goto** statement, the **while** is superfluous; for example, Listing 6.2 (*iterativecounttofive.cpp*) could be rewritten without the **while** statement as shown in Listing 6.16 (*gotoloop.cpp*).

Listing 6.16: gotoloop.cpp

```
#include <iostream>

using namespace std;

int main() {
    int count = 1;           // Initialize counter
top:
    if (count > 5)
        goto end;
    cout << count << endl;   // Display counter, then
    count++;                 // Increment counter
    goto top;
end:
    ; // Target is an empty statement
}
```

Early programming languages like FORTRAN and early versions of BASIC did not have structured statements like **while**, so programmers were forced to use **goto** statements to write loops. The problem with using **goto** statements is that it is easy to develop program logic that is very difficult to understand, even for the original author of the code. See the Wikipedia article about *spaghetti code* (http://en.wikipedia.org/wiki/Spaghetti_code). The structured programming revolution of the 1960s introduced constructs such as the **while** statement and resulted in the disappearance of the use of **goto** in most situations. All modern programming languages have a form of the **while** statement, so the **goto** statement in C++ is largely ignored except for the case of breaking out of a nested loop. You similarly should restrict your use of the **goto** statement to the abnormal exit of nested loops.

6.3.3 The continue Statement

When a program's execution encounters a **break** statement inside a loop, it skips the rest of the body of the loop and exits the loop. The **continue** statement is similar to the **break** statement, except the **continue** statement does not necessarily exit the loop. The **continue** statement skips the rest of the body of the loop and immediately checks the loop's condition. If the loop's condition remains true, the loop's execution resumes at the top of the loop. Listing 6.17 (*continueexample.cpp*) shows the **continue** statement in action.

Listing 6.17: continueexample.cpp

```
#include <iostream>

using namespace std;

int main() {
```

```

int input, sum = 0;
bool done = false;
while (!done) {
    cout << "Enter positive integer (999 quits): ";
    cin >> input;
    if (input < 0) {
        cout << "Negative value " << input << " ignored"
            << endl;
        continue; // Skip rest of body for this iteration
    }
    if (input != 999) {
        cout << "Tallying " << input << endl;
        sum += input;
    }
    else
        done = (input == 999); // 999 entry exits loop
}
cout << "sum = " << sum << endl;
}

```

Programmers do not use the `continue` statement as frequently as the `break` statement since it is easy to transform the code the would use `continue` into an equivalent form that does not. Listing 6.18 (`nocontinueexample.cpp`) works exactly like Listing 6.17 (`continueexample.cpp`), but it avoids the `continue` statement.

Listing 6.18: `nocontinueexample.cpp`

```

#include <iostream>

using namespace std;

int main() {
    int input, sum = 0;
    bool done = false;
    while (!done) {
        cout << "Enter positive integer (999 quits): ";
        cin >> input;
        if (input < 0) {
            cout << "Negative value " << input << " ignored"
                << endl;
        }
        else
            if (input != 999) {
                cout << "Tallying " << input << endl;
                sum += input;
            }
            else
                done = (input == 999); // 999 entry exits loop
    }
    cout << "sum = " << sum << endl;
}

```

Figure 6.5 shows how we can rewrite any program that uses a `continue` statement into an equivalent form that does not use `continue`. The transformation is simpler than for `break` elimination (see Figure 6.4), since the loop's condition remains the same, and no additional variable is needed. The version

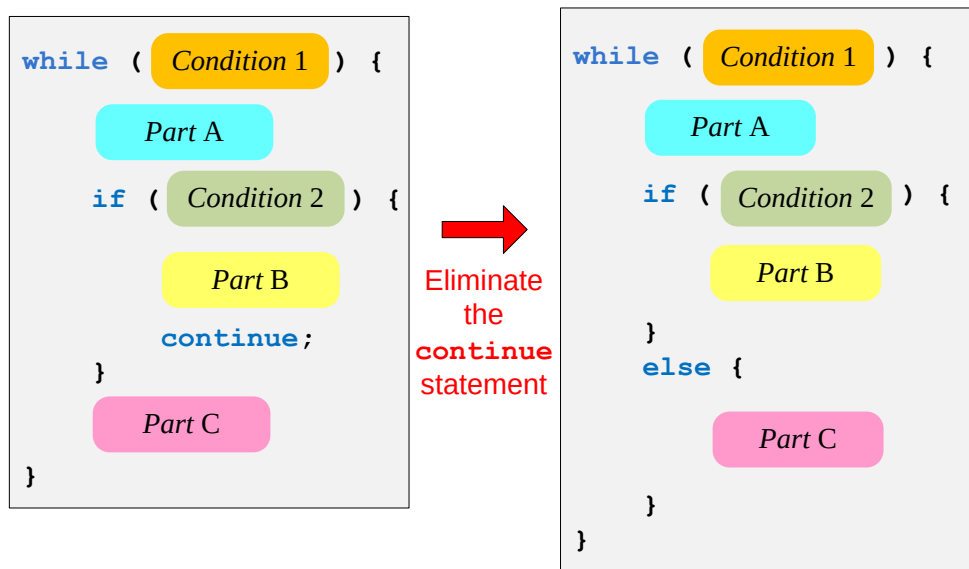


Figure 6.5: The code on the left generically represents any loop that uses a `continue` statement. It is possible to transform the code on the left to eliminate the `continue` statement, as the code on the right shows.

that uses `continue` is no more efficient than the version that uses `else`; in fact, the Visual C++ and GNU C++ compilers generate the same machine language code for Listing 6.17 (`continueexample.cpp`) and Listing 6.18 (`nocontinueexample.cpp`). Also, the logic of the `else` version is no more complex than the `continue` version. Therefore, unlike the `break` statement above, there is no compelling reason to use the `continue` statement. Sometimes a programmer may add `continue` statement at the last minute to an existing loop body to handle an exceptional condition (like ignoring negative numbers in the example above) that initially went unnoticed. If the body of the loop is lengthy, a conditional statement with a `continue` can be added easily near the top of the loop body without touching the logic of the rest of the loop. The `continue` statement thus merely provides a convenient alternative for the programmer. The `else` version is preferred.

6.4 Infinite Loops

An infinite loop is a loop without an exit. Once the program flow enters an infinite loop's body it cannot escape. Some infinite loops are by design; for example, a long-running server application, like a Web server, may need to continuously check for incoming connections. This server application can perform this checking within a loop that runs indefinitely. All too often, however, beginning programmers create infinite loops by accident, and these infinite loops represent logic errors in their programs.

Intentional infinite loops should be made obvious. For example,

```
while (true) {
    /* Do something forever . . . */
}
```

The Boolean literal `true` is always true, so it is impossible for the loop's condition to be false. The only ways to exit the loop is via a `break` statement, `return` statement (see Chapter 9), or an `exit` call (see Section 8.1) embedded somewhere within its body.

It is easy to write an intentional infinite loop. Accidental infinite loops are quite common, but can be puzzling for beginning programmers to diagnose and repair. Consider Listing 6.19 (`findfactors.cpp`) that attempts to print all the integers with their associated factors from 1 to 20.

Listing 6.19: `findfactors.cpp`

```
#include <iostream>

using namespace std;

int main() {
    // List of the factors of the numbers up to 20
    int n = 1;
    const int MAX = 20;
    while (n <= MAX) {
        int factor = 1;
        cout << n << ": ";
        while (factor <= n)
            if (n % factor == 0) {
                cout << factor << " ";
                factor++;
            }
        cout << endl; // Go to next line for next n
        n++;
    }
}
```

It displays

```
1: 1
2: 1 2
3: 1
```

and then “freezes up” or “hangs,” ignoring any user input (except the key sequence `Ctrl C` on most systems which interrupts and terminates the running program). This type of behavior is a frequent symptom of an unintentional infinite loop. The factors of 1 display properly, as do the factors of 2. The first factor of 3 is properly displayed and then the program hangs. Since the program is short, the problem may be easy to locate. In some programs, though, the error may be challenging to find. Even in Listing 6.19 (`findfactors.cpp`) the debugging task is nontrivial since it involves nested loops. (Can you find and fix the problem in Listing 6.19 (`findfactors.cpp`) before reading further?)

In order to avoid infinite loops, we must ensure that the loop exhibits certain properties:

- The loop's condition must not be a tautology (a Boolean expression that can never be false). For example,

```
while (i >= 1 || i <= 10) {
    /* Body omitted */
}
```



```
}

```

is an infinite loop since any value chosen for **i** will satisfy one or both of the two subconditions. Perhaps the programmer intended to use a **&&** instead of **| |** to stay in the loop as long as **i** remains in the range 1...10.

In Listing 6.19 (findfactors.cpp) the outer loop condition is

```
n <= MAX

```

If **n** is 21 and **MAX** is 20, then the condition is false, so this is not a tautology. Checking the inner loop condition:

```
factor <= n

```

we see that if **factor** is 3 and **n** is 2, then the expression is false; therefore, it also is not a tautology.

- The condition of a **while** must be true initially to gain access to its body. The code within the body must modify the state of the program in some way so as to influence the outcome of the condition that is checked at each iteration. This usually means code within the body of the loop modifies one of the variables used in the condition. Eventually the variable assumes a value that makes the condition false, and the loop terminates.

In Listing 6.19 (findfactors.cpp) the outer loop's condition involves the variable **n** and constant **MAX**. **MAX** cannot change, so to avoid an infinite loop it is essential that **n** be modified within the loop. Fortunately, the last statement in the body of the outer loop increments **n**. **n** is initially 1 and **MAX** is 20, so unless the circumstances arise to make the inner loop infinite, the outer loop should eventually terminate.

The inner loop's condition involves the variables **n** and **factor**. No statement in the inner loop modifies **n**, so it is imperative that **factor** be modified in the loop. The good news is **factor** is incremented in the body of the inner loop, but the bad news is the increment operation is protected within the body of the **if** statement. The inner loop contains one statement, the **if** statement. That **if** statement in turn has two statements in its body:

```
while (factor <= n)
    if (n % factor == 0) {
        cout << factor << " ";
        factor++;
    }

```

If the condition of the **if** is ever false, the state of the program will not change when the body of the inner loop is executed. This effectively creates an infinite loop. The statement that modifies **factor** must be moved outside of the **if** statement's body:

```
while (factor <= n)
    if (n % factor == 0)
        cout << factor << " ";
    factor++;

```

This new version runs correctly.

Programmers can use a debugger to step through a program to see where and why an infinite loop arises. Another common technique is to put print statements in strategic places to examine the values of the variables involved in the loop's control. The original inner loop can be so augmented:

```

while (factor <= n) {
    cout << "factor = " << factor
        << "    n = " << n << endl;
    if (n % factor == 0) {
        cout << factor << " ";
        factor++;
    }
}

```

It produces the following output:

```

1: factor = 1    n = 1
1
2: factor = 1    n = 2
1 factor = 2    n = 2
2
3: factor = 1    n = 3
1 factor = 2    n = 3
factor = 2    n = 3
factor = 2    n = 3
factor = 2    n = 3
factor = 2    n = 3
factor = 2    n = 3
.
.
.

```

The program continues to print the same line until the user interrupts its execution. The output demonstrates that once **factor** becomes equal to 2 and **n** becomes equal to 3 the program's execution becomes trapped in the inner loop. Under these conditions:

1. $2 < 3$ is true, so the loop continues and
2. $3 \% 2$ is equal to 1, so the **if** statement will not increment **factor**.

It is imperative that **factor** be incremented each time through the inner loop; therefore, the statement incrementing **factor** must be moved outside of the **if**'s guarded body.

6.5 Iteration Examples

We can implement some sophisticated algorithms in C++ now that we are armed with **if** and **while** statements. This section provides several examples that show off the power of conditional execution and iteration.

6.5.1 Drawing a Tree

Suppose we must write a program that draws a triangular tree, and the user provides the tree's height. A tree that is five levels tall would look like

```

    *
   ***
  *****
 *****
*****

```

whereas a three-level tree would look like

```

    *
   ***
  *****

```

If the height of the tree is fixed, we can write the program as a simple variation of Listing 2.4 (arrow.cpp) which uses only printing statements and no loops. Our program, however, must vary its height and width based on input from the user.

Listing 6.20 (starttree.cpp) provides the necessary functionality.

Listing 6.20: starttree.cpp

```

#include <iostream>

using namespace std;

int main() {
    int height;    // Height of tree
    cout << "Enter height of tree: ";
    cin >> height; // Get height from user
    int row = 0;    // First row, from the top, to draw
    while (row < height) { // Draw one row for every unit of height
        // Print leading spaces
        int count = 0;
        while (count < height - row) {
            cout << " ";
            count++;
        }
        // Print out stars, twice the current row plus one:
        // 1. number of stars on left side of tree
        //    = current row value
        // 2. exactly one star in the center of tree
        // 3. number of stars on right side of tree
        //    = current row value
        count = 0;
        while (count < 2*row + 1) {

```

```

        cout << " ";
        count++;
    }
    // Move cursor down to next line
    cout << endl;
    // Change to the next row
    row++;
}
}

```

When a user runs Listing 6.20 (starttree.cpp) and enters **7**, the program displays

```

Enter height of tree: 7
      *
     ***
    *****
   *********
  ***********
 *****
*****

```

Listing 6.20 (starttree.cpp) uses two sequential **while** loops both nested within a **while** loop. The outer **while** loop is responsible for drawing one row of the tree each time its body is executed:

- The program will execute the outer **while** loop's body as long as the user enters a value greater than zero; if the user enters zero or less, the program terminates and does nothing. This is the expected behavior.
- The last statement in the body of the outer **while**:

```
row++;
```

ensures that the variable **row** increases by one each time through the loop; therefore, it eventually will equal **height** (since it initially had to be less than **height** to enter the loop), and the loop will terminate. There is no possibility of an infinite loop here.

- The body of the outer loop consists of more than one statement; therefore, the body must be enclosed within curly braces. Whenever a group of statements is enclosed within curly braces a *block* is formed. Any variable declared within a block is local to that block. A variable's scope (the section of the source code in which the variable exists and can be used) is from its point of declaration to the end of the block in which it is declared. For example, the variables **height** and **row** are declared in the block that is **main**'s body; thus, they are local to **main**. The variable **count** is declared within the block that is the body of the outer **while** statement; therefore, **count** is local to the outer **while** statement. An attempt to use **count** *outside* the body of the outer **while** statement would be an error.

What does it mean for a variable **x** to be *local* to a particular section of code? It means **x** does not exist outside its scope. There may be other variables in the program named **x**, but they are different variables. If it seems odd that you can have two different variables in the same program with the same name, consider the fact that there can be two people in the same room with the same name. They are different people, but

they have the same name. Similarly, the meaning of a variable depends on its context, and its name is not necessarily unique.

The two inner loops play distinct roles:

- The first inner loop prints spaces. The number of spaces printed is equal to the height of the tree the first time through the outer loop and decreases each iteration. This is the correct behavior since each succeeding row moving down contains fewer leading spaces but more asterisks.
- The second inner loop prints the row of asterisks that make up the tree. The first time through the outer loop, row is zero, so no left side asterisks are printed, one central asterisk is printed (the top of the tree), and no right side asterisks are printed. Each time through the loop the number of left-hand and right-hand stars to print both increase by one and the same central asterisk is printed; therefore, the tree grows one wider on each side each line moving down. Observe how the $2 \times \text{row} + 1$ value expresses the needed number of asterisks perfectly.
- While it seems asymmetrical, note that no third inner loop is required to print trailing spaces on the line after the asterisks are printed. The spaces would be invisible, so there is no reason to print them!

6.5.2 Printing Prime Numbers

A *prime number* is an integer greater than one whose only factors (also called divisors) are one and itself. For example, 29 is a prime number (only 1 and 29 divide into it with no remainder), but 28 is not (2, 4, 7, and 14 are factors of 28). Prime numbers were once merely an intellectual curiosity of mathematicians, but now they play an important role in cryptography and computer security.

The task is to write a program that displays all the prime numbers up to a value entered by the user. Listing 6.21 (printprimes.cpp) provides one solution.

Listing 6.21: printprimes.cpp

```
#include <iostream>

using namespace std;

int main() {
    int max_value;
    cout << "Display primes up to what value? ";
    cin >> max_value;
    int value = 2; // Smallest prime number
    while (value <= max_value) {
        // See if value is prime
        bool is_prime = true; // Provisionally, value is prime
        // Try all possible factors from 2 to value - 1
        int trial_factor = 2;
        while (trial_factor < value) {
            if (value % trial_factor == 0) {
                is_prime = false; // Found a factor
                break; // No need to continue; it is NOT prime
            }
            trial_factor++;
        }
        if (is_prime)
            cout << value << " "; // Display the prime number
    }
}
```

```

        value++;    // Try the next potential prime number
    }
    cout << endl;    // Move cursor down to next line
}

```

Listing 6.21 (printprimes.cpp), with an input of 90, produces:

```

Display primes up to what value? 90
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89

```

The logic of Listing 6.21 (printprimes.cpp) is a little more complex than that of Listing 6.20 (starttree.cpp). The user provides a value for **max_value**. The main loop (outer **while** iterates over all the values from two to **max_value**:

- Two new variables, local to the body of the outer loop, are introduced: **trial_factor** and **is_prime**. **is_prime** is initialized to true, meaning **value** is assumed to be prime unless our tests prove otherwise. **trial_factor** takes on all the values from two to **value - 1** in the inner loop:

```

int trial_factor = 2;
while (trial_factor < value) {
    if (value % trial_factor == 0) {
        is_prime = false;    // Found a factor
        break;    // No need to continue; it is NOT prime
    }
    trial_factor++;
}

```

The expression **value % trial_factor** is zero when **trial_factor** divides into **value** with no remainder—exactly when **trial_factor** is a factor of **value**. If the executing program determines that any of the values of **trial_factor** is a factor of **value**, then it sets **is_prime** to false and exits the loop via the **break**. If the loop continues to completion, the program never sets **is_prime** to false, which means it found no factors and **value** is indeed prime.

- The **if** statement after the inner loop:

```

if (is_prime)
    cout << value << " ";    // Display the prime number

```

simply checks the status of **is_prime**. If **is_prime** is true, then **value** must be prime, so it prints **value** along with an extra space for separation from output it may produce during subsequent iterations.

Some important questions we can ask include:

1. If the user enters a 2, will it be printed?

In this case **max_value = value = 2**, so the condition of the outer loop

```

value <= max_value

```

is true, since `2 <= 2`. `is_prime` is set to true, but the condition of the inner loop

```
trial_factor < value
```

is not true (2 is not less than 2). Thus, the inner loop is skipped, `is_prime` is not changed from true, and 2 is printed. This behavior is correct because 2 is the smallest prime number (and the only even prime).

2. if the user enters a number less than 2, is anything printed?

The `while` condition ensures that values less than two are not considered. The body of the `while` will never be entered. Only the newline is printed, and no numbers are displayed. This behavior is correct.

3. Is the inner loop guaranteed to always terminate?

In order to enter the body of the inner loop, `trial_factor` must be less than `value`. `value` does not change anywhere in the loop. `trial_factor` is not modified anywhere in the `if` statement within the loop, and it is incremented within the loop immediately after the `if` statement. `trial_factor` is, therefore, incremented during each iteration of the loop. Eventually, `trial_factor` will equal `value`, and the loop will terminate.

4. Is the outer loop guaranteed to always terminate?

In order to enter the body of the outer loop, `value` must be less than or equal to `max_value`. `max_value` does not change anywhere in the loop. `value` is increased in the last statement within the body of the outer loop, and `value` is not modified anywhere else. Since the inner loop is guaranteed to terminate as shown in the previous answer, eventually `value` will exceed `max_value` and the loop will end.

We can rearrange the logic of the inner `while` to avoid the `break` statement. The current version is:

```
while (trial_factor < value) {
    if (value % trial_factor == 0) {
        is_prime = false; // Found a factor
        break; // No need to continue; it is NOT prime
    }
    trial_factor++;
}
```

We can rewrite it as:

```
while (is_prime && trial_factor < value) {
    is_prime = (value % trial_factor != 0);
    trial_factor++; // Try next factor
}
```

This version without the `break` introduces a slightly more complicated condition for the `while` but removes the `if` statement within its body. `is_prime` is initialized to true before the loop. Each time through the loop it is reassigned. `trial_factor` will become false if at any time `value % trial_factor` is zero. This is exactly when `trial_factor` is a factor of `value`. If `is_prime` becomes false, the loop cannot continue, and if `is_prime` never becomes false, the loop ends when `trial_factor` becomes equal to `value`. Due to operator precedence, the parentheses are not necessary. The parentheses do improve readability, since an expression including both `==` and `!=` is awkward for humans to parse. When parentheses are placed where they are not needed, as in

```
x = (y + 2);
```

the compiler simply ignores them, so there is no efficiency penalty in the compiled code.

We can shorten the loop even further:

```
while (is_prime && trial_factor < value)
    is_prime = (value % trial_factor++ != 0);
```

This version uses the post-increment operator within the test expression (see Section 4.8). Recall that with the post-increment operator the value of the variable is used in the surrounding expression (if any), and then the variable is incremented. Since the `while`'s body now contains only one statement, the curly braces are not needed.

6.6 Summary

- The `while` statement allows the execution of code sections to be repeated multiple times.
- The condition of the `while` controls the execution of statements within the `while`'s body.
- The statements within the body of a `while` are executed over and over until the condition of the `while` is false.
- If the `while`'s condition is initially false, the body is not executed at all.
- In an infinite loop, the `while`'s condition never becomes false.
- The statements within the `while`'s body must eventually lead to the condition being false; otherwise, the loop will be infinite.
- Do not confuse `while` statements with `if` statements; their structure is very similar (`while` reserved word instead of the `if` word), but they behave differently.
- Infinite loops are rarely intentional and are usually accidental.
- An infinite loop can be diagnosed by putting a printing statement inside its body.
- An assignment expression has a value; the expression's value is the same as the expression on the right side of the assignment operator. This fact can be used to streamline the control of a loop that repeats based on user input.
- A loop contained within another loop is called a nested loop.
- Iteration is a powerful mechanism and can be used to solve many interesting problems.
- A block is any section of source code enclosed within curly braces. A compound statement is one example of a block.
- A variable declared within a block is local to that block.
- Complex iteration using nested loops mixed with conditional statements can be difficult to do correctly.
- Sometimes simple optimizations can speed up considerably the execution of loops.

- The **break** statement immediately exits a loop, skipping the rest of the loop's body, without checking to see if the condition is true or false. Execution continues with the statement immediately following the body of the loop.
- In a nested loop, the **break** statement exits only the loop in which the **break** is found.
- The **goto** statement directs the program's execution to a labeled statement within the function. The **goto** statement is legitimately used *only* to exit completely from the depths of a nested loop.
- The **continue** statement immediately checks the loop's condition, skipping the rest of the loop's body. If the condition is true, the execution continues at the top of the loop as usual; otherwise, the loop is terminated and execution continues with the statement immediately following the loop's body. false.
- In a nested loop, the **continue** statement affects only the loop in which the **continue** is found.

6.7 Exercises

1. In Listing 6.4 (addnonnegatives.cpp) could the condition of the **if** statement have used **>** instead of **>=** and achieved the same results? Why?
2. In Listing 6.4 (addnonnegatives.cpp) could the condition of the **while** statement have used **>** instead of **>=** and achieved the same results? Why?
3. Use a loop to rewrite the following code fragment so that it uses just one **cout** and one **endl**.

```
cout << 2 << endl;
cout << 4 << endl;
cout << 6 << endl;
cout << 8 << endl;
cout << 10 << endl;
cout << 12 << endl;
cout << 14 << endl;
cout << 16 << endl;
```

4. In Listing 6.4 (addnonnegatives.cpp) what would happen if the statement containing **cin** is moved out of the loop? Is moving the assignment out of the loop a good or bad thing to do? Why?
5. How many asterisks does the following code fragment print?

```
int a = 0;
while (a < 100) {
    cout << "★";
    a++;
}
cout << endl;
```

6. How many asterisks does the following code fragment print?

```
int a = 0;
while (a < 100)
    cout << "★";
cout << endl;
```

7. How many asterisks does the following code fragment print?

```
int a = 0;
while (a > 100) {
    cout << "*";
    a++;
}
cout << endl;
```

8. How many asterisks does the following code fragment print?

```
int a = 0;
while (a < 100) {
    int b = 0;
    while (b < 55) {
        cout << "*";
        b++;
    }
    cout << endl;
}
```

9. How many asterisks does the following code fragment print?

```
int a = 0;
while (a < 100) {
    if (a % 5 == 0)
        cout << "*";
    a++;
}
cout << endl;
```

10. How many asterisks does the following code fragment print?

```
int a = 0;
while (a < 100) {
    int b = 0;
    while (b < 40) {
        if ((a + b) % 2 == 0)
            cout << "*";
        b++;
    }
    cout << endl;
    a++;
}
```

11. How many asterisks does the following code fragment print?

```
int a = 0;
while (a < 100) {
    int b = 0;
    while (b < 100) {
        int c = 0;
        while (c < 100) {
```

```
        cout << "★";
        c++;
    }
    b++;
}
a++;
}
cout << endl;
```

12. What is printed by the following code fragment?

```
int a = 0;
while (a < 100)
    cout << a++;
cout << endl;
```

13. What is printed by the following code fragment?

```
int a = 0;
while (a > 100)
    cout << a++;
cout << endl;
```

14. Rewrite the following code fragment using a **break** statement and eliminating the **done** variable. Your code should behave identically to this code fragment.

```
bool done = false;
int n = 0, m = 100;
while (!done && n != m) {
    cin >> n;
    if (n < 0)
        done = true;
    cout << "n = " << endl;
}
```

15. Rewrite the following code fragment so it does not use a **break** statement. Your code should behave identically to this code fragment.

```
// Code with break ...
```

16. Rewrite the following code fragment so it eliminates the **continue** statement. Your new code's logic should be simpler than the logic of this fragment.

```
int x = 100, y;
while (x > 0) {
    cin >> y;
    if (y == 25) {
        x--;
        continue;
    }
    cin >> x;
    cout << "x = " << x << endl;
}
```

17. Suppose you were given some code from the 1960s in a language that did not support structured statements like `while`. Your task is to modernize it and adapt it to C++. The following code fragment has been adapted to C++ already, but you must now structure it with a `while` statement to replace the `gotos`. Your code should be `goto` free and still behave identically to this code fragment.

```

    int i = 0;
top:  if (i >= 10)
        goto end;
    cout << i << endl;
    i++;
    goto top;
end:

```

18. Simplify with `goto`...
19. What is printed by the following code fragment?

```

int a = 0;
while (a < 100);
    cout << a++;
cout << endl;

```

20. Write a C++ program that accepts a single integer value entered by the user. If the value entered is less than one, the program prints nothing. If the user enters a positive integer, n , the program prints an $n \times n$ box drawn with `*` characters. If the users enters 1, for example, the program prints

```

*

```

If the user enters a 2, it prints

```

**
**

```

An entry of three yields

```

***
***
***

```

and so forth. If the user enters 7, it prints

```

*****
*****
*****
*****
*****
*****
*****

```


Chapter 7

Other Conditional and Iterative Statements

The `if/else` and `while` statements are sufficient to implement any algorithms that involve conditional execution and looping. The `break` and `continue` statements are convenient but are not necessary. C++ provides some additional conditional and iterative statements that are more convenient to use in some circumstances. These additional statements include

- `switch`: an alternative to some multi-way `if/else` statements
- the conditional operator: an expression that exhibits the behavior of an `if/else` statement
- `do/while`: a loop that checks its condition after its body is executed
- `for`: a loop convenient for counting

These alternate constructs allow certain parts of algorithms to be expressed more clearly and succinctly. This chapter explores these other forms of expressing conditional execution and iteration.

7.1 The switch Statement

The `switch` statement provides a convenient alternative for some multi-way `if/else` statements like the one in Listing 5.15 (`restyleddigittoword.cpp`). Listing 7.1 (`switchdigittoword.cpp`) is a new implementation of Listing 5.15 (`restyleddigittoword.cpp`) that uses a `switch` statement instead of a multi-way `if/else` statement.

Listing 7.1: switchdigittoword.cpp

```
#include <iostream>

using namespace std;

int main() {
    int value;
    cout << "Please enter an integer in the range 0...5: ";
```

```
cin >> value;
switch (value) {
    case 0:
        cout << "zero";
        break;
    case 1:
        cout << "one";
        break;
    case 2:
        cout << "two";
        break;
    case 3:
        cout << "three";
        break;
    case 4:
        cout << "four";
        break;
    case 5:
        cout << "five";
        break;
    default:
        if (value < 0)
            cout << "Too small";
        else
            cout << "Too large";
        break;
}
cout << endl;
}
```

The general form of a **switch** is:

```
switch ( integral expression ) {  
    case integral constant 1 :  
        statement sequence 1  
        break;  
    case integral constant 2 :  
        statement sequence 2  
        break;  
    case integral constant 3 :  
        statement sequence 3  
        break;  
        ●  
        ●  
        ●  
    case integral constant n :  
        statement sequence n  
        break;  
    default:  
        default statement sequence  
}
```

In a **switch** statement

- The reserved word **switch** identifies a **switch** statement.
- The required parenthesized expression that follows the word **switch** must evaluate to an integral value. Any integer type, characters, and Boolean expressions are acceptable. Floating point expressions and other non-integer types are forbidden.

- The body of the **switch** is enclosed by required curly braces.
- Each occurrence of the word **case** is followed by an integral *constant* and a colon (:). We call the integral constant a **case label**. This label can be either a literal value or a **const** symbolic value (see Section 3.5). In particular, non-**const** variables and other expressions are expressly forbidden.

The case label defines a position within the code; it is not an executable statement. A case label represents a target to which the program's execution flow can jump.

If the **case** label matches the **switch**'s expression, then the statements that follow that label are executed up until the **break** statement is encountered. The statements and **break** statement that follow each **case** label are optional. One way to execute one set of statements for more than one **case** label is to provide empty statements for one or more of the labels, as in:

```
cin >> key;    // get key from user
switch (key) {
    case 'p':
    case 'P':
        cout << "You choose \"P\"\" << endl;
        break;
    case 'q':
    case 'Q':
        done = true;
        break;
}
```

Here either an upper- or lowercase *P* result in the same action— *You chose P* is printed. If the user enters either an upper- or lowercase *Q*, the **done** Boolean variable is set to true. If the user enters neither *P* nor *Q*, none of the statements in the **switch** is executed.

The **break** statement is optional. When a **case** label is matched, the statements that follow are executed until a **break** statement is encountered. The control flow then transfers out of the body of the **switch**. In this way, the **break** within a **switch** works just like a **break** within a loop: the rest of the body of the statement is skipped and program execution resumes at the next statement following the body. A missing **break** statement, a common error, when its omission is not intentional, causes the statements of the succeeding **case** label to be executed. The process continues until a **break** is encountered or the end of the **switch** body is reached.

- The **default** label is matched if none of the **case** labels match. It serves as a catch all option like the final **else** in a multi-way **if/else** statement. The **default** label is optional. If it is missing and none of the **case** labels match the expression, then no statement within the **switch**'s body is executed.

The **switch** statement has two restrictions that make it less general than the multi-way **if/else**:

- The **switch** argument must be an integral expression.
- Case labels must be constant integral values. Integral literals and constants are acceptable. Variables or expressions are **not** allowed.

To illustrate these restrictions, consider the following **if/else** statement that translates easily to an equivalent **switch** statement:

```
if (x == 1) {  
    // Do 1 stuff here . . .  
}  
else if (x == 2) {  
    // Do 2 stuff here . . .  
}  
else if (x == 3) {  
    // Do 3 stuff here . . .  
}
```

The corresponding **switch** statement is:

```
switch (x) {  
    case 1:  
        // Do 1 stuff here . . .  
        break;  
    case 2:  
        // Do 2 stuff here . . .  
        break;  
    case 3:  
        // Do 3 stuff here . . .  
        break;  
}
```

Now consider the following **if/else**:

```
if (x == y) {  
    // Do "y" stuff here . . .  
}  
else if (x > 2) {  
    // Do "> 2" stuff here . . .  
}  
else if (z == 3) {  
    // Do 3 stuff here . . .  
}
```

This code cannot be easily translated into a **switch** statement. The variable **y** cannot be used as a **case** label. The second choice checks for an inequality instead of an exact match, so direct translation to a **case** label is impossible. In the last condition, a different variable is checked, **z** instead of **x**. The control flow of a **switch** statement is determined by a single value (for example, the value of **x**), but a multi-way **if/else** statement is not so constrained.

Where applicable, a **switch** statement allows programmers to compactly express multi-way selection logic. Most programmers find a **switch** statement easier to read than an equivalent multi-way **if/else** construct.

A positive consequence of the **switch** statement's restrictions is that it allows the compiler to produce more efficient code for a **switch** than for an equivalent **if/else**. If a choice must be made from one of several or more options, and the **switch** statement can be used, then the **switch** statement will likely be faster than the corresponding multi-way **if/else**.

7.2 The Conditional Operator

As purely a syntactical convenience, C++ provides an alternative to the `if/else` construct called the *conditional operator*. It has limited application but is convenient nonetheless. The following code fragment assigns one of two things to `x`:

- the result of `y/z`, if `z` is nonzero, or
- zero, if `z` is zero; we wish to avoid the run-time error of division by zero.

```
// Assign a value to x:
if (z != 0)
    x = y/z; // Division is possible
else
    x = 0;    // Assign a default value instead
```

This code has two assignment statements, but only one is executed at any given time. The conditional operator makes for a more compact statement:

```
// Assign a value to x:
x = (z != 0) ? y/z : 0;
```

The general form of a conditional expression is:

(condition) ? expression 1 : expression 2

- *condition* is a normal Boolean expression that might appear in an `if` statement. Parentheses around the condition are not required but should be used to improve the readability.
- *expression 1* is the overall value of the conditional expression if *condition* is true.
- *expression 2* is the overall value of the conditional expression if *condition* is false.

The conditional operator uses two symbols (`?` and `:`) and three operands. Since it has three operands it is classified as a *ternary* operator (C++'s only one). The overall type of a conditional expression is the more dominant of *exp₁* and *exp₂*. The conditional expression can be used anywhere an expression can be used. It is not a statement itself; it is used within a statement.

As another example, the *absolute value* of a number is defined in mathematics by the following formula:

$$|n| = \begin{cases} n, & \text{when } n \geq 0 \\ -n, & \text{when } n < 0 \end{cases}$$

In other words, the absolute value of a positive number or zero is the same as that number; the absolute value of a negative number is the additive inverse (negative of) of that number. The following C++ expression represents the *absolute value* of the variable `n`:

```
(n < 0) ? -n : n
```

Some argue that the conditional operator is cryptic, and thus its use reduces a program's readability. To seasoned C++ programmers it is quite understandable, but it is used sparingly because of its very specific nature.

7.3 The do/while Statement

An executing program checks the condition of a **while** statement (Section 6.1) before executing any of the statements in its body; thus, we say a **while** loop is a *top-checking* loop. Sometimes this sequence of checking then executing is inconvenient; for example, consider

The **while** statement (Section 6.1) checks its condition before its body is executed; thus, it is a *top-checking* loop. Sometimes this sequence of checking the condition first then executing the body is inconvenient; for example, consider Listing 7.2 (goodinputonly.cpp).

Listing 7.2: goodinputonly.cpp

```
#include <iostream>

using namespace std;

int main() {
    int in_value = -1;
    cout << "Please enter an integer in the range 0-10: ";
    // Insist on values in the range 0...10
    while (in_value < 0 || in_value > 10)
        cin >> in_value;
    // in_value at this point is guaranteed to be within range
    cout << "Legal value entered was " << in_value << endl;
}
```

The loop in Listing 7.2 (goodinputonly.cpp) traps the user in the **while** until the user provides a number in the desired range. Here's how it works:

- The condition of the **while** specifies a set that includes all values that are *not* in the desired range. The initialization of **in_value** to `-1` ensures the condition of the **while** will be true initially, and, thus, the program always will execute the loop's body at least one time.
- The user does not get a chance to enter a value until program's execution is inside the loop.
- The only way the user can escape the loop is to enter a value that violates the condition—precisely a value in the desired range.

The initialization of **in_value** before the loop check is somewhat artificial. It is there only to ensure entry into the loop's body. It seems unnatural to check for a valid value *before* the user gets a chance to enter it. A loop that checks its condition after its body is executed at least once would be more appropriate. The **do/while** statement is a *bottom-checking* loop that behaves exactly in this manner. Listing 7.3 (betterinputonly.cpp) uses a **do/while** statement to check for valid input.

Listing 7.3: betterinputonly.cpp

```
#include <iostream>

using namespace std;

int main() {
    int in_value;
    cout << "Please enter an integer in the range 0-10: ";
    // Insist on values in the range 0...10
    do
        cin >> in_value;
    while (in_value < 0 || in_value > 10);
    // in_value at this point is guaranteed to be within range
    cout << "Legal value entered was " << in_value << endl;
}
```

Notice that there is no need to initialize `in_value` since its value is not used until after it is assigned through the input stream `cin`. Figure 7.1 compares the flowcharts of a `while` and `do/while` loop.

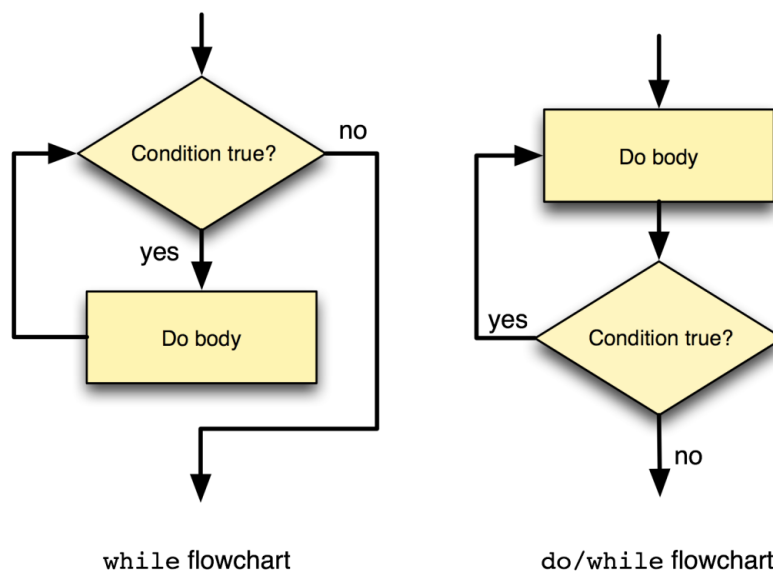


Figure 7.1: The flowcharts for while and do/while loops

The `do/while` statement has the general form:

```
do
    statement
while ( condition );
```

- The reserved words **do** and **while** identify a **do/while** statement. The **do** and **while** keywords delimit the loop's body, but curly braces are still required if the body consists of more than one statement.
- The *condition* is associated with the **while** at the end of the loop. The *condition* is a Boolean expression and must be enclosed within parentheses.
- The *statement* is exactly like the *statement* in the general form of the **while** loop (see Section 6.1). It can be a compound statement enclosed within curly braces.

The body of a **do/while** statement, unlike the **while** statement, is guaranteed to execute at least once.

The **do/while** loop is a convenience to the programmer and is not an essential programming construct. It is easy to transform any code that uses a **do/while** statement into code that behaves identically that uses a **while** statement instead. In practice, programmers use **while** loops much more frequently than **do/while** loops because more algorithms require top-checking loops than bottom-checking loops. The **do/while** statement is included in C++ for a reason, however. Transforming an algorithm that can be expressed more naturally with a bottom-checking loop into one that uses a top-checking loop can lead to awkward code. Use **do/while** when appropriate.

7.4 The for Statement

Recall Listing 6.2 (`iterativecounttofive.cpp`). It simply counts from one to five. Counting is a frequent activity performed by computer programs. Certain program elements are required in order for any program to count:

- A variable must be used to keep track of the count; in Listing 6.2 (`iterativecounttofive.cpp`), **count** is the aptly named counter variable.
- The counter variable must be given an initial value. In the case of Listing 6.2 (`iterativecounttofive.cpp`), the initial value is 1.
- The variable must be modified (usually incremented) as the program counts. The statement

```
count++;
```

increments **count** in Listing 6.2 (`iterativecounttofive.cpp`).

- A way must be provided to determine if the counting has completed. In Listing 6.2 (`iterativecounttofive.cpp`), the condition of the **while** statement determines if the counting is complete or must continue.

C++ provides a specialized loop that packages these four programming elements into one convenient statement. Called the **for** statement, its general form is

```
for ( initialization ; condition ; modification )  
    statement
```

- The reserved word **for** identifies a **for** statement.
- The loop is controlled by a special variable called the *loop variable*.
- The header, contained in parentheses, contains three parts, each separated by semicolons:
 - **Initialization.** The *initialization* part assigns an initial value to the loop variable. The loop variable may be declared here as well; if it is declared here, then its scope is limited to the **for** statement. This means you may use that loop variable only within the loop. It also means you are free to reuse that variable's name outside the loop to declare a different variable with the same name as the loop variable.

The initialization part is performed one time.
 - **Condition.** The *condition* part is a Boolean expression, just like the condition of a **while** statement. The condition is checked each time *before* the body is executed.
 - **Modification.** The *modification* part generally changes the loop variable. The change should be such that the condition will eventually become false so the loop will terminate. The modification is performed during each iteration *after* the body is executed.

Notice that the last part (*modification*) is not following by a semicolon; semicolons are used strictly to separate the three parts.
- The *statement* is like the body of any other loop. It may be a compound statement within curly braces.

Any **for** loop can be rewritten as a **while** loop. The general form of the **for** loop given above can be written equivalently as

```

        initialization
    while ( condition ) {
        statement
        modification
    }

```

Listing 7.4 (forcounttofive.cpp) uses a **for** statement to count to five.

Listing 7.4: forcounttofive.cpp

```

#include <iostream>

using namespace std;

int main() {
    for (int count = 1; count <= 5; count++)
        cout << count << endl;    // Display counter
}

```

With a **while** loop, the four counting components (variable declaration, initialization, condition, and modification) can be scattered throughout the code. With a **for** loop, a programmer should be able to determine all the important information about the loop's control by looking at one statement.

Recall Listing 6.12 (timestable.cpp) that prints a multiplication table on the screen. We can organize its code better by converting all the **while** statements to **for** statements. The result uses far less code, as shown in Listing 7.5 (bettertimestable.cpp).

Listing 7.5: bettertimestable.cpp

```

#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int size;    // The number of rows and columns in the table
    cout << "Please enter the table size: ";
    cin >> size;
    // Print a size x size multiplication table

    // First, print heading
    cout << "          ";
    for (int column = 1; column <= size; column++)

```



```

        cout << setw(4) << column; // Print heading for this column.
    cout << endl;
    // Print line separator
    cout << "    +";
    for (int column = 1; column <= size; column++)
        cout << "----"; // Print separator for this column.
    cout << endl;
    // Print table contents
    for (int row = 1; row <= size; row++) {
        cout << setw(4) << row << " |"; // Print row label.
        for (int column = 1; column <= size; column++)
            cout << setw(4) << row*column; // Display product
        cout << endl; // Move cursor to next row
    }
}

```

A **for** loop is ideal for stepping through the rows and columns. The information about the control of both loops is now packaged in the respective **for** statements instead of being spread out in various places in **main**. In the **while** version, it is easy for the programmer to forget to update one or both of the counter variables (**row** and/or **column**). The **for** makes it harder for the programmer to forget the loop variable update, since it is done right up front in the **for** statement header.

It is considered bad programming practice to do either of the following in a **for** statement:

- **Modify the loop control variable within the body of the loop**—if the loop variable is modified within the body, then the logic of the loop's control is no longer completely isolated to the **for** statement's header. The programmer must look elsewhere within the statement to understand completely how the loop works.
- **Prematurely exit the loop with a **break****—this action also violates the concept of keeping all the loop control logic in one place (the **for**'s header).

The language allows both of these practices, but experience shows that it is best to avoid them. If it seems necessary to violate this advice, consider using a different kind of loop. The **while** and **do/while** loops do not imply the same degree of control regularity expected in a **for** loop.

Listing 7.6 (permuteabcd.cpp) is a rewrite of Listing 6.13 (permuteabc.cpp) that replaces its **while** loops with **for** loops and adds an additional character.

Listing 7.6: permuteabcd.cpp

```

// File permuteabcd.cpp

#include <iostream>

using namespace std;

int main() {
    for (char first = 'A'; first <= 'D'; first++)
        for (char second = 'A'; second <= 'D'; second++)
            if (second != first) // No duplicate letters
                for (char third = 'A'; third <= 'D'; third++)
                    if (third != first && third != second)
                        for (char fourth = 'A'; fourth <= 'D'; fourth++)
                            if (fourth != first && fourth != second && fourth != third)

```

```
        cout << first << second << third << fourth << endl;
    }
```

Notice that since all the variable initialization and incrementing is taken care of in the **for** statement headers, we no longer need compound statements in the loop bodies, so the curly braces are unnecessary. Listing 7.6 (permuteabcd.cpp) prints all 24 permutations of **ABCD**:

```
ABCD
ABDC
ACBD
ACDB
ADBC
ADCB
BACD
BADC
BCAD
BCDA
BDAC
BDCA
CABD
CADB
CBAD
CBDA
CDAB
CDBA
DABC
DACB
DBAC
DBCA
DCAB
DCBA
```

Listing 7.7 (forprintprimes.cpp) is a rewrite of Listing 6.21 (printprimes.cpp) that replaces its **while** loops with **for** loops.

Listing 7.7: forprintprimes.cpp

```
#include <iostream>

using namespace std;

int main() {

    int max_value;
    cout << "Display primes up to what value? ";
    cin >> max_value;
    for (int value = 2; value <= max_value; value++) {
        // See if value is prime
        bool is_prime = true; // Provisionally, value is prime
        // Try all possible factors from 2 to value - 1
```

```

    for (int trial_factor = 2;
         is_prime && trial_factor < value;
         trial_factor++)
        is_prime = (value % trial_factor != 0);
    if (is_prime)
        cout << value << " ";    // Display the prime number
}
cout << endl;    // Move cursor down to next line
}

```

As shown in Listing 7.7 (forprintprimes.cpp), the conditional expression in the **for** loop is not limited to a simple test of the loop control variable; it can be any legal Boolean expression. Programmers can use logical the *and* (**&&**), *or* (**|**), and *not* (**!**) operators to create complex Boolean expressions, if necessary. The modification part of the **for** loop is not limited to simple arithmetic and can be quite elaborate. For example:

```

for (double d = 1000; d >= 1; cin >> d) {
    /* Body goes here */
}

```

Here **d** is reassigned from the input stream. If necessary, multiple variables can be initialized in the initialization part:

```

for (int i = 0, j = 100; i < j; i++) {
    /* Body goes here */
}

```

While the **for** statement supports such complex headers, simpler is usually better. Ordinarily the **for** loop should manage just one control variable, and the initialization, condition, and modification parts should be straightforward. If a particular programming situation warrants an overly complicated **for** construction, consider using another kind of loop.

Any or all of the parts of the **for** statement (initialization, condition, modification, and body) may be omitted:

- **Initialization.** If the initialization is missing, as in

```

for (; i < 10; i++)
    /* Body goes here */

```

then no initialization is performed by the **for** loop, and it must be done elsewhere.

- **Condition.** If the condition is missing, as in

```

for (int i = 0; ; i++)
    /* Body goes here */

```

then the condition is true by default. A **break** or **goto** must appear in the body unless an infinite loop is intended.

- **Modification.** If the modification is missing, as in

```

for (int i = 0; i < 10; )
    /* Body goes here */

```

then the **for** performs no automatic modification; the modification must be done by a statement in the body to avoid an infinite loop.

- **Body.** An empty body, as in

```
for (int i = 0; i < 10; i++) {}
```

or

```
for (int i = 0; i < 10; i++);
```

results in an empty loop. Some programmers use an empty loop to produce a non-portable delay in the program's execution. A programmer may, for example, need to slow down a graphical animation. Such an attempt using an empty loop is non-portable for several reasons. If the program actually executes the loop, slower computers will delay longer than faster computers. The timing of the program's delay will differ from one computer to another. Worse yet, some compilers may detect that such code has no functional effect and optimize away the empty loop. This means the compiler will ignore the **for** statement altogether.



As mentioned in Section 5.3, be careful about accidentally putting a semicolon at the end of the **for** header, as in

```
for (int i = 0; i < 10; i++);  
    /* Intended body goes here */
```

The semicolon terminates the **for** statement, and the intended body that follows is not the body, even though it may be properly indented.

One common C/C++ idiom to make an intentional infinite loop is to use a **for** statement with all control information missing:

```
for ( ;; )  
    /* Body goes here */
```

Omitting all the parts of the **for** header is a statement from the programmer that says “I know what I am doing—I really want an infinite loop here.” In reality the loop may not be infinite at all; its body could contain a **break** or **goto** statement.

While the **for** statement supports the omission of parts of its header, such constructs should be avoided. The intention of the **for** loop is to allow the programmer to see all the aspects of the loop's control in one place. If some of these control responsibilities are to be handled elsewhere (not in the **for**'s header) then consider using another kind of loop.

Programmers usually select a simple name for the control variable of a **for** statement. Recall that variable names should be well chosen to reflect the meaning of their use within the program. It may come as a surprise that **i** is probably the most common name used for an integer control variable in a **for** loop. This practice has its roots in mathematics where variables such as *i*, *j*, and *k* are commonly used to index vectors and matrices. Such mathematical structures have programming analogs in arrays and **vectors**, which we explore in Chapter 11. Computer programmers make considerable use of **for** loops in array and vector processing, so programmers have universally adopted this convention of short control variable names. Thus, it generally is acceptable to use simple identifiers like **i** as loop control variables.

C++ allows the **break**, **continue**, and **goto** statements to be used in the body of a **for** statement. Like with the **while** and **do/while** statements, **break** causes immediate loop termination, **continue**

causes the condition to be immediately checked to determine if the iteration should continue, and `goto` jumps to a label somewhere in the function. As previously mentioned, however, `for` loop control should be restricted to its header, and the use of `break`, `continue`, and `goto` within `for` loops should be avoided.

Any `for` loop can be rewritten with a `while` loop and behave identically. For example, consider the `for` loop

```
for (int i = 1; i <= 10; i++)
    cout << i << endl;
```

and next consider the `while` loop that behaves exactly the same way:

```
int i = 1;
while (i <= 10) {
    cout << i << endl;
    i++;
}
```

Which is better? The `for` loop conveniently packages the loop control information in its header, but in the `while` loop this information is distributed throughout the small section of code. The `for` loop thus provides a better organization of the loop control code. Does one loop outperform the other? No, most compilers produce essentially the same code for both constructs. Thus, the `for` loop is preferred in this example.

7.5 Summary

- The `switch` statement is an alternative to some multi-way `if/else` statements.
- Not all multi-way `if/else` statements can be converted directly to a `switch` statement.
- The expression to evaluate within the `switch` statement must evaluate to an integral value (`int`, `short`, `long`, `char`, or `bool`).
- The `case` labels within a `switch` statement must be integral **literals** or **constants** (`int`, `short`, `long`, `char`, or `bool`). Specifically, `case` labels may **not** be variables or other expressions.
- Program execution jumps to the `case` label with the value that matches the `switch` expression.
- Once a `case` label is matched, program execution continues within the `switch` statement until a `break` statement is encountered.
- If no `case` labels match the `switch` expression, the program execution jumps to the `default` label, if it is present. If no `case` labels match and no `default` label is present, no part of the `switch` body is executed.
- The conditional operator is an expression that evaluates to one of two values depending on a given condition.
- The `do/while` is a bottom-checking loop, unlike the `while` loop, which is a top-checking loop.
- The body of a `do/while` loop is always executed at least once, regardless of the condition. The body of a `while` loop is not executed if the condition is initially `false`.

- The **for** loop is a top-checking loop that, when used properly, concentrates all the information about its control in one convenient location.
- The three parts of the **for** loop control are initialization, condition, and modification.
- The **for** statement is best used for a loop that can be controlled by a single variable with a definite starting value, a definite ending value, and a regular way to update the variable's value.
- A **for** loop is ideal for counting.
- Any or all of the three parts in the **for** loop header can be omitted; however, if you feel the need to omit one or more of the parts, the **while** statement may be a better choice.
- Best practice avoids modifying the control variable of a **for** statement within the loop's body; the modification should be limited to the third part of the **for** header.
- The `<iomanip>` library provides the **setw** object that provides special formatting for output sent to **cout**.

7.6 Exercises

1. Consider the following code fragment.

```
int x;
cin >> x;
switch (x + 3) {
    case 5:
        cout << x << endl;
        break;
    case 10:
        cout << x - 3 << endl;
        break;
    case 20:
        cout << x + 3 << endl;
        break;
}
```

- (a) What is printed when the user enters 2?
- (b) What is printed when the user enters 5?
- (c) What is printed when the user enters 7?
- (d) What is printed when the user enters 17?
- (e) What is printed when the user enters 20?
- (f) What is printed when the user enters 5?

2. Consider the following code fragment.

```
char ch;
cin >> ch;
switch (ch) {
    case 'a':
```

```

        cout << "*" << endl;
        break;
    case 'A':
        cout << "***" << endl;
        break;
    case 'B':
    case 'b':
        cout << "****" << endl;
    case 'C':
    case 'c':
        cout << "*****" << endl;
        break;
    default:
        cout << "*****" << endl;
}

```

- (a) What is printed when the user enters *a*?
 - (b) What is printed when the user enters *A*?
 - (c) What is printed when the user enters *b*?
 - (d) What is printed when the user enters *B*?
 - (e) What is printed when the user enters *C*?
 - (f) What is printed when the user enters *c*?
 - (g) What is printed when the user enters *t*?
3. What is printed by the following code fragment?

```

int x = 0;
do {
    cout << x << " ";
    x++;
} while (x < 10);
cout << endl;

```

4. What is printed by the following code fragment?

```

int x = 20;
do {
    cout << x << " ";
    x++;
} while (x < 10);
cout << endl;

```

5. What is printed by the following code fragment?

```

for (int x = 0; x < 10; x++)
    cout << "*";
cout << endl;

```

6. Rewrite the following code fragment so that a **switch** is used instead of the **if/else** statements.

```
int value;
char ch;
cin >> ch;
if (ch == 'A')
    value = 10;
else if (ch == 'P')
    value = 20;
else if (ch == 'T')
    value = 30;
else if (ch == 'V')
    value = 40;
else
    value = 50;
cout << value << endl;
```

7. Rewrite the following code fragment so that a multi-way **if/else** is used instead of the **switch** statement.

```
int value;
char ch;
cin >> ch;
switch( ch) {
case 'A':
    value = 10;
    break;
case 'P':
    cin >> value;
    break;
case 'T':
    value = ch;
    break;
case 'V':
    value = ch + 1000;
    break;
default:
    value = 50;
}
cout << value << endl;
```

8. Rewrite the following code fragment so that a multi-way **if/else** is used instead of the **switch** statement.

```
int value;
char ch;
cin >> ch;
switch (ch) {
case 'A':
    cout << ch << endl;
    value = 10;
    break;
case 'P':
```



```

case 'E':
    cin >> value;
    break;
case 'T':
    cin >> ch;
    value = ch;
case 'C':
    value = ch;
    cout << "value=" << value << ", ch=" << ch << endl;
    break;
case 'V':
    value = ch + 1000;
    break;
}
cout << value << endl;

```

9. Rewrite the following code fragment so a **while** loop is used instead of the **for** statement.

```

for (int i = 100; i > 0; i--)
    cout << i << endl;

```

10. Rewrite the following code fragment so that it uses the conditional operator instead of an **if** statement:

```

if (value % 2 != 0)    // Is value even?
    value = value + 1; // If not, make it even.

```

11. Rewrite the following code fragment so that it uses the conditional operator instead of an **if/else** statement:

```

if (value % 2 == 0)    // Is value even?
    value = 0;        // If so, make it zero.
else
    value = value + 1; // Otherwise, make it even.

```

12. Would the following multi-way **if/else** be a good candidate to rewrite as a **switch** statement? If so, rewrite the code using a **switch**; otherwise, explain why it is impractical to do so.

```

int x, y;
cin >> x >> y;
if (x < 10)
    y = 10;
else if (x == 5)
    y = 5;
else if (x == y)
    y = 0;
else if (y > 10)
    x = 10;
else
    x = y;

```

Chapter 8

Using Functions

Suppose you must write a C++ program that computes the square root of a number supplied by the user. Listing 8.1 (computesquareroot.cpp) provides a simple implementation.

Listing 8.1: computesquareroot.cpp

```
// File squareroot.cpp

#include <iostream>

using namespace std;

int main() {
    double input;

    // Get value from the user
    cout << "Enter number: ";
    cin >> input;
    double diff;
    // Compute a provisional square root
    double root = 1.0;

    do { // Loop until the provisional root
        // is close enough to the actual root
        root = (root + input/root) / 2.0;
        cout << "root is " << root << endl;
        // How bad is the approximation?
        diff = root * root - input;
    }
    while (diff > 0.0001 || diff < -0.0001);

    // Report approximate square root
    cout << "Square root of " << input << " = " << root << endl;
}
```

The program is based on a simple algorithm, Newton's Method, that uses successive approximations to zero in on an answer that is within 0.0001 of the true answer.

One sample run is

```
Enter number: 2
root is 1.5
root is 1.41667
root is 1.41422
Square root of 2 = 1.41422
```

The actual square root is approximately 1.4142135623730951 and so the result is within our accepted tolerance (0.0001). Another run is

```
Enter number: 100
root is 50.5
root is 26.2401
root is 15.0255
root is 10.8404
root is 10.0326
root is 10.0001
root is 10
Square root of 100 = 10
```

which is, of course, the exact answer.

While this code may be acceptable for many applications, better algorithms exist that work faster and produce more precise answers. Another problem with the code is this: What if you are working on a significant scientific or engineering application and must use different formulas in various parts of the source code, and each of these formulas involve square roots in some way? In mathematics, for example, you use square root to compute the distance between two geometric points (x_1, y_1) and (x_2, y_2) as

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

and, using the quadratic formula, the solution to the equation $ax^2 + bx + c = 0$ is

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

In electrical engineering and physics, the root mean square of a set of values $\{a_1, a_2, a_3, \dots, a_n\}$ is

$$\sqrt{\frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{n}}$$

Suppose we are writing one big program that, among many other things, needs to compute distances and solve quadratic equations. Must we copy and paste the relevant portions of the square root code in Listing 8.1 (computesquareroot.cpp) to each location in our source code that requires a square root computation? Also, what if we develop another program that requires computing a root mean square? Will we need to copy the code from Listing 8.1 (computesquareroot.cpp) into every program that needs to compute square roots, or is there a better way to package the square root code and reuse it?

Code is made reusable by packaging it in *functions*. A function is a unit of reusable code. In Chapter 9 we will write our own reusable functions, but in this chapter we examine some of the functions available in the C++ standard library. C++ provides a collection of standard precompiled C and C++ code stored in libraries. Programmers can use parts of this library code within their own code to build sophisticated programs.

8.1 Introduction to Using Functions

In mathematics, a *function* computes a result from a given value; for example, from the function definition $f(x) = 2x + 3$, we can compute $f(5) = 13$ and $f(0) = 3$. A function in C++ works like a mathematical function. To introduce the function concept, we will look at the standard C++ function that implements mathematical square root.

In C++, a function is a named sequence of code that performs a specific task. A program itself consists of a collection of functions. One example of a function is the mathematical square root function. Such a function, named **sqrt**, is available to C and C++ programs (see Section 8.2). The square root function accepts one numeric value and produces a **double** value as a result; for example, the square root of 16 is 4, so when presented with 16.0, **sqrt** responds with 4.0. Figure 8.1 visualizes the square root function. For



Figure 8.1: Conceptual view of the square root function

the programmer using the **sqrt** function within a program, the function is a black box; the programmer is concerned more about *what* the function does, not *how* it does it.

This **sqrt** function is exactly what we need for our square root program, Listing 8.1 (computesquareroot.cpp). The new version, Listing 8.2 (standardsquareroot.cpp), uses the library function **sqrt** and eliminates the complex logic of the original code.

Listing 8.2: standardsquareroot.cpp

```
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    double input;

    // Get value from the user
    cout << "Enter number: ";
    cin >> input;
```

```
// Compute the square root
double root = sqrt(input);

// Report result
cout << "Square root of " << input << " = " << root << endl;
}
```

The line

```
#include <cmath>
```

directs the preprocessor to augment our source code with the declarations of a collection of mathematical functions in the `cmath` library. The `sqrt` function is among them. Table 8.1 lists some of the other commonly used mathematical functions available in the `cmath` library. The compiler needs this augmented code so it can check to see if we are using the `sqrt` function properly.

The expression

```
sqrt(input)
```

is a *function invocation*, also known as a *function call*. A function provides a service to the code that uses it. Here, our `main` function is the *caller*¹ that uses the service provided by the `sqrt` function. We say `main` calls, or invokes, `sqrt` passing it the value of `input`. The expression `sqrt(input)` evaluates to the square root of the value of the variable `input`. Behind the scenes—inside the black box as it were—precompiled C code uses the value of the `input` variable to compute its square root. There is nothing special about this precompiled C code that constitutes the `sqrt` function; it was written by a programmer or team of programmers working for the library vendor using the same tools we have at our disposal. In Chapter 9 we will write our own functions, but for now we will enjoy the functions that others have provided for us.

When calling a function, a pair of parentheses follow the function’s name. Information that the function requires to perform its task must appear within these parentheses. In the expression

```
sqrt(input)
```

`input` is the information the function needs to do its work. We say `input` is the *argument*, or *parameter*, passed to the function. We also can say “we are passing `input` to the `sqrt` function.”

While we might say “we are passing `input` to the `sqrt` function,” the program really is not giving the function access to `main`’s `input` variable. The `sqrt` function itself cannot change the value of `main`’s `input` variable, it simply uses the variable’s value to perform the computation.

The following simple analogy may help explain how the communication works between `main` and `sqrt`. The `main` function has work to do, but instead of doing all the work itself, it delegates some of the work (in this case the hard part) to `sqrt`. When `main` needs to compute the square root of `input`, it writes down the value of its `input` variable on a piece of paper and hands it to `sqrt`. `main` then sits idly until `sqrt` finishes its work. The `sqrt` function accepts `main`’s note and begins working on the task (computing the square root of the number on the note `main` gave it). When it is finished, `sqrt` does two things: `sqrt` hands back to `main` a different piece of paper with the answer, and `sqrt` throws away the piece of paper `main` originally passed to it. When `main` receives the note from `sqrt` it uses the information on the note and then discards the note. The `main` function then can continue with its other business.

¹The term *client* can be used as well, although we reserve the term *client* for code that interacts with objects (see Chapter 13).

The `sqrt` function thus has no access to `main`'s original `input` variable; it has only a copy of `input`, as if “written on a piece of paper.” (Similarly, if the `sqrt` function uses any variables to do its work, `main` is oblivious to them and has no way to access them.) After `sqrt` is finished and returns to `main` its computed answer, `sqrt` discards its copy of `input` (by analogy, the function “throws away” the paper with the copy of `input` that `main` gave it). Thus, during a function call the parameter is a temporary, transitory value used only to communicate information to the function. The parameter lives only as long as the function is executing.

Figure 8.2 illustrates a program's execution involving simple function calls. Figure 8.2 shows that a

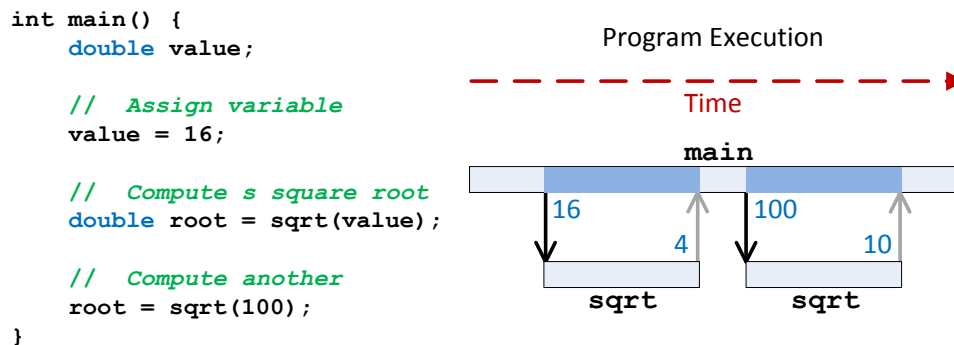


Figure 8.2: The diagram on the right visualizes the execution of the program on the left. Time flows from left to right. A rectangular bar represents the time that a function is active. A C++ program's execution begins with its `main` function. Here, `main` calls the `sqrt` function twice. The shaded parts of `main`'s bar shows the times `main` has to wait for `sqrt` to complete.

program's execution begins in its `main` function. Here `main` calls the `sqrt` function twice. A vertical bar represents the time that a function is *active*, or “alive.” A function's variables exist while a function is active. Observe that the `main` function is active for the duration of the program's execution. The `sqrt` becomes active twice, exactly the two times `main` calls it.

The `sqrt` function can be called in other ways, as Listing 8.3 (`usingsqrt.cpp`) illustrates:

Listing 8.3: `usingsqrt.cpp`

```

/*
 * This program shows the various ways the
 * sqrt function can be used.
 */

#include <iostream>
#include <cmath>

using namespace std;

int main() {
    double x = 16.0;
    // Pass a literal value and display the result
    cout << sqrt(16.0) << endl;
}

```

```

// Pass a variable and display the result
cout << sqrt(x) << endl;
// Pass an expression
cout << sqrt(2 * x - 5) << endl;
// Assign result to variable
double y = sqrt(x);
// Use result in an expression
y = 2 * sqrt(x + 16) - 4;
// Use result as argument to a function call
y = sqrt(sqrt(256.0));
cout << y << endl;
}

```

The **sqrt** function accepts a single numeric argument. The parameter that a caller can pass to **sqrt** can be a literal number, a numeric variable, an arithmetic expression, or even a function invocation that produces an acceptable numeric result.

Some C++ functions, like **sqrt**, compute a value and return it to the caller. The caller can use this result in various ways, as shown in Listing 8.3 (usingsqrt.cpp). The next to the last statement passes the result of calling **sqrt** to **sqrt**, thereby computing $\sqrt{\sqrt{256}}$, which is 4.

If the caller code attempts to pass a parameter to the function that is incompatible with the type expected by the function, the compiler will issue an error.

```
cout << sqrt("16") << endl; // Illegal, a string is not a number
```

The compiler is able to determine that the above statement is illegal based on the additional information the preprocessor added to the code via the

```
#include<cmath>
```

directive.

Listing 8.3 (usingsqrt.cpp) shows that a program can call the **sqrt** function as many times and in as many places as needed. As noted in Figure 8.1, to the caller of the square root function, the function is a black box; the caller is concerned strictly about *what* the function does, not *how* the function accomplishes its task.

We safely can treat all functions as black boxes. We can use the service that a function provides without being concerned about its internal details. We are guaranteed that we can influence the function's behavior only via the parameters that we pass, and that nothing else we do can affect what the function does or how it does it. Furthermore, the function cannot affect any of our code, apart from what we do with the value it computes.

Some functions take more than one parameter; for example, the C++ **max** function requires two arguments in order to produce a result. The **max** function selects and returns the larger of the two parameters. The **max** function is visualized in Figure 8.3. The **max** function could be used as

```
cout << "The larger of " << 4 << " and " << 7
      << " is " << max(4, 7) << endl;
```

Notice that the parameters are contained in parentheses following the function's name, and the parameters are separated by commas.

From the caller's perspective a function has three important parts:



Figure 8.3: Conceptual view of the maximum function

- **Name.** Every function has a name that identifies the location of the code to be executed. Function names follow the same rules as variable names; a function name is another example of an identifier (see Section 3.3).
- **Parameter type(s).** A caller must provide the exact number and types of parameters that that a function expects. If a caller attempts to call a function with too many or too few parameters, the compiler will issue an error message and not compile the code. Similarly, if the caller passes parameters that are not compatible with the types specified for the function, the compiler will report appropriate error messages.
- **Result type.** A function can compute a result and return this value to the caller. The caller's use of this result must be compatible with the function's specified result type. The result type returned to the caller and the parameter types passed in by the caller can be completely unrelated.

These three crucial pieces of information are formally described for each function in a specification known as a function *prototype*. The prototype for the `sqrt` function is

```
double sqrt(double)
```

and the `max` function's prototype can be expressed as ²

```
int max(int, int)
```

In a function prototype, the return type is listed first, followed by the function's name, and then the parameter types appear in parentheses. Sometimes it is useful to list parameter names in the function's prototype, as in

```
double sqrt(double n)
```

or

```
int max(int a, int b)
```

The specific parameter names are irrelevant. The names make it easier to describe what the function does; for example, `sqrt` computes the square root of `n` and `max` determines the larger of `a` and `b`.

When using a library function the programmer must include the appropriate `#include` directive in the source code. The file specified in an `#include` directive contains prototypes for library functions. In order to use the `sqrt` function, a program must include the

² The prototype for the actual library function `max` uses *generic types*; generic types are beyond the scope of this introductory book, so the prototype provided here is strictly for illustrating a point.


```
#include <cmath>
```

preprocessor directive. For the **max** function, include

```
#include <algorithm>
```

although under Visual C++, **#include**ing **<iostream>** is sufficient.

Armed with the function prototypes, the compiler can check to see if the calling code is using the library functions correctly. An attempt to use the **sqrt** function as

```
cout << sqrt(4.0, 7.0) << endl; // Error
```

will result in an error because the prototype for **sqrt** specifies only one numeric parameter, not two.

Some functions do not accept parameters; for example, the C++ function to generate a pseudorandom number, **rand**, is called with no arguments:

```
cout << rand() << endl;
```

The **rand** function returns an **int** value, but the caller does not pass the function any information to do its task. The **rand** prototype is

```
int rand()
```

Notice the empty parentheses that indicate this function does not accept any parameters.

Unlike mathematical functions that must produce a result, C++ does not require a function to return a value to its caller. The C++ function **exit** expects an integer value from the caller, but it does not return a result back to the caller. A prototype for a function that returns nothing uses **void** as the return type, as in:

```
void exit(int);
```

The **exit** function immediately terminates the program's execution. The integer argument passed to **exit** is returned to the operating system which can use the value to determine if the program terminated normally or due to an error. C++ programs automatically return zero when **main** finishes executing—no **exit** call is necessary.

Note that since **exit** does not return a value to the caller, code such as

```
cout << exit(8) << endl; // Illegal!
```

will not compile since the expression **exit(8)** evaluates to nothing, and the **cout** stream object requires an actual value of some kind to print. A **void** function is useful for the *side effects* it produces instead a value it computes. Example side effects include printing something on the console, sending data over a network, or animating a graphical image.

8.2 Standard Math Functions

The **cmath** library provides much of the functionality of a scientific calculator. Table 8.1 lists only a few of the available functions.

mathfunctions Module	
double sqrt(double x)	Computes the square root of a number: $\text{sqrt}(x) = \sqrt{x}$
double exp(double x)	Computes e raised a power: $\text{exp}(x) = e^x$
double log(double x)	Computes the natural logarithm of a number: $\log(x) = \log_e x = \ln x$
double log10(double x)	Computes the common logarithm of a number: $\log(x) = \log_{10} x$
double cos(double)	Computes the cosine of a value specified in radians: $\cos(x) = \cos x$; other trigonometric functions include sine, tangent, arc cosine, arc sine, arc tangent, hyperbolic cosine, hyperbolic sine, and hyperbolic tangent
double pow(double x, double y)	Raises one number to a power of another: $\text{pow}(x, y) = x^y$
double fabs(double x)	Computes the absolute value of a number: $\text{fabs}(x) = x $

Table 8.1: A few of the functions from the `cmath` library

The **cmath** library also defines a constant named **HUGE_VAL**. Programmers can use this constant to represent infinity or an undefined value such the slope of a vertical line or a fraction with a zero denominator. A complete list of the numeric functions available to C++ can be found at <http://www.cplusplus.com/reference/clibrary/cmath/>.



Be careful to put the function's arguments in the proper order when calling a function; for example, the call **pow(10, 2)** computes $10^2 = 100$, but the call **pow(2, 10)** computes $2^{10} = 1,024$.

A C++ program that uses any of the functions from the **cmath** library must use the following preprocessor **#include** directive:

```
#include <cmath>
```

Functions in the **cmath** library are ideal for solving problems like the one shown in Figure 8.4. Suppose a spacecraft is at a fixed location in space relative to some planet. The spacecraft's distance to the planet, therefore, also is fixed. A satellite is orbiting the planet in a circular orbit. We wish to compute how much farther away the satellite will be from the spacecraft when it has progressed 10 degrees along its orbital path.

We will let the origin of our coordinate system (0,0) be located at the center of the planet which corresponds also to the center of the circular orbital path. The satellite is initially at point (x_1, y_1) and the spacecraft is stationary at point (p_x, p_y) . The spacecraft is located in the same plane as the satellite's orbit. We need to compute the difference in the distances between the moving point (satellite) and the fixed point (spacecraft) at two different times during the satellite's orbit.

Facts from mathematics provide solutions to the following two problems:

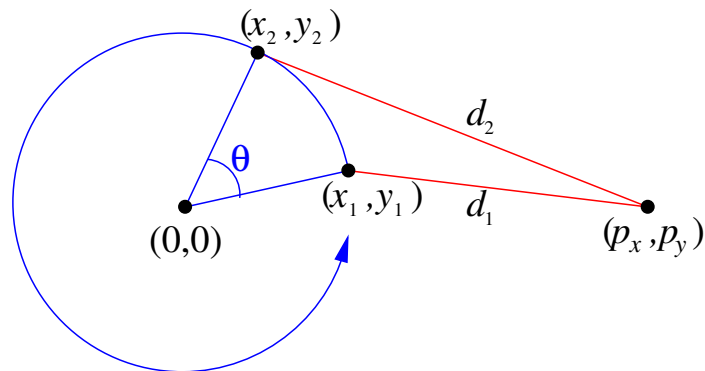


Figure 8.4: Orbital distance problem

1. **Problem:** We must recompute the location of the moving point as it moves along the circle.

Solution: Given an initial position (x_1, y_1) of the moving point, a rotation of θ degrees around the origin will yield a new point at (x_2, y_2) , where

$$\begin{aligned} x_2 &= x_1 \cos \theta - y_1 \sin \theta \\ y_2 &= x_1 \sin \theta + y_1 \cos \theta \end{aligned}$$

2. **Problem:** The distance between the moving point and the fixed point must be recalculated as the moving point moves to a new position.

Solution: The distance d_1 in Figure 8.4 between two points (p_x, p_y) and (x_1, y_1) is given by the formula

$$d_1 = \sqrt{(x_1 - p_x)^2 + (y_1 - p_y)^2}$$

Similarly, the distance d_2 in Figure 8.4 is

$$d_2 = \sqrt{(x_2 - p_x)^2 + (y_2 - p_y)^2}$$

Listing 8.4 (orbitdist.cpp) uses these mathematical results to compute the difference in the distances.

Listing 8.4: orbitdist.cpp

```
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    // Location of orbiting point is (x,y)
    double x;    // These values change as the
    double y;    // satellite moves
    const double PI = 3.14159;

    // Location of fixed point is always (100, 0),
    // AKA (p_x, p_y). Change these as necessary.
    const double p_x = 100;
```

```

const double p_y = 0;

// Radians in 10 degrees
const double radians = 10 * PI/180;

// Precompute the cosine and sine of 10 degrees
const double COS10 = cos(radians);
const double SIN10 = sin(radians);

// Get starting point from user
cout << "Enter initial satellite coordinates (x,y):";
cin >> x >> y;

// Compute the initial distance
double d1 = sqrt((p_x - x)*(p_x - x) + (p_y - y)*(p_y - y));

// Let the satellite orbit 10 degrees
double x_old = x; // Remember x's original value
x = x*COS10 - y*SIN10; // Compute new x value
// x's value has changed, but y's calculate depends on
// x's original value, so use x_old instead of x.
y = x_old*SIN10 + y*COS10;

// Compute the new distance
double d2 = sqrt((p_x - x)*(p_x - x) + (p_y - y)*(p_y - y));

// Print the difference in the distances
cout << "Difference in distances: " << d2 - d1 << endl;
}

```

We can use the square root function to improve the efficiency of our primes program. Instead of trying all the factors of n up to $n - 1$, we need only try potential factors up to the square root of n . Listing 8.5 (moreefficientprimes.cpp) uses the `sqrt` function to reduce the number of factors that need be considered.

Listing 8.5: moreefficientprimes.cpp

```

#include <iostream>
#include <cmath>

using namespace std;

int main() {

    int max_value;
    cout << "Display primes up to what value? ";
    cin >> max_value;
    for (int value = 2; value <= max_value; value++) {
        // See if value is prime
        bool is_prime = true; // Provisionally, value is prime
        double r = value, root = sqrt(r);
        // Try all possible factors from 2 to the square
        // root of value
        for (int trial_factor = 2;
            is_prime && trial_factor <= root; trial_factor++)
            is_prime = (value % trial_factor != 0);
    }
}

```

```

        if (is_prime)
            cout << value << " ";    // Display the prime number
    }
    cout << endl;    // Move cursor down to next line
}

```

The **sqrt** function comes in three forms:

```

double sqrt(double)
float sqrt(float)
long double sqrt(long double)

```

The function names are the same, but the parameter types differ. We say that the **sqrt** function is *overloaded*. (Overloaded functions are covered in more detail in Section 10.3.) When a caller invokes the **sqrt** function, the compiler matches the call to the closest matching prototype. If the caller passes a **double** parameter, the compiler generates code to call the **double** version. If the caller instead passes a **float** variable, the compiler selects the **float** version of **sqrt**. When an **int** is passed to **sqrt**, the compiler cannot decide which version to use, because an **int** can be converted automatically to either a **float**, **double**, or **long double**. The compiler thus needs some help to resolve the ambiguity, so we introduced an additional variable of type **double** so the compiler will use the **double** version of the **sqrt** function. Another option is to use a type cast to convert the integer value into one of the types acceptable to the **sqrt** function.

8.3 Maximum and Minimum

C++ provides standard functions for determining the maximum and minimum of two numbers. Listing 8.6 (maxmin.cpp) exercises the standard **min** and **max** functions.

Listing 8.6: maxmin.cpp

```

#include <iostream>
#include <algorithm>

using namespace std;

int main() {
    int value1, value2;
    cout << "Please enter two integer values: ";
    cin >> value1 >> value2;
    cout << "max = " << max(value1, value2)
         << ", min = " << min(value1, value2) << endl;
}

```

To use the standard **max** and **min** functions in in program you must include the **<algorithm>** header.

8.4 clock Function

The **clock** function from the **<ctime>** library requests from the operating system the amount of time an executing program has been running. The units returned by the call **clock()** is system dependent, but it

can be converted into seconds with the constant `CLOCKS_PER_SEC`, also defined in the `ctime` library. Under Visual C++, the `CLOCKS_PER_SEC` constant is 1,000, which means the call `clock()` returns the number of milliseconds that the program has been running.

Using two calls to the `clock` function you can measure *elapsed time*. Listing 8.7 (`timeit.cpp`) measures how long it takes a user to enter a character from the keyboard.

Listing 8.7: `timeit.cpp`

```
#include <iostream>
#include <ctime>

using namespace std;

int main() {
    char letter;
    cout << "Enter a character: ";
    clock_t seconds = clock();    // Record starting time
    cin >> letter;
    clock_t other = clock();      // Record ending time
    cout << static_cast<double>(other - seconds)/CLOCKS_PER_SEC
         << " seconds" << endl;
}
```

The type `clock_t` is a type defined in the `<ctime>` header file. `clock_t` is equivalent to an `unsigned long`, and you can perform arithmetic on `clock_t` values and variables just as if they are `unsigned longs`. In the expression

```
static_cast<double>(other - seconds)/CLOCKS_PER_SEC
```

the cast is required to force floating-point division; otherwise, the result is truncated to an integer value.

Listing 8.8 (`measureprimespeed.cpp`) measures how long it takes a program to display all the prime numbers up to half a million using the algorithm from Listing 7.7 (`forprintprimes.cpp`).

Listing 8.8: `measureprimespeed.cpp`

```
#include <iostream>
#include <ctime>
#include <cmath>

using namespace std;

// Display the prime numbers between 2 and 500,000 and
// time how long it takes

int main() {
    clock_t start_time = clock(),    // Record start time
            end_time;
    for (int value = 2; value <= 500000; value++) {
        // See if value is prime
        bool is_prime = true;    // Provisionally, value is prime
        // Try all possible factors from 2 to n - 1
        for (int trial_factor = 2;
             is_prime && trial_factor < value;
             trial_factor++)
```

```

        is_prime = (value % trial_factor != 0);
    if (is_prime)
        cout << value << " ";    // Display the prime number
    }
    cout << endl;    // Move cursor down to next line
    end_time = clock();
    // Print the elapsed time
    cout << "Elapsed time: "
        << static_cast<double>(end_time - start_time)/CLOCKS_PER_SEC
        << " sec." << endl;
}

```

On one system, the program took 93 seconds, on average, to print all the prime numbers up to 500,000. By comparison, the newer, more efficient version, Listing 8.5 (`moreefficientprimes.cpp`), which uses the square root optimization takes only 15 seconds to display all the primes up to 500,000. Exact times will vary depending on the speed of the computer.

As it turns out, much of the program's execution time is taken up printing the output, not computing the prime numbers to print. We can compare the algorithms better by redirecting the program's output to a file. If the executable program is named `primes.exe`, you can redirect its output at the command line by issuing the command

```
primes > run1.out
```

This creates a text file named `run1.out` that can be viewed with any text editor. Its contents are exactly what would have been printed to the screen if the redirection is not used.

When run using redirection, the time difference is even more dramatic: The unoptimized version generates the prime numbers up to 500,000 in 77 seconds, while the optimized square root version requires only 2 seconds to generate the same number of primes! An even faster prime generator can be found in Listing 11.12 (`fasterprimes.cpp`); it uses a completely different algorithm to generate prime numbers.

The `ctime` header must be `#included` to use the standard `time` function in a program.

8.5 Character Functions

The C library provides a number of character functions that are useful to C++ programmers. Listing 8.9 (`touppercase.cpp`) converts lowercase letters to uppercase letters.

Listing 8.9: `touppercase.cpp`

```

#include <iostream>
#include <cctype>

using namespace std;

int main() {
    for (char lower = 'a'; lower <= 'z'; lower++) {
        char upper = toupper(lower);
        cout << lower << " => " << upper << endl;
    }
}

```

```

    }
}

```

The first lines printed by Listing 8.9 (touppercase.cpp) are

```

a => A
b => B
c => C
d => D

```

Interestingly, the `toupper` function returns an `int`, not a `char`. At the enhanced warning level 4 for Visual C++ a cast is required to assign the result to the variable `upper`:

```
char upper = static_cast<char>(toupper(lower));
```

Some of the more useful character functions are described in Table 8.2.

charfunctions Module	
<code>int toupper(int ch)</code>	Returns the uppercase version of the given character; returns the original character if no uppercase version exists (such as for punctuation or digits)
<code>int tolower(int ch)</code>	Returns the lowercase version of the given character; returns the original character if no lowercase version exists (such as for punctuation or digits)
<code>int isupper(int ch)</code>	Returns a nonzero value (true) if <code>ch</code> is an uppercase letter ('A'–'Z'); otherwise, it returns 0 (false)
<code>int islower(int ch)</code>	Returns a nonzero value (true) if <code>ch</code> is a lowercase letter ('a'–'z'); otherwise, it returns 0 (false)
<code>int isalpha(int ch)</code>	Returns a nonzero value (true) if <code>ch</code> is a letter from the alphabet ('A'–'Z' or 'a'–'z'); otherwise, it returns 0 (false)
<code>int isdigit(int ch)</code>	Returns a nonzero value (true) if <code>ch</code> is a digit ('0'–'9'); otherwise, it returns 0 (false)

Table 8.2: A few of the functions from the `cctype` library

Other functions exist to determine if a character is a punctuation character like a comma or semicolon (`ispunct`), a space, tab, or newline character (`isspace`).

To use the standard C character functions in your C++ program, you must include the `<cctype>` header file.

8.6 Random Numbers

Some applications require behavior that appears random. Random numbers are useful particularly in games and simulations. For example, many board games use a die (one of a pair of dice) to determine how many places a player is to advance. (See Figure 8.5.) A die or pair of dice are used in other games of chance. A die is a cube containing spots on each of its six faces. The number of spots range from one to six. A player rolls a die or sometimes a pair of dice, and the side(s) that face up have meaning in the game being played. The value of a face after a roll is determined at random by the complex tumbling of the die. A software adaptation of a game that involves dice would need a way to simulate the random roll of a die.

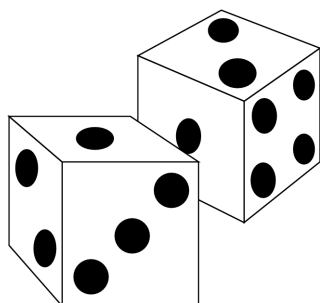


Figure 8.5: A pair of dice

All algorithmic random number generators actually produce *pseudorandom* numbers, not true random numbers. A pseudorandom number generator has a particular period, based on the nature of the algorithm used. If the generator is used long enough, the pattern of numbers produced repeats itself exactly. A sequence of true random numbers would not contain such a repeating subsequence. The good news is that all practical algorithmic pseudorandom number generators have periods that are large enough for most applications.

C++ programmers can use two standard C functions for generating pseudorandom numbers: **srand** and **rand**:

```
void srand(unsigned)
int rand()
```

srand establishes the first value in the sequence of pseudorandom integer values. Each call to **rand** returns the next value in the sequence of pseudorandom values. Listing 8.10 (simplerandom.cpp) shows how a sequence of 100 pseudorandom numbers can be printed.

Listing 8.10: simplerandom.cpp

```
#include <iostream>
#include <cstdlib>

using namespace std;

int main() {
    srand(23);
    for (int i = 0; i < 100; i++) {
        int r = rand();
```

```

        cout << r << " ";

    }
    cout << endl;
}

```

The numbers printed by the program appear to be random. The algorithm is given a seed value to begin, and a formula is used to produce the next value. The seed value determines the sequence of numbers generated; identical seed values generate identical sequences. If you run the program again, the same sequence is displayed because the same seed value, 23, is used. In order to allow each program run to display different sequences, the seed value must be different for each run. How can we establish a different seed value for each run? The best way to make up a “random” seed at run time is to use the `time` function which is found in the `ctime` library. The call `time(0)` returns the number of seconds since midnight January 1, 1970. This value obviously differs between program runs, so each execution will use a different seed value, and the generated pseudorandom number sequences will be different. Listing 8.11 (`betterrandom.cpp`) incorporates the `time` function to improve its randomness over multiple executions.

Listing 8.11: `betterrandom.cpp`

```

#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    srand(static_cast<unsigned>(time(0)));
    for (int i = 0; i < 100; i++) {
        int r = rand();
        cout << r << " ";

    }
    cout << endl;
}

```

Each execution of Listing 8.11 (`betterrandom.cpp`) produces a different pseudorandom number sequence. The actual type of value that `time` returns is `time_t`, so the result from a call to `time` must be cast to `unsigned int` before being used with `srand`.

Notice that the numbers returned by `rand` can be rather large. The pseudorandom values range from 0 to a maximum value that is implementation dependent. The maximum value for Visual C++’s `rand` function is 32,767, which corresponds to the largest 16-bit `signed int` value. The `cstdlib` header defines the constant `RAND_MAX` that represents the largest value in the range. The following statement

```
cout << RAND_MAX << endl;
```

would print the value of `RAND_MAX` for a particular system.

Ordinarily we need values in a more limited range, like 1...100. Simple arithmetic with the modulus operator can produce the result we need. If n is any non-negative integer and m is any positive integer, the expression

$$n \% m$$

produces a value in the range $0 \dots -1$.

This means the statement

```
int r = rand() % 100;
```

can assign only values in the range $0 \dots 99$ to `r`. If we really want values in the range $1 \dots 100$, what can we do? We simply need only add one to the result:

```
int r = rand() % 100 + 1;
```

This statement produces pseudorandom numbers in the range $1 \dots 100$.

We now have all we need to write a program that simulates the rolling of a die.

Listing 8.12 (`die.cpp`) simulates rolling die.

Listing 8.12: `die.cpp`

```
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    // Set the random seed value
    srand(static_cast<unsigned>(time(0)));

    // Roll the die three times
    for (int i = 0; i < 3; i++) {
        // Generate random number in the range 1...6
        int value = rand() % 6 + 1;

        // Show the die
        cout << "+-----+" << endl;
        switch (value) {
            case 1:
                cout << "|           |" << endl;
                cout << "|      *      |" << endl;
                cout << "|           |" << endl;
                break;
            case 2:
                cout << "| *           |" << endl;
                cout << "|           |" << endl;
                cout << "|      *      |" << endl;
                break;
            case 3:
                cout << "|           * |" << endl;
                cout << "|      *      |" << endl;
                cout << "| *           |" << endl;
                break;
            case 4:
                cout << "| *      *    |" << endl;
                cout << "|           |" << endl;
                cout << "| *      *    |" << endl;
        }
    }
}
```

```

        break;
    case 5:
        cout << "| * * |" << endl;
        cout << "| * |" << endl;
        cout << "| * * |" << endl;
        break;
    case 6:
        cout << "| * * * |" << endl;
        cout << "| |" << endl;
        cout << "| * * * |" << endl;
        break;
    default:
        cout << " *** Error: illegal die value ***" << endl;
        break;
    }
    cout << "+-----+" << endl;
}
}

```

The output of one run of Listing 8.12 (die.cpp) is

```

+-----+
| * * |
| |
| * * |
+-----+
+-----+
| * * * |
| * * * |
| * * * |
+-----+
+-----+
| * |
| |
+-----+

```

Since the values are pseudorandomly generated, actual output will vary from one run to the next.

8.7 Summary

- The C and C++ standard library provides a collection of routines that can be incorporated into code that you write.
- When faced with the choice of using a standard library function or writing your own code to solve the same problem, choose the library function. The standard function will be tested thoroughly, well documented, and likely more efficient than the code you would write.
- The function is a standard unit of reuse in C++.

- Code that uses a function is known as *calling* code.
- A function has a name, a list of parameters (which may be empty), and a result type (which may be **void**). A function performs some computation or action that is useful to callers. Typically a function produces a result based on the parameters passed to it.
- Callers communicate information to a function via its parameters (also known as arguments).
- In order to use a standard function, callers must **#include** the proper header file.
- The arguments passed to a function by a caller consist of a comma-separated list enclosed by parentheses.
- Calling code (code outside the function's definition that invoke the function) must pass the correct number and types of parameters that the function expects.
- The C standard library (**<cmath>**) includes a variety of mathematical functions.
- The **time** function from the **<ctime>** library returns the number of seconds since midnight, January 1, 1970.
- The C standard library (**<cctype>**) includes a variety of character manipulation functions.
- The **rand** function (from **<stdlib>**) returns a pseudorandom integer. The first pseudorandom number can be set with the **srand** function. The constant **RAND_MAX** is largest possible value returned by **rand**.

8.8 Exercises

1. Suppose you need to compute the square root of a number in a C++ program. Would it be a good idea to write the code to perform the square root calculation? Why or why not?
2. In C++ source code what is one way to help you distinguish a variable name from a function name?
3. Which one of the following values could be computed by the **rand** function?

4.534-1@RAND_MAX@ + 1
4. What does **clock_t** represent?
5. What does **CLOCKS_PER_SEC** represent?
6. Ordinarily how often should a program call the **srand** function?
7. In Listing 8.2 (standardsquareroot.cpp), what does the **main** function do while the **sqrt** function is computing the square root of the argument that **main** provides?
8. Consider each of the following code fragments below that could be part of a C++ program. Each fragment contains a call to a standard C/C++ library function. Answer each question in one of the following three ways:
 - If the code fragment contains a compile-time error, write the word *error* for the answer.
 - If the code fragment contains no compile-time errors and you can determine its output at compile-time, provide the fragment's literal output.

- If the code fragment contains no compile-time errors but you cannot determine its exact output at compile-time, provide one possible evaluation and write the word *example* for the answer and provide one possible literal output that the code fragment could produce.

- (a) `cout << sqrt(4.5) << endl;`
- (b) `cout << sqrt(4.5, 3.1) << endl;`
- (c) `cout << rand(4) << endl;`
- (d) `double d = 16.0;`
`cout << sqrt(d) << endl;`
- (e) `cout << srand() << endl;`
- (f) `cout << rand() << endl;`
- (g) `int i = 16;`
`cout << sqrt(i) << endl;`
- (h) `cout << srand(55) << endl;`
- (i) `cout << tolower('A') << endl;`
- (j) `cout << exp() << endl;`
- (k) `cout << sqrt() << endl;`
- (l) `cout << toupper('E') << endl;`
- (m) `cout << toupper('e') << endl;`
- (n) `cout << toupper("e") << endl;`
- (o) `cout << exp(4.5) << endl;`
- (p) `cout << toupper('h', 5) << endl;`
- (q) `cout << ispunct('!') << endl;`
- (r) `cout << tolower("F") << endl;`
- (s) `char ch = 'D';`
`cout << tolower(ch) << endl;`
- (t) `cout << exp(4.5, 3) << endl;`

- (u) `cout << toupper('7') << endl;`
 - (v) `double a = 5, b = 3;`
`cout << exp(a, b) << endl;`
 - (w) `cout << exp(3, 5, 2) << endl;`
 - (x) `cout << tolower(70) << endl;`
 - (y) `double a = 5;`
`cout << exp(a, 3) << endl;`
 - (z) `double a = 5;`
`cout << exp(3, a) << endl;`
9. From geometry: Write a computer program that given the lengths of the two sides of a right triangle adjacent to the right angle computes the length of the hypotenuse of the triangle. (See Figure 8.6.) If you are unsure how to solve the problem mathematically, do a web search for the *Pythagorean theorem*.

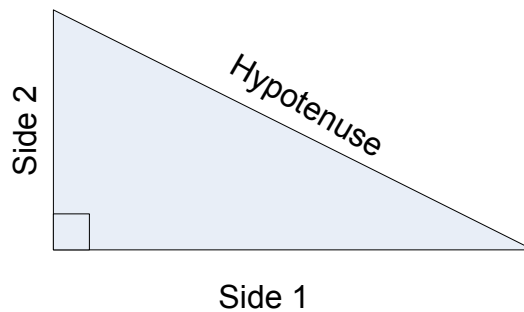


Figure 8.6: Right triangle

10. From trigonometry: Write a computer program that given the lengths of the two sides of a right triangle adjacent to the right angle computes the length of the hypotenuse of the triangle. If you are unsure how to solve the problem mathematically, do a web search for the *Pythagorean theorem*.
- 11.

Chapter 9

Writing Functions

As programs become more complex, programmers must structure their programs in such a way as to effectively manage their complexity. Most humans have a difficult time keeping track of too many pieces of information at one time. It is easy to become bogged down in the details of a complex problem. The trick to managing complexity is to break down the problem into more manageable pieces. Each piece has its own details that must be addressed, but these details are hidden as much as possible within that piece. The problem is ultimately solved by putting these pieces together to form the complete solution.

So far all of our programs have been written within one function—**main**. As the number of statements within a function increases, the function can become unwieldy. The code within such a function that does all the work by itself is called *monolithic code*. Monolithic code that is long and complex is undesirable for several reasons:

- **It is difficult to write correctly.** All the details in the entire piece of code must be considered when writing any statement within that code.
- **It is difficult to debug.** If the sequence of code does not work correctly, it is often difficult to find the source of the error. The effects of an erroneous statement that appears earlier in the code may not become apparent until a correct statement later uses the erroneous statement's incorrect result.
- **It is difficult to extend.** All the details in the entire sequence of code must be well understood before it can be modified. If the code is complex, this may be a formidable task.

Using a divide and conquer strategy, a programmer can decompose a complicated function (like **main**) into several simpler functions. The original function can then do its job by delegating the work to these other functions. In this way the original function can be thought of as a “work coordinator.”

Besides their code organization aspects, functions allow us to bundle functionality into reusable parts. In Chapter 8 we saw how library functions can dramatically increase the capabilities of our programs. While we should capitalize on library functions as much as possible, sometimes we need a function exhibiting custom behavior that is not provided by any standard function. Fortunately we can create our own functions, and the same function may be used (called) in numerous places within a program. If the function's purpose is general enough and we write the function properly, we may be able to reuse the function in other programs as well.

9.1 Function Basics

Recall the “handwritten” square root code we saw in Listing 8.1 (`computesquareroot.cpp`). We know that the better option is the standard library function `sqrt`; however, we will illustrate custom function development by writing our own square root function based on the code in Listing 8.1 (`computesquareroot.cpp`). In Listing 9.1 (`customsquareroot.cpp`) we see the definition for the `square_root` function.

Listing 9.1: `customsquareroot.cpp`

```
#include <iostream>
#include <iomanip>
#include <cmath>

using namespace std;

// Compute an approximation of
// the square root of x
double square_root(double x) {
    double diff;
    // Compute a provisional square root
    double root = 1.0;

    do { // Loop until the provisional root
        // is close enough to the actual root
        root = (root + x/root) / 2.0;
        //cout << "root is " << root << endl;
        // How bad is the approximation?
        diff = root * root - x;
    } while (diff > 0.0001 || diff < -0.0001);
    return root;
}

int main() {
    // Compare the two ways of computing the square root
    for (double d = 1.0; d <= 10.0; d += 0.5)
        cout << setw(7) << square_root(d) << " : " << sqrt(d) << endl;
}
```

The `main` function in Listing 9.1 (`customsquareroot.cpp`) compares the behavior of our custom `square_root` function to the `sqrt` library function. Its output:

```
1 : 1
1.22474 : 1.22474
1.41422 : 1.41421
1.58116 : 1.58114
1.73205 : 1.73205
1.87083 : 1.87083
2 : 2
2.12132 : 2.12132
2.23607 : 2.23607
2.34521 : 2.34521
2.44949 : 2.44949
2.54952 : 2.54951
2.64577 : 2.64575
2.73861 : 2.73861
2.82843 : 2.82843
2.91548 : 2.91548
3 : 3
3.08221 : 3.08221
3.16228 : 3.16228
```

shows a few small differences in the results. Clearly we should use the standard `sqrt` function instead of ours.

There are two aspects to every C++ function:

- **Function definition.** The definition of a function specifies the function's return type and parameter types, and it provides the code that determines the function's behavior. In Listing 9.1 (customsquareroot.cpp) the definition of the `square_root` function appears above the `main` function.
- **Function invocation.** A programmer uses a function via a function invocation. The `main` function invokes both our `square_root` function and the `sqrt` function. Every function has exactly one definition but may have many invocations.

A function definition consists of four parts:

- **Name**—every function in C++ has a name. The name is an identifier (see Section 3.3). As with variable names, the name chosen for a function should accurately portray its intended purpose or describe its functionality.
- **Type**—every function has a return type. If the function returns a value to its caller, its type corresponds to the type of the value it returns. The special type `void` signifies that the function does not return a value.
- **Parameters**—every function must specify the types of parameters that it accepts from callers. The parameters appear in a parenthesized comma-separated list like in a function prototype (see Section 8.1). Unlike function prototypes, however, parameters usually have names associated with each type.
- **Body**—every function definition has a body enclosed by curly braces. The body contains the code to be executed when the function is invoked.

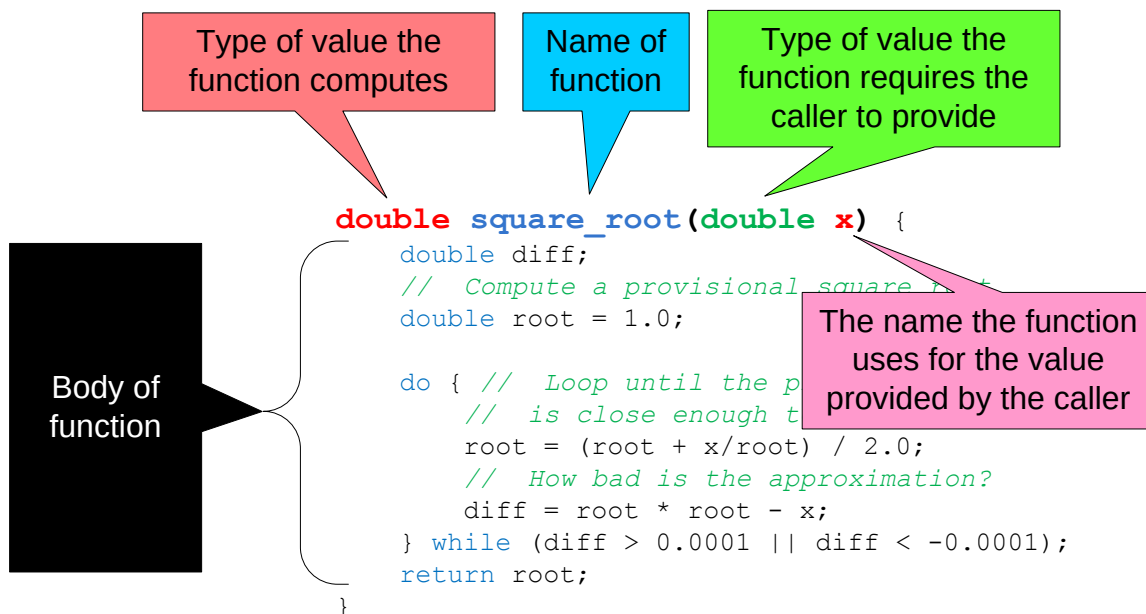


Figure 9.1: Function definition dissection

Figure 9.1 dissects a our **square_root** function definition.

The simplest function accepts no parameters and returns no value to the caller. Listing 9.2 (simplefunction.cpp) is a variation of Listing 4.1 (adder.cpp) that contains such a simple function definition.

Listing 9.2: simplefunction.cpp

```

#include <iostream>

using namespace std;

// Definition of the prompt function
void prompt() {
    cout << "Please enter an integer value: ";
}

int main() {
    int value1, value2, sum;
    cout << "This program adds together two integers." << endl;
    prompt(); // Call the function
    cin >> value1;
    prompt(); // Call the function again
    cin >> value2;
    sum = value1 + value2;
    cout << value1 << " + " << value2 << " = " << sum << endl;
}

```

```
}
```

The **prompt** function simply prints a message. The program runs as follows:

1. The program's execution, like in all C++ programs, begins with the first executable statement in the function named **main**. The first line in the **main** function simply declares some variables needed for compiler housekeeping, so the next line actually begins the executable code.
2. The first executable statement prints the message of the program's intent.
3. The next statement is a call of the **prompt** function. At this point the program's execution transfers to the body of the **prompt** function. The code within **prompt** is executed until the end of its body or until a **return** statement is encountered. Since **prompt** contains no **return** statement, all of **prompt**'s body (the one print statement) will be executed.
4. When **prompt** is finished, control is passed back to the point in **main** immediately *after* the call of **prompt**.
5. The next action after **prompt** call reads the value of **value1** from the keyboard.
6. A second call to **prompt** transfers control back to the code within the **prompt** function. It again prints its message.
7. When the second call to **prompt** is finished, control passes back to **main** at the point of the second input statement that assigns **value2** from the keyboard.
8. The remaining two statements in **main** are executed, and then the program's execution terminates.

As another simple example, consider Listing 9.3 (countto10.cpp).

Listing 9.3: countto10.cpp

```
#include <iostream>

using namespace std;

int main() {
    for (int i = 1; i <= 10; i++)
        cout << i << endl;
}
```

which simply counts to ten:

```
1
2
3
4
5
6
7
8
9
10
```

If counting to ten in this way is something we want to do frequently within a program, we can write a function as shown in Listing 9.4 (countto10func.cpp) and call it as many times as necessary.

Listing 9.4: countto10func.cpp

```
#include <iostream>

using namespace std;

// Count to ten and print each number on its own line
void count_to_10() {
    for (int i = 1; i <= 10; i++)
        cout << i << endl;
}

int main() {
    cout << "Going to count to ten . . .";
    count_to_10();
    cout << "Going to count to ten again. . .";
    count_to_10();
}
```

Our **prompt** and **countto10** functions are a bit underwhelming. The **prompt** function could be eliminated, and each call to **prompt** could be replaced with the statement in its body. The same could be said for the **countto10** function, although it is convenient to have the simple one-line statement that hides the complexity of the loop. Using the **prompt** function does have one advantage, though. If **prompt** is removed and the two calls to **prompt** are replaced with the print statement within **prompt**, we have to make sure that the two messages printed are identical. If we simply call **prompt**, we know the two messages printed will be identical because only one possible message can be printed (the one in the body of **prompt**).

We can alter the behavior of a function through a mechanism called *parameter passing*. If a function is written to accept information from the caller, the caller must supply the information in order to use the function. The caller communicates the information via one or more parameters as required by the function. The **countto10** function does us little good if we sometimes want to count up to a different number. Listing 9.5 (countton.cpp) generalizes Listing 9.4 (countto10func.cpp) to count as high as the caller needs.

Listing 9.5: countton.cpp

```
#include <iostream>

using namespace std;

// Count to n and print each number on its own line
void count_to_n(int n) {
    for (int i = 1; i <= n; i++)
        cout << i << endl;
}

int main() {
    cout << "Going to count to ten . . .";
    count_to_n(10);
    cout << "Going to count to five . . .";
    count_to_n(5);
}
```

When the caller, in this case **main**, issues the call

```
count_to_n(10);
```

the argument 10 is assigned to **n** before the function's statements begin executing.

A caller must pass exactly one integer parameter (or other type that is assignment-compatible with integers) to **countton** during a call. An attempt to do otherwise one will result in a compiler error or warning:

```
count_to_n();           // Error, missing parameter during the call
count_to_n(3, 5);       // Error, too many parameters during the call
count_to_n(3.2);        // Warning, possible loss of data (double to int)
```

We can enhance the **prompt** function's capabilities as shown in Listing 9.6 (betterprompt.cpp)

Listing 9.6: betterprompt.cpp

```
#include <iostream>

using namespace std;

// Definition of the prompt function
int prompt() {
    int result;
    cout << "Please enter an integer value: ";
    cin >> result;
    return result;
}

int main() {
    int value1, value2, sum;
    cout << "This program adds together two integers." << endl;
    value1 = prompt(); // Call the function
    value2 = prompt(); // Call the function again
    sum = value1 + value2;
    cout << value1 << " + " << value2 << " = " << sum << endl;
}
```

In this version, **prompt** takes care of the input, so **main** does not have to use any input statements. The assignment statement within **main**:

```
value1 = prompt();
```

implies **prompt** is no longer a **void** function; it must return a value that can be assigned to the variable **value1**. Furthermore, the value that **prompt** returns must be assignment compatible with an **int** because **value1**'s declared type is **int**. A quick look at the first line of **prompt**'s definition confirms our assumption:

```
int prompt()
```

This indicates that **prompt** returns an **int** value.

Because **prompt** is declared to return an **int** value, it must contain a **return** statement. A **return** statement specifies the exact value to return to the caller. When a **return** is encountered during a function's execution, control immediately passes back to the caller. The value of the function call is the value specified by the **return** statement, so the statement

```
value1 = prompt();
```

assigns to the variable **value1** the value indicated when the **return** statement executes.

Note that in Listing 9.6 (`betterprompt.cpp`), we declared a variable named **result** inside the **prompt** function. This variable is local to the function, meaning we cannot use this particular variable outside of **prompt**. It also means we are free to use that same name outside of the **prompt** function in a different context, and that use will not interfere with the **result** variable within **prompt**. We say that **result** is a *local variable*.

We can further enhance our **prompt** function. Currently **prompt** always prints the same message. Using parameters, we can customize the message that **prompt** prints. Listing 9.7 (`evenbetterprompt.cpp`) shows how parameters are used to provide a customized message within **prompt**.

Listing 9.7: `evenbetterprompt.cpp`

```
#include <iostream>

using namespace std;

// Definition of the prompt function
int prompt(int n) {
    int result;
    cout << "Please enter integer #" << n << ": ";
    cin >> result;
    return result;
}

int main() {
    int value1, value2, sum;
    cout << "This program adds together two integers." << endl;
    value1 = prompt(1);    // Call the function
    value2 = prompt(2);    // Call the function again
    sum = value1 + value2;
    cout << value1 << " + " << value2 << " = " << sum << endl;
}
```

In Listing 9.7 (`evenbetterprompt.cpp`), the parameter influences the message that it printed. The user is now prompted to enter value #1 or value #2. The call

```
value1 = prompt(1);
```

passes the value 1 to the **prompt** function. Since **prompt**'s parameter is named **n**, the process works as if the assignment statement

```
n = 1;
```

were executed as the first action within **prompt**.

In the first line of the function definition:

```
int prompt(int n)
```

n is called the *formal parameter*. A formal parameter is used like a variable within the function's body, but it is declared in the function's parameter list; it is not declared in the function's body. A *formal* parameter is a parameter as used in the *formal* definition of the function.

At the point of the function call:

```
value1 = prompt(1);
```

the parameter (or argument) passed into the function, 1, is called the *actual parameter*. An *actual parameter* is the parameter *actually* used during a call of the function. When a function is called, any actual parameters are assigned to their corresponding formal parameters, and the function begins executing. Another way to say it is that during a function call, the actual parameters are *bound* to their corresponding formal parameters.

The parameters used within a function definition are called *formal parameters*. Formal parameters behave as local variables within the function's body; as such, the name of a formal parameter will not conflict with any local variable or formal parameter names from other functions. This means as a function developer you may choose a parameter name that best represents the parameter's role in the function.



If you are writing a function, you cannot predict the caller's actual parameters. You must be able to handle any value the caller sends. The compiler will ensure that the types of the caller's parameters are compatible with the declared types of your formal parameters.

To remember the difference between formal and actual parameters, remember this:

- A *formal* parameter is a parameter declared and used in a function's *formal* definition.
- An *actual* parameter is a parameter supplied by the caller when the caller *actually* uses (invokes or calls) the function.

When the call

```
value1 = prompt(1);
```

is executed in **main**, and the statement

```
cout << "Please enter integer #" << n << ": ";
```

within the body of **prompt** is executed, **n** will have the value 1. Similarly, when the call

```
value2 = prompt(2);
```

is executed in **main**, and the statement

```
cout << "Please enter integer #" << n << ": ";
```

within the body of **prompt** is executed, **n** will have the value 2. In the case of

```
value1 = prompt(1);
```

n within **prompt** is bound to 1, and in the case of

```
value2 = prompt(2);
```


`n` within `prompt` is bound to 2.



A function's definition requires that all formal parameters be declared in the parentheses following the function's name. A caller does not provide actual parameter type declarations when calling the function. Given the `square_root` function defined in Listing 9.1 (`customsquareroot.cpp`), the following caller code fragment is illegal:

```
double number = 25.0;
// Legal, pass the variable's value to the function
cout << square_root(number) << endl;
// Illegal, do not declare the parameter during the call
cout << square_root(double number) << endl;
```

The function definition is responsible for declaring the types of its parameters, not the caller.

9.2 Using Functions

The general form of a function definition is

```
type name ( parameterlist )
{
    body
}
```

- The *type* of the function indicates the type of value the function returns. Often a function will perform a calculation and the result of the calculation must be communicated back to the place where the function was invoked. The special type `void` indicates that the function does not return a value.
- The *name* of the function is an identifier (see Section 3.3). The function's name should indicate the purpose of the function.
- The *parameterlist* is a comma separated list of pairs of the form

type name

where *type* is a C++ type and *name* is an identifier representing a parameter. The caller of the function communicates information into the function via parameters. The parameters specified in the parameter list of a function definition are called *formal parameters*. A parameter is also known as an *argument*. The parameter list may be empty; an empty parameter list indicates that no information may be passed into the function by the caller.

- The *body* is the sequence of statements, enclosed within curly braces, that define the actions that the function is to perform. The statements may include variable declarations, and any variables declared within the body are local to that function.

The body may contain only one statement, many statements, or no statements at all; regardless, the curly braces always are required.

Observe that multiple pieces of information can be passed into a function via multiple parameters, but only one piece of information can be passed out of the function via the return value. Recall the greatest common divisor (also called greatest common factor) from elementary mathematics. To determine the GCD of 24 and 18 we list all of their common factors and select the largest one:

24: 1, 2, 3, 4, 6, 8, 12, 24
 18: 1, 2, 3, 6, 9, 18

The greatest common divisor function is useful when reducing fractions

to lowest terms; for example, consider the fraction $\frac{18}{24}$. The greatest common divisor of 18 and 24 is 6, and

so we divide the numerator and the denominator of the fraction by 6: $\frac{18 \div 6}{24 \div 6} = \frac{3}{4}$. The GCD function has applications in other areas besides reducing fractions to lowest terms. Consider the problem of dividing a piece of plywood 24 inches long by 18 inches wide into square pieces of maximum size without wasting any material. Since the $\text{GCF}(24, 18) = 6$, we can cut the plywood into twelve 6 inch $\times$ 6 inch square pieces as shown in Figure 9.2. If we cut the plywood into squares of any other size without wasting the

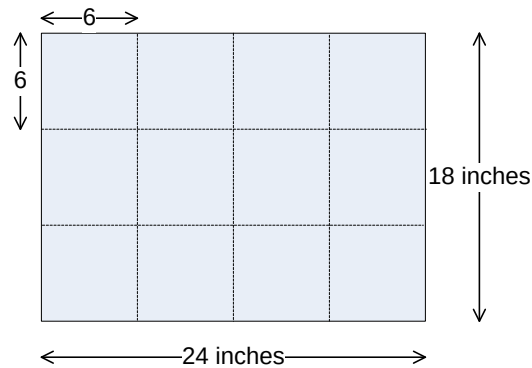


Figure 9.2: Cutting plywood

any of the material, the squares would have to be smaller than 6 inches $\times$ 6 inches; for example, we could make forty-eight 3 inch $\times$ 3 inch squares as shown in pieces as shown in Figure 9.3. If we cut squares

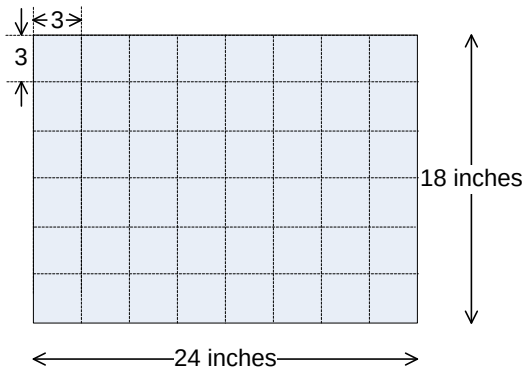


Figure 9.3: Squares too small

larger than 6 inches $\times$ 6 inches, not all the plywood can be used to make the squares. Figure 9.4. shows how some larger squares would fare. The GCD function plays a vital role in cryptography, enabling secure

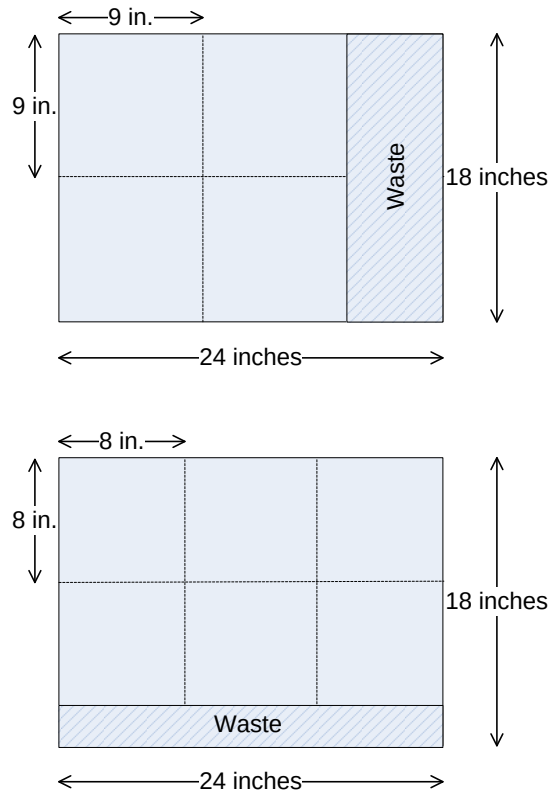


Figure 9.4: Squares too large

communication across an insecure network.

The following code defines a function that computes the greatest common divisor of two integers. It determines largest factor (divisor) common to its parameters:

```
int gcd(int num1, int num2) {
    // Determine the smaller of num1 and num2
    int min = (num1 < num2) ? num1 : num2;
    // 1 is definitely a common factor to all ints
    int largestFactor = 1;
    for (int i = 1; i <= min; i++)
        if (num1 % i == 0 && num2 % i == 0)
            largestFactor = i; // Found larger factor
    return largestFactor;
}
```

This function is named **gcd** and expects two integer arguments. Its formal parameters are named **num1** and **num2**. It returns an integer result. Its body declares three local variables: **min**, **largestFactor**, and **i** (**i** is local to the **for** statement). The last line in its body is a **return** statement. A return statement is required for functions that return a value. A **void** function is not required to have a **return** statement. If a **void** function does have a **return** statement, it must simply consist of **return** followed by a semicolon (in other words, it cannot return a value, like **gcd**'s **return** statement does). A **void** function that does not contain a **return** statement simply returns at the end of its body.

Recall from Section 6.5 that local variables have meaning only within their scope. This means that when you write a function you can name a local variable without fear that its name may be used already in another part of the program. Two different functions can use local variables named **x**, and these are two different variables that have no influence on each other. Anything local to a function definition is hidden to all code outside that function definition.

Since a formal parameter is a local variable, you can reuse the names of formal parameters in different functions without a problem.

Another advantage of local variables is that they occupy space in the computer's memory only when the function is executing. Space is allocated for local variables and parameters when the function begins executing. When the function is finished and control returns to the caller, the variables and parameters go out of scope, and the memory they held is freed up for other purposes within the running program. This process of local variable allocation and deallocation happens each time a caller invokes the function. More information about how C++ handles memory management during a program's execution can be found in Section F.11.

Once we have written a complete function definition we can use the function within our program. We invoke a programmer-defined function in exactly the same way as a standard library function like **sqrt** (Section 8.2) or **rand** (Section 8.6). If the function returns a value (that is, it is not declared **void**), then we can use its invocation anywhere an expression of that type is allowed. The parameters used for the function call are known as actual parameters. The function **gcd** can be called as part of an assignment statement:

```
int factor = gcd(val, 24);
```

This call uses the variable **val** as its first actual parameter and the literal value 24 as its second actual parameter. Variables, expressions, and literals can be freely used as actual parameters. The function then computes and returns its result. This result is assigned to the variable **factor**.

How does the function call and parameter mechanism work? It's actually quite simple. The actual parameters, in order, are assigned (bound) to each of the formal parameters in the function definition, then control is passed to the body of the function. When the function's body is finished executing, control passes back to the point in the program where the function was called. The value returned by the function, if any, replaces the function call expression. In the statement

```
int factor = gcd(val, 24);
```

an integer value is assigned to **factor**. The expression on the right is a function call, so the function is invoked to determine what to assign. The value of the variable **val** is assigned to the formal parameter **num1**, and the literal value 24 is assigned to the formal parameter **num2**. The body of the **gcd** function is then executed. When the **return** statement in the body is encountered, program execution returns back to where the function was called. The argument of the return statement becomes the value that is assigned to **factor**. This process of copying actual parameters to formal parameters works exactly like assignment, so widening and narrowing (see Section 4.2) is performed automatically as needed. For example, if **val** is declared to be a **char**, its value would automatically be copied to a temporary location and converted to an **int**. This temporary value would then be bound to the formal parameter **num1**. Note that **gcd** could be

called from many different places within the same program, and, since different parameter values could be passed at each of these different invocations, `gcd` could compute a different result at each invocation.

Other invocation examples include:

- `cout << gcd(36, 24);`

This example simply prints the result of the invocation. The value 36 is bound to `num1` and 24 is bound to `num2` for the purpose of the function call. The value 12 will be printed, since 12 is the greatest common divisor of 36 and 24..

- `x = gcd(x - 2, 24);`

The execution of this statement would evaluate `x - 2` and bind its value to `num1`. `num2` would be assigned 24. The result of the call is then assigned to `x`. Since the right side of the assignment statement is evaluated *before* being assigned to the left side, the original value of `x` is used when calculating `x - 2`, and the function return value then updates `x`.

- `x = gcd(x - 2, gcd(10, 8));`

This example shows two invocations in one statement. Since the function returns an integer value its result can itself be used as an actual parameter in a function call. Passing the result of one function call as an actual parameter to another function call is called *function composition*.

The compiler will report an error if a function call does not agree with the function's definition. Possible problems include:

- **Number of actual parameters do not agree with the number of formal parameters.** The number of parameters must agree exactly. For example, the statement

```
int factor = gcd(24);    // Error: too few parameters
```

is illegal given the above definition of `gcd`, since only one actual parameter is provided when two are required.

- **Passing an actual parameter that is not assignment compatible with the formal parameter.** For example, passing the `cout` object when an `int` has been defined, as in

```
int factor = gcd(36, cout); // Error: second parameter is wrong type
```

The compiler will detect that `cout` is not a valid `int` and report an error.

- **Using the result in a context where an expression of that type is not allowed.** For example, a function that returns `void` cannot be used where an `int` is expected:

```
cout << srand(2); // Error: srand does not return anything
```

The compiler will disallow this code.

9.3 Pass by Value

The default parameter passing mechanism in C++ is classified as *pass by value*, also known as *call by value*. This means the value of the actual parameter is copied to the formal parameter for the purpose of executing the function's code. Since it is working on a copy of the actual parameter, the function's execution cannot affect the value of the actual parameter owned by the caller.

Listing 9.8 (passbyvalue.cpp) illustrates the consequences of pass by value.

Listing 9.8: passbyvalue.cpp

```
#include <iostream>

using namespace std;

/*
 * increment(x)
 *     Illustrates pass by value protocol.
 */
void increment(int x) {
    cout << "Beginning execution of increment, x = "
         << x << endl;
    x++; // Increment x
    cout << "Ending execution of increment, x = "
         << x << endl;
}

int main() {
    int x = 5;
    cout << "Before increment, x = " << x << endl;
    increment(x);
    cout << "After increment, x = " << x << endl;
}
```

For additional drama we chose to name the actual parameter the same as the formal parameter. Since the actual parameter and formal parameter are declared and used in different contexts and represent completely different memory locations, their names can be the same without any problems.

Listing 9.8 (passbyvalue.cpp) produces

```
Before increment, x = 5
Beginning execution of increment, x = 5
Ending execution of increment, x = 6
After increment, x = 5
```

The memory for the variable **x** in **main** is unaffected since **increment** works on a copy of the actual parameter.

C++ supports another way of passing parameters called *pass by reference*. Pass by reference is introduced in Section 10.8.

A function communicates its return value to the caller in the same way that the caller might pass a parameter by value. In the **prompt** function we saw earlier:

```
int prompt(int n) {
    int result;
    cout << "Please enter integer #" << n << ": ";
    cin >> result;
    return result;
}
```

the `return` statement is

```
return result;
```

The variable `result` is local to `prompt`. We informally may say we are returning the `result` variable, but, in fact, we really are returning only the *value* of the `result` variable. The caller has no access to the local variables declared within any function it calls. In fact, the local variables for a function exist only when the function is active (that is, executing). When the function returns to its caller all of its local variables disappear from memory. During subsequent invocations, the function's local variables reappear when the function becomes active and disappear again when it finishes.

9.4 Function Examples

This section contains a number of examples of how we can use functions to organize a program's code.

9.4.1 Better Organized Prime Generator

Listing 9.9 (`primefunc.cpp`) is a simple enhancement of Listing 8.5 (`moreefficientprimes.cpp`). It uses the square root optimization and adds a separate `is_prime` function.

Listing 9.9: `primefunc.cpp`

```
#include <iostream>
#include <cmath>

using namespace std;

/*
 *  is_prime(n)
 *      Determines the primality of a given value
 *      n an integer to test for primality
 *      Returns true if n is prime; otherwise, returns false
 */
bool is_prime(int n) {
    bool result = true; // Provisionally, n is prime
    double r = n, root = sqrt(r);
    // Try all possible factors from 2 to the square
    // root of n
    for (int trial_factor = 2;
         result && trial_factor <= root; trial_factor++)
        result = (n % trial_factor != 0);
    return result;
}
```

```

/*
 *  main
 *      Tests for primality each integer from 2
 *      up to a value provided by the user.
 *      If an integer is prime, it prints it;
 *      otherwise, the number is not printed.
 */
int main() {
    int max_value;
    cout << "Display primes up to what value? ";
    cin >> max_value;
    for (int value = 2; value <= max_value; value++)
        if (is_prime(value)) // See if value is prime
            cout << value << " "; // Display the prime number
    cout << endl; // Move cursor down to next line
}

```

Listing 9.9 (primefunc.cpp) illustrates several important points about well-organized programs:

- The complete work of the program is no longer limited to the **main** function. The effort to test for primality is delegated to a separate function. **main** is focused on a simpler task: generating all the numbers to be considered and using another function (**is_prime**) to do the hard work of determining if a given number is prime. **main** is now simpler and more logically *coherent*. A function is coherent when it is focused on a single task. Coherence is a desirable property of functions. If a function becomes too complex by trying to do too many different things, it can be more difficult to write correctly and debug when problems are detected. A complex function should be decomposed into several, smaller, more coherent functions. The original function would then call these new simpler functions to accomplish its task. Here, **main** is not concerned about *how* to determine if a given number is prime; **main** simply delegates the work to **is_prime** and makes use of the **is_prime** function's findings.
- Each function is preceded by a thorough comment that describes the nature of the function. It explains the meaning of each parameter, and it indicates what the function should return. The comment for **main** may not be as thorough as for other functions; this is because **main** usually has no parameters, and it always returns a code to the operating system upon the program's termination.
- While the exterior comment indicates *what* the function is to do, comments within each function explain in more detail *how* the function accomplishes its task.

The call to **is_prime** return true or false depending on the value passed to it. The means a condition like

```
if (is_prime(value) == true) . . .
```

can be expressed more compactly as

```
if (is_prime(value)) . . .
```

because if **is_prime(value)** is true, **true** $\sim$ **true** is true, and if **is_prime(value)** is false, **false** $\sim$ **true** is false. The expression **is_prime(value)** suffices.

Just as it is better for a loop to have exactly one entry point and exactly one exit point, preferably a function will have a single **return** statement. Simple functions with a small number of **returns** are generally tolerable, however. Consider the following version of **is_prime**:


```

bool is_prime(int n) {
    for (int trialFactor = 2;
        trialFactor <= sqrt(static_cast<double>(n));
        trialFactor++)
        if (n % trialFactor == 0) // Is trialFactor a factor?
            return false; // Yes, return right away
    return true; // Tried them all, must be prime
}

```

This version uses two `return` statements, but eliminates the need for a local variable (`result`). Because a `return` statement exits the function immediately, no `break` statement is necessary. The two `return` statements are close enough textually in source code that the logic is fairly transparent.

9.4.2 Command Interpreter

Some functions are useful even if they accept no information from the caller and return no result. Listing 9.10 (`calculator.cpp`) uses such a function.

Listing 9.10: calculator.cpp

```

#include <iostream>
#include <cmath>

using namespace std;

/*
 * help_screen
 *   Displays information about how the program works
 *   Accepts no parameters
 *   Returns nothing
 */
void help_screen() {
    cout << "Add:   Adds two numbers" << endl;
    cout << "      Example: a 2.5 8.0" << endl;
    cout << "Subtract: Subtracts two numbers" << endl;
    cout << "      Example: s 10.5 8.0" << endl;
    cout << "Print:   Displays the result of the latest operation"
        << endl;

    cout << "      Example: p" << endl;
    cout << "Help:   Displays this help screen" << endl;
    cout << "      Example: h" << endl;
    cout << "Quit:   Exits the program" << endl;
    cout << "      Example: q" << endl;
}

/*
 * menu
 *   Display a menu
 *   Accepts no parameters
 *   Returns the character entered by the user.
 */
char menu() {
    // Display a menu
}

```

```

    cout << "=== A)dd S)ubtract P)rint H)elp Q)uit ===" << endl;
    // Return the char entered by user
    char ch;
    cin >> ch;
    return ch;
}

/*
 * main
 *   Runs a command loop that allows users to
 *   perform simple arithmetic.
 */
int main() {
    double result = 0.0, arg1, arg2;
    bool done = false; // Initially not done
    do {
        switch (menu()) {
            case 'A': // Addition
            case 'a':
                cin >> arg1 >> arg2;
                result = arg1 + arg2;
                cout << result << endl;
                break;
            case 'S': // Subtraction
            case 's':
                cin >> arg1 >> arg2;
                result = arg1 - arg2;
                // Fall through, so it prints the result
            case 'P': // Print result
            case 'p':
                cout << result << endl;
                break;
            case 'H': // Display help screen
            case 'h':
                help_screen();
                break;
            case 'Q': // Quit the program
            case 'q':
                done = true;
                break;
        }
    }
    while (!done);
}

```

The `help_screen` function needs no information from `main`, nor does it return a result. It behaves exactly the same way each time it is called. The `menu` function returns the character entered by the user.

9.4.3 Restricted Input

Listing 7.3 (`betterinputonly.cpp`) forces the user to enter a value within a specified range. We now can easily adapt that concept to a function. Listing 9.11 (`betterinputfunc.cpp`) uses a function named `get_int_range` that does not return until the user supplies a proper value.

Listing 9.11: betterinputfunc.cpp

```

#include <iostream>

using namespace std;

/*
 *  get_int_range(first, last)
 *      Forces the user to enter an integer within a
 *      specified range
 *      first is either a minimum or maximum acceptable value
 *      last is the corresponding other end of the range,
 *      either a maximum or minimum *      value
 *      Returns an acceptable value from the user
 */
int get_int_range(int first, int last) {
    // If the larger number is provided first,
    // switch the parameters
    if (first > last) {
        int temp = first;
        first = last;
        last = temp;
    }
    // Insist on values in the range first...last
    cout << "Please enter a value in the range "
         << first << "..." << last << ": ";
    int in_value;    // User input value
    bool bad_entry;
    do {
        cin >> in_value;
        bad_entry = (in_value < first || in_value > last);
        if (bad_entry) {
            cout << in_value << " is not in the range "
                 << first << "..." << last << endl;
            cout << "Please try again: ";
        }
    }
    while (bad_entry);
    // in_value at this point is guaranteed to be within range
    return in_value;
}

/*
 *  main
 *      Tests the get_int_range function
 */
int main() {
    cout << get_int_range(10, 20) << endl;
    cout << get_int_range(20, 10) << endl;
    cout << get_int_range(5, 5) << endl;
    cout << get_int_range(-100, 100) << endl;
}

```

Listing 9.11 (betterinputfunc.cpp) forces the user to enter a value within a specified range. This functionality could be useful in many programs.

In Listing 9.11 (`betterinputfunc.cpp`)

- The high and low values are specified by parameters. This makes the function more flexible since it could be used elsewhere in the program with a completely different range specified and still work correctly.
- The function is supposed to be called with the lower number passed as the first parameter and the higher number passed as the second parameter. The function will also accept the parameters out of order and automatically swap them to work as expected; thus,

```
num = get_int_range(20, 50);
```

will work exactly like

```
num = get_int_range(50, 20);
```

- The Boolean variable `bad_entry` is used to avoid evaluating the Boolean expression twice (once to see if the bad entry message should be printed and again to see if the loop should continue).

9.4.4 Better Die Rolling Simulator

Listing 9.12: `betterdie.cpp`

```
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

/*
 * initialize_die
 *   Initializes the randomness of the die
 */
void initialize_die() {
    // Set the random seed value
    srand(static_cast<unsigned>(time(0)));
}

/*
 * show_die(spots)
 *   Draws a picture of a die with number of spots
 *   indicated
 *   spots is the number of spots on the top face
 */
void show_die(int spots) {
    cout << "+-----+" << endl;
    switch (spots) {
        case 1:
            cout << "|           |" << endl;
            cout << "|    *    |" << endl;
            cout << "|           |" << endl;
            break;
        case 2:
            cout << "| *       |" << endl;
```

```

        cout << " |      |" << endl;
        cout << " |      * |" << endl;
        break;
    case 3:
        cout << " |      * |" << endl;
        cout << " |      * |" << endl;
        cout << " | *      |" << endl;
        break;
    case 4:
        cout << " | *      * |" << endl;
        cout << " |      |" << endl;
        cout << " | *      * |" << endl;
        break;
    case 5:
        cout << " | *      * |" << endl;
        cout << " |      * |" << endl;
        cout << " | *      * |" << endl;
        break;
    case 6:
        cout << " | * * * |" << endl;
        cout << " |      |" << endl;
        cout << " | * * * |" << endl;
        break;
    default:
        cout << " *** Error: illegal die value ***" << endl;
        break;
}
cout << "+-----+" << endl;
}

/*
 * roll
 * Returns a pseudorandom number in the range 1...6
 */
int roll() {
    return rand() % 6 + 1;
}

/*
 * main
 * Simulates the roll of a die three times
 */
int main() {

    // Initialize the die
    initialize_die();

    // Roll the die three times
    for (int i = 0; i < 3; i++)
        show_die(roll());
}

```

In Listing 9.12 (betterdie.cpp), **main** is no longer concerned with the details of pseudorandom number generation, nor is it responsible for drawing the die. These important components of the program are now

in functions, so their details can be perfected independently from **main**.

Note how the result of the call to **roll** is passed directly as an argument to **show_die**.

9.4.5 Tree Drawing Function

Listing 9.13: treefunc.cpp

```
#include <iostream>

using namespace std;

/*
 * tree(height)
 *   Draws a tree of a given height
 *   height is the height of the displayed tree
 */
void tree(int height) {
    int row = 0;           // First row, from the top, to draw
    while (row < height) { // Draw one row for every unit of height
        // Print leading spaces
        int count = 0;
        while (count < height - row) {
            cout << " ";
            count++;
        }
        // Print out stars, twice the current row plus one:
        //   1. number of stars on left side of tree
        //   = current row value
        //   2. exactly one star in the center of tree
        //   3. number of stars on right side of tree
        //   = current row value
        count = 0;
        while (count < 2*row + 1) {
            cout << "*";
            count++;
        }
        // Move cursor down to next line
        cout << endl;
        // Change to the next row
        row++;
    }
}

/*
 * main
 *   Allows users to draw trees of various heights
 */
int main() {
    int height;           // Height of tree
    cout << "Enter height of tree: ";
    cin >> height;        // Get height from user
    tree(height);
}
```

```
}
```

Observe that the name **height** is being used as a local variable in **main** and as a formal parameter name in **tree**. There is no conflict here, and the two **height**s represent two different locations in memory. Furthermore, the fact that the statement

```
tree(height);
```

uses **main**'s **height** as an actual parameter and **height** happens to be the name as the formal parameter is simply a coincidence. During the call, the value of **main**'s **height** variable is copied into to the formal parameter in **tree** also named **height**. The compiler can keep track of which **height** is which based on where each is declared.

9.4.6 Floating-point Equality

Recall from Listing 4.2 (*imprecise.cpp*) that floating-point numbers are not mathematical real numbers; a floating-point number is finite, and is represented internally as a quantity with a binary mantissa and exponent. Just as $1/3$ cannot be represented finitely in the decimal (base 10) number system, $1/10$ cannot be represented exactly in the binary (base 2) number system with a fixed number of digits. Often, no problems arise from this imprecision, and in fact many software applications have been written using floating-point numbers that must perform precise calculations, such as directing a spacecraft to a distant planet. In such cases even small errors can result in complete failures. Floating-point numbers can and are used safely and effectively, but not without appropriate care.

To build our confidence with floating-point numbers, consider Listing 9.14 (*simplefloataddition.cpp*), which adds two double-precision floating-point numbers and checks for a given value.

Listing 9.14: simplefloataddition.cpp

```
#include <iostream>

using namespace std;

int main() {
    double x = 0.9;
    x += 0.1;
    if (x == 1.0)
        cout << "OK" << endl;
    else
        cout << "NOT OK" << endl;
}
```

All seems well judging by Listing 9.14 (*simplefloataddition.cpp*). Next, consider Listing 9.15 (*badfloatcheck.cpp*) which attempts to control a loop with a double-precision floating-point number.

Listing 9.15: badfloatcheck.cpp

```
#include <iostream>

using namespace std;

int main() {
    // Count to ten by tenths
```

```
for (double i = 0.0; i != 1.0; i += 0.1)
    cout << "i = " << i << endl;
}
```

When compiled and executed, Listing 9.15 (badfloatcheck.cpp) begins as expected, but it does not end as expected:

```
i = 0
i = 0.1
i = 0.2
i = 0.3
i = 0.4
i = 0.5
i = 0.6
i = 0.7
i = 0.8
i = 0.9
i = 1
i = 1.1
i = 1.2
i = 1.3
i = 1.4
i = 1.5
i = 1.6
i = 1.7
i = 1.8
i = 1.9
i = 2
i = 2.1
```

We expect it stop when the loop variable `i` equals 1, but the program continues executing until the user types **Ctrl-C**. We are adding 0.1, just as in Listing 9.14 (simplefloataddition.cpp), but now there is a problem. Since 0.1 cannot be represented exactly within the constraints of the double-precision floating-point representation, the repeated addition of 0.1 leads to round off errors that accumulate over time. Whereas $0.1 + 0.9$ rounded off may equal 1, 0.1 added to itself 10 times may be 1.000001 or 0.999999, neither of which is exactly 1.

Listing 9.15 (badfloatcheck.cpp) demonstrates that the `==` and `!=` operators are of questionable worth when comparing floating-point values. The better approach is to check to see if two floating-point values are *close enough*, which means they differ by only a very small amount. When comparing two floating-point numbers x and y , we essentially must determine if the absolute value of their difference is small; for example, $|x - y| < 0.00001$. The C `abs` function was introduced in Section 8.2, and we can incorporate it into an equals function, as shown in Listing 9.16 (floatequals.cpp).

Listing 9.16: floatequals.cpp

```
#include <iostream>
#include <cmath>

using namespace std;
```



```

/*
 * equals(a, b, tolerance)
 *   Returns true if a = b or |a - b| < tolerance.
 *   If a and b differ by only a small amount
 *   (specified by tolerance), a and b are considered
 *   "equal." Useful to account for floating-point
 *   round-off error.
 *   The == operator is checked first since some special
 *   floating-point values such as HUGE_VAL require an
 *   exact equality check.
 */
bool equals(double a, double b, double tolerance) {
    return a == b || fabs(a - b) < tolerance;
}

int main() {
    for (double i = 0.0; !equals(i, 1.0, 0.0001); i += 0.1)
        cout << "i = " << i << endl;
}

```

The third parameter, named **tolerance**, specifies how close the first two parameters must be in order to be considered equal. The **==** operator must be used for some special floating-point values such as **HUGE_VAL**, so the function checks for **==** equality as well. Since C++ uses short-circuit evaluation for Boolean expressions involving logical *OR* (see Section 5.2), if the **==** operator indicates equality, the more elaborate check is not performed.

You should use a function like **equals** when comparing two floating-point values for equality.

9.4.7 Multiplication Table with Functions

When we last visited our multiplication table program in Listing 7.5 (`bettertimetable.cpp`), we used nested **for** loops to display a user-specified size. We can decompose the code into functions as shown in Listing 9.17 (`timetablefunction.cpp`). Our goal is to have a collection of functions that each are very simple. We also want the program's overall structure to be logically organized.

Listing 9.17: `timetablefunction.cpp`

```

#include <iostream>
#include <iomanip>

using namespace std;

// Print the column labels for an n x n multiplication table.
void col_numbers(int n) {
    cout << "          ";
    for (int column = 1; column <= n; column++)
        cout << setw(4) << column; // Print heading for this column.
    cout << endl;
}

// Print the table's horizontal line at the top of the table
// beneath the column labels.
void col_line(int n) {

```

```

    cout << "      +";
    for (int column = 1; column <= n; column++)
        cout << "----";          // Print separator for this row.
    cout << endl;
}

// Print the title of each column across the top of the table
// including the line separator.
void col_header(int n) {
    // Print column titles
    col_numbers(n);

    // Print line separator
    col_line(n);
}

// Print the title that appears before each row of the table's
// body.
void row_header(int n) {
    cout << setw(4) << n << " |"; // Print row label.
}

// Print the line of text for row n
// This includes the row number and the
// contents of each row.
void print_row(int row, int columns) {
    row_header(row);
    for (int col = 1; col <= columns; col++)
        cout << setw(4) << row*col; // Display product
    cout << endl;                  // Move cursor to next row
}

// Print the body of the n x n multiplication table
void print_contents(int n) {
    for (int current_row = 1; current_row <= n; current_row++)
        print_row(current_row, n);
}

// Print a multiplication table of size n x n.
void timestable(int n) {
    // First, print column heading
    col_header(n);

    // Print table contents
    print_contents(n);
}

// Forces the user to enter an integer within a
// specified range first is either a minimum or maximum
// acceptable value last is the corresponding other end
// of the range, either a maximum or minimum value
// Returns an acceptable value from the user
int get_int_range(int first, int last) {
    // If the larger number is provided first,
    // switch the parameters

```

```

    if (first > last) {
        int temp = first;
        first = last;
        last = temp;
    }
    // Insist on values in the range first...last
    cout << "Please enter a value in the range "
         << first << "..." << last << ": ";
    int in_value;    // User input value
    bool bad_entry;
    do {
        cin >> in_value;
        bad_entry = (in_value < first || in_value > last);
        if (bad_entry) {
            cout << in_value << " is not in the range "
                 << first << "..." << last << endl;
            cout << "Please try again: ";
        }
    }
    while (bad_entry);
    // in_value at this point is guaranteed to be within range
    return in_value;
}

int main() {
    // Get table size from user; allow values in the
    // range 1...18.
    int size = get_int_range(1, 18);

    // Print a size x size multiplication table
    timestable(size);
}

```

In Listing 9.17 (`timestablefunction.cpp`), each function plays a very specific role; for example, `row_header` prints the label for a particular row and then prints the vertical bar that separates the row label from the body of the table. We reuse the `get_int_range` function from Listing 9.11 (`betterinputfunc.cpp`). Notice that there no longer appear to be any nested loops in the program. The nested loop is not really gone, it now is hidden by the program's functional composition. Observe that the `print_contents` function contains a loop. Each time through the loop it calls `print_row`, but `print_row` itself contains a loop. The nested iteration, therefore, is still present.

Realistically, the functional decomposition within Listing 9.17 (`timestablefunction.cpp`) is extreme. The relative simplicity of the program does not really justify eight separate functions each with such a narrow focus; however, more complex software systems are decomposed in this very manner. Not only does Listing 9.17 (`timestablefunction.cpp`) give us insight into how we can take a complicated problem and break it down into simpler, more manageable pieces, we can use the program to better understand how the function invocation process works. To see how, consider the situation where a user wishes to print a 3×3 multiplication table using Listing 9.17 (`timestablefunction.cpp`). Figure 9.5 an arrow pointing down corresponds to a function call. The labels on the down arrows represent parameters passed during the call. The up arrows indicate the return from a function. The label on an up arrow is the function's return value. Functions of type `void` have no labels on their return arrows.

Each horizontal bar in Figure 9.5 represents the duration of the program's execution that the function is *active*. The `main` function always is the first function executed in a C++ program, and in Listing 9.17

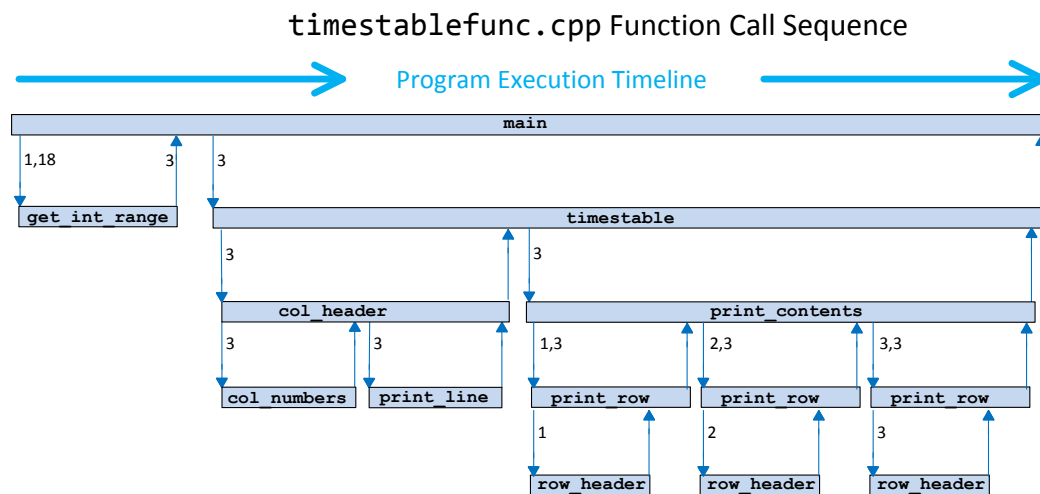


Figure 9.5: Traces the activation of the various functions in Listing 9.17 (timestablefunction.cpp) when the user enters 3 for a 3×3 multiplication table. Time flows from left to right. The horizontal bars show the lifetimes of the various functions involved.

(`timetablefunction.cpp`) `main` immediately calls the `get_int_range` function. During the lifetime of `get_int_range`'s execution notice that `main` is still active. This means any local variables it may maintain are still stored in memory and have not lost their values. This means that `main`'s `value` variable exists for the effective life of the program. This is not true for the variables used in `get_int_range`: `temp`, `in_value`, `bad_entry`, and the parameters `first` and `last`. These variables maintained by `get_int_range` appear automatically when `get_int_range` begins executing (the left end of its bar) and their space is released when `get_int_range` is finished (the right end of its bar)¹. During the printing of a 3×3 table the program calls `print_row` three times, and for each call the function's parameters `row` and `column` and local variable `col` come to life and then disappear when the function returns.

9.5 Commenting Functions

It is good practice to comment a function's definition with information that aids programmers who may need to use or extend the function. The essential information includes:

- **The purpose of the function.** The function's purpose is not always evident merely from its name. This is especially true for functions that perform complex tasks. A few sentences explaining what the function does can be helpful.
- **The role of each parameter.** The parameter names and types are obvious from the definition, but the

¹Technically, the run-time environment does not allocate the space for a local variable until after its point of declaration. For variables declared within blocks, like `temp` with the `if` body, the variable is discarded at the end of the block's execution.

purpose of a parameter may not be apparent merely from its name. It is helpful indicate the purpose of each parameter, if any.

- **The nature of the return value.** While the function may do a number of interesting things as indicated in the function's purpose, what exactly does it return to the caller? It is helpful to clarify exactly what value the function produces, if any.

Other information is often required in a commercial environment:

- **Author of the function.** Specify exactly who wrote the function. An email address can be included. If questions about the function arise, this contact information can be invaluable.
- **Date that the function's implementation was last modified.** An additional comment can be added each time the function is updated. Each update should specify the exact changes that were made and the person responsible for the update.
- **References.** If the code was adapted from another source, list the source. The reference may consist of a Web URL.

The following fragment shows the beginning of a well-commented function definition:

```
/*
 *  distance(x1, y1, x2, y2)
 *      Computes the distance between two geometric points
 *      x1 is the x coordinate of the first point
 *      y1 is the y coordinate of the first point
 *      x2 is the x coordinate of the second point
 *      y2 is the y coordinate of the second point
 *      Returns the distance between (x1,y1) and (x2,y2)
 *      Author: Joe Algori (joe@eng-sys.net)
 *      Last modified: 2010-01-06
 *      Adapted from a formula published at
 *      http://en.wikipedia.org/wiki/Distance
 */
double distance(double x1, double y1, double x2, double y2) {
    ...
}
```

From the information provided

- callers know what the function can do for them,
- callers know how to use the function,
- subsequent programmers can contact the original author if questions arise about its use or implementation,
- subsequent programmers can check the Wikipedia reference if questions arise about its implementation, and
- the quality of the algorithm may be evaluated based upon the quality of its source of inspiration (Wikipedia).

9.6 Custom Functions vs. Standard Functions

Armed with our knowledge of function definitions, we can rewrite Listing 8.1 (`computesquareroot.cpp`) so the program uses a custom square root function. Listing 9.18 (`squarerootfunction.cpp`) shows one possibility.

Listing 9.18: `squarerootfunction.cpp`

```
#include <iostream>

using namespace std;

double square_root(double x) {
    double diff;
    // Compute a provisional square root
    double root = 1.0;

    do { // Loop until the provisional root
        // is close enough to the actual root
        root = (root + x/root) / 2.0;
        cout << "root is " << root << endl;
        // How bad is the approximation?
        diff = root * root - x;
    }
    while (diff > 0.0001 || diff < -0.0001);
    return root;
}

int main() {
    double input;

    // Get value from the user
    cout << "Enter number: ";
    cin >> input;
    // Report square root
    cout << "Square root of " << input << " = "
        << square_root(input) << endl;
}
```

Is Listing 9.18 (`squarerootfunction.cpp`) better than Listing 8.2 (`standardsquareroot.cpp`) which uses the standard `sqrt` function from the `cmath` library? Generally speaking, if you have the choice of using a standard library function or writing your own custom function that provides the same functionality, choose to use the standard library routine. The advantages of using the standard library routine include:

- Your effort to produce the custom code is eliminated entirely; you can devote more effort to other parts of the application's development.
- If you write your own custom code, you must thoroughly test it to ensure its correctness; standard library code, while not immune to bugs, generally has been subjected to a complete test suite. Library code is used by many developers, and thus any lurking errors are usually exposed early; your code is exercised only by the programs you write, and errors may not become apparent immediately. If your programs are not used by a wide audience, bugs may lie dormant for a long time. Standard library routines are well known and trusted; custom code, due to its limited exposure, is suspect until it gains wider exposure and adoption.

- Standard routines are typically tuned to be very efficient; it takes a great deal of effort to make custom code efficient.
- Standard routines are well-documented; extra work is required to document custom code, and writing good documentation is hard work.

Listing 9.19 (squarerootcomparison.cpp) tests our custom square root function over a range of 1,000,000,000 floating-point values.

Listing 9.19: squarerootcomparison.cpp

```
#include <iostream>
#include <cmath>

using namespace std;

// Consider two floating-point numbers equal when
// the difference between them is very small.
bool equals(double a, double b, double tolerance) {
    return a == b || fabs(a - b) < tolerance;
}

double square_root(double x) {
    // Compute a provisional square root
    double root = 1.0;

    do { // Loop until the provisional root
        // is close enough to the actual root
        root = (root + x/root) / 2.0;
    }
    while (!equals(root*root, x, 0.0001));
    return root;
}

int main() {
    for (double d = 0.0; d < 100000.0; d += 0.0001) {
        if (!equals(square_root(d), sqrt(d), 0.001))
            cout << d << ": Expected " << sqrt(d) << ", but computed "
                << square_root(d) << endl;
    }
}
```

Listing 9.19 (squarerootcomparison.cpp) uses our **equals** function from Listing 9.16 (floatequals.cpp). The third parameter specifies a tolerance; if the difference between the first two parameters is less than the specified tolerance, the first two parameters are considered equal. Our new custom **square_root** function uses the **equals** function. The **main** function uses the **equals** function as well. Observe, however, that the tolerance used within the square root computation is smaller than the tolerance **main** uses to check the result. The **main** function, therefore, uses a less strict notion of equality.

The output of Listing 9.19 (squarerootcomparison.cpp):

```
0: Expected 0, but computed 0.0078125
0.0001: Expected 0.01, but computed 0.0116755
0.0008: Expected 0.0282843, but computed 0.0298389
0.0009: Expected 0.03, but computed 0.0313164
0.001: Expected 0.0316228, but computed 0.0327453
```

shows that our custom square root function produces results outside of **main**'s acceptable tolerance for five values. Five wrong answers out of one billion tests represents a 0.0000005% error rate. While this error rate is very small, indicates our **square_root** function is not perfect. One of values that causes the function to fail may be very important to a particular application, so our function is not trustworthy.

9.7 Summary

- The development of larger, more complex programs is more manageable when the program consists of multiple programmer-defined functions.
- Every function has one definition but can have many invocations.
- A function definition includes the function's name, parameters, return type, and body.
- A function name, like a variable name, is an identifier.
- Formal parameters are the parameters as they appear in a function's definition; actual parameters are the arguments supplied by the caller.
- Formal parameters are essentially variables local to the function; actual parameters may be variables, expressions, or literal values.
- The values of the actual parameters are copied into the formal parameters when a function is invoked.
- Callers must pass to functions the number of parameters specified in the function definition. The types of the actual parameters must be compatible with the types of the formal parameters.
- With C++'s default call-by-value parameter passing mechanism, modifying a formal parameter within a function will not affect the value of the caller's actual parameter. The formal parameter is simply a copy of the caller's actual parameter.
- Variables declared within a function are local to that function definition. Local variables cannot be seen by code outside the function definition.
- During a program's execution, local variables live only when the function is executing. When a particular function call is finished, the space allocated for its local variables is freed up.
- Programmers should supply comments for each function indicating its purpose and the role(s) of its parameter(s) and return value. Additional information about the function's author, date of last modification, and other information may be required in some situations.

9.8 Exercises

1. Is the following a legal C++ program?

```
int proc(int x) {  
    return x + 2;  
}  
  
int proc(int n) {  
    return 2*n + 1;  
}  
  
int main() {  
    int x = proc(5);  
}
```

2. Is the following a legal C++ program?

```
int proc(int x) {  
    return x + 2;  
}  
  
int main() {  
    int x = proc(5),  
        y = proc(4);  
}
```

3. Is the following a legal C++ program?

```
#include <iostream>  
  
void proc(int x) {  
    std::cout << x + 2 << std::endl;  
}  
  
int main() {  
    int x = proc(5);  
}
```

4. Is the following a legal C++ program?

```
#include <iostream>  
  
void proc(int x) {  
    std::cout << x + 2 << std::endl;  
}  
  
int main() {  
    proc(5);  
}
```

5. Is the following a legal C++ program?

```
#include <iostream>

int proc(int x, int y) {
    return 2*x + y*y;
}

int main() {
    std::cout << proc(5, 4) << std::endl;
}
```

6. Is the following a legal C++ program?

```
#include <iostream>

int proc(int x, int y) {
    return 2*x + y*y;
}

int main() {
    std::cout << proc(5) << std::endl;
}
```

7. Is the following a legal C++ program?

```
#include <iostream>

int proc(int x) {
    return 2*x*x;
}

int main() {
    std::cout << proc(5, 4) << std::endl;
}
```

8. Is the following a legal C++ program?

```
#include <iostream>

proc(int x) {
    std::cout << 2*x*x << std::endl;
}

int main() {
    proc(5);
}
```

9. The programmer was expecting the following program to print 200. What does it print instead? Why does it print what it does?

```
#include <iostream>

void proc(int x) {
```

```

    x = 2*x*x;
}

int main() {
    int num = 10;
    proc(num);
    std::cout << num << std::endl;
}

```

10. Is the following program legal since the variable **x** is used in two different places (**proc** and **main**)? Why or why not?

```

#include <iostream>

int proc(int x) {
    return 2*x*x;
}

int main() {
    int x = 10;
    std::cout << proc(x) << std::endl;
}

```

11. Is the following program legal since the actual parameter has a different name from the formal parameter (**y** vs. **x**)? Why or why not?

```

#include <iostream>

int proc(int x) {
    return 2*x*x;
}

int main() {
    int y = 10;
    std::cout << proc(y) << std::endl;
}

```

12. Complete the **distance** function started in Section 9.5. Test it with several points to convince yourself that is correct.

What happens if a caller passes too many parameters to a function?

13. What happens if a caller passes too few parameters to a function?

14. What are the rules for naming a function in C++?

15. Consider the following function definitions:

```

#include <iostream>

int fun1(int n) {
    int result = 0;
    while (n) {

```

```

        result += n;
        n--;
    }
    return result;
}

void fun2(int stars) {
    for (int i = 0; i < stars; i++)
        std::cout << "★";
    std::cout << std::endl;
}

double fun3(double x, double y) {
    return 2*x*x + 3*y;
}

bool fun4(char ch) {
    return ch >= 'A' && ch <= 'Z';
}

bool fun5(int a, int b, int c) {
    return (a <= b) ? (b <= c) : false;
}

int fun6() {
    return rand() % 2;
}

```

Examine each of the following print statements. If the statement is illegal, explain why it is illegal; otherwise, indicate what the statement will print.

- (a) `cout << fun1(5) << endl;`
- (b) `cout << fun1() << endl;`
- (c) `cout << fun1(5, 2) << endl;`
- (d) `cout << fun2(5) << endl;`
- (e) `fun2(5);`
- (f) `cout << fun3(5, 2) << endl;`
- (g) `cout << fun3(5.0, 2.0) << endl;`
- (h) `cout << fun3('A', 'B') << endl;`
- (i) `cout << fun3(5.0) << endl;`
- (j) `cout << fun3(5.0, 0.5, 1.2) << endl;`
- (k) `cout << fun4('T') << endl;`
- (l) `cout << fun4('t') << endl;`
- (m) `cout << fun4(5000) << endl;`
- (n) `fun4(5000);`
- (o) `cout << fun5(2, 4, 6) << endl;`

- (p) `cout << fun5(4, 2, 6) << endl;`
- (q) `cout << fun5(2, 2, 6) << endl;`
- (r) `cout << fun5(2, 6) << endl;`
- (s) `if (fun5(2, 2, 6))`
 `cout << "Yes"<< endl;`
 `else`
 `cout << "No"<< endl;`
- (t) `cout << fun6() << endl;`
- (u) `cout << fun6(4) << endl;`
- (v) `cout << fun3(fun1(3), 3) << endl;`
- (w) `cout << fun3(3, fun1(3)) << endl;`
- (x) `cout << fun1(fun1(fun1(3))) << endl;`
- (y) `cout << fun6(fun6()) << endl;`

Chapter 10

Managing Functions and Data

This chapter covers some additional aspects of functions in C++. Recursion, a key concept in computer science is introduced.

10.1 Global Variables

All variables to this point have been local to functions or local to blocks within the bodies of conditional or iterative statements. Local variables have some very desirable properties:

- A local variable occupies memory only when the variable is in scope. When the program execution leaves the scope of a local variable, it frees up the memory for that variable. This freed up memory is then available for use by the local variables in other functions during their invocations.
- We can use the same variable name in different functions without any conflict. The compiler derives all of its information about a local variable used within a function from the declaration of that variable in that function. The compiler will not look for the declaration of a local variable in the definition of another function. Thus, there is no way a local variable in one function can interfere with a local variable declared in another function.

A local variable is transitory, so its value is lost in between function invocations. Sometimes it is desirable to have a variable that lives as long as the program is running; that is, until the **main** function completes. In contrast to a local variable, a *global* variable is declared outside of all functions and is not local to any particular function. In fact, any function that appears in the text of the source code after the point of the global variable's declaration may legally access and/or modify that global variable.

Listing 10.1 (`globalcalculator.cpp`) is a modification of Listing 9.10 (`calculator.cpp`) that uses a global variable named **result** that is shared by several functions in the program.

Listing 10.1: `globalcalculator.cpp`

```
#include <iostream>
#include <cmath>

using namespace std;
```

```
/*
 * help_screen
 *     Displays information about how the program works
 *     Accepts no parameters
 *     Returns nothing
 */
void help_screen() {
    cout << "Add: Adds two numbers" << endl;
    cout << "     Example: a 2.5 8.0" << endl;
    cout << "Subtract: Subtracts two numbers" << endl;
    cout << "     Example: s 10.5 8.0" << endl;
    cout << "Print: Displays the result of the latest operation"
        << endl;
    cout << "     Example: p" << endl;
    cout << "Help: Displays this help screen" << endl;
    cout << "     Example: h" << endl;
    cout << "Quit: Exits the program" << endl;
    cout << "     Example: q" << endl;
}

/*
 * menu
 *     Display a menu
 *     Accepts no parameters
 *     Returns the character entered by the user.
 */
char menu() {
    // Display a menu
    cout << "=== A)dd S)ubtract P)rint H)elp Q)uit ===" << endl;
    // Return the char entered by user
    char ch;
    cin >> ch;
    return ch;
}

/*
 * Global variables used by several functions
 */
double result = 0.0, arg1, arg2;

/*
 * get_input
 *     Assigns the globals arg1 and arg2 from user keyboard
 *     input
 */
void get_input() {
    cin >> arg1 >> arg2;
}

/*
 * report
 *     Reports the value of the global result
 */
void report() {
    cout << result << endl;
}
```

```
}

/*
 * add
 *   Assigns the sum of the globals arg1 and arg2
 *   to the global variable result
 */

void add() {
    result = arg1 + arg1;
}

/*
 * subtract
 *   Assigns the difference of the globals arg1 and arg2
 *   to the global variable result
 */

void subtract() {
    result = arg1 - arg2;
}

/*
 * main
 *   Runs a command loop that allows users to
 *   perform simple arithmetic.
 */
int main() {
    bool done = false; // Initially not done
    do {
        switch (menu()) {
            case 'A': // Addition
            case 'a':
                get_input();
                add();
                report();
                break;
            case 'S': // Subtraction
            case 's':
                get_input();
                subtract();
                report();
            case 'P': // Print result
            case 'p':
                report();
                break;
            case 'H': // Display help screen
            case 'h':
                help_screen();
                break;
            case 'Q': // Quit the program
            case 'q':
                done = true;
                break;
        }
    }
}
```



```

    }
    while (!done);
}

```

Listing 10.1 (globalcalculator.cpp) uses global variables **result**, **arg1**, and **arg2**. These names no longer appear in the **main** function. These global variables are accessed and/or modified in four different functions: **get_input**, **report**, **add**, and **subtract**.

When in the course of translating the statements within a function to machine language, the compiler resolves a variable it encounters as follows:

- If the variable has a local declaration (that is, it is a local variable or parameter), the compiler will use the local variable or parameter, even if a global variable of the same name exists. Local variables, therefore, take precedence over global variables. We say the local declaration *hides* the global declaration in the scope of the local variable.
- If the variable has no local declaration but is declared as a global variable, the compiler will use the global variable.
- If the variable has neither a local declaration nor a global declaration, then the variable is undefined, and its use is an error.

In the situation where a local variable hides a global variable of the same name, there is a way to access both the local variable and like-named global variable within the local variable's scope. Suppose a program has a global variable named **x** and a function with a local variable named **x**. The statement

```
x = 10;
```

within the scope of the local variable will assign the local **x**. The following statement will assign the global variable **x** in the scope of the local variable of the same name:

```
::x = 10;
```

The `::` operator is called the *scope resolution* operator. This special syntax may be used whenever a global variable is accessed within a function, but usually it only used when necessary to access a hidden global variable.

If the value of a local variable is used by a statement before that variable has been given a value, either through initialization or assignment, the compiler will issue a warning. For example, the Visual C++ compiler will issue a warning about code in the following function:

```

void uninitialized() {
    int x;    // Declare the variable
    cout << x; // Then use it
}

```

The warning is

warning C4700: uninitialized local variable 'x' used

(The only way to avoid this warning in Visual C++ is to turn off *all* warnings.) A local variable has an undefined value after it is declared without being initialized. Its value should not be used until it has been properly assigned. Global variables, however, do not need to be initialized before they are used. Numeric

global variables are automatically assigned the value zero. This means the initialization of **result** in Listing 10.1 (globalcalculator.cpp) is superfluous, since **result** will be assigned zero automatically. Boolean global variables are automatically assigned zero as well, as zero represents **false** (see Section 5.1).

When it is acceptable to use global variables, and when is it better to use local variables? In general, local variables are preferred to global variables for several reasons:

- When a function uses local variables exclusively and performs no other input operations (like using the **cin** object), its behavior is influenced only by the parameters passed to it. If a non-local variable appears, the function's behavior is affected by every other function that can modify that non-local variable. As a simple example, consider the following trivial function that appears in a program:

```
int increment(int n) {
    return n + 1;
}
```

Can you predict what the following statement within that program will print?

```
cout << increment(12) << endl;
```

If your guess is 13, you are correct. The **increment** function simply returns the result of adding one to its argument. The **increment** function behaves the same way each time it is called with the same argument.

Next, consider the following three functions that appear in some program:

```
int process(int n) {
    return n + m; // m is a global integer variable
}

void assign_m() {
    m = 5;
}

void inc_m() {
    m++;
}
```

Can you predict the what the following statement within the program will print?

```
cout << process(12) << endl;
```

We cannot predict what this statement in isolation will print. The following scenarios all produce different results:

```
assign_m();
cout << process(12) << endl;
```

prints 17,

```
m = 10;
cout << process(12) << endl;
```

prints 22,

```

m = 0;
inc_m();
inc_m();
cout << process(12) << endl;

```

prints 14, and

```

assign_m();
inc_m();
inc_m();
cout << process(12) << endl;

```

prints 19. The identical printing statements print different values depending on the cumulative effects of the program's execution up to that point.

It may be difficult to locate an error if that function fails because it may be the fault of *another* function that assigned an incorrect value to the global variable. The situation may be more complicated than the simple examples above; consider:

```

assign_m();
.
.   /* 30 statements in between, some of which may change a,
.       b, and m */
.
if (a < 2 && b <= 10)
    m = a + b - 100;
.
.   /* 20 statements in between, some of which may change m
.
cout << process(12) << endl;

```

- A nontrivial program that uses non-local variables will be more difficult for a human reader to understand than one that does not. When examining the contents of a function, a non-local variable requires the reader to look elsewhere (outside the function) for its meaning:

```

// Linear function
double f(double x) {
    return m*x + b;
}

```

What are **m** and **b**? How, where, and when are they assigned or re-assigned?

- A function that uses only local variables can be tested for correctness in isolation from other functions, since other functions do not affect the behavior of this function. This function's behavior is only influenced only by its parameters, if it has any.

The exclusion of global variables from a function leads to *functional independence*. A function that depends on information outside of its scope to correctly perform its task is a dependent function. When a function operates on a global variable it depends on that global variable being in the correct state for the function to complete its task correctly. Nontrivial programs that contain many dependent functions are more difficult debug and extend. A truly independent function that use no global variables and uses no programmer-defined functions to help it out can be tested for correctness in isolation. Additionally, an independent function can be copied from one program, pasted into another program, and work without modification. Functional independence is a desirable quality.

Unlike global variables, global constants are generally safe to use. Code within functions that use global constants are dependent on those constants, but since constants cannot be changed, developers need not worry that other functions that have access to the global constants might disturb their values.

The use of global constants within functions has drawbacks in terms of program maintenance. As a program evolves, code is added and removed. If a global constant is removed or its meaning changes during the course of the program's development, the change will affect any function using the global constant.

Listing 10.2 (digitaltimer.cpp) uses global constants to assist the display of a digital timer.

Listing 10.2: digitaltimer.cpp

```
#include <iostream>
#include <iomanip>
#include <ctime>

using namespace std;

// Some conversions from seconds
const clock_t SEC_PER_MIN = 60,           // 60 sec = 1 min
              SEC_PER_HOUR = 60 * SEC_PER_MIN, // 60 min = 1 hr
              SEC_PER_DAY = 24 * SEC_PER_HOUR; // 24 hr = 24 hr

/*
 * print_time
 *   Displays the time in hr:min:sec format
 *   seconds is the number of seconds to display
 */

void print_time(clock_t seconds) {
    clock_t hours = 0, minutes = 0;

    // Prepare to display time =====
    cout << endl;
    cout << "      ";

    // Compute and display hours =====
    hours = seconds/SEC_PER_HOUR;
    cout << setw(2) << setfill('0') << hours << ":";

    // Remove the hours from seconds
    seconds %= SEC_PER_HOUR;

    // Compute and display minutes =====
    minutes = seconds/SEC_PER_MIN;
    cout << setw(2) << setfill('0') << minutes << ":";
    // Remove the minutes from seconds
    seconds %= SEC_PER_MIN;

    // Compute and display seconds =====
    cout << setw(2) << setfill('0') << seconds << endl;
}

int main() {
    clock_t start = clock();           // Record starting time
    clock_t elapsed = (clock() - start)/CLOCKS_PER_SEC, // Elapsed time in sec.
```

```

        previousElapsed = elapsed;
    // Counts up to 24 hours (1 day), then stops
    while (elapsed < SEC_PER_DAY) {
        // Update the display only every second
        if (elapsed - previousElapsed >= 1) {
            // Remember when we last updated the display
            previousElapsed = elapsed;
            print_time(elapsed);
        }
        // Update elapsed time
        elapsed = (clock() - start)/CLOCKS_PER_SEC;
    }
}

```

In Listing 10.2 (digitaltimer.cpp):

- The **main** function controls the time initialization and update and deals strictly in seconds. The logic in **main** is kept relatively simple.
- The code that extracts the hours, minutes, and seconds from a given number of seconds is isolated in **print_time**. The **print_time** function can now be used anytime a value in seconds needs to be expressed in the *hours : minutes : seconds* format.
- The second conversion constants (**SEC_PER_HOUR**, **SEC_PER_MIN**, and **SEC_PER_DAY**) are global constants so that both functions can access them if necessary. In this case the functions use different constants, but it makes sense to place all the conversion factors in one place.

Since the two functions divide the responsibilities in a way that each can be developed independently, the design is cleaner and the program is easier to develop and debug. The use of constants ensures that the shared values cannot be corrupted by either function.

The exclusion from a function's definition of global variables and global constants does not guarantee that it will always produce the same results given the same parameter values; consider

```

int compute(int n) {
    int favorite;
    cout << "Please enter your favorite number: ";
    cin >> favorite;
    return n + favorite;
}

```

The **compute** function avoid globals, yet we cannot predict the value of the expression **compute(12)**. Recall the **increment** function from above:

```

int increment(int n) {
    return n + 1;
}

```

Its behavior is totally predictable. Furthermore, **increment** does not modify any global variables, meaning it cannot in any way influence the overall program's behavior. We say that **increment** is a *pure function*. A pure function cannot perform any input or output (for example, use the **cout** and **cin** objects), nor may it use global variables. While **increment** is pure, the **compute** function is impure. The following function is impure also, since it performs output:

```
int increment_and_report(int n) {
    cout << "Incrementing " << n << endl;
    return n + 1;
}
```

A pure function simply computes its return value and has no other observable side effects.

A function that uses only pure functions and otherwise would be considered pure is itself a pure function; for example:

```
int double_increment(int n) {
    return increment(n) + 1;
}
```

`double_increment` is a pure function since `increment` is pure; however, `double_increment_with_report`:

```
int double_increment_with_report(int n) {
    return increment_and_report(n) + 1;
}
```

is not a pure function since it calls `increment_and_report` which is impure.

10.2 Static Variables

Space in the computer's memory for local variables and function parameters is allocated at run time when the function begins executing. When the function is finished and returns, the memory used for the function's local variables and parameters is freed up for other purposes. If a function is never called, the variable's local variables and parameters will never occupy the computer's memory.

Because a function's locals are transitory, a function cannot ordinarily retain any information between calls. C++ provides a way in which a variable local to a function can be retained in between calls. Listing 10.3 (`counter.cpp`) shows how declaring a local variable `static` allows it to remain in the computer's memory for the duration of the program's execution.

Listing 10.3: counter.cpp

```
#include <iostream>
#include <iomanip>

using namespace std;

/*
 * count
 *     Keeps track of a count.
 *     Returns the current count.
 */

int count() {
    // cnt's value is retained between calls because it
    // is declared static
    static int cnt = 0;
    return ++cnt;    // Increment and return current count
}
```

```
int main() {
    // Count to ten
    for (int i = 0; i < 10; i++)
        cout << count() << ' ';
    cout << endl;
}
```

In Listing 10.3 (counter.cpp), the `count` function is called 10 times. Each time it returns a tally of the number of times it has been called:

```
1 2 3 4 5 6 7 8 9 10
```

By contrast, if you remove the word `static` from Listing 10.3 (counter.cpp), recompile it, and rerun it, it prints:

```
1 1 1 1 1 1 1 1 1 1
```

because new memory is allocated for the `cnt` variable each time `count` is called.

The local declaration

```
static int cnt = 0;
```

allocates space for `cnt` and assigns zero to it once—at the beginning of the program's execution. The space set aside for `cnt` is not released until the program finishes executing.

Recall Listing 9.7 (evenbetterprompt.cpp) that included the following function:

```
int prompt(int n) {
    int value;
    cout << "Please enter integer #" << n << ": ";
    cin >> value;
    return value;
}
```

The caller, `main`, used `prompt` function as follows:

```
int value1, value2, sum;
cout << "This program adds together two integers." << endl;
value1 = prompt(1);    // Call the function
value2 = prompt(2);    // Call the function again
sum = value1 + value2;
```

The first call to `prompt` prints the message

```
Please enter integer #1:
```

and awaits the user's input. The second call prints the message

```
Please enter integer #2:
```

Another caller might use `prompt` within a loop like this:

```
int sum = 0;
for (int i = 1; i < 10; i++)
    sum += prompt(i);
```

Notice that it is the caller's responsibility to keep track of the proper number to pass to `prompt`. The caller may make a mistake and pass the wrong number or may not want to manage such details. It would be better to move the responsibility of tracking input count to `prompt`. `static` variables make that possible, as Listing 10.4 (`promptwithstatic.cpp`)

Listing 10.4: `promptwithstatic.cpp`

```
#include <iostream>

using namespace std;

/*
 * prompt requests an integer from the user and
 * keeps track of the cumulative number of entries.
 * Returns the value entered by the user.
 */
int prompt() {
    static int count = 0;
    int value;
    cout << "Please enter integer #" << ++count << ": ";
    cin >> value;
    return value;
}

int main() {
    int value1, value2, sum;
    cout << "This program adds together two integers." << endl;
    value1 = prompt();    // Call the function
    value2 = prompt();    // Call the function again
    sum = value1 + value2;
    cout << value1 << " + " << value2 << " = " << sum << endl;
}
```

Listing 10.4 (`promptwithstatic.cpp`) behaves just like Listing 9.7 (`evenbetterprompt.cpp`) but in the new version `main` does not have to keep track of the number of user entries.

Local `static` variables were inherited from the C programming language, but their need has diminished with the introduction of objects in C++ (see Chapter 14). Functions with `static` variables provide a way to implement executable code with persistent state. Objects provide a more natural and more flexible way to achieve the same effect.

10.3 Overloaded Functions

In C++, a program can have multiple functions with the same name. When two or more functions within a program have the same name, the function is said to be *overloaded*. The functions must be different somehow, or else the compiler would not know how to associate a call with a particular function definition. The compiler identifies a function by more than its name; a function is uniquely identified by its *signature*. A function signature consists of the function's name and its parameter list. In the parameter list, only the types of the formal parameters are important, not their names. If the parameter types do not match exactly, both in number and position, then the function signatures are different. Consider the following overloaded functions:

1. `void f() { /* ... */ }`

This version has no parameters, so its signature differs from all the others which each have at least one parameter.

2. `void f(int x) { /* ... */ }`

This version differs from Version 3, since its single parameter is an `int`, not a `double`.

3. `void f(double x) { /* ... */ }`

This version differs from Version 2, since its single parameter is a `double`, not an `int`.

4. `void f(int x, double y) { /* ... */ }`

This version differs from Version 5 because, even though Versions 4 and 5 have the same number of parameters with the same types, the order of the types is different.

5. `void f(double x, int y) { /* ... */ }`

See the comments for Version 4.

Overloaded functions are a convenience for programmers. If overloaded functions were not allowed (many programming languages do not support function overloading), new function names must be created for different functions that perform basically the same task but accept different parameter types. It is better for the programmer to choose the same name for the similar functions and let the compiler properly resolve the differences. Overloading becomes a more important issue for constructors, special functions called during object creation (Chapter 14).

10.4 Recursion

The *factorial* function is widely used in combinatorial analysis (counting theory in mathematics), probability theory, and statistics. The factorial of n is often expressed as $n!$. Factorial is defined for non-negative integers as

$$n! = n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdots 2 \cdot 1$$

and $0!$ is defined to be 1. Thus $6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$. Mathematicians precisely define factorial in this way:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{otherwise.} \end{cases}$$

This definition is *recursive* since the `!` function is being defined, but `!` is also used in the definition. A C++ function can be defined recursively as well. Listing 10.5 (`factorialtest.cpp`) includes a factorial function that exactly models the mathematical definition.

Listing 10.5: `factorialtest.cpp`

```
#include <iostream>

using namespace std;

/*
 * factorial(n)
 * Computes n!
 * Returns the factorial of n.
 */
int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    // Try out the factorial function
    cout << " 0! = " << factorial(0) << endl;
    cout << " 1! = " << factorial(1) << endl;
    cout << " 6! = " << factorial(6) << endl;
    cout << "10! = " << factorial(10) << endl;
}
```

Observe that the `factorial` function in Listing 10.5 (`factorialtest.cpp`) uses no loop to compute its result. The `factorial` function simply calls itself. The call `factorial(6)` is computed as follows:

```
factorial(6) = 6 * factorial(5)
             = 6 * 5 * factorial(4)
             = 6 * 5 * 4 * factorial(3)
             = 6 * 5 * 4 * 3 * factorial(2)
             = 6 * 5 * 4 * 3 * 2 * factorial(1)
             = 6 * 5 * 4 * 3 * 2 * 1 * factorial(0)
             = 6 * 5 * 4 * 3 * 2 * 1 * 1
             = 6 * 5 * 4 * 3 * 2 * 1
             = 6 * 5 * 4 * 3 * 2
             = 6 * 5 * 4 * 6
             = 6 * 5 * 24
             = 6 * 120
             = 720
```

Note that the `factorial` function can be slightly optimized by changing the `if`'s condition from `(n == 0)` to `(n < 2)`. This change results in a function execution trace that eliminates two function calls at the end:

```
factorial(6) = 6 * factorial(5)
             = 6 * 5 * factorial(4)
             = 6 * 5 * 4 * factorial(3)
```

```

= 6 * 5 * 4 * 3 * factorial(2)
= 6 * 5 * 4 * 3 * 2 * 1
= 6 * 5 * 4 * 3 * 2
= 6 * 5 * 4 * 6
= 6 * 5 * 24
= 6 * 120
= 720

```

Figure 10.1 shows the call sequence for `factorial(6)` invoked from with a `main` function.

`factorial(6)` Function Call Sequence (called from `main`)

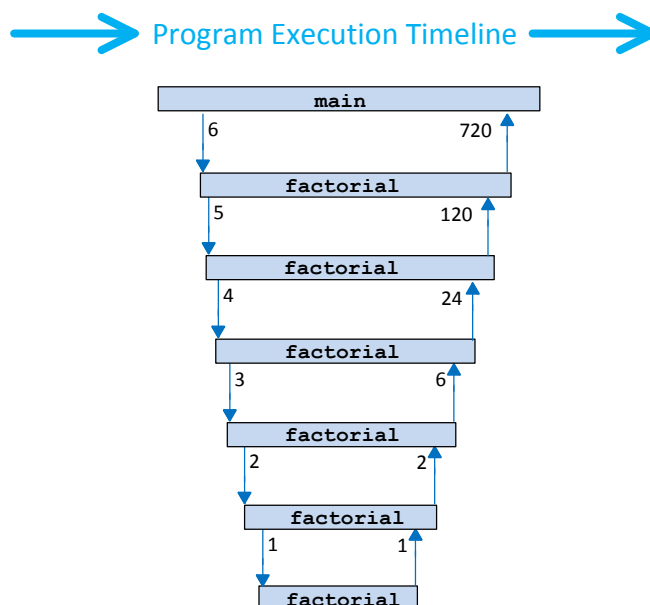


Figure 10.1: Traces the function activations of the recursive function `factorial` when called from `main` with an argument of 6. The arrows into an activation bar indicates the argument passed by the caller; the arrows out show the value passed back to the caller. The length of a bar represents the time during which that invocation of the function is active.

A correct simple recursive function definition is based on four key concepts:

1. The function must optionally call itself within its definition; this is the *recursive case*.
2. The function must optionally *not* call itself within its definition; this is the *base case*.
3. Some sort of conditional execution (such as an `if/else` statement) selects between the recursive case and the base case based on one or more parameters passed to the function.

4. Each invocation that does not correspond to the base case must call itself with parameter(s) that move the execution closer to the base case. The function's recursive execution must converge to the base case.

Each recursive invocation must bring the function's execution closer to its base case. The **factorial** function calls itself in the **else** clause of the **if/else** statement. Its base case is executed if the condition of the **if** statement is true. Since the factorial is defined only for non-negative integers, the initial invocation of **factorial** must be passed a value of zero or greater. A zero parameter (the base case) results in no recursive call. Any other positive parameter results in a recursive call with a parameter that is closer to zero than the one before. The nature of the recursive process progresses towards the base case, upon which the recursion terminates.

We can easily write a non-recursive factorial function, as Listing 10.6 (`nonrecursfact.cpp`) shows.

Listing 10.6: `nonrecursfact.cpp`

```
#include <iostream>

using namespace std;

/*
 * factorial(n)
 *   Computes n!
 *   Returns the factorial of n.
 */
int factorial(int n) {
    int product = 1;
    for (int i = n; i > 0; i--)
        product *= i;
    return product;
}

int main() {
    // Try out the factorial function
    cout << " 0! = " << factorial(0) << endl;
    cout << " 1! = " << factorial(1) << endl;
    cout << " 6! = " << factorial(6) << endl;
    cout << "10! = " << factorial(10) << endl;
}
```

Which **factorial** function is better, the recursive or non-recursive version? Generally, if the same basic algorithm is being used by both the recursive and non-recursive functions, then the non-recursive function will be more efficient. A function call is a relatively expensive operation compared to a variable assignment or comparison. The body of the non-recursive **factorial** function invokes no functions, but the recursive version calls a function—it calls itself—during all but the last recursive invocation. The iterative version of **factorial** is therefore more efficient than the recursive version.

Even though the iterative version of the factorial function is technically more efficient than the recursive version, on most systems you could not tell the difference. The reason is the factorial function “grows” fast, meaning it returns fairly large results for relatively small arguments. For example, with 32-bit integers, the 13! is the largest integer value that can be computed. The execution time difference between the two versions is negligible on most systems.

Recall the **gcd** functions from Section 9.2. It computed the greatest common divisor (also known as greatest common factor) of two integer values. It works, but it is not very efficient. A better algorithm is

used in Listing 10.7 (gcd.cpp). It is based on one of the oldest algorithms known, developed by Euclid around 300 B.C.

Listing 10.7: gcd.cpp

```
#include <iostream>

using namespace std;

/*
 * gcd(m, n)
 *   Uses Euclid's method to compute
 *   the greatest common divisor
 *   (also called greatest common
 *   factor) of m and n.
 *   Returns the GCD of m and n.
 */
int gcd(int m, int n) {
    if (n == 0)
        return m;
    else
        return gcd(n, m % n);
}

int iterative_gcd(int num1, int num2) {
    // Determine the smaller of num1 and num2
    int min = (num1 < num2) ? num1 : num2;
    // 1 is definitely a common factor to all ints
    int largestFactor = 1;
    for (int i = 1; i <= min; i++)
        if (num1 % i == 0 && num2 % i == 0)
            largestFactor = i; // Found larger factor
    return largestFactor;
}

int main() {
    // Try out the gcd function
    for (int num1 = 1; num1 <= 100; num1++)
        for (int num2 = 1; num2 <= 100; num2++)
            cout << "gcd(" << num1 << ", " << num2
                << ") = " << gcd(num1, num2) << endl;
}
```

Note that this **gcd** function is recursive. The algorithm it uses is much different from our original iterative version. Because of the difference in the algorithms, this recursive version is actually much more efficient than our original iterative version. A recursive function, therefore, cannot be dismissed as inefficient just because it is recursive.

Listing 10.8 (histobar.cpp) provides another example of a recursive function. The **segments1** function uses iteration to draw segments that make up a bar that could be part of a histogram. The **segments2** function does that same thing, except it uses recursion.

Listing 10.8: histobar.cpp

```
#include <iostream>
```

```

using namespace std;

// Draws a bar n segments long
// using iteration.
void segments1(int n) {
    while (n > 0) {
        cout << "*";
        n--;
    }
    cout << endl;
}

// Draws a bar n segments long
// using recursion.
void segments2(int n) {
    if (n > 0) {
        cout << "*";
        segments2(n - 1);
    }
    else
        cout << endl;
}

int main() {
    segments1(3);
    segments1(10);
    segments1(0);
    segments1(5);
    cout << "-----" << endl;
    segments2(3);
    segments2(10);
    segments2(0);
    segments2(5);
}

```

The output of Listing 10.8 (histobar.cpp) shows that the two functions produce the same results:

```

***
*****

*****
-----

***
*****

*****

```

10.5 Making Functions Reusable

A function definition packages in one place functionality that we can exercise (call) from many different places within a program. Thus far, however, we have not seen how we can reuse easily function definitions in other *programs*. For example, our `is_prime` function in Listing 9.9 (`primefunc.cpp`) works well within Listing 9.9 (`primefunc.cpp`), and we could put it to good use in other programs that need to test primality (encryption software, for example, makes heavy use of prime numbers). We could use the copy-and-paste feature of our favorite text editor to copy the `is_prime` function definition from Listing 9.9 (`primefunc.cpp`) into the new encryption program we are developing.

It is possible to reuse a function in this way only if the function definition does not use any programmer-defined global variables, programmer-defined global constants, nor other programmer-defined functions. If a function does use any of these programmer-defined external entities, they must be included for the function to compile. Said another way, the code in the function definition ideally will use only local variables and parameters. Such a function is a truly independent function can be reused easily in multiple programs.

The notion of copying source code from one program to another is not ideal, however. It is too easy for the copy to be incomplete or for some other error to be introduced during the copy. Furthermore, such code duplication is wasteful. If 100 programs on a particular system all need to use the `is_prime` function, under this scheme they must all include the `is_prime` code. This redundancy wastes space. Finally, in perhaps the most compelling demonstration of the weakness of this copy-and-paste approach, what if a bug is discovered in the `is_prime` function that all 100 programs are built around? When the error is discovered and fixed in one program, the other 99 programs will still contain the bug. Their source code must be updated, and they each then must be recompiled. The situation would be the same if a correct `is_prime` function were updated to be made more efficient. The problem is this: all the programs using `is_prime` define their *own* `is_prime` function; while the function definitions are meant to be identical, there is no mechanism tying all these common definitions together. We would really like to reuse the function as is without copying it.

Fortunately, C++ provides a way to develop functions in separate files and combine the code from these independently developed functions into one program. We can compile the source files separately, and the linker can combine the compiled code into an executable. What we need is a way for the compiler to verify that the calling code in one source file is correctly invoking the function defined in another source file.

10.9 provides the first step in making a reusable `is_prime` function.

Listing 10.9: `prime.h`

```
bool is_prime(int);
```

The simple one-line code in 10.9 can be stored in a file called `prime.h`. In Visual Studio, you simply add a new item to your project specified as a header file, name it `prime.h`, and you are ready to type the one line of code into the newly created file. This file contains the prototype for our `is_prime` function (see our earlier discussion of function prototypes in Section 8.1). Caller code that intends to use our `is_prime` function must `#include` this file so that compiler can check to see if the caller is using our `is_prime` function properly. An attempt, for example, to pass two arguments to `is_prime` would result in a compiler error since the prototype specifies a single integer argument.

A second file, which could be named `prime.cpp`, appears in Listing 10.10 (`prime.cpp`).

Listing 10.10: `prime.cpp`

```
// prime.cpp
#include <cmath>      // Needed for sqrt
```

```

#include "prime.h"    // is_prime prototype

/*
 * is_prime(n)
 *   Determines the primality of a given value
 *   n an integer to test for primality
 *   Returns true if n is prime; otherwise, returns false
 */
bool is_prime(int n) {
    bool result = true;    // Provisionally, n is prime
    double r = n, root = sqrt(r);
    // Try all possible factors from 2 to the square
    // root of n
    for (int trial_factor = 2; result && trial_factor <= root; trial_factor++)
        result = (n % trial_factor != 0);
    return result;
}

```

The code in Listing 10.10 (prime.cpp) is placed in prime.cpp, a different file from prime.h. It provides an implementation of the `is_prime` function. Notice the `#include` preprocessor directive in Listing 10.10 (prime.cpp) that references the file prime.h. While not required, this serves as a good check to see if the implementation code in this file is faithful to the prototype specified in prime.h. This prime.cpp file is compiled separately, and the compiler will report an error if the implementation of `is_prime` disagrees with the information in the header file.

Note that the file prime.cpp does not contain a `main` function; `main` will appear in another file. Also observe that we do not need to `#include` the `iostream` header, since the `cout` and `cin` objects are not used anywhere in this file. The `cmath` header is `#included` since `is_prime` uses the `sqrt` function.

The final piece of the program is the calling code. Suppose Listing 10.11 (primetester.cpp) is added to the project in a file named primetester.cpp.

Listing 10.11: primetester.cpp

```

#include <iostream>
#include "prime.h"

using namespace std;

/*
 * main
 *   Tests for primality each integer from 2
 *   up to a value provided by the user.
 *   If an integer is prime, it prints it;
 *   otherwise, the number is not printed.
 */
int main() {
    int max_value;
    cout << "Display primes up to what value? ";
    cin >> max_value;
    for (int value = 2; value <= max_value; value++)
        if (is_prime(value)) // See if value is prime
            cout << value << " "; // Display the prime number
    cout << endl; // Move cursor down to next line
}

```


Note that the file `primetester.cpp` uses a function named `is_prime`, but its definition is missing. The definition for `is_prime` is found, of course, in `prime.cpp`.

Visual Studio will automatically compile and link the `.cpp` files when it builds the project. Each `.cpp` is compiled independently on its own merits.

If you are using the Visual Studio Command Line tool, in order to build the program you would type

```
cl /EHsc /Za /W3 primetester.cpp prime.cpp
```

The executable file's name is determined by the name of the first source file listed, in this case `primetester`.

If you are using the GCC tools instead of Visual Studio, in order to make the executable program named `primetester` (or `primetester.exe` under the Microsoft Windows version of the GCC tools), you would issue the command

```
g++ -o primetester -Wall -O1 -std=c++11 prime.cpp primetester.cpp
```

The GNU C++ compiler will separately compile the two source files producing two machine language object files. The linker will then use those object files to create the executable. When the linker has created the executable, it automatically deletes the two object files.

The `is_prime` function is now more readily available to other programs. If it becomes an often used function in many programs, it can be compiled and placed into a special file called a *library*. In this form it need not be recompiled each time a new program is built that requires it. If our `is_prime` is placed in a dynamic library, its code can be loaded and linked at run time and shared by many executing programs. We do not cover library creation in this text.

In Listing 8.7 (`timeit.cpp`), Listing 8.8 (`measureprimespeed.cpp`), and Listing 10.2 (`digitaltimer.cpp`) we used the `clock` function from the `<ctime>` library to measure the elapsed time of sections of various executing programs. In each of these programs the programmer must be aware of the `clock_t` type and `CLOCKS_PER_SEC` constant, both defined in the `<ctime>` header file. Furthermore, the programmer must use the `clock` function properly and correctly perform some arithmetic and include a messy type cast operation. Armed with our knowledge of global variables (Section 10.1) and separate compilation, we can provide a better programmer interface to the lower-level timing functions provided to the C library.

Listing 10.12 (`timermodule.h`) specifies some convenient timing functions.

Listing 10.12: `timermodule.h`

```
// Header file timermodule.h

// Reset the timer so it reads 0 seconds
void reset_timer();

// Start the timer. The timer will
// begin measuring elapsed time.
void start_timer();

// Stop the timer. The timer will
// retain the current elapsed time, but
```

```

// it will not measure any time while
// it is stopped.
void stop_timer();

// Return the cumulative time (in seconds)
// kept by the timer since it last was reset
double elapsed_time();

```

Listing 10.13 (timermodule.cpp) implements the functions declared in Listing 10.12 (timermodule.h).

Listing 10.13: timermodule.cpp

```

// File timermodule.h
// Implements the program timer module

#include <ctime>

// Global variable that keeps track of the
// elapsed time.
double elapsed;

// Global variable that counts the number of
// clock ticks since the most recent start time.
clock_t start_time;

// Global flag that indicates whether or not the
// timer is running.
bool running;

// Reset the timer so it reads 0 seconds
void reset_timer() {
    elapsed = 0.0;
    running = false; // Ensure timer is not running
}

// Start the timer. The timer will
// begin measuring elapsed time.
// Starting the timer if it already is running has no effect
void start_timer() {
    // Starting an already running timer has no effect
    if (!running) {
        running = true; // Note that the timer is running
        start_time = clock(); // Record start time
    }
}

// Stop the timer. The timer will
// retain the current elapsed time, but
// it will not measure any time while
// it is stopped.
// Stopping the timer if it is not currently running has no
// effect.
void stop_timer() {
    // Stopping a non-running timer has no effect
    if (running) {

```

```

        clock_t stop_time = clock(); // Record stop time
        running = false;           // Stop the clock
        // Add to the elapsed time how long it has been since we last
        // started the timer
        elapsed += static_cast<double>((stop_time - start_time))
                  / CLOCKS_PER_SEC;
    }
}

// Return the cumulative running time (in seconds)
// kept by the timer since it last was reset
double elapsed_time() {
    if (running) { // Compute time since last reset
        clock_t current_time = clock(); // Record current time
        return elapsed + static_cast<double>((current_time - start_time))
                      / CLOCKS_PER_SEC;
    }
    else // Timer stopped; elapsed already computed in stop_timer
        return elapsed;
}

```

Observe that the code in Listing 10.13 (timermodule.cpp) allows client code to stop the timer and restart it later without losing any previously accumulated time. The implementation uses three global variables—**elapsed**, **start_time**, and **running**—to maintain the state of the timer. One or more of these global variables is influenced by three functions—**start_timer**, **stop_timer**, **reset_time**. The fourth function returns the value of the **elapsed** variable.

Listing 10.14 (bettertimeit.cpp) simplifies Listing 8.7 (timeit.cpp) with our new timer module.

Listing 10.14: bettertimeit.cpp

```

#include <iostream>
#include "timermodule.h" // Out timer module

using namespace std;

int main() {
    char letter;
    cout << "Enter a character: ";
    start_timer(); // Start timing
    cin >> letter;
    stop_timer(); // Stop timing
    cout << elapsed_time() << " seconds" << endl;
}

```

The code within Listing 10.14 (bettertimeit.cpp) is simpler and cleaner than the code in Listing 8.7 (timeit.cpp). All traces of the **clock_t** type and the messy arithmetic and casting are gone. The timer module provides a simple interface to callers that hides the details on how the timing actually happens.

Despite the ease of use of our timer module, it has a severe limitation. Suppose you wish to measure how long it takes for a function to execute and also, during that function's execution, separately time a smaller section of code within that function. When the function is finished executing, you would like to know how long it took the function to do its job and how long a portion of its code took to execute. We essentially need two independent timers, but with our timer module it is not possible to conduct simultaneously more than one timing. We will see a far superior way to model a program execution timer in

Section ??.

 We will use objects to enable us to maintain as many simultaneous stopwatches as we need.

10.6 Pointers

Ordinarily we need not be concerned about where variables live in the computer's memory during a program's execution. The compiler generates machine code that takes care of those details for us. Some systems software like operating systems and device drivers need to access specific memory locations in order to interoperate with hardware. Systems programmers, therefore, must be able to write code that can access such lower-level detail. Developers of higher-level applications sometimes need to access the address of variables to achieve specialized effects.

Each byte in a computer's memory is numbered with a unique address. The first address is 0, and the locations are numbered sequentially up to some maximum value allowed by the operating system and hardware. A C++ variable is stored in memory, so each variable lives at a particular address.

If **x** is a variable of any type, the expression

&x

represents the *address* of **x**. The expression

&x

is really just a number—the numeric address of the variable's memory location, but except for some situations in systems programming, programmers rarely need to treat this value as a number. The **&** operator is called the *address of* operator.

While an address is really just a non-negative integer value, C++ uses a special notation when dealing with addresses. In the following declaration

```
int *p;
```

the variable **p** is not an **int** itself; the ***** symbol in the declaration signifies that **p** is a *pointer* to an **int**. This means that **p** can be assigned to the address of an **int**. The following sequence of code

```
int x;  
x = 3;  
int *p;  
p = &x;
```

assigns the address of **x** to **p**. We can visualize the execution of these four statements in Figure 10.2. The ***** symbol used as shown above during a variable declaration indicates that the variable is a *pointer*. It will be used to refer to another variable or some other place in memory. In this case, the sequence of assignments allows pointer **p** to refer to variable **x**.

In order to access memory via a pointer, we use the unary ***** operator. When not used in the context of a declaration, the unary ***** operator is called the *pointer dereferencing* operator. Continuing the code sequence above,

```
int x;  
x = 3;  
int *p;  
p = &x;  
*p = 5;
```

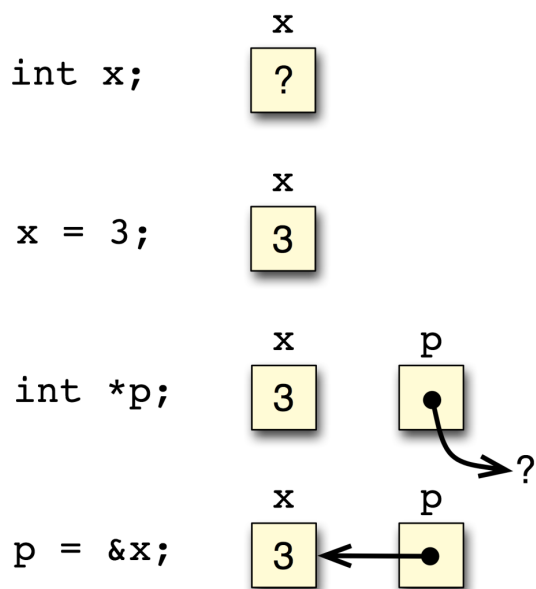


Figure 10.2: Pointer declaration and assignment

the statement

```
*p = 5;
```

copies the value 5 into the address referenced by the pointer `p`. This means that `x`'s value is changed. Figure 10.3 illustrates the full sequence. Notice that the assignment to `*p` modifies variable `x`'s value. The pointer `p` provides another way to access the memory allocated for `x`.

It is important to note that the statement

```
*p = 5;
```

is the first assignment statement we have seen that uses more than just a single variable name on the left of the assignment operator. The statement is legal because the expression `*p` represents a memory location that can store a value. Here, `*p` stands in the place of the variable `x`, and we easily can assign a value to `x`.

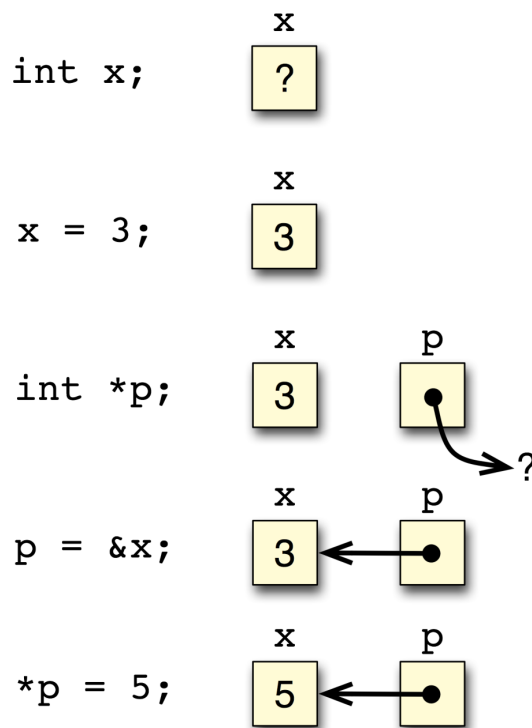


Figure 10.3: Assignment via a pointer

The unary `*` operator has two distinct meanings depending on the context:

1. At the pointer's declaration, for example,

```
double *p;
```



the `*` indicates that `p` is a pointer to an `double`; it is not itself a `double`.

2. After the pointer's declaration, for example,

```
*p = 0.08;
```

the expression `*p` represents the contents of memory at the address assigned to `p`.

The `*` operator is the inverse of the `&` operator. `&` finds the address of a variable, and, when this address is assigned to a pointer variable like `p`, the `*` operator accesses the memory referenced by the pointer:

```
int v;           // v is a normal integer variable
int *p = &v;    // Pointer variable p points to v
```

```
*p = 5;           // Assigns 5 to variable v
```

We may declare a pointer and not assign it, as in

```
int *p;
```

We say **p** is an *uninitialized pointer*, sometimes called a *wild pointer*. If **p** is a local variable, its contents are undetermined bits. Because of **p**'s declared type, these bits are interpreted as an address, so the net effect is that the uninitialized pointer **p** points to a random location in the computer's memory. An attempt to dereference **p**, as in

```
*p = 500;
```

is certainly asking for trouble. This statement attempts to write the value 500 at some unknown-to-the-programmer memory location. Often the address is not part of the area of memory set aside for the executing program, so the operating system steps in and issues a run-time error. This is the best possible result for misusing a wild pointer. It is possible, however, that the spurious address is within the executing program's domain. In this case the value 500 may overwrite another variable or the compiled machine language instructions of the program itself! Such errors are difficult to track down because the overwritten value of the variable cannot be detected until the program attempts to use the variable. The statement that misuses the uninitialized pointer may be far away in the source code (even in a different source file) from the code that attempts to use the clobbered variable. When the program fails, the programmer naturally looks around in the code where the failure occurred—the code in the vicinity where the clobbered variable appears. Unfortunately, the misbehaving code is fine, and the error lies elsewhere. There often is no easy way to locate the true source of the error.

Suppose **p** is a variable that points in an `int`. The statement

```
*p = 5;
```

assigns the value 5 to the memory location to which **p** points. Compare this legal C++ statement to the following illegal C++ statement:

```
p = 5; // Illegal statement as is
```

This second statement attempts to make **p** refer to the memory address 5; it does not assign the value 5 to the memory to which **p** refers. C++ is very strict about disallowing the mixing of pointers and non-pointers across assignment.



Systems programmers sometimes need to assign a pointer to a particular address in memory, and C++ permits the assignment with a special kind of type cast, the `reinterpret_cast`:

```
p = reinterpret_cast<int *>(5); // Legal
```

The familiar `static_cast` (see Section 4.2) will not work. Why is C++ so strict when it comes to assignments of pointers to non-pointers and vice versa? It is easy to make a mistake such as omitting the `*` operator when it is needed, so the special cast forces the programmer to pause and consider whether the mixed assignment truly is necessary or whether attempting to do so would be a mistake.

C++ provides the reserved word `nullptr` to represent a pointer to “nothing.” It stands for “null pointer.” This keyword is new to the current ISO C++11 standard and is supported by both Visual C++ 12.0 in Visual Studio 2013 and GCC 4.8. The following statement

```
p = nullptr;
```

indicates that `p` is pointing nowhere. On most platforms, `nullptr` maps to address zero, which is out of bounds for any running program. Dereferencing `p` thus would result in a run-time error, and the problem can be located quickly in a debugger.

C++ allows the literal value `0` to be used in place of `nullptr`. The statement

```
p = 0;
```

achieves the same result as assigning `nullptr` to `p`. This is how C++ programmers assigned a pointer to point to nothing before the `nullptr` keyword was available. Since newer compilers support existing C++ source code, the literal zero assignment still works. You should use the `nullptr` keyword if your compiler supports it because it improves the source code readability. Since `0` can represent both an integer value and a pointer to any type, both of the following statements are legal if `p` has been declared to be a pointer to an integer:

```
p = 0;
*p = 0;
```

The first statement assigns null to `p`, while the second statement sets the data to which `p` points to zero. Said another way, the first statement changes what `p` points to; the second statement changes where `p` points. Superficially, the two statements look very similar and are easy to confuse. Next, consider the statements

```
p = nullptr;    // OK
*p = nullptr;   // Error, will not compile
```

The second statement contains an error because the `nullptr` literal may be assigned only to a pointer type.



The `nullptr` reserved word is part of the C++11 standard. The name `nullptr` is simply an identifier (for example, a variable or function name) for older compilers. Before the `nullptr` constant became available the literal `0` (zero) was considered the null pointer reference. For backwards compatibility with older code, C++11 allows you to use `0` in place of `nullptr`, but if possible you should avoid this practice when writing new code. The `nullptr` literal allows the compiler to perform better type checking. To see why, suppose the programmer believes the variable `p` is a pointer to an integer, but `p` is instead a simple integer:

```
// Programmer believes p is a pointer to an integer
p = 0;
```

In this case the compiler is powerless to detect a problem because `p` is an integer, and zero is a valid integer value. If `p` really is an integer rather a pointer, the compiler will flag the following code:

```
// Programmer believes p is a pointer to an integer
p = nullptr;
```

because the `nullptr` may not be assigned to a non-pointer type.

The `nullptr` constant and the notion of a pointer pointing nowhere becomes more useful when building dynamic data structures (see, for example, Section ??).

10.7 Reference Variables

C++ supports another kind of variable that is many ways similar to a pointer. When the `&` symbol is used as part of the type name during a variable declaration, as in

```
int x;  
int& r = x;
```

we say `r` is a *reference variable*. This declaration creates a variable `r` that refers to the same memory location as the variable `x`. We say that `r` *aliases* `x`. Unlike a pointer variable, we may treat `r` as if it were an `int` variable—no dereferencing with `*` is necessary. Listing 10.15 (`referencevar.cpp`) demonstrates how reference variables can alias other variables.

Listing 10.15: `referencevar.cpp`

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    int x = 5;  
    int y = x;  
    int& r = x;  
    cout << "x = " << x << endl;  
    cout << "y = " << y << endl;  
    cout << "r = " << r << endl;  
    cout << "Assign 7 to x" << endl;  
    x = 7;  
    cout << "x = " << x << endl;  
    cout << "y = " << y << endl;  
    cout << "r = " << r << endl;  
    cout << "Assign 8 to y" << endl;  
    y = 8;  
    cout << "x = " << x << endl;  
    cout << "y = " << y << endl;  
    cout << "r = " << r << endl;  
    cout << "Assign 2 to r" << endl;  
    r = 2;  
    cout << "x = " << x << endl;  
    cout << "y = " << y << endl;  
    cout << "r = " << r << endl;  
}
```

The output Listing 10.15 (`referencevar.cpp`):

```

x = 5
y = 5
r = 5
Assign 7 to x
x = 7
y = 5
r = 7
Assign 8 to y
x = 7
y = 8
r = 7
Assign 2 to r
x = 2
y = 8
r = 2

```

clearly demonstrates that the variables **x** and **r** represent the same quantity. Reassigning **x** changes **r** in exactly the same way, and reassigning **r** changes **x** in exactly the same way. The variable **y**, on the other hand, is independent from both **x** and **r**. Reassigning either **x** or **r** does not affect **y**, and reassigning **y** affects neither **x** nor **r**.

Reference variables are similar to pointer variables, but there are some important differences. Reference syntax is simpler than pointer syntax because it is not necessary to dereference a reference variable in order to assign the memory location to which it refers. If **num** is an **int**, **ptr** is a pointer to an **int**, and **ref** is a reference to an **int**, consider the following statements:

```

num = ref;    // Assign num from ref, no need to dereference
num = *ptr;   // Assign num from ptr, must dereference with *

```

A references variable has two big limitations over a pointer variable:

- a reference variable must be initialized with an actual variable when it is declared. A pointer variable may be declared without an initial value and assigned later. Consider the following statements:

```

int *p;    // Legal, we will assign p later
int &r;    // Illegal, we must initialize r when declaring it

```

Attempting to compile this code under Visual C++ prompts the compiler to issue the error

error C2530: 'r' : references must be initialized

- there is no way to bind a reference variable to a different variable during its lifetime. Consider the following code fragment:

```

int x = 5, y = 7;
int *p = &x;    // Binds p to point to x
int& r = x;    // Binds r to x
p = &y;    // Reassign p to point to y
r = y;    // Assign y's value to x (via r)

```

The statement

```
r = y;
```

Does not bind **r** to the **y** variable. The declaration of **r** binds **r** to the **x** variable for the life of **r**. This statement simply assigns **y**'s value to **x** via the reference **r**. In contrast, we may freely bind pointer variables to any variables we choose at any time.

A reference variable, therefore, in the examples provided here works like pointer that must be bound to a variable and may not be redirected to point anywhere else. Also, unlike pointers, `nullptr` may not be assigned to a reference variable. There are some other differences between references and pointers that we will not explore here. Reference variables provide a simpler syntax than pointer variables since we do not use the pointer dereferencing operator (`*`) when working with references.

There is one other major difference between pointers and references—C++ adopted pointers as is from the C programming language, but C does not provide references. C++ programmers often use library functions written in C, so it is important to not mix references with C code.

Section 10.8 reveals a very important practical application of pointers and references in their role of enabling pass by reference to functions.

10.8 Pass by Reference

The default technique for passing parameters to functions is pass by value (see Section 9.3). C++ also supports *pass by reference*, also known as *call by reference*, which allows functions to alter the values of formal parameters passed by callers.

Consider Listing 10.16 (`faultyswap.cpp`), which uses a function that attempts to interchange the values of its two integer parameters.

Listing 10.16: `faultyswap.cpp`

```
#include <iostream>

using namespace std;

/*
 * swap(a, b)
 *   Attempts to interchange the values of
 *   its parameters a and b. That it does, but
 *   unfortunately it only affects the local
 *   copies.
 */
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

/*
 * main
 *   Attempts to interchange the values of
 *   two variables using a faulty swap function.
 */
int main() {
```

```

int var1 = 5, var2 = 19;
cout << "var1 = " << var1 << ", var2 = " << var2 << endl;
swap(var1, var2);
cout << "var1 = " << var1 << ", var2 = " << var2 << endl;
}

```

The output of Listing 10.16 (faultyswap.cpp) is

```

var1 = 5, var2 = 19
var1 = 5, var2 = 19

```

Unfortunately, the **swap** function simply interchanges copies of the actual parameters, not the actual parameters themselves. We really would like to write a function that interchanges the caller's variables.

Pass by reference is necessary to achieve the desired effect. C++ can do pass by reference in two ways: pointer parameters and reference parameters.

10.8.1 Pass by Reference via Pointers

Pointers (see Section 10.6) allow us to access the memory locations of variables. We can use this capability to allow a function to modify the values of variables that are owned by its caller. Listing 10.17 (swapwithpointers.cpp) provides a correct version of our variable interchange program.

Listing 10.17: swapwithpointers.cpp

```

#include <iostream>

using namespace std;

/*
 * swap(a, b)
 *   Interchanges the values of memory
 *   referenced by its parameters a and b.
 *   It effectively interchanges the values
 *   of variables in the caller's context.
 */
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

/*
 * main
 *   Interchanges the values of two variables
 *   using the swap function.
 */
int main() {
    int var1 = 5, var2 = 19;
    cout << "var1 = " << var1 << ", var2 = " << var2 << endl;
    swap(&var1, &var2);
}

```

```
    cout << "var1 = " << var1 << ", var2 = " << var2 << endl;  
}
```

The output of Listing 10.17 (swapwithpointers.cpp) is

```
var1 = 5, var2 = 19  
var1 = 19, var2 = 5
```

which is the result we were trying to achieve. The **swap** function can manipulate **main**'s variables directly since we passed it pointers to those variables.

In Listing 10.17 (swapwithpointers.cpp):

- The formal parameters to **swap**, **a** and **b**, are pointers to integers; they are not integers themselves. In order to access the integer to which the pointer named **a** refers, it must be dereferenced. That is why any use of **a** in **swap**'s body is prefixed with the pointer dereferencing operator, *****. The statement

```
int temp = *a;
```

assigns to the local variable **temp** the value of the variable to which **a** points. Since **main** passes the address of **var1** as the first parameter in its call to **swap**, in this case **a** points to **var1**, so ***a** is effectively another way to access the memory location of **var1** in **main**. The function thus assigns the value of **main**'s **var1** variable to **temp**.

- In **swap**'s statement

```
*a = *b;
```

since ***a** is effectively **main**'s **var1** variable and ***b** is effectively **main**'s **var2** variable, this statement assigns the value of **main**'s **var2** to its **var1** variable.

- The **swap** function's

```
*b = temp;
```

statement assigns the local **temp** value to **main**'s **var2** variable, since **swap**'s **b** parameter points to **main**'s **var2**.

- Within **main**, the call

```
swap(&var1, &var2);
```

passes the addresses of its local variables to **swap**.

In reality, pass by reference with pointers is still using pass by value. Instead of passing copies of values, we are passing copies of addresses. In Listing 10.17 (swapwithpointers.cpp), for example, the values of the addresses of **var1** and **var2** are copied to the formal parameters **a** and **b**. The difference is we are not attempting to reassign **a** or **b**; we are reassigning memory to which **a** and **b** point. Whether we use the original address or a copy of the address, it is still the same address—the same numeric location in memory.

10.8.2 Pass by Reference via References

Both C and C++ support pass by reference with pointers (see Section 10.8.1). Since C++ programs often use C libraries, C++ programmers must be familiar with the pointer technique for pass by reference. C++, however, provides a simpler way of implementing pass by reference using *reference parameters*. (See Section 10.7 for an introduction to reference variables.) Listing 10.18 (swapwithreferences.cpp) uses reference parameters in our variable interchange program.

Listing 10.18: swapwithreferences.cpp

```
#include <iostream>

using namespace std;

/*
 *  swap(a, b)
 *      Interchanges the values of memory
 *      referenced by its parameters a and b.
 *      It effectively interchanges the values
 *      of variables in the caller's context.
 */
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

/*
 *  main
 *      Interchanges the values of two variables
 *      using the swap function.
 */
int main() {
    int var1 = 5, var2 = 19;
    cout << "var1 = " << var1 << ", var2 = " << var2 << endl;
    swap(var1, var2);
    cout << "var1 = " << var1 << ", var2 = " << var2 << endl;
}
```

The syntax of Listing 10.18 (swapwithreferences.cpp) is a bit cleaner than that of Listing 10.17 (swapwithpointers.cpp). In Listing 10.18 (swapwithreferences.cpp):

- The formal parameters to **swap**, **a** and **b**, are *references* to integers; this is signified by the **&** symbol following **int** in their declarations. Because **a** is a reference we use it exactly like an integer; there is no need to reference it with *****. Because **a** is a reference, however, it is an alias to another variable or memory location elsewhere. This means if we modify **a**, we also modify the variable it references, in this case **var1** in **main**. The statement

```
int temp = a;
```

assigns to the local variable **temp** the value of **a**, but since **a** is another way to get to **var1**, this statement ultimately assigns **var1**'s value to **temp**.

- Within **main**, the call

```
swap(var1, var2);
```

does not require any special decoration. It looks like a normal pass by value function invocation. A programmer must be aware that **swap** uses reference parameters and that any function that uses reference parameters can change the actual values passed to it.

Reference parameters were introduced into C++ so that some of the more advanced object-oriented features could be implemented more cleanly. Some argue that for simpler situations like the **swap** function, pointer pass by reference is more desirable than reference parameter pass by reference because with pointer pass by reference, the caller is forced to pass addresses of the actual parameters that may be modified. In this way there can be no doubt that pass by reference is going on. With reference parameters, pass by value and pass by reference cannot be distinguished at the call site, since a call with reference parameters looks exactly like a pass by value invocation.

In general, pure pass by value functions are preferred to pass by reference functions. Functions using pass by reference cause *side effects*. This means they can change the state of the program in ways that can be determined only by looking inside of the function and seeing how it works. Functions that access global variables (see Section 10.1) can also cause side effects. Program development is much easier when functions can be treated as black boxes that perform computations in isolation without the possibility of affecting anything outside of their local context. The result of a function's work can be assigned to a variable thus changing the state of the program, but that change is the responsibility of the caller, not the responsibility of the function itself. With pass by value, the function's parameters and local variables come into existence when the function executes, the parameters and local variables disappear when the function is finished, and nothing else is affected by the function's execution.

Side-effect-free functions can be developed and tested in isolation from the rest of the program. Once programmers are satisfied with their correctness, they need not be touched again as the remainder of the system is developed. Functions with side effects, however, have dependencies to other parts of the program, and changes elsewhere in the system may require programmers to modify and re-evaluate existing functions.

10.9 Higher-order Functions

CAUTION! SECTION UNDER CONSTRUCTION

The functions we have seen so far have accepted only data as parameters. Since functions can contain conditional statements and loops, they can do different things based on the data they receive. The code within each function, however, is fixed at compile time. Consider the following function:

```
int evaluate(int x, int y) {
    return x + y;
}
```

The call

```
evaluate(10, 2)
```

evaluates to 12. Since the **evaluate** function returns the sum of its two parameters. The call

```
evaluate(-4, 6);
```

returns 2. The function returns a different result this time because we passed different parameters to it. The **evaluate** function in a sense behaves differently depending the arguments passed by its caller; however,

it always *adds* its two parameters. There is no way we can call **evaluate** and expect it to multiply its two parameters instead. The **evaluate** function is hard-coded to perform addition.

What if we wanted the **evaluate** function to be able to perform different arithmetic operations at different times during a program's execution? Unfortunately, **evaluate** as it currently is written cannot adapt to perform a different arithmetic operation. The good news is that we can rewrite **evaluate** so that it can flexibly adapt to a caller's changing arithmetic needs.

C++ allows programmers to pass functions as parameters to other functions. A function even may return a function as a result. A function that accepts one or more functions as parameters or returns a function as a result is known as a *higher-order function*. As we will see, higher-order functions open up new programming possibilities enabling us to customize the behavior of a function by plugging into it different functions to achieve different effects.

C++ achieves higher-order functions via *function pointers*. During a program's execution, the compiled machine language code for a function must reside in the computer's memory for the program to be able to invoke the function. That means every function has a memory address just as each variable has its own specific memory address. A pointer to a function holds the starting address for the compiled code of a particular function.

Listing 10.19 (arithmeticeval.cpp) provides an example of a higher-order function in action.

Listing 10.19: arithmeticeval.cpp

```
#include <iostream>

using namespace std;

int add(int x, int y) {
    return x + y;
}

int multiply(int x, int y) {
    return x * y;
}

int evaluate(int (*f)(int, int), int x, int y) {
    return f(x, y);
}

int main() {
    cout << add(2, 3) << endl;
    cout << multiply(2, 3) << endl;
    cout << evaluate(&add, 2, 3) << endl;
    cout << evaluate(&multiply, 2, 3) << endl;
}
```

The first parameter of the **evaluate** function,

```
int (*f)(int, int)
```

represents a function pointer. The parameter's name is **f**, and **f** is a pointer to a function that accepts two integer parameters and returns an integer result. In Listing 10.19 (arithmeticeval.cpp), the first parameter a caller must pass to **evaluate** is the address of a function with a prototype that matches the formal parameter specified in **evaluate**'s definition. Both the **add** and **multiply** functions qualify because they accept two integer parameters and return an integer result. The expression


```
evaluate(&add, 2, 3)
```

passes the address of the **add** function to **evaluate**. In the body of the **evaluate** function we see that **evaluate** invokes the function specified by its first parameter (**f**), passing to this function its second (**x**) and third (**y**) parameters as the two parameters that function expects. The net effect, therefore, of the call

```
evaluate(&add, 2, 3)
```

is the same as

```
add(2, 3)
```

C++ has a somewhat relaxed syntax for function pointers that cannot be applied to pointers to data. When invoking **evaluate** in source code we may omit the ampersand in front of the function argument, as is

```
cout << evaluate(add, 2, 3) << endl;
```

Since the compiler knows that **add** is a function and since parentheses do not follow the word **add**, the compiler deduces that this is not a call to **add** but rather the address of the **add** function.



When the name of a previously declared function appears by itself within C++ source code it represents a pointer to that function.

Function pointers are not restricted to function parameters. Given the definition of **add** above, the following code fragment is legal C++:

```
// Declare func to be a pointer to a function that accepts
// two integer parameters and returns an integer result
int (*func)(int, int);

// Assign add to func
func = add;

// Call add through func
cout << func(7, 2) << endl;
```

This code fragment as part of a complete C++ program would print 9.

Higher-order functions via function pointers provide a powerful tool for developing flexible programs. With functions as parameters we can dynamically customize the behavior of a function, essentially “plugging in” new functionality by passing in different functions. We will put higher-order functions to good use in Chapter 12.

10.10 Summary

- A global variable is available to any function defined after the global variable.

- Local variables are not automatically initialized, but global numeric variables are automatically initialized to zero.
- A global variable exists for the life of the program, but local variables are created during a function call and are discarded when the function's execution has completed.
- A local variable with the same name as a global variable hides the global from the code within the function. The scope resolution operator (`::`) can be used to access the hidden global variable.
- Modifying a global variable directly affects the behavior of any function that uses that global variable. A function that uses a global variable cannot be tested in isolation since its behavior will vary depending on how code outside the function modifies the global variable.
- The behavior of an independent function is determined strictly by the parameters passed into it. An independent function will not use global variables.
- Local variables are preferred to global variables, since the indiscriminate use of global variables leads to functions that are less flexible, less reusable, and more difficult to understand.
- Unlike normal local variables, **static** local variables exist throughout the life of the program.
- Unlike normal local variables, a **static** local variable retains its value in between function invocations.
- All **static** local variables are initialized once, at the beginning of the program's execution.
- A function's signature consists of its name along with the types of parameters it accepts.
- A C++ program may have multiple functions with the same names as long as their signatures differ.
- A recursive function must optionally call itself or not as determined by a conditional statement. The call of itself is the recursive case, and the base case does not make the recursive call. Each recursive call should move the computation closer to the base case.
- Function prototypes can be placed in a `.h` header file which is **#included** by calling code. The function definitions are then placed in a `.cpp` file which can be compiled separately from the calling code.
- A pointer is a variable representing a memory address.
- The address of a variable can be determined with the address-of operator (**&**). This address can be assigned to a pointer variable.
- The ***** symbol is used during declaration to signify a pointer variable.
- When not used during a declaration, the unary ***** operator is the pointer dereferencing operator. The pointer dereferencing operator in conjunction with a pointer variable is used to examine or modify the contents of the memory referenced by the pointer.
- The pointer dereferencing operator (*****) and address-of operator (**&**) perform the inverse operations of each other. The **&** operator yields the address of an object, while the ***** operator provides the object at a given address.
- In call-by-reference parameter passing, an actual parameter value is not copied into the formal parameter. Instead, the address of an actual parameter is passed as the formal parameter. The code within the function can manipulate the value of the actual parameter through its address. The result is that functions can modify actual parameters via pass by reference.

- C++ implements pass by reference two different ways: via pointer parameters and reference parameters. The C programming language supports pass by reference only with pointer parameters.
- You should use pass by reference when a function would need to return more than one result. Since a function can return only one value, multiple variables can be passed by reference to a function, and the function can modify their values directly.
- Higher-order functions accept other functions as parameters and may return functions as results.
- Higher-order functions enable programmers to develop highly flexible functions with pluggable functionality.

10.11 Exercises

1. Consider the following C++ code:

```
#include <iostream>

using namespace std;

int sum1(int n) {
    int s = 0;
    while (n > 0) {
        s++;
        n--;
    }
    return s;
}

int input;

int sum2() {
    int s = 0;
    while (input > 0) {
        s++;
        input--;
    }
    return s;
}

int sum3() {
    int s = 0;
    for (int i = input; i > 0; i--)
        s++;
    return s;
}

int main() {
    // See each question below for details
}
```

- (a) What is printed if **main** is written as follows?

```
int main() {
    input = 5;
    cout << sum1(input) << endl;
    cout << sum2() << endl;
    cout << sum3() << endl;
}
```

- (b) What is printed if **main** is written as follows?

```
int main() {
    input = 5;
    cout << sum1(input) << endl;
    cout << sum3() << endl;
    cout << sum2() << endl;
}
```

- (c) What is printed if **main** is written as follows?

```
int main() {
    input = 5;
    cout << sum2() << endl;
    cout << sum1(input) << endl;
    cout << sum3() << endl;
}
```

- (d) Which of the functions **sum1**, **sum2**, and **sum3** produce a side effect? What is the side effect?
- (e) Which function may not use the **input** variable?
- (f) What is the scope of the variable **input**? What is its lifetime?
- (g) What is the scope of the variable **i**? What is its lifetime?
- (h) Which of the functions **sum1**, **sum2**, and **sum3** manifest good functional independence? Why?

2. Consider the following C++ code:

```
#include <iostream>

using namespace std;

int next_int1() {
    static int cnt = 0;
    cnt++;
    return cnt;
}

int next_int2() {
    int cnt = 0;
    cnt++;
    return cnt;
}

int global_count = 0;
```

```

int next_int3() {
    global_count++;
    return global_count;
}

int main() {
    for (int i = 0; i < 5; i++)
        cout << next_int1() << " "
              << next_int2() << " "
              << next_int3() << endl;
}

```

- (a) What does the program print?
 - (b) Which of the functions `next_int1`, `next_int2`, and `next_int3` is the best function for the intended purpose? Why?
 - (c) What is a better name for the function named `next_int2`?
 - (d) The `next_int3` function works in this context, but why is it not a good implementation of function that always returns the next largest integer?
3. The following C++ program is split up over three source files. The first file, `counter.h`, consists of

```

int read();
int increment();
int decrement();

```

The second file, `counter.cpp`, contains

```

static int count;

int read() {
    return count;
}

int increment() {
    if (count < 5)
        count++;
}

int decrement() {
    if (count > 0)
        count--;
}

```

The third file, `main.cpp`, is incomplete:

```

#include <iostream>
#include "counter.h"

using namespace std;

```

```
int main() {
    // Add code here
}
```

- (a) Add statements to **main** that enable it to produce the following output:

```
3
2
4
```

The restriction is that the only output statement you are allowed to use (three times) is

```
cout << read() << endl;
```

- (b) Under the restriction of using the same output statement above, what code could you add to **main** so that it would produce the following output?

```
6
```

4. Consider the following C++ code:

```
#include <iostream>

using namespace std;

int max(int n) {
    return n;
}

int max(int m, int n) {
    return (m >= n)? m : n;
}

int max(int m, int n, int r) {
    int x = m;
    if (n > x)
        x = n;
    if (r > x)
        x = r;
    return x;
}

int main() {
    cout << max(4) << endl;
    cout << max(4, 5) << endl;
    cout << max(5, 4) << endl;
    cout << max(1, 2, 3) << endl;
    cout << max(2, 1, 3) << endl;
    cout << max(2, 1, 2) << endl;
}
```

- (a) Is the program legal since there are three different functions named **max**?
- (b) What does the program print?

5. Consider the following function:

```
int proc(int n) {
    if (n < 1)
        return 1;
    else
        return proc(n/2) + proc(n - 1);
}
```

Evaluate each of the following expressions:

- (a) **proc(0)**
 - (b) **proc(1)**
 - (c) **proc(2)**
 - (d) **proc(3)**
 - (e) **proc(5)**
 - (f) **proc(10)**
 - (g) **proc(0)**
6. Rewrite the **gcd** function so that it implements Euclid's method but uses iteration instead of recursion.
7. If **x** is a variable, how would you determine its address in the computer's memory?
8. What is printed by the following code fragment?

```
int x = 5, y = 3, *p = &x, *q = &y;
cout << "x = " << x << ", y = " << y << endl;
x = y;
cout << "x = " << x << ", y = " << y << endl;
x = 7;
cout << "x = " << x << ", y = " << y << endl;
*p = 10;
cout << "x = " << x << ", y = " << y << endl;
p = q;
*p = 20;
cout << "x = " << x << ", y = " << y << endl;
```

9. Given the declarations:

```
int x, y, *p, *q;
```

indicate what each of the following code fragments will print.

- (a) **p = &x;**
x = 5;
cout << *p << endl;

- (b) `x = 5;`
`p = &x;`
`cout << *p << endl;`
- (c) `p = &x;`
`*p = 8;`
`cout << *p << endl;`
- (d) `p = &x;`
`q = &y;`
`x = 100;`
`y = 200;`
`*q = *p;`
`cout << x << ' ' << y << endl;`
`cout << *p << ' ' << *q << endl;`
- (e) `p = &x;`
`q = &y;`
`x = 100;`
`y = 200;`
`q = p;`
`cout << x << ' ' << y << endl;`
`cout << *p << ' ' << *q << endl;`
- (f) `x = 5;`
`y = 10;`
`p = q = &y;`
`cout << *p << ' ' << *q << endl;`
`*p = 100;`
`*q = 1;`
`cout << x << ' ' << y << endl;`
- (g) `x = 5;`
`y = 10;`
`p = q = &x;`
`*p = *q = y;`
`cout << x << ' ' << y << endl;`

10. The following function does not behave as expected:

```
/*
 * (Faulty function)
 *
 * get_range
 *   Establishes a range of integers. The lower value must
 *   be greater than or equal to min, and the upper value
 *   must be less than or equal to max.
 *   min is the lowest acceptable lower value.
 *   max is the highest acceptable upper value.
 *   lower is assigned the lower limit of the range
```



```

    *      upper is assigned the upper limit of the range
    */
void get_range(int min, int max, int lower, int upper) {
    cout << "Please enter a data range within the bounds "
          << min << "... " << max << ": ";
    do { // Loop until acceptable values are provided
        cin >> lower >> upper;
        if (lower < min)
            cout << lower << " is too low, please try again." << endl;
        if (upper > max)
            cout << upper << " is too high, please try again." << endl;
    }
    while (lower < min || upper > max);
}

```

- (a) Modify the function so that it works using pass by reference with pointers.
 - (b) Modify the function so that it works using pass by reference with reference parameters.
11. Classify the following functions as pure or impure. **x** is a global variable and **LIMIT** is a global constant.

- (a)

```
int f1(int m, int n) {
    return 2*m + 3*n;
}
```
- (b)

```
int f2(int n) {
    return n - LIMIT;
}
```
- (c)

```
int f3(int n) {
    return n - x;
}
```
- (d)

```
void f4(int n) {
    cout << 2*n << endl;
}
```
- (e)

```
int f5(int n) {
    int m;
    cin >> m;
    return m * n;
}
```
- (f)

```
int f6(int n) {
    int m = 2*n, p;
    p = 2*m - 5;
    return p - n;
}
```

Chapter 11

Aggregate Data

The variables we have used to this point can assume only one value at a time. As we have seen, we can use individual variables to create some interesting and useful programs; however, variables that can represent only one value at a time do have their limitations. Consider Listing 11.1 (averagenumbers.cpp) which averages five numbers entered by the user.

Listing 11.1: averagenumbers.cpp

```
#include <iostream>

using namespace std;

int main() {
    double n1, n2, n3, n4, n5;
    cout << "Please enter five numbers: ";
    // Allow the user to enter in the five values.
    cin >> n1 >> n2 >> n3 >> n4 >> n5;
    cout << "The average of " << n1 << ", " << n2 << ", "
         << n3 << ", " << n4 << ", " << n5 << " is "
         << (n1 + n2 + n3 + n4 + n5)/5 << endl;;
}
```

A sample run of Listing 11.1 (averagenumbers.cpp) looks like:

```
Please enter five numbers: 9 3.5 0.2 100 15.3
The average of 9.0, 3.5, 0.2, 100.0, 15.3 is 25.6
```

The program conveniently displays the values the user entered and then computes and displays their average.

Suppose the number of values to average must increase from five to 25. If we use Listing 11.1 (averagenumbers.cpp) as a guide, twenty additional variables must be introduced, and the overall length of the program will necessarily grow. Averaging 1,000 numbers using this approach is impractical.

Listing 11.2 (averagenumbers2.cpp) provides an alternative approach for averaging numbers.

Listing 11.2: averagenumbers2.cpp

```

#include <iostream>

using namespace std;

int main() {
    double sum = 0.0, num;
    const int NUMBER_OF_ENTRIES = 5;
    cout << "Please enter " << NUMBER_OF_ENTRIES
         << " numbers: ";
    for (int i = 0; i < NUMBER_OF_ENTRIES; i++) {
        cin >> num;
        sum += num;
    }
    cout << "The average of " << NUMBER_OF_ENTRIES
         << " values is " << sum/NUMBER_OF_ENTRIES << endl;
}

```

Listing 11.2 (averagenumbers2.cpp) behaves slightly differently from Listing 11.1 (averagenumbers.cpp), as the following sample run using the same data shows:

```

Please enter five numbers: 9 3.5 0.2 100 15.3
The average of the five values is 25.6

```

Listing 11.2 (averagenumbers2.cpp) can be modified to average 25 values much more easily than Listing 11.1 (averagenumbers.cpp) that must use 25 separate variables—just change the constant **NUMBER_OF_ENTRIES**. In fact, the coding change to average 1,000 numbers is no more difficult. However, unlike the original average program, this new version does not display the numbers entered. This is a significant difference; it may be necessary to retain all the values entered for various reasons:

- All the values can be redisplayed after entry so the user can visually verify their correctness.
- The values may need to be displayed in some creative way; for example, they may be placed in a graphical user interface component, like a visual grid (spreadsheet).
- The values entered may need to be processed in a different way after they are all entered; for example, we may wish to display just the values entered above a certain value (like greater than zero), but the limit is not determined until after all the numbers are entered.

In all of these situations we must retain the values of all the variables for future recall.

We need to combine the advantages of both of the above programs; specifically we want

- the ability to retain individual values, and
- the ability to dispense with creating individual variables to store all the individual values

These may seem like contradictory requirements, but C++ provides several standard data structures that simultaneously provide both of these advantages. In this chapter we will examine two of these data structures: *vectors* and *arrays*.

11.1 Vectors

A vector in C++ is an object that manages a block of memory that can hold multiple values simultaneously; a vector, therefore, represents a collection of values. A vector has a name, and we may access the values it contains via their position within the block of memory managed by the vector. A vector stores a sequence of values, and the values must all be of the same type. A collection of values all of the same type is said to be *homogeneous*.

11.1.1 Declaring and Using Vectors

In order to use a vector object within a C++ program, you must add the preprocessor directive

```
#include <vector>
```

The **vector** type is part of the standard (**std**) namespace, so its full name is **std::vector**, just like the full name of **cout** is **std::cout** (see Section 2.1). If you include the statement

```
using namespace std;
```

or

```
using std::vector;
```

at the top of your source file, you can use the shorter name **vector** within your code. The remainder of the examples in this chapter assume the necessary **using** statement and so uses the short name, **vector**.

We may declare a vector object that can hold integers as simply as

```
vector<int> vec_a;
```

The type within the angle brackets may be any valid C++ data type. When declared this way, the vector **vec_a** initially is empty.

We can declare a vector with a particular initial size as follows:

```
vector<int> vec_b(10);
```

Here **vec_b** initially holds 10 integers. All 10 elements are zero by default. Note that the vector's size appears within parentheses following the vector's name.

We may declare a vector object of a given size and specify the initial value of all of its elements:

```
vector<int> vec_c(10, 8);
```

In this example **vec_c** initially holds 10 integers, all having the value 8. Note that the first number within the parentheses following the vector's name indicates the number of elements, and the second number specifies the initial value of all the elements.

We may declare a vector and specify each and every element separately:

```
vector<int> vec_d{10, 20, 30, 40, 50};
```

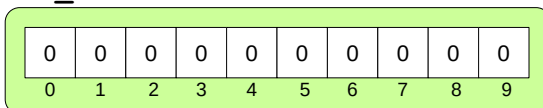
Note that the elements appear within curly braces, not parentheses. The list of elements within the curly braces constitutes a *vector initializer list*. This kind of declaration is practical only for relatively small vectors. Figure 11.1 provides a conceptual illustration of the vectors **vec_a**, **vec_b**, **vec_c**, and **vec_d**.

```
vector<int> vec_a;
vector<int> vec_b(10);
vector<int> vec_c(10, 8);
vector<int> vec_d{ 10, 20, 30, 40 };
```

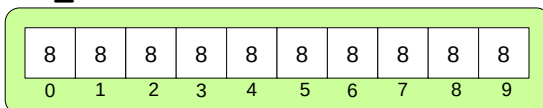
vec_a



vec_b



vec_c



vec_d

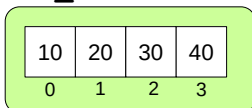


Figure 11.1: Four different vector declaration statements and conceptual illustrations of the resulting vectors

Like any other variable, a vector can be local or global, and it must be declared before it is used.



The vector initializer list syntax:

```
vector<int> vec_d{10, 20, 30, 40, 50};
```

is a C++11 language feature and, therefore, is not supported by older C++ compilers; in particular, the C++ compilers available for Visual Studio editions prior to Visual Studio 2013 do not support this initializer list syntax.

Once we have a non-empty vector object, we can access its elements using the array's name, the square bracket operator, and a integer index:

```
vector<int> list(3);           // Declare list to be a vector of three ints
list[0] = 5;                  // Make the first element 5
list[1] = -3;                 // Make the second element -3
```

```
list[2] = 12 ;           // Make the last element 12
cout << list[1] << endl; // Print the element at index 1
```

This code fragment shows how the square brackets allow us to access an individual element based on that element's position within the vector. The number within the square brackets indicates the distance from the beginning of the vector. The expression `list[0]` therefore indicates the element at the very beginning (a distance of zero from the beginning), and `list[1]` is the second element (a distance of one away from the beginning). After executing these assignment statements, the `list` vector conceptually looks like Figure 11.2. C++ classifies the square brackets, `[]`, as a binary operator, since it requires two operands: a

```
vector<int> list(3);
list[0] = 5;
list[1] = -3;
list[2] = 12;
```

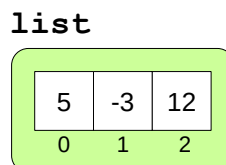


Figure 11.2: The numbers below the boxes represent indices, or positions, within the vector. Note that the first element in a vector is found at index 0, not 1.

vector's name and an index.

Vectors may hold any valid C++ data type. The following code fragment declares three vectors of differing types:

```
vector<int> list;
vector<double> collection{ 1.0, 3.5, 0.5, 7.2 };
vector<char> letters{ 'a', 'b', 'c' };
```

Here `list` is empty but can contain integer values, `collection` is a vector of double-precision floating-point numbers initially containing the values contained in the initializer list, and `letters` holds the lowercase versions of the first three letters of the English alphabet. Figure 11.3 illustrates these three vector objects. We can observe two key points from Figures 11.1–11.3:

- Vectors store their elements in a contiguous block of memory. This means, for example, the memory occupied by the element at index 2 follows immediately after the memory occupied by the element at index 1 and immediately before the element at index 3.
- Elements in a vector are located by a numeric index. The first element is at index *zero*, not one.

In an expression such as

```
list[3]
```

```
vector<int> list;
vector<double> collection{ 1.0, 3.5, 0.5, 7.2 };
vector<char> letters{ 'a', 'b', 'c' };
```

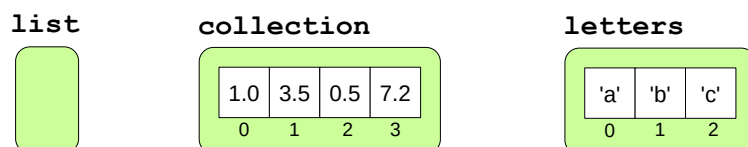


Figure 11.3: Vectors containing different types of elements. Note that while two different vector objects may contain different types of elements, all the elements within a particular vector object must all have the same type.

the expression within the square brackets, in this case 3, is called an *index* or *subscript*. The subscript terminology comes from mathematicians who use subscripts to reference elements in a mathematical sequence (for example, V_2 represents the second element in the mathematical sequence V). Unlike the convention often used in mathematics, however, the first element in a vector is at position *zero*, not one. The expression `list[2]` can be read aloud as “**list** sub two.” As a consequence of a zero beginning index, if vector **a** holds n elements, the last element in **a** is `a[n - 1]`, not `a[n]`.

An element of a vector accessed via its index behaves just like a variable of that type; for example, suppose **nums** has been declared as

```
vector<double> nums(10);
```

This declaration specifies a collection of 10 double-precision floating-point elements. The following code fragment shows how we can manipulate some of those elements:

```
// Print the fourth element
cout << nums[3] << endl;
// The third element is the average of the first and last elements
nums[2] = (nums[0] + nums[9])/2;
// Assign elements at indices 1 and 4 from user input
cin >> nums[1] >> nums[4];
```

Note that the `<<` operator associated with the `cout` output stream object is not designed to work with vector objects as a whole:

```
cout << nums << endl;    // <--- Will not compile!
```

The expression within the `[]` operator must be compatible with an integer. Suppose **a** is a vector, **x** is a numeric variable, **max** is a function that returns a numeric value, and **b** is a vector that holds numeric values. The following examples illustrate the variety of expressions that qualify as legal vector indices:

- an numeric literal: `a[34]`
- an numeric variable: `a[x]`

- an arithmetic expression: `a[x + 3]`
- an integer result of a function call that returns a numeric value: `a[max(x, y)]`
- an element of a vector that contains numeric values: `a[b[3]]`

A floating-point index is permissible but discouraged. The compiler will issue a warning about using a floating-point index with good reason. A vector may have an element at index 2 or index 3, but it is not possible to have an element located between indices 2 and 3; therefore, any floating-point index must be truncated to an integer in order to select the proper element within the vector.

11.1.2 Traversing a Vector

The action of moving through a vector visiting each element is known as *traversal*. `for` loops are ideal for vector traversals. If `a` is an integer vector containing 10 elements, the following loop prints each element in `a`:

```
for (int i = 0; i < 10; i++)
    cout << a[i] << endl;
```

The loop control variable, `i`, steps through each valid index of vector `a`. Variable `i`'s value starts at 0 and ends at 9, the last valid position in vector `a`.

The following loop prints contents of vector `a` in reverse order:

```
for (int i = 9; i >= 0; i--)
    cout << a[i] << endl;
```

The following code produces a vector named `set` containing the integer sequence 0, 1, 2, 3, ..., 999:

```
vector<int> set(1000);
for (int i = 0; i < 1000; i++)
    set[i] = i;
```

We now have all the tools we need to build a program that flexibly averages numbers while retaining all the values entered. Listing 11.3 (vectoraverage.cpp) uses a vector and a loop to achieve the generality of Listing 11.2 (averagenumbers2.cpp) with the ability to retain all input for later redisplay.

Listing 11.3: vectoraverage.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    double sum = 0.0;
    const int NUMBER_OF_ENTRIES = 5;
    vector<double> numbers(NUMBER_OF_ENTRIES);

    cout << "Please enter " << NUMBER_OF_ENTRIES
         << " numbers: ";
    // Allow the user to enter in the values.
    for (int i = 0; i < NUMBER_OF_ENTRIES; i++) {
```



```

        cin >> numbers[i];
        sum += numbers[i];
    }
    cout << "The average of ";
    for (int i = 0; i < NUMBER_OF_ENTRIES - 1; i++)
        cout << numbers[i] << ", ";
    // No comma following last element
    cout << numbers[NUMBER_OF_ENTRIES - 1] << " is "
        << sum/NUMBER_OF_ENTRIES << endl;
}

```

The output of Listing 11.3 (vectoraverage.cpp) is similar to the original Listing 11.1 (averagenumbers.cpp) program:

```

Please enter 5 numbers: 9 3.5 0.2 100 15.3
The average of 9.0, 3.5, 0.2, 100.0, 15.3 is 25.6

```

Unlike the original program, however, we now conveniently can extend this program to handle as many values as we wish. We need only change the definition of the **NUMBER_OF_ENTRIES** constant to allow the program to handle any number of values. This centralization of the definition of the vector's size eliminates duplicating a hard-coded value and leads to a program that is more maintainable. Suppose every occurrence of **NUMBER_OF_ENTRIES** were replaced with the literal value 5. The program would work exactly the same way, but changing the size would require touching many places within the program. When duplicate information is scattered throughout a program, it is a common error to update some but not all of the information when a change is to be made. If all of the duplicate information is not updated to agree, the inconsistencies result in errors within the program. By faithfully using the **NUMBER_OF_ENTRIES** constant throughout the program instead of the literal numeric value, the problems with inconsistency can be avoided.

The first loop in Listing 11.3 (vectoraverage.cpp) collects all five input values from the user. The second loop only prints the first four because it also prints a trailing comma after each element. Since no comma should be displayed after the last element, the program prints the last element after the loop is finished.

The compiler will insist that the programmer use a numeric value for an index, but the programmer must ensure that the index used is within the bounds of the vector. Since the index may consist of an arbitrary integer expression whose value cannot be determined until run time, the compiler cannot check for out-of-bound vector accesses; for example, in the code

```

int x;
vector<int> v(10); // Make a vector with 10 spaces available
cin >> x;         // User enters x at run time
v[x] = 1;         // Is this OK? What is x?

```

the compiler cannot predict what number the user will enter. This means that misuse of a vector index can lead to run-time errors. To illustrate the problem, consider Listing 11.4 (vectoroutofbounds.cpp).

Listing 11.4: vectoroutofbounds.cpp

```

#include <iostream>
#include <vector>
using namespace std;

```

```

int main() {
    const int SIZE = 3;
    vector<int> a{5, 5, 5};
    // Print the contents of the vector
    cout << "a contains ";
    for (int i = 0; i < SIZE; i++)
        cout << a[i] << " ";
    cout << endl;
    // Change all the 5s in vector a to 8s
    for (int i = 0; i <= SIZE; i++)    // Bug: <= should be <
        a[i] = 8;
    // Reprint the contents of the vector
    cout << "a contains ";
    for (int i = 0; i < SIZE; i++)
        cout << a[i] << " ";
    cout << endl;
}

```

Listing 11.4 (vectoroutofbounds.cpp) contains a logic error; the reassignment loop goes one past the end of vector **a**. Attempting to access elements outside the bounds of a vector produces what is known as *undefined behavior*. The C++ language standard uses this term to indicate a program's behavior is not specified, and compiler writers are free to do whatever they want. Often running programs that venture into undefined behavior will crash, but sometimes they may continue executing with no indication of a problem and appear to behave correctly most of the time. In other words, the actual program behavior is system dependent and compiler dependent. Consider code that represents undefined behavior to be a logic error, since its action is inconsistent across platforms and compilers. Simply said, the program's behavior is unpredictable. Unpredictable behavior is incorrect behavior.

In most cases, an out-of-bounds access simply accesses memory outside the vector. If this includes memory that does not belong to the executing program, modern operating systems will terminate the program and produce an error message. Under Visual C++ when the program is built as a debug version (the default when using the IDE), the program prints the contents of the vector the first time but crashes before it can print it out a second time. Microsoft Windows then displays the dialog box shown in Figure 11.4. The program running under Linux or OS X may simply print in the console:



Segmentation fault

If your program is using a vector, and it terminates with such a message, you should check its source code carefully for places where out-of-bounds vector accesses are possible.

The following code fragment shows some proper and improper vector accesses:

```

vector<int> numbers(10);    // Declare the vector
numbers[0] = 5;             // Put value 5 first
numbers[9] = 2;             // Put value 2 last
numbers[-1] = 5;           // Out of bounds; first valid index is 0
numbers[10] = 5;           // Out of bounds; last valid index is 9
numbers[1.3] = 5;          // Compiler warning, 1.3 is not an int

```

In a vector traversal with a **for** loop such as

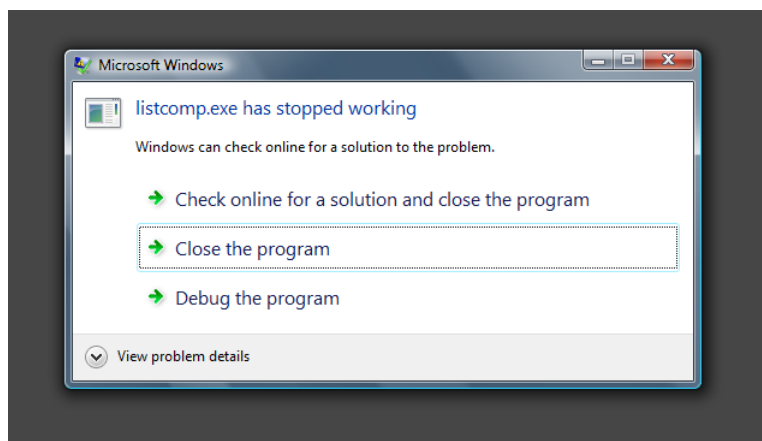


Figure 11.4: Memory access error under Visual C++

```
for (int i = 0; i < SIZE; i++)
    cout << a[i] << endl;
```

it is easy to ensure that `i` cannot fall outside the bounds of vector `a`, but you should check an arbitrary index value before using it. In the following code:

```
int x;
vector<int> a(10); // Make a vector with 10 spaces available
cin >> x;          // User enters x at run time
// Ensure x is within the bounds of the vector
if (0 <= x && x < 10)
    a[x] = 1;      // This is safe now
else
    cout << "Index provided is out of range" << endl;
```

the `if` statement ensures the vector access is within the vector's bounds.

C++11 supports a variation of the `for` statement that uses special syntax for vectors. Commonly known as the *range-based for* or “foreach” statement, this version of the `for` statement permits vector traversal without an index variable keeping track of the position. The following code fragment uses a range-based `for` statement to print the contents of an integer vector named `vec`:

```
for (int n : vec)
    cout << n << ' ';
```

You can read this statement as “for each `int n` in `vec`, `cout << n << ' ';`” The colon (`:`) therefore is pronounced “in.” In the first iteration of this range-based `for` loop the variable `n` represents the first element in the vector `vec`. In the second iteration `n` represents the second element. The third time through `n` is the third element, and so forth. The declared variable assumes the role of a different vector element during each iteration of the loop. Note that the range-based `for` loop requires no control variable to keep track of the current index within the vector; the loop itself takes care of that detail, freeing the programmer from that task.

The general form of this range-based **for** statement is

for (*type elementvariable : vectorvariable*)
statement using elementvariable

If the element variable within the range-based **for** loop is declared to be a reference, the code within the body of the loop may modify the vector's elements. Listing 11.5 (foreachexample.cpp) allows a user to populate a vector with 10 numbers and then prints the vector's contents.

Listing 11.5: foreachexample.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    // Declare a vector of ten numbers
    vector<double> vec(10);

    // Allow the user to populate the vector
    cout << "Please enter 10 numbers: ";
    for (double& elem : vec)
        cin >> elem;

    // Print the vector's contents
    for (double elem : vec)
        cout << elem << endl;
}
```

Note that the first range-based **for** statement in Listing 11.5 (foreachexample.cpp) uses a reference variable to assign each element in the vector. The second range-based **for** statement does not need to use a reference variable because it does not change any of the contents of vector **vec**.

It is not always possible to use the range-based **for** statement when traversing vectors. The range-based **for** statement iterates forward through the vector elements and cannot move backwards. Also, the range-based **for** statement is not convenient if you want to consider only a portion of the elements in the vector. Examples include visiting every other element in the vector or considering only the first third of the elements. In these specialized cases a standard **for** loop with an integer control variable is the better choice.

11.1.3 Vector Methods

A vector is an object, and objects differ from the simple types like **int**, **double**, and **bool**, in a number of ways. Most objects have access to special functions called *methods*. C++ programmers often refer to methods as *member functions*. A method is a function associated with a class of objects. A method invocation involves a slightly different syntax than a function invocation; for example, if **obj** is an object that supports a method named **f** that accepts no parameters, we can invoke **f** on behalf of **obj** with the statement:

obj.f();

The dot operator connects an object with a method to invoke. Other than this special invocation syntax, methods work very much like the global functions introduced in Chapter 8. A method may accept parameters and may return a value.

Vectors support a number of methods, but we will focus on seven of them:

- **push_back**—inserts a new element onto the back of a vector
- **pop_back**—removes the last element from a vector
- **operator []**—provides access to the value stored at a given index within the vector
- **at**—provides bounds-checking access to the value stored at a given position within the vector
- **size**—returns the number of values currently stored in the vector
- **empty**—returns true if the vector contains no elements; returns false if the vector contains one or more elements
- **clear**—makes the vector empty.

We have seen how to declare a vector of a particular size and use the space provided; however, we are not limited by a vector's initial size. In order to add an element to a vector, we use the **push_back** method as follows:

```
vector<int> list;           // Make an empty vector that can hold integers
list.push_back(5);         // Add 5 to the end of list
list.push_back(-3);        // Add -3 to the end of the list
list.push_back(12);        // Add 12 to the end of list
```

After executing the three **push_back** calls above, the **list** vector conceptually looks just like Figure 11.2. The size of the vector adjusts automatically as new elements are inserted onto the back. Each **push_back** method call increases the number of elements in a vector's by one.

The **pop_back** method performs the opposite action of **push_back**. A call to **pop_back** removes the last element from a vector, effectively reducing the number of elements in the vector by one. The following code fragment produces a vector named **list** that contains only the element 5.

```
vector<int> list;           // Declare list to be a vector
list.push_back(5);         // Add 5 to the end of list
list.push_back(-3);        // Add -3 to the end of the list
list.push_back(12);        // Add 12 to the end of list
list.pop_back();           // Removes 12 from the list
list.pop_back();           // Removes -3 from the list
```

We have been using the **operator []** method to access an element in the vector. The word **operator** is reserved in C++, and that makes this method even more interesting. The expression

```
vec.operator[] (2)
```

is the long way to write

```
vec[2]
```

Programmers use the shorter syntax exclusively, but the longer expression better illustrates the fact that the square bracket (`[]`) operator really is a class method.

As we have seen, a programmer must be vigilant to avoid using an out-of-bounds index with the `operator[]` method. The vector class provides an additional method, `at`, that provides index bounds checking. The expression `vec[x]` in and of itself provides no bounds checking and so may represent undefined behavior. The functionally equivalent expression `vec.at(x)` will check to ensure that the index `x` is within the bounds of the vector. If `x` is outside the acceptable range of indices, the method is guaranteed to produce a run-time error. Listing 11.6 (`vectoroutofbounds2.cpp`) is a variation of Listing 11.4 (`vectoroutofbounds.cpp`) that uses the `at` method instead of the `operator[]` method.

Listing 11.6: `vectoroutofbounds2.cpp`

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    const int SIZE = 3;
    vector<int> a{5, 5, 5};
    // Print the contents of the vector
    cout << "a contains ";
    for (int i = 0; i < SIZE; i++)
        cout << a.at(i) << " ";
    cout << endl;
    // Change all the 5s in vector a to 8s
    for (int i = 0; i <= SIZE; i++)    // Bug: <= should be <
        a.at(i) = 8;
    // Reprint the contents of the vector
    cout << "a contains ";
    for (int i = 0; i < SIZE; i++)
        cout << a.at(i) << " ";
    cout << endl;
}
```

When compiled and executed Listing 11.6 (`vectoroutofbounds2.cpp`) is guaranteed to produce a run-time error.

The `size` method returns the number of elements in a vector. The following code fragment prints the contents of vector `list`:

```
int n = list.size();
for (int i = 0; i < n; i++)
    cout << list[i] << " ";
cout << endl;
```

The exact type that the `size` method returns here is a `vector<int>::size_type`. This type is defined within the `vector<int>` class. It is compatible with the `unsigned` type and may be assigned to an `int` variable as shown above. We can avoid the additional local variable `n` as follows:

```
for (unsigned i = 0; i < list.size(); i++)
    cout << list[i] << " ";
cout << endl;
```

Notice that `i`'s declared type is `unsigned`, not `int`. This prevents a warning when comparing `i` to `list.size()`. A comparison between signed and unsigned integer types potentially is dangerous, and

the compiler will alert us to that fact. To see why you should not take this warning lightly, consider the following code that we would expect to print nothing:

```
unsigned u = 0;
int i = 0;
while (i < u - 1) {
    cout << i << endl;
    i++;
}
```

but instead it prints as many **unsigned** values as the system supports! This is because even though 0 is not less than -1 , -1 is a signed value, not an unsigned value. The unsigned data type cannot represent signed numbers. An attempt to compute **unsigned** 0 minus 1 on a 32-bit system produces 4,294,967,295, which definitely is not less than zero. To be safe, assign the value of the **size** method to an **int** variable or use **unsigned** variables to control loop iterations. Better yet, use the range-based **for** statement whenever possible.

The **empty** method is a convenience method; if **vec** is a vector, the expression **vec.empty()** is equivalent to the Boolean expression **vec.size() != 0**. Note that the **empty** method does not *make* the vector empty; it simply returns true *if* the vector is empty and false if it is not.

The **clear** method removes all the elements from a vector leaving it empty. Invoking **clear** on an initially empty vector has no effect. Immediately after clearing a vector, the vector's **size** method will return zero, its **empty** method will return true, and a call to its **operator[]** method with an index of any value (including zero) will exhibit undefined behavior.

11.1.4 Vectors and Functions

A function can accept a vector as a parameter as shown in Listing 11.7 (vectortofunc.cpp)

Listing 11.7: vectortofunc.cpp

```
#include <iostream>
#include <vector>

using namespace std;

/*
 * print(v)
 * Prints the contents of an int vector
 * v is the vector to print
 */
void print(vector<int> v) {
    for (int elem : v)
        cout << elem << " ";
    cout << endl;
}

/*
 * sum(v)
 * Adds up the contents of an int vector
 * v is the vector to sum
 * Returns the sum of all the elements
 * or zero if the vector is empty.
 */
```

```

*/
int sum(vector<int> v) {
    int result = 0;
    for (int elem : v)
        result += elem;
    return result;
}

int main() {
    vector<int> list{ 2, 4, 6, 8, };
    // Print the contents of the vector
    print(list);
    // Compute and display sum
    cout << sum(list) << endl;
    // Zero out all the elements of list
    int n = list.size();
    for (int i = 0; i < n; i++)
        list[i] = 0;
    // Reprint the contents of the vector
    print(list);
    // Compute and display sum
    cout << sum(list) << endl;
}

```

Listing 11.7 (vectortofunc.cpp) produces

```

2 4 6 8
20
0 0 0 0
0

```

The **print** function's definition:

```
void print(vector<int> v) {
```

shows that a vector formal parameter is declared just like a non-vector formal parameter. In this case, the **print** function uses pass by value, so during the program's execution an invocation of **print** will copy the data in the actual parameter (**list**) to the formal parameter (**v**). Since a vector potentially can be quite large, it generally is inefficient to pass a vector by value as shown above. Pass by value requires a function invocation to create a new vector object for the formal parameter and copy all the elements of the actual parameter into the new vector which is local to the function. A better approach uses pass by reference, with a twist:

```
void print(const vector<int>& v) {
    for (int elem : v)
        cout << elem << " ";
    cout << endl;
}

```

The **&** symbol indicates that a caller invoking **print** will pass **v** by reference (see Section 10.8). This copies the address of the actual parameter (owned by the caller) to the formal parameter **v** instead of

making a copy of all the data in the caller's vector. Passing the address is much more efficient because on most systems an address is the same size as a single `int`, whereas a vector could, for example, contain 1,000,000 `ints`. With pass by value a function invocation would have to copy all those 1,000,000 integers from the caller's actual parameter into the function's formal parameter.

Section 10.8 indicated that call-by-value parameter passing is preferred to call-by-reference parameter passing. This is because a function using pass by value cannot modify the actual variable the caller passed to it. Observe closely that our new `print` function declares its formal parameter `v` to be a `const` reference. This means the function cannot modify the actual parameter passed by the caller. Passing a vector object as a constant reference allows us to achieve the efficiency of pass by reference with the safety of pass by value.

Like the `print` function, the `sum` function in Listing 11.7 (`vectortofunc.cpp`) does not intend to modify the contents of its vector parameter. Since the `sum` function needs only look at the vector's contents, its vector parameter should be declared as a `const` reference. In general, if a function's purpose *is* to modify a vector, the reference should not be `const`. Listing 11.8 (`makerandomvector.cpp`) uses a function named `make_random` that fills a vector with pseudorandom integer values.

Listing 11.8: `makerandomvector.cpp`

```
#include <iostream>
#include <vector>
#include <cstdlib>    // For rand

using namespace std;

/*
 * print(v)
 * Prints the contents of an int vector
 * v is the vector to print; v is not modified
 */
void print(const vector<int>& v) {
    for (int elem : v)
        cout << elem << " ";
    cout << endl;
}

/*
 * make_random(v)
 * Fills an int vector with pseudorandom numbers
 * v is the vector to fill; v is modified
 * size is the maximum size of the vector
 */
void make_random(vector<int>& v, int size) {
    v.clear();    // Empties the contents of vector
    int n = rand() % size + 1;    // Random size for v
    for (int i = 0; i < n; i++)
        v.push_back(rand());    // Populate with random values
}

int main() {
    srand(2);    // Set pseudorandom number generator seed
    vector<int> list;
    // Print the contents of the vector
    cout << "Vector initially: ";
```

```

    print(list);
    make_random(list, 20);
    cout << "1st random vector: ";
    print(list);
    make_random(list, 5);
    cout << "2nd random vector: ";
    print(list);
    make_random(list, 10);
    cout << "3rd random vector: ";
    print(list);
}

```

The `make_random` function in Listing 11.8 (`makerandomvector.cpp`) calls the vector method `clear` which makes a vector empty. We call `clear` first because we want to ensure the vector is empty before we add more elements. The function then proceeds to add a random number of random integers to the empty vector.

A function may return a vector object. Listing 11.9 (`primelist.cpp`) is a practical example of a function that returns a vector.

Listing 11.9: `primelist.cpp`

```

#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

/*
 * print(v)
 * Prints the contents of an int vector
 * v is the vector to print; v is not modified
 */
void print(const vector<int>& v) {
    for (int elem : v)
        cout << elem << " ";
    cout << endl;
}

/*
 * is_prime(n)
 * Determines the primality of a given value
 * n an integer to test for primality
 * Returns true if n is prime; otherwise, returns false
 */
bool is_prime(int n) {
    if (n < 2)
        return false;
    else {
        bool result = true; // Provisionally, n is prime
        double r = n, root = sqrt(r);
        // Try all possible factors from 2 to the square
        // root of n
        for (int trial_factor = 2; result && trial_factor <= root;
            trial_factor++)
            result = (n % trial_factor != 0);
    }
}

```

```

        return result;
    }
}

/*
 * primes(begin, end)
 * Returns a vector containing the prime
 * numbers in the range begin...end.
 * begin is the first number in the range
 * end is the last number in the range
 */
vector<int> primes(int begin, int end) {
    vector<int> result;
    for (int i = begin; i <= end; i++)
        if (is_prime(i))
            result.push_back(i);
    return result;
}

int main() {
    int low, high;
    cout << "Please enter lowest and highest values in "
          << "the range: ";
    cin >> low >> high;
    vector<int> prime_list = primes(low, high);
    print(prime_list);
}

```

The **primes** function declares a local vector variable named **result**. The function examines every value in the range **begin...end**, inclusive. The **primes** function uses **is_prime** as a helper function. If the **is_prime** function classifies a value as a prime, code within the **primes** function adds the value to the **result** vector. After **primes** has considered all the values in the provided range, **result** will contain all the prime numbers in that range. In the end, **primes** returns the vector containing all the prime numbers in the specified range.

When returning a local variable that is a built-in scalar type like **int**, **double**, **char**, etc., a C++ function normally makes a copy of the local variable to return to the caller. Making a copy is necessary because local variables exist only while the function in which they are declared is actively executing. When the function is finished executing and returns back its caller, the run-time environment reclaims the memory held by the function's local variables and parameters so their space can be used by other functions.

The return value in **primes** is not a simple scalar type—it is an object that can be quite large, especially if the caller passes in a large range. Modern C++ compilers generate machine code that eliminates the need to copy the local vector **result**. The technique is known as *return value optimization*, and it comes into play when a function returns an object declared within the function. With return value optimization, the compiler “knows” that the variable will disappear and that the variable is to be returned to the caller; therefore, it generates machine language code that makes the space for the result in the caller's memory space, not the called function. Since the caller is maintaining the space for the object, it persists after the function returns. Because of return value optimization you can return vectors by value without fear of a time-consuming copy operation.

Due to the fact that vectors may contain a large number of elements, you usually should pass vectors to functions as reference parameters rather than value parameters:



- If the function *is* meant to modify the contents of the vector, declare the vector as a non-**const** reference (that is, omit the **const** keyword when declaring the parameter).
- If the function is *not* meant to modify the contents of the vector, declare the vector as a **const** reference.

It generally is safe to return a vector by value from a function if that vector is declared local to the function. Modern C++ compilers generate optimized code that avoid the overhead of copying the result back to the caller.

11.1.5 Multidimensional Vectors

The vectors we have seen thus far have been one dimensional—simple sequences of values. C++ supports higher-dimensional vectors. A *two-dimensional vector* is best visualized as a table with rows and columns. The statement

```
vector<vector<int>> a(2, vector<int>(3));
```

effectively declares **a** to be a two-dimensional (2D) vector of integers. It literally creates a vector with two elements, and each element is itself a vector containing three integers. Note that the type of **a** is a vector of vector of integers. A 2D vector is sometimes called a *matrix*. In this case, the declaration specifies that 2D vector **a** contains two rows and three columns. Figure 11.5 shows the logical structure of the vector created by the following sequence of code:

```
vector<vector<int>> a(2, vector<int>(3));
a[0][0] = 5;
a[0][1] = 19;
a[0][2] = 3;
a[1][0] = 22;
a[1][1] = -8;
a[1][2] = 10;
```

The two-dimensional vector **a** is said to be a 2×3 vector, meaning it has two rows and three columns (as shown in Figure 11.5). Rows are arranged horizontally, and the values in columns are arranged vertically. In each of the assignment statements above, for example

```
a[1][0] = 22;
```

the first index (here 1) signifies the row and the second index (here 0) denotes the column of the element within the vector. Literally, the expression **a**[1][0] means (**a**[1])[0]; that is, the element at index 0 of the vector at index 1 within **a**.

Using a syntax similar to the initialization lists of one-dimensional vectors, we can declare and initialize the 2D vector **a** from above as:

```
vector<vector<int>> a({ 5, 19, 3},
                    {22, -8, 10});
```

a

0	5	19	3
1	22	-8	10
	0	1	2

Figure 11.5: A 2×3 two-dimensional vector

Note that each row appears within its own set of curly braces, and each row looks like 1D vector initialization list.

To access an element of a 2D vector, use two subscripts:

```
a[r][c] = 4;           // Assign element at row r, column c
cout << a[m][n] << endl; // Display element at row m, column n
```

The following function prints the contents of a 2D vector of `doubles`:

```
void print(const vector<vector<double>>& m) {
    for (unsigned row = 0; row < m.size(); row++) {
        for (unsigned col = 0; col < m[row].size(); col++)
            cout << setw(5) << m[row][col];
        cout << endl;
    }
}
```

We can use range-based `for` statements to simplify the code:

```
void print(const vector<vector<double>>& m) {
    for (const vector<double>& row : m) { // For each row
        for (int elem : row)           // For each element in a row
            cout << setw(5) << elem;
        cout << endl;
    }
    return sum;
}
```

The declaration of the parameter `m` is somewhat complicated. We can simplify the syntax by using a C++ `typedef` statement. The statement

```
typedef vector<vector<double>> Matrix;
```

creates a new name **Matrix** for the existing type of a 2D vector containing **double**s. If you omit the word **typedef** from the above statement, it becomes a declaration of a 2D vector named **Matrix**. The **typedef** keyword changes the meaning of the statement so that instead of declaring a variable it defines a simpler name for the type. **typedef** statements usually appear near the top of a source file and most often have global scope. It is uncommon to see a local **typedef** statement within a function body.

Given the **typedef** statement defining the new type name **Matrix**, we may express the parameter for the **print** function more simply:

```
void print(const Matrix& m) {
    for (const vector<double>& row : m) { // For each row
        for (int elem : row) // For each element in a row
            cout << setw(5) << elem;
        cout << endl;
    }
    return sum;
}
```

We can take advantage of the type inference capability of C++11 to simplify the **print** function even further (see Section 3.9):

```
void print(const Matrix& m) {
    for (auto row : m) { // For each row
        for (int elem : row) // For each element in a row
            cout << setw(5) << elem;
        cout << endl;
    }
    return sum;
}
```

Here we replaced the explicit type **const vector<double>&** with the word **auto**. The compiler is able to infer the type of the variable **row** from the context: **m** is a **Matrix** (that is, a **vector<vector<double>>**), so **row** must be an element of the 2D vector (which itself is a 1D vector).

Listing 11.10 (twodimvector.cpp) experiments with 2D vectors and takes advantage of the **typedef** statement to simplify the code.

Listing 11.10: twodimvector.cpp

```
#include <iostream>
#include <iomanip>
#include <vector>

using namespace std;

typedef vector<vector<double>> Matrix;

// Allow the user to enter the elements of a matrix
void populate_matrix(Matrix& m) {
    cout << "Enter the " << m.size() << " rows of the matrix. " << endl;
    for (unsigned row = 0; row < m.size(); row++) {
        cout << "Row #" << row << " (enter " << m[row].size() << " values): ";
        for (double& elem : m[row])
            cin >> elem;
    }
}
```

```

}

void print_matrix(const Matrix m) {
    for (auto row : m) {
        for (double elem : row)
            cout << setw(5) << elem;
        cout << endl;
    }
}

int main() {
    int rows, columns;
    cout << "How many rows? ";
    cin >> rows;
    cout << "How many columns? ";
    cin >> columns;
    // Declare the 2D vector
    Matrix mat(rows, vector<double>(columns));
    // Populate the vector
    populate_matrix(mat);
    // Print the vector
    print_matrix(mat);
}

```

An expression that uses just one index with a 2D vector represents a single row within the 2D vector. This row is itself a 1D vector. Thus, if **a** is a 2D vector and **i** is an integer, then the expression

a[i]

is a 1D vector representing row **i**.

We can build vectors with dimensions higher than two. Each “slice” of a 3D vector is simply a 2D vector, a 4D vector is a vector of 3D vectors, etc. For example, the statement

matrix[x][y][z][t] = 1.0034;

assigns 1.0034 to an element in a 4D vector of **doubles**. In practice, vectors with more than two dimensions are rare, but advanced scientific and engineering applications sometimes require higher-dimensional vectors.

11.2 Arrays

C++ is an object-oriented programming language, and a vector is an example of a software object. The C programming language does not directly support object-oriented programming and, therefore, does not have vectors available to represent aggregate types. The C language uses a more primitive construct called an *array*. C++, which began as an extension of C, supports arrays as well as vectors. Some C++ libraries use arrays instead of vectors. In addition, C++ programs can use any of the large number of C libraries that have been built up over the past 40+ years, and many of these libraries process arrays.

An array is a variable that refers to a block of memory that, like a vector, can hold multiple values simultaneously. An array has a name, and the values it contains are accessed via their position within the block of memory designated for the array. Also like a vector, the elements within an array must all be of

the same type. Arrays may be local or global variables. Arrays are built into the core language of both C and C++. This means you do not need to add any `#include` directives to use an array within a program.

Arrays come in two varieties, static and dynamic. A programmer must supply the size of a static array when declaring it; for example, the following code:

```
// list is an array of 25 integers
int list[25];
```

declares `list` to be an array of 25 integers. The value within the square brackets specifies the number of elements in the array, and the size is fixed for the life of the array. The value within the square brackets must be a constant value determined at compile time. It can be a literal value or a symbolic constant, but it cannot be a variable. This is in contrast to vector declarations in which the initial vector size may be specified by a variable with a value determined at run time:

```
int x;
cin >> x;           // Get x's value from the user at run time
int list_1[x];       // Illegal for a static array
vector<int> list_2(x); // OK for a vector
```

It is possible to declare an array and initialize it with a sequence of elements, with a syntax similar to that of vectors:

```
double collection[] = { 1.0, 3.5, 0.5, 7.2 };
```

The compiler can count the elements in the initialization list, so you need not supply a number within the square brackets. If you provide a number within the square brackets, it should be at least as large as the number of elements in the initialization list. The equals symbol is required for array initialization. You optionally can use the equals symbol as shown here when initializing vectors, but it is not required for vectors.

Programmers need not worry about managing the memory used by static arrays. The compiler and run-time environment automatically ensure the array has enough space to hold all of its elements. The space held by local arrays is automatically freed up when the local array goes out of the scope of its declaration. Global arrays live for the lifetime of the executing program. Memory management for static arrays, therefore, works just like scalar variables.

A dynamic array is simply a pointer (see Section 10.6) that points to a dynamically allocated block of memory; for example, the following code:

```
// collection can point to a dynamically allocated array of integers
int *collection;
```

declares `collection` to be a pointer to an `int`. This means `collection` holds a memory address. This declaration statement itself does not allocate any memory for the array's contents, so you may not begin using `collection` at this point. First, you must allocate the memory for its elements. The statement

```
// Directs collection to point to the beginning of block of 25 integers
collection = new int[25];
```

allocates space for a block of 25 integers and assigns `collection` to point to the beginning of this newly allocated memory. The value within the square brackets in a dynamic array allocation may be a variable determined at run time:

```
int x;
```

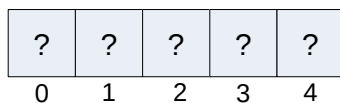


```
cin >> x; // Get x's value from the user at run time
int *list_3 = new int[x]; // OK for a dynamic array
```

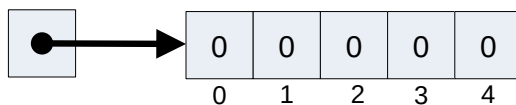
Figure 11.6 compares the conceptual differences among of static and and dynamic arrays and vectors. While a dynamic array is accessed via a pointer, a static array array behaves like a constant pointer; that

```
int list_1[10];
int *list_2 = new int[5];
vector<int> list_3(5);
```

list_1



list_2



list_3

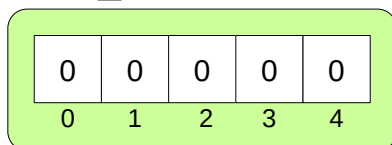


Figure 11.6: Comparing static arrays, dynamic arrays, and vectors

is, a pointer that cannot be reassigned to point elsewhere. Appendix F provides more details about array declaration and initialization.

It is important to note that since a programmer must explicitly allocate the space for a dynamic array using the **new** operator, the program should contain a corresponding call to **delete** to free up the array's memory when the array is no longer needed. C++ uses a special syntax for array deletion:

```
int *list = new int[10]; // Allocate the array dynamically
// Use the array
delete [] list; // Deallocate the array's memory
```

Note that the square brackets must be empty. The run-time environment keeps track of the array's allocated size and knows how much memory to reclaim.

Figure 11.6 shows that, like vectors, array elements may be located by their index, and the very first element in an array is located at index 0. The similarity to vectors is not accidental—the developers of the C++ vector library designed vectors to behave as much as possible like C arrays. Once declared, programmers treat static and dynamic arrays the same. The following code fragment creates a static array and populates it with pseudorandom values:

```
int random_numbers[25];
for (int i = 0; i < 25; i++)
    random_numbers[i] = rand();
```

Unlike vectors, an array has a fixed size. It is impossible to change the size of a static array (short of editing the source code and recompiling). In order to change at run time the size of a dynamic array, you must

1. allocate a different block of memory of the desired size,
2. copy all of the existing elements into the new memory,
3. use `delete` to deallocate the previous memory block to avoid a memory leak, and
4. reassign the original dynamic array pointer to point to new memory block.

Appendix F provides more detail about this process.

Unlike with vectors, it is not possible to assign one array variable to another through a simple assignment statement copying all the elements from one array into another. If the arrays are static arrays, the simple assignment is illegal. If the arrays are dynamic arrays, the assignment simply makes the two pointers point to the same block of memory, which is not the same effect as vector assignment. Furthermore, simple assignment of one dynamic array to another creates a memory leak. Appendix F shows how to do proper array assignment.

Because an array is not an object it has no associated methods. It is the programmer's responsibility to keep track of the size of an array. An array, both static and dynamic, is simply the starting address of a raw block of memory. When passing an array to a function you also must pass additional information so the function can process the array properly. The following `print` function prints the contents of an array:

```
void print(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << ' ';
    cout << endl;
}
```

Inside the function there is no way to determine `a`'s size; therefore, the second parameter specifies the array's size. A caller could invoke the function as

```
int list[25];
for (int i = 0; i < 25; i++)
    list[i] = rand() % 1000; // Fill with pseudorandom values
print(list, 25);           // Print the elements of list
```

Another popular option is to pass a pointer to the beginning of the array and another pointer to just past the end of the array:

```
void print(int *begin, int *end) {
    for (int *elem = begin; elem != end; elem++)
```

```

        cout << *elem << ' ';
    cout << endl;
}

```

Notice that this code does not appear to be using arrays at all! This code takes advantage of *pointer arithmetic*. If **elem** is a pointer to a value in memory, the expression **elem++** makes **elem** point to the next location in memory that can store the type of value to which **elem** points. Since the memory devoted to an array is a continuous, contiguous block of memory addresses, pointer arithmetic simply redirects a pointer from one element in an array to another element in that array. Listing 11.11 (pointerarithmetic.cpp) provides some example of pointer arithmetic in action.

Listing 11.11: pointerarithmetic.cpp

```

#include <iostream>
using namespace std;

int main() {
    // Make an array
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;           // p points to the first element
    cout << *p << endl;      // Prints 10, does not change p
    cout << p[0] << endl;    // Prints 10, does not change p
    cout << p[1] << endl;    // Prints 20, does not change p
    cout << *p << endl;      // Prints 10, does not change p
    p++;                    // Advances p to the next element
    cout << *p << endl;      // Prints 20, does not change p
    p += 2;                 // Advance p two places
    cout << *p << endl;      // Prints 40, does not change p
    cout << p[0] << endl;    // Prints 40, does not change p
    cout << p[1] << endl;    // Prints 50, does not change p
    p--;                    // Moves p back one place
    cout << *p << endl;      // Prints 30, does not change p
}

```

Listing 11.11 (pointerarithmetic.cpp) shows how a programmer can initialize all the elements in an array using a syntax similar to vector initializer list. Listing 11.11 (pointerarithmetic.cpp) prints

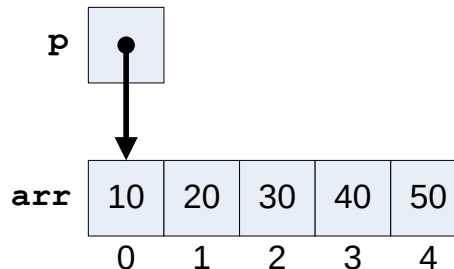
```

10
10
20
10
20
40
40
50
30

```

Figure 11.7 illustrates how Listing 11.11 (pointerarithmetic.cpp) works. Listing 11.11 (pointerarithmetic.cpp) also shows how we can use the square bracket array access operator with a pointer. The expression **p[i]** refers to the element in memory *i* positions from the location pointed to by **p**.

```
int arr[] = {10, 20, 30, 40, 50};
int *p = arr;
```



```
p++;
```

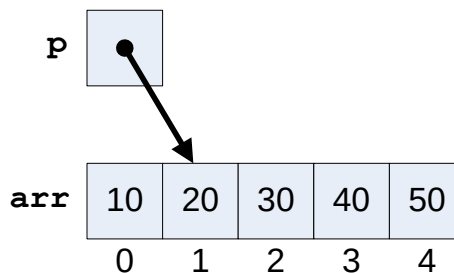


Figure 11.7: Pointer arithmetic. Incrementing pointer `p` moves it to the next element in the array.

Going back to the `print` function that uses pointers:

```
void print(int *begin, int *end) {
    for (int *elem = begin; elem != end; elem++)
        cout << *elem << ' ';
    cout << endl;
}
```

the pointer `elem` first points to the same element to which `begin` points (that is, the first element in the array), and within the loop it moves to the next element repeatedly until it passes the last element. A caller could invoke the function as

```
int list[25]; // Allocate a static array
for (int i = 0; i < 25; i++)
    list[i] = rand() % 1000; // Fill with pseudorandom values
print(list, list + 25); // Print the elements of list
```

The advantage of the begin/end pointer approach is that it allows a programmer to pass a *slice* of the array to the function. If we wish to print the elements of the array from index 3 to index 7, call the function as

```
print(list + 3, list + 8); // Print elements at indices 3, 4, 5, 6, 7
```

Feature	Vectors	Arrays
First element at index 0	Yes	Yes
Keeps track of its own size	Yes	No
Capacity expands automatically as needed	Yes	No
May be empty	Yes	No

Table 11.1: Vectors compared to arrays

Appendix F provides a more comprehensive look at the correspondence between arrays and pointers and pointer arithmetic.

It is important to note is that arrays are not objects and, therefore, have no associated methods. The square bracket notation when used with arrays does not represent a special operator method. The square bracket array access notation is part of the core C++ language inherited from C. The creators of the C++ **vector** library designed vectors to behave as much as possible like primitive C arrays. Arrays have been part of the C language since its beginning, but vectors (added by C++) have adopted the syntactical features of arrays. Both arrays and vectors use square brackets for element access, and both locate their first element at index zero. Both provide access to a block of memory that that can hold multiple elements. A vector object adds some additional capabilities and conveniences that make vectors the better choice for C++ developers. Table 11.1 summarizes many of the differences between vectors and arrays.

Vectors have a close association with arrays—a vector object is a thin wrapper around a dynamic array. A vector object simply manages several pointers that point to its dynamic array. A vector adds value by providing convenient methods for adding elements and resizing the array it manages. A vector keeps track of its own size. Arrays provide none of convenient methods that vectors do. The overhead that a vector imposes over a raw dynamic array is negligible. You should prefer vectors over arrays when writing C++ code.

What if you are using vectors, but you need to use a C library that accepts only arrays? Fortunately it is easy to “unwrap” the array that the vector manages. If **vec** is a vector, the expression **&vec[0]** is the address of the first element within the vector. Given a function such as

```
void print(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << ' ';
    cout << endl;
}
```

that expects an array and its size, you can invoke it with your **vec** vector object as

```
print(&vec[0], vec.size());
```

Note that the first parameter is the starting address of the wrapped array, and the second parameter is the number of elements. If instead you have a function that uses a pointer range:

```
void print(int *begin, int *end) {
    for (int *elem = begin; elem != end; elem++)
        cout << *elem << ' ';
    cout << endl;
}
```

you can invoke it with your **vec** vector object as

```
print(&vec[0], &vec[0] + vec.size());
```

Note that the first parameter is the starting address of the wrapped array, and the second parameter is memory address just past the end of the wrapped array.

As these examples show, you may use vector objects instead of arrays without any danger of not being able to use libraries that deal only with arrays.

If you happen to be using an array and need to use a function that expects a vector instead, it is easy to make a vector out of an existing array. If **arr** is a raw integer array containing **n** elements, the statement

```
vector<int> vec(arr, arr + n);
```

creates a new vector object **vec** with elements that are identical to **arr**. One disadvantage to this technique is that it copies all the elements in **arr** to a new dynamic array managed by the vector.

We will see in Chapter 18 that it is relatively easy to write a function in a generic way so that it can accept and process either a vector or an array with equal efficiency.

Programmers must manage arrays and associated pointers very carefully because array and pointer misuse is a common source of difficult to find and repair bugs within programs. Vectors effectively take care of much of the memory management problems that plague raw arrays. Additionally, vectors provide functionality and convenience that arrays cannot. As we have seen, it is easy to adapt a vector to a context that requires an array. For these reasons, additional information about arrays is restricted to Appendix F, and the remainder of this book concentrates on C++ vectors rather than arrays.

11.3 Prime Generation with a Vector

Listing 11.12 (*fasterprimes.cpp*) uses an algorithm developed by the Greek mathematician Eratosthenes who lived from 274 B.C. to 195 B.C. The principle behind the algorithm is simple: Make a list of all the integers two and larger. Two is a prime number, but any multiple of two cannot be a prime number (since a multiple of two has two as a factor). Go through the rest of the list and mark out all multiples of two (4, 6, 8, ...). Move to the next number in the list (in this case, three). If it is not marked out, it must be prime, so go through the rest of the list and mark out all multiples of that number (6, 9, 12, ...). Continue this process until you have listed all the primes you want.

Listing 11.12: fasterprimes.cpp

```
// File primesieve.cpp

#include <iostream>
#include <vector>
#include <ctime>

using namespace std;

// Display the prime numbers between 2 and 500,000 and
// time how long it takes

// Largest potential prime considered
//const int MAX = 500000;
const int MAX = 500;

// Each position in the Boolean vector indicates
// if the number of that position is not prime:
// false means "prime," and true means "composite."
```

```

// Initially all numbers are prime until proven otherwise
vector<bool> nonprimes(MAX); // Global vector initialized to all false

int main() {
    clock_t start_time = clock(); // Record start time

    // First prime number is 2; 0 and 1 are not prime
    nonprimes[0] = nonprimes[1] = true;

    // Start at the first prime number, 2.
    for (int i = 2; i < MAX; i++) {
        // See if i is prime
        if (!nonprimes[i]) {
            cout << i << " ";
            // It is prime, so eliminate all of its
            // multiples that cannot be prime
            for (int j = 2*i; j < MAX; j += i)
                nonprimes[j] = true;
        }
    }
    cout << endl; // Move cursor down to next line
    // Print the elapsed time
    cout << "Elapsed time: "
         << static_cast<double>(clock() - start_time)/CLOCKS_PER_SEC
         << " seconds" << endl;
}

```

Listing 11.13: fasterprimes2.cpp

```

#include <iostream>
#include <vector>
#include <ctime>

using namespace std;

// Display the prime numbers between 2 and 500,000 and
// time how long it takes

// Largest potential prime considered
const int MAX = 2000000;

// Each position in the Boolean vector indicates
// if the number of that position is not prime:
// false means "prime," and true means "composite."
// Initially all numbers are prime until proven otherwise
vector<bool> nonprimes(MAX); // Global vector initialized to all false

void count_primes1() {
    int count = 0;
    clock_t start_time = clock(); // Record start time
    for (int value = 2; value <= MAX; value++) {
        // See if value is prime
        bool is_prime = true; // Provisionally, value is prime
        // Try all possible factors from 2 to the value - 1
        for (int trial_factor = 2;

```

```

        is_prime && trial_factor < value; trial_factor++)
        is_prime = (value % trial_factor != 0);
    if (is_prime)
        count++; // Count the prime number
}
// Print the elapsed time
cout << "Count = " << count << " ";
cout << "Elapsed time: "
    << static_cast<double>(clock() - start_time)/CLOCKS_PER_SEC
    << " seconds" << endl;
}

void count_primes2() {
    int count = 0;
    clock_t start_time = clock(); // Record start time
    for (int value = 2; value <= MAX; value++) {
        // See if value is prime
        bool is_prime = true; // Provisionally, value is prime
        double r = value, root = sqrt(r);
        // Try all possible factors from 2 to the square
        // root of value
        for (int trial_factor = 2;
            is_prime && trial_factor <= root; trial_factor++)
            is_prime = (value % trial_factor != 0);
        if (is_prime)
            count++; // Count the prime number
    }
    // Print the elapsed time
    cout << "Count = " << count << " ";
    cout << "Elapsed time: "
        << static_cast<double>(clock() - start_time)/CLOCKS_PER_SEC
        << " seconds" << endl;
}

void count_primes3() {
    int count = 0;
    clock_t start_time = clock(); // Record start time

    // First prime number is 2; 0 and 1 are not prime
    nonprimes[0] = nonprimes[1] = true;

    // Start at the first prime number, 2.
    for (int i = 2; i < MAX; i++) {
        // See if i is prime
        if (!nonprimes[i]) {
            count++; // It's prime, so count it
            // It is prime, so eliminate all of its
            // multiples that cannot be prime
            for (int j = 2*i; j < MAX; j += i)
                nonprimes[j] = true;
        }
    }
    // Print the elapsed time
    cout << "Count = " << count << " ";
    cout << "Elapsed time: "

```



```

        << static_cast<double>(clock() - start_time)/CLOCKS_PER_SEC
        << " seconds" << endl;
    }

int main() {
    count_primes1();
    count_primes2();
    count_primes3();
}

```

Recall Listing 8.8 (measureprimespeed.cpp), which also prints all the prime numbers up to 500,000. Using redirection (see Section 8.4), Listing 8.8 (measureprimespeed.cpp) takes 77 seconds. In comparison, Listing 11.12 (fasterprimes.cpp) takes about one second to perform the same task on the system. This is comparable to the square root version, Listing 8.5 (moreefficientprimes.cpp), which takes two seconds to run. If the goal is to print the prime numbers up to 1,000,000, the original version averages 271 seconds, or about four and one-half minutes. The square root version averages 4.5 seconds. The new, vector-based version averages 2 seconds. For 5,000,000 values the unoptimized original version takes a little over an hour and 33 minutes, the square root version takes a respectable 39 seconds, but vector version averages only 9 seconds to run.

11.4 Summary

- A vector represents a collection of homogeneous data values.
- A programmer must declare the type of values contained by a vector when declaring the vector itself.
- A programmer may specify a vector's initial size when declaring a vector.
- An element of a vector is accessed via a non-negative integer index using the `[]` operator; for example,

```
v[3] = 4.1;
```

The index of the desired element is placed within the square brackets.

- The first element in a vector is at index zero. If the vector contains n elements, the last element is at index $n - 1$.
- A vector's initial size may be specified when it is declared; for example,

```
vector<int> numbers(10); // numbers holds 10 integers
```

- A vector's contents may be initialized in various ways when the vector is declared, as in

```
vector<int> numbers(10); // holds ten 0s
vector<int> numbers(10, 5); // holds ten 5s
vector<int> numbers{ 2, 4, 6, 8 }; // holds elements 2, 4, 6, 8
```

- Any expression compatible with an integer can appear within the vector's `[]` operator. Any integer literal, variable, and expression is acceptable within `[]`. A strict compiler will issue a warning if a floating-point expression is used within the square brackets.
- A `for` loop is a convenient way to traverse the contents of a vector.

- Like other variables, a variable representing a vector can be local, **static** local, or global.
- A vector can be passed to a function.
- For efficiency reasons a vector parameter to a function should be declared as a reference.
- If a function is not supposed to modify the contents of a vector, best programming practices insist the vector reference parameter should be declared **const**.
- It is the programmer's responsibility to stay within the bounds of a vector. Venturing outside the bounds of a vector results in undefined behavior that introduces a logic error and often leads to a program crash.
- Two-dimensional vectors conceptually store their elements in rows and columns. C++ supports vectors of arbitrary dimensions.
- In a 2D vector expression like **a[i][j]** the first index (here **i**) represents a row, and the second index (here **j**) specifies a column.
- The **push_back** method adds a element to the end of a vector.
- The **pop_back** method removes the last element from a vector.
- The **size** method returns the number of elements that a vector holds.
- The **empty** method returns **true** if a vector contains no elements; otherwise, it returns **false**.
- The **clear** method removes all the elements from a vector.
- The range-based **for** statement allows traversal of a vector without an explicit counter variable.
- A vector is an object that wraps a more primitive data structure inherited from the C language: an array.
- Arrays come in two varieties: static and dynamic.
- Static arrays have sizes determined at compile time, while the size of dynamic arrays may be supplied at run time, if necessary.
- Programmers need **not** explicitly allocate and deallocate the memory for static arrays; programmers **must** explicitly allocate and deallocate the memory for dynamic arrays.
- Arrays, unlike vectors, have no associated methods. An array cannot even keep track of its own size.
- C++ programmers can easily extract an array from a vector to take advantage of available C library functions that process arrays.
- The **typedef** reserved word enables programmers to define simpler names for complicated C++ types.

11.5 Exercises

1. Can you declare a vector to hold a mixture of `int`s and `doubles`?
2. What happens if you attempt to access an element of an vector using a negative index?
3. Given the declaration

```
vector<int> list(100);
```

- (a) What expression represents the very first element of `list`?
 - (b) What expression represents the very last element of `list`?
 - (c) Write the code fragment that prints out the contents of `list`.
 - (d) Is the expression `list[3.0]` legal or illegal?
4. Given the declarations

```
vector<int> list{2, 3, 1, 14, 4};
int x = 2;
```

evaluate each of the following expressions:

- (a) `list[1]`
 - (b) `list[x]`
 - (c) `list.size()`
 - (d) `list.empty()`
 - (e) `list.at(3)`
 - (f) `list[x] + 1`
 - (g) `list[x + 1]`
 - (h) `list[list[x]]`
 - (i) `list[list.size() - 1]`
5. Is the following code fragment legal or illegal?


```
vector<int> list1(5), list2{ 3, 3, 3, 3, 3 };
list1 = list2;
```
 6. Provide a single declaration statement that declares an integer vector named `list` that contains the values 45, -3, 16 and 8?
 7. Does a vector keep track of the number of elements it contains?
 8. Does an array keep track of the number of elements it contains?
 9. Complete the following function that adds up all the *positive* values in an integer vector. For example, if vector `vec` contains the elements 3, -3, 5, 2, -1, and 2, the call `sum_positive(vec)` would evaluate to 12, since $3 + 5 + 2 + 2 = 12$. The function returns zero if the vector is empty. The function does not affect the contents of the vector.

```
int sum_positive(const vector<int>& v) {
    // Add your code...
}
```

10. Complete the following function that counts the even numbers in an integer vector. For example, if vector **vec** contains the elements 3, 5, 2, -1, and 2, the call **count_evens(vec)** would evaluate to 4, since $2 + 2 = 4$. The function returns zero if the vector is empty. The function does not affect the contents of the vector.

```
int count_evens(const vector<int>& v) {
    // Add your code...
}
```

11. Complete the following function that counts the even numbers in a 2D vector of integers.

```
int count_evens(const vector<vector<int>>& v) {
    // Add your code...
}
```

12. Complete the following function that compares two integer vectors to see if they contain exactly the same elements in exactly the same positions. The function returns true if the vectors are equal; otherwise, it returns false. For example, if vector **vec1** contains the elements 3, 5, 2, -1, and 2, and vector **vec2** contains the elements 3, 5, 2, -1, and 2, the call **equals(vec1, vec2)** would evaluate to true. If instead vector **vec2** contains the elements 3, 2, 5, -1, and 2, the call **equals(vec1, vec2)** would evaluate to false (the second and third elements are not in the same positions). Two vectors of unequal sizes cannot be equal. The function does not affect the contents of the vectors.

```
bool equals(const vector<int>& v1, const vector<int>& v2) {
    // Add your code...
}
```

13. Complete the following function that determines if all the elements in one vector also appear in another. The function returns true if all the elements in the second vector also appear in the first; otherwise, it returns false. For example, if vector **vec1** contains the elements 3, 5, 2, -1, 7, and 2, and vector **vec2** contains the elements 5, 7, and 2, the call **contains(vec1, vec2)** would evaluate to true. If instead vector **vec2** contains the elements 3, 8, -1, and 2, the call **contains(vec1, vec2)** would evaluate to false (8 does not appear in the first vector). Also If vector **vec2** contains the elements 5, 7, 2, and 5, the call **contains(vec1, vec2)** would evaluate to false (5 appears twice in **vec2** but only once in **vec1**, so **vec1** does not contain all the elements that appear in **vec2**). The function does not affect the contents of the vectors.

```
bool contains(const vector<int>& v1, const vector<int>& v2) {
    // Add your code...
}
```

14. Suppose your task is to implement the function with the prototype

```
void proc(vector<int> v);
```

When you implement the body of **proc**, how can you determine the size of vector **v**?

15. Consider the declaration

```
vector<vector<int>> collection(100, vector<int>(200));
```

- (a) What does the expression **collection[15][29]** represent?

- (b) How many elements does **collection** hold?
- (c) Write the C++ code that prints all the elements in **collection**. All the elements in the same row should appear on the same line, and but each successive row should appear on its own line.
- (d) What does the expression **collection[15]** represent?

16. Consider the declaration

```
vector<vector<vector<vector<int>>>>>
    mesh(100, vector<int>(200, vector<int>(100, vector<int>(50))));
```

How many elements does **mesh** hold?

- 17. How is a C++ array different from a vector?
- 18. What advantages does a C++ vector provide over an array?
- 19. Provide the statement(s) that declare and create a static array named **a** that can hold 20 integers.
- 20. Provide the statement(s) that declare and ceate a dynamic array named **a** that can hold 20 integers.
- 21. What extra attention does a programmer need to give to a static array when its use within a program is finished?
- 22. What extra attention does a programmer need to give to a dynamic array when its use within a program is finished?
- 23. What extra attention does a programmer need to give to a vector array when its use within a program is finished?
- 24. Consider the following function that processes the elements of an array using a range:

```
bool proc(const int *begin, const int *end) {
    // Details omitted . . .
}
```

and an array and vector declared as shown here:

```
int a[10];
vector<int> v;
```

- (a) Provide the statement that correctly calls **proc** with array **a**.
- (b) Provide the statement that correctly calls **proc** with vector **v**.

Chapter 12

Sorting and Searching

Chapters 11 introduced the fundamentals of making and using vectors. In this chapter we explore some common algorithms for ordering elements within a vector and for locating elements by their value rather than by their index.

12.1 Sorting

We will use the generic term *sequence* to refer to either a vector or an array. Sorting—arranging the elements within a sequence into a particular order—is a common activity. For example, a sequence of integers may be arranged in ascending order (that is, from smallest to largest). A sequence of words (strings) may be arranged in lexicographical (commonly called alphabetic) order. Many sorting algorithms exist, and some perform much better than others. We will consider one sorting algorithm that is relatively easy to implement.

The selection sort algorithm is relatively easy to implement, and it performs acceptably for smaller sequences. If A is a sequence, and i represents an index with the sequence, selection sort works as follows:

1. Set $i = 0$.
2. Examine all the elements $A[j]$, where $j > i$. If any of these elements is less than $A[i]$, then exchange $A[i]$ with the smallest of these elements. (This ensures that all elements after position i are greater than or equal to $A[i]$.)
3. If i is less than the length of A , increase i by 1 and goto Step 2.

If the condition in Step 3 is not met, the algorithm terminates with a sorted sequence. The command to “goto Step 2” in Step 3 represents a loop. We can begin to translate the above description into C++ as follows:

```
// n is A's length
for (int i = 0; i < n - 1; i++) {
    // Examine all the elements
    // A[j], where j > i.
    // If any of these A[j] is less than A[i],
    // then exchange A[i] with the smallest of these elements.
```

```
}

```

The directive at Step 2 beginning with “Examine all the elements $A[j]$, where $j > i$ ” also requires a loop. We continue refining our implementation with:

```
// n is A's length
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        // Examine all the elements
        // A[j], where j > i.
    }
    // If any A[j] is less than A[i],
    // then exchange A[i] with the smallest of these elements.
}

```

In order to determine if any of the elements is less than $A[i]$, we introduce a new variable named **small**. The purpose of **small** is to keep track of the position of the smallest element found so far. We will set **small** equal to **i** initially because we wish to locate any element less than the element found at position **i**.

```
// n is A's length
for (int i = 0; i < n - 1; i++) {
    // small is the position of the smallest value we've seen
    // so far; we use it to find the smallest value less than A[i]
    int small = i;
    for (int j = i + 1; j < n; j++) {
        if (A[j] < A[small])
            small = j; // Found a smaller element, update small
    }
    // If small changed, we found an element smaller than A[i]
    if (small != i)
        // exchange A[small] and A[i]
}

```

Listing 12.1 (sortintegers.cpp) provides the complete C++ implementation of the **selection_sort** function within a program that tests it out.

Listing 12.1: sortintegers.cpp

```
#include <iostream>
#include <vector>

using namespace std;

/*
 * swap(a, b)
 *   Interchanges the values of memory
 *   referenced by its parameters a and b.
 *   It effectively interchanges the values
 *   of variables in the caller's context.
 */
void swap(int& a, int& b) {
    int temp = a;
    a = b;

```

```

    b = temp;
}

/*
 *  selection_sort
 *      Arranges the elements of vector a into ascending order.
 *      a is a vector that contains integers.
 */
void selection_sort(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Note: i, small, and j represent positions within a
        // a[i], a[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        int small = i;
        // See if a smaller value can be found later in the vector
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[small])
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            swap(a[i], a[small]);
    }
}

/*
 *  print
 *      Prints the contents of a vector of integers.
 *      a is the vector to print.
 *      a is not modified.
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << '}';
}

int main() {
    vector<int> list{23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10};
    cout << "Before: ";
    print(list);
    cout << endl;
    selection_sort(list);
    cout << "After: ";
    print(list);
    cout << endl;
}

```



```
}

```

Listing 12.1 (sortintegers.cpp) uses a fancier **print** routine, separating the vector's elements with commas. The program's output is

```
Before: {23,-3,4,215,0,-3,2,23,100,88,-10}
After:  {-10,-3,-3,0,2,4,23,23,88,100,215}
```

We really do not need to write our own function to interchange the values of two integers as shown in Listing 12.1 (sortintegers.cpp). The C++ standard library includes **std::swap** that works just like the **swap** function in Listing 12.1 (sortintegers.cpp); therefore, if you remove the definition of **swap** from Listing 12.1 (sortintegers.cpp), the program will work just as well.

12.2 Flexible Sorting

What if we want to change the behavior of the sorting function in Listing 12.1 (sortintegers.cpp) so that it arranges the elements in descending order instead of ascending order? It is actually an easy modification; simply change the line

```
if (a[j] < a[small])
```

to be

```
if (a[j] > a[small])
```

Suppose we want to change the sort so that it sorts the elements in ascending order except that all the even numbers in the vector appear before all the odd numbers? This would take a little more effort, but it still is possible to do.

The next question is more intriguing: How can we rewrite the **selection_sort** function so that, by passing an additional parameter, it can sort the vector in any way we want?

We can make our sort function more flexible by making it higher-order function (see Section 10.9) that accepts an ordering function as a parameter. Listing 12.2 (flexibleintsor.cpp) arranges the elements in a vector two different ways using the same **selection_sort** function.

Listing 12.2: flexibleintsor.cpp

```
#include <iostream>
#include <vector>

using namespace std;

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
bool less_than(int a, int b) {
    return a < b;
}
```

```

}

/*
 * greater_than(a, b)
 * Returns true if a > b; otherwise, returns
 * false.
 */
bool greater_than(int a, int b) {
    return a > b;
}

/*
 * selection_sort(a, compare)
 * Arranges the elements of a in an order determined
 * by the compare function.
 * a is a vector of integers.
 * compare is a function that compares the ordering of
 * two integers.
 */
void selection_sort(vector<int>& a, bool (*compare)(int, int)) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Note: i, small, and j represent positions within a
        // a[i], a[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        int small = i;
        // See if a smaller value can be found later in the vector
        for (int j = i + 1; j < n; j++)
            if (compare(a[j], a[small]))
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            swap(a[i], a[small]); // Uses std::swap
    }
}

/*
 * print
 * Prints the contents of an integer vector
 * a is the vector to print.
 * a is not modified.
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << ' ';
}

```

```
int main() {
    vector<int> list{ 23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10 };
    cout << "Original:  ";
    print(list);
    cout << endl;
    selection_sort(list, less_than);
    cout << "Ascending: ";
    print(list);
    cout << endl;
    selection_sort(list, greater_than);
    cout << "Descending: ";
    print(list);
    cout << endl;
}
```

Listing 12.2 (flexibleintsor.cpp) takes advantage of the standard **swap** function (see Section 12.1).

The output of Listing 12.2 (flexibleintsor.cpp) is

```
Original:  {23,-3,4,215,0,-3,2,23,100,88,-10}
Ascending: {-10,-3,-3,0,2,4,23,23,88,100,215}
Descending: {215,100,88,23,23,4,2,0,-3,-3,-10}
```

The comparison function passed to the sort routine customizes the sort's behavior. The basic structure of the sorting algorithm does not change, but its notion of ordering is adjustable. If the second parameter to **selection_sort** is **less_than**, the sort routine arranges the elements into ascending order. If the caller passes **greater_than** instead, **selection_sort** rearranges vector's elements into descending order. More creative orderings are possible with more elaborate comparison functions.

Selection sort is a relatively efficient simple sort, but more advanced sorts are, on average, much faster than selection sort, especially for large data sets. One such general purpose sort is *Quicksort*, devised by C. A. R. Hoare in 1962. Quicksort is the fastest known general purpose sort. Since sorting is a common data processing activity, the standard C library provides a function named **qsort** that implements Quicksort. More information about Quicksort and **qsort** is available at <http://en.wikipedia.org/wiki/Quicksort>.

12.3 Search

Searching a vector for a particular element is a common activity. We will consider the two most common search strategies: linear search and binary search.

12.3.1 Linear Search

Listing 12.3 (linearsearch.cpp) uses a function named **locate** that returns the position of the first occurrence of a given element in a vector of integers; if the element is not present, the function returns **-1**.

Listing 12.3: linearsearch.cpp

```

#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

/*
 * locate(a, seek)
 * Returns the index of element seek in vector a.
 * Returns -1 if seek is not an element of a.
 * a is the vector to search.
 * seek is the element to find.
 */
int locate(const vector<int>& a, int seek) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        if (a[i] == seek)
            return i; // Return position immediately
    return -1; // Element not found
}

/*
 * format(i)
 * Prints integer i right justified in a 4-space
 * field. Prints "****" if i > 9,999.
 */
void format(int i) {
    if (i > 9999)
        cout << "****" << endl; // Too big!
    else
        cout << setw(4) << i;
}

/*
 * print(v)
 * Prints the contents of an int vector.
 * v is the vector to print.
 */
void print(const vector<int>& v) {
    for (int i : v)
        format(i);
}

/*
 * display(a, value)
 * Draws an ASCII art arrow showing where
 * the given value is within the vector.
 * a is the vector.
 * value is the element to locate.
 */
void display(const vector<int>& a, int value) {

```

```
int position = locate(a, value);
if (position >= 0) {
    print(a);                // Print contents of the vector
    cout << endl;
    position = 4*position + 7; // Compute spacing for arrow
    cout << setw(position);
    cout << "  ^  " << endl;
    cout << setw(position);
    cout << "  |  " << endl;
    cout << setw(position);
    cout << "  +-- " << value << endl;
}
else {
    cout << value << " not in ";
    print(a);
    cout << endl;
}
cout << "=====" << endl;
}

int main() {
    vector<int> list{ 100, 44, 2, 80, 5, 13, 11, 2, 110 };
    display(list, 13);
    display(list, 2);
    display(list, 7);
    display(list, 100);
    display(list, 110);
}
```

The output of Listing 12.3 (linearsearch.cpp) is

```

100  44   2  80   5  13  11   2 110
                        ^
                        |
                        +-- 13
=====
100  44   2  80   5  13  11   2 110
                        ^
                        |
                        +-- 2
=====
7 not in 100  44   2  80   5  13  11   2 110
=====
100  44   2  80   5  13  11   2 110
      ^
      |
      +-- 100
=====
100  44   2  80   5  13  11   2 110
                                   ^
                                   |
                                   +-- 110
=====

```

The key function in Listing 12.3 (linearsrch.cpp) is **locate**; all the other functions simply lead to a more interesting display of **locate**'s results. If **locate** finds a match, it immediately returns the position of the matching element; otherwise, if **locate** considers all the elements of the vector and finds no match, it returns -1 . -1 is a good indication of failure, since -1 is not a valid index in a C++ vector.

The other functions are

- **format** prints an integer right justified within a four-space field. Extra spaces pad the beginning of the number if necessary.
- **print** prints out the elements in any vector using the **format** function to properly format the values. This alignment simplifies the **display** function.
- **display** uses **locate**, **print**, and **setw** to provide a graphical display of the operation of the **locate** function.

The kind of search performed by **locate** is known as *linear search*, since the process takes a straight line path from the beginning to the end of the vector, and it considers each element in order. Figure 12.1 illustrates linear search.

12.3.2 Binary Search

Linear search is acceptable for relatively small vectors, but the process of examining each element in a large vector is time consuming. An alternative to linear search is *binary search*. In order to perform binary search a vector's elements must be in sorted order. Binary search exploits this sorted structure of the vector using a clever but simple strategy that quickly zeros in on the element to find:

```
int list[] = { 100, 44, 2, 80, 5, 13, 11, 2, 110 };
int x = locate(list, 9, 13);
```

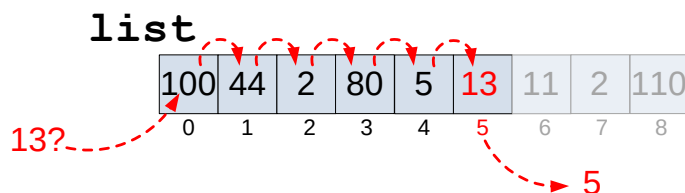


Figure 12.1: Linear search. The grayed out elements are not considered during the search process.

1. If the vector is empty, return -1 .
2. Check the element in the middle of the vector. If the element is what you are seeking, return its position. If the middle element is larger than the element you are seeking, perform a binary search on the first half of the vector. If the middle element is smaller than the element you are seeking, perform a binary search on the second half of the vector.

This approach is analogous to looking for a telephone number in the phone book in this manner:

1. Open the book at its center. If the name of the person is on one of the two visible pages, look at the phone number.
2. If not, and the person's last name is alphabetically less the names on the visible pages, apply the search to the left half of the open book; otherwise, apply the search to the right half of the open book.
3. Discontinue the search with failure if the person's name should be on one of the two visible pages but is not present.

Listing 12.4 (binarysearch.cpp) contains a C++ function that implements the binary search algorithm.

Listing 12.4: binarysearch.cpp

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

/*
 *  binary_search(a, seek)
 *      Returns the index of element seek in vector a;
 *      returns -1 if seek is not an element of a
 *      a is the vector to search; a's contents must be
 *      sorted in ascending order.
 *      seek is the element to find.
 */
int binary_search(const vector<int>& a, int seek) {
```

```

    int first = 0,           // Initially the first position
        last = a.size() - 1, // Initially the last position
        mid;                // The middle of the vector
    while (first <= last) {
        mid = first + (last - first + 1)/2;
        if (a[mid] == seek)
            return mid;      // Found it
        else if (a[mid] > seek)
            last = mid - 1;   // continue with 1st half
        else // a[mid] < seek
            first = mid + 1;  // continue with 2nd half
    }
    return -1;               // Not there
}

/*
 *  format(i)
 *      Prints integer i right justified in a 4-space
 *      field. Prints "****" if i > 9,999.
 */
void format(int i) {
    if (i > 9999)
        cout << "****" << endl; // Too big!
    else
        cout << setw(4) << i;
}

/*
 *  print(v)
 *      Prints the contents of an int vector.
 *      v is the vector to print.
 */
void print(const vector<int>& v) {
    for (int i : v)
        format(i);
}

/*
 *  display(a, value)
 *      Draws an ASCII art arrow showing where
 *      the given value is within the vector.
 *      a is the vector.
 *      value is the element to locate.
 */
void display(const vector<int>& a, int value) {
    int position = binary_search(a, value);
    if (position >= 0) {
        print(a); // Print contents of the vector
        cout << endl;
        position = 4*position + 7; // Compute spacing for arrow
        cout << setw(position);
        cout << "  ^  " << endl;
        cout << setw(position);
    }
}

```



```

        cout << " | " << endl;
        cout << setw(position);
        cout << " +--- " << value << endl;
    }
    else {
        cout << value << " not in ";
        print(a);
        cout << endl;
    }
    cout << "=====" << endl;
}

int main() {
    // Check binary search on even- and odd-length vectors and
    // an empty vector
    vector<int> even_list{ 1, 2, 3, 4, 5, 6, 7, 8 },
               odd_list{ 1, 2, 3, 4, 5, 6, 7, 8, 9 },
               empty_list;

    for (int i = -1; i <= 10; i++)
        display(even_list, i);
    for (int i = -1; i <= 10; i++)
        display(odd_list, i);
    for (int i = -1; i <= 10; i++)
        display(empty_list, i);
}

```

In the `binary_search` function:

- The initializations of `first` and `last`:

```

    int first = 0,           // Initially the first position
        last = a.size() - 1, // Initially the last position

```

ensure that `first` is less than or equal to `last` for a nonempty vector. If the vector is empty, `first` is zero, and `last` is equal to `n - 1` which equals `-1`. So in the case of an empty vector `binary_search` will skip the loop body and immediately return `-1`. This is correct behavior because an empty vector cannot possibly contain any item we seek.

- The variable `mid` represents the midpoint value between `first` and `last`, and it is computed in the statement

```

    mid = first + (last - first + 1)/2;

```

This arithmetic may look a bit complex. The more straightforward way to compute the average of two values would be

```

    mid = (first + last)/2; // Alternate calculation

```

The problem with this method of computing the midpoint is it fails to produce the correct results more times than the version found in Listing 12.4 (`binarysearch.cpp`). The problem arises if the vector is large, and `first` and `last` both have relatively large values. The expression `first + last` can overflow the range of integers producing a meaningless result. The subsequent division by two is too late to help. The result of the expression `first + last` would overflow the range of integers more often than the expression `first + (last - first + 1)/2` because `(last - first + 1)/2`

would be a much smaller value to add to **first**. If you are thinking “I would never have thought of that problem,” don’t worry too much. A large number of competent, professional software engineers have fallen prey to this oversight (see <http://googleresearch.blogspot.com/2006/06/extra-extra-read-all-about-it-nearly.html>).

The calculation of **mid** ensures that **first** $\leq$ **mid** $\leq$ **last**. This particular

- If **mid** is the location of the sought element (checked in the first **if** statement), the loop terminates, and returns the correct position.
- The second **if** statement ensures that either **last** decreases or **first** increases each time through the loop. Thus, if the loop does not terminate for other reasons, eventually **first** will be larger than **last**, and the loop will terminate. If the loop terminates for this reason, the function returns -1 . This is the correct behavior.
- The second **if** statement excludes the irrelevant elements from further search. The number of elements remaining to consider is effectively cut in half.

Figure 12.2 illustrates how binary search works.

```
int list[] = { 10, 14, 20, 28, 29, 33, 34, 45, 48 };
int x = binary_search(list, 9, 33);
```

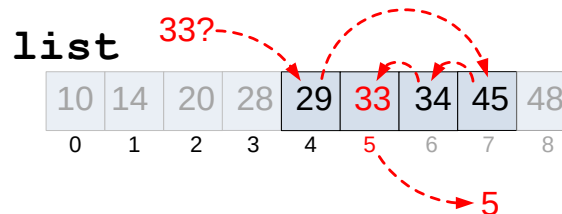


Figure 12.2: Binary search. The grayed out elements are not considered during the search.

The implementation of the binary search algorithm is more complicated than the simpler linear search algorithm. Ordinarily simpler is better, but for algorithms that process data structures that potentially hold large amounts of data, more complex algorithms employing clever tricks that exploit the structure of the data (as binary search does) often dramatically outperform simpler, easier-to-code algorithms.

For a fair comparison of linear vs. binary search, suppose we want to locate an element in a sorted vector. That the vector is ordered is essential for binary search, but it can be helpful for linear search as well. The revised linear search algorithm is

```
// This version requires vector a to be sorted in
// ascending order.
int linear_search(const vector<int>& a, int seek) {
    int n = a.size();
    for (int i = 0; i < n && a[i] <= seek; i++)
        if (a[i] == seek)
            return i; // Return position immediately
```

```

    return -1; // Element not found
}

```

Notice that, as in the original version of linear search, the loop will terminate when all the elements have been examined, but it also will terminate early when it encounters an element larger than the sought element. Since the vector is sorted, there is no need to continue the search once you have begun seeing elements larger than your sought value; **seek** cannot appear after a larger element in a sorted vector.

Suppose a vector to search contains n elements. In the worst case—looking for an element larger than any currently in the vector—the loop in linear search takes n iterations. In the best case—looking for an element smaller than any currently in the vector—the function immediately returns without considering any other elements. The number of loop iterations thus ranges from 1 to n , and so on average linear search requires $\frac{n}{2}$ comparisons before the loop finishes and the function returns.

Now consider binary search of a vector that contains n elements. After each comparison the size of the vector left to consider is one-half the original size. If the sought item is not found on the first probe, the number of remaining elements to search is $\frac{n}{2}$. After the next time through the loop, the number of elements left to consider is one-half of $\frac{n}{2}$, or $\frac{n}{4}$. After the third iteration, search space in the vector drops to one-half of $\frac{n}{4}$, which is $\frac{n}{8}$. This process of cutting the search space in half continues each time through the loop until the process locates the sought element or runs out of elements to consider. The problem of determining how many times a set of things can be divided in half until only one element remains can be solved with a base-2 logarithm. For binary search, the worst case scenario of not finding the sought element requires the loop to make $\log_2 n$ iterations.

How does this analysis help us determine which search is better? The quality of an algorithm is judged by two key characteristics:

- How much time (processor cycles) does it take to run?
- How much space (memory) does it take to run?

In our situation, both search algorithms process the sequence with only a few extra local variables, so for large sequences they both require essentially the same space. The big difference here is speed. Binary search performs more elaborate computations each time through the loop, and each operation takes time, so perhaps binary search is slower. Linear search is simpler (fewer operations through the loop), but perhaps its loop executes many more times than the loop in binary search, so overall it is slower.

We can deduce the faster algorithm in two ways: empirically and analytically. An empirical test is an experiment; we carefully implement both algorithms and then measure their execution times. The analytical approach analyzes the source code to determine how many operations the computer's processor must perform to run the program on a problem of a particular size.

Listing 12.5 (searchcompare.cpp) measures the running times of the two kinds of searches to compare the two algorithms empirically.

Listing 12.5: searchcompare.cpp

```

#include <iostream>
#include <iomanip>
#include <vector>
#include <ctime>

using namespace std;

/*

```

```

* binary_search(a, seek)
* Returns the index of element seek in vector a;
* returns -1 if seek is not an element of a
* a is the vector to search; a's contents must be
* sorted in ascending order.
* seek is the element to find.
*/
int binary_search(const vector<int>& a, int seek) {
    int n = a.size();
    int first = 0,      // Initially the first element in vector
        last = n - 1,  // Initially the last element in vector
        mid;           // The middle of the vector
    while (first <= last) {
        mid = first + (last - first + 1)/2;
        if (a[mid] == seek)
            return mid;      // Found it
        else if (a[mid] > seek)
            last = mid - 1;   // continue with 1st half
        else // a[mid] < seek
            first = mid + 1;  // continue with 2nd half
    }
    return -1;      // Not there
}

/*
* linear_search(a, seek)
* Returns the index of element seek in vector a.
* Returns -1 if seek is not an element of a.
* a is the vector to search.
* seek is the element to find.
* This version requires vector a to be sorted in
* ascending order.
*/
int linear_search(const vector<int>& a, int seek) {
    int n = a.size();
    for (int i = 0; i < n && a[i] <= seek; i++)
        if (a[i] == seek)
            return i;      // Return position immediately
    return -1;      // Element not found
}

/*
* Tests the execution speed of a given search function on a
* vector.
* search - the search function to test
* v      - the vector to search
* trials - the number of trial runs to average
* Returns the elapsed time in seconds
* The C++ chrono library defines the types
* system_clock::time_point and microseconds.
*/
double time_execution(int (*search)(const vector<int>&, int),
                     vector<int>& v, int trials) {
    int n = v.size();

```

```

// Average the time over a specified number of trials
double elapsed = 0.0;
for (int iters = 0; iters < trials; iters++ ) {
    clock_t start_time = clock(); // Start the timer
    for (int i = 0; i < n; i++) // Search for all elements
        search(v, i);
    clock_t end_time = clock(); // Stop the timer
    elapsed += static_cast<double>(end_time - start_time)/CLOCKS_PER_SEC;
}
return elapsed/trials; // Mean elapsed time per run
}

int main() {
    cout << "-----" << endl;
    cout << "   Vector           Linear           Binary" << endl;
    cout << "   Size           Search           Search" << endl;
    cout << "-----" << endl;

    // Test the sorts on vectors with 1,000 elements up to
    // 10,000 elements.
    for (int size = 0; size <= 50000; size += 5000) {
        vector<int> list(size); // Make a vector of the appropriate size

        // Ensure the elements are ordered low to high
        for (int i = 0; i < size; i++)
            list[i] = i;

        cout << setw(7) << size;
        // Search for all the elements in list using linear search
        // Compute running time averaged over five runs
        cout << fixed << setprecision(3) << setw(12)
            << time_execution(linear_search, list, 5)
            << " sec";

        // Search for all the elements in list binary search
        // Compute running time averaged over 25 runs
        cout << fixed << setprecision(3) << setw(12)
            << time_execution(binary_search, list, 25)
            << " sec" << endl;
    }
}

```

Listing 12.5 (searchcompare.cpp) applies linear search and binary search to vectors of various sizes and displays the results. The vector sizes range from 0 to 50,000. The program uses the function `time_execution` to compute the average running times of linear search and binary search. The `main` function directs which search `time_execution` should perform by passing as the first parameter a pointer to the appropriate function. The second parameter to `time_execution` specifies the number of runs the function should use to compute the average. Notice that in this program we average linear search over five runs and execute binary search over 25 runs. We subject the binary search function to more runs since it executes so quickly, and five runs is not an adequate sample size to evaluate its performance, especially for smaller vectors where the binary search's execution time is close to the resolution of the timer. Besides, since binary search does execute so quickly, we easily can afford to let `time_execution` run more tests and compute a more accurate average.

In Listing 12.5 (searchcompare.cpp) we use two new stream manipulators, **fixed** and **setprecision**, to dress up the output. The **fixed** manipulator adjusts **cout** to use a fixed number of decimal places, and the **setprecision** manipulator specifies the number of digits to display after the decimal point. The two manipulators in tandem allow us to align the columns of numbers by their decimal points.

A sample run of Listing 12.5 (searchcompare.cpp) displays

Vector Size	Linear Search	Binary Search
0	0.000 sec	0.000 sec
5000	0.112 sec	0.001 sec
10000	0.444 sec	0.002 sec
15000	1.003 sec	0.003 sec
20000	2.172 sec	0.007 sec
25000	3.444 sec	0.005 sec
30000	5.254 sec	0.006 sec
35000	7.216 sec	0.008 sec
40000	9.701 sec	0.009 sec
45000	11.709 sec	0.018 sec
50000	12.287 sec	0.012 sec

With a vector of size 50,000 linear search takes on average about 12 seconds on one system, while binary search requires a small fraction of a one second. If we increase the vector's size to 500,000, linear search runs in 830 seconds (13 minutes, 50 seconds), while binary search still takes less than one second! Empirically, binary search performs dramatically better than linear search.

In addition to using the empirical approach, we can judge which algorithm is better by analyzing the source code for each function. Each arithmetic operation, assignment, logical comparison, and vector access requires time to execute. We will assume each of these activities requires one unit of processor “time.” This assumption is not strictly true, but it will give acceptable results for relative comparisons. Since we will follow the same rules when analyzing both search algorithms, the relative results for comparison purposes will be fairly accurate.

We first consider linear search:

```
int linear_search(const vector<int>& a, int seek) {
    int n = a.size();
    for (int i = 0; i < n && a[i] != seek; i++)
        if (a[i] == seek)
            return i;    // Return position immediately
    return -1;    // Element not found
}
```

We determined that, on average, the loop makes $\frac{n}{2}$ iterations for a vector of length n . The initialization of **i** happens only one time during each call to **linear_search**. All other activity involved with the loop except the **return** statements happens $\frac{n}{2}$ times. The function returns either **i** or **-1**, and only one return is executed during each call. Table 12.1 shows the breakdown for linear search.

The running time of the **linear_search** function thus can be expressed as a simple linear function: $L(n) = 3n + 4$.

Action	Operations	Operation Cost	Iterations	Cost
<code>n = a.size()</code>	<code>=, a.size</code>	2	1	2
<code>i = 0</code>	<code>=</code>	1	1	1
<code>i < size && a[i] <= seek</code>	<code><=, &&, [], <=</code>	4	$n/2$	$2n$
<code>a[i] == seek</code>	<code>[], ==</code>	2	$n/2$	n
<code>return i or return -1</code>	<code>return</code>	1	1	1
Total Cost				$3n + 4$

Table 12.1: Analysis of Linear Search

Action	Operations	Operation Cost	Iterations	Cost
<code>n = a.size()</code>	<code>=, a.size</code>	2	1	2
<code>first = 0</code>	<code>=</code>	1	1	1
<code>last = n - 1</code>	<code>=, -</code>	2	1	2
<code>first <= last</code>	<code><=</code>	1	$\log_2 n$	$\log_2 n$
<code>mid = first + (last - first + 1)/2</code>	<code>=, +, -, +, /</code>	5	$\log_2 n$	$5 \log_2 n$
<code>v[mid] == seek</code>	<code>[], ==</code>	2	$\log_2 n$	$2 \log_2 n$
<code>v[mid] > seek</code>	<code>[], ></code>	2	$\log_2 n$	$2 \log_2 n$
<code>last = mid - 1 or first = mid + 1</code>	<code>=, ±</code>	2	$\log_2 n$	$2 \log_2 n$
<code>return mid or return -1</code>	<code>return</code>	1	1	1
Total Cost				$12 \log_2 n + 6$

Table 12.2: Analysis of Binary Search

Next, we consider binary search:

```

int binary_search(const vector<int>& a, int seek) {
    int n = a.size(),    // Number of elements
        first = 0,       // Initially the first element in vector
        last = n - 1,    // Initially the last element in vector
        mid;             // The middle of the vector
    while (first <= last) {
        mid = first + (last - first + 1)/2;
        if (a[mid] == seek)
            return mid;    // Found it
        else if (a[mid] > seek)
            last = mid - 1; // continue with 1st half
        else // a[mid] < seek
            first = mid + 1; // continue with 2nd half
    }
    return -1;    // Not there
}

```

We determined that in the worst case the loop in `binary_search` iterates $\log_2 n$ times if the vector contains n elements. The two initializations before the loop are performed once per call. Most of the actions within the loop occur $\log_2 n$ times, except that only one `return` statement can be executed per call, and in the `if/else` statement only one path can be chosen per loop iteration. Table 12.2 shows the complete analysis of binary search.

We will call our binary search function $B(n)$. Figure 12.3 shows the plot of the two functions $L(n) = 3n + 4$ and $B(n) = 12\log_2 n + 6$. For $n < 17$, the linear function $3n + 4$ is less than the binary function

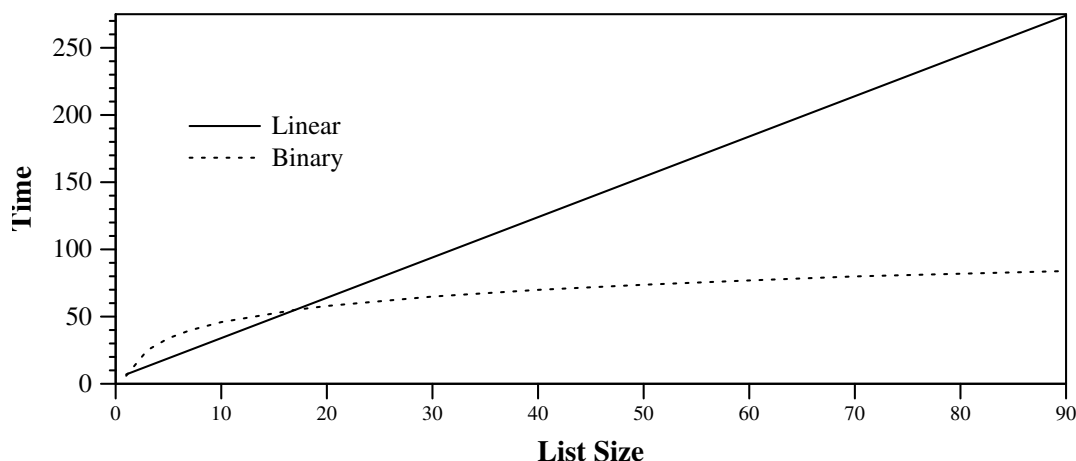


Figure 12.3: A graph of the functions derived from analyzing the linear and binary search routines

$12\log_2 n + 6$. This means that linear search should perform better than binary search for vector sizes less than 17. This is because the code for linear search is less complicated, and it can complete its work on smaller vectors before the binary search finishes its more sophisticated computations. At $n = 17$, however, the two algorithms should perform about the same because

$$L(17) = 3(17) + 4 = 51 + 4 = 55 \approx 55.05 = 49.05 + 6 = 12(4.09) + 6 = 12\log_2 17 + 6 = B(17)$$

Figure 12.3 shows that for all $n > 17$ binary search outperforms linear search, and the performance gap increases rapidly as n grows. This wide performance discrepancy agrees with our empirical observations we obtained from Listing 12.5 (`searchcompare.cpp`). Unfortunately we cannot empirically compare the running times of the two searches for vectors small enough to demonstrate that linear search is faster for very small vectors. As the output of Listing 12.5 (`searchcompare.cpp`) shows, both searches complete their work in time less than the resolution of our timer for vectors with 1,000 elements. Both empirically and analytically, we see that binary search is fast even for very large vectors, while linear search is impractical for large vectors.

12.4 Vector Permutations

Sometimes it is useful to consider all the possible arrangements of the elements within a vector. A sorting algorithm, for example, must work correctly on any initial arrangement of elements in a vector. To test a sort function, a programmer could check to see if it produces the correct result for all arrangements of a relatively small vector. A rearrangement of a collection of ordered items is called a *permutation*. Listing 12.6 (`vectorpermutations.cpp`) prints all the permutations of the contents of a given vector.

Listing 12.6: `vectorpermutations.cpp`

```
#include <iostream>
#include <vector>
```



```

using namespace std;

/*
 * print
 * Prints the contents of a vector of integers
 * a is the vector to print; a is not modified
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << "{";
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << "}";
}

/*
 * Prints all the permutations of vector a in the
 * index range begin...end, inclusive. The function's
 * behavior is undefined if begin or end
 * represents an index outside of the bounds of vector a.
 */
void permute(vector<int>& a, int begin, int end) {
    if (begin == end) {
        print(a);
        cout << endl;
    }
    else {
        for (int i = begin; i <= end; i++) {
            // Interchange the element at the first position
            // with the element at position i
            swap(a[begin], a[i]);
            // Recursively permute the rest of the vector
            permute(a, begin + 1, end);
            // Interchange the current element at the first position
            // with the current element at position i
            swap(a[begin], a[i]);
        }
    }
}

/*
 * Tests the permutation functions
 */
int main() {
    // Get number of values from the user
    cout << "Please enter number of values to permute: ";
    int number;
    cin >> number;
    // Create the vector to hold all the values
    vector<int> list(number);
    // Initialize the vector
    for (int i = 0; i < number; i++)

```

```

        list[i] = i;

    // Print original list
    print(list);
    cout << endl << "-----" << endl;
    // Print all the permutations of list
    permute(list, 0, number - 1);
    cout << endl << "-----" << endl;
    // Print list after all the manipulations
    print(list);
}

```

A sample run of Listing 12.6 (vectorpermutations.cpp) when the user enters 4 prints

```

Please enter number of values to permute: 4
{0,1,2,3}
-----
{0,1,2,3}
{0,1,3,2}
{0,2,1,3}
{0,2,3,1}
{0,3,2,1}
{0,3,1,2}
{1,0,2,3}
{1,0,3,2}
{1,2,0,3}
{1,2,3,0}
{1,3,2,0}
{1,3,0,2}
{2,1,0,3}
{2,1,3,0}
{2,0,1,3}
{2,0,3,1}
{2,3,0,1}
{2,3,1,0}
{3,1,2,0}
{3,1,0,2}
{3,2,1,0}
{3,2,0,1}
{3,0,2,1}
{3,0,1,2}
-----
{0,1,2,3}

```

The **permute** function in Listing 12.6 (vectorpermutations.cpp) is a recursive function, as it calls itself inside of its definition. We have seen how recursion can be an alternative to iteration; however, the **permute** function here uses *both* iteration *and* recursion together to generate all the arrangements of a vector. At first glance, the combination of these two algorithm design techniques as used here may be

difficult to follow, but we actually can understand the process better if we *ignore* some of the details of the code.

First, notice that in the recursive call the argument **begin** is one larger, and **end** remains the same. This means as the recursion progresses the ending index never changes, and the beginning index keeps increasing until it reaches the ending index. The recursion terminates when **begin** becomes equal to **end**.

In its simplest form the function looks like this:

```
void permute(int *a, int begin, int end) {
    if (begin == end)
        // Print the whole vector
    else
        // Do the interesting part of the algorithm
}
```

Let us zoom in on the interesting part of the algorithm (less the comments):

```
for (int i = begin; i <= end; i++) {
    swap(a[begin], a[i]);
    permute(a, begin + 1, end);
    swap(a[begin], a[i]);
}
```

If the mixture of iteration and recursion is confusing, eliminate iteration!

If a loop iterates a fixed number of times, you may replace the loop with the statements in its body duplicated that number times; for example, we can rewrite the code

```
for (int i = 0; i < 5; i++)
    cout << i << endl;
```

as

```
cout << 0 << endl;
cout << 1 << endl;
cout << 2 << endl;
cout << 3 << endl;
cout << 4 << endl;
```

Notice that the loop is gone. This process of transforming a loop into the series of statements that the loop would perform is known as *loop unrolling*. Compilers sometimes unroll loops to make the code's execution faster. After unrolling the loop the loop control variable (in this case **i**) is gone, so there is no need to initialize it (done once) and, more importantly, no need to check its value and update it during each iteration of the loop.

Our purpose for unrolling the loop in **perform** is not to optimize it. Instead we are trying to understand better how the algorithm works. In order to unroll **perform**'s loop, we will consider the case for vectors containing exactly three elements. In this case the **for** in the **perform** function would be hardcoded as

```
for (int i = 0; i <= 2; i++) {
    swap(a[begin], a[i]);
    permute(a, begin + 1, end);
    swap(a[begin], a[i]);
}
```

and we can transform this code into

```

swap(a[begin], a[0]);
permute(a, begin + 1, end);
swap(a[begin], a[0]);

swap(a[begin], a[1]);
permute(a, begin + 1, end);
swap(a[begin], a[1]);

swap(a[begin], a[2]);
permute(a, begin + 1, end);
swap(a[begin], a[2]);

```

Once the loop is gone, we see we have simply a series of recursive calls of **permute** sandwiched by calls to **swap**. The first call to **swap** interchanges an element in the vector with the first element. The second call to **swap** reverses the effects of the first swap. This series of **swap-permute-swap** operations allows each element in the vector to have its turn being the first element in the permuted vector. The **permute** recursive call generates all the permutations of the rest of the list. Figure 12.4 traces the recursive process of generating all the permutations of the vector **{0,1,2}**. The leftmost third of Figure 12.4 shows the original contents of the vector and the initial call of **permute**. The three branches represent the three iterations of the **for** loop: **i** varying from **begin** (0) to **end** (2). The vectors indicate the state of the vector after the first swap but before the recursive call to **permute**.

The middle third of Figure 12.4 shows the state of the vector during the first recursive call to **permute**. The two branches represent the two iterations of the **for** loop: **i** varying from **begin** (1) to **end** (2). The vectors indicate the state of the vector after the first swap but before the next recursive call to **permute**. At this level of recursion the element at index zero is fixed, and the remainder of the processing during this chain of recursion is restricted to indices greater than zero.

The rightmost third of Figure 12.4 shows the state of the vector during the second recursive call to **permute**. At this level of recursion the elements at indices zero and one are fixed, and the remainder of the processing during this chain of recursion is restricted to indices greater than one. This leaves the element at index two, but this represents the base case of the recursion because **begin** (2) equals **end** (2). The function makes no more recursive calls to itself. The function merely prints the current contents of the vector.

The arrows in Figure 12.4 represent a call to, or a return from, **permute**. They illustrate the recursive call chain. The arrows pointing left to right represent a call, and the arrows pointing from right to left represent a return from the function. The numbers associated with arrow indicate the order in which the calls and returns occur during the execution of **permute**.

The second column from the left shows the original contents of the vector after the first **swap** call but before the first recursive call to **permute**. The swapped elements appear in red. The third column shows the contents of the vector at the second level of recursion. In the third column the elements at index zero are fixed, as this recursion level is using **begin** with a value of one instead of zero. The **for** loop within this recursive call swaps the elements highlighted in red. The rightmost column is the point where **begin** equals **end**, and so the **permute** function does not call itself effectively terminating the recursion.

While Listing 12.6 (vectorpermutations.cpp) is a good exercise in vector manipulation and recursion, the C++ standard library provides a function named **next_permutation** that rearranges the elements of a vector. Listing 12.7 (stlpermutations.cpp) uses **next_permutation** within a loop to print all the permutations of the vector's elements.

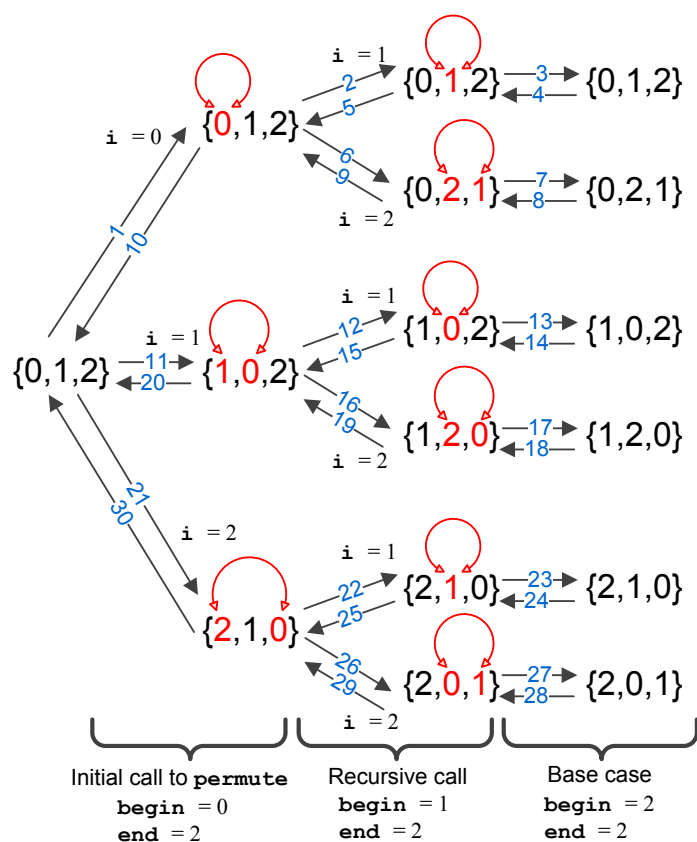


Figure 12.4: A tree mapping out the recursive process of the `permute` function operating on the vector `{}`.

Listing 12.7: `stlpermutations.cpp`

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

/*
 * print
 * Prints the contents of an int vector
 * a is the vector to print; a is not modified
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << "{";
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
```

```

        cout << ',' << a[i]; // Print the rest
    }
    cout << "};";
}

int main() {
    vector<int> nums { 0, 1, 2, 3 };
    cout << "-----" << endl;
    do {
        print(nums);
        cout << endl;
    } // Compute the next ordering of elements
    while (next_permutation(begin(nums), end(nums)));
}

```

12.5 Randomly Permuting a Vector

Section 12.4 showed how we can generate all the permutations of a vector in an orderly fashion. More often, however, we need to produce one of those permutations chosen at random. For example, we may need to randomly rearrange the contents of an ordered vector so that we can test a sort function to see if it will produce the original ordered sequence. We could generate all the permutations, put each one in a vector of vectors, and select a permutation at random from that vector of vectors. This approach is very inefficient, especially as the length of the vector to permute grows larger. Fortunately, we can randomly permute the contents of a vector easily and quickly. Listing 12.8 (`randompermute.cpp`) contains a function named `permute` that randomly permutes the elements of an vector.

Listing 12.8: `randompermute.cpp`

```

#include <iostream>
#include <vector>
#include <ctime>
#include <cstdlib>

using namespace std;

/*
 * print
 * Prints the contents of an int vector
 * a is the vector to print; a is not modified
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << "{";
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << "};";
}

```

```
/*
 * Returns a pseudorandom number in the range begin...end - 1,
 * inclusive. Returns 0 if begin >= end.
 */
int random(int begin, int end) {
    if (begin >= end)
        return 0;
    else {
        int range = end - begin;
        return begin + rand()%range;
    }
}

/*
 * Randomly permute a vector of integers.
 * a is the vector to permute, and n is its length.
 */
void permute(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Select a pseudorandom location from the current
        // location to the end of the collection
        swap(a[i], a[random(i, n)]);
    }
}

// Tests the permute function that randomly permutes the
// contents of a vector
int main() {
    // Initialize random generator seed
    srand(static_cast<int>(time(0)));

    // Make the vector {1,2,3,4,5,6,7,8}
    vector<int> vec { 1, 2, 3, 4, 5, 6, 7, 8 };

    // Print vector before
    print(vec);
    cout << endl;

    permute(vec);

    // Print vector after
    print(vec);
    cout << endl;
}
```

One run of Listing 12.8 (randompermute.cpp) produces

```
1 2 3 4 5 6 7 8
2 7 1 6 4 8 3 5
```

Notice that the `permute` function in Listing 12.8 (`randompermute.cpp`) uses a simple un-nested loop and no recursion. The `permute` function varies the `i` index variable from 0 to the index of the next to last element in the vector. Within the loop, the function obtains via `rand` (see Section 8.6) a pseudorandom index greater than or equal to `i`. It then exchanges the elements at position `i` and the random position. At this point all the elements at index `i` and smaller are fixed and will not change as the function's execution continues. The loop then increments index `i`, and the process continues until all the `i` values have been considered.

To be correct, our `permute` function must be able to generate any valid permutation of the vector. It is important that our `permute` function is able to produce all possible permutations with equal probability; said another way, we do not want our `permute` function to generate some permutations more often than others. The `permute` function in Listing 12.8 (`randompermute.cpp`) is fine, but consider a slight variation of the algorithm:

```
// Randomly permute a vector?
void faulty_permute(vector<int>& a) {
    int n = a.size()
    for (int i = 0; i < n; i++) {
        // Select a pseudorandom position somewhere in the vector
        swap(a[i], a[random(0, n)]);
    }
}
```

Do you see the difference between `faulty_permute` and `permute`? The `faulty_permute` function chooses its the random index from all valid vector indices, whereas `permute` restricts the random index to valid indices greater than or equal to `i`. This means that `faulty_permute` can exchange any element within vector `a` with the element at position `i` during any loop iteration. While this approach superficially may appear to be just as good as `permute`, it in fact produces an uneven distribution of permutations. Listing 12.9 (`comparepermutations.cpp`) exercises each `permutation` function 1,000,000 times on the vector `{1, 2, 3}` and tallies each permutation. There are exactly six possible permutations of this three-element vector.

Listing 12.9: `comparepermutations.cpp`

```
#include <iostream>
#include <vector>
#include <ctime>
#include <cstdlib>

using namespace std;

/*
 * print
 * Prints the contents of an int vector
 * a is the vector to print; a is not modified
 * n is the number of elements in the vector
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << "{";
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ', ' << a[i]; // Print the rest
    }
    cout << "}" << endl;
}
```



```

    }
    cout << "};";
}

/*
 * Returns a pseudorandom number in the range begin...end - 1,
 * inclusive. Returns 0 if begin >= end.
 */
int random(int begin, int end) {
    if (begin >= end)
        return 0;
    else {
        int range = end - begin;
        return begin + rand()%range;
    }
}

/*
 * Randomly permute a vector of integers.
 * a is the vector to permute, and n is its length.
 */
void permute(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Select a pseudorandom location from the current
        // location to the end of the collection
        swap(a[i], a[random(i, n)]);
    }
}

/* Randomly permute a vector? */
void faulty_permute(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n; i++) {
        // Select a pseudorandom position somewhere in the collection
        swap(a[i], a[random(0, n)]);
    }
}

/* Classify a vector as one of the six permutations */
int classify(const vector<int>& a) {
    switch (100*a[0] + 10*a[1] + a[2]) {
        case 123: return 0;
        case 132: return 1;
        case 213: return 2;
        case 231: return 3;
        case 312: return 4;
        case 321: return 5;
    }
    return -1;
}

/* Report the accumulated statistics */
void report(const vector<int>& a) {
    cout << "1,2,3: " << a[0] << endl;
}

```

```

    cout << "1,3,2: " << a[1] << endl;
    cout << "2,1,3: " << a[2] << endl;
    cout << "2,3,1: " << a[3] << endl;
    cout << "3,1,2: " << a[4] << endl;
    cout << "3,2,1: " << a[5] << endl;
}

/*
 * Fill the given vector with zeros.
 * a is the vector, and n is its length.
 */
void clear(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        a[i] = 0;
}

int main() {
    // Initialize random generator seed
    srand(static_cast<int>(time(0)));

    // permutation_tally vector keeps track of each permutation pattern
    // permutation_tally[0] counts {1,2,3}
    // permutation_tally[1] counts {1,3,2}
    // permutation_tally[2] counts {2,1,3}
    // permutation_tally[3] counts {2,3,1}
    // permutation_tally[4] counts {3,1,2}
    // permutation_tally[5] counts {3,2,1}
    vector<int> permutation_tally { 0, 0, 0, 0, 0, 0 };

    // original always holds the vector {1,2,3}
    const vector<int> original { 1, 2, 3 };

    // working holds a copy of original is gets permuted and tallied
    vector<int> working;

    // Run each permutation one million times
    const int RUNS = 1000000;

    cout << "---- Random permute #1 ----" << endl;
    clear(permutation_tally);
    for (int i = 0; i < RUNS; i++) { // Run 1,000,000 times
        // Make a copy of the original vector
        working = original;
        // Permute the vector with the first algorithm
        permute(working);
        // Count this permutation
        permutation_tally[classify(working)]++;
    }
    report(permutation_tally); // Report results

    cout << "---- Random permute #2 ----" << endl;
    clear(permutation_tally);
    for (int i = 0; i < RUNS; i++) { // Run 1,000,000 times
        // Make a copy of the original vector

```

```

    working = original;
    // Permute the vector with the second algorithm
    faulty_permute(working);
    // Count this permutation
    permutation_tally[classify(working)]++;
}
report(permutation_tally); // Report results
}

```

In Listing 12.9 (comparepermutations.cpp)'s output, permute #1 corresponds to our original **permute** function, and permute #2 is the **faulty_permute** function. The output of Listing 12.9 (comparepermutations.cpp) reveals that the faulty permutation function favors some permutations over others:

```

--- Random permute #1 -----
1,2,3: 166608
1,3,2: 166820
2,1,3: 166350
2,3,1: 166702
3,1,2: 166489
3,2,1: 167031
--- Random permute #2 -----
1,2,3: 148317
1,3,2: 184756
2,1,3: 185246
2,3,1: 185225
3,1,2: 148476
3,2,1: 147980

```

In one million runs, the **permute** function provides an even distribution of the six possible permutations of {1, 2, 3}. The **faulty_permute** function generates the permutations {1, 3, 2}, {2, 1, 3}, and {2, 3, 1} more often than the permutations {1, 2, 3}, {3, 1, 2}, and {3, 2, 1}.

To see why **faulty_permute** misbehaves, we need to examine all the permutations it can produce during one call. Figure 12.5 shows a hierarchical structure that maps out how **faulty_permute** transforms the contents of its vector parameter each time through the **for** loop. The top of the tree shows the

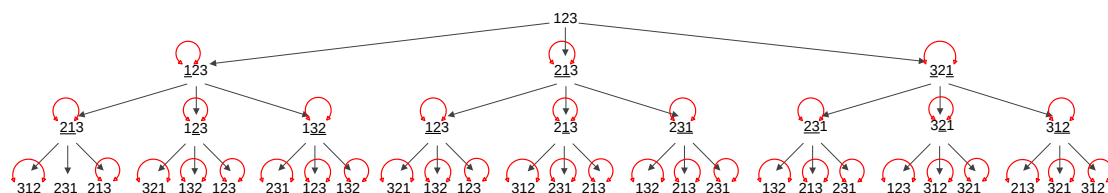


Figure 12.5: A tree mapping out the ways in which **faulty_permute** can transform the vector 1, 2, 3 at each iteration of its **for** loop

original vector, {1, 2, 3}. The second row shows the three possible resulting configurations after the

first iteration of the `for` loop. The leftmost 3-tuple represents the element at index zero swapped with the element at index zero (effectively no change). The second 3-tuple on the second row represents the interchange of the elements at index 0 and index 1. The third 3-tuple on the second row results from the interchange of the elements at positions 0 and 2. The underlined elements represent the elements most recently swapped. If only one item in the 3-tuple is underlined, the function merely swapped the item with itself. The bottom row of 3-tuples contains all the possible outcomes of the `faulty_permute` function given the vector `{1, 2, 3}`.

As Figure 12.5 shows, the vector `{1, 3, 2}`, `{2, 1, 3}`, and `{2, 3, 1}` each appear five times in the last row, while `{1, 2, 3}`, `{3, 1, 2}`, and `{3, 2, 1}` each appear only four times. There are a total of 27 possible outcomes, so some permutations appear $\frac{4}{27} = 14.815\%$ of the time, while the others appear $\frac{5}{27} = 18.519\%$ of the time. Notice that these percentages agree with our experimental results from Listing 12.9 (`comparepermutations.cpp`).

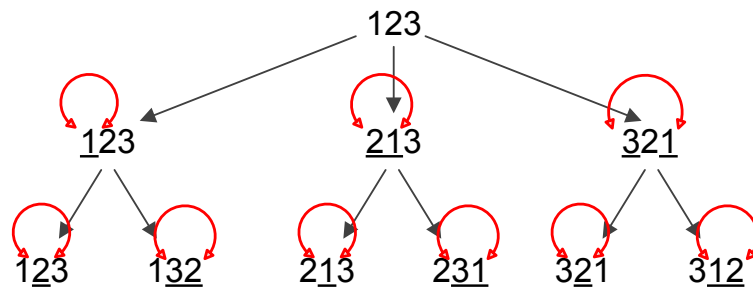


Figure 12.6: A tree mapping out the ways in which `permute` can transform the vector 1, 2, 3 at each iteration of its `for` loop

Compare Figure 12.5 to Figure 12.6. The second row of the tree for `permute` is identical to the second row of the tree for `faulty_permute`, but the third row is different. The second time through its loop the `permute` function does not attempt to exchange the element at index zero with any other elements. We see that none of the first elements in the 3-tuples in row three are underlined. The third row contains exactly one instance of each of the possible permutations of `{1, 2, 3}`. This means that the correct `permute` function is not biased towards any of the individual permutations, and so the function can generate all the permutations with equal probability. The `permute` function has a $\frac{1}{6} = 16.667\%$ probability of generating a particular permutation; this number agrees with our the experimental results of Listing 12.9 (`comparepermutations.cpp`).

12.6 Summary

- Various algorithms exist for sorting vectors. Selection sort is a simple algorithm for sorting a vector.
- Linear search is useful for finding elements in an unordered vector. Binary search can be used on ordered vectors, and due to the nature of the algorithm, binary search is very fast, even on large vectors.

- Permutations
- More permutations

12.7 Exercises

1. Complete the following function that reorders the contents of a vector so they are reversed from their original order. For example, a vector containing the elements 2, 6, 2, 5, 0, 1, 2, 3 would be transformed into 3, 2, 1, 0, 5, 2, 6, 2. Note that your function must physically rearrange the elements within the vector, not just print the elements in reverse order.

```
void reverse(vector<int>& v) {
    // Add your code...
}
```

2. Complete the following function that reorders the contents of a vector so that all the even numbers appear before any odd number. The even values are sorted in ascending order with respect to themselves, and the odd numbers that follow are also sorted in ascending order with respect to themselves. For example, a vector containing the elements 2, 1, 10, 4, 3, 6, 7, 9, 8, 5 would be transformed into 2, 4, 6, 8, 10, 1, 3, 5, 7, 9. Note that your function must physically rearrange the elements within the vector, not just print the elements in reverse order.

```
void special_sort(vector<int>& v) {
    // Add your code...
}
```

3. Complete the following function that shifts all the elements of a vector backward one place. The last element that gets shifted off the back end of the vector is copied into the first (0th) position. For example, if a vector containing the elements 2, 1, 10, 4, 3, 6, 7, 9, 8, 5 is passed to the function, it would be transformed into 5, 2, 1, 10, 4, 3, 6, 7, 9, 8. Note that your function must physically rearrange the elements within the vector, not just print the elements in the shifted order.

```
void rotate(vector<int>& v) {
    // Add your code...
}
```

4. Complete the following function that determines if the number of even and odd values in a vector is the same. The function would return true if the vector contains 5, 1, 0, 2 (two evens and two odds), but it would return false for the vector containing 5, 1, 0, 2, 11 (too many odds). The function should return true if the vector is empty, since an empty vector contains the same number of evens and odds (0). The function does not affect the contents of the vector.

```
bool balanced(const vector<int>& v) {
    // Add your code...
}
```

5. Complete the following function that returns true if vector **a** contains duplicate elements; it returns false if all the elements in **a** are unique. For example, the vector 2, 3, 2, 1, 9 contains duplicates (2 appears more than once), but the vector 2, 1, 0, 3, 8, 4 does not (none of the elements appear more than once).

An empty vector has no duplicates.

The function does not affect the contents of the vector.

```
bool has_duplicates(const vector<int>& v) {  
    // Add your code...  
}
```

6. Can binary search be used on an unsorted vector? Why or why not?
7. Can linear search be used on an unsorted vector? Why or why not?

Chapter 13

Standard C++ Classes

In the hardware arena, a desktop computer is built by assembling

- a motherboard (a circuit board containing sockets for a processor and assorted supporting cards),
- a processor,
- memory,
- a video card,
- an input/output card (USB ports, parallel port, and mouse port),
- a disk controller,
- a disk drive,
- a case,
- a keyboard,
- a mouse, and
- a monitor.

(Some of these components like the I/O, disk controller, and video may be integrated with the motherboard.)

The video card is itself a sophisticated piece of hardware containing a video processor chip, memory, and other electronic components. A technician does not need to assemble the card; the card is used as is off the shelf. The video card provides a substantial amount of functionality in a standard package. One video card can be replaced with another card from a different vendor or with another card with different capabilities. The overall computer will work with either card (subject to availability of drivers for the operating system) because standard interfaces allow the components to work together.

Software development today is increasingly *component based*. Software components are used like hardware components. A software system can be built largely by assembling pre-existing software building blocks. C++ supports various kinds of software building blocks. The simplest of these is the *function* that we investigated in Chapter 8 and Chapter 9. A more powerful technique uses built-in and user designed software *objects*.

C++ is object-oriented. It was not the first OO programming language, but it was the first OO language that gained widespread use in a variety of application areas. An OO programming language allows the programmer to define, create, and manipulate objects. Variables representing objects can have considerable functionality compared to the primitive numeric variables like `ints` and `doubles`. Like a normal variable, every C++ object has a type. We say an object is an instance of a particular *class*, and *class* means the same thing as *type*. An object's type is its class. We have been using the `cout` and `cin` objects for some time. `cout` is an instance of the `ostream` class—which is to say `cout` is of type `ostream`. `cin` is an instance of the `istream` class.

Code that uses an object is a *client* of that object; for example, the following code fragment

```
cout << "Hello" << endl;
```

uses the `cout` object and, therefore, is a client of `cout`. Many of the functions we have seen so far have been clients of the `cout` and/or `cin` objects. Objects provide services to their clients.

13.1 String Objects

A string is a sequence of characters, most often used to represent words and names. The C++ standard library provides the class `string` which specifies *string objects*. In order to use string objects, you must provide the preprocessor directive

```
#include <string>
```

The `string` class is part of the standard namespace, which means its full type name is `std::string`. If you use the

```
using namespace std;
```

or

```
using std::string;
```

statements in your code, you can use the abbreviated name `string`.

You declare a `string` object like any other variable:

```
string name;
```

You may assign a literal character sequence to a `string` object via the familiar string quotation syntax:

```
string name = "joe";
cout << name << endl;
name = "jane";
cout << name << endl;
```

You may assign one `string` object to another using the simple assignment operator:

```
string name1 = "joe", name2;
name2 = name1;
cout << name1 << " " << name2 << endl;
```

In this case, the assignment statement copies the characters making up `name1` into `name2`. After the assignment both `name1` and `name2` have their own copies of the characters that make up the string; they

do not share their contents. After the assignment, changing one string will not affect the other string. Code within the **string** class defines how the assignment operator should work in the context of **string** objects.

Like the **vector** class (Section 11.1.3), the **string** class provides a number of methods. Some **string** methods include:

- **operator[]**—provides access to the value stored at a given index within the vector
- **operator=**—assigns one string to another
- **operator+=**—appends a string or single character to the end of a **string** object
- **at**—provides bounds-checking access to the character stored at a given index
- **length**—returns the number of characters that make up the string
- **size**—returns the number of characters that make up the string (same as **length**)
- **find**—locates the index of a substring within a **string** object
- **substr**—returns a new **string** object made of a substring of an existing **string** object
- **empty**—returns true if the string contains no characters; returns false if the string contains one or more characters
- **clear**—makes the vector empty.

The following code fragment

```
string word = "computer";
cout << "\"" << word << "\" contains " << word.length()
    << " letters." << endl;
```

prints

```
"computer" contains 8 letters.
```

The expression:

```
word.length()
```

invokes the **length** method on behalf of the **word** object. The **string** class provides a method named **size** that behaves exactly like the **length** method.

The **string** class defines a method named **operator[]** that allows a programmer to access a character within a **string**, as in

```
cout << "The letter at index 3 is " << word.operator[] (3) << endl;
```

Here the **operator[]** method uses the same syntax as the **length** method, but the **operator[]** method expects a single integer parameter. The above code fragment is better written as

```
cout << "The letter at index 3 is " << word[3] << endl;
```

The expression

```
word.operator[] (3)
```

is equivalent to the expression

```
word[3]
```

We see that `operator[]` works exactly like its namesake in the `std::vector` class Section 11.1.3.

The following code fragment exercises some of the methods available to string objects:

Listing 13.1: stringoperations.cpp

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    // Declare a string object and initialize it
    string word = "fred";
    // Prints 4, since word contains four characters
    cout << word.length() << endl;
    // Prints "not empty", since word is not empty
    if (word.empty())
        cout << "empty" << endl;
    else
        cout << "not empty" << endl;
    // Makes word empty
    word.clear();
    // Prints "empty", since word now is empty
    if (word.empty())
        cout << "empty" << endl;
    else
        cout << "not empty" << endl;
    // Assign a string using operator= method
    word = "good";
    // Prints "good"
    cout << word << endl;
    // Append another string using operator+= method
    word += "-bye";
    // Prints "good-bye"
    cout << word << endl;
    // Print first character using operator[] method
    cout << word[0] << endl;
    // Print last character
    cout << word[word.length() - 1] << endl;
    // Prints "od-by", the substring starting at index 2 of length 5
    cout << word.substr(2, 5);
    string first = "ABC", last = "XYZ";
    // Splice two strings with + operator
    cout << first + last << endl;
    cout << "Compare " << first << " and ABC: ";
    if (first == "ABC")
        cout << "equal" << endl;
    else
```

```

    cout << "not equal" << endl;
    cout << "Compare " << first << " and XYZ: ";
    if (first == "XYZ")
        cout << "equal" << endl;
    else
        cout << "not equal" << endl;
}

```

The statement

```
word = "good";
```

is equivalent to

```
word.operator=("good");
```

Here we see the explicit use of the dot (.) operator to invoke the method. Similarly,

```
word += "-bye";
```

is the syntactically sweetened way to write

```
word.operator+=("-bye");
```

The **+** operator performs *string concatenation*, making a new string by appending one string to the back of another.

With the **substr** method we can extract a new string from another, as shown in Figure 13.1.

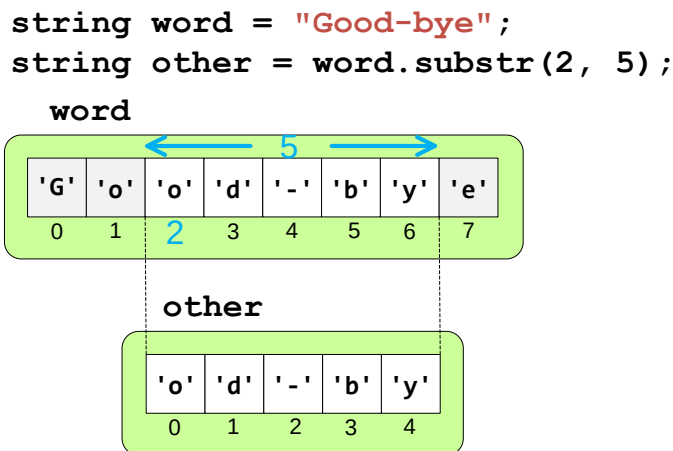


Figure 13.1: Extracting the new string "od-by" from the string "Good-bye"

In addition to string methods, the standard **string** library provides a number of global functions that process strings. These functions use operator syntax and allow us to compare strings via **<**, **==**, **>=**, etc. A more complete list of **string** methods and functions can be found at <http://www.cplusplus.com/reference/string/>.

13.2 Input/Output Streams

We have used `iostream` objects from the very beginning. `cout` is the output stream object that prints to the screen. `cin` is the input stream object that receives values from the keyboard. The precise type of `cout` is `ostream`, and `cin`'s type is `istream`.

Like other objects, `cout` and `cin` have methods. The `<<` and `>>` operators actually are the methods `operator<<` and `operator>>`. (The operators `<<` and `>>` normally are used on integers to perform left and right bitwise shift operations; see Appendix E for more information.) The following code fragment

```
cin >> x;
cout << x;
```

can be written in the explicit method call form as:

```
cin.operator>>(x);
cout.operator<<(x);
```

The first statement calls the `operator>>` method on behalf of the `cin` object passing in variable `x` by reference. The second statement calls the `operator<<` method on behalf of the `cout` object passing the value of variable `x`. A statement such as

```
cout << x << endl;
```

is a more pleasant way of expressing

```
cout.operator<<(x).operator<<(endl);
```

Reading the statement left to right, the expression `cout.operator<<(x)` prints `x`'s value on the screen and returns the `cout` object itself. The return value (simply `cout`) then is used to invoke the `operator<<` method again with `endl` as its argument.

A statement such as

```
cin >> x >> y;
```

can be written

```
cin.operator>>(x).operator>>(y);
```

As is the case of `operator<<` with `cout`, reading left to right, the expression `cin.operator>>(x)` calls the `operator>>` method passing variable `x` by reference. It reads a value from the keyboard and assigns it to `x`. The method call returns `cin` itself, and the return value is used immediately to invoke `operator>>` passing variable `y` by reference.

You probably have noticed that it is easy cause a program to fail by providing input that the program was not expecting. For example, compile and run Listing 13.2 (`naiveinput.cpp`).

Listing 13.2: `naiveinput.cpp`

```
#include <iostream>

using namespace std;

int main() {
    int x;
```

```

// I hope the user does the right thing!
cout << "Please enter an integer: ";
cin >> x;
cout << "You entered " << x << endl;
}

```

Listing 13.2 (naiveinput.cpp) works fine as long as the user enters an integer value. What if the user enters the word “five,” which arguably is an integer? The program produces incorrect results. We can use some additional methods available to the `cin` object to build a more robust program. Listing 13.3 (betterinput.cpp) detects illegal input and continues to receive input until the user provides an acceptable value.

Listing 13.3: betterinput.cpp

```

#include <iostream>
#include <limits>

using namespace std;

int main() {
    int x;
    // I hope the user does the right thing!
    cout << "Please enter an integer: ";
    // Enter and remain in the loop as long as the user provides
    // bad input
    while (!(cin >> x)) {
        // Report error and re-prompt
        cout << "Bad entry, please try again: ";
        // Clean up the input stream
        cin.clear(); // Clear the error state of the stream
        // Empty the keyboard buffer
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << "You entered " << x << endl;
}

```

We learned in Section 6.1 that the expression

```
cin >> x
```

has a Boolean value that we may use within a conditional or iterative statement. If the user enters a value with a type compatible with the declared type of the variable, the expression evaluates to true; otherwise, it is interpreted as false. The negation

```
!(cin >> x)
```

is true if the input is bad, so the only way to execute the body of the loop is provide illegal input. As long as the user provides bad input, the program’s execution stays inside the loop.

While determining whether or not a user’s entry is correct seems sufficient for the programmer to make corrective measures, it is not. Two additional steps are necessary:

- The bad input characters the user provided cause the `cin` object to enter an error state. The input stream object remains in an error state until the programmer manually resets it. The call

```
cin.clear();
```

resets the stream object so it can process more input.

- Whatever characters the user typed in that cannot be assigned to the given variable remain in the keyboard input buffer. Clearing the stream object does not remove the leftover keystrokes. Asking the user to retry without clearing the bad characters entered from before results in the same problem—the stream object re-enters the error state and the bad characters remain in the keyboard buffer. The solution is to flush from the keyboard buffer all of the characters that the user entered since the last valid data entry. The statement

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

removes from the buffer all the characters, up to and including the newline character (`'\n'`). The function call

```
numeric_limits<streamsize>::max()
```

returns the maximum number of characters that the buffer can hold, so the **ignore** method reads and discards characters until it reads the newline character (`'\n'`) or reaches the end of the buffer, whichever comes first.

Once the stream object has been reset from its error state and the keyboard buffer is empty, user input can proceed as usual.

The **ostream** and **istream** classes have a number of other methods, but we will not consider them here.

istream objects use whitespace (spaces and tabs) as delimiters when they get input from the user. This means you cannot use the **operator>>** to assign a complete line of text from the keyboard if that line contains embedded spaces. Listing 13.4 (`faultyreadline.cpp`) illustrates.

Listing 13.4: `faultyreadline.cpp`

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    string line;
    cout << "Please enter a line of text: ";
    cin >> line;
    cout << "You entered: \"" << line << "\"" << endl;
}
```

A sample run of Listing 13.4 (`faultyreadline.cpp`) reveals:

```
Please enter a line of text: Mary had a little lamb.
You entered: "Mary"
```

As you can see, Listing 13.4 (`faultyreadline.cpp`) does not assign the complete line of text to the sting variable **line**. The text is truncated at the first space in the input.

To read in a complete line of text from the keyboard, including any embedded spaces that may be present, use the global `getline` function. As Listing 13.5 (`readline.cpp`) shows, the `getline` function accepts an `istream` object and a `string` object to assign.

Listing 13.5: `readline.cpp`

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    string line;
    cout << "Please enter a line of text: ";
    getline(cin, line);
    cout << "You entered: \"" << line << "\"" << endl;
}
```

A sample run of Listing 13.5 (`readline.cpp`) produces:

```
Please enter a line of text: Mary has a little lamb.
You entered: "Mary has a little lamb."
```

13.3 File Streams

Many applications allow users to create and manipulate data. Truly useful applications allow users to store their data to files; for example, word processors can save and load documents.

Vectors would be more useful if they were *persistent*. Data is persistent when it exists between program executions. During one execution of a particular program the user may create and populate a vector. The user then saves the contents of the vector to disk and quits the program. Later, the user can run the program again and reload the vector from the disk and resume work.

C++ `fstream` objects allow programmers to build persistence into their applications. Listing 13.6 (`numberlist.cpp`) is a simple example of a program that allows the user to save the contents of a vector to a text file and load a vector from a text file.

Listing 13.6: `numberlist.cpp`

```
// File file_io.cpp

#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;

/*
 * print_vector(v)
 * Prints the contents of vector v.
 */
```



```

*   v is a vector holding integers.
*/
void print_vector(const vector<int>& vec) {
    cout << "{";
    int len = vec.size();
    if (len > 0) {
        for (int i = 0; i < len - 1; i++)
            cout << vec[i] << ","; // Comma after elements
        cout << vec[len - 1]; // No comma after last element
    }
    cout << "}" << endl;
}

/*
*   save_vector(filename, v)
*   Writes the contents of vector v.
*   filename is name of text file created. Any file
*   by that name is overwritten.
*   v is a vector holding integers. v is unchanged by the
*   function.
*/
void save_vector(const string& filename, const vector<int>& vec) {
    ofstream out;
    out.open(filename);
    if (out.good()) { // Make sure the file was opened properly
        int n = vec.size();
        for (int i = 0; i < n; i++)
            out << vec[i] << " "; // Space delimited
        out << endl;
        out.close();
    }
    else
        cout << "Unable to save the file" << endl;
}

/*
*   load_vector(filename, v)
*   Reads the contents of vector v from text file
*   filename. v's contents are replaced by the file's
*   contents. If the file cannot be found, the vector v
*   is empty.
*   v is a vector holding integers.
*/
void load_vector(const string& filename, vector<int>& vec) {
    ifstream in;
    in.open(filename);
    if (in.good()) { // Make sure the file was opened properly
        vec.clear(); // Start with empty vector
        int value;
        while (in >> value) // Read until end of file
            vec.push_back(value);
        in.close();
    }
    else
        cout << "Unable to load in the file" << endl;
}

```

```

}

int main() {
    vector<int> list;
    bool done = false;
    char command;
    while (!done) {
        cout << "I)nsert <item> P)rint "
              << "S)ave <filename> L)oad <filename> "
              << "E)rase Q)uit: ";
        cin >> command;
        int value;
        string filename;
        switch (command) {
            case 'I':
            case 'i':
                cin >> value;
                list.push_back(value);
                break;
            case 'P':
            case 'p':
                print_vector(list);
                break;
            case 'S':
            case 's':
                cin >> filename;
                save_vector(filename, list);
                break;
            case 'L':
            case 'l':
                cin >> filename;
                load_vector(filename, list);
                break;
            case 'E':
            case 'e':
                list.clear();
                break;
            case 'Q':
            case 'q':
                done = true;
                break;
        }
    }
}

```

Listing 13.6 (numberlist.cpp) is command driven with a menu, and when the user types

```
I)nsert <item> P)rint S)ave <filename> L)oad <filename> E)rase Q)uit: S data1.text
```

the program saves the current contents of the vector to a file named data1.text. The user can erase the contents of the vector:

```
I)nsert <item> P)rint S)ave <filename> L)oad <filename> E)rase Q)uit: E
```

and then restore the contents with a load command:

```
I)nsert <item> P)rint S)ave <filename> L)oad <filename> E)rase Q)uit: L dat
```

The user also may quit the program, and later re-run the program and load in the previously saved list of numbers. The user can save different number lists to different files using different file names.

An **ofstream** object writes data to files. We can use the **open** method to associate the object with the file to be created. The **open** method accepts a string that represents a file name. The **save_vector** function in Listing 13.6 (numberlist.cpp) passes a **string** object to **open**, but the **open** method can work with string literals (quoted strings) as well. Consider the following code fragment:

```
ofstream fout;
int x = 10;
fout.open("myfile.dat");
if (fout.good()) { // Make sure the file was opened properly
    fout << "x = " << x << endl;
    fout.close();
}
else
    cout << "Unable to write to the file \"myfile.dat\"\" << endl;
```

After the call to **open**, programmers should verify that the method correctly opened the file by calling the file stream object's **good** method. An output file stream may fail for various reasons, including the disk being full or insufficient permissions to create a file in a given folder.

Once we have its associated file open, we can use an **ofstream** object like the **cout** output stream object, except the data is recorded in a text file instead of being printed on the screen. Just like with **cout**, you can use the **<<** operator and send an **ofstream** object stream manipulators like **endl** and **setw**.

After the executing program has written all the data to the file and needs to write no more it must use the **close** method to properly close the file. Failure to do so may mean that some or all of the data written to the file is not completely saved to the disk. The **close** method accepts no parameters.

An **ifstream** object reads data from files. We can use the **open** method to associate the **ifstream** object with a particular file to be read. As with **ofstream** objects, the **open** method of the **ifstream** class accepts a string file name to identify the file to read.

After calling **open**, the program should call **good** to ensure the file was successfully opened. An input stream object often fails to open properly because the file does not exist; perhaps the file name is misspelled, or the path to the file is incorrect. An input stream can also fail because of insufficient permissions or because of bad sectors on the disk.

Once its associated file is opened, an input file stream object behaves like the **cout** object, except its data comes from a text file instead the keyboard. This means the familiar **>>** operator and **getline** function are completely compatible with **ifstream** objects.

As with an output stream object, your program should close the file associated with an input stream object via the object's **close** method when the program has finished reading the file.

13.4 Complex Numbers

C++ supports mathematical complex numbers via the `complex` class. Recall from mathematics that a complex number has a real component and an imaginary component. Often written as $a + bi$, a is the real part, an ordinary real number, and bi is the imaginary part where b is a real number and $i^2 = -1$.

The `complex` class in C++ is a template class like `vector`. In the angle brackets you specify the precision of the complex number's components:

```
complex<float> fc;
complex<double> dc;
complex<long double> ldc;
```

Here, the real component and imaginary coefficient of `fc` are single-precision floating-point values. `dc` and `ldc` have the indicated precisions. Listing 13.7 (`complex.cpp`) is small example that computes the product of complex conjugates (which should be real numbers).

Listing 13.7: `complex.cpp`

```
// File complex.cpp
#include <iostream>
#include <complex>

using namespace std;

int main() {
    // c1 = 2 + 3i, c2 = 2 - 3i; c1 and c2 are complex conjugates
    complex<double> c1(2.0, 3.0), c2(2.0, -3.0);

    // Compute product "by hand"
    double real1 = c1.real(),
           imag1 = c1.imag(),
           real2 = c2.real(),
           imag2 = c2.imag();
    cout << c1 << " * " << c2 << " = "
         << real1*real2 + imag1*real2 + real1*imag2 - imag1*imag2 << endl;

    // Use complex arithmetic
    cout << c1 << " * " << c2 << " = " << c1*c2 << endl;
}
```

Listing 13.7 (`complex.cpp`) prints

```
(2,3) * (2,-3) = 13
(2,3) * (2,-3) = (13,0)
```

Observe that the program displays the complex number $2 - 3i$ as the ordered pair `(2, -3)`. The first element of the pair is the real part, and the second element is the imaginary coefficient. If the imaginary part is zero, the number is a real number (or, in this case, a `double`).

Imaginary numbers have scientific and engineering applications that exceed the scope of this book, so this concludes our brief into C++'s `complex` class. If you need to solve problems that involve complex numbers, more information can be found at <http://www.cplusplus.com/reference/std/complex/>.

13.5 Better Pseudorandom Number Generation

Listing 12.9 (comparepermutations.cpp) showed that we must use care when randomly permuting the contents of a vector. A naïve approach can introduce accidental bias into the result. It turns out that our simple technique for generating pseudorandom numbers using **rand** and modulus has some issues itself.

Suppose we wish to generate pseudorandom numbers in the range $0 \dots 9,999$. This range spans 10,000 numbers. Under Visual C++ **RAND_MAX** is 32,767, which is large enough to handle a maximum value of 9,999. The expression **rand() % 10000** will evaluate to number in our desired range. A good pseudorandom number generator should be just as likely to produce one number as another. In a program that generates one billion pseudorandom values in the range $0 \dots 9,999$, we would expect any given number to appear approximately $\frac{1,000,000,000}{10,000} = 100,000$ times. The actual value for a given number will vary slightly from one run to the next, but the average over one billion runs should be very close to 100,000.

Listing 13.8 (badrand.cpp) evaluates the quality of the **rand** with modulus technique by generating one billion pseudorandom numbers within a loop. It counts the number of times the pseudorandom number generator produces 5 and also it counts the number of times 9,995 appears. Note that 5 is near the beginning of the range $0 \dots 9,999$, and 9,995 is near the end of that range. To verify the consistency of its results, it repeats this test 10 times. The program reports the results of each individual trial, and in the end it computes the average of the 10 trials.

Listing 13.8: badrand.cpp

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    // Initialize a random seed value
    srand(static_cast<unsigned>(time(nullptr)));

    // Verify the largest number that rand can produce
    cout << "RAND_MAX = " << RAND_MAX << endl;

    // Total counts over all the runs.
    // Make these double-precision floating-point numbers
    // so the average computation at the end will use floating-point
    // arithmetic.
    double total5 = 0.0, total9995 = 0.0;

    // Accumulate the results of 10 trials, with each trial
    // generating 1,000,000,000 pseudorandom numbers
    const int NUMBER_OF_TRIALS = 10;

    for (int trial = 1; trial <= NUMBER_OF_TRIALS; trial++) {
        // Initialize counts for this run of a billion trials
        int count5 = 0, count9995 = 0;
        // Generate one billion pseudorandom numbers in the range
        // 0...9,999 and count the number of times 5 and 9,995 appear
        for (int i = 0; i < 1000000000; i++) {
```

```

// Generate a pseudorandom number in the range 0...9,999
int r = rand() % 10000;
if (r == 5)
    count5++; // Number 5 generated, so count it
else if (r == 9995)
    count9995++; // Number 9,995 generated, so count it
}
// Display the number of times the program generated 5 and 9,995
cout << "Trial #" << setw(2) << trial << " 5: " << count5
    << "    9995: " << count9995 << endl;
total5 += count5; // Accumulate the counts to
total9995 += count9995; // average them at the end
}
cout << "-----" << endl;
cout << "Averages for " << NUMBER_OF_TRIALS << " trials: 5: "
    << total5 / NUMBER_OF_TRIALS << "    9995: "
    << total9995 / NUMBER_OF_TRIALS << endl;
}

```

The output of Listing 13.8 (badrand.cpp) shows that our pseudorandom number generator favors 5 over 9,995:

```

RAND_MAX = 32767
Trial # 1 5: 122295    9995: 91255
Trial # 2 5: 121789    9995: 91862
Trial # 3 5: 122440    9995: 91228
Trial # 4 5: 121602    9995: 91877
Trial # 5 5: 122599    9995: 91378
Trial # 6 5: 121599    9995: 91830
Trial # 7 5: 122366    9995: 91598
Trial # 8 5: 121839    9995: 91387
Trial # 9 5: 122295    9995: 91608
Trial #10 5: 121898    9995: 91519
-----
Averages for 10 trials: 5: 122072    9995: 91554.2

```

The first line verifies that the largest pseudorandom number that Visual C++ can produce through **rand** is 32,767. The next 10 lines that the program show the result of each trial, monitoring the activity of the one billion number generations. Since we are dealing with pseudorandom numbers, the results for each trial will not be exactly the same, but over one billion runs each they should be close. Note how consistent the results are among the runs.

While we expected both 5 and 9,995 to appear about the same number of times—each approximately 100,000 times—in fact the number 5 appeared consistently more than 100,000 times, averaging 122,072 times. The number 9,995 appeared consistently less than 100,000 times, averaging 91,554.2. Note that $\frac{122,072}{91,554.2} = 1.33$; this means the value 5 appeared 1.33 times more often than 9,995. Looking at it another way, $1.33 \approx \frac{4}{3}$, so for every four times the program produced a 5 it produced 9,995 only three times. As we soon will see, this ratio of 4:3 is not accidental.

Figure 13.2 shows why the expression `rand() % 10000` does not produce an even distribution. Figure 13.2 shows an abbreviated list of all the numbers the `rand` function can produce before applying

Elements in each row are equivalent modulus 10,000

10,000 rows	2,768 rows	0	10,000	20,000	30,000	Four ways to obtain any value in the range 0 ... 2,767
		1	10,001	20,001	30,001	
		2	10,002	20,002	30,002	
		3	10,003	20,003	30,003	
	7,232 rows	.	.	.	.	Only three ways to obtain any value in the range 2,768 ... 9,999
		.	.	.	.	
		.	.	.	.	
		2,766	12,766	22,766	32,766	
		2,767	12,767	22,767	32,767	
		2,768	12,768	22,768		
		.	.	.		
		.	.	.		
		.	.	.		
		.	.	.		
		.	.	.		
		.	.	.		
		9,997	19,997	29,997		
		9,998	19,998	29,998		
		9,999	19,999	29,999		

Figure 13.2: If shown in full, the table would contain 10,000 rows and 32,768 individual numbers. The values in each row are equivalent modulus 10,000. All the columns except the rightmost column contain 10,000 entries.

the modulus operation. If you add the missing rows that the ellipses represent, the table would contain 10,000 rows. All of the four values in each row are equivalent modulus 10,000; thus, for example,

$$2 = 10,002 = 20,002 = 30,002$$

and

$$1,045 = 11,045 = 21,045 = 31,045$$

Since the `rand` function cannot produce any values in the range 32,678...39,999, the rightmost column is not complete. Because the leftmost three columns are complete, the modulus operator can produce values in the range 0...2,767 four different ways; for example, the following code fragment

```
cout << 5 % 10000 << ' ' << 10005 % 10000 << ' '
    << 20005 % 10000 << ' ' << 30005 % 10000 << endl;
```

prints

```
5 5 5 5
```

The **rand** function cannot return a value greater than 32,767; specifically, in our program above, **rand** cannot produce 39,995. Listing 13.8 (badrand.cpp), therefore, using **rand** and modulus we can produce 9,995 in only three different ways: 9,995, 19,995, and 29,995. Based on our analysis, Listing 13.8 (badrand.cpp) can generate the number 5 four different ways and 9,995 three different ways. This 4:3 ratio agrees with our empirical observations of the behavior of Listing 13.8 (badrand.cpp). The consequences of this bias means that values in the relatively small range 0...2,767 will appear disproportionately more frequently than numbers in the larger range 2,768...9,999. Such bias definitely is undesirable in a pseudorandom number generator.

We must use more sophisticated means to produce better pseudorandom numbers. Fortunately C++11 provides several standard classes that provide high-quality pseudorandom generators worthy of advanced scientific and mathematical applications.

The **rand** function itself has another weakness that makes it undesirable for serious scientific, engineering, and mathematical applications. **rand** uses a linear congruential generator algorithm (see http://en.wikipedia.org/wiki/Linear_congruential_generator). **rand** has a relatively small period. This means that the pattern of the sequence of numbers it generates will repeat itself exactly if you call **rand** enough times. For Visual C++, **rand**'s period is 2,147,483,648. Listing 13.9 (randperiod.cpp) verifies the period of **rand**.

Listing 13.9: randperiod.cpp

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    // Set random number seed value
    srand(42);

    // Need to use numbers larger than regular integers; use long long ints
    for (long long i = 1; i < 4294967400LL; i++) {
        int r = rand();
        if (1 <= i && i <= 10)
            cout << setw(10) << i << ":" << setw(6) << r << endl;
        else if (2147483645 <= i && i <= 2147483658)
            cout << setw(10) << i << ":" << setw(6) << r << endl;
        else if (4294967293LL <= i && i <= 4294967309LL)
            cout << setw(10) << i << ":" << setw(6) << r << endl;
    }
}
```

Listing 13.9 (randperiod.cpp) uses the C++ standard **long long int** integer data type because it needs to count above the limit of the **int** type, 2,147,483,647. The short name for **long long int** is just **long long**. Visual C++ uses four bytes to store both **int** and **long** types, so their range of values are identical. Under Visual C++, the type **long long** occupies eight bytes which allows the **long long** data type to span the range 9,223,372,036,854,775,808...9,223,372,036,854,775,807. To represent a literal **long long** within C++ source code, we append the **LL** suffix, as in **12LL**. The expression **12** represents the **int** (4-byte version) of 12, but **12LL** represents the **long long** (8-byte version) of 12.

Listing 13.9 (randperiod.cpp) prints the first 10 pseudorandom numbers it generates, then it prints

numbers 2,147,483,645 through 2,147,483,658. Finally the program prints its 4,294,96,729th through 4,294,967,309th pseudorandom numbers. Listing 13.9 (randperiod.cpp) displays

```
1: 175
2: 400
3: 17869
4: 30056
5: 16083
6: 12879
7: 8016
8: 7644
9: 15809
10: 1769
2147483645: 25484
2147483646: 21305
2147483647: 6359
2147483648: 0
2147483649: 175
2147483650: 400
2147483651: 17869
2147483652: 30056
2147483653: 16083
2147483654: 12879
2147483655: 8016
2147483656: 7644
2147483657: 15809
2147483658: 1769
4294967293: 25484
4294967294: 21305
4294967295: 6359
4294967296: 0
4294967297: 175
4294967298: 400
4294967299: 17869
4294967300: 30056
4294967301: 16083
4294967302: 12879
4294967303: 8016
4294967304: 7644
4294967305: 15809
4294967306: 1769
4294967307: 32409
4294967308: 29950
4294967309: 13471
```

Notice that after 2,147,483,648 iterations the program begins to print the same numbers in the same sequential order as it began. 2,147,483,648 iterations later (after 4,294,967,296 total iterations) the sequence once again repeats. A careful observer could detect this repetition and thus after some time be able to pre-

dict the next pseudorandom value that the program would produce. A predictable pseudorandom number generator is not a good random number generator. Such a generator used in a game of chance would render the game perfectly predictable by clever players. A better pseudorandom number generator would have a much longer period.

The Mersenne twister (see http://en.wikipedia.org/wiki/Mersenne_twister) is a widely-used, high-quality pseudorandom number generator. It has a very long period, $2^{19,937} - 1$, which is approximately $4.3154 \times 10^{6,001}$. If an implementation of the Mersenne twister could generate 1,000,000,000 (one billion) pseudorandom numbers every second, a program that generated such pseudorandom numbers exclusively and did nothing else would need to run about $1.3684 \times 10^{5,985}$ years before it begin to repeat itself. It is safe to assume that an observer will not be able to wait around long enough to be able to witness a repeated pattern in the sequence.

The standard C++ library contains the **mt19937** class from which programmers can instantiate objects used to generate pseudorandom numbers using the Mersenne twister algorithm. Generating the pseudorandom numbers is one thing, but ensuring that the numbers fall uniformly distributed within a specified range of values is another concern. Fortunately, the standard C++ library provides a multitude of classes that allow us to shape the production of an **mt19937** object into a mathematically sound distribution.

Our better pseudorandom generator consists of three pieces:

- an object that produces a random seed value,
- a pseudorandom number generator object that we construct with the random seed object, and
- a distribution object that uses the pseudorandom number generator object to produce a sequence of pseudorandom numbers that are uniformly distributed.

The C++ classes for these objects are

- The seed object is an instance of the **random_device** class.
- The pseudorandom number generator object is an instance of the **mt19937** class.
- The distribution object is an instance of the **uniform_int_distribution** class.

We use the **random_device** object in place of **srand**. The **mt19937** object performs the role of the **rand** function, albeit with much better characteristics. The **uniform_int_distribution** object constrains the pseudorandom values to a particular range, replacing the simple but problematic modulus operator. Listing 13.10 (`highqualityrandom.cpp`) upgrades Listing 13.8 (`badrand.cpp`) with improved random number generation is based on these classes.

Listing 13.10: `highqualityrandom.cpp`

```
#include <iostream>
#include <iomanip>
#include <random>

using namespace std;

int main() {
    random_device rdev;    // Used to establish a seed value
    // Create a Mersenne Twister random number generator with a seed
    // value derived from rd
    mt19937 mt(rdev());
```

```

// Create a uniform distribution object. Given a random
// number generator, dist will produce a value in the range
// 0...9999.
uniform_int_distribution<int> dist(0, 9999);

// Total counts over all the runs.
// Make these double-precision floating-point numbers
// so the average computation at the end will use floating-point
// arithmetic.
double total5 = 0.0, total9995 = 0.0;

// Accumulate the results of 10 trials, with each trial
// generating 1,000,000,000 pseudorandom numbers
const int NUMBER_OF_TRIALS = 10;

for (int trial = 1; trial <= NUMBER_OF_TRIALS; trial++) {
    // Initialize counts for this run of a billion trials
    int count5 = 0, count9995 = 0;
    // Generate one billion pseudorandom numbers in the range
    // 0...9,999 and count the number of times 5 and 9,995 appear
    for (int i = 0; i < 1000000000; i++) {
        // Generate a pseudorandom number in the range 0...9,999
        int r = dist(mt);
        if (r == 5)
            count5++; // Number 5 generated, so count it
        else if (r == 9995)
            count9995++; // Number 9,995 generated, so count it
    }
    // Display the number of times the program generated 5 and 9,995
    cout << "Trial #" << setw(2) << trial << " 5: " << setw(6) << count5
        << " 9995: " << setw(6) << count9995 << endl;
    total5 += count5; // Accumulate the counts to
    total9995 += count9995; // average them at the end
}
cout << "-----" << endl;
cout << "Averages for " << NUMBER_OF_TRIALS << " trials: 5: "
    << setw(6) << total5 / NUMBER_OF_TRIALS << " 9995: "
    << setw(6) << total9995 / NUMBER_OF_TRIALS << endl;
}

```

One run of Listing 13.10 (highqualityrandom.cpp) reports

```

Trial # 1 5: 99786    9995: 100031
Trial # 2 5: 99813    9995: 99721
Trial # 3 5: 99595    9995: 100144
Trial # 4 5: 100318   9995: 100243
Trial # 5 5: 99570    9995: 100169
Trial # 6 5: 99860    9995: 99724
Trial # 7 5: 99821    9995: 100263
Trial # 8 5: 99851    9995: 99887
Trial # 9 5: 100083   9995: 100204
Trial #10 5: 100202   9995: 99943
-----
Averages for 10 trials: 5: 99889.9 9995: 100033

```

During this particular program run we see that in 1,000,000,000 attempts the program generates the value 5 on average 99,889.9 times and generates 9,995 on average 100,033 times. Both of these counts approximately equal the expected 100,000 target. Examining the 10 trials individually, we see that neither the count for 5 nor the count for 9,995 is predisposed to be greater than or less than the other. In some trials the program generates 5 slightly less than 100,000 times, and in others it appears slightly greater than 100,000 times. The same is true for 9,995. These multiple trials show that in over 1,000,000,000 iterations the program consistently generates 5 approximately 100,000 times and 9,995 approximately 100,000 times. Listing 13.10 (`highqualityrandom.cpp`) shows us that the pseudorandom numbers generated by the Mersenne Twister object in conjunction with the distribution object are much better uniformly distributed than those produced by `rand` with the modulus operation.

Notice in Listing 13.10 (`highqualityrandom.cpp`) how the three objects work together:

- The `uniform_int_distribution` object produces a pseudorandom number from the `mt19937` generator object. The `mt19937` object generates a pseudorandom number, and the `uniform_int_distribution` object constrains this pseudorandom number to the desired range.
- Programmers create an `mt19937` object with a `random_device` object. The `random_device` object provides the seed value, potentially from a hardware source, to the `mt19937` generator object. We also can pass a fixed integer value to `mt19937`'s constructor if we want the generator to produce a perfectly reproducible sequence of values; for example, the following code fragment

```

mt19937 gen(20); // Use fixed seed instead of random_device
uniform_int_distribution<int> dist(0, 9999);
cout << dist(gen) << endl;
cout << dist(gen) << endl;
cout << dist(gen) << endl;

```

always prints

```

7542
6067
6876

```

The use of `random_device`, `mt19937`, and `uniform_int_distribution` is a little more complicated than using `srand` and `rand` with the modulus operator, but the extra effort is worth it for

many applications. This object-oriented approach is more modular because it allows us to substitute an object of a different pseudorandom number generator class in place of `mt19937` if we so choose. We also may swap out the normal distribution for a different distribution. Those familiar with probability theory may be familiar with a variety of different probability distributions, such as Bernoulli, Poisson, binomial, chi-squared, etc. The C++ standard library contains distribution classes that model all of these probability distributions and many more. Programmers can mix and match generator objects and distribution objects as needed to achieve specialized effects. While this flexibility is very useful and has its place, the `random_device`, `mt19937`, and `uniform_int_distribution` classes as used in Listing 13.10 (`highqualityrandom.cpp`) suffice for most applications needing to simulate random processes.

13.6 Summary

CAUTION! SECTION UNDER CONSTRUCTION

Add content

13.7 Exercises

CAUTION! SECTION UNDER CONSTRUCTION

Add exercises.

Chapter 14

Custom Objects

In earlier times programmers wrote software in the machine language of the computer system because compilers have yet to be invented. The introduction of variables in association with higher-level programming languages marked a great step forward in the late 1950s. No longer did programmers need to be concerned with the lower-level details of the processor and absolute memory addresses. Named variables and functions allow programmers to abstract away such machine-level details and concentrate on concepts that transcend computers, such as integers and characters. Objects provide a level of abstraction above that of variables. Objects allow programmers to go beyond simple values—developers can focus on more complex things like geometric shapes, bank accounts, and aircraft wings. Programming objects that represent these real-world things can possess capabilities that go far beyond the simple variables we have studied to this point.

A C++ object typically consists of a collection of data and code. By bundling data and code together, objects store information and provide services to other parts of the software system. An object forms a computational unit that makes up part of the overall computer application. A programming object can model a real-world object more naturally than can a collection of simple variables since it can encapsulate considerable complexity. Objects make it easier for developers to build complex software systems.

C++ is classified as an object-oriented language. Most modern programming languages have some degree of object orientation. This chapter shows how programmers can define, create, and use custom objects.

14.1 Object Basics

Consider the task of dealing with geometric points. Mathematicians represent a single point as an ordered pair of real numbers, usually expressed as (x, y) . In C++, the `double` type serves to approximate a subset of the mathematical real numbers. A point with coordinates within the range of double-precision floating-point numbers, therefore, can be modeled by two `double` variables. Notice something that conceptually is one thing requires two variables in C++. As a consequence, a function that computes the distance between two points requires four parameters— x_1 , y_1 , x_2 , and y_2 —rather than two points— (x_1, y_1) and (x_2, y_2) . Ideally, we should be able to use one variable to represent a point.

One approach to represent a point could use a two-element vector, for example:

```
vector<double> pt { 3.2, 0.0 };
```

This approach has several problems:

- We must use numeric indices instead of names to distinguish between the two components of a point object. We may agree that `pt[0]` means the x coordinate of point `pt` and `pt[1]` means the y coordinate of point `pt`, but the compiler is powerless to detect the error if a programmer uses an expression like `pt[19]` or `pt[-3]`.
- We cannot restrict the vector's size to two. A programmer may accidentally push extra items onto the back of a vector representing a point object.
- We cannot use a vector to represent objects in general. Consider a bank account object. A bank account object could include, among many other diverse things, an account number (an integer), a customer name (a string), and an interest rate (a double-precision floating-point number). A vector implementation of such an object is impossible because the elements in a vector must all be of the same type.

In addition to storing data, we want our objects to be active agents that can do computational tasks. We need to be able to associate code with a class of objects. We need a fundamentally different programming construct to represent objects.

Before examining how C++ specifically handles objects, we first will explore what capabilities are desirable. Consider an automobile. An automobile user, the driver, uses the car for transportation. The user's interface to the car is fairly simple, considering an automobile's overall complexity. A driver provides input to the car via its steering wheel, accelerator and brake pedals, turn signal control, shift lever, etc. The automobile produces output to the driver with its speedometer, tachometer, various instrument lights and gauges, etc. These standardized driver-automobile interfaces enable an experienced driver to drive any modern car without the need for any special training for a particular make or model.

The typical driver can use a car very effectively without understanding the details of how it works. To drive from point *A* to point *B* a driver does not need to know the number of cylinders in the engine, the engine's horsepower, or whether the vehicle is front-wheel drive or rear-wheel drive. A driver may look under the hood at the engine, but the driver cannot confirm any details about what is *inside* the engine itself without considerable effort or expense. Many details are of interest only to auto enthusiasts or mechanics. There may be only a select few automotive engineers capable of understanding and appreciating other more esoteric details about the vehicle's design and implementation.

In some ways programming objects as used in object-oriented programming languages are analogous to automobile components. An object may possess considerable capability, but a programmer using the object needs to know only *what* the object can do without needing to know *how* it works. An object provides an interface to any client code that wishes to use that object. A typical object selectively exposes some parts of itself to clients and keeps other parts hidden from clients. The object's designer, on the other hand, must know the complete details of the object's implementation and must be an expert on both the *what* the object does and *how* it works.

C++ uses the `class` keyword to define the structure of an object. A class serves as a pattern or template from which a programmer may produce objects. We will concentrate on five things facilitating object-oriented programming with C++ classes:

1. A class can specify the data that constitute an object's state.
2. A class can define code to be executed on an object's behalf that provides services to clients that use the object.
3. A class may define code that automatically initializes a newly-created object ensuring that it begins its life in a well-defined state.

4. A class may define code that automatically cleans up any resources an object may be using when its life is finished.
5. The C++ language provides a way for the developer of a class to specify which parts of its objects are visible to clients and which parts are hidden from clients.

A class is a programmer-defined type. An object is an *instance* of a class. The terms *object* and *instance* may be used interchangeably.

14.2 Fields

The simplest kind of object stores only data. We can define a point type as follows:

```
class Point {  
public:  
    double x;  
    double y;  
};
```

Notice the semicolon that follows the close curly brace of the class definition. This semicolon is required, but it is easy to overlook. By convention class names begin with a capital letter, but class names are just identifiers like variable names and function names. Here, our class name is **Point**. The body of the class appears within the curly braces.

The **Point** class specifies two double-precision floating-point data components named **x** and **y**. These are called *fields* or *data members*. They both appear within the class body after the **public** label. We say that **x** and **y** are public fields of the **Point** class; this means client code has full access to the **x** and **y** fields. Any client may examine and modify the **x** and **y** components of a **Point** object.

Once this **Point** class definition is available, a client can create and use **Point** objects as shown in Listing 14.1 (mathpoints.cpp).

Listing 14.1: mathpoints.cpp

```
#include <iostream>  
  
using namespace std;  
  
// The Point class defines the structure of software  
// objects that model mathematical, geometric points  
class Point {  
public:  
    double x;    // The point's x coordinate  
    double y;    // The point's y coordinate  
};  
  
int main() {  
    // Declare some point objects  
    Point pt1, pt2;  
    // Assign their x and y fields  
    pt1.x = 8.5; // Use the dot notation to get to a part of the object  
    pt1.y = 0.0;  
    pt2.x = -4;
```



```

pt2.y = 2.5;
// Print them
cout << "pt1 = (" << pt1.x << ", " << pt1.y << ")" << endl;
cout << "pt2 = (" << pt2.x << ", " << pt2.y << ")" << endl;
// Reassign one point from the other
pt1 = pt2;
cout << "pt1 = (" << pt1.x << ", " << pt1.y << ")" << endl;
cout << "pt2 = (" << pt2.x << ", " << pt2.y << ")" << endl;
// Are pt1 and pt2 aliases? Change pt1's x coordinate and see.
pt1.x = 0;
cout << "pt1 = (" << pt1.x << ", " << pt1.y << ")" << endl;
// Note that pt2 is unchanged
cout << "pt2 = (" << pt2.x << ", " << pt2.y << ")" << endl;
}

```

Listing 14.1 (mathpoints.cpp) prints

```

pt1 = (8.5,0)
pt2 = (-4,2.5)
pt1 = (-4,2.5)
pt2 = (-4,2.5)
pt1 = (0,2.5)
pt2 = (-4,2.5)

```

It is important to note that **Point** is *not* an object. It represents a class of objects. It is a type. The variables **pt1** and **pt2** are the objects, or instances, of the class **Point**. Each of the objects **pt1** and **pt2** has its own copies of fields named **x** and **y**. Figure 14.1 provides a conceptual view of point objects **pt1** and **pt2**. On most systems double-precision floating-point numbers require eight bytes of memory. Since each

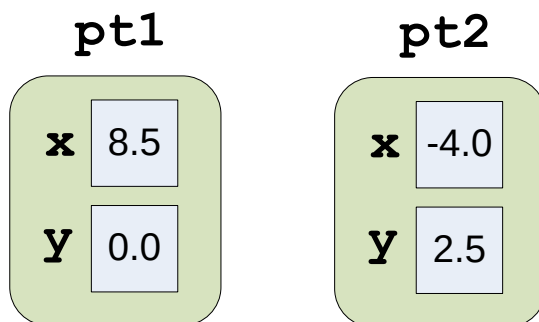


Figure 14.1: Two Point objects with their individual data fields

Point object stores two **doubles**, a **Point** object uses at least 16 bytes of memory. In practice, an object may be slightly bigger than the sum its individual components because most computer architectures restrict

how data can be arranged in memory. This means some objects include a few extra bytes for “padding.” We can use the `sizeof` operator to determine the exact number of bytes an object occupies. Under Visual C++, the expression `sizeof pt1` evaluates to 16.

A client may use the dot (`.`) operator with an object to access one of the object’s members. The expression `pt1.x` represents the x coordinate of object `pt1`. The dot operator is a binary operator; its left operand is a class instance (object), and its right operand is a member of the class.

The assignment statement in Listing 14.1 (`mathpoints.cpp`)

```
pt1 = pt2;
```

and the statements that follow demonstrate that one object may be assigned to another directly without the need to copy each individual member of the object.

As another example, suppose we wish to implement a simple bank account object. We determine that the necessary information for each account consists of a name, ID number, and a balance (amount of money in the account). We can define our bank account class as

```
class Account {
public:
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current balance;
    double balance;
};
```

The `name`, `id`, and `balance` fields constitute the `Account` class, and the fields represent three different types. Listing 14.2 (`bankaccount.cpp`) uses an array of `Account` objects.

Listing 14.2: `bankaccount.cpp`

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

class Account {
public:
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
};

// Allows the user to enter via the keyboard information
// about an account and adds that account to the database.
void add_account(vector<Account>& accts) {
    string name;
    int number;
    double amount;
```

```

    cout << "Enter name, account number, and account balance: ";
    cin >> name >> number >> amount;
    Account acct;
    acct.name = name;
    acct.id = number;
    acct.balance = amount;
    accts.push_back(acct);
}

// Print all the accounts in the database
void print_accounts(const vector<Account>& accts) {
    int n = accts.size();
    for (int i = 0; i < n; i++)
        cout << accts[i].name << ", " << accts[i].id
            << ", " << accts[i].balance << endl;
}

//void swap(Account& er1, Account& er2) {
//    Account temp = er1;
//    er1 = er2;
//    er2 = temp;
//}

bool less_than_by_name(const Account& e1, const Account& e2) {
    return e1.name < e2.name;
}

bool less_than_by_id(const Account& e1, const Account& e2) {
    return e1.id < e2.id;
}

bool less_than_by_balance(const Account& e1, const Account& e2) {
    return e1.balance < e2.balance;
}

// Sorts a bank account database into ascending order
// The comp parameter determines the ordering
void sort(vector<Account>& db,
        bool (*comp)(const Account&, const Account&)) {
    int size = db.size();
    for (int i = 0; i < size - 1; i++) {
        int smallest = i;
        for (int j = i + 1; j < size; j++)
            if (comp(db[j], db[smallest]))
                smallest = j;
        if (smallest != i)
            swap(db[i], db[smallest]);
    }
}

// Allows a user interact with a bank account database.
int main() {

```

```

// The simple database of bank accounts
vector<Account> customers;

// User command
char cmd;

// Are we done yet?
bool done = false;

do {
    cout << "[A]dd [N]ame [I]D [B]alance [Q]uit==> ";
    cin >> cmd;
    switch (cmd) {
        case 'A':
        case 'a':
            // Add an account
            add_account(customers);
            break;
        case 'P':
        case 'p':
            // Print customer database
            print_accounts(customers);
            break;
        case 'N':
        case 'n':
            // Sort database by name
            sort(customers, less_than_by_name);
            print_accounts(customers);
            break;
        case 'I':
        case 'i':
            // Sort database by ID (account number)
            sort(customers, less_than_by_id);
            print_accounts(customers);
            break;
        case 'B':
        case 'b':
            // Sort database by account balance
            sort(customers, less_than_by_balance);
            print_accounts(customers);
            break;
        case 'Q':
        case 'q':
            done = true;
            break;
    }
} while (!done);
}

```

Listing 14.2 (bankaccount.cpp) stores bank account objects in a vector (see Section 11.1). Also, a bank account object contains a **string** object as a field. This shows that objects can contain other objects and implies that our objects can have arbitrarily complex structures.

A sample run of Listing 14.2 (bankaccount.cpp) prints

```

[A]dd [N]ame [I]D [B]alance [Q]uit==> a
Enter name, account number, and account balance: Sheri 34 100.34
[A]dd [N]ame [I]D [B]alance [Q]uit==> a
Enter name, account number, and account balance: Mary 10 1323.00
[A]dd [N]ame [I]D [B]alance [Q]uit==> a
Enter name, account number, and account balance: Larry 88 55.05
[A]dd [N]ame [I]D [B]alance [Q]uit==> a
Enter name, account number, and account balance: Terry 33 423.50
[A]dd [N]ame [I]D [B]alance [Q]uit==> a
Enter name, account number, and account balance: Gary 11 7.27
[A]dd [N]ame [I]D [B]alance [Q]uit==> n
Gary,11,7.27
Larry,88,55.05
Mary,10,1323
Sheri,34,100.34
Terry,33,423.5
[A]dd [N]ame [I]D [B]alance [Q]uit==> i
Mary,10,1323
Gary,11,7.27
Terry,33,423.5
Sheri,34,100.34
Larry,88,55.05
[A]dd [N]ame [I]D [B]alance [Q]uit==> b
Gary,11,7.27
Larry,88,55.05
Sheri,34,100.34
Terry,33,423.5
Mary,10,1323
[A]dd [N]ame [I]D [B]alance [Q]uit==> q

```

The program allows users to sort the bank account database in several different ways using different comparison functions. Notice that the **less_than_by_name** and similar comparison functions use **const** reference parameters for efficiency (see Section 11.1.4).

Observe that the **add_account** function has a local variable named **name**. All **Account** objects have a field named **name**. The **add_account** function uses the **name** identifier in both ways without a problem. The compiler can distinguish between the two uses of the identifier because one is qualified with an object variable before the dot (**.**) operator and the other is not.

14.3 Methods

The objects we have seen so far have been passive entities that have no built-in functionality. In addition to defining the structure of the data for its objects, a class can define functions that operate on behalf of its objects.

Recall the bank account class, **Account**, from Section 14.2:

```
class Account {
```

```

public:
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
};

```

Suppose this design is given to programmer Sam who must write a **withdraw** function. Sam is a careful programmer, so his **withdraw** function checks for overdrafts:

```

/*****
 *  withdraw(acct, amt)
 *      Deducts amount amt from Account acct, if possible.
 *      Returns true if successful; otherwise, it returns false.
 *      A call can fail if the withdraw would
 *      cause the balance to fall below zero
 *
 *      acct: a bank account object
 *      amt:  funds to withdraw
 *
 *      Author: Sam Coder
 *      Date: September 3, 2012
 *****/
bool withdraw(Account& acct, double amt) {
    bool result = false; // Unsuccessful by default
    if (acct.balance - amt >= 0) {
        acct.balance -= amt;
        result = true; // Success
    }
    return result;
}

```

The following code fragment shows the expected code a client might write when withdrawing funds from the bank:

```

// Good client code
// -----
// A simple database of bank customers
vector<Account> accountDB;

// Populate the database via some function
accountDB = initialize_db();

// Get transaction information
int number;
double debit;
cout << "Please enter account number and amount to withdraw:";
cin >> number >> debit;

// Locate account index

```

```

int n = accountDB.size(), i = 0;
while (i < n) {
    if (accountDB[i].id == number)
        break; // Found the account
    i++;
}

// Perform the transaction, if possible
if (i < n) {
    if (withdraw(accountDB[i], debit))
        cout << "Withdrawal successful" << endl;
    else
        cout << "Cannot perform the withdraw" << endl;
}
else
    cout << "Account does not exist" << endl;

```

Unfortunately, nothing prevents a client from being sloppy:

```

// Bad client code
// -----
// A simple database of bank customers
vector<Account> accountDB;

// Populate the database via some function
accountDB = initialize_db();

// Get transaction information
int number;
double debit;
cout << "Please enter account number and amount to withdraw:";
cin >> number >> debit;

// Locate account index
int n = accountDB.size(), i = 0;
while (i < n) {
    if (accountDB[i].id == number)
        break; // Found the account
    i++;
}

// Perform the transaction
if (i < n)
    accountDB[i] -= debit; // What if debit is too big?
else
    cout << "Account does not exist" << endl;

```

Nothing in an **Account** object itself can prevent an overdraft or otherwise prevent clients from improperly manipulating the balance of an account object.

We need to be able to protect the internal details of our bank account objects and yet permit clients to

interact with them in a well-defined, controlled manner.

Consider a non-programming example. If I deposit \$1,000.00 dollars into a bank, the bank then has custody of my money. It is still my money, so I theoretically can reclaim it at any time. The bank stores money in its safe, and my money is in the safe as well. Suppose I wish to withdraw \$100 dollars from my account. Since I have \$1,000 total in my account, the transaction should be no problem. What is wrong with the following scenario:

1. Enter the bank.
2. Walk past the teller into a back room that provides access to the safe.
3. The door to the safe is open, so enter the safe and remove \$100 from a stack of \$20 bills.
4. Exit the safe and inform a teller that you got \$100 out of your account.
5. Leave the bank.

This is not the process a normal bank uses to handle withdrawals. In a perfect world where everyone is honest and makes no mistakes, all is well. In reality, many customers might be dishonest and intentionally take more money than they report. Even though I faithfully counted out my funds, perhaps some of the bills were stuck to each other and I made an honest mistake by picking up six \$20 bills instead of five. If I place the bills in my wallet with other money that is already, I may never detect the error. Clearly a bank needs a more controlled procedure for handling customer withdrawals.

When working with programming objects, in many situations it is better to restrict client access to the internals of an object. Client code should not be able to change directly bank account objects for various reasons, including:

- A withdrawal should not exceed the account balance.
- Federal laws dictate that deposits above a certain amount should be reported to the Internal Revenue Service, so a bank would not want customers to be able to add funds to an account in a way to circumvent this process.
- An account number should never change for a given account for the life of that account.

How do we protect the internal details of our bank account objects and yet permit clients to interact with them in a well-defined, controlled manner? The trick is to hide completely from clients the object's fields and provide special functions, called methods, that have access to the hidden fields. These methods provide the only means available to clients of changing the object's internal state.

In the following revised **Account** class:

```
class Account {  
    // String representing the name of the account's owner  
    string name;  
    // The account number  
    int id;  
    // The current account balance  
    double balance;  
public:  
    // Methods will be added here . . .  
};
```


all the fields no longer reside in the public section of the class definition. This makes the following client code impossible:

```
Account acct;
// Set the account's balance
acct.balance = 100; // Illegal
// Withdraw some funds
acct.balance -= 20; // Illegal
```

The **balance** field is in the private area of the class. You may use the literal **private** label, as in

```
class Account {
private:
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
public:
    // Methods will be added here . . .
};
```

Because any parts of a class not explicitly labeled are implicitly private, the **private** label is not necessary. Said another way, all members of a class are automatically private unless otherwise labeled. Some programmers like to put the public members before the private members within a class definition, as in

```
class Account {
public:
    // Methods will be added here . . .
private:
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
};
```

In this case the **private** label is necessary.

In order to enforce the spirit of the **withdraw** function, we will make it a method, and add a **deposit** method to get funds into an account:

```
class Account {
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
public:
    /*****
```

```

    * deposit(amt)
    *   Adds amount amt to the account's balance.
    *
    *   Author: Sam Coder
    *   Date: September 3, 2012
    *****/
void deposit(double amt) {
    balance += amt;
}

/*****
 * withdraw(amt)
 *   Deducts amount amt from the account's balance,
 *   if possible.
 *   Returns true if successful; otherwise, it returns false.
 *   A call can fail if the withdraw would
 *   cause the balance to fall below zero
 *
 *   amt: funds to withdraw
 *
 *   Author: Sam Coder
 *   Date: September 3, 2012
 *****/
bool withdraw(double amt) {
    bool result = false; // Unsuccessful by default
    if (balance - amt >= 0) {
        balance -= amt;
        result = true; // Success
    }
    return result;
}
};

```

In this new definition of **Account**, the members named **deposit** and **withdraw** are not fields; they are method definitions. A method definition looks like a function definition, but it appears within a class definition. Because of this, a method is also known as a *member function*.

A client accesses a method with the dot (.) operator:

```

// Withdraw money from the Account object named acct
acct.withdraw(100.00);

```

The **withdraw** method definition uses three variables: **amt**, **result**, and **balance**. The variables **amt** and **result** are local to **withdraw**—**amt** is the method's parameter, and **result** is declared within the body of **withdraw**. Where is **balance** declared? It is the field declared in the private section of the class. The **withdraw** method affects the **balance** field of the object upon which it is called:

```

// Affects the balance field of acct1 object
acct1.withdraw(100.00);
// Affects the balance field of acct2 object
acct2.withdraw(25.00);

```

Methods may be overloaded just like global functions (see Section 10.3). This means multiple methods in the same class may have the same names, but their signatures must be different. Recall that a function's signature consists of its name and parameter types; a method's signature too consists of its name and parameter types.

We saw in Section 14.2 that each object provides storage space for its own data fields. An object does not require any space for its methods. This means the only things about an individual object that an executing program must maintain are the object's fields. While the exact organization of memory varies among operating systems, Figure F.10 shows a typical layout. All the data processed by a program appears in one of three sections: stack, heap, or static memory. As with simple data types like `ints`, the fields of an object declared local to a function or method reside in the segment of memory known as the stack. Also like simple data types, the fields of an object allocated with `new` appear on the heap. The fields in global and `static` local objects reside in the static section of the executing program's memory.

Consider the following simple class:

```
class Counter {
    int count;
public:
    // Allow clients to reset the counter to zero
    void clear() {
        count = 0;
    }

    // Allow clients to increment the counter
    void inc() {
        count++;
    }

    // Return a copy of the counter's current value
    int get() {
        return count;
    }
};
```

For this code we see that each `Counter` object will store a single integer value (its `count` field). Under Visual C++ `sizeof Counter` is the same as `sizeof int`: four. A local `Counter` object consumes four bytes of stack space, a global `Counter` object uses four bytes of static memory, and a dynamically-allocated `Counter` object uses four bytes of heap space.

The segment of memory in Figure F.10 labeled *code* stores the machine language for all the program's functions and methods. The compiler translates methods into machine language as it does regular functions. Internally, the method `inc` in the `Counter` class is identified by a longer name, `Counter::inc`. Although `Counter::inc` is a method, in the compiled code it works exactly like a normal function unrelated to any class. In the client code

```
Counter ctr1, ctr2; // Declare a couple of Counter objects
ctr1.clear(); // Reset the counters to zero
ctr2.clear();
ctr1.inc(); // Increment the first counter
```

the statement

```
ctr1.clear();
```

sets the private `count` field of `ctr1` to zero, while the statement

```
ctr2.clear();
```

sets `ctr2`'s `count` field to zero. Since all `Counter` objects share the same `reset` method, how does each call to `Counter::clear` reset the field of the proper `Counter` object? The trick is this: While it appears that the `reset` method of the `Counter` class accepts no parameters, it actually receives a secret parameter that corresponds to the address of the object on left side of the dot. The C++ source code statement

```
ctr1.clear();    // Actual C++ code
```

internally is treated like

```
Counter::clear(&ctr1);    // <-- Not real C++ code
```

in the compiled code. The code within the method can influence the field of the correct object via the pointer it receives from the caller. The `clear` method contains the single statement

```
count = 0;
```

Since the `count` variable is not declared locally within the `clear` method and it is not a global variable, it must be the field named `count` of the object pointed to by the secret parameter passed to `clear`.

Section 15.2 shows how programmers can access this secret pointer passed to methods.

14.4 Constructors

One crucial piece still is missing. How can we make sure the fields of an object have reasonable initial values before a client begins using the object? A class may define a *constructor* that looks similar to a method definition. The code within a constructor executes on behalf of an object when a client creates the object. For some classes, the client can provide information for the constructor to use when initializing the object. As with functions and methods, class constructors may be overloaded. Listing 14.3 (bankaccountmethods.cpp) exercises an enhanced `Account` class that offers `deposit` and `withdraw` methods, as well as a constructor.

Listing 14.3: bankaccountmethods.cpp

```
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

class Account {
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
public:
    // Initializes a bank account object
    Account(const string& customer_name, int account_number,
           double amount):
```

```
        name(customer_name), id(account_number), balance(amount) {
    if (amount < 0) {
        cout << "Warning: negative account balance" << endl;
        balance = 0.0;
    }
}

// Adds amount amt to the account's balance.
void deposit(double amt) {
    balance += amt;
}

// Deducts amount amt from the account's balance,
// if possible.
// Returns true if successful; otherwise, it returns false.
// A call can fail if the withdraw would
// cause the balance to fall below zero
bool withdraw(double amt) {
    bool result = false; // Unsuccessful by default
    if (balance - amt >= 0) {
        balance -= amt;
        result = true; // Success
    }
    return result;
}

// Displays information about the account object
void display() {
    cout << "Name: " << name << ", ID: " << id
        << ", Balance: " << balance << endl;
}

};

int main() {
    Account acct1("Joe", 2312, 1000.00);
    Account acct2("Moe", 2313, 500.29);
    acct1.display();
    acct2.display();
    cout << "-----" << endl;
    acct1.withdraw(800.00);
    acct2.deposit(22.00);
    acct1.display();
    acct2.display();
}
```

Listing 14.3 (bankaccountmethods.cpp) produces

```
Name: Joe, ID: 2312, Balance: 1000
Name: Moe, ID: 2313, Balance: 500.29
-----
Name: Joe, ID: 2312, Balance: 200
Name: Moe, ID: 2313, Balance: 522.29
```

The following characteristics differentiate a constructor definition from a regular method definition:

- A constructor has the same name as the class.
- A constructor has no return type, not even **void**.

The constructor in Listing 14.3 (bankaccountmethods.cpp) initializes all the fields with values supplied by the client. The comma-separated list between the colon and the curly brace that begins the body of the constructor is called the *constructor initialization list*. An initialization list contains the name of each field with its initial value in parentheses. All of the fields that make up an object must be initialized before the body of the constructor executes. In this case the code within the constructor adjusts the balance to zero and issues a warning if a client attempts to create an account with an initial negative balance. The constructor thus ensures an account object's balance can never begin with a negative value. The **withdraw** method ensures that, once created, an **Account** object's balance will never be negative. Notice that the client provides the required constructor parameters at the point of the object's declaration:

```
// Client creating two Account objects
Account acct1("Joe", 2312, 1000.00);
Account acct2("Moe", 2313, 500.29);
```

Since the **Account** class contains a constructor definition which requires arguments, it now is impossible for a client to declare an **Account** object like

```
Account acct3; // Illegal, must supply arguments for constructor
```

If **acct** is a **Account** object defined as in Listing 14.3 (bankaccountmethods.cpp), the following code is illegal:

```
acct.balance -= 100; // Illegal, balance is private
```

Clients do not have direct access to the **balance** field since it is private. Clients instead should use the appropriate method call to adjust the balance of an **Account** object:

```
Account acct("Joe", 1033, 50.00); // New bank account object
acct.deposit(1000.00); // Add some funds
acct.withdraw(2000.00); // Method should disallow this operation
```

The details of depositing and withdrawing funds are the responsibility of the object itself, not the client code. The attempt to withdraw the \$2,000 dollars above would not change the account's balance, and the client can check the return value of **withdraw** to provide appropriate feedback to the user about the error or take steps correct the situation.

A constructor that specifies no parameters is called a *default constructor*. If the programmer does not specify any constructor for a class, the compiler will provide a default constructor that does nothing. If the programmer defines any constructor for a class, the compiler will not generate a default constructor. See Section 15.1 for the consequences of this constructor policy.



If you do not define a constructor for your class, the compiler automatically will create one for you—a default constructor that accepts no parameters. The compiler-generated constructor does not do anything to affect the state of newly created instances.

If you define any constructor for your class, the compiler will not provide a default constructor.

14.5 Destructors

A class constructor ensures that when a client creates an instance of that class the object will begin its life in a well-defined state. A class *destructor* takes care of any necessary clean up at the end of an object's life. Listing 14.4 (`destroyaccount.cpp`) modifies our **Account** class so that it uses dynamic C strings for names. The constructor is responsible for the memory allocation, and the destructor is responsible for the memory clean up.

Listing 14.4: `destroyaccount.cpp`

```
#include <iostream>
#include <string>

using namespace std;

class Account {
    // String representing the name of the account's owner
    string name;
    // The account number
    int id;
    // The current account balance
    double balance;
public:
    // Initializes a bank account object
    Account(const string& customer_name, int account_number, double amount):
        name(name), id(account_number), balance(amount) {
        cout << "Creating Account object #" << id << endl;
        if (amount < 0) {
            cout << "Warning: negative account balance" << endl;
            balance = 0.0;
        }
    }

    // Releases the memory allocated to this account object
    ~Account() {
        cout << "Destroying Account object #" << id << endl;
    }

    // Adds amount amt to the account's balance.
    void deposit(double amt) {
        balance += amt;
    }
}
```

```

// Deducts amount amt from the account's balance,
// if possible.
// Returns true if successful; otherwise, it returns false.
// A call can fail if the withdraw would
// cause the balance to fall below zero
bool withdraw(double amt) {
    bool result = false; // Unsuccessful by default
    if (balance - amt >= 0) {
        balance -= amt;
        result = true; // Success
    }
    return result;
}

// Displays information about the account object
void display() {
    cout << "Name: " << name << ", ID: " << id
        << ", Balance: " << balance << endl;
}

};

// Allows a user interact with a bank account database.
int main() {
    Account acct("Jerry", 193423, 1250.00);
    acct.display();
    // The destructor automatically displays the number of the
    // account being destroyed when main is finished
}

```

Like a constructor, a destructor has the same name as its class, except that the destructor's name begins with a tilde (~) symbol. This tilde symbol distinguishes a destructor from a constructor. Also unlike a constructor, a destructor may not be overloaded, and a destructor may not accept any parameters.

The output of Listing 14.4 (destroyaccount.cpp) is

```

Creating Account object #193423
Name: Jerry, ID: 193423, Balance: 1250
Destroying Account object #193423

```

The constructor prints the first line when the **main** function initializes the **acct** object. The **acct** variable's **display** method prints the second line. The destructor prints the last line at the end of **main**'s execution when the **acct** variable goes out of scope.

14.6 Defining a New Numeric Type

C++'s class feature allows us to define our own complete types. We will define a new numeric type that models mathematical rational numbers. In mathematics, a *rational* number is defined as the ratio of two integers, where the second integer must be nonzero. Commonly called a *fraction*, a rational number's two integer components are called *numerator* and *denominator*. Rational numbers possess certain properties;

for example, two fractions can have different numerators and denominators but still be considered equal ($\frac{1}{2} = \frac{2}{4}$). Listing 14.5 (simplerational.cpp) shows how to define and use rational numbers.

Listing 14.5: simplerational.cpp

```
#include <iostream>
#include <cstdlib>

using namespace std;

// Models a mathematical rational number
class SimpleRational {
    int numerator;
    int denominator;
public:
    // Initializes the components of a Rational object
    SimpleRational(int n, int d): numerator(n), denominator(d) {
        if (d == 0) {
            // Display error message
            cout << "Zero denominator error" << endl;
            exit(1); // Exit the program
        }
    }

    // The default constructor makes a zero rational number
    // 0/1
    SimpleRational(): numerator(0), denominator(1) {}

    // Allows a client to reassign the numerator
    void set_numerator(int n) {
        numerator = n;
    }

    // Allows a client to reassign the denominator.
    // Disallows an illegal fraction (zero denominator).
    void set_denominator(int d) {
        if (d != 0)
            denominator = d;
        else {
            // Display error message
            cout << "Zero denominator error" << endl;
            exit(1); // Exit the program
        }
    }

    // Allows a client to see the numerator's value.
    int get_numerator() {
        return numerator;
    }

    // Allows a client to see the denominator's value.
    int get_denominator() {
        return denominator;
    }
};
```

```

int main() {
    SimpleRational fract(1, 2); // The fraction 1/2
    cout << "The fraction is " << fract.get_numerator()
         << "/" << fract.get_denominator() << endl;
    fract.set_numerator(19);
    fract.set_denominator(4);
    cout << "The fraction is " << fract.get_numerator()
         << "/" << fract.get_denominator() << endl;
}

```

The **SimpleRational** class defines a new numeric type that C++ does not natively provide—the *rational number* type. (It is named **SimpleRational** because it is our first cut at a rational number class; a better version is to come in Listing 16.1 (rational.cpp).) This **SimpleRational** class defines two overloaded constructors. One constructor accepts the numerator and denominator values from the client. The other constructor allows a client to declare a **SimpleRational** object as

```
SimpleRational frac;
```

without supplying the initial numerator and denominator values. This default constructor assigns $\frac{0}{1}$ to the object. Both constructors ensure that a **SimpleRational** object's denominator will not be zero.

Our new numeric type certainly leaves a lot to be desired. We cannot display one of our rational number objects with **cout** very conveniently. We cannot use the standard arithmetic operators like **+** or *******, and we cannot compare two rational numbers using **==** or **<**. In Section ?? we address these shortcomings.

14.7 Encapsulation

When developing complex systems, allowing indiscriminate access to an object's internals can be disastrous. It is all too easy for a careless, confused, or inept programmer to change an object's state in such a way as to corrupt the behavior of the entire system. A malicious programmer may intentionally tweak one or more objects to sabotage the system. In either case, if the software system controls a medical device or military missile system, the results can be deadly.

C++ provides several ways to protect the internals of an object from the outside world, but the simplest strategy is the one we have been using: fields and methods, generically referred to as class members, can be qualified as either **public** or **private**.

The compiler enforces the inaccessibility of private members. In Listing 14.5 (simplerational.cpp), for example, client code cannot directly modify the **denominator** instance variable of a **Rational** object making it zero. A client may influence the values of **numerator** and **denominator** only via methods provided by the class designer.

Accessibility rules, also called visibility rules or permissions, determine what parts of a class and/or object are accessible to the outside world. C++ provides a great deal of flexibility in assigning access permissions, but some general principles exist that, if followed, foster programs that are easier to build and extend.

- In general, fields should be **private**. Clients should not be able to arbitrarily change the state of an object. Allowing such might allow client code to put an object into an undefined state (for example, changing the denominator of a fraction to zero). An object's state should only change as a result of calling the object's methods.

The built-in primitive types like `int` and `double` offer no protection from client access. One exception to the private fields rule applies to simple objects that programmers naturally would treat as primitive types. Recall the geometric `Point` class found in Listing 14.1 (`mathpoints.cpp`). The x and y fields of a point object safely may assume any legitimate floating-point value, and it may be reasonable in some applications for clients to treat a `Point` object as a primitive type. In this case it is appropriate to make the x and y fields public.

- Methods that provide a service to client code should be part of the `public` section of the class. The author of the class must ensure that the `public` methods cannot place the object into an illegal state. For example, the method

```
void set_denominator(int d) {  
    denominator = d;  
}
```

if part of the `Rational` class would permit client code to sabotage a valid fraction with a simple statement:

```
fract.set_denominator(0);
```

- Methods that assist the service methods but that are not meant to be used by the outside world should be in the `private` section of the class. This allows a `public` method to be decomposed into simpler, perhaps more coherent activities without the threat of client code accessing these more primitive methods. These private methods are sometimes called *helper methods* or *auxiliary methods*.

Why would a programmer intentionally choose to limit access to parts of an object? Restricting access obviously limits the client's control over the objects it creates. While this may appear to be a disadvantage at first glance, this access restriction actually provides a number of advantages:

- **Flexibility in implementation.** A class conceptually can be separated into two parts:
 - **The class interface—the public part.** Clients see and can use the public parts of an object. The public methods and public variables of a class constitute the *interface* of the class. A class's interface specifies *what* it does.
 - **The class implementation—the hidden part.** Clients cannot see any private methods or private variables. Since this private information is invisible to clients, class developers are free to do whatever they want with the private parts of the class. A class's implementation specifies *how* it accomplishes what it needs to do.

We would like our objects to be black boxes: clients shouldn't need to know *how* the objects work but merely rely on *what* objects can do.



A good rule of thumb in class design is this: make data `private`, and make methods that provide a service to clients `public`.

Many real-world objects follow this design philosophy. Consider a digital wristwatch. Its display gives its user the current time and, perhaps, date. It can produce different output in different modes; for examples, elapsed time in stopwatch mode or wake up time in alarm mode. It presents to its user

only a few buttons for changing modes, starting and stopping stopwatches, and setting the time. *How* it does what it does is irrelevant to most users; most users are concerned with *what* it does. Its user risks great peril by opening the watch and looking at its intricate internal details. The user is meant to interact with the watch only through its interface—the display and buttons.

Similarly, an automobile presents an accelerator pedal to its user. The user knows that pushing the pedal makes the car go faster. That the pedal is connected to the fuel injection system (and possibly other systems, like cruise control) through a cable, wire, or other type of linkage is of concern only to the automotive designer or mechanic. Most drivers prefer to be oblivious to the under-the-hood details.

Changing the interface of a class can disturb client code that already has been written to use objects of that class. For example, what if the maintainers of the **Rational** class decide that **Rational** objects should be immutable; that is, after a client creates a **Rational** object the client cannot adjust the **numerator** or **denominator** values. The **set_numerator** and **set_denominator**, therefore, would disappear. Unfortunately, both of these methods are public and thus part of **Rational**'s interface to client. Existing client code may be using these methods, and removing them, making them private, altering the types of their parameters or any other changes to the interface would render existing client code incompatible. Client code that has been written to use **set_numerator** according to its original interface no longer will be correct. We say the change in **Rational**'s interface *breaks* the client code.

Class authors have no flexibility to alter the interface of a class once the class has been released for clients to use. On the other hand, altering the private information in a class will not break existing client code that uses that class, since private class information is invisible to clients. When the private parts of a class change, clients need only recompile their code; client programmers do not need to modify their source code. A class, therefore, becomes less resilient to change as more of its components become exposed to clients. To make classes as flexible as possible, which means maximizing the ability to make improvements to the class in the future, hide as much information as possible from clients.

- **Reducing programming errors.** Client code cannot misuse the parts of a class that are private since the client cannot see the private parts of a class. Properly restricting client access can make it impossible for client code to put objects into an undefined state. In fact, if a client can coax an object into an illegal state via the class interface, then the design and/or implementation of the class is faulty. As an example, if a client can somehow make a **Rational** object's denominator zero, then one of the methods or a constructor must contain a logic error. Clients should never be able to place an object into an illegal state.
- **Hiding complexity.** Objects can provide a great deal of functionality. Even though a class may provide a fairly simple interface to clients, the services it provides may require a significant amount of complex code to accomplish their tasks. One of the challenges of software development is dealing with the often overwhelming complexity of the task.

It is difficult, if not impossible, for one programmer to be able to comprehend at one time all the details of a large software system. Classes with well-designed interfaces and hidden implementations provide a means to reduce this complexity. Since private components of a class are hidden, their details cannot contribute to the complexity the client programmer must manage. The client programmer needs not be concerned with exactly how an object works, but the details that make the object work are present nonetheless. The trick is exposing details only when necessary:

- **Class designer.** The class designer must be concerned with the hidden implementation details of the class. Since objects of the class may be used in many different contexts, the class designer usually does not have to worry about the context in which the class will be used. From the

perspective of the class designer, the complexity of the client code that may use the class is therefore eliminated.

- **Applications developer using a class.** The developer of the client code must be concerned with the details of the application code being developed. The application code will use objects. The hidden details of the class these objects represent are of no concern to the client developers. From the perspective of the client code designer, therefore, the complexity of the code that makes the objects work is eliminated.

This concept of information hiding is called *encapsulation*. Details are exposed to particular parties only when appropriate. In sum, the proper use of encapsulation results in

- software that is more flexible and resilient to change,
- software that is more robust and reliable, and
- a software development process that programmers can more easily comprehend and manage.

Finally, the C++ encapsulation model has its limits. It is not possible to protect an object from code within itself. Any method within a class has full access to any member defined within that class. If it appears that parts of a class should be protected from some of its methods, the class should be split up into multiple classes with suitable restrictions among the resulting component classes.

14.8 Summary

- The **class** reserved word introduces a programmer-defined type.
- A semicolon (;) must follow the closing curly brace of a class declaration.
- Variables of a class are called *objects* or *instances* of that class.
- The **public** label within a class specifies the parts of objects of that class that are visible to code outside of that class.
- The parts of a class that are not in the public section are by default private and thus inaccessible to code outside of the class. The **private** label explicitly declares a section of a class to be private. The **public** and **private** labels may appear multiple times within a class declaration allowing programmers to order class members as they chose without being forced to put all the private members in one place.
- The dot (.) operator is used to access elements of an object.
- A data member of a class is known as a *field*. Equivalent terms include *data member*, *instance variable*, and *attribute*.
- A function defined in a class that operates on objects of that class is called a *method*. Equivalent terms include *member function* and *operation*.
- A constructor represents code that executes automatically when an object comes into existence. A constructor has the same name as the class and no specified return type (not even **void**).
- A constructor initialization list assigns initial values to an object's fields. The initialization list appears after the closing parenthesis of the constructor's parameter list but before the opening curly brace of the constructor's body.

- A destructor represents code that executes automatically when an object goes out of scope. Destructors are useful for cleaning up any renewable resources (like dynamically allocated heap memory) that the object may have allocated during its lifetime.
- Encapsulation and data hiding offers several benefits to programmers:
 - Flexibility—class authors are free to change the private details of a class. Existing client code need not be changed to work with the new implementation.
 - Reducing programming errors—if client code cannot touch directly the hidden details of an object, the internal state of that object is completely under the control of the class author. With a well-designed class, clients cannot place the object in an ill-defined state (thus leading to incorrect program execution).
 - Hiding complexity—the hidden internals of an object might be quite complex, but clients cannot see and should not be concerned with those details. Clients need to know *what* an object can do, not *how* it accomplishes the task.

14.9 Exercises

1. Given the definition of the **Rational** number class in Listing 14.5 (simplerational.cpp), complete the function named **add**:

```
Rational add(const Rational& r1, const Rational& r2) {
    // Details go here
}
```

that returns the rational number representing the sum of its two parameters.

2. Given the definition of the geometric **Point** class in Listing 15.3 (point.cpp), complete the function named **distance**:

```
double distance(const Point& r1, const Point& r2) {
    // Details go here
}
```

that returns the distance between the two points passed as parameters.

3. Given the definition of the **Rational** number class in Section 14.3, complete the following function named **reduce**:

```
Rational reduce(const Rational& r) {
    // Details go here
}
```

that returns the rational number that represents the parameter reduced to lowest terms; for example, the fraction 10/20 would be reduced to 1/2.

4. Given the definition of the **Rational** number class in Section 14.3, complete the following function named **reduce**:

```
void reduce(Rational& r) {
    // Details go here
}
```

that uses pass by reference to reduce to lowest terms the parameter passed to it; for example, the fraction 10/20 would be reduced to 1/2.

5. What is the purpose of a class constructor?
6. May a class constructor be overloaded?
7. What are the consequences of declaring a method to be `const`?
8. Why are `const` methods necessary?
9. Given the definition of the **Rational** number class in Section 14.3, complete the following method named **reduce**:

```
class Rational {
    // Other details omitted here ...

    // Returns an object of the same value reduced
    // to lowest terms
    Rational reduce() const {
        // Details go here
    }
};
```

that returns the rational number that represents the object reduced to lowest terms; for example, the fraction 10/20 would be reduced to 1/2.

10. Given the definition of the **Rational** number class in Section 14.3, complete the following method named **reduce**:

```
class Rational {
    // Other details omitted here ...

    // Reduces the object to lowest terms
    void reduce() {
        // Details go here
    }
};
```

that reduces the object on whose behalf the method is called to lowest terms; for example, the fraction 10/20 would be reduced to 1/2.

11. Given the definition of the geometric **Point** class in Section 14.2 add a method named **distance**:

```
class Point {
    // Other details omitted

    // Returns the distance from this point to the
    // parameter p
    double distance(const Point& p) {
        // Details go here
    }
};
```

that returns the distance between the point on whose behalf the method is called and the parameter **p**.

12. Consider the following C++ code:

```
#include <iostream>

using namespace std;

class IntPoint {
public:
    int x;
    int y;
    IntPoint(int x, int y): x(x), y(y) {}
};

class Rectangle {
    IntPoint corner;    // Location of the rectangle's lower-left corner
    int width;          // The rectangle's width
    int height;         // The rectangle's width
public:
    Rectangle(IntPoint pt, int w, int h):
        corner((pt.x < -100) ? -100 : (pt.x > 100 ? 100 : pt.x),
               (pt.y < -100) ? -100 : (pt.y > 100 ? 100 : pt.y)),
        width((w < 0) ? 0 : w), height((h < 0) ? 0 : h) {}

    int perimeter() const {
        return 2*width + 2*height;
    }

    int area() const {
        return width * height;
    }

    int get_width() const {
        return width;
    }

    int get_height() const {
        return height;
    }

    // Returns true if rectangle r overlaps this
    // rectangle object.
    bool intersect(const Rectangle& r) const {
        // Details omitted
    }

    // Returns the length of a diagonal rounded to the nearest
    // integer.
    int diagonal() const {
        // Details omitted
    }
}
```



```

    // Returns the geometric center of the rectangle with
    // the (x,y) coordinates rounded to the nearest integer.
    IntPoint center() const {
        // Details omitted
    }

    bool is_inside(const IntPoint& pt) const {
        // Details omitted
    }
};

int main() {
    Rectangle rect1(IntPoint(2, 3), 5, 7),
               rect2(IntPoint(2, 3), 1, 3),
               rect3(IntPoint(2, 3), 15, 3),
               rect4(IntPoint(2, 3), 5, 3);
    cout << rect1.get_width() << endl;
    cout << rect1.get_height() << endl;
    cout << rect2.get_width() << endl;
    cout << rect2.get_height() << endl;
    cout << rect3.get_width() << endl;
    cout << rect3.get_height() << endl;
    cout << rect4.get_width() << endl;
    cout << rect4.get_height() << endl;
    cout << rect1.get_perimeter() << endl;
    cout << rect1.get_area() << endl;
    cout << rect2.get_perimeter() << endl;
    cout << rect2.get_area() << endl;
    cout << rect3.get_perimeter() << endl;
    cout << rect3.get_area() << endl;
    cout << rect4.get_perimeter() << endl;
    cout << rect4.get_area() << endl;
}

```

- (a) What does the program print?
- (b) With regard to a **Rectangle** object's lower-left corner, what are the minimum and maximum values allowed for the x coordinate? What are the minimum and maximum values allowed for the y coordinate?
- (c) What is a **Rectangle** object's minimum and maximum width?
- (d) What is a **Rectangle** object's minimum and maximum height?
- (e) What happens when a client attempts to create a **Rectangle** object with parameters that are outside the acceptable ranges?
- (f) Implement the **diagonal** method.
- (g) Implement the **center** method.
- (h) Implement the **intersect** method.
- (i) Implement the **is_inside** method.
- (j) Complete the following function named **corner**:

```

    IntPoint corner(const Rectangle& r) {
        // Details go here
    };

```

that returns the lower-left corner of the **Rectangle** object **r** passed to it. You may not modify the **Rectangle** class.

(k)

13. Develop a **Circle** class that, like the **Rectangle** class above, provides methods to compute perimeter and area. The **Rectangle** instance variables are not appropriate for circles; specifically, circles do have corners, and there is no need to specify a width and height. A center point and a radius more naturally describe a circle. Build your **Circle** class appropriately.

14. Given the **Rectangle** and **Circle** classes from questions above, write an **encloses** function:

```

// Returns true if rectangle rect is large enough to
// completely enclose circle circ
bool encloses(const Rectangle& rect, const Circle& circ) {
    // Details omitted
}

```

so that it returns true if circle **circ**'s dimensions would allow it to fit completely within rectangle **rect**. If **circ** is too big, the function returns false. The positions of **rect** and **circ** do not influence the result.

15. Consider the following C++ code:

```

class Widget {
    int value;
public:
    Widget();
    Widget(int v);
    int get();
    void bump();
};

Widget::Widget() {
    value = 40;
}

Widget::Widget(int v) {
    if (v >= 40)
        value = v;
    else
        value = 0;
}

int Widget::get() const {
    return value;
}

void Widget::bump() {
    if (value < 50)

```

```
        value++;
    }

    int main() {
        Widget w1, w2(5);
        cout << w1.get() << endl;
        cout << w2.get() << endl;
        w1.bump(); w2.bump();
        cout << w1.get() << endl;
        cout << w2.get() << endl;
        for (int i = 0; i < 20; i++) {
            w1.bump();
            w2.bump();
        }
        cout << w1.get() << endl;
        cout << w2.get() << endl;
    }
}
```

- (a) What does the program print?
- (b) If **wid** is a **Widget** object, what is the minimum value the expression **wid.get()** can return?
- (c) If **wid** is a **Widget** object, what is the maximum value the expression **wid.get()** can return?

Chapter 15

Fine Tuning Objects

In Chapter 14 we introduced the basics of object-oriented programming: private data, public methods, and automatic initialization and destruction. In this chapter examine some details that enable C++ programmers to fine tune class design.

15.1 Pointers to Objects and Object Arrays

Given the `Point` class from Section 14.2, the statement

```
Point pt;
```

declares the variable `pt` to be a `Point` object. As with primitive data, we can declare pointers to objects:

```
Point pt;
Point *p_pt;
```

Here, `p_pt` is a pointer to a `Point` object. Before we use the pointer we must initialize it to point to a valid object. We can assign the pointer to refer to an existing object, as in

```
p_pt = &pt;
```

or use the `new` operator to dynamically allocate an object from the heap:

```
p_pt = new Point;
```

If the class has a constructor that that accepts parameters, we need to provide the appropriate arguments when using `new`. Recall the `Account` class from Section 14.4. Given the declarations

```
Account acct("Joe", 3143, 90.00);
Account *acct_ptr;
```

before we use `acct_ptr` it must point to a valid object. As in the `Point` class example, we can assign the pointer to refer to an existing object, as in

```
acct_ptr = &acct;
```

or use the `new` operator to dynamically allocate an object from the heap:

```
acct_ptr = new Account("Moe", 400, 1300.00);
```

Note that we include the arguments expected by the **Account** class constructor. This statement allocates memory for one **Account** object and executes the constructor's code to initialize the newly created object. Figure 15.1 illustrates a pointer to an account object. As with any dynamically allocated entity, a

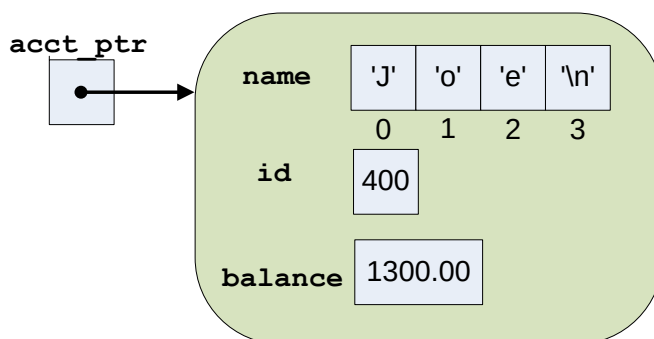


Figure 15.1: A pointer to an **Account** object

programmer must be careful to use the **delete** operator to deallocate any objects created via **new**.

C++ supports vectors and arrays of objects, but they present special challenges. First, consider a simple situation. The **Point** class defines no constructor, so the following code is valid:

```
vector<Point> pts(100);    // Okay
```

The compiler happily generates code that at run time will allocate enough space for 100 **Point** objects. No special initializations are needed since **Point** has no constructor. What if a class defines a constructor that accepts arguments and does not supply also a constructor that requires no arguments? Consider the **Account** class. The following statement is illegal:

```
vector<Account> accts(100); // Illegal, the Account class has
                           // no default constructor
```

When creating the space for the **accts** elements, C++ expects a default constructor properly to properly initialize all of the vector's elements. The only constructor in the **Account** class requires arguments, and so the compiler refuses to accept such a declaration. The compiler cannot ensure that the vector's elements are properly initialized before the programmer begins using the vector.

One solution uses an vector of pointers, as in

```
vector<Account *> accts(100); // A vector of account pointers
```

Note that this does not create any **Account** objects. The programmer subsequently must iterate through the vector and use **new** to create individually each account element. An example of this would be

```
vector<Account *> accts(100); // A vector of account pointers
for (int i = 0; i < 100; i++) {
    // Get information from the user
    cin >> name >> id >> amount;
```

```

    // Create the new account object
    accts[i] = new Account(name, id, amount);
}

```

The dot operator syntax to access a field of a object through a pointer is a bit awkward:

```

Point *p = new Point;
(*p).x = 253.7;
(*p).y = -00.5;

```

The parentheses are required since the dot (.) has higher precedence than the pointer dereferencing operator (*). Without the parentheses, the statement would be evaluated as if the parentheses were placed as shown here

```

*(p.x) = 253.7; // Error

```

The variable `p` is not a `Point` object, but a pointer to a `Point` object, so it must be dereferenced with the `*` operator before applying the `.` operator. C++ provides a simpler syntax to access fields of an object through a pointer. The *structure pointer operator* eliminates the need for the parentheses:

```

Point *p = new Point;
p->x = 253.7;
p->y = -00.5;

```

The pair of symbols `->` constitute one operator (no space in between is allowed), and the operator is meant to look like an arrow pointing right. There is no associated left pointing arrow in C++.

You can use the `->` operator to access the methods of an object referenced by a pointer:

```

Account *acct_ptr = new Account("Joe", 400, 1300.00);
if (acct_ptr->withdraw(10.00))
    cout << "Withdrawal successful" << endl;
else
    cout << "*** Insufficient funds ***" << endl;

```

An executing program will *not* automatically deallocate the memory allocated via `new`; It is the programmer's responsibility to manually deallocate the memory when the object is no longer needed. The `delete` operator with no `[]` decoration frees up a single dynamically allocated object:

```

// acct_ptr points to an Account object previously allocated
// via new
delete acct_ptr;

```

The following function has a logic error because it fails to free up memory allocated with `new`:

```

void faulty_func() {
    Account *acct_ptr = new Account("Joe", 400, 1300.00);
    if (acct_ptr->withdraw(10.00))
        cout << "Withdrawal successful" << endl;
    else
        cout << "*** Insufficient funds ***" << endl;
    acct_ptr->display();
}

```

Here, `acct_ptr` is a local variable, so it occupies space on the stack. It represents a simple address, essentially a number. The space for the `acct_ptr` variable is automatically deallocated when `faulty_func` completes. The problem is `acct_ptr` points to memory that was allocated with `new`, and this memory is not freed up within the function. The pointer `acct_ptr` is the only way to get to that memory, so when the function is finished, that memory is lost for the life of the program. The condition is known as a *memory leak*. If the program runs to completion quickly, the leak may go undetected. In a longer running program such as a web server, memory leaks can cause the program to crash after a period of time. The problem arises when code that leaks memory is executed repeatedly and eventually all of available memory becomes used up.

The corrected function would be written

```
void faulty_func() {
    Account *acct_ptr = new Account("Joe", 400, 1300.00);
    if (acct_ptr->withdraw(10.00))
        cout << "Withdrawal successful" << endl;
    else
        cout << "*** Insufficient funds ***" << endl;
    acct_ptr->display();
    delete acct_ptr;
}
```

Recall (see 10.6) that in C++ there is one literal value to which a pointer of any type may be assigned—`nullptr`:

```
Account *p_rec = nullptr;    // p_rec is null
```

A pointer with the value `nullptr` is interpreted to mean a pointer that is pointing to nothing. An attempt to `delete` a null pointer is legal and does nothing.

Pointers to objects are commonly used to build elaborate linked data structures. Section ?? describes one such data structure.

15.2 The `this` Pointer

Recall our `Counter` class from Section 14.3, reproduced here:

```
class Counter {
    int count;
public:
    // Allow clients to reset the counter to zero
    void clear() {
        count = 0;
    }

    // Allow clients to increment the counter
    void inc() {
        count++;
    }

    // Return a copy of the counter's current value
    int get() {
```

```

        return count;
    }
};

```

In Section 14.3 we saw that an expression such as `ctr1.clear()` passes the address of `ctr1` as a secret parameter during the call to the `clear` method. This is how the method determines which `count` field to reset—`ctr1.clear()` resets `ctr1`'s `count` field, and `ctr2.clear()` would reset `ctr2`'s `count` field.

Within a method definition a programmer may access this secret parameter via the reserved word `this`. Within a method body the `this` expression represents a pointer to the object upon which the method was called. The above `Counter` class may be expressed as

```

// Uses this directly
class Counter {
    int count;
public:
    // Allow clients to reset the counter to zero
    void clear() {
        this->count = 0;
    }

    // Allow clients to increment the counter
    void inc() {
        this->count++;
    }

    // Return a copy of the counter's current value
    int get() {
        return this->count;
    }
};

```

Within a method of the `Counter` class, `this->count` is an alternate way of writing just `count`. Some programmers always use the `this` pointer as shown here to better communicate to human readers that the `count` variable has to be a field, and it cannot be a local or global variable.

The `this` pointer is handy when a method parameter has the same name as a field:

```

class Point {
    double x;
    double y;
private:
    void set_x(double x) {
        // Assign the parameter's value to the field
        this->x = x;
    }
    // Other details omitted . . .
};

```

In the `set_x` method the parameter `x` has the same name as field `x`. This is legal in C++; a method parameter or local variable of a method may have the same name as a field within that class. The problem is that the local variable or parameter hides the access to the field. Any unqualified use of the name `x` refers

to the parameter **x**, not the field **x**. One solution would be to name the parameter something else: perhaps **_x** or **x_param**. A strong argument can be made, though, that **x** is the best name for the field, *and* **x** is also the best name for the parameter to the **set_x** method. To get access to the field in this case, use the **this** pointer. Since **this** is a reserved word, the expression **this->x** cannot be mistaken for anything other than the **x** field of the object upon which the method was invoked.

Another use of the **this** pointer involves passing the current object off to another function or method. Suppose a global function named **log** exists that accepts a **Counter** object as shown here:

```
void log(Counter c) {
    // Details omitted . . .
}
```

If within the **clear** method we wish to call the **log** function passing it the object on whose behalf the **clear** method was executing, we could write

```
class Counter {
    // . . .
public:
    void clear() {
        count = 0;
        // Pass this object off to the log function
        log(*this);
    }
    // . . .
};
```

We pass the expression ***this** as the actual parameter to **log** because the **log** function expects an object (or object reference), not a pointer to an object. Remember the syntax of pointers: If **this** is a pointer, ***this** is the object **this** to which **this** points.

Since **this** serves as an implicit parameter passed to methods, it is illegal to use the **this** expression outside of the body of a method.

15.3 const Methods

Given the **SimpleRational** class in Listing 14.5 (`simplerational.cpp`), the following code produces a compiler error:

```
const SimpleRational fract(1, 2); // Constant fraction 1/2
fract.set_numerator(2);          // Error, cannot change a constant
```

This behavior is desirable, since the **set_numerator** method can change the state of a **SimpleRational** object, and our **fract** object is supposed to be constant. Unfortunately, this correct behavior is accidental. The compiler does not analyze our methods to determine exactly what they do. Consider the following code that also will not compile:

```
const SimpleRational fract(1, 2); // Constant fraction 1/2
cout << fract.get_numerator();    // Error!
```

Since the **get_numerator** method does not modify a **SimpleRational** object, we would expect that invoking it on a constant object should be acceptable, but the compiler rejects it. Again, the compiler cannot

understand what `set_numerator` is supposed to do; specifically, it cannot determine that a method will not change the state of an object. The programmer must supply some additional information to help the compiler.

If a method is not supposed to change the state of an object, that method should be declared `const`. In the `SimpleRational` class, the methods `get_numerator` and `get_denominator` simply return, respectively, copies of the fraction's numerator and denominator. Neither method is intended to modify any instance variables. If we look at the code for those methods, we see that indeed neither method changes anything about a `SimpleRational` object. What if the programmer made a mistake—perhaps a spurious copy and paste error—and the statement

```
numerator = 0;
```

made its way into the `get_numerator` method? Unfortunately, the way things stand now, the compiler cannot detect this error, and `get_numerator` will contain a serious logic error.

We can remove the possibility of such an error by declaring `get_numerator` (and `get_denominator`) `const`:

```
class SimpleRational {
public:
    /* ... stuff omitted ... */
    int get_numerator() const {
        return numerator;
    }
    int get_denominator() const {
        return denominator;
    }
    /* ... other stuff omitted ... */
};
```

The `const` keyword goes after the closing parenthesis of the parameter list and before the opening curly brace of the method's body. If we accidentally attempt to reassign an instance variable in a `const` method, the compiler will report an error.

Declaring a method `const` is not merely a good defensive programming strategy used by a class developer. Methods declared to be `const` can be called with `const` objects, while it is illegal to invoke a non-`const` method with a `const` object. With the new `const` version of `SimpleRational`'s `get_numerator` method, the following code

```
const Rational fract(1, 2);    // Constant fraction 1/2
fract.set_numerator(2);        // Error, cannot change a constant
```

is still illegal, since `Rational::set_numerator` is not `const`, but

```
const Rational fract(1, 2);    // Constant fraction 1/2
cout << fract.get_numerator(); // OK with const get_numerator
```

now is legal.

Any method that has no need to change any field within the object should be declared `const`. Any method that is supposed to change a field in an object should not be declared `const`. A `const` method can be used with both `const` and non-`const` objects, but a non-`const` method cannot be used with `const` objects. For maximum flexibility, always declare a method to be `const` unless doing so would prevent the method from doing what it is supposed to do.

15.4 Separating Method Declarations and Definitions

It is common in larger C++ projects to separate a method implementation from its declaration. A simple example illustrates how to do this; consider the class **MyClass**:

```
class MyClass {
public:
    void my_method() const {
        cout << "Executing \"my_method\"" << endl;
    }
};
```

This version of **MyClass** uses what is known as an *inline method definition*; that is, the method **my_method** is defined completely with a body in the same place as it is declared. Compare the inline version to the following equivalent representation split across two files: **myclass.h** and **myclass.cpp**. The code in **myclass.h** is shown in Listing 15.1 (**myclass.h**).

Listing 15.1: **myclass.h**

```
// File myclass.h

class MyClass {
public:
    void my_method() const;    // Method declaration
};
```

No body for **my_method** appears in the declaration of the class **MyClass** in **myclass.h**; instead, the method implementation appears elsewhere, in **myclass.cpp**. Listing 15.2 (**myclass.cpp**).

Listing 15.2: **myclass.cpp**

```
// File myclass.cpp

// Include the class declaration
#include "myclass.h"
#include <iostream>

// Method definition
void MyClass::my_method() const {
    std::cout << "Executing \"my_method\"" << std::endl;
}
```

Without the prefix **MyClass::** the definition of **my_method** in **myclass.cpp** simply would be global function definition like all the ones we have seen since Chapter 9. The class name and scope resolution operator (**::**) binds the definition of the method to the class to which it belongs.

The **.h** header file would be **#included** by all the source files that need to use **MyClass** objects; the **myclass.cpp** file would be compiled separately and linked into the rest of the project's **.cpp** files.

A **Point** class written inline as

```
class Point {
    double x;
    double y;
public:
```

```

    Point(double x, double y): x(x), y(y) {}
    double get_x() const { return x; }
    double get_y() const { return y; }
    void set_x(double v) { x = v; }
    void set_y(double v) { y = v; }
};

```

could be split up into the declaration:

```

class Point {
    double x;
    double y;
public:
    Point(double x, double y);    // No constructor implementation
    double get_x() const;        // and no method
    double get_y() const;        // implementations`
    void set_x(double v);
    void set_y(double v);
};

```

and method implementations:

```

// Point constructor
Point::Point(double x, double y): x(x), y(y) {}

// Get the x value
double Point::get_x() const {
    return x;
}

// Get the y value
double Point::get_y() const {
    return y;
}

// Set the x value
void Point::set_x(double v) {
    x = v;
}

// Set the y value
void Point::set_y(double v) {
    y = v;
}

```

The class name prefix such as `Point::` is necessary not only so a method definition such as `get_x` can be distinguished from a global function definition, but also to distinguish it from a different class that contains a method with the same name and parameter types. A method signature for a method is just like a global function signature, except a method signature includes the class name as well. Each of the following represent distinct signatures:

- `get_x()` is the signature for a global function named `get_x` that accepts no parameters.

- **Point::get_x()** is the signature for a method of the **Point** class named **get_x()** that accepts no parameters.
- **LinearEquation::get_x()** is the signature for a method of the **LinearEquation** class named **get_x()** that accepts no parameters.

Recall from Section 10.3 that the return type is not part of a function's signature. The same is true for methods.

Many C++ programmers avoid the inline style of class declarations and use the separate class declaration and method definition files for several reasons:

- If the class is meant to be used in multiple programs, the compiler must recompile the methods each time the header file is **#included** by some C++ source file. When the method declarations and definitions are separate, the compiler can compile the code for the definitions once, and the linker can combine this compiled code with the client code that uses it.
- Client programmers can look at the contents of header files. If method definitions are inlined, client programmers can see exactly how the methods work. This can be a disadvantage; for example, a client programmer may make assumptions about how fast a method takes to execute or the particular order in which a method processes data in a vector. These assumptions can influence how the client code calls the method. If the class maintainer changes the implementation of the method, the client programmer's previous assumptions may be invalid. A certain ordering of the data that before the change resulted in faster processing may now be slower. An improvement in a graphics processing algorithm may cause an animation to run too quickly when the method is rewritten. For the class designer's maximum flexibility, client programmers should not be able to see the details of a method's implementation because then they cannot form such assumptions.

Client programmers need to know *what* the method does, not *how* it accomplishes it.

To enforce this hidden method implementation:

1. Separate the method declarations from their definitions. Put the class declaration in the header file and the method definitions in a **.cpp** file.
 2. Compile the **.cpp** file into an object file.
 3. Provide the client the **.h** file and the compiled object file but not the source file containing the method definitions.
 4. The client code can now be compiled by including the **.h** file and linked with the object file, but the client programmer has no access to the source code of the method definitions.
- Some programmers find the inline style difficult since it provides too much detail. It complicates determining *what* objects of the class are supposed to do because the *how* is sprinkled throughout the class declaration.

Despite the disadvantages mentioned above, inline methods are sometimes appropriate. For simple classes defined in the same file that uses them, the inline style is handy, as Listing 15.3 (point.cpp) shows.

Listing 15.3: point.cpp

```
#include <iostream>

class Point {
    double x;
    double y;
```

```

public:
    Point(double x, double y): x(x), y(y) {}
    double get_x() const { return x; }
    double get_y() const { return y; }
    void set_x(double v) { x = v; }
    void set_y(double v) { y = v; }
};

double dist(const Point& pt1, const Point& pt2) {
    // Compute distance between pt1 and pt2 and return it
    // This is a function stub; add the actual code later
    return 0.0; // Just return zero for now
}

int main() {
    Point p1(2.5, 6), p2(0.0, 0.0);
    std::cout << dist(p1, p2) << std::endl;
}

```

Here the `Point` class is very simple, and the constructor and method implementations are obvious. The inline structure makes more sense in this situation.

Listing 15.4 (`trafficlight.h`) contains the interface for a class used to create objects that simulate stop-caution-go traffic signals.

Listing 15.4: `trafficlight.h`

```

class Trafficlight {
public:
    // Class constants available to clients
    enum SignalColor { Red, Green, Yellow };
private:
    SignalColor color; // The light's current color: Red, Green, or Yellow
public:

    Trafficlight(SignalColor initial_color);
    void change();
    SignalColor get_color() const;
};

```

Notice the `Trafficlight` class defines a public enumerated type (`SignalColor`) that clients may use. Because the enumerated type is defined inside the class, clients must use the fully-qualified name `Trafficlight::SignalColor` when declaring variables or parameters of this type.

The traffic signal implementation code in Listing 15.5 (`trafficlight.cpp`) defines the `Trafficlight` methods.

Listing 15.5: `trafficlight.cpp`

```

#include "trafficlight.h"

// Ensures a traffic light object is in the state of
// red, green, or yellow. A rogue value makes the
// traffic light red
Trafficlight::Trafficlight(Trafficlight::SignalColor initial_color) {
    switch (initial_color) {

```

```

    case Red:
    case Green:
    case Yellow:
        color = initial_color;
        break;
    default:
        color = Red; // Red by default, just in case
}
}

// Ensures the traffic light's signal sequence
void Trafficlight::change() {
    // Red --> green, green --> yellow, yellow --> red
    // Treat enum type as int, compute successor color, then
    // covert back to enum type.
    color = static_cast<SignalColor>((color + 1) % 3);
}

// Returns the light's current color so a client can
// act accordingly
Trafficlight::SignalColor Trafficlight::get_color() const {
    return color;
}

```

The code within Listing 15.5 (trafficlight.cpp) must **#include** the trafficlight.h header so the compiler is exposed to the **TrafficLight** class declaration; otherwise, when compiling trafficlight.cpp the compiler would not know if the method implementations faithfully agreed with declarations. If the method definition of **TrafficLight::change** in trafficlight.cpp specified parameters but its declaration within trafficlight.h did not, that would be a problem. Furthermore, without including trafficlight.h, the type **SignalColor** would be an undefined type within trafficlight.cpp.

Notice that outside the class declaration in Listing 15.4 (trafficlight.h) we must prefix the method names and enumerated type **SignalColor** with **TrafficLight::**. Without this prefix the compiler would treat the identifiers as globals. **TrafficLight::SignalColor** is the type declared in the **TrafficLight** class, but **SignalColor** by itself is treated as a global type (that does not exist).

Listing 15.6 (trafficmain.cpp) shows how a client could use the **TrafficLight** class.

Listing 15.6: trafficmain.cpp

```

#include <iostream>
#include "trafficlight.h"

using namespace std;

void print(Trafficlight lt) {
    Trafficlight::SignalColor color = lt.get_color();
    cout << "+-----+" << endl;
    cout << "|         |" << endl;
    if (color == Trafficlight::Red)
        cout << "| (R)  |" << endl;
    else
        cout << "| ( ) |" << endl;
    cout << "|         |" << endl;
    if (color == Trafficlight::Yellow)

```

```

        cout << "| (Y) |" << endl;
    else
        cout << "| ( ) |" << endl;
    cout << "|      |" << endl;
    if (color == Trafficlight::Green)
        cout << "| (G) |" << endl;
    else
        cout << "| ( ) |" << endl;
    cout << "|      |" << endl;
    cout << "+-----+" << endl;
}

int main() {
    Trafficlight light(Trafficlight::Green);
    while (true) {
        print(light);
        light.change();
        cin.get();
    }
}

```

Listing 15.6 (trafficmain.cpp) first prints

```

+-----+
|      |
| ( ) |
|      |
| ( ) |
|      |
| (G) |
|      |
+-----+

```

When the user presses the **Enter** key the program prints

```

+-----+
|      |
| ( ) |
|      |
| (Y) |
|      |
| ( ) |
|      |
+-----+

```

a second **Enter** press prints


```

+-----+
|       |
|  (R)  |
|       |
|  ( )  |
|       |
|  ( )  |
|       |
+-----+

```

A third **Enter** press completes the cycle:

```

+-----+
|       |
|  ( )  |
|       |
|  ( )  |
|       |
|  (G)  |
|       |
+-----+

```

The program's execution continues in this manner within its infinite loop until the user presses **Ctrl C**.

15.5 Preventing Multiple Inclusion

C++ follows the *one definition rule*, which means a variable, function, class, instance variable, or method may not have more than one definition in the same context. The code within the following function definition violates the one definition rule:

```

int sum_and_display(const vector<int>& list) {
    int s = 0, size = list.size();
    // Add up the values in the list
    int i = 0;
    while (i < size) {
        s += list[i];
        i++;
    }
    // Print the contents of the list
    int i = 0; // Illegal, variable i already defined above
    while (i < size) {
        cout << setw(5) << list[i] << endl;
        i++;
    }
}

```

```

        cout << "-----" << endl;
        cout << setw(5) << s << endl;
    }

```

The line

```
int s = 0, size = list.size();
```

is not a problem even though the identifier **size** appears twice. The first appearance denotes the local variable named **size**, and the second use is a call to the **size** method of the **std::vector** class. This statement defines (declares) only the variable **size**; the **size** method already has been defined in the **std::vector** class (and the compiler processed its definition from the `<vector>` header file).

The variable **i** has two distinct definitions in the same context, so its redeclaration right before the display code is an error.

Like a variable, a class may have only one definition. When we build general purpose classes meant to be used widely in our programs we must take care that we do not violate the one definition rule. In fact, we can violate the one definition rule accidentally even if we define a class only once!

To see how we can accidentally violate the one definition rule, consider the following simple example. Suppose we have a simple counter class declared in `count.h`:

```

class Counter {
    int count;
public:
    // Allow clients to reset the counter to zero
    void clear();

    // Allow clients to increment the counter
    void inc();

    // Return a copy of the counter's current value
    int get() const;
};

```

with its implementation in `count.cpp`:

```

// Allow clients to reset the counter to zero
void Counter::clear() {
    count = 0;
}

// Allow clients to increment the counter
void Counter::inc() {
    count++;
}

// Return a copy of the counter's current value
int Counter::get() const {
    return count;
}

```

Next, consider the following two header files defining objects that themselves use **Counter** objects, `wid-get.h`:

```
// File widget.h

#include "count.h"

class Widget {
    Counter meter;
public:
    // Other stuff omitted
};
```

and gadget.h:

```
// File gadget.h

#include "count.h"

class Gadget {
    Counter ticker;
public:
    // Other stuff omitted
};
```

The preprocessor directive

```
#include "count.h"
```

is required in the file `widget.h`, because the compiler must be acquainted with the type **Counter** to properly handle the instance variable **meter**. Similarly, the `count.h` header must be included in `gadget.h` in order for the compiler to accept **Gadget**'s declaration of its **ticker** field.

Client code that wishing to use **Widget** objects must use the appropriate `#include` statement:

```
#include "widget.h"

int main() {
    Widget myWidget;
    // Use the myWidget object
}
```

Similarly, client code that uses **Gadget** objects must include at the top of its file:

```
#include "gadget.h"

int main() {
    Gadget myGadget;
    // Use the myGadget object
}
```

Both of these client programs will build without any problems. Sometimes, however, a program may need to use both **Widget** objects *and* **Gadget** objects. Since the `widget.h` header file does not know anything about the **Gadget** class and the `gadget.h` header file has no information about **Widgets**, we must include both header files in code that uses both classes. The appropriate include directives would be

```
#include "widget.h"
```

```
#include "gadget.h"

int main() {
    Widget myWidget;
    Gadget myGadget;
    // Use the myWidget and myGadget objects
}
```

Including one of the header files without the other is insufficient since the compiler must be able to check if the client is using both types correctly. This client code, however, will not compile. In this case the problem is with `count.h`. Including both `widget.h` and `gadget.h` includes the definition of the **Counter** class twice, so the compiler sees two definitions of **Counter**. Even though the two definitions are identical, this violates the one definition rule and so results in a compiler error.

The issue here is known as *multiple inclusion*, and it can arise when header files `#include` other header files. Multiple inclusion is a problem, but it often is necessary because the programmer may have a legitimate need for both types of objects within the same program. Fortunately, the solution is simple. The standard way to prevent multiple inclusion is to wrap a class definition as follows:

```
#ifndef _COUNT_H_
#define _COUNT_H_

class Counter {
    int count;
public:
    // Allow clients to reset the counter to zero
    void clear();

    // Allow clients to increment the counter
    void inc();

    // Return a copy of the counter's current value
    int get() const;
};

#endif
```

Do not use semicolons at the end of the lines beginning with `#ifndef`, `#define`, and `#endif` because these are preprocessor directives, not C++ statements. The word following the `#ifndef` and `#define` preprocessor directives can be any valid identifier, but best practice produces a unique word tied to the name of the header file in which it appears. The convention is to use all capital letters and underscores as shown above (an identifier cannot contain a dot). Putting underscores before and after the header file name further makes it less likely that name will be used elsewhere within the program. If the header file is named `myheader.h`, the preprocessor crapper would be

```
#ifndef _MYHEADER_H_
#define _MYHEADER_H_

/* Declare your classes here */

#endif
```

Operator	Class	Function	Example	Chapter
<code>operator<<</code>	<code>std::ostream</code>	Right-hand operand is sent to the stream specified by the left-hand operand	<code>cout << x;</code>	Chapter 2
<code>operator>></code>	<code>std::istream</code>	Right-hand operand is extracted from the stream specified by the left-hand operand	<code>cin >> x;</code>	Chapter 4
<code>operator[]</code>	<code>std::vector</code>	Right-hand operand (the integer within the square brackets) is used to locate an element within the left-hand operand (a vector)	<code>x = v[2];</code>	Section 11.1
<code>operator==</code>	<code>std::string</code>	Right-hand operand (a string) is compared with the left-hand operator (also a string) to determine if they contain exactly the same characters in exactly the same order	<code>if (word == "Please") proceed();</code>	Chapter 13

Table 15.1: Some Overloaded Operators for Objects

Like a C++ program, the preprocessor can maintain a collection of variables. The preprocessor directive **#ifndef** evaluates to true the preprocessor has not seen the definition of a given variable; **#ifndef** returns false if it has seen the definition of the variable. The **#define** directive defines a variable. The net effect of the **#ifndef/#define/#endif** directives is that the preprocessor will not include the header file more than once when it is processing a C++ source file. This means the compiler will see the class definition exactly once, thus satisfying the one definition rule.

The **#ifndef...#define...#endif** preprocessor directives should be used to wrap the class declaration in the header file and do not appear in the `.cpp` file containing the method definitions. Since you cannot always predict how widespread the use of a class will become, it is good practice to use this preprocessor trick for all general purpose classes you create. By doing so you will avoid the problem of multiple inclusion.

15.6 Overloaded Operators

Many of the C++ operators can be overloaded to work with programmer-defined types. Table 15.1 shows some operators that have been defined to work with several of the standard object classes.

We can define how specific operators work for the types we devise. We may express operators for classes either global functions or member functions.

15.6.1 Operator Functions

Consider a simple mathematical point class:

```
class Point {
public:
    double x;
    double y;
};
```

Suppose we wish to define the operation of addition on points as follows:

$$(x_1, y_1) + (x_2, y_2) = (x_1 + y_1, x_2 + y_2)$$

Thus, the x coordinate of the sum of two points is the sum of the x coordinates, and the y coordinate is the sum of the y coordinates.

We can overload the addition operator to work on **Point** objects with the following global function:

```

Point operator+(const Point& p1, const Point& p2) {
    Point result;
    result.x = p1.x + p2.x;
    result.y = p1.y + p2.y;
    return result;
}

```

With a slightly more sophisticated `Point` definition:

```

class Point {
    double x; // Fields are now private
    double y;
public:
    // Add a constructor
    Point(double _x, double _y);
    // Add some methods to access the private data members
    double getX() const;
    double getY() const;
};

```

with the expected constructor and method implementations:

```

// Initialize fields
Point::Point(double _x, double _y): x(_x), y(_y) {}

double Point::getX() const {
    return x; // Return a copy of the x member
}

double Point::getY() const {
    return y; // Return a copy of the y member
}

```

we can write the operator function in one line:

```

Point operator+(const Point& p1, const Point& p2) {
    return Point(p1.getX() + p2.getX(), p1.getY() + p2.getY());
}

```

The output stream `<<` operator is often overridden for custom classes. The `cout` object (and, therefore, the `ostream` class) has overloaded methods named `operator<<` that allow the primitive types like integers and floating-point numbers to be printed. If we create a new type, such as `Stopwatch`, the `ostream` class has no `operator<<` method built in to handle objects of our new type. In order to use `cout`'s `<<` operator with programmer-defined objects we must define a global operator function of the form:

```

ostream& operator<<(ostream& os, const X& x)

```

where `X` represents a programmer-defined type. Notice that the function returns type `ostream&`. This is because the first parameter is also an `ostream&`, and the function returns the same object that was passed into it. If `x` is a programmer-defined type, the expression

```

operator<<(cout, x)

```

thus evaluates to a reference to `cout`. This process of returning the object that was passed allows the `<<` operator to be chained, as in

```
cout << x << y << endl;
```

This is the beautified syntax for

```
operator<<(operator<<(operator<<(cout, x), y), endl);
```

If you examine this expression carefully, you will see that the first argument to all the calls of `operator<<` is simply `cout`.

For our enhanced `Point` class, `operator<<` could look like:

```
ostream& operator<<(ostream& os, const Point& pt) {
    os << '(' << pt.getX() << ',' << pt.getY() << ')';
    return os;
}
```

The function simply returns the same `ostream` object reference that was passed into it via the first parameter.

Given the above definitions of `operator+` and `operator<<`, clients can write code such as

```
Point my_point(1, 2), your_point(0.45, 0);
cout << "Point 1: " << my_point << endl;
cout << "Point 2: " << your_point << endl;
cout << my_point << " + " << your_point
    << " = " << my_point + your_point << endl;
```

which displays

```
Point 1: (1.0, 2.0)
Point 2: (0.45, 0.0)
(1.0, 2.0) + (0.45, 0.0) = (1.45, 2.0)
```

When class developers provide such an `operator<<` function, clients can print out objects just as easily as printing the base data types.

The `<<` operator is not overloaded for the `std::vector` class, but we now easily can do it ourselves as needed. For a vector of integers, the following function

```
ostream& operator<<(ostream& os, const vector<int>& vec) {
    os << '{';
    int n = vec.size();
    if (n != 0) {
        os << vec[0]
        for (int i = 1; i < n; i++)
            os << ',' << vec[i];
    }
    os << '}';
    return os;
}
```

provides the necessary functionality. Given this `operator<<` function, clients can display vectors as they would a built in type:

```
vector<int> list(10, 5);
cout << list << endl;
```

This code fragment prints

```
{5, 5, 5, 5, 5, 5, 5, 5, 5, 5}
```

15.6.2 Operator Methods

A class may define operator methods. A method for a unary operator accepts no parameters, and a method for a binary operator accepts only one parameter. The missing parameter is the object upon which the operator is applied, the `this` pointer. To see how operator methods work, consider an enhanced rational class:

```
class EnhancedRational {
    int numerator;
    int denominator;
public:
    /* Other details omitted */

    // Unary plus returns the double-precision floating-point
    // equivalent of the rational number.
    double operator+() const {
        return static_cast<double>(numerator)/denominator;
    }

    // Binary plus returns the rational number that results
    // from adding another rational number (the parameter) to
    // this object.
    Rational operator+(Rational other) const {
        int den = denominator * other.denominator,
            num = numerator * other.denominator
                + other.numerator * denominator;
        Rational result(num, den);
        return result;
    }
};
```

The following code fragment shows how clients can use these operator methods:

```
Rational fract1(1, 2), fract2(1, 3), fract3;
double value = +fract1;    // Assigns 0.5
fract3 = fract1 + fract2;  // fract3 is 5/6
```

Remember that the statement

```
fract3 = fract1 + fract2;
```


is syntactic sugar for the equivalent statement

```
fract3 = fract1.operator+(fract2);
```

Here we see how the righthand operand of the plus operator, **fract2**, becomes the single parameter to the binary **operator+** method of **EnhancedRational**.

The statement

```
double value = +fract1;
```

is the syntactically sugared version of

```
double value = fract1.operator+();
```

While programmers may change the meaning of many C++ operators in the context of objects, the precedence and associativity of all C++ operators are fixed. In the **Rational** class, for example, it is impossible to enable the **+** operator to have a higher precedence than the ***** in the context of **Rational** objects or any other classes of objects.

15.7 static Members

Variables declared in a class declaration are known as instance variables because each instance (object) of that class maintains its own copy of the variables. This means, for example, that changing the numerator of one **Rational** object will not affect the numerator of any other **Rational** object.

Sometimes it is convenient to have variables or constants that are shared by all objects of a class. Global variables and constants certainly will work, but globals are not tied to any particular class. C++ uses the **static** keyword within a class to specify that all objects of that class share a field; for example,

```
class Widget {
    int value;
    static int quantity;
};
```

Each individual **Widget** object has its own **value** variable, but only one **quantity** variable exists and is shared by all **Widget** objects.

One unusual thing about static fields is that must be *defined* outside of the class declaration. For the **Widget** class above, the statement

```
int Widget::quantity;
```

must appear somewhere in the source code outside of the class declaration.

Consider a widget factory in which each widget object must have a unique serial number. Serial numbers are sequential, and a new widget object's serial number is one larger than the widget produced immediately before. Important for warranty claims, a client should not be able to alter a serial number of a widget object. The **Widget** class in Listing 15.7 (serialnumber.cpp) shows how to use a **static** variable to manage serial numbers for widget objects.

Listing 15.7: serialnumber.cpp

```
#include <iostream>
```

```

using namespace std;

class Widget {
    // All Widget objects share serial_number_source
    static int serial_number_source;
    // Each Widget object is supposed to have its own
    // unique serial_number
    int serial_number;
public:
    // A Widget object's serial number is initialized from
    // the shared serial_number_source variable which is
    // incremented each time a Widget object is created
    Widget(): serial_number(serial_number_source++) {}
    // Client's can look at the serial number but not touch
    int get_serial_number() const {
        return serial_number;
    }
};

// Initialize the initial serial number; the first
// Widget made will have serial number 1
int Widget::serial_number_source = 1;

// Make some widgets and check their serial numbers
int main() {
    Widget w1, w2, w3, w4;
    cout << "w1 serial number = " << w1.get_serial_number() << endl;
    cout << "w2 serial number = " << w2.get_serial_number() << endl;
    cout << "w3 serial number = " << w3.get_serial_number() << endl;
    cout << "w4 serial number = " << w4.get_serial_number() << endl;
}

```

The output of Listing 15.7 (serialnumber.cpp) is

```

w1 serial number = 1
w2 serial number = 2
w3 serial number = 3
w4 serial number = 4

```

Each **Widget** object has its own **serial_number** variable, but all **Widget** objects have access to the shared **serial_number_source** variable. The executing program initializes **serial_number_source** one time at the beginning of the program's execution before it calls the **main** function. This means **serial_number_source** is properly initialized to 1 before the program creates any **Widget** objects. Each time the client creates a new **Widget** object, the constructor assigns the individual object's serial number from the **static** variable. The constructor also increments **serial_number_source**'s value, so the next object created will have a serial number one higher than the previous **Widget**.

C++ programmers often use class static fields to provide public constants available to clients. Consider Listing 15.8 (trafficlight.h) that models a simple traffic light a little differently from Listing 15.4 (trafficlight.h). It uses symbolic integer constants instead of enumeration types.

Listing 15.8: trafficsignal.h

```

#ifndef _TRAFFICSIGNAL_H_
#define _TRAFFICSIGNAL_H_

class TrafficSignal {
    int color; // The light's current color: RED, GREEN, or YELLOW
public:
    // Class constants available to clients
    static const int RED = 0;
    static const int GREEN = 1;
    static const int YELLOW = 2;

    TrafficSignal(int initial_color);
    void change();
    int get_color() const;
};

#endif

```

The state of a traffic light object—which of its lamps is illuminated—is determined by an integer value: 0 represents *red*, 1 stands for *green*, and 2 means *yellow*. It is much more convenient for clients to use the symbolic constants **RED**, **GREEN**, and **YELLOW** than to try to remember which integer values stand for which colors. These constants are **public**, so clients can freely access them, but, since they are constants, clients cannot alter their values. An additional benefit to being **const** is this: You may initialize a **static const** field within the class body itself. You do not need to re-define a **static const** field outside the class body as you do for a **non-const static** field.

Listing 15.9 (trafficsignal.cpp) provides the implementation of the methods of **TrafficSignal** class.

Listing 15.9: trafficsignal.cpp

```

#include "trafficsignal.h"

// Ensures a traffic light object is in the state of
// red, green, or yellow. A rogue integer value makes the
// traffic light red
TrafficSignal::TrafficSignal(int initial_color) {
    switch (initial_color) {
        case RED:
        case GREEN:
        case YELLOW:
            color = initial_color;
            break;
        default:
            color = RED; // Red by default
    }
}

// Ensures the traffic light's signal sequence
void TrafficSignal::change() {
    // Red --> green, green --> yellow, yellow --> red
    color = (color + 1) % 3;
}

// Returns the light's current color so a client can

```

```
// act accordingly
int TrafficSignal::get_color() const {
    return color;
}
```

Within the methods of the **TrafficSignal** class we can use the simple names **RED**, **GREEN**, and **YELLOW**. Code outside of the **TrafficSignal** class can access the color constants because they are public but must use the fully-qualified names **TrafficSignal::RED**, **TrafficSignal::GREEN**, and **TrafficSignal::YELLOW**.

A client can create a traffic light object as

```
TrafficSignal light(TrafficSignal::YELLOW);
```

Since the client code is outside the **TrafficSignal** class it must use the full **TrafficSignal::YELLOW** name. This statement makes an initially yellow traffic light. Since the **RED**, **GREEN**, and **YELLOW** public fields are constants, clients cannot access them directly to control the state of a traffic light object.

It may not be obvious, but the color constants in the **TrafficSignal** class *must* be declared **static**. To see why, consider a minor change to **TrafficSignal** in the class we will call **TrafficSignalAlt**:

```
class TrafficSignalAlt {
    int color; // The light's current color: RED, GREEN, or YELLOW
public:
    // Constant instance fields available to clients
    const int RED; // Note: NOT static
    const int GREEN;
    const int YELLOW;

    TrafficSignalAlt(int initial_color);
    void change();
    int get_color() const;
};
```

The **TrafficSignalAlt** class is identical to **TrafficSignal**, except that the color constants are constant instance fields instead of constant class (**static**) fields. Observe that just like the **TrafficSignal** class, the **TrafficSignalAlt** class has no default constructor. Recall from Section 14.4 that a default constructor accepts no arguments, and that if a programmer provides any constructor for a class, the compiler will not automatically provide a default constructor. This means the client cannot write code such as

```
TrafficSignal light;
```

or

```
TrafficSignalAlt light2;
```

During the object's creation the client *must* provide an integer argument representing a traffic light color. If **RED**, **GREEN**, and **YELLOW** are constant instance variables (that is, constant non-**static** fields), every **TrafficSignalAlt** object has its own copy of the fields, and the **RED**, **GREEN**, and **YELLOW** fields cannot exist outside of any traffic light object. This leads to a chicken-and-egg problem—how can we create the first **TrafficSignalAlt** object using the symbolic constants **RED**, **GREEN**, or **YELLOW**? These constants do not exist unless we have a traffic light object, yet we need a traffic light object to have any of these constants!

During a program's execution, **static** class fields are created before **main** begins executing. This means any data pertaining to a class that must exist before any object of that class is created must be declared **static**. Static class variables exist outside of any instance of that class.

C++ allows methods to be declared **static**. A **static** method executes on behalf of the class, not an instance of the class. This means that a **static** method may not access any instance variables (that is non-**static** fields) of the class, nor may they call other non-**static** methods. Since a **static** method executes on behalf of the class, it has no access to the fields of any particular instance of that class. That explains the restriction against **static** methods accessing non-**static** data members. Since a non-**static** method may access instance variables of an object upon which it is called, a **static** method may not call a non-**static** method and thus indirectly have access to instance variables. The restriction goes only one way—any class method, **static** or non-**static**, may access a **static** data member or call a **static** method. Said another way, all non-**static** methods have the **this** implicit parameter (Section 15.2). No **static** method has the **this** parameter.

15.8 Classes vs. structs

All members of a C++ class are **private** by default. The class

```
class Point {  
public:  
    double x;  
    double y;  
};
```

must use the **public** label so clients can access a **Point** object's fields. The **struct** keyword is exactly like the **class** keyword, except that the default access to members is **public**:

```
struct Point {  
    double x;    // These fields now are public  
    double y;  
};
```

The C language supports the **struct** feature, but not the **class** keyword. C **structs** do not support methods, constructors, and destructors. In C++ (unlike in C), a **struct** can contain methods, constructors, and destructors. By default, members in **structs** are public, but you can apply the **private** and **public** labels as in a class to fine tune client access.

Despite their similarities, C++ programmers favor **classes** over **structs** for programmer-defined types with methods. The **struct** construct is useful for declaring simple composite data types that are meant to be treated like primitive types. Programmers can manipulate directly integers, for example. Integers do not have methods or any hidden parts. Likewise, a geometric point object consists of two coordinates that can assume any valid floating-point values. It makes sense to allow client code to manipulate directly the coordinates, rather than forcing clients to use methods like **set_x** and **set_y**. On the other hand, it is unwise to allow clients to modify directly the denominator of a **Rational** object, since a fraction with a zero denominator is undefined.



In C++, by default everything in an object defined by a **struct** is accessible to clients that use that object. In contrast, clients have no default access to the internals of an object that is an instance of a **class**. The default member access for **struct** instances is **public**, and the default member access for **class** instances is **private**.

The **struct** feature is, in some sense, redundant. It is a carryover from the C programming language. By retaining the **struct** keyword, however, C++ programs can use C libraries that use C **structs**. Any C++ program that expects to utilize a C library using a **struct** must restrict its **struct** definitions to the limited form supported by C. Such **struct** definitions may not contain non-**public** members, methods, constructors, etc.

15.9 Summary

1. While the dot (.) operator is used to access a member of an object, the arrow (->) operator is used to access a member of an object via a pointer.
2. A method declared **const** may not modify the value of an instance variable within an object. The compiler enforces this restriction.
3. The class interface includes instance variables declarations and method prototypes. The method implementations can appear in a separate file.
4. Method implementations can appear in class declarations so that a class interface and implementation is combined in one place.
5. A non-**const** method may not be called with a **const** object. Any public method can be called with a non-**const** object.
6. Operators may be overloaded for programmer-defined types. Overloaded operators may be global functions and methods.
7. A programmer-defined operator obeys the same precedence and associativity rules as the built-in operator it overloads.
8. Unlike an instance variable, a **static** field is shared by all objects of their class.
9. Unlike instance variables, a **static** field must be declared outside of the class declaration.
10. A **static** method executes on behalf of the class, not an instance of the class.
11. A **static** method may not access any instance variables of the class, nor may they call other non-**static** methods.
12. Any class method, **static** or non-**static**, may access a **static** data member or call a **static** method.

15.10 Exercises

1. Exercise

Chapter 16

Building some Useful Classes

This chapter uses the concepts from the past few chapters to build some complete, practical classes.

16.1 A Better Rational Number Class

Listing 16.1 (rational.cpp) enhances the **SimpleRational** class (Listing 14.5 (simplerational.cpp)) providing a more complete type.

Listing 16.1: rational.cpp

```
#include <iostream>

using namespace std;

class Rational {
    int numerator;
    int denominator;

    // Compute the greatest common divisor (GCD) of two integers
    static int gcd(int m, int n) {
        if (n == 0)
            return m;
        else
            return gcd(n, m % n);
    }

    // Compute the least common multiple (LCM) of two integers
    static int lcm(int m, int n) {
        return m * n / gcd(m, n);
    }

public:
    Rational(int n, int d): numerator(n), denominator(d) {
        if (d == 0) { // Disallow an undefined fraction
            cout << "*****Warning---Illegal Rational" << endl;
            numerator = 0; // Make up a reasonable default fraction
            denominator = 1;
        }
    }
};
```



```

    }
}

// Default fraction is 0/1
Rational(): numerator(0), denominator(1) {}

int get_numerator() const {
    return numerator;
}

int get_denominator() const {
    return denominator;
}

Rational reduce() const {
    // Find the factor that numerator and denominator have in common . . .
    int factor = gcd(numerator, denominator);
    // . . . then divide it out in the new fraction
    return Rational(numerator/factor, denominator/factor);
}

// Equal fractions have identical numerators and denominators
bool operator==(const Rational& fract) const {
    // First, find the reduced form of this fraction and the parameter . . .
    Rational f1 = reduce(),
            f2 = fract.reduce();
    // . . . then see if their components match.
    return (f1.numerator == f2.numerator)
        && (f1.denominator == f2.denominator);
}

// Compute the sum of fract and the current rational number
Rational operator+(const Rational& fract) const {
    // Find common denominator
    int commonDenominator = lcm(denominator, fract.denominator);
    // Add the adjusted numerators
    int newNumerator = numerator * commonDenominator/denominator
        + fract.numerator * commonDenominator/fract.denominator;
    return Rational(newNumerator, commonDenominator);
}

// Compute the product of fract and the current rational number
Rational operator*(const Rational& fract) const {
    return Rational(numerator * fract.numerator,
        denominator * fract.denominator).reduce();
}
};

// Allow a Rational object to be displayed in a nice
// human-readable form.
ostream& operator<<(ostream& os, const Rational& r) {
    os << r.get_numerator() << "/" << r.get_denominator();
    return os;
}

```

```
int main() {
    Rational f1(1, 2), f2(1, 3);
    cout << f1 << " + " << f2 << " = " << (f1 + f2) << endl;
    cout << f1 << " * " << f2 << " = " << (f1 * f2) << endl;
}
```

Listing 16.1 (rational.cpp) produces

```
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
```

This rational number type has some notable features:

- **Constructors.** The overloaded constructors permit convenient initialization and make it impossible to create an undefined fraction.
- **Private static methods.** **Rational** provides two **private static** methods: **gcd** and **lcm**. The algorithm for the recursive **gcd** (greatest common divisor) method was introduced in Section 10.4. The **lcm** (least common multiple) method is derived from the mathematical relationship:

$$\text{gcd}(m,n) \times \text{lcm}(m,n) = m \times n$$

These two methods are declared **private** because they are not meant to be used directly by client code. *Greatest common divisor* and *least common multiple* are concepts from number theory of which **Rational** clients have no direct need. Client code expects functionality typical of rational numbers, such as addition and reduction; these two private methods are used by other, public, methods that provide functionality more closely related to rational numbers. These two private methods are **static** methods because they do not use instance variables. An object is not required to compute the greatest common divisor of two integers. It is legal for **gcd** and **lcm** to be instance methods, but instance methods should be used only where necessary, since they have the power to alter the state of an object. Faulty coding that accidentally modifies an instance variable can be difficult to track down. If a class method is used, however, the compiler can spot any attempt to access an instance variable immediately.

- **Public instance methods.** None of the instance methods (**operator==**, **reduce**, **operator+**, and **operator***) modify the state of the object upon which they are invoked. Thus, the **Rational** class still produces immutable objects. The methods **operator+**, **operator***, and **reduce** use the **private** helper methods to accomplish their respective tasks.
- **Global << operator.** **operator<<** is a global function that allows a **Rational** object to be sent to the **cout** object to be displayed as conveniently as a built-in type.

16.2 Stopwatch

The linear search vs. binary search comparison program (Listing 12.5 (searchcompare.cpp)) accessed the system clock in order to time the execution of a section of code. The program used the **clock** function from the standard C library and an additional variable to compute the elapsed time. The following skeleton code fragment:

```

clock_t seconds = clock();    // Record starting time

/*
 * Do something here that you wish to time
 */

clock_t other = clock();      // Record ending time
cout << static_cast<double>(other - seconds)/CLOCKS_PER_SEC
     << " seconds" << endl;

```

certainly works and can be adapted to any program, but it has several drawbacks:

- A programmer must take care to implement the timing code correctly for each section of code to be timed. This process is error prone:

- `clock_t` is a specialized type that is used infrequently. It is not obvious from its name that `clock_t` is equivalent to an unsigned integer, so a programmer may need to consult a library reference to ensure its proper use.
- The programmer must specify the correct arithmetic:

```
(other - seconds)/CLOCKS_PER_SEC
```

- The type cast to `double` of the time difference is necessary but easily forgotten or applied incorrectly. If a programmer incorrectly applies parentheses as so

```
static_cast<double>((other - seconds)/CLOCKS_PER_SEC)
```

the result will lose precision. Worse yet, the following parenthetical grouping

```
static_cast<double>(other) - seconds/CLOCKS_PER_SEC
```

will result in an incorrect value, since division has precedence over subtraction.

- The timing code is supplemental to the actual code that is being profiled, but it may not be immediately obvious by looking at the complete code which statements are part of the timing code and which statements are part of the code to be timed.

Section 10.5 offered a solution to the above shortcomings of using the raw types, constants, and functions available from the C time library. Listing 10.13 (timermodule.cpp) hides the details of the C time library and provides a convenient functional interface to callers. Unfortunately, as mentioned in Section 10.5, the functional approach has a serious limitation. The code in Listing 10.13 (timermodule.cpp) uses global variables to maintain the state of the timer. There is only one copy of each global variable. This means programmers using the timer functions cannot independently measure the elapsed time of overlapping events; for example, you cannot measure how long it takes for a function to execute and simultaneously measure how long a section of code within that function takes to execute.

A programmer could time multiple, simultaneous activities by using the raw C library `clock` function directly, but then we are back to where we began: messy, potentially error-prone code.

Objects provide a solution. Consider the following client code that uses a stopwatch object to keep track of the time:

```

Stopwatch timer;    // Declare a stopwatch object

timer.start();      // Start timing

```

```

/*
 * Do something here that you wish to time
 */

timer.stop();    // Stop the clock
cout << timer.elapsed() << " seconds" << endl;

```

This code using a **Stopwatch** object is as simple as the code that uses the timer functions from Listing 10.13 (timermodule.cpp). As an added benefit, a developer can think of a **Stopwatch** object as if it is a real physical stopwatch object: push a button to start the clock (call the **start** method), push a button to stop the clock (call the **stop** method), and then read the elapsed time (use the result of the **elapsed** method). What do you do if you need to time two different things at once? You use two stopwatches, of course, so a programmer would declare and use two **Stopwatch** objects. Since each object maintains its own instance variables, each **Stopwatch** object can keep track of its own elapsed time independently of all other active **Stopwatch** objects.

Programmers using a **Stopwatch** object in their code are much less likely to make a mistake because the details that make it work are hidden and inaccessible. With objects we can wrap all the messy details of the timing code into a convenient package. Given our experience designing our own types through C++ classes, we now are adequately equipped to implement such a **Stopwatch** class. 16.2 provides the header file defining the structure and capabilities of our **Stopwatch** objects.

Listing 16.2: stopwatch.h

```

#ifndef _STOPWATCH_H_DEFINED_
#define _STOPWATCH_H_DEFINED_

#include <ctime>

class Stopwatch {
    clock_t start_time;
    bool running;
    double elapsed_time;
public:
    Stopwatch();
    void start();           // Start the timer
    void stop();            // Stop the timer
    void reset();           // Reset the timer
    double elapsed() const; // Reveal the elapsed time
    bool is_running() const; // Is the stopwatch currently running?
};

#endif

```

From this class declaration we see that when clients create a **Stopwatch** object a constructor is available to take care of any initialization details. Four methods are available to clients: **start**, **stop**, **reset**, and **elapsed**. The **reset** method is included to set the clock back to zero to begin a new timing. Note that the “messy” detail of the **clock_t** variable is private and, therefore, clients cannot see or directly affect its value within a **Stopwatch** object.

This **Stopwatch** class (Listing 16.2 (stopwatch.h)) addresses the weaknesses of the non-object-oriented approach noted above:

- The timing code can be implemented in methods of the **Stopwatch** class. Once the methods are correct, a programmer can use **Stopwatch** objects for timing the execution of sections of code without worrying about the details of how the timing is actually done. Client code cannot introduce errors in the timing code if the timing code is hidden within the **Stopwatch** class.
- The details of the timing code no longer intertwine with the code to be timed, since the timing code is located in the **Stopwatch** class. This makes it easier for programmers to maintain the code they are timing.
- The **Stopwatch** class provides a convenient interface for the programmer that replaces the lower-level details of calling system time functions.

Listing 16.3 (stopwatch.cpp) provides the **Stopwatch** implementation.

Listing 16.3: stopwatch.cpp

```
#include <iostream>
#include "Stopwatch.h"

using namespace std;

// Creates a Stopwatch object
// A newly minted object is not running and is in a "reset" state
Stopwatch::Stopwatch(): start_time(0), running(false), elapsed_time(0.0) {}

// Starts the stopwatch to begin measuring elapsed time.
// Starting a stopwatch that already is running has no effect.
void Stopwatch::start() {
    if (!running) {
        running = true;           // Set the clock running
        start_time = clock();      // Record start time
    }
}

// Stops the stopwatch. The stopwatch will retain the
// current elapsed time, but it will not measure any time
// while it is stopped.
// If the stopwatch is already stopped, the method has
// no effect.
void Stopwatch::stop() {
    if (running) {
        clock_t stop_time = clock(); // Record stop time
        running = false;
        // Accumulate elapsed time since start
        elapsed_time += static_cast<double>((stop_time - start_time))
                        /CLOCKS_PER_SEC;
    }
}

// Reports the cumulative time in seconds since the
// stopwatch was last reset.
// This method does not affect the state of the stopwatch.
double Stopwatch::elapsed() const {
    if (running) { // Compute time since last reset
        clock_t current_time = clock(); // Record current time
```

```

        // Add time from previous elapsed to the current elapsed
        // since the latest call to the start method.
        return elapsed_time
            + static_cast<double>((current_time - start_time))
              /CLOCKS_PER_SEC;
    }
    else // Timer stopped; elapsed already computed in the stop method
        return elapsed_time;
}

// Returns the stopwatch's status (running or not) to the client.
// This method does not affect the state of the stopwatch.
bool Stopwatch::is_running() const {
    return running;
}

// Resets the stopwatch so a subsequent start begins recording
// a new time. Stops the stopwatch if it currently is running.
void Stopwatch::reset() {
    running = false;
    elapsed_time = 0.0;
}

```

Note that our design allows a client to see the running time of a **Stopwatch** object without needing to stop it. An alternate design might print an error message and perhaps exit the program's execution if a client attempts to see the elapsed time of a running stopwatch.

Some aspects of the **Stopwatch** class are notable:

- **Stopwatch** objects use three instance variables:
 - The **start_time** instance variable records the time when the client last called the **start** method.
 - The **elapsed_time** instance variable keeps track of the time since the latest call to the **reset** method.
 - The **running** Boolean instance variable indicates whether or not the clock is running.
 - The constructor sets the initial values of the instance variables **start_time**, **elapsed_time**, and **running**.
 - The **start** method notes the system time *after* the assignment to **running**. If these two statements were reversed, the elapsed time would include the time to do the assignment to **running**. The elapsed time should as closely as possible just include the statements in the client code between the **start** and **stop** method calls.
- Notice that **start_time** is not assigned if it currently is not zero. This allows a client to stop the stopwatch and restart it later without losing an earlier segment of time.
- In the **stop** method, the system time is noted *before* the assignment to **running** so the elapsed time does not include the assignment to **running**. This provides a more accurate accounting of the client code execution time.

The **stop** method computes the accumulated elapsed time.

- The **elapsed** method either returns the elapsed time computed by the **stop** method or computes the current running time without altering the **elapsed_time** variable. Clients should avoid calling **elapsed** when a **Stopwatch** object is running since doing so would interfere with the accurate timing of client code execution.

Compare the **main** function of Listing 12.5 (searchcompare.cpp) to that of Listing 16.4 (bettersearchcompare.cpp):

Listing 16.4: bettersearchcompare.cpp

```
#include <iostream>
#include <iomanip>
#include <ctime>
#include <vector>
#include "Stopwatch.h"

using namespace std;

/*
 *  binary_search(v, seek)
 *  Returns the index of element seek in vector v;
 *  returns -1 if seek is not an element of v
 *  v is the vector to search; v's contents must be
 *  sorted in ascending order.
 *  seek is the element to find
 */
int binary_search(const vector<int>& v, int seek) {
    int first = 0,           // Initially the first element in vector
        last = v.size() - 1, // Initially the last element in vector
        mid;                // The middle of the vector
    while (first <= last) {
        mid = first + (last - first + 1)/2;
        if (v[mid] == seek)
            return mid;      // Found it
        else if (v[mid] > seek)
            last = mid - 1;   // continue with 1st half
        else // v[mid] < seek
            first = mid + 1;  // continue with 2nd half
    }
    return -1;              // Not there
}

// This version requires vector v to be sorted in
// ascending order.
/*
 *  linear_search(v, seek)
 *  Returns the index of element seek in vector v;
 *  returns -1 if seek is not an element of a
 *  v is the vector to search; v's contents must be
 *  sorted in ascending order.
 *  seek is the element to find
 */
int linear_search(const vector<int>& v, int seek) {
    size_t n = v.size();
    for (size_t i = 0; i < n && v[i] <= seek; i++)
        if (v[i] == seek)
```

```

        return i;    // Return position immediately
    return -1;    // Element not found
}

int main() {
    const size_t SIZE = 30000;
    vector<int> list(SIZE);

    Stopwatch timer;

    // Ensure the elements are ordered low to high
    for (size_t i = 0; i < SIZE; i++)
        list[i] = i;
    // Search for all the elements in list using linear search
    timer.start();
    for (size_t i = 0; i < SIZE; i++)
        linear_search(list, i);
    // Print the elapsed time
    timer.stop();
    cout << "Linear elapsed: " << timer.elapsed() << " seconds" << endl;
    // Prepare for a new timing
    timer.reset();
    // Search for all the elements in list using binary search
    timer.start();
    for (size_t i = 0; i < SIZE; i++)
        binary_search(list, i);
    // Print the elapsed time
    timer.stop();
    cout << "Binary elapsed: " << timer.elapsed() << " seconds" << endl;
}

```

This new, object-oriented version is simpler and more readable.

The design of the **Stopwatch** class allows clients to create multiple **Stopwatch** instances, and each instance will keep track of its own time. In practice when profiling executing programs, such generality usually is unnecessary. Rarely do developers need to time overlapping code, so one timer object per program usually is enough. Multiple sections of code can be checked with the same **Stopwatch** object; simply start it, stop it, check the time, and then reset it and start it again when another section of code is to be timed.

16.3 Sorting with Logging

Section 12.2 shows how to use function pointers to customize the ordering that selection sort performs on an array of integers. The **selection_sort** function in Listing 12.2 (**flexibleintsrt.cpp**) accepts a function pointer parameter in addition to the array and the array's size. The function pointer points to a function that accepts two integer parameters and returns true or false. The function is supposed to use some kind of ordering rule to determine if its first integer parameter precedes its second integer parameter.

Suppose we wish to analyze the number of comparisons and the number of swaps the sort function performs on a given array with a particular ordering strategy. One way to do this is have the sort function itself keep track of the number of times it calls the comparison function and **swap** function and return this information when it finishes. To do so we would have to define an object to hold the two pieces of data

(comparisons and swaps) since a function can return only one value, not two. Also if we do this, we must significantly alter the code of the sort algorithm itself. We would prefer to keep the sort algorithm focused on its task of sorting and remain uncluttered from this additional logging code.

If instead of passing a function pointer to our sort function we pass a specially crafted object. We can design our object to do whatever we want; specifically, we can design our special object to perform the necessary comparisons and keep track of how many comparisons it performs. We could let the object do the swap, and it could log the swaps it performs.

Listing 16.5 (`loggingflexiblesort.cpp`) is a variation of Listing 12.2 (`flexibleintsort.cpp`) that uses a comparison *object* instead of a comparison *function*.

Listing 16.5: `loggingflexiblesort.cpp`

```
#include <iostream>
#include <vector>

using namespace std;

/*
 * Comparer objects manage the comparisons and element
 * interchanges on the selection sort function below.
 */
class Comparer {
    // Keeps track of the number of comparisons
    // performed
    int compare_count;
    // Keeps track of the number of swaps performed
    int swap_count;
    // Function pointer directed to the function to
    // perform the comparison
    bool (*comp)(int, int);
public:
    // The client must initialize a Comparer object with a
    // suitable comparison function.
    Comparer(bool (*f)(int, int)):
        compare_count(0), swap_count(0), comp(f) {}

    // Resets the counters to make ready for a new sort
    void reset() {
        compare_count = swap_count = 0;
    }

    // Method that performs the comparison. It delegates
    // the actual work to the function pointed to by comp.
    // This method logs each invocation.
    bool compare(int m, int n) {
        compare_count++;
        return comp(m, n);
    }

    // Method that performs the swap.
    // Interchange the values of
    // its parameters a and b which are
    // passed by reference.
    // This method logs each invocation.
```

```

    void swap(int& m, int& n) {
        swap_count++;
        int temp = m;
        m = n;
        n = temp;
    }

    // Returns the number of comparisons this object has
    // performed since it was created.
    int comparisons() const {
        return compare_count;
    }

    // Returns the number of swaps this object has
    // performed since it was created.
    int swaps() const {
        return swap_count;
    }
};

/*
 *  selection_sort(a, compare)
 *      Arranges the elements of vector a in an order determined
 *      by the compare object.
 *      a is a vector of ints.
 *      compare is a function that compares the ordering of
 *      two integers.
 *      The contents of a are physically rearranged.
 */
void selection_sort(vector<int>& a, Comparer& compare) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Note: i, small, and j represent positions within a
        // a[i], a[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        int small = i;
        // See if a smaller value can be found later in the array
        for (int j = i + 1; j < n; j++)
            if (compare.compare(a[j], a[small]))
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            compare.swap(a[i], a[small]);
    }
}

/*
 *  print
 *      Prints the contents of an integer vector
 *      a is the vector to print.
 *      a is not modified.
 */

```

```

void print(const vector<int>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << '}';
}

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
bool less_than(int a, int b) {
    return a < b;
}

/*
 * greater_than(a, b)
 * Returns true if a > b; otherwise, returns
 * false.
 */
bool greater_than(int a, int b) {
    return a > b;
}

int main() {
    // Make a vector of integers from an array
    vector<int> original { 23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10 };

    // Make a working copy of the original vector
    vector<int> working = original;
    cout << "Before: ";
    print(working);
    cout << endl;
    Comparator lt(less_than), gt(greater_than);
    selection_sort(working, lt);
    cout << "Ascending: ";
    print(working);
    cout << " (" << lt.comparisons() << " comparisons, "
        << lt.swaps() << " swaps)" << endl;
    cout << "-----" << endl;
    // Make another copy of the original vector
    working = original;
    cout << "Before: ";
    print(working);
    cout << endl;
    selection_sort(working, gt);
    cout << "Descending: ";
    print(working);
    cout << " (" << gt.comparisons() << " comparisons, "
        << gt.swaps() << " swaps)" << endl;
}

```

```

cout << "-----" << endl;
// Sort a sorted vector
cout << "Before: ";
print(working);
cout << endl;
// Reset the greater than comparer so we start counting at
// zero
gt.reset();
selection_sort(working, gt);
cout << "Descending: ";
print(working);
cout << " (" << gt.comparisons() << " comparisons, "
    << gt.swaps() << " swaps)" << endl;
}

```

Notice that a **Comparison** object wraps a comparison function pointer, contains a **swap** method, and maintains two integer counters. The comparison object passed to the sort routine customizes the sort's behavior (via its function pointer) and keeps track of the number of comparisons and swaps it performs (via its integer counters). As in Listing 12.2 (flexibleintsor.cpp), the basic structure of the sorting algorithm remains the same regardless of the ordering determined by the comparison object.

The output of Listing 16.5 (loggingflexiblesort.cpp) is

```

Before: {23,-3,4,215,0,-3,2,23,100,88,-10}
Ascending: {-10,-3,-3,0,2,4,23,23,88,100,215} (55 comparisons, 7 swaps)
-----
Before: {23,-3,4,215,0,-3,2,23,100,88,-10}
Descending: {215,100,88,23,23,4,2,0,-3,-3,-10} (55 comparisons, 5 swaps)
-----
Before: {215,100,88,23,23,4,2,0,-3,-3,-10}
Descending: {215,100,88,23,23,4,2,0,-3,-3,-10} (55 comparisons, 0 swaps)

```

We see from the results that the number of comparisons is dictated by the algorithm itself, but the number of element swaps depends on the ordering of the elements and the nature of the comparison. Sorting an already sorted array with selection sort does not reduce the number of comparisons the function must perform, but, as we can see, it requires no swaps.

16.4 Linked Lists

An object in C++ can hold just about any type of data, but there are some limitations. Consider the following **struct** definition:

```

struct Node {
    int data;
    Node next;    // Error, illegal self reference
};

```

(Here we use a **struct** instead of a **class** since we will consider a **Node** object a primitive data type that requires no special protection from clients.) How much space should the compiler set aside for a **Node**

object? A **Node** contains an integer and a **Node**, but this contained **Node** field itself would contain an integer and a **Node**, and the nested containment would go on forever. Such a structure understandably is illegal in C++, and the compiler will issue an error. You are not allowed to have a **class** or **struct** field of the same type within the **class** or **struct** being defined.

Another object definition looks similar, but it is a legal structure:

```
struct Node {
    int data;
    Node *next;    // Self reference via pointer is legal
};
```

The reason this second version is legal is because the compiler now can compute the size of a **Node** object. A pointer is simply a memory address under the hood, so all pointer variables are the same size regardless of their declared type. The pointer solves the infinitely nested containment problem.

This ability of an object to refer to an object like itself is not merely an interesting curiosity; it has practical applications. Suppose we wish to implement a sequence structure like a vector. We can use the self-referential structure defined above to build a list of **Node** objects linked together via pointers. The following declaration

```
Node *n1, *n2, *n2, *n4;
```

specifies four node pointers that will reference elements in our list. We can build a list containing the elements 23, 10, 0, and 3 as follows

```
// Pointers to node objects
Node *n1, *n2, *n2, *n4;
// Create the list nodes
n1 = new Node;
n2 = new Node;
n3 = new Node;
n4 = new Node;
// Assign the data fields
n1->data = 23;
n2->data = 10;
n3->data = 0;
n4->data = 3;
// Link the nodes together
n1->next = n2;
n2->next = n3;
n3->next = n4;
n4->next = nullptr;
```

The last assignment of the null pointer (**nullptr**) indicates that node **n4** has no other nodes following it. This sequence of code produces the null-terminated linked list illustrated in Figure 16.1. In Figure 16.1 the × symbol within the **next** pointer field represents the null pointer.

The hard-coded list in Figure 16.1 does not exploit the dynamic nature of linked lists. Armed with our knowledge of C++ classes, encapsulation, and methods, we can build a client-friendly, dynamic linked list type.

The code found in Listing 16.6 (intlist.h), Listing 16.7 (intlist.cpp), and Listing 16.8 (listmain.cpp) demonstrates the power of linked lists. Listing 16.6 (intlist.h) is the header file for a simple integer linked list class.

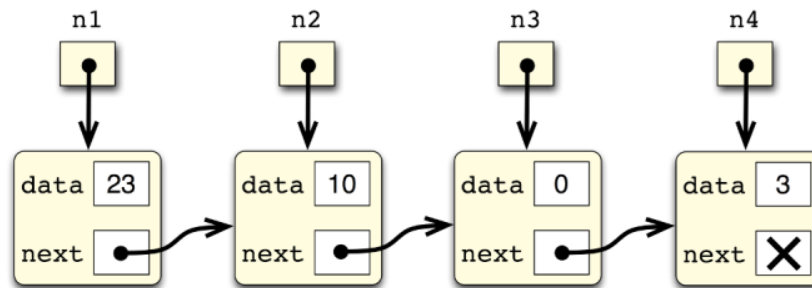


Figure 16.1: A null-terminated linked list

Listing 16.6: intlist.h

```

class IntList {
    /*
     * An object that holds an element in a linked list.
     * This type is local to the IntList class and inaccessible
     * to outside this class.
     */
    struct Node {
        int data;          // A data element of the list
        Node *next;        // The node that follows this one in the list
        Node(int d);       // Constructor
    };

    Node *head;           // Points to the first item in the list

    // Private helper methods

    /*
     * insert(p, n)
     * Inserts n onto the back of the
     * list pointed to by p.
     * p points to a node in a linked list
     * n is the element to insert.
     * Returns a pointer to the list that
     * contains n.
     */
    Node *insert(Node *p, int n);

    /*
     * print(p)
     * Prints the contents of a linked list of integers.
     * p points to a linked list of nodes.
     */
}

```

```

void print(Node *p) const;

/*
 * Returns the length of the linked list pointed to by p.
 */
int length(Node *p) const;

/*
 * dispose(p)
 * Deallocate the memory held by the list pointed to by p.
 */
void dispose(Node *p);

public:
/*
 * The constructor makes an initially empty list
 */
IntList();

/*
 * The destructor frees up the memory held by the list
 */
~IntList();

/*
 * insert(n)
 * Inserts n onto the back of the list.
 * n is the element to insert.
 */
void insert(int n);

/*
 * print()
 * Prints the contents of the linked list of integers.
 */
void print() const;

/*
 * Returns the length of the linked list.
 */
int length() const;

/*
 * clear()
 * Removes all the elements in the linked list.
 */
void clear();
};

```

The **Node** class is declared within the **IntList** class. We say that **Node** is a *nested class*. Since the declaration of **Node** appears in the private section of **IntList**, **Node** is a type known only to code within the **IntList** class. The complete name of the **Node** type is **IntList::Node**.

Notice that the **IntList** class has several private methods in addition to its public methods. Code outside the class cannot execute these private methods directly. These private methods are helper methods

that the public methods invoke to accomplish their tasks. We say that a public method *delegates* the work to its private helper methods. Why is this delegation necessary here? The private methods use recursion that requires a parameter of type `IntList::Node` which is unknown outside the `IntList` class. A client is therefore unable to use the private methods directly, even if they were made public. The public methods do not expose to the client any details about the class's implementation; specifically they keep the `IntList::Node` type and the `head` instance variable hidden from clients.

Observe that the overloaded `print` and `length` methods (both private and public) of the `IntList` class are declared `const`. Neither printing a list nor requesting its length should modify an `IntList` object. Clients can, therefore, use the `print` and `length` methods with a constant `IntList` object. An attempt to use `insert` or `clear` on a constant `IntList` object will yield a compiler error. The error here makes sense because `insert` definitely will modify a list object, and `clear` potentially will modify a list object (`clear` will not modify an empty list).

The `insert` method of the `IntList` class will use `new` to dynamically allocate `Node` objects when adding elements to the collection. The `IntList` class destructor:

```
~IntList();
```

will be responsible for deallocating the list nodes when an `IntList` object goes out of scope. The destructor simply will call the `clear` method to clean up the memory held by the list.

Why not just let the client call `clear` directly to free up the list without depending on the destructor? A client may forget to call `clear` when finished with a list object. The resulting memory leak could cause a long-running program to run out of memory. With the destructor, the client programmer needs not worry about deallocating the list since the destructor does it automatically; for example, consider the following code:

```
void f() {
    IntList my_list;    // Constructor called here
    // Add some numbers to the list
    my_list.insert(22);
    my_list.insert(5);
    my_list.insert(-44);
    // Print the list
    my_list.print();
} // Destructor for my_list called when the function is finished
```

The variable `my_list` is local to function `f`. When function `f` finishes executing the variable `my_list` goes out of scope. At this point the `IntList` destructor executes on behalf of `my_list`. The destructor ensures that `my_list`'s dynamically allocated memory is released. In the context of a pointer to `IntList`, as in

```
void f2() {
    IntList *lptr;      // Pointer, constructor NOT called yet
    lptr = new IntList; // Constructor called here
    // Add some numbers to the list
    lptr->insert(22);
    lptr->insert(5);
    lptr->insert(-44);
    // Print the list
    lptr->print();
    delete lptr;        // Destructor called here
}
```


`lptr` is not an object; it is a pointer to an object. Declaring `lptr` does not create an object, and, therefore, the constructor does not execute. The statement

```
lptr = new IntList;
```

does create an object, and so the `IntList` constructor executes on behalf of the object to which `lptr` points. When `lptr` goes out of scope at the end of function `f2`'s execution, the stack variable `lptr` goes away, but since it is a pointer, not an object, no destructor code executes. The client must explicitly free up memory with `delete`:

```
delete lptr;           // Destructor called here
```

unless the programmer intends for the function to return the pointer to the client (in which case the client is responsible for calling `delete` when finished with the object).

Listing 16.7 (`intlist.cpp`) implements the methods declared in 16.6.

Listing 16.7: `intlist.cpp`

```
// intlist.cpp

#include "intlist.h"
#include <iostream>

// Private IntList operations

/*
 * Node constructor
 */
IntList::Node::Node(int n): data(n), next(nullptr) {}

/*
 * insert(p, n)
 * Inserts n onto the back of the
 * list pointed to by p.
 * p points to a node in a linked list
 * n is the element to insert.
 * Returns a pointer to the list that
 * contains n.
 */
IntList::Node *IntList::insert(IntList::Node *p, int n) {
    if (p) // Is p non-null?
        p->next = insert(p->next, n);
    else // p is null, make a new node
        p = new IntList::Node(n);
    return p;
}

/*
 * Returns the length of the linked list pointed to by p.
 */
int IntList::length(IntList::Node *p) const {
    if (p)
        return 1 + length(p->next); // 1 + length of rest of list
    else
        return 0; // Empty list has length zero
}
```

```

}

/*
 * print(p)
 * Prints the contents of a linked list of integers.
 * p points to a linked list of nodes.
 */
void IntList::print(IntList::Node *p) const {
    while (p) { // While p is not null
        std::cout << p->data << " "; // Print current node's data
        p = p->next; // Move to next node
    }
    std::cout << std::endl;
}

/*
 * dispose(p)
 * Deallocate the memory held by the list pointed to by p.
 */
void IntList::dispose(IntList::Node *p) {
    if (p) {
        dispose(p->next); // Free up the rest of the list
        delete p; // Deallocate this node
    }
}

// Public IntList operations

/*
 * The constructor makes an initially empty list.
 * The list is empty when head is null.
 */
IntList::IntList(): head(nullptr) {}

/*
 * The destructor frees up the memory held by the list
 */
IntList::~IntList() {
    clear(); // Deallocate space held by the list nodes
}

/*
 * insert(n)
 * Inserts n onto the back of the list.
 * n is the element to insert.
 */
void IntList::insert(int n) {
    head = insert(head, n); // Delegate work to private helper method
}

/*
 * print()
 * Prints the contents of the linked list of integers.
 */
void IntList::print() const {

```

```

    print(head); // Delegate work to private helper method
}

/*
 * Returns the length of the linked list.
 */
int IntList::length() const {
    return length(head); // Delegate work to private helper method
}

/*
 * clear()
 * Removes all the elements in the linked list.
 */
void IntList::clear() {
    dispose(head); // Deallocate space for all the nodes
    head = nullptr; // Null head signifies list is empty
}

```

Since the code in Listing 16.7 (intlist.cpp) appears outside of the class declaration, any use of the **Node** type requires its full name: **IntList::Node**. Note the **::** use in the **Node** constructor:

```
IntList::Node::Node(int n): data(n), next(nullptr) {}
```

Three private methods in Listing 16.7 (intlist.cpp) (**insert**, **length**, and **dispose**) are recursive. The **print** method could be recursive, but for variety it uses iteration.

Listing 16.8 (listmain.cpp) provides some sample client code that exercises a linked list.

Listing 16.8: listmain.cpp

```

// list_main.cpp

#include "intlist.h"
#include <iostream>

using namespace std;

int main() {
    bool done = false;
    char command;
    int value;
    IntList list;

    while (!done) {
        cout << "I)nsert <item> P)rint L)ength E)rase Q)uit >>";
        cin >> command;
        switch (command) {
            case 'I': // Insert a new element into the list
            case 'i':
                if (cin >> value)
                    list.insert(value);
                else
                    done = true;
                break;
            case 'P': // Print the contents of the list

```

```

        case 'p':
            list.print();
            break;
        case 'L': // Print the list's length
        case 'l':
            cout << "Number of elements: " << list.length() << endl;
            break;
        case 'E': // Erase the list
        case 'e':
            list.clear();
            break;
        case 'Q': // Exit the loop (and the program)
        case 'q':
            done = true;
            break;
    }
}
}

```

The client code in Listing 16.8 (listmain.cpp) allows a user to interactively add items to a list, print the list, determine the list's size, and clear the list. Observe that the client code does not use pointers at all. All the pointer manipulations are hidden within the **IntList** class. The pointer data and code and the **Node** struct itself is private within **IntList**, out of the reach of programmers who use these linked lists. Pointer programming can be tricky, and it is easy to introduce subtle, hard to find bugs; thus, encapsulation once again results in easier and more robust application development.

As shown in the **IntList** methods, a pointer variable by itself can be used as a condition within a conditional statement or loop. A null pointer is interpreted as **false**, and any non-null pointer is **true**. This means if **p** is a pointer, the statement

```

if (p)
    /* Do something */

```

is a shorthand for

```

if (p != nullptr)
    /* Do something */

```

and the statement

```

if (!p)
    /* Do something */

```

is a shorthand for

```

if (p == nullptr)
    /* Do something */

```

Most C and C++ programmers use the shorter syntax.

In order to better understand how the recursive methods work, think about the structure of a **Node** object with this interpretation: A **Node** object holds a data item and a pointer to rest of the list that follows. A pointer to a **Node** is either null or non-null. A null pointer represents the empty list. A non-null pointer represents a non-empty list consisting of two parts: the first element in the list and the rest of the list. If the **next** field of a **Node** object is null, the rest of the list is empty.

Armed with this view of lists we can now examine the behavior of the linked list methods in more detail:

- **insert**. To understand how the recursive private **insert** method works we must *think recursively*. A list is either empty or non-empty. Inserting a new item onto a list is simple:

- If the list is empty, make a new list with one element in it. The single element in the resulting list is the item you want to insert. Symbolically (not in C++), we can let $\emptyset$ stand for the empty list and $x \rightarrow \emptyset$ stand for the list containing just the element x . We therefore can represent the insertion into an empty list as:

$$\text{insert}(\emptyset, x) = x \rightarrow \emptyset;$$

- If the list is not empty, merely **insert** the new item into the *rest of the list*. Notice the recursive description. Symbolically, we can express this as

$$\text{insert}(x \rightarrow \text{rest-of-list}, a) = x \rightarrow \text{insert}(\text{rest-of-list}, a)$$

To see how **insert** works, we will build a list with three elements. First we will insert 1. The **list** variable in **main** is initialized to null, so the first time **main** calls **insert** it passes **nullptr** as the first parameter, and 1 as the second parameter. Let $\emptyset$ represent the empty list, so we are in effect making the call

```
list = insert( $\emptyset$ , 1);
```

Within **insert**, since the parameter **p** is null, the condition

```
if (p) // Is p non-null?
```

is not met, so the statements within the **else** body are executed. Those statements reassign **p** to point to a newly created node holding 1. The node's **next** pointer is null, so **p** points to a list holding just one element. We can represent this list as $1 \rightarrow \emptyset$. **p** is returned, and **main** assigns the result to **list**, so **list** now points to a linked list containing the single element 1. **list** now is the list $1 \rightarrow \emptyset$. Observe that this first call to **insert** results in no recursive call.

We can symbolize this insertion of 1 into an empty list as

$$\text{list} = \text{insert}(\emptyset, 1) = 1 \rightarrow \emptyset$$

The second call to **insert**, which produces a list holding two elements, is more interesting. This time we will insert 2 into our list. Again, **list** is passed as the first parameter and 2 is passed as the second, but this time **list** is non-null. Within the method, the condition

```
if (p) // Is p non-null?
```

is satisfied, so the recursive call takes place:

```
p->next = insert(p->next, n);
```

Note that **p->next** is assigned, not **p** itself. The **p->next** field represents the *rest of the list*, so we now have a new problem to solve: insert 2 into the remainder of the list. At this point the rest of the list is empty so the recursive call simply creates a new list containing 2 (that is, $2 \rightarrow \emptyset$) and returns it to be assigned to **p->next**. **p** itself is not reassigned, so it is pointing to the same node as before, but that node's **next** field was reassigned to point $2 \rightarrow \emptyset$. **p** thus becomes $1 \rightarrow 2 \rightarrow \emptyset$ and is returned and assigned to **list** in **main**. **list** is now $1 \rightarrow 2 \rightarrow \emptyset$.

We can symbolize this insertion of 2 into the list $1 \rightarrow \emptyset$ as

$$\text{list} = \text{insert}(1 \rightarrow \emptyset, 2) = 1 \rightarrow \text{insert}(\emptyset, 2) = 1 \rightarrow 2 \rightarrow \emptyset$$

Finally, we will insert 3 into **list**. The process unfolds as follows:

```
list = insert(1→2→∅, 3)
      = 1→insert(2→∅, 3)
      = 1→2→insert(∅, 3)
      = 1→2→3→∅
```

- **length**. Lists are either empty or non-empty. The length of the empty list is zero. The length of a non-empty list is at least one because a non-empty list contains at least one element. Writing a recursive list **length** method is as simple as this:

- If the list is empty, its length is zero. Symbolically, we can write **length**($\emptyset$) = 0.
- If the list is non-empty, its length is one (counting its first element) plus the **length** of the rest of the list.

This English description translates directly into C++ code.

Symbolically, we can write **length**($x \rightarrow \text{rest}$) = 1 + **length**(*rest*).

We can visualize the recursion process for determining the length of $2 \rightarrow 10 \rightarrow 7 \rightarrow \emptyset$ as:

```
length(2→10→7→∅) = 1 + length(10→7→∅)
                  = 1 + 1 + length(7→∅)
                  = 1 + 1 + 1 + length(∅)
                  = 1 + 1 + 1 + 0
                  = 3
```

- **print**. The **print** method is iterative, so the loop continues while **p** is non-null. Each time through the loop **p** is updated to point to the next node in the list. Eventually **p** will point to a node with a **next** pointer equal to **nullptr**. After printing the data in this last node, **p** becomes null, and the loop terminates.
- **dispose**. The **dispose** method behaves similarly to the other two recursive methods. Notice, however, that it makes the recursive call deleting the nodes in the rest of the list *before* it deletes the current node. An attempt to access data via a pointer after using **delete** to deallocate that data results in undefined behavior; therefore, it is a logic error to attempt to do so. This means the code in **dispose** should not be written as

```
if (p) { // Logic error! Do not do it this way!
    delete p; // Deallocate this node first
    dispose(p->next); // Then free up the rest of the list
}
```

Here we are attempting to use **p->next**, which itself uses **p**, after deleting **p**'s memory.

16.5 Automating Testing

We know that a clean compile does not imply that a program will work correctly. We can detect errors in our code as we interact with the executing program. The process of exercising code to reveal errors or demonstrate the lack thereof is called testing. The informal testing that we have done up to this point has been adequate, but serious software development demands a more formal approach. As you gain more experience developing software you will realize that good testing requires the same skills and creativity as programming itself.

Until recently testing was often an afterthought. Testing was not seen to be as glamorous as designing and coding. Poor testing led to buggy programs that frustrated users. Also, tests were written largely after the program's design and coding were complete. The problem with this approach is major design flaws may not be revealed until late in the development cycle. Changes late in the development process are invariably more expensive and difficult to deal with than changes earlier in the process.

Weaknesses in the standard approach to testing led to a new strategy: test-driven development. In test-driven development the testing is automated, and the design and implementation of good tests is just as important as the design and development of the actual program. In pure TDD, tests are developed before any application code is written, and any application code produced is immediately subjected to testing.

16.9 defines the structure of a rudimentary test object.

Listing 16.9: tester.h

```
#define _TESTER_H_

#include <vector>
#include <string>

using std::vector;
using std::string;

class Tester {
    int error_count; // Number of errors detected
    int total_count; // Number of tests executed

    // Determines if double-precision floating-point
    // values d1 and d2 are "equal."
    // Returns true if their difference is less than tolerance.
    bool equals(double d1, double d2, double tolerance) const;

    // Displays vector a in human-readable form
    void print_vector(const vector<int>& a);
public:
    // Initializes a Tester object
    Tester();

    // Determines if an expected integer result (expected)
    // matches the actual result (actual). msg is the message
    // that describes the test.
    void check_equals(const string& msg, int expected, int actual);

    // Determines if an expected double result (expected)
    // matches the actual result (actual) or they differ by at
    // most tolerance. msg is the message that describes the test.
    void check_equals(const string& msg, double expected,
                     double actual, double tolerance);

    // Determines if an expected string result (expected)
    // matches the actual result (actual). msg is the message
    // that describes the test.
    void check_equals(const string& msg,
                     const vector<int>& expected,
                     const vector<int>& actual);
```

```

    // Reports the final results: number of tests passed and
    // failed and the total number of tests run.
    void report_results() const;
};

#endif

```

A simple test object keeps track of the number of tests performed and the number of failures. The client uses the test object to check the results of a computation against a predicted result. Notice that the **equals** method, which checks for the equality of two double-precision floating-point numbers is private, as it is meant to be used internally by the other methods within the class. The **equals** method works the same way as the **equals** function we examined in Listing 9.16 (floatequals.cpp).

Listing 16.10 (tester.cpp) implements the **Tester** methods.

Listing 16.10: tester.cpp

```

#include <iostream>
#include <cmath>
#include "tester.h"

using namespace std;

Tester::Tester(): error_count(0), total_count(0) {
    cout << "+-----" << endl;
    cout << "|   Testing   " << endl;
    cout << "+-----" << endl;
}

// d1 and d2 are "equal" if their difference is less than
// a specified tolerance
bool Tester::equals(double d1, double d2, double tolerance) const {
    return d1 == d2 || abs(d1 - d2) < tolerance;
}

// Prints the contents of a vector of integers.
void Tester::print_vector(const vector<int>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << '}';
}

// Compare integer outcomes
void Tester::check_equals(const string& msg, int expected, int actual) {
    cout << "[" << msg << "]" ";
    total_count++; // Count this test
    if (expected == actual)
        cout << "OK" << endl;
    else {
        error_count++; // Count this failed test
    }
}

```



```

        cout << "*** Failed! Expected: " << expected
              << ", actual: " << actual << endl;
    }
}

// Compare double-precision floating-point outcomes
void Tester::check_equals(const string& msg, double expected,
                        double actual, double tolerance) {
    cout << "[" << msg << "]" ";
    total_count++; // Count this test
    if (equals(expected, actual, tolerance))
        cout << "OK" << endl;
    else {
        error_count++; // Count this failed test
        cout << "*** Failed! Expected: " << expected
              << ", actual: " << actual << endl;
    }
}

// Compare string outcomes
void Tester::check_equals(const string& msg, const vector<int>& expected,
                        const vector<int>& actual) {
    cout << "[" << msg << "]" ";
    total_count++; // Count this test
    if (expected == actual)
        cout << "OK" << endl;
    else {
        error_count++; // Count this failed test
        cout << "*** Failed! Expected: ";
        print_vector(expected);
        cout << " Actual: ";
        print_vector(actual);
        cout << endl;
    }
}

// Display final test statistics
void Tester::report_results() const {
    cout << "+-----" << endl;
    cout << "| " << total_count << " tests run, "
          << total_count - error_count << " passed, "
          << error_count << " failed" << endl;
    cout << "+-----" << endl;
}
}

```

Listing 16.11: testvectorstuff.cpp

```

#include <iostream>
#include <vector>
#include "tester.h"

using namespace std;

```

```

// sort has a bug (it does not do anything)
void sort(vector<int>& vec) {
    // Not yet implemented
}

// sum has a bug (misses first element)
int sum(const vector<int>& vec) {
    int total = 0;
    for (size_t i = 1; i < vec.size(); i++)
        total += vec[i];
    return total;
}

int main() {
    Tester t; // Declare a test object
    // Some test cases to test sort
    vector<int> vec { 4, 2, 3 };
    sort(vec);
    t.check_equals("Sort test #1", {2, 3, 4}, vec);
    vec = {2, 3, 4};
    sort(vec);
    t.check_equals("Sort test #2", {2, 3, 4}, vec);
    // Some test cases to test sum
    t.check_equals("Sum test #1", sum({0, 3, 4}), 7);
    t.check_equals("Sum test #2", sum({-3, 0, 5}), 2);
}

```

The program's output is

```

+-----+
|  Testing
+-----+
[Sort test #1] *** Failed!  Expected: {2,3,4}   Actual:   {4,2,3}
[Sort test #2] OK
[Sum test #1] OK
[Sum test #2] *** Failed!  Expected: 5, actual: 2

```

Notice that the **sort** function has yet to be implemented, but we can test it anyway. The first test is bound to fail. The second test checks to see if our **sort** function will not disturb an already sorted vector, and we pass this test with no problem. This is an example of *coincidental correctness*.

In the **sum** function, the programmer was careless and used 1 as the beginning index for the vector. Notice that the first test does not catch the error, since the element in the zeroth position (zero) does not affect the outcome. A tester must be creative and devious to try and force the code under test to demonstrate its errors.

16.6 Convenient High-quality Pseudorandom Numbers

In Section 13.5 we used some classes from the standard C++ library to generate high-quality pseudorandom numbers. Listing 13.10 (`highqualityrandom.cpp`) used three kinds of objects—`random_device`, `mt19937`, and `uniform_int_distribution`—to produce good pseudorandom sequences.

The C++ class construct allows us to creatively combine multiple sources of functionality into one convenient package. Listing 16.12 (`uniformrandom.h`) contains the `UniformRandomGenerator` class.

Listing 16.12: `uniformrandom.h`

```
#ifndef _UNIFORM_RANDOM_DEFINED_
#define _UNIFORM_RANDOM_DEFINED_

#include <random>

using std::uniform_int_distribution;
using std::random_device;
using std::mt19937;

class UniformRandomGenerator {

    // A uniform distribution object
    uniform_int_distribution<int> dist;
    // A Mersenne Twister random number generator with a seed
    // obtained from a random_device object
    mt19937 mt;
public:
    // The smallest pseudorandom number this generator can produce
    const int MIN;

    // The largest pseudorandom number this generator can produce
    const int MAX;

    // Create a pseudorandom number generator that produces values in
    // the range low...high
    UniformRandomGenerator(int low, int high) : dist(low, high),
        mt(random_device()()),
        MIN(low), MAX(high) {}

    // Return a pseudorandom number in the range MIN...MAX
    int operator()() {
        return dist(mt);
    }
};
#endif
```

The `UniformRandomGenerator` class provides a simplified interface to programmers who need access to high-quality pseudorandom numbers. Behind the scenes, every `UniformRandomGenerator` object contains its own `uniform_int_distribution` object and `mt19937` object. The constructor accepts the minimum and maximum values in the range of pseudorandom numbers desired. The constructor uses this range to construct the appropriate `uniform_int_distribution` object for this range. The constructor also initializes the `mt19937` object field. The `UniformRandomGenerator` constructor passes to the constructor of the `mt19937` class a temporary `random_device` object. Since

a **UniformRandomGenerator** object uses the **random_device** only for creating its **mt19937** field and does not need it later, **UniformRandomGenerator** objects do not contain a **random_device** field.

To create a **UniformRandomGenerator** object that produces pseudorandom integers in the range $-100 \dots 100$, a client need only write

```
UniformRandomGenerator gen(-100, 100);
```

The **UniformRandomGenerator** class also provides an **operator()** method. This allows a client to “call” an object as if it were a function. Given a **UniformRandomGenerator** object named **gen**, we can assign a pseudorandom number to an integer variable **x**, with the statement

```
int x = gen();
```

This statement may appear to be calling a global function named **gen**; in fact, it is calling the **operator()** method of the **UniformRandomGenerator** class on behalf of object **gen**. The expression **gen()** is syntactic sugar for **gen.operator()()**. The expression **gen.operator()()** may look unusual, but it simply is invoking the **UniformRandomGenerator::operator()** method on behalf of **gen** passing no arguments in the empty last pair of parentheses.

Listing 16.13 (**testuniformrandom.cpp**) is a simplified remake of Listing 13.10 (**highqualityrandom.cpp**). In Listing 16.13 (**testuniformrandom.cpp**) we see that with our **UniformRandomGenerator** class we can generate high-quality pseudorandom numbers with a single object that is simple to use.

Listing 16.13: **testuniformrandom.cpp**

```
#include <iostream>
#include "uniformrandom.h"

using namespace std;

int main() {
    // Pseudorandom number generator with range 0...9,999
    UniformRandomGenerator rand(0, 9999);

    // Total counts over all the runs.
    // Make these double-precision floating-point numbers
    // so the average computation at the end will use floating-point
    // arithmetic.
    double total5 = 0.0, total9995 = 0.0;

    // Accumulate the results of 10 trials, with each trial
    // generating 1,000,000,000 pseudorandom numbers
    const int NUMBER_OF_TRIALS = 10;

    for (int trial = 1; trial <= NUMBER_OF_TRIALS; trial++) {
        // Initialize counts for this run of a billion trials
        int count5 = 0, count9995 = 0;
        // Generate one billion pseudorandom numbers in the range
        // 0...9,999 and count the number of times 5 and 9,995 appear
        for (int i = 0; i < 1000000000; i++) {
            // Generate a pseudorandom number in the range 0...9,999
            int r = rand();
            if (r == 5)
                count5++;           // Number 5 generated, so count it
        }
    }
}
```

```

        else if (r == 9995)
            count9995++; // Number 9,995 generated, so count it
        }
        // Display the number of times the program generated 5 and 9,995
        cout << "Trial #" << setw(2) << trial << " 5: " << setw(6) << count5
            << "    9995: " << setw(6) << count9995 << endl;
        total5 += count5; // Accumulate the counts to
        total9995 += count9995; // average them at the end
    }
    cout << "-----" << endl;
    cout << "Averages for " << NUMBER_OF_TRIALS << " trials: 5: "
        << setw(6) << total5 / NUMBER_OF_TRIALS << "    9995: "
        << setw(6) << total9995 / NUMBER_OF_TRIALS << endl;
}

```

Note that in Listing 16.13 (testuniformrandom.cpp), the statement

```
int r = rand();
```

is not a call to our familiar `std::rand` function; rather, `rand` here is a `UniformRandomGenerator` object, and the statement is invoking its `operator()` method.

16.7 Summary

- The object-oriented features of C++ allow developers to design a variety of useful classes to make programming tasks easier.
- Sophisticated computation can be packaged within objects that are relatively easy for clients to use.

16.8 Exercises

1. Recall the private `insert` method from Listing 16.7 (intlist.cpp) and consider the following new `insert` function which contains only a couple of minor modifications to the original version:

```

IntList::Node *IntList::insert(Node *p, int n) {
    // Check that we are not at the end of the list and
    // that the current list element is less than the new
    // item to add.
    if (p && p->data < n)
        p->next = insert(p->next, n); // Insert into rest of list
    else { // Insert new item at the front of the list
        Node *temp = new Node;
        temp->data = n;
        temp->next = p;
        p = temp;
    }
    return p;
}

```

- (a) Without executing the code, can you predict how this new version behaves differently, if at all, from the original version?
- (b) Replace the **insert** function in Listing 16.7 (intlist.cpp) with this new version to check your prediction. Can you explain why this new **insert** function behaves the way it does?

Chapter 17

Inheritance and Polymorphism

In Chapter 16 we saw how it is possible to design classes from which clients can produce objects that exhibit somewhat sophisticated behavior. The **Rational**, **Stopwatch**, **LinkedList**, and **Tester** classes each were each built from scratch. C++ provides a way to create a class from an existing class by a process known as *inheritance*. Through this process the new class inherits all the characteristics of the existing class, and the class developer can extend and customize the inherited functionality as well as add new features.

17.1 I/O Stream Inheritance

Recall from Section 13.3 that writing data to a file is almost as easy as printing it on the screen. Once an **ofstream** object is set up, we can use the `<<` operator in the same way we use it with the **cout** object:

```
ofstream fout("myfile.dat");
int x = 10;
if (fout.good()) {    // Make sure the file was opened properly
    fout << "x = " << x << endl;
    fout.close();
}
else
    cout << "Unable to write to the file \"myfile.dat\" " << endl;
```

Section 15.6 showed how `operator<<` may be overloaded for a programmer-defined type. For a vector of integers we can write the function

```
ostream& operator<<(ostream& os, const vector<int>& vec) {
    os << '{';
    int n = vec.size();
    if (n > 0) {    // Is the vector non-empty?
        os << vec[0]; // Send first element to the stream
        for (int i = 1; i < n; i++)
            os << ', ' << vec[i];    // Send remaining elements
    }
    os << '}';
```



```

    return os;
}

```

Can we easily adapt our << operator so that we can use it with **ofstream** objects as well?

The answer, perhaps surprisingly, is we do not have to adapt our **operator<<** function; it works as is with **ofstream** objects. How can this be, since the **cout** object has type **ostream** which is not the same class as **ofstream**?

C++ allows some automatic conversions among the built in numeric types; for example, an **int** is widened to a **double**, and a **double** is truncated to an **int**. **bool**s and **int**s are freely interconvertible. These types are built into the language, so standard conversions apply. When it comes to programmer-defined types, however, the compiler is unable automatically to generate code to convert from one type to another. Most of the time it would make no sense to do so—imagine attempting to convert an object of type **Stopwatch** (16.2) to an object of type **Tester** (16.9).

So, how is it that an object of type **ofstream** can be converted automatically to an object of type **ostream**? You might think that it is because they are both part of the standard library, so the conversion has been programmed into them. The fact is, no conversion takes place! Any object of type **ofstream** object is automatically treated as if it were an **ostream** object because the **ostream** and **ofstream** classes are related in a special way. The **ofstream** class is *derived* from the **ostream** class. We say that **ostream** is the *base class* and **ofstream** is the *derived class*. Sometimes the term *superclass* is used for the base class, in which case the derived class is called the *subclass*. The base/derived class terminology is used interchangeably with super/subclass, although the C++ community tends to prefer the terms base/derived class to super/subclass. The process is known as *inheritance* because the derived class inherits all the characteristics of its base class. The terms *derivation*, *subclassing* and *specialization* are sometimes used in place of the term *inheritance*.

As a consequence of the derivation process, an instance of the derived class can be treated as if it were an instance of the base class. Listing 17.1 (printstreams.cpp) shows how a function can process instances of two different classes related by inheritance.

Listing 17.1: printstreams.cpp

```

#include <fstream>

using namespace std;

void print(ostream& os, int n) {
    os.width(10); // Right justified in 10 spaces
    os.fill('*'); // Fill character is *
    os << n << endl;
}

int main() {
    // Pretty print to screen
    print(cout, 35);

    // Pretty print to text file
    ofstream fout("temp.data");
    if (fout.good()) {
        print(fout, 36);
        fout.close();
    }
}

```

Observe that the `print` function in Listing 17.1 (`printstreams.cpp`) expects a client to pass an `ostream` reference as the first parameter, but the `main` function can send it both `ostream` and `ofstream` object references with equal ease.

This ability to allow a subclass object to be used in any context that expects a superclass object is known as the *Liskov Substitution Principle*, after computer scientist Barbara Liskov.

The term *is a* has a special meaning in the context of inheritance. Suppose we have class **B** and class **D** derived from **B**. **B** is the base class and **D** is the derived class. Since we can treat an instance of a derived class as if it were an instance of its base class, if we declared a **D** object as

```
D d; // d is a D object
```

we can say *d is a D* (as it is declared), and we also can say *d is a B*.

17.2 Inheritance Mechanics

The developers of the `ofstream` class did not begin with a blank slate. The `ostream` class existed first, and the developers specified the `ofstream` class in such a way so any `ofstream` object would be treated as a specific kind of `ostream`. In this section we will examine a very simple example that illustrates the mechanics of class inheritance in C++.

Suppose we have class **B** defined as shown here:

```
class B {
    // Details omitted
};
```

To derive a new class **D** from **B** we use the following syntax:

```
class D: public B {
    // Details omitted
};
```

Here **B** is the base class and **D** is the derived class. **B** is the pre-existing class, and **D** is the new class based on **B**.

To see how inheritance works, consider classes **B** and **D** with additional detail:

```
class B {
    // Other details omitted
public:
    void f();
};

void B::f() {
    std::cout << "In function 'f'" << std::endl;
}

class D: public B {
    // Other details omitted
public:
    void g();
};
```

```
};

void D::g() {
    std::cout << "In function 'g'" << std::endl;
}
```

The client code

```
B myB;
D myD;
myB.f();
myD.f();
myD.g();
```

prints

```
In function 'f'
In function 'f'
In function 'g'
```

Even though the source code for class **D** does not explicitly show the definition of a method named **f**, it has such a method that it inherits from class **B**.

Note that inheritance works in one direction only. Class **D** inherits method **f** from class **B**, but class **B** cannot inherit **D**'s **g** method. Given the definitions of classes **B** and **D** above, the following code is illegal:

```
B myB;
myB.g(); // Illegal, a B object is NOT a D object
```

If we omit the word **public** from class **D**'s definition, as in

```
class D: B {
    // Details omitted
};
```

all the public members of **B** inherited by **D** objects will be *private* by default; for example, if base class **B** looks like the following:

```
class B {
public:
    void f();
};
```



```
void B::f() {
    std::cout << "In function 'f'" << std::endl;
}
```

the following code is not legal:

```
D myD;
myD.f(); // Illegal, method f now is private!
```

This means a client may not treat a **D** object exactly as if it were a **B** object. This violates the Liskov Substitution Principle, and the *is a* relationship does not exist.

While this *private inheritance* is useful in rare situations, the majority of object-oriented software design uses public inheritance. C++ is one of the few object-oriented languages that supports private inheritance.

There is no limit to the number of classes a developer may derive from a single base class. In the following code:

```
class D1: public B { /* Details omitted */ };
class D2: public B { /* Details omitted */ };
class D3: public B { /* Details omitted */ };
```

the classes **D1**, **D2**, and **D3** are all derived from class **B**.

A developer may derive a class from more than one base class in a process known as *multiple inheritance*. In the following code:

```
class B1 { /* Details omitted */ };
class B2 { /* Details omitted */ };

class D: public B1, public B2 { /* Details omitted */ };
```

Here class **D** has two base classes, **B1** and **B2**. If a particular function or method expects a **B1** object as an argument, the caller may pass a **D** object; similarly, any context that requires a **B2** object will accept a **D** object. In object-oriented design, multiple inheritance is not as common as single inheritance (one base class).

The next section provides a simple example that shows how inheritance works.

17.3 Uses of Inheritance

Inheritance is a design tool that allows developers to take an existing class and produce a new class that provides enhanced behavior or different behavior. The enhanced or new behavior does not come at the expense of existing code; that is, when using inheritance programmers do not touch any source code in the base class. Also, developers can leverage existing code (in the base class) without duplicating it in the derived classes.

Listing 17.2 (fancytext.cpp) provides a complete program with simple classes that demonstrate how we can use inheritance to enhance the behavior of unadorned text objects.

Listing 17.2: fancytext.cpp

```
#include <string>
#include <iostream>

using namespace std;

// Base class for all Text derived classes
class Text {
    string text;
public:
    // Create a Text object from a client-supplied string
    Text(const string& t): text(t) {}

    // Allow clients to see the text field
    virtual string get() const {
        return text;
    }

    // Concatenate another string onto the
    // back of the existing text
    virtual void append(const string& extra) {
        text += extra;
    }
};

// Provides minimal decoration for the text
class FancyText: public Text {
    string left_bracket;
    string right_bracket;
    string connector;
public:
    // Client supplies the string to wrap plus some extra
    // decorations
    FancyText(const string& t, const string& left,
              const string& right, const string& conn):
        Text(t), left_bracket(left),
        right_bracket(right), connector(conn) {}

    // Allow clients to see the decorated text field
    string get() const override {
        return left_bracket + Text::get() + right_bracket;
    }
}
```

```

    // Concatenate another string onto the
    // back of the existing text, inserting the connector
    // string
    void append(const string& extra) override {
        Text::append(connector + extra);
    }
};

// The text is always the word FIXED
class FixedText: public Text {
public:
    // Client does not provide a string argument; the
    // wrapped text is always "FIXED"
    FixedText(): Text("FIXED") {}

    // Nothing may be appended to a FixedText object
    void append(const string&) override {
        // Disallow concatenation
    }
};

int main() {
    Text t1("plain");
    FancyText t2("fancy", "<<", ">>", "***");
    FixedText t3;
    cout << t1.get() << endl;
    cout << t2.get() << endl;
    cout << t3.get() << endl;
    cout << "-----" << endl;
    t1.append("A");
    t2.append("A");
    t3.append("A");
    cout << t1.get() << endl;
    cout << t2.get() << endl;
    cout << t3.get() << endl;
    cout << "-----" << endl;
    t1.append("B");
    t2.append("B");
    t3.append("B");
    cout << t1.get() << endl;
    cout << t2.get() << endl;
    cout << t3.get() << endl;
}

```

The output of Listing 17.2 (fancytext.cpp) is

```

plain
<<fancy>>
FIXED
-----
plainA
<<fancy***A>>
FIXED
-----
plainAB
<<fancy***A***B>>
FIXED

```

In Listing 17.2 (fancytext.cpp), **Text** serves as the base class for two other classes, **FancyText** and **FixedText**. A **Text** object wraps a `std::string` object, and since the string object is private to the **Text** class, clients cannot get to the string object directly. The clients may see the string via the **get** method, and can modify the wrapped string only in a limited way via the **append** method. Listing 17.2 (fancytext.cpp) contains two new keywords:

- The **virtual** keyword appears in front of the definitions of the **get** and **append** methods in the **Text** class. The **virtual** specifier indicates that the designer of the **Text** class intends for derived classes to be able to customize the behavior of the **get** and **append** methods.
- The **override** keyword appears at the end of the method definition headers for the **get** and **append** methods in the **FancyText** class and the **append** method for the **FixedText**. This means the exact behavior of these methods will be different in some way from their implementation in the **Text** class. The **FixedText** class overrides only the **append** method; it inherits the **get** method without alteration.

Consider the **FixedText::get** method:

```

class FancyText: public Text {
    // . . .
public:
    // . . .
    string get() const override {
        return left_bracket + Text::get() + right_bracket;
    }
};

```

The **FancyText** class alters the way **get** works. We say the **FancyText** class *overrides* its inherited **get** method. The keyword **override** emphasizes the fact that the code in the **get** method in **FancyText** intends to do something differently from the code in **Text**'s **get** method. In this case the **FancyText::get** method builds a string result by concatenating three other strings: the first string is a front bracketing string, the second is the wrapped string, and the third string is a trailing bracketing string. Notice that the second string is obtained with the expression

```
Text::get()
```

This calls the base class version of the **get** method. We say that the **FancyText::get** method delegates some of its work to the base class version, **Text::get**. Notice that the statement

```
return left_bracket + text + right_bracket; // Illegal
```

would be illegal because `text` is private to the base class. The member specifier `private` means inaccessible outside of the class, period. Derived classes have no special privilege in this regard. In order to perform the delegation we must use `Text::get()` and not simply `get()`; the unqualified expression `get()` is equivalent to `this->get`, which calls `FancyText::get`. This means invoking the unqualified `get` within the definition of `FancyText::get` is a recursive call which is not what we want.

A method declared virtual in a base class is automatically virtual in its derived classes. You may repeat the `virtual` specifier in derived classes, but the use of `override` makes this redundant.



The `override` keyword was added to the language in C++11. Prior to C++11 when a method in a derived class had the same signature as a virtual method in its base class, the method implicitly overrode its base class version. The problem was that a programmer could intend to override a method in the derived class but get the signature wrong. The resulting method *overloaded* the original method rather than overriding it. If a programmer uses the `override` specifier and uses a signature that does not match the base class version, the compiler will report an error. The `override` specifier provides a way for programmers to explicitly communicate their intentions.

For backwards compatibility the `override` keyword is optional. Its presence enables the compiler to verify that the method is actually overriding a `virtual` method in the base class. Without it, the programmer must take care to faithfully reproduce the signature of the method to override.

The `override` keyword is a *context-sensitive keyword*, meaning it is a keyword only when appearing as it does here in the declaration of a method header. In other contexts it behaves like an identifier.

The `FixedText` class does not change how the `get` method works for `FixedText` objects; thus, the `FixedText::get` method does not add any special decorations.

The `FancyText::append` method overrides the inherited `append` method by inserting a special separator string in between the existing wrapped text string and the string to append. Like `FancyText::get`, the `FancyText::append` method delegates the actual concatenation operation to its base class version because code within the `FancyText` class cannot directly influence its `text` field.

The `FixedText::append` method does not allow the existing wrapped text string to be modified.

The constructor for `Text` requires a single `std::string` parameter. The constructor for `FancyText` requires four string arguments:

```
class FancyText: public Text {
    // . . .
public:
    FancyText(const string& t, const string& left,
              const string& right, const string& conn):
        Text(t), left_bracket(left),
              right_bracket(right), connector(conn) {}
    // . . .
}
```



```
};
```

We want to assign the constructor’s first parameter, `t`, to the inherited member `text`, but `text` is private in the base class. This means the `FancyText` constructor cannot initialize it directly. Since the constructor of its base class knows what to do with this parameter, the first expression in the constructor initialization list (the part between the `:` and the `{}`):

```
... Text(t) ...
```

explicitly calls the base class constructor, passing it `t`. This base class initialization expression must be the first thing in the initialization list because C++ requires that the parts of an object inherited from its base class be initialized before any new parts (added by the derived class) are initialized. The next three expressions in the initialization list:

```
... left_bracket(left), right_bracket(right), connector(conn) ...
```

initialize the `left_bracket`, `right_bracket`, and `connector` fields as usual. The body of the constructor is empty as no other initialization work is necessary.

The `FixedText` class is somewhat simpler than the `FancyText` class. It adds no fields and does not override the `get` method. Its constructor accepts no parameters because clients are not permitted to determine the contents of the wrapped string—the `text` field is always the word `FIXED`. It inherits the `text` field and `get` method (as is) from the `Text` class.

Given the `Text` and `FancyText` classes as defined in Listing 17.2 (`fancytext.cpp`), the following client code is legal:

```
Text t1("ABC");
FancyText t2("XYZ", "[", "]", ":");
cout << t1.get() << " " << t2.get() << endl;
t1 = t2;
cout << t1.get() << " " << t2.get() << endl;
```

Since `t2`’s declared type is `FancyText`, by virtue of inheritance, `t2` is a `Text` object as well. The above code fragment prints

```
ABC <<XYZ>>
XYZ <<XYZ>>
```

Notice that the assignment

```
t1 = t2;
```

copied into object `t1` only the fields that `FancyText` objects have in common with `Text` objects; that is, the `text` field. Since `t1` is a plain `Text` object, it does not have the `left_bracket`, `right_bracket`, and `connector` fields capable of storing the additional data contained in a `FancyText` object. This process of losing derived class data when assigning to a base class instance is known as *object slicing*. The parts that will not fit into the base class instance are “sliced” off. Figure 17.1 illustrates object slicing. Note that the assignment in the other direction:

```
t2 = t1; // Illegal
```

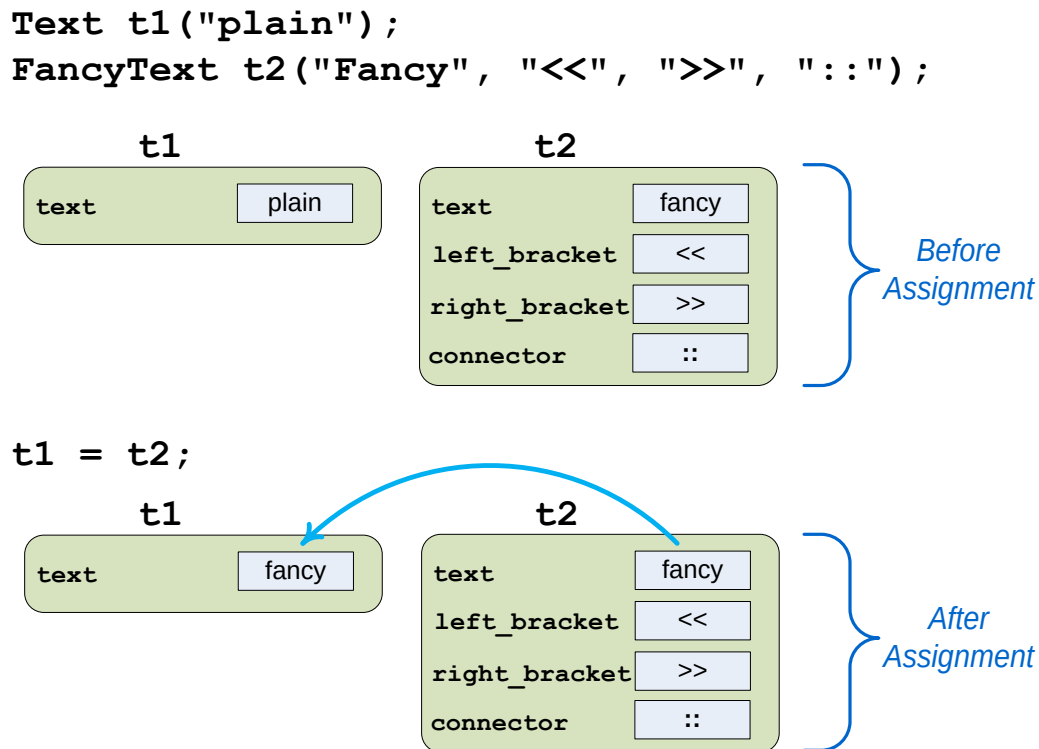


Figure 17.1: Object slicing during assignment of a derived class instance to a base class variable

is not possible; even though any **FancyText** object *is a* **Text** object, we cannot say any **Text** object is a **FancyText** object (it could be a **FixedText** object or just a **Text** object without any special decoration). Figure 17.2 shows how such an attempted assignment would be meaningless because the base class instance has missing information needed by the derived class instance. Failure to properly assign all the fields in the derived class instance would produce an object that is not well defined, so C++ does not allow the assignment.

FixedText class instances do not contain any additional data that plain **Text** instances do not have, but the assignment rules remain the same:

```
Text t1("ABC");
FixedText t3;
cout << t1.get() << " " << t3.get() << endl;
t1 = t3;
cout << t1.get() << " " << t3.get() << endl;
```

is legal and produces

```
ABC  FIXED
FIXED  FIXED
```

```
Text t1("plain");
FancyText t2("Fancy", "<<", ">>", "::");
```

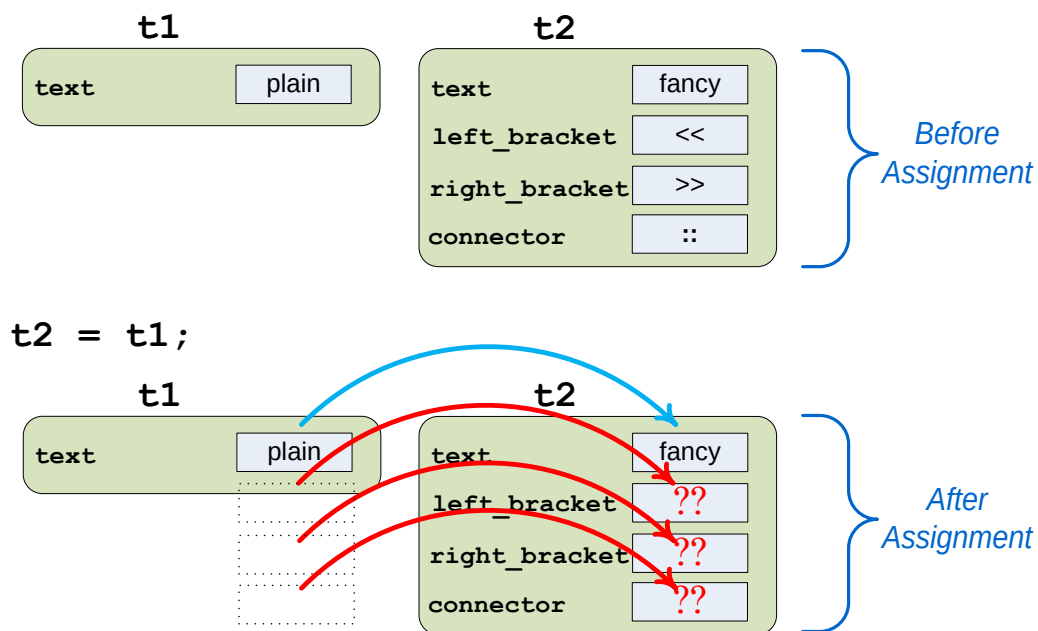


Figure 17.2: Assigning a base class instance to a derived class variable is not possible and is illegal in C++

but the statement

```
t3 = t1; // Illegal
```

is illegal and will not compile.



It always is legal to assign a derived class instance to a variable of a base type. This is because a derived class instance *is a* specific kind of base class instance. In contrast, it never is legal to assign a base class instance to a variable of a derived type. This is because the *is a* relationship is only one directional, from a derived class to its base class.

The **Text**, **FancyText**, and **FixedText** classes form a small *class hierarchy*. The relationships in this class hierarchy can be represented in a graphical form using the Unified Modeling Language (UML). Figure 17.3 shows the UML diagram for our simple **Text** class hierarchy. A rectangle represents a class. **Text**, the base class, appears at the top of the Figure 17.3. The two derived classes appear below **Text**. The inheritance arrow points from the derived classes to the base class. The arrow represents the *is a* relationship which flows upward. This visual layout and level of detail provided by the UML diagram more succinctly communicates the relationships amongst the classes than does the C++ source code.

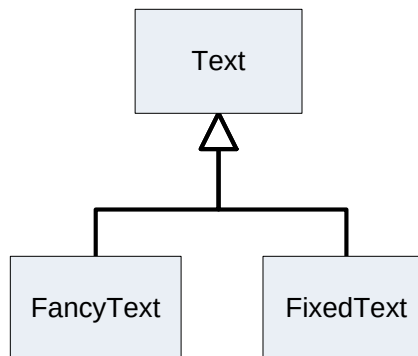


Figure 17.3: The Unified Modeling Language class hierarchy diagram for the family of text classes

The UML is a large, complex graphical language that may be used to model many facets of the software development process. More information about the UML is available at <http://www.uml.org>.

17.4 Polymorphism

Recall the **Text** class hierarchy from Listing 17.2 (`fancytext.cpp`) illustrated in Figure 17.3. Consider the following client code fragment:

```

Text t1("ABC");
FancyText t2("XYZ", "[", "]", ":");
cout << t1.get() << " " << t2.get() << endl;

```

What is the nature of the machine code generated by the expression `t1.get()`? The compiler translates this higher-level expression into the machine language instruction that causes the program's execution to jump to another part of the compiled code. Where does the program's execution jump to? The variable `t1`'s declared type is **Text**, the program's execution jumps to the compiled **Text::get** method (also the address of `t1` is passed as the implicit **this** parameter to the method so the compiled code can access `t1`'s **text** field).

For the expression `t2.get()`, the program's execution jumps to **FancyText**'s compiled **get** method, passing the address of object `t2` as the implicit **this** parameter.

When the compiler can determine which method to execute based on the declared type of an object, the process is known as *static binding* or *early binding*. Static binding is used in all cases for non-virtual methods and in the cases we have seen so far for virtual methods.

The situation is different if we use pointers to objects instead of the objects themselves. Consider the following code:

```

Text t1("ABC");
FancyText t2("XYZ", "[", "]", ":");
cout << t1.get() << " " << t2.get() << endl;

```

```
Text *p1, *p2;
p1 = &t1;
p2 = &t2;
```

The variables **p1** and **p2** are declared to be pointers to **Text** objects. The variable **t1** is a **Text** object, so the assignment

```
p1 = &t1;
```

makes perfect sense. The variable **t2** has a declared type of **FancyText**, but a **FancyText** object *is a* **Text** object, so the assignment

```
p2 = &t2;
```

is legal since **p2** is indeed pointing to a special kind of **Text** object.

What does the following code fragment print?

```
Text t1("ABC");
FancyText t2("XYZ", "[", "]", ":");
cout << t1.get() << " " << t2.get() << endl;
Text *p1, *p2;
p1 = &t1;
p2 = &t2;
cout << p1->get() << " " << p2->get() << endl;
```

This code displays

```
ABC <<XYZ>>
ABC <<XYZ>>
```

Even though **p2**'s declared type is "pointer to a **Text** object" not "pointer to a **FancyText** object," the expression **p2->get()** calls **FancyText::get**, not **Text::get**. How does the compiler determine which method to call in this case? The answer may be surprising: The compiler does *not* determine which method to call!

In the case of a virtual method invoked via a pointer, the running program, not the compiler, determines exactly which code to execute. The process is known as *dynamic binding* or *late binding*. Static binding is relatively easy to understand: the method to execute depends on the declared type of the variable upon which the method is invoked. The compiler keeps track of the declared type of every variable, so the choice is easy. Inheritance and the *is a* relationship make things more complicated. In the example above, **p2**'s declared type is "pointer to **Text**." If the compiler were given the authority to select the method to call for the expression **p2->get()**, its only choice would be **Text::get**; however, **p2** actually is pointing to a **FancyText** object. How does the executing program know which code to execute?

We know that every instance of a class contains its own copy of the fields declared within the class. In the code

```
Text word1("Wow"), word2("Wee");
```

the **word1** object's **text** field in the **std::string** object representing the string **"Wow"**, and **word2**'s **text** field is **"Wee"**. In reality, if a class contains at least one virtual method, all instances of that class

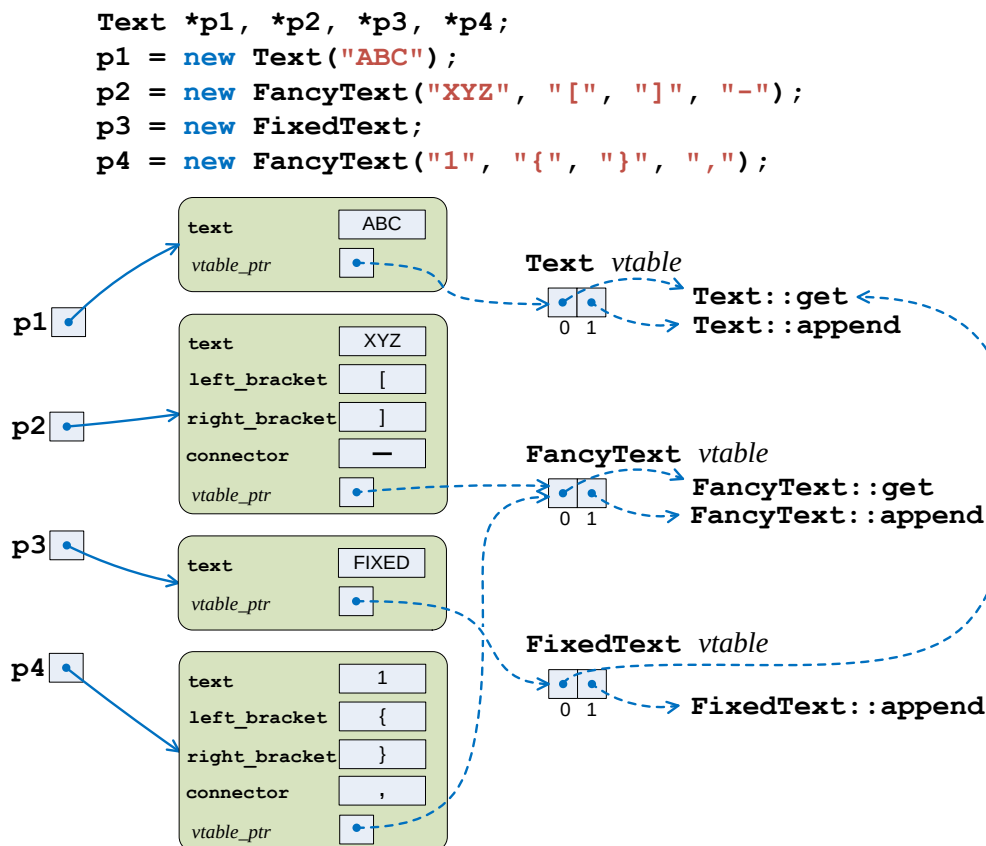


Figure 17.4: Several objects and their associated vtables. You will not find any evidence in the C++ source code of the dashed pointers and vtables shown in this diagram. When virtual methods are involved the compiler produces executable code that builds these hidden data structures behind the scenes. When a client calls a virtual method via a pointer to an object, the run-time environment locates the exact method to execute by following the vtable pointer stored in the object itself. Observe that since the `FixedText` class does not override the inherited `get` method, its vtable references the same `get` code as that of `Text`'s vtable.

will contain one extra “hidden” field, a pointer to an array of virtual method pointers. This array of method pointers commonly is called the *vtable*. One vtable exists for each class, but all instances of that class must store a pointer to that shared vtable. The compiler assigns an index to each virtual method in a class. In our example, the `Text` class contains two virtual methods, so the index of `get` might be 0, and the index of `append` might be 1.

Figure 17.4 illustrates a typical scenario with vtables. In this case the compiler translates the expression `p1->get()` into the machine language equivalent of `p1->vtable[0]()`, which invokes the correct `get` method for the instance pointed to by `p1`. Similarly for the `append` method, `p1->append("suffix")` internally becomes `p1->vtable[1]("suffix")`. If `p1` points to a `Text` instance, `p1->vtable` points to the vtable array for the `Text` class. If `p1` points instead to a `FancyText` instance, `p1->vtable` points to the vtable array of the `FancyText` class. In either case the correct method is selected during the program's execution.

You may argue that perhaps the compiler could be made to be able to determine in all situations which virtual method to call. In the code from above:

```
Text t1("ABC");
FancyText t2("XYZ", "[", "]", ":");
cout << t1.get() << " " << t2.get() << endl;
Text *p1, *p2;
p1 = &t1;
p2 = &t2;
cout << p1->get() << " " << p2->get() << endl;
```

cannot the compiler deduce **p2**'s exact type since **p2** is assigned to point to **t2**, and it knows **t2**'s type is **FancyText**?

Such extended analysis capabilities would further complicate the compiler, and C++ compilers are already very complicated pieces of software. Attempting to add this ability would prove futile anyway because the compiler cannot in general always determine which method to invoke. To see why, consider the following code fragment:

```
Text *p;
if (std::rand() % 2 == 0)
    p = new Text("ABC");
else
    p = new FancyText("XYZ", "[", "]", ":");
cout << p->get() << endl;
```

This code generates a pseudorandom number at run time. If it generates an even number, it directs **p** to point to a plain **Text** object; otherwise, it directs **p** to point to a **FancyText** object. Which method will this code call, **Text::get** or **FancyText::get**? Since the code generates the pseudorandom number at run time, the compiler is powerless to determine the exact type of the object to which **p** points; thus, the compiler cannot determine which **get** method to call. The compiler can, however, generate code that jumps to the method found at the address stored in **p**'s associated vtable at the index corresponding to the **get** method.

Listing 17.3 (vtablesize.cpp) demonstrates that the vtable pointer does occupy space within an object that contains a virtual method.

Listing 17.3: vtablesize.cpp

```
#include <iostream>

using namespace std;

// A class with no virtual methods
class NoVTable {
    int data;
public:
    void set(int d) { data = d; }
    int get() { return data; }
};

// A class with virtual methods
class HasVTable {
    int data;
public:
```

```

    virtual void set(int d) { data = d; }
    virtual int get() { return data; }
};

int main() {
    NoVTable no_vtable;
    HasVTable has_vtable;
    no_vtable.set(10);
    has_vtable.set(10);
    cout << "no_vtable size = " << sizeof no_vtable << endl;
    cout << "has_vtable size = " << sizeof has_vtable << endl;
}

```

The output of Listing 17.3 (vtablesize.cpp) is

```

no_vtable size = 4
has_vtable size = 8

```

A **NoVTable** requires four bytes for its integer field, but a **HasVTable** object occupies eight bytes—four bytes for its integer field and four bytes for its secret vtable pointer.

Dynamic binding enables a powerful technique in object-oriented programming called *polymorphism*. A polymorphic method behaves differently depending on the actual type of object upon which it is invoked. Consider Listing 17.4 (polymorphicvector.cpp).

Listing 17.4: polymorphicvector.cpp

```

#include <string>
#include <vector>
#include <iostream>

using namespace std;

// Base class for all Text derived classes
class Text {
    string text;
public:
    Text(const string& t): text(t) {}
    virtual string get() const {
        return text;
    }
};

// Provides minimal decoration for the text
class FancyText: public Text {
    string left_bracket;
    string right_bracket;
    string connector;
public:
    FancyText(const string& t, const string& left,
              const string& right, const string& conn):
        Text(t), left_bracket(left), right_bracket(right),

```



```

        connector(conn) {}
        string get() const override {
            return left_bracket + Text::get() + right_bracket;
        }
};

// The text is always the word FIXED
class FixedText: public Text {
public:
    FixedText(): Text("FIXED") {}
};

int main() {
    vector<Text*> texts { new Text("Wow"),
                        new FancyText("Wee", "[", "]", "-"),
                        new FixedText,
                        new FancyText("Whoa", ":", ":", ":") };

    for (auto t : texts)
        cout << t->get() << endl;
}

```

We know from Section 11.1 that a vector is a collection of homogeneous elements. *Homogeneous* means the elements in a vector must all be of the same type. Homogeneity takes on a deeper meaning when inheritance and the *is a* relationship is involved. In Listing 17.4 (polymorphicvector.cpp) the declared type of the `texts` vector is `vector<Text*>`. With inheritance, not only can `texts` hold pointers to simple `Text` objects, it also simultaneously can hold pointers to `FixedText` and `FancyText` objects. Listing 17.4 (polymorphicvector.cpp) prints

```

Wow
[Wee]
FIXED
:Whoa:

```

As we can see, the expression `t->get()` in the `main` function is polymorphic; the actual `get` method invoked—`Text::get`, `FancyText::get`, or `FixedText::get`—depends on the exact type of `t`. As `t` assumes the value of each element in the vector during the loop's execution, the exact type of object that `t` points to varies. Even though all the elements of the `texts` vector are pointers to `Text` objects, only one of the elements points to a pure `Text` object; the rest of the elements point to `FancyText` or `FixedText` objects.

Why must we use pointers to objects rather than the objects themselves to achieve polymorphism? Remember that a pointer stores a memory address (see Section 10.6). All pointers, no matter what type they point to, are all the same size (4 bytes on 32-bit systems and 8 bytes on 64-bit systems). `Text` objects and `FancyText` objects are not the same size (see Figure 17.1 for a conceptual picture); `FancyText` objects are bigger, containing three extra string fields. All the elements of a vector must be the same size. If we made `texts` a vector of `Text` objects rather than a vector of pointers to `Text` objects, when we assign a `FancyText` object to an element in the `texts` vector, the assignment would slice the extra fields in the `FancyText` object. Pointer assignment avoids the slicing problem.

The main reason for using pointers is that C++ uses static binding for all methods (virtual and non-virtual) invoked on behalf of an object; the compiler chooses the method based on the declared type of the

object. In contrast, C++ uses dynamic binding for virtual method calls made via pointers to objects; the exact type of the object determines the method selection.

A polymorphic method in C++ requires four key ingredients:



1. The method must appear in a class that is part of an inheritance hierarchy.
2. The method must be declared **virtual** in the base class at the top of the hierarchy.
3. Derived classes override the behavior of the inherited virtual methods as needed.
4. Clients must invoke the method via a pointer to an object, not directly through the object itself.

In summary, polymorphism requires inheritance and the *is a* relationship. The base class defines the method signature, and the derived classes override the method with their own custom behavior.

[adapter design pattern example]

17.5 Protected Members

As a more practical, yet still relatively simple example, suppose we need to model two-dimensional shape objects, such as rectangles, squares, triangles, ellipses, and circles. Our application will determine how to pack parts into a containing area, so it is important to be able to determine an individual shape object's area. Figure 17.5 illustrates packing some two-dimensional shapes into a two-dimensional container. We thus want to determine the minimum area of the container that can hold a given collection of parts. A real-world application could be computing the size of the smallest circuit board that can hold a collection of electronic components. Our program will be much simpler and give us only a rough approximation; it does not take into account the geometry and orientation of the individual shapes but merely computes the total area of the components. The actual answer generally will be larger than the result computed by our program.

An interesting problem arises if a method in a derived class needs to use an inherited variable. If the variable is declared **private** in the base class, a derived class method cannot access it. The **private** specifier means “not available outside the class, period.” We know it is unwise in general to make instance variables public, since that breaks encapsulation. Since encapsulation is a desirable property, C++ provides a third level of access protection within a class—**protected**. A **protected** member, whether it be data or a method, is inaccessible to all code outside the class, *except* for code within a derived class. The use of **protected** is illustrated in the shapes code.

We begin by defining the base class for all shape objects: **Shape**. Listing 17.5 (shape.h) provides the code for **Shape**.

Listing 17.5: shape.h

```
#ifndef _SHAPE_H_
#define _SHAPE_H_
```

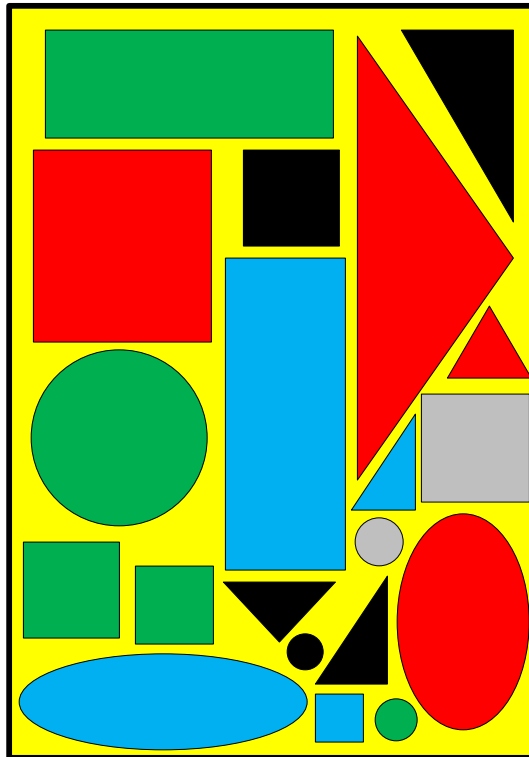


Figure 17.5: Packing Two-dimensional Shapes into a Two-dimensional Container

```

/*
 * Shape is the base class for all shapes
 */
class Shape {
public:
    // Longest distance across the shape
    virtual double span() const = 0;
    // The shape's area
    virtual double area() const = 0;
};

#endif

```

The “assignment to zero” of the two virtual methods in the **Shape** class make them special; they are called *pure virtual methods* or *pure virtual functions*. This special syntax signifies that these methods have no implementations. It is not possible to provide a body for these methods. A class that contains at least one pure virtual method is an *abstract class*. It is not possible to create an instance of an abstract class. The compiler enforces this restriction; the following statement;

```
Shape myShape;    // Illegal
```

is illegal. A non-abstract class is called a *concrete class*. All the classes we have seen to this point except for **Shape** have been concrete classes.

An abstract class represents an abstract concept. *Shape* is an abstract concept. We can have circles, rectangles, and lots of other kinds of shapes, but can we have something that is “just a shape” without being a particular kind of shape? Even though we cannot create instances of abstract classes, abstract classes are useful for organizing a class hierarchy.

We can derive a concrete class from our **Shape** class as shown in Listing 17.6 (rectangle.h).

Listing 17.6: rectangle.h

```
#ifndef _RECTANGLE_H_
#define _RECTANGLE_H_

#include "shape.h"

class Rectangle: public Shape {
protected:
    double length;
    double width;
public:
    Rectangle(double len, double wid);
    // Length of the longer side
    double span() const override;
    double area() const override;
};

#endif
```

Note the appearance of the **protected** access specifier. The methods in any class derived directly or indirectly from **Rectangle** will be able to access the **length** and **width** fields. These fields will be inaccessible to code outside these classes. Listing 17.7 (rectangle.cpp) provides the implementation of **Rectangle**’s methods.

Listing 17.7: rectangle.cpp

```
// File rectangle.cpp

#include "rectangle.h"
#include <algorithm> // For max function

// Generally for rectangles length >= width, but the
// constructor does not enforce this.
Rectangle::Rectangle(double len, double wid): length(len), width(wid) {}

// Length of the longer side--determine which is longer
double Rectangle::span() const {
    return std::max(length, width);
}

double Rectangle::area() const {
    return length * width;
}
```

From mathematics we know that a square is a special kind of rectangle, so Listing 17.8 (square.h) and Listing 17.9 (square.cpp) specify a **Square** class.

Listing 17.8: square.h

```
#ifndef _SQUARE_H_
#define _SQUARE_H_

#include "rectangle.h"

// A square is a special case of a rectangle

class Square: public Rectangle {
public:
    // Length and width are equal in a square, so specify the
    // length of only one side
    Square(double side);
    // The inherited methods work as is; no need to
    // change their behavior.
};

#endif
```

Listing 17.9: square.cpp

```
// File square.cpp

#include "square.h"

// Defer the work of initialization to the base class constructor
Square::Square(double side): Rectangle(side, side) {}
```

Next, we add a triangle shape (Listing 17.10 (triangle.h) and Listing 17.11 (triangle.cpp)).

Listing 17.10: triangle.h

```
#ifndef _TRIANGLE_H_
#define _TRIANGLE_H_

#include "shape.h"

class Triangle: public Shape {
protected:
    double side1;    // Triangles have three sides
    double side2;
    double side3;
public:
    Triangle(double s1, double s2, double s3);
    double span() const override;
    double area() const override;
};

#endif
```

Listing 17.11: triangle.cpp

```
// File triangle.cpp

#include "triangle.h"
#include <algorithm> // For max function

Triangle::Triangle(double s1, double s2, double s3) :
    side1(s1), side2(s2), side3(s3) {}

// The span of a triangle is the length of the longest side
double Triangle::span() const {
    return std::max(side1, std::max(side2, side3));
}

// Not having the base and height of the triangle explicitly, we
// use Heron's formula to compute the area
double Triangle::area() const {
    // Compute semiperimeter
    double s = (side1 + side2 + side3)/2;
    // Compute area using Heron's formula
    return s*(s - side1)*(s - side2)*(s - side3);
}
```

An ellipse is a simple curved shape that we can add to our class hierarchy. Listing 17.12 (ellipse.h) and Listing 17.13 (ellipse.cpp) define the ellipse class.

Listing 17.12: ellipse.h

```
#ifndef _ELLIPSE_H_
#define _ELLIPSE_H_

#include "shape.h"

class Ellipse: public Shape {
protected:
    double major_radius; // The longer radius of the ellipse
    double minor_radius; // The shorter radius of the ellipse
public:
    Ellipse(double major, double minor);
    double span() const override;
    double area() const override;
};

#endif
```

Listing 17.13: ellipse.cpp

```
// File ellipse.cpp

#include "ellipse.h"
#include <algorithm> // For max function

// PI is local to this file
```

```

static const double PI = 3.14159;

// Note: This constructor does not enforce
// major_axis >= minor_axis
Ellipse::Ellipse(double major, double minor):
    major_radius(major), minor_radius(minor) {}

// Greatest distance across is the length of the longer radius
double Ellipse::span() const {
    return std::max(major_radius, minor_radius);
}

double Ellipse::area() const {
    return PI * major_radius * minor_radius;
}

```

A circle is just an ellipse with equal major and minor radii. Listing 17.14 (circle.h) and Listing 17.15 (circle.cpp) define the **Circle** class.

Listing 17.14: circle.h

```

#ifndef _CIRCLE_H_
#define _CIRCLE_H_

#include "ellipse.h"

// A circle is a special case of an ellipse

class Circle: public Ellipse {
public:
    // In a circle the major and minor radii are the same, so
    // we need specify only one value when creating a circle.
    Circle(double radius);
    // Inherited methods work as is, no need to change their
    // behavior.
};

#endif

```

Listing 17.15: circle.cpp

```

// File circle.cpp

#include "circle.h"

// PI is local to this file
static const double PI = 3.14159;

Circle::Circle(double radius): Ellipse(radius, radius) {}

```

These shape classes form the class hierarchy shown in Figure 17.6.

Listing 17.16: testshapes.cpp

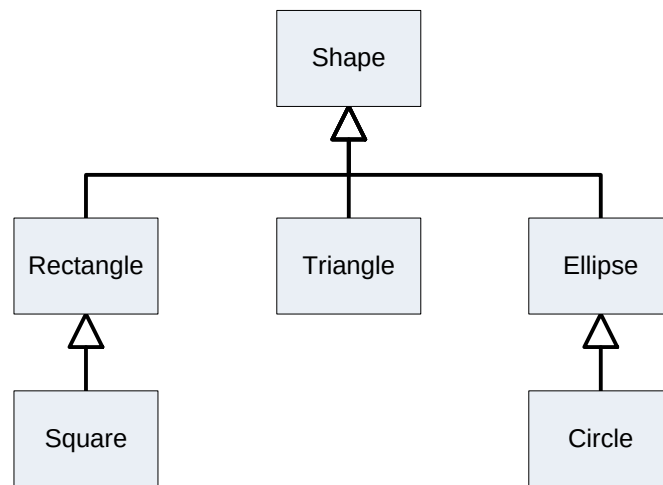


Figure 17.6: Inheritance hierarchy of shape classes

```
// File testshapes.cpp

#include <iostream>
#include <vector>

#include "rectangle.h"
#include "triangle.h"
#include "circle.h"
#include "ellipse.h"

using namespace std;

int main() {
    Rectangle rect(3, 4);
    Circle circ(4.5);
    Triangle tri(3, 4, 5);
    Ellipse elli(3, 4);
    vector<Shape *> shape_list;

    shape_list.push_back(&circ);
    shape_list.push_back(&tri);
    shape_list.push_back(&rect);
    shape_list.push_back(&elli);

    int n = shape_list.size();
    double area_total = 0.0, max_span = 0.0;

    for (int i = 0; i < n; i++) {
```



```

    // Examine the area each shape
    cout << "Area = " << shape_list[i]->area() << endl;
    // Accumulate the areas of all the shapes
    area_total += shape_list[i]->area();
    // Account for the longest object
    if (max_span < shape_list[i]->span())
        max_span = shape_list[i]->span();
}
// Report the total area of all the shapes combined
cout << "Total shape area is " << area_total << endl;
// Report the minimum length of the container
cout << "Longest shape is " << max_span << endl;
}

```

Observe that neither **Square** nor **Circle** needed access to the protected fields of their base classes. Consider Listing 17.17 (`drawablerectangle.cpp`) that derives a new class from **Rectangle** to allow clients to draw rectangle objects in a console window using text graphics.

Listing 17.17: `drawablerectangle.cpp`

```

#include <iostream>
#include "rectangle.h"

class DrawableRectangle: public Rectangle {
public:
    // Delegate construction to the base class
    DrawableRectangle(double length, double width):
        Rectangle(length, width) {}

    // Draw a rectangle using text graphics
    void draw() const {
        // Access the inherited protected fields
        int rows = static_cast<int>(length + 0.5),
            columns = static_cast<int>(width + 0.5);
        // Draw the rectangle
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < columns; c++)
                std::cout << '#';
            std::cout << std::endl;
        }
    }
};

int main() {
    DrawableRectangle rec1(3, 2),
                      rec2(10, 5),
                      rec3(4, 8);

    rec1.draw();
    std::cout << "-----" << std::endl;
    rec2.draw();
    std::cout << "-----" << std::endl;
    rec3.draw();
}

```

Listing 17.17 (`drawablerectangle.cpp`) displays

```

##
##
##
-----
####
####
####
####
####
####
####
####
####
####
-----
#####
#####
#####
#####

```

The **DrawableRectangle::draw** method needs access to the fields **length** and **width** to be able to draw the rectangle. It is important to remember that every **DrawableRectangle** object contains these fields because it inherits them from **Rectangle**; however, if the **length** and **width** fields were declared private instead of protected in **Rectangle**, **DrawableRectangle::draw** would not be able to access these fields.

A class supports three levels of protection for its members:



1. **private**: This is the default. Private members are accessible to any code within the class itself but are inaccessible to code outside of the class.
2. **protected**: Protected members are accessible to code within the class itself and to code within derived classes that use public inheritance. Protected members are inaccessible to other (non-derived) classes.
3. **public**: Public members are accessible to code anywhere.

In sum, private means not accessible outside the class, period. Public means open to everyone. Protected means “public” to subclasses and “private” to all other classes.

17.6 Fine Tuning Inheritance

Recall the flexible sorting code found in Listing 16.5 (loggingflexiblesort.cpp). It uses objects to enable the following functionality:

- The ordering imposed by a selection sort function can be varied by using custom comparison functions.
- The object used to determine the sort's ordering collects data about the number of comparisons and element interchanges the sort function performs.

Listing 17.20 (polymorphicsort.cpp) provides a slight variation of Listing 16.5 (loggingflexiblesort.cpp). Besides splitting the code up into multiple source files, it adds two virtual methods that enable future developers to customize the **compare** and **swap** methods in derived classes.

Listing 17.18 (comparer.h) contains the declaration of the **Comparer** class.

Listing 17.18: comparer.h

```
#ifndef _COMPARER_H_
#define _COMPARER_H_
/*
 * Comparer objects manage the comparisons and element
 * interchanges on the selection sort function below.
 */
class Comparer {
    // The object's data is private, so it is inaccessible to
    // clients and derived classes

    // Keeps track of the number of comparisons
    // performed
    int compare_count;

    // Keeps track of the number of swaps performed
    int swap_count;

    // Function pointer directed to the function to
    // perform the comparison
    bool (*comp)(int, int);

protected:
    // Method that actually performs the comparison
    // Derived classes may customize this method
    virtual bool compare_impl(int m, int n);

    // Method that actually performs the swap
    // Derived classes may customize this method
    virtual void swap_impl(int& m, int& n);

public:
    // The client must initialize a Comparer object with a
    // suitable comparison function.
    Comparer(bool (*f)(int, int));

    // Resets the counters to make ready for a new sort
    void reset();

    // Method that performs the comparison. It delegates
    // the actual work to the function pointed to by comp.
    // This method logs each invocation.
    bool compare(int m, int n);
}
```

```

    // Method that performs the swap.
    // Interchange the values of
    // its parameters a and b which are
    // passed by reference.
    // This method logs each invocation.
    void swap(int& m, int& n);

    // Returns the number of comparisons this object has
    // performed since it was created.
    int comparisons() const;

    // Returns the number of swaps this object has
    // performed since it was created.
    int swaps() const;
};
#endif

```

Listing 17.19 (comparer.cpp) provides the implementation of the **Comparer** class methods.

Listing 17.19: comparer.cpp

```

#include "comparer.h"

// Method that actually performs the comparison
// Derived classes may customize this method
bool Comparer::compare_impl(int m, int n) {
    return comp(m, n);
}

// Method that actually performs the swap
// Derived classes may customize this method
void Comparer::swap_impl(int& m, int& n) {
    int temp = m;
    m = n;
    n = temp;
}

// The client must initialize a Comparer object with a
// suitable comparison function.
Comparer::Comparer(bool (*f)(int, int)) :
    compare_count(0), swap_count(0), comp(f) {}

// Resets the counters to make ready for a new sort
void Comparer::reset() {
    compare_count = swap_count = 0;
}

// Method that performs the comparison. It delegates
// the actual work to the function pointed to by comp.
// This method logs each invocation.
bool Comparer::compare(int m, int n) {
    compare_count++;
    return compare_impl(m, n);
}

```

```

// Method that performs the swap.
// Interchange the values of
// its parameters a and b which are
// passed by reference.
// This method logs each invocation.
void Comparer::swap(int& m, int& n) {
    swap_count++;
    swap_impl(m, n);
}

// Returns the number of comparisons this object has
// performed since it was created.
int Comparer::comparisons() const {
    return compare_count;
}

// Returns the number of swaps this object has
// performed since it was created.
int Comparer::swaps() const {
    return swap_count;
}

```

Notice that even though the **virtual** keyword appears before the method declarations in `comparer.h`, it does not appear before the method implementations in `comparer.cpp`.

Listing 17.20 (`polymorphicsort.cpp`) provides the client code that tests the new **Comparer** class.

Listing 17.20: `polymorphicsort.cpp`

```

#include <iostream>
#include <vector>
#include "comparer.h"

using namespace std;

/*
 * selection_sort(a, compare)
 *     Arranges the elements of vector a in an order determined
 *     by the compare object.
 *     a is a vector of ints.
 *     compare is a function that compares the ordering of
 *     two integers.
 *     The contents of a are physically rearranged.
 */
void selection_sort(vector<int>& a, Comparer& compare) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Note: i, small, and j represent positions within a
        // a[i], a[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        int small = i;
        // See if a smaller value can be found later in the array
    }
}

```

```

        for (int j = i + 1; j < n; j++)
            if (compare.compare(a[j], a[small]))
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            compare.swap(a[i], a[small]);
    }
}

/*
 * print
 * Prints the contents of an integer vector
 * a is the vector to print.
 * a is not modified.
 */
void print(const vector<int>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << '}';
}

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
bool less_than(int a, int b) {
    return a < b;
}

/*
 * greater_than(a, b)
 * Returns true if a > b; otherwise, returns
 * false.
 */
bool greater_than(int a, int b) {
    return a > b;
}

int main() {
    // Make a vector of integers
    vector<int> original { 23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10 };

    // Make a working copy of the original vector
    vector<int> working = original;
    cout << "Before: ";
    print(working);
    cout << endl;
    Comparer lt(less_than), gt(greater_than);
    selection_sort(working, lt);
}

```

```

cout << "Ascending: ";
print(working);
cout << " (" << lt.comparisons() << " comparisons, "
    << lt.swaps() << " swaps)" << endl;
cout << "-----" << endl;
// Make another copy of the original vector
working = original;
cout << "Before: ";
print(working);
cout << endl;
selection_sort(working, gt);
cout << "Descending: ";
print(working);
cout << " (" << gt.comparisons() << " comparisons, "
    << gt.swaps() << " swaps)" << endl;
cout << "-----" << endl;
// Sort a sorted vector
cout << "Before: ";
print(working);
cout << endl;
// Reset the greater than comparer so we start counting at
// zero
gt.reset();
selection_sort(working, gt);
cout << "Descending: ";
print(working);
cout << " (" << gt.comparisons() << " comparisons, "
    << gt.swaps() << " swaps)" << endl;
}

```

The functions in Listing 17.20 (polymorphicsort.cpp) are identical to the functions in Listing 16.5 (loggingflexiblesort.cpp), and, not surprisingly, the output of Listing 17.20 (polymorphicsort.cpp) is identical to the output of Listing 16.5 (loggingflexiblesort.cpp):

```

Before: {23,-3,4,215,0,-3,2,23,100,88,-10}
Ascending: {-10,-3,-3,0,2,4,23,23,88,100,215} (55 comparisons, 7 swaps)
-----
Before: {23,-3,4,215,0,-3,2,23,100,88,-10}
Descending: {215,100,88,23,23,4,2,0,-3,-3,-10} (55 comparisons, 5 swaps)
-----
Before: {215,100,88,23,23,4,2,0,-3,-3,-10}
Descending: {215,100,88,23,23,4,2,0,-3,-3,-10} (55 comparisons, 0 swaps)

```

The question arises: What advantage does the new **Comparer** class have over the original one?

The design of this new **Comparer** class is interesting:

- Since its counter fields are private, only methods in the **Comparer** class itself can access a **Comparer** object's data.
- The two virtual methods, `compare_impl` and `swap_impl`, are protected, so clients cannot access them directly. Derived classes, however, can see them and override them. The suffix `_impl` stands

for “implementation,” so `compare_impl` represents the implementation details of the `compare` method and `swap_impl` represents the implementation details of the `swap` method.

- The public `compare` and `swap` methods both delegate part of their work to the protected `compare_impl` and `swap_impl` methods.
- The `compare` and `swap` methods are not declared `virtual`, so derived classes cannot override them.
- The `compare` and `swap` methods manage the `compare_count` and `swap_count` counter fields. Since derived classes cannot see these fields, there is nothing that a designer of a derived class can do when overriding `compare_impl` or `swap_impl` to disturb the correct accounting of the number of times a client calls `compare` or `swap`.
- The `comparisons` and `swaps` methods that report the results to the client are non-virtual, so derived classes may not override their behavior.

Observe that the designer of the `Comparer` class allows the nature of the comparisons and swaps in derived classes to be flexible, but it is rigid on the enforcement of how the accounting is performed and reported. The proper use of `protected` and `private` specifiers in a base class as shown in `Comparer` affords class designers a great deal of control over exactly what derived class designers can do. Derived classes may adapt some behaviors, but other behaviors are non-negotiable.

What kind of customization would a programmer want to do to the `Comparer` class beyond changing how the comparison is performed? Consider the `LogComparer` class declared in Listing 17.21 (`logcomparer.h`).

Listing 17.21: `logcomparer.h`

```
#ifndef _LOGCOMPARER_H_
#define _LOGCOMPARER_H_

#include <fstream>
#include <string>
#include "comparer.h"

/*
 * Comparer objects manage the comparisons and element
 * interchanges on the selection sort function below.
 */
class LogComparer: public Comparer {
    // Output stream to which logging messages are directed
    std::ofstream fout;

protected:
    // Method that actually performs the comparison
    bool compare_impl(int m, int n) override;

    // Method that actually performs the swap
    void swap_impl(int& m, int& n) override;

public:
    // The client must initialize a LogComparer object with a
    // suitable comparison function and the file name of a text
    // file to which the object will direct logging messages
```



```

    LogComparer(bool (*f)(int, int), const std::string& filename);

    // The destructor must close the log file
    ~LogComparer();
};

#endif

```

The `LogComparer` class overrides the `compare_impl` and `swap_impl` methods. Instances of the `LogComparer` class use an `ofstream` object to record logging information to a text file.

The implementation of the `LogComparer` methods can be found in Listing 17.22 (`logcomparer.cpp`).

Listing 17.22: `logcomparer.cpp`

```

#include "logcomparer.h"
#include <cstdlib>
#include <iostream>

using namespace std;

// Method that actually performs the comparison
// Derived classes may override this method
bool LogComparer::compare_impl(int m, int n) {
    fout << "Comparing " << m << " to " << n << std::endl;
    // Base class method does the comparison
    return Comparer::compare_impl(m, n);
}

// Method that actually performs the swap
// Derived classes may override this method
void LogComparer::swap_impl(int& m, int& n) {
    fout << "Swapping " << m << " and " << n << endl;
    int temp = m;
    m = n;
    n = temp;
}

// The client must initialize a LogComparer object with a
// suitable comparison function and the file name of the
// text file to receive logging messages.
LogComparer::LogComparer(bool (*f)(int, int), const string& filename):
    Comparer(f) {
    fout.open(filename);
    if (!fout.good()) {
        fout.close();
        cout << "Could not open log file " << filename
              << " for writing" << endl;
        exit(1); // Terminate the program
    }
}

// Destructor closes the log file
LogComparer::~LogComparer() {
    fout.close();
}

```

The constructor attempts to open a file output stream to write logging information to a text file. If the constructor cannot for any reason open the text file for writing, it terminates the program's execution. The destructor closes the file stream object when the **LogComparer** object's life is over.

The two overridden methods, **compare_impl** and **swap_impl**, write text to the log file. Both methods use the **LogComparer** object's **fout** field rather than the **cout** console output stream object.

Notice that even though the **override** keyword appears after the method declarations in **logcomparer.h**, it does not appear after the method implementations in **logcomparer.cpp**.

The client code in Listing 17.23 (**loggingsort.cpp**) uses a **LogComparer** object to create a text file named **sort.log**.

Listing 17.23: **loggingsort.cpp**

```
#include <iostream>
#include <vector>
#include "logcomparer.h"

using namespace std;

/*
 *  selection_sort(a, compare)
 *      Arranges the elements of vector a in an order determined
 *      by the compare object.
 *      a is a vector of ints.
 *      compare is a function that compares the ordering of
 *      two integers.
 *      The contents of a are physically rearranged.
 */
void selection_sort(vector<int>& a, Comparer& compare) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        // Note: i, small, and j represent positions within a
        // a[i], a[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        int small = i;
        // See if a smaller value can be found later in the array
        for (int j = i + 1; j < n; j++)
            if (compare.compare(a[j], a[small]))
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            compare.swap(a[i], a[small]);
    }
}

/*
 *  print
 *      Prints the contents of an integer vector
 *      a is the vector to print.
 */
```

```

/*      a is not modified.
*/
void print(const vector<int>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << '>';
}

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
*/
bool less_than(int a, int b) {
    return a < b;
}

int main() {
    // Make a vector of integers from an array
    int a[] = { 23, -3, 4, 215, 0, -3, 2 };
    int len = (sizeof a)/(sizeof a[0]);
    vector<int> vec(a, a + len);

    // Make a working copy of the original vector
    cout << "Before:  ";
    print(vec);
    cout << endl;
    LogComparer lt(less_than, "sort.log");
    selection_sort(vec, lt);
    cout << "Ascending: ";
    print(vec);
    cout << " (" << lt.comparisons() << " comparisons, "
        << lt.swaps() << " swaps)" << endl;
}

```

Listing 17.23 (loggingsort.cpp) prints to the console

```

Before:  {23,-3,4,215,0,-3,2}
Ascending:  {-3,-3,0,2,4,23,215} (21 comparisons, 4 swaps)

```

and writes the file sort.log the text

```

Comparing -3 to 23
Comparing 4 to -3
Comparing 215 to -3
Comparing 0 to -3

```

```

Comparing -3 to -3
Comparing 2 to -3
Swapping 23 and -3
Comparing 4 to 23
Comparing 215 to 4
Comparing 0 to 4
Comparing -3 to 0
Comparing 2 to -3
Swapping 23 and -3
Comparing 215 to 4
Comparing 0 to 4
Comparing 23 to 0
Comparing 2 to 0
Swapping 4 and 0
Comparing 4 to 215
Comparing 23 to 4
Comparing 2 to 4
Swapping 215 and 2
Comparing 23 to 4
Comparing 215 to 4
Comparing 215 to 23

```

If during development the selection sort function has a problem, the programmer can review the contents of the log file to examine how the sort progresses and perhaps determine where the problem lies.

The design of the **Comparer** class provides an overall structure that allows inheriting classes to fine tune the details without disturbing the overarching framework. Inheritance and polymorphism allow us to add functionality to an existing code base. Virtual methods provide extension points by which derived classes can add custom behavior.

17.7 Summary

- A new class can be derived from an existing class via *inheritance*.
- Other names for inheritance include *subclassing*, *derivation*, and *specialization*.
- In the code

```

class Gadget: public Widget {
    // Details omitted
};

```

- **Widget** is the *base class* (also called the *superclass*), and **Gadget** is the *derived class* (also called the *subclass*).
 - **Gadget** inherits all the instance variables and methods from **Widget**.
 - Code within the **Gadget** class can see and use the public and protected members inherited from **Widget**.
- The file streams **ofstream** and **ifstream** are subclasses of, respectively, **ostream** and **istream**. Because of the *is a* relationship imposed by inheritance, an **ofstream** object can be used in any

context that expects an **ostream** object reference or pointer, and **ifstream** objects can be used where an **istream** object is required.

- The specifier **protected** works like “private” as far as most code outside the class is concerned, but it works like “public” to code within derived classes. Subclass methods can see and manipulate inherited protected data and call inherited protected methods, but subclasses cannot directly see inherited private data and cannot call directly inherited private methods.
- *Polymorphism* in object-oriented languages is a process by which the code executed during a method invocation is determined at run time, not compile time.
- Polymorphism in C++ requires three key ingredients:
 - inheritance—a derived class overrides a method in its base class
 - virtual methods—an overridden **virtual** method can use dynamic binding (code to execute is determined at run time) rather than static binding (compiler determines the code to execute)
 - method calls through pointers or references—a pointer’s declared type is of the base class, but the actual type of the object to which it points may be a derived class. The *is a* relationship enables the process to work correctly.
- A derived class may override a public method that is declared virtual in the base class. Such virtual methods enable polymorphism.
- A *pure virtual* method (or pure virtual function) has no defined body and is indicated by the “= 0” syntax.
- A class that contains at least one pure virtual function is known as an *abstract* class. It is illegal to attempt to create an instance of an abstract class. Abstract classes serve to organize hierarchies of classes related through inheritance. Classes that are not abstract are called *concrete* classes.

17.8 Exercises

1. Overload the **operator<<** function so it accepts an **ostream** reference and a two-dimensional vector of Boolean values. The function should print the contents of the 2D vector in a table; for example, a four-row, five column print out might look like:

```

true  false  true  true  true
false true   false false false
false false  false false false
true  false  true  true  true

```

Note that it prints the words *true* and *false*, not simply 0 or 1. Once you have tested your function using the **cout** object, demonstrate that you can save the contents of any 2D Boolean vector to a text file.

2. Consider the following C++ code:

```

class Widget {
public:
    virtual int f() { return 1; }
}

```

```
};

class Gadget: public Widget {
public:
    virtual int f() { return 2; }
};

class Gizmo: public Widget {
public:
    virtual int f() { return 3; }
};

void do_it(Widget *w) {
    cout << w->f() << " ";
}

int main() {
    vector<Widget *> widgets;
    Widget wid;
    Gadget gad;
    Gizmo giz;
    widgets.push_back(&wid);
    widgets.push_back(&gad);
    widgets.push_back(&giz);
    for (size_t i = 0; i < widgets.size(); i++)
        do_it(widgets[i]);
}
```

- (a) What does the program print?
- (b) Would the program still compile and run if the `f` method within the `Widget` class were a pure virtual function?
- (c) How would the program run differently if the `virtual` keyword were removed from all the code?
- (d) Would the program behave the same if the `virtual` keyword were removed from all the classes except for `Widget`?

Chapter 18

Generic Programming

CAUTION! CHAPTER UNDER CONSTRUCTION

The flexible sorting examples in Listing 12.2 (`flexibleintsort.cpp`), Listing 16.5 (`loggingflexiblesort.cpp`), and Listing 17.20 (`polymorphicsort.cpp`) allow us to arrange the elements in integer vectors in creative ways and perform other interesting activities such as logging. Those examples demonstrate the power of function pointers, inheritance, and polymorphism. As flexible and powerful as these techniques are, they all contain one major limitation: they deal with vectors of *integers* only. Consider the **Comparer** class from Listing 17.18 (`comparer.h`) and Listing 17.19 (`comparer.cpp`), and the “flexible” **selection_sort** function from Listing 17.23 (`loggingsort.cpp`). The **selection_sort** function accepts a vector of integers and a **Comparer** reference. What if we need to sort a vector of double-precision floating-point numbers or a vector of **string** objects? Unfortunately, all of this “flexible” code cannot handle this seemingly minor variation.

In this chapter we look at C++’s template mechanism that enables programmers to develop truly generic algorithms and data structures. We will see how the standard C++ library has embraced the template technology to provide a wealth of generic algorithms and data structures that greatly assist developers in the construction of quality software.

18.1 Function Templates

Consider the following comparison function found in Listings 12.2, 16.5, and 17.23:

```
/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
bool less_than(int a, int b) {
    return a < b;
}
```

Listing 18.1 (`testlessthan.cpp`) tests the **less_than** function with various arguments.

Listing 18.1: testlessthan.cpp

```
#include <iostream>

using namespace std;

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
bool less_than(int a, int b) {
    return a < b;
}

int main() {
    cout << less_than(2, 3) << endl;
    cout << less_than(2.2, 2.7) << endl;
    cout << less_than(2.7, 2.2) << endl;
}
```

The compiler generates warnings for the last two statements. The `less_than` function expects two integer arguments, but the calling code sends two double-precision floating-point values. The automatic conversion from `double` to `int` can lose information, hence the warnings. The output of Listing 18.1 (testlessthan.cpp) shows that we should take these warnings seriously:

```
1
0
0
```

Obviously $2 < 3$ and $2.7 \not< 2.2$, but $2.2 \not< 2.7$? The automatic `double` to `int` conversion truncates, so the `less_than` function treats both 2.2 and 2.7 as 2, and clearly $2 \not< 2$.

The situation is even worse for the following code:

```
string word1 = "ABC", word2 = "XYZ";
cout << (word1 < word2) << endl;
cout << (word2 < word1) << endl;
// The next statement will not compile
//cout << less_than(word1, word2) << endl;
```

The C++ standard library provides a global `operator<` function that compares two `std::string` objects lexicographically. The first two `cout` statements produce the expected output:

```
1
0
```

but the last line that is commented out will not compile if we remove the commenting symbols. This is because the C++ library does not provide a standard conversion from `std::string` to `int`, and `less_than` requires two `int` parameters.

As another example, consider the following function that computes the sum of the elements in a vector of integers:

```
int sum(const vector<int>& v) {
    int result = 0;
    for (int elem : v)
        result += elem;
    return result;
}
```

The following client code works well:

```
vector<int> v {10, 20, 30};
cout << sum(v) << endl;
```

and prints 60. The following code does not compile:

```
vector<double> v {10, 20, 30};
cout << sum(v) << endl;
```

The second code fragment attempts to pass a vector of double-precision floating-point values to the `sum` function. Unfortunately, the `sum` function accepts only vectors that contain integers. More precisely, `sum` accepts only arguments of type `vector<int>`, and a `vector<double>` object is not a `vector<int>`. The solution is easy; just copy and paste the original `sum` function and change all the occurrences of “`int`” to “`double`,” creating an overloaded `sum` function:

```
double sum(const vector<double>& v) {
    double result = 0;
    for (double elem : v)
        result += elem;
    return result;
}
```

This works, but the duplicated effort is unsatisfying. These two overloaded `sum` functions are identical except for the types involved. The actions of the two functions—the initialization, vector traversal and arithmetic—are essentially the same. In general, code duplication is undesirable. A programmer that discovers and repairs an error in one of the functions must remember to apply the corresponding correction to the function’s overloaded counterpart.

Both the `less_than` and `sum` functions share a common shortcoming: they are tied in some way to a particular data type. It would be convenient to be able to specify the common pattern and let the compiler fill in the types as required. In the case of `less_than` we would like to be able to write a function that can use the `<` operator on parameters of any type for which `<` is compatible. For `sum`, we would like to create a generic function that works for vectors containing any numeric type.

C++ enables programmers to write such generic functions via *templates*. A function template specifies a pattern of code, and either the programmer or the compiler supplies the exact type as needed. Listing 18.2 (templatelessthan.cpp) uses C++’s template mechanism to create a generic `less_than` function.

Listing 18.2: templatelessthan.cpp

```
#include <iostream>
#include <string>

using namespace std;
```

```

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
template <typename T>
bool less_than(T a, T b) {
    return a < b;
}

int main() {
    cout << less_than(2, 3) << endl;
    cout << less_than(2.2, 2.7) << endl;
    cout << less_than(2.7, 2.2) << endl;

    string word1 = "ABC", word2 = "XYZ";
    cout << less_than(word1, word2) << endl;
    cout << less_than(word2, word1) << endl;
}

```

The output of Listing 18.2 (templatelessthan.cpp) is

```

1
1
0
1
0

```

In the `less_than` function definition:

```

template <typename T>
bool less_than(T a, T b) {
    return a < b;
}

```

`template` and `typename` are reserved words, and `T` is a *type parameter*. The `template` keyword indicates that the function definition that follows is not a normal function definition but rather is a pattern or template from which the compiler can attempt to produce the correct function definition. A function template is also known as a *generic function*. The `typename` keyword indicates that the identifier that follows (in this case `T`) is a placeholder for a C++ type name. The type parameter `T` stands for an actual type to be determined elsewhere. `T` is an identifier and so can have any name legal for a variable, function, or class; some programmers prefer more descriptive names such as `CompareType` or `TypeParam`. Single capital letters such as `T`, `S`, `U`, and `V` are popular type parameter names among C++ programmers. Just as a regular parameter of a function or method represents a value, a template parameter represents a type.

You may use the reserved word `class` interchangeably with the keyword `typename` in this context:

```

template <class T>
bool less_than(T a, T b) {
    return a < b;
}

```

In Listing 18.2 (templatelessthan.cpp) the compiler generates three different **less_than** functions: one that accepts two integers, one that accepts two double-precision floating-point values, and one that accepts two string objects. Although the programmer cannot see them in the source code, the compiler in effect creates the following literal function definitions:

```
bool less_than(int a, int b) {
    return a < b;
}

bool less_than(double a, double b) {
    return a < b;
}

bool less_than(string a, string b) {
    return a < b;
}
```

When the compiler generates an actual function definition from a template function, the process is called *template instantiation*, not to be confused with an object instantiation from a class.

The compiler instantiates functions only as needed. If Listing 18.2 (templatelessthan.cpp) did not contain the **std::string** objects, when processing the source code the compiler would not instantiate the version of **less_than** that accepts two **std::string** objects.

For efficiency purposes, the better way to write **less_than** is

```
template <typename T>
bool less_than(const T& a, const T& b) {
    return a < b;
}
```

Notice that parameters **a** and **b** are passed by reference, not by value. This means the caller passes only the addresses of objects **a** and **b** and does not need to make copies to send to the function. This can make a big difference if **T** represents large objects. Since the parameters are **const** references, the function cannot change the state of the caller's actual parameters; therefore, the function offers call-by-value safety without the overhead of copying the parameters.

If a caller attempts a statement such as

```
cout << less_than(2, 2.2) << endl;
```

the compiler generates an error. This is because the template definition specifies that its two type parameters are identical. The literal **2** is an **int**, and the literal **2.2** is a **double**. The compiler can automatically instantiate **less_than** functions that accept two **ints** or two **doubles** but not mixed types. In the expression

```
less_than(2, 2.2)
```

the compiler cannot know which instantiated version of the template to call.

One solution requires the programmer to explicitly instantiate the function as shown here:

```
cout << less_than<int>(2, 2.2) << endl;
```

This forces the compiler to instantiate and call the integer version. The compiler will issue a warning about truncating the 2.2. The statement

```
cout << less_than<double>(2, 2.2) << endl;
```

would use the double-precision floating-point version of `less_than`, and it compiles cleanly with no warnings since widening the `int` 2 to a `double` is automatic.

We can rewrite the `less_than` template function to avoid the need for explicit instantiation. For the most flexibility, we can express the `less_than` template function as

```
template <typename T, typename V>
bool less_than(const T& a, const V& b) {
    return a < b;
}
```

This version uses two type parameters, `T` and `V`, and they can represent different types. With this new definition all of the following statements are acceptable:

```
cout << less_than(2, 3) << endl;
cout << less_than(2.2, 2.7) << endl;
cout << less_than(2, 2.2) << endl;
cout << less_than(2.2, 2) << endl;
```

No explicit template instantiation is necessary. Given this calling code, the compiler will automatically instantiate four overloaded `less_than` functions:

- `less_than(2, 3)` instantiates `bool less_than(const int& a, const int& b)`
- `less_than(2.2, 2.7)` instantiates `bool less_than(const double& a, const double& b)`
- `less_than(2, 2.2)` instantiates `bool less_than(const int& a, const double& b)`
- `less_than(2.2, 2)` instantiates `bool less_than(const double& a, const int& b)`

While this more flexible `less_than` template function allows us to compare mixed numeric expressions, in general it makes more sense to expect the two parameters to be the same type.

Returning to our vector summation function, we can express `sum` generically as

```
template <typename T>
T sum(const vector<T>& v) {
    T result = 0;
    for (T elem : v)
        result += elem;
    return result;
}
```

As in the case of the generic `less_than` function, the keyword `template` introduces a template definition. The angle brackets (`<>`) that follow the `template` keyword contain declarations of template parameters.

When the compiler sees calling code like

```
vector<int> v {10, 20, 30};
cout << sum(v) << endl;
```

it knows `v`'s declared type is `vector<int>`; therefore, it uses the pattern in the template to instantiate an actual function that looks like the following:

```
// The compiler automatically generates this from the template:
int sum(const vector<int>& v) {
    int result = 0;
    for (int elem : v)
        result += elem;
    return result;
}
```

Notice how the compiler simply substitutes `int` for every occurrence of the type parameter `T`. If the compiler sees code like

```
vector<double> v {10, 20, 30};
cout << sum(v) << endl;
```

it again uses the template to instantiate the following function:

```
// The compiler automatically generates this:
double sum(const vector<double>& v) {
    double result = 0;
    for (double elem : v)
        result += elem;
    return result;
}
```

In a program that has code like

```
vector<double> v {10, 20, 30};
vector<int> w {10, 20, 30};
cout << sum(v) << endl;
cout << sum(w) << endl;
```

the compiler will instantiate *both* versions of the `sum` function. The compiler generates these automatically and secretly. The programmer never sees these instantiated template functions because they do not appear in any source code. Unlike the preprocessor that sends a modified version of the source code to the compiler (see Section 1.2 to review the preprocessor's role in the program's build process), a modern C++ compiler generates the machine language versions of these instantiated functions directly from the programmer's source code.

Recall our `swap` function from Listing 10.18 (`swapwithreferences.cpp`). Callers can use it to interchange the values of two integer variables. With our knowledge of C++ templates, we now can write a generic version that interchanges the values of two variables of *any* type:

```
/*
 * swap(a, b)
 *   Interchanges the values of memory
 *   referenced by its parameters a and b.
 *   It effectively interchanges the values
 *   of variables in the caller's context.
 */
template <typename ElemType>
void swap(ElemType& a, ElemType& b) {
    ElemType temp = a;
    a = b;
```

```

    b = temp;
}

```

The only restriction is the two parameters must be the same type. Here the type parameter's name is **ElemType**. This **swap** function works fine, but we do not need to write such a function ourselves; the C++ standard library includes a **std::swap** template function that does the same thing as our **swap** function shown above.

The following function will print the contents of a vector of any type:

```

/*
 *  print
 *      Prints the contents of a vector
 *      a is the vector to print.
 *      a is not modified.
 */
template <typename T>
void print(const vector<T>& a) {
    int n = a.size();
    cout << '{';
    if (n > 0) {
        cout << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    cout << '}';
}

```

The restriction here is that the elements in the vector must be printable to the **cout** object using **<<**.

Listing 18.3 (templateflexsort.cpp) uses our new generic functions to increase the flexibility of our flexible sorting program from Listing 12.2 (flexibleintsor.cpp).

Listing 18.3: templateflexsort.cpp

```

#include <iostream>
#include <string>
#include <vector>
#include <utility> // For generic swap function

using namespace std;

/*
 *  less_than(a, b)
 *      Returns true if a < b; otherwise, returns
 *      false.
 */
template <typename T>
bool less_than(const T& a, const T& b) {
    return a < b;
}

/*
 *  greater_than(a, b)
 *      Returns true if a > b; otherwise, returns

```

```

    *    false.
    */
template <typename T>
bool greater_than(const T& a, const T& b) {
    return a > b;
}

/*
 *    selection_sort(a, n, compare)
 *    Arranges the elements of vector vec in an order determined
 *    by the compare function.
 *    vec is a vector.
 *    compare is a function that compares the ordering of
 *    two types that support the < operator.
 *    The contents of vec are physically rearranged.
 */
template <typename T>
void selection_sort(vector<T>& vec,
                    bool (*compare)(const T&, const T&)) {
    int n = vec.size();
    for (int i = 0; i < n - 1; i++) {
        // Note: i, small, and j represent positions within vec.
        // vec[i], vec[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than vec[i]
        int small = i;
        // See if a smaller value can be found later in the vector
        for (int j = i + 1; j < n; j++)
            if (compare(vec[j], vec[small]))
                small = j; // Found a smaller value
        // Swap vec[i] and vec[small], if a smaller value was found
        if (i != small)
            swap(vec[i], vec[small]); // Uses std::swap from <utility>
    }
}

/*
 *    print
 *    Prints the contents of a vector
 *    vec is the vector to print.
 *    The function does not modify vec.
 */
template <typename T>
void print(const vector<T>& vec) {
    int n = vec.size();
    cout << '{';
    if (n > 0) {
        cout << vec[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << vec[i]; // Print the rest
    }
    cout << '}' ;
}

```



```

}

int main() {
    vector<int> list{23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10};
    cout << "Original:  ";
    print(list);
    cout << endl;
    selection_sort(list, less_than<int>);
    cout << "Ascending:  ";
    print(list);
    cout << endl;
    selection_sort(list, greater_than<int>);
    cout << "Descending: ";
    print(list);
    cout << endl;
    cout << "-----" << endl;
    vector<string> words { "tree", "girl", "boy", "apple", "dog", "cat", "bird" };
    cout << "Original:  ";
    print(words);
    cout << endl;
    selection_sort(words, less_than<string>);
    cout << "Ascending:  ";
    print(words);
    cout << endl;
    selection_sort(words, greater_than<string>);
    cout << "Descending: ";
    print(words);
    cout << endl;
}

```

Notice that all the functions except for `main` are generic functions. The `selection_sort` function is the most interesting one. It accepts a generic array and a function pointer that accepts two generic parameters of the same type. In one of its invocations in `main`:

```
selection_sort(list, greater_than<int>);
```

the expression `greater_than<int>` is an explicit template instantiation, and it tells the compiler we want to send to the `selection_sort` function a pointer to the `greater_than` function instantiated with integer parameters. This explicit instantiation is required because the compiler cannot automatically instantiate the `greater_than` function without an actual call to the function passing real parameters, and a pointer to a function is not a function invocation. At this point in the program we are not calling the `greater_than` function but only passing a pointer to it. Calling it would provide the necessary type information to the compiler via the actual parameters used, but a pointer to a function does not contain parameter information. The compiler needs the assistance of this explicit template instantiation.

The call to `swap` within `selection_sort` is actually invoking the `std::swap` function from the standard library. The `#include <utility>` directive brings in the declaration for `std::swap`, which itself is a template function.

The compiler can instantiate an actual function from a template function only if the type substituted for the template parameter supports its use within the function. Consider the following code:

```

// Make two ofstream objects
std::ofstream fout1("file1.data"), fout2("file2.data");

```

```
// Compare them
if (less_than(fout1, fout2)    // <-- Error!
    cout << "fout1 is less than fout2" << endl;
```

The expression `less_than(fout1, fout2)` is illegal because no `operator<` exists to compare two `std::ofstream` objects; thus, the compiler cannot produce the machine language that represents the expression `a < b` when `a` and `b` are `ofstreams`. Ultimately, therefore, the compiler cannot instantiate an actual `less_than` function that accepts two `ofstream` objects.

When programmers use templates incorrectly or construct incorrect template functions or classes, the compiler can detect and report the problems. All current C++ compilers share one weakness with template processing: poor human-readable error messages. The error messages the compiler produces in these situations tend to be lengthy and difficult to decipher even for experienced C++ programmers. This is because the compiler bases its error messages on its processing of the *instantiated* code, not the original *un-instantiated* code. Since programmers see only the un-instantiated source, deciphering the compiler's template error messages tends to be more challenging than usual.

As Listing 18.4 (templatescale.cpp) shows, a template can accept *non-type parameters*.

Listing 18.4: templatescale.cpp

```
#include <iostream>
using namespace std;

template <int N>
int scale(int value) {
    return value * N;
}

int main() {
    cout << scale<3>(5) << endl;
    cout << scale<4>(10) << endl;
}
```

Note that in Listing 18.4 (templatescale.cpp) the `typename` keyword is missing, and in its place is the actual type `int`. This means parameter `N` is a non-type template parameter. The program prints

```
15
40
```

Listing 18.5 (templatescale2.cpp) uses both a type-parameter and a non-type parameter.

Listing 18.5: templatescale2.cpp

```
#include <iostream>
using namespace std;

template <typename T, int N>
T scale(const T& value) {
    return value * N;
}
```

```
int main() {  
    cout << scale<double, 3>(5.3) << endl;  
    cout << scale<int, 4>(10) << endl;  
}
```

Listing 18.5 (templatescale2.cpp) displays the following during its execution:

```
15.9  
40
```

The compiler will instantiate two **scale** functions:

```
// Instantiated by the compiler due to scale<double, 3>:  
double scale(const double& value) {  
    return value * 3;  
}
```

and

```
// Instantiated by the compiler due to scale<int, 4>:  
int scale(const int& value) {  
    return value * 4;  
}
```

The types of non-type template parameters permitted by C++ are very limited. Integral types (**int**, **unsigned**, **long**, etc.) are acceptable, as are pointers and references. Other types such as floating-point and programmer-defined types may not be non-type template parameters.

The C++ Standard Library contains many generic functions, some of which we have seen already (all in the **std** namespace):

- **max**—computes the maximum of two values
- **min**—computes the maximum of two values
- **swap**—interchanges the values of two variables

18.2 Class Templates

Function templates (Section 18.1) enable us to build generic functions. C++ supports *class templates* as well. With class templates we can specify the pattern or structure of a class of objects in a type-independent way. The class template mechanism is a key tool for creating generic types.

As a simple example, consider a basic **Point** class that represents two-dimensional point objects. Mathematical point objects should have real-valued coordinates that we can approximate with double-precision floating-point values. A point on a graphical display, on the other hand, better uses integer coordinates because screen pixels have discrete, whole number locations.

Rather than providing two separate classes, we can write one class template let the compiler instantiate the coordinates as the particular program requires. Listing 18.6 (genericpoint.h) contains the generic class.

Listing 18.6: genericpoint.h

```

#ifndef _GENERICPOINT_H_
#define _GENERICPOINT_H_

template <typename T>
class Point {
public:
    T x;
    T y;
    Point(T x, T y): x(x), y(y) {}
};

#endif

```

If a client declares a **Point** object as

```
Point<int> pixel(10, 10);
```

the compiler will instantiate a **Point<int>** class that looks like

```

class Point {
public:
    int x;
    int y;
    Point(int x, int y): x(x), y(y) {}
};

```

If a client instead declares a **Point** object as

```
Point<float> pixel(10.0, 20.0);
```

the compiler will instantiate a **Point<double>** class that looks like

```

class Point {
public:
    double x;
    double y;
    Point(double x, double y): x(x), y(y) {}
};

```

Neither of these instantiations appear in any source code, so a programmer will not see them. The compiler substitutes actual types (**int** or **double**) for the parameterized type (**T**).

As a more significant example, Listing 18.7 (genericcomparer.h) contains the declaration of a **Comparer** class that is the generic equivalent to Listing 17.18 (comparer.h).

Listing 18.7: genericcomparer.h

```

#ifndef _GENERICCOMPARER_H_
#define _GENERICCOMPARER_H_

/*
 * Comparer objects manage the comparisons and element
 * interchanges on the selection sort function below.
 */
template <typename T>

```

```

class Comparer {
    // The object's data is private, so it is inaccessible to
    // clients and derived classes

    // Keeps track of the number of comparisons
    // performed
    int compare_count;

    // Keeps track of the number of swaps performed
    int swap_count;

    // Function pointer directed to the function to
    // perform the comparison
    bool (*comp)(const T&, const T&);

protected:
    // Method that actually performs the comparison
    // Derived classes may customize this method
    virtual bool compare_impl(const T& m, const T& n) {
        return comp(m, n);
    }

    // Method that actually performs the swap
    // Derived classes may customize this method
    virtual void swap_impl(T& m, T& n) {
        T temp = m;
        m = n;
        n = temp;
    }

public:
    // The client must initialize a Comparer object with a
    // suitable comparison function.
    Comparer(bool (*f)(const T&, const T&)):
        compare_count(0), swap_count(0), comp(f) {}

    // Resets the counters to make ready for a new sort
    void reset() {
        compare_count = swap_count = 0;
    }

    // Method that performs the comparison. It delegates
    // the actual work to the function pointed to by comp.
    // This method logs each invocation.
    bool compare(const T& m, const T& n) {
        compare_count++;
        return compare_impl(m, n);
    }

    // Method that performs the swap.
    // Interchange the values of
    // its parameters a and b which are
    // passed by reference.
    // This method logs each invocation.
    void swap(T& m, T& n) {

```

```

        swap_count++;
        swap_impl(m, n);
    }

    // Returns the number of comparisons this object has
    // performed since it was created.
    int comparisons() const {
        return compare_count;
    }

    // Returns the number of swaps this object has
    // performed since it was created.
    int swaps() const {
        return swap_count;
    }
};

#endif

```

As with Listing 17.18 (comparer.h), the **Comparer** objects specified in Listing 18.7 (genericcomparer.h) compare and swap any types of values and are meant to be used in a vector or array sorting program. Given the function

```

bool less_than(int m, int n) {
    return m < n;
}

```

the following code fragment instantiates a **Comparer** template that works with integers:

```

Comparer comp(less_than); // Make an integer comparer object

```

Listing 18.8 (genericstatisticsort.cpp) provides the client code that tests the new **Comparer** class.

Listing 18.8: genericstatisticsort.cpp

```

#include <iostream>
#include <vector>
#include <string>
#include "genericcomparer.h"

using namespace std;

/*
 *  selection_sort(a, compare)
 *      Arranges the elements of vector a in an order determined
 *      by the compare object.
 *      a is a vector.
 *      compare is a function that compares the ordering of
 *      two integers.
 *      The contents of a are physically rearranged.
 */
template <typename T>
void selection_sort(vector<T>& a, Comparer<T>& compare) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {

```

```

        // Note: i, small, and j represent positions within a
        // a[i], a[small], and a[j] represents the elements at
        // those positions.
        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        int small = i;
        // See if a smaller value can be found later in the array
        for (int j = i + 1; j < n; j++)
            if (compare.compare(a[j], a[small]))
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            compare.swap(a[i], a[small]);
    }
}

/*
 * Prints the contents of a vector
 * a is the vector to print.
 * a is not modified.
 */
template <typename T>
ostream& operator<<(ostream& os, const vector<T>& a) {
    int n = a.size();
    os << '{';
    if (n > 0) {
        os << a[0]; // Print the first element
        for (int i = 1; i < n; i++)
            cout << ',' << a[i]; // Print the rest
    }
    os << '}';
    return os;
}

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
template <typename T>
bool less_than(const T& a, const T& b) {
    return a < b;
}

/*
 * greater_than(a, b)
 * Returns true if a > b; otherwise, returns
 * false.
 */
template <typename T>
bool greater_than(const T& a, const T& b) {
    return a > b;
}

```

```

int main() {
    // Make a vector of integers
    vector<int> original { 23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10 };

    // Make a working copy of the original vector
    vector<int> working = original;
    cout << "Before: ";
    cout << working << endl;
    Comparer<int> lt(less_than<int>), gt(greater_than<int>);
    selection_sort(working, lt);
    cout << "Ascending: ";
    cout << working << endl;
    cout << " (" << lt.comparisons() << " comparisons, "
        << lt.swaps() << " swaps)" << endl;
    cout << "-----" << endl;
    // Make another copy of the original vector
    working = original;
    cout << "Before: ";
    cout << working << endl;
    selection_sort(working, gt);
    cout << "Descending: ";
    cout << working << endl;
    cout << " (" << gt.comparisons() << " comparisons, "
        << gt.swaps() << " swaps)" << endl;
    cout << "-----" << endl;

    // Make a vector of strings
    vector<string> str_original { "tree", "boy", "apple", "girl", "dog", "cat" };

    // Make a working copy of the original vector
    vector<string> str_working = str_original;
    cout << "Before: ";
    cout << str_working << endl;
    Comparer<string> str_lt(less_than<string>), str_gt(greater_than<string>);
    selection_sort(str_working, str_lt);
    cout << "Ascending: ";
    cout << str_working << endl;
    cout << " (" << str_lt.comparisons() << " comparisons, "
        << str_lt.swaps() << " swaps)" << endl;
    cout << "-----" << endl;
    // Make another copy of the original vector
    str_working = str_original;
    cout << "Before: ";
    cout << str_working << endl;
    selection_sort(str_working, str_gt);
    cout << "Descending: ";
    cout << str_working << endl;
    cout << " (" << str_gt.comparisons() << " comparisons, "
        << str_gt.swaps() << " swaps)" << endl;
    cout << "-----" << endl;
}

```

Listing 18.8 (genericstatisticsort.cpp) demonstrates that the generic **Comparer** class works equally as well with integers and strings; the program's output is


```

Before:   {23,-3,4,215,0,-3,2,23,100,88,-10}
Ascending: {-10,-3,-3,0,2,4,23,23,88,100,215}
          (55 comparisons, 7 swaps)
-----
Before:   {23,-3,4,215,0,-3,2,23,100,88,-10}
Descending: {215,100,88,23,23,4,2,0,-3,-3,-10}
          (55 comparisons, 5 swaps)
-----
Before:   {tree,boy,apple,girl,dog,cat}
Ascending: {apple,boy,cat,dog,girl,tree}
          (15 comparisons, 3 swaps)
-----
Before:   {tree,boy,apple,girl,dog,cat}
Descending: {tree,girl,dog,cat,boy,apple}
          (15 comparisons, 4 swaps)
-----

```

Recall the **IntList** linked list class from Listings 16.6 and 16.7. It implemented a linked list of integer elements. Suppose we instead want to make a linked list of **std::string** objects. Without C++ templates, we would have to copy the **IntList** code and modify it so it works with **string** objects instead of integers. We could call this class **StringList**. Note that the code in the **IntList** and **StringList** classes would be almost identical; the only major difference would be the type of elements the list can hold. C++'s class template feature is ideal for designing generic container classes. Listing 18.9 (`l1ist.h`) defines a such a generic linked list in its **LList** class.

Listing 18.9: `l1ist.h`

```

// l1ist.h
#include <iostream>

template <typename T>
class LList {
    /*
     * An object that holds an element in a linked list.
     * This type is local to the LList class and inaccessible
     * to outside this class.
     */
    struct Node {
        T data;           // A data element of the list
        Node *next;       // The node that follows this one in the list
        // Constructor
        Node(const T& d): data(d), next(nullptr) {}
    };

    Node *head; // Points to the first item in the list

    // Private helper methods

    /*
     * insert(p, elem)
     * Inserts elem onto the back of the

```

```

    *   list pointed to by p.
    *   p points to a node in a linked list
    *   elem is the element to insert.
    *   Returns a pointer to the list that
    *   contains elem.
    */
Node *insert(Node *p, const T& elem) {
    if (p) // Is p non-null?
        p->next = insert(p->next, elem);
    else // p is null, make a new node
        p = new LList::Node(elem);
    return p;
}

/*
 * print(p)
 * Prints the contents of the linked list
 * p points to a linked list of nodes.
 */
void print(Node *p) const {
    while (p) { // While p is not null
        std::cout << p->data << " "; // Print current node's data
        p = p->next; // Move to next node
    }
    std::cout << std::endl;
}

/*
 * Returns the length of the linked list pointed to by p.
 */
int length(Node *p) const {
    if (p)
        return 1 + length(p->next); // 1 + length of rest of list
    else
        return 0; // Empty list has length zero
}

/*
 * dispose(p)
 * Deallocate the memory held by the list pointed to by p.
 */
void dispose(Node *p) {
    if (p) {
        dispose(p->next); // Free up the rest of the list
        delete p; // Deallocate this node
    }
}

public:
    /*
     * The constructor makes an initially empty list
     */
    LList(): head(nullptr) {}

```

```

/*
 * The destructor frees up the memory held by the list
 */
~LList() {
    clear(); // Deallocate space held by the list nodes
}

/*
 * insert(elem)
 * Inserts elem onto the back of the list.
 * elem is the element to insert.
 */
void insert(const T& elem) {
    head = insert(head, elem); // Delegate work to private helper method
}

/*
 * print()
 * Prints the contents of the linked list.
 * Defers the actual work to the private
 * overloaded print method.
 */
void print() const {
    print(head); // Delegate work to private helper method
}

/*
 * Returns the length of the linked list.
 * Defers the actual work to the private
 * overloaded length method.
 */
int length() const {
    return length(head); // Delegate work to private helper method
}

/*
 * clear()
 * Removes all the elements in the linked list.
 * The private dispose function does the hard work.
 */
void clear() {
    dispose(head); // Deallocate space for all the nodes
    head = nullptr; // Null head signifies list is empty
}
};

```

Notice that in Listing 18.9 (l1ist.h) the linked lists elements have the generic type **T**. Observe carefully where the **T** type parameter appears in the code. Listing 18.10 (templatelistmain.cpp) shows how a client can instantiate the **LList** template for integer elements. The code in Listing 18.10 (templatelistmain.cpp) contains only trivial changes to just two lines of the code from Listing 16.8 (listmain.cpp).

Listing 18.10: templatelistmain.cpp

```

// templatelistmain.cpp

```

```

#include "l1list.h"           // Was ==> #include "intlist.h"
#include <iostream>

using namespace std;

int main() {
    bool done = false;
    char command;
    int value;
    LList<int> list;           // Was ==> IntList list;

    while (!done) {
        cout << "I)nsert <item> P)rint L)ength E)rase Q)uit >>";
        cin >> command;
        switch (command) {
            case 'I': // Insert a new element into the list
            case 'i':
                if (cin >> value)
                    list.insert(value);
                else
                    done = true;
                break;
            case 'P': // Print the contents of the list
            case 'p':
                list.print();
                break;
            case 'L': // Print the list's length
            case 'l':
                cout << "Number of elements: " << list.length() << endl;
                break;
            case 'E': // Erase the list
            case 'e':
                list.clear();
                break;
            case 'Q': // Exit the loop (and the program)
            case 'q':
                done = true;
                break;
        }
    }
}

```

The statement

```
LList<int> list;
```

instantiates an actual class where all the occurrences of **T** become **int**. It also declares **list** to be an object of this instantiated class.

Just as we may instantiate function templates with multiple different types within the same program, we can do the same for class templates. The class template **LList** in Listing 18.9 (l1list.h) makes the following client code possible:

```

LList<int> intlist;
LList<string> strlist;

```

```
intlist.insert(5);
strlist.insert("Wow");
intlist.print();
strlist.print();
```

This code fragment simultaneously manages lists of integers and lists of string objects.

By now you probably have deduced the fact that `std::vector` is a class template. The `vector` class template is much more sophisticated than our simple `LList` template, but the basic mechanism for its design, a class template, is the same.

18.3 Iterators

Section 11.2 and the extended information in Appendix F demonstrate how we can use a pointer to access the elements within an array. A vector is an object that manages a primitive array, and the developers of the vector class designed vectors to look and feel as much as possible as arrays. They adopted the square bracket (`[]`) operator to access elements. They also devised a way for programmers to use pointer-like objects to access the elements within a vector. These pointer-like objects are called *iterators*.

An iterator is an object that allows a client to traverse and access elements of a data structure in an implementation independent way. C++11 defines two global functions, `begin` and `end`, that produce iterators to the front and back, respectively, of a data structure like a vector or static array. Functions in the standard library that accept iterators as arguments rather than arrays or vectors work equally well with both vectors and arrays. Since they accept iterator arguments, these standard functions additionally are able to work with other, more sophisticated data structures. We will examine some of these standard functions later in this chapter.

In order to behave like a pointer, an iterator object provides the following methods:

- `operator*`: used to access the element at the iterator's current position. The syntax is exactly like pointer dereferencing (see Section 10.6).
- `operator++`: used to move the iterator to the next element within the data structure. The syntax is exactly like pointer arithmetic (see Section 11.2).
- `operator!=`: used to determine whether two iterator objects currently refer to different elements within the data structure.

Listing 18.11 (`simpleiterator.cpp`) shows how to use these methods to manipulate an iterator object.

Listing 18.11: `simpleiterator.cpp`

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Make a simple integer vector
    vector<int> vec {10, 20, 30, 40, 50};
    // Direct an iterator to the vector's first element
    vector<int>::iterator iter = begin(vec);
    // Print the element referenced by the iterator
    cout << *iter << endl;
```

```

    // Advance the iterator
    iter++;
    // See where the iterator is now
    cout << *iter << endl;
}

```

Listing 18.11 (simpleiterator.cpp) prints

```

10
20

```

The statement

```
vector<int>::iterator iter = begin(vec);
```

declares and initializes an iterator object, **iter**. The type of **iter** is **vector<int>::iterator**. This complicated expression indicates that **iterator** is a type defined within the **vector<int>** type. A shorter way to express this statement takes advantage of the compiler's ability to infer the variable's type from its context:

```
auto iter = begin(vec);
```

(Section 3.9 introduced C++'s type inference capabilities.) The compiler can deduce the type of **iter** based on the return type of the **begin** function.

While the **begin** function returns an iterator pointing to the first element in a data structure, the **end** function returns an iterator pointing to the element *just past the last element* in the data structure. In the code fragment

```
vector<int> vec {10, 20, 30, 40, 50};
auto iter = end(vec);
```

the iterator object **iter** does not reference a viable element in the vector. As such, we never attempt to dereference the iterator; the expression ***iter** in this case is undefined. We normally use the iterator returned by the **end** function to test to see if we are past the end of our data structure, Listing 18.12 (iteratorloop.cpp) demonstrates.

Listing 18.12: iteratorloop.cpp

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Make a simple integer vector
    vector<int> vec {10, 20, 30, 40, 50};
    // Print the contents of the vector
    for (auto iter = begin(vec); iter != end(vec); iter++)
        cout << *iter << ' ';
    cout << endl;
}

```

Listing 18.12 (iteratorloop.cpp) prints

```
10 20 30 40 50
```

The expression

```
iter != end(vec)
```

checks each time through the loop to ensure the iterator object has not run off the back end of the vector.

The **begin** and **end** functions are overloaded to work with vector objects, arrays, and other container classes found in the standard library. Listing 18.13 (iteratorlooparray.cpp) is the array version of Listing 18.12 (iteratorloop.cpp).

Listing 18.13: iteratorlooparray.cpp

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Make a static integer array
    int arr[] = {10, 20, 30, 40, 50};
    // Print the contents of the array
    for (auto iter = begin(arr); iter != end(arr); iter++)
        cout << *iter << ' ';
    cout << endl;
}
```

The output of Listing 18.13 (iteratorlooparray.cpp) is identical to Listing 18.12 (iteratorloop.cpp):

```
10 20 30 40 50
```

Any type for which the **begin** and **end** functions have been overloaded may participate in a range-based **for** statement. In fact, the compiler internally transforms a range-based **for** construct into its equivalent **begin/end** form during the compilation process. The following code

```
for (auto i : vec)
    cout << i << ' ';
```

is short for (one possible internal compiler representation)

```
for (auto _i = begin(vec); _i != end(vec); _i++)
    cout << *_i << ' ';
```

We can treat an iterator as a pointer. Recall from Section 11.2 that we can manipulate pointers with pointer arithmetic. If **p** points to the first element of an array, **p + 5** points to the sixth element in the array (because **p + 0** is the same as **p**). The expression **p++** makes the **p** point to the next element in the array, while **p--** moves **p** backwards one position. We can use similar arithmetic with iterators as Listing 18.14 (iterarith.cpp) illustrates.

Listing 18.14: iterarith.cpp

```
#include <iostream>
#include <vector>

using namespace std;

// Print the contents of vector v, traversing with a
// caller supplied increment value (inc)
void print(const vector<int>& v, int inc) {
    for (auto p = begin(v); p != end(v); p += inc)
        cout << *p << ' ';
    cout << endl;
}

// Print the contents of the vector v backwards,
// traversing with a caller supplied decrement value (dec)
void print_reverse(const vector<int>& v, int dec) {
    auto p = end(v);
    while (p != begin(v)) {
        p -= dec;
        cout << *p << ' ';
    }
    cout << endl;
}

int main() {
    vector<int> vec(20);
    for (int i = 0; i < 20; i++)
        vec[i] = i;

    print(vec, 1);
    print(vec, 2);
    print(vec, 4);
    print(vec, 5);
    print(vec, 10);

    cout << endl;

    print_reverse(vec, 1);
    print_reverse(vec, 2);
    print_reverse(vec, 4);
    print_reverse(vec, 5);
    print_reverse(vec, 10);
}
```

Listing 18.14 (iterarith.cpp) prints


```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
0 2 4 6 8 10 12 14 16 18
0 4 8 12 16
0 5 10 15
0 10

19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
18 16 14 12 10 8 6 4 2 0
16 12 8 4 0
15 10 5 0
10 0

```

Some specialized data structures in the C++ standard library support specialized iterators have some limits on the kinds of arithmetic they support; for example, some container types support forward but not backward traversal. Subtraction is not an option for these kinds of iterators.

18.4 Iterator Ranges

Iterators are valuable for specifying *ranges* over sequences (vectors and arrays). We can specify a range with a pair of iterators: one iterator points to the first element in the sequence, and the other iterator points to the position just past the end of the sequence. The global functions **begin** and **end** fit well into this definition of range.

Suppose **vec** is a vector. The two iterators **begin(vec)** and **end(vec)** span the complete range of values in **vec**. In the following statement:

```
auto b = begin(vec), e = b + 1;
```

the range of two iterators **b** and **e** includes just the first element in **vec**. Compare this range to the one represented by the following statement:

```
auto b = begin(vec), e = b + 10;
```

Here the range of two iterators **b** and **e** spans the first 10 elements of **vec**. The range of the iterators **b** and **e** defined by the statement

```
auto e = end(vec), b = e - 5;
```

encompasses the last five elements of **vec**.

The iterator pair **begin(vec)**, **end(vec)** specify the complete range of values in vector **vec**. The following function uses a iterators to count the number of times a value appears within a vector:

```
int count(vector<int>::iterator iter_begin,
          vector<int>::iterator iter_end, int seek) {
    int cnt = 0;
    for (vector<int>::iterator cursor = iter_begin;
         cursor != iter_end; iter++)
        if (*cursor == seek)
            cnt++;
}
```

```

    return cnt;
}

```

Section 11.2 introduced a technique for traversing an array via a pointer range. Rather than passing an array (literally the address of the first element) and its size (number of elements) to a traversal function, the range-based approach passes the array (again, literally a pointer to the first element) and a pointer to position just past the end of the array. Notice how the pointers in range-based array technique correspond directly to the iterator objects returned by the **begin** and **end** functions. Since iterator objects behave like pointers, we can write truly generic code that can process both arrays and vectors.

Listing 18.15 (`genericloggingflexiblesort.cpp`) uses template functions and template classes to make our logging flexible sort code truly generic. It also uses inheritance to derive a class from a template class.

Listing 18.15: `genericloggingflexiblesort.cpp`

```

#include <iostream>
#include <vector>
#include <cstdlib>
#include <fstream>
#include <string>

using namespace std;

/*
 * Comparer objects manage the comparisons and element
 * interchanges on the selection sort function below.
 */
template <typename T>
class Comparer {
    // The object's data is private, so it is inaccessible to
    // clients and derived classes

    // Keeps track of the number of comparisons
    // performed
    int compare_count;

    // Keeps track of the number of swaps performed
    int swap_count;

    // Function pointer directed to the function to
    // perform the comparison
    bool (*comp)(const T&, const T&);

protected:
    // Method that actually performs the comparison
    // Derived classes may override this method
    virtual bool compare_impl(const T& m, const T& n);

    // Method that actually performs the swap
    // Derived classes may override this method
    virtual void swap_impl(T& m, T& n);

public:
    // The client must initialize a Comparer object with a
    // suitable comparison function.

```

```

    Comparer(bool (*f)(const T&, const T&));

    // Resets the counters to make ready for a new sort
    void reset();

    // Method that performs the comparison. It delegates
    // the actual work to the function pointed to by comp.
    // This method logs each invocation.
    bool compare(const T& m, const T& n);

    // Method that performs the swap.
    // Interchange the values of
    // its parameters a and b which are
    // passed by reference.
    // This method logs each invocation.
    void swap(T& m, T& n);

    // Returns the number of comparisons this object has
    // performed since it was created.
    int comparisons() const;

    // Returns the number of swaps this object has
    // performed since it was created.
    int swaps() const;
};

// Method that actually performs the comparison
// Derived classes may override this method
template <typename T>
bool Comparer<T>::compare_impl(const T& m, const T& n) {
    return comp(m, n);
}

// Method that actually performs the swap
// Derived classes may override this method
template <typename T>
void Comparer<T>::swap_impl(T& m, T& n) {
    T temp = m;
    m = n;
    n = temp;
}

// The client must initialize a Comparer object with a
// suitable comparison function.
template <typename T>
Comparer<T>::Comparer(bool (*f)(const T&, const T&)):
    compare_count(0), swap_count(0), comp(f) {}

// Resets the counters to make ready for a new sort
template <typename T>
void Comparer<T>::reset() {
    compare_count = swap_count = 0;
}

```

```

// Method that performs the comparison. It delegates
// the actual work to the function pointed to by comp.
// This method logs each invocation.
template <typename T>
bool Comparer<T>::compare(const T& m, const T& n) {
    compare_count++;
    return compare_impl(m, n);
}

// Method that performs the swap.
// Interchange the values of
// its parameters a and b which are
// passed by reference.
// This method logs each invocation, so
// we do not use std::swap here.
template <typename T>
void Comparer<T>::swap(T& m, T& n) {
    swap_count++;
    swap_impl(m, n);
}

// Returns the number of comparisons this object has
// performed since it was created.
template <typename T>
int Comparer<T>::comparisons() const {
    return compare_count;
}

// Returns the number of swaps this object has
// performed since it was created.
template <typename T>
int Comparer<T>::swaps() const {
    return swap_count;
}

/*
 *   selection_sort(a, compare)
 *       Arranges the elements of a sequence in an order determined
 *       by the compare object.
 *       begin points to the beginning of the sequence.
 *       end points to the imaginary element just past the last
 *       element of the sequence.
 *       compare is an object that compares the ordering of two
 *       elements and records the actions it performs.
 *       The function physically rearranges the contents of the
 *       sequence.
 */
template <typename T, typename V>
void selection_sort(const T begin, const T end, Comparer<V>& compare) {
    for (auto i = begin; i != end - 1; i++) {
        // Note: i, small, and j represent positions within the
        // sequence.
        // a[i], a[small], and a[j] represents the elements at
        // those positions.

```

```

        // small is the position of the smallest value we've seen
        // so far; we use it to find the smallest value less
        // than a[i]
        auto small = i;
        // See if a smaller value can be found later in the sequence
        for (auto j = i + 1; j != end; j++)
            if (compare.compare(*j, *small))
                small = j; // Found a smaller value
        // Swap a[i] and a[small], if a smaller value was found
        if (i != small)
            compare.swap(*i, *small);
    }
}

/*
 * print
 * Prints the contents of a sequence
 * begin points to the beginning of the sequence.
 * end points to the imaginary element just past the last.
 * The function does not modify the sequence.
 */
template <typename T>
void print(const T begin, const T end) {
    cout << '{';
    if (begin != end) {
        T iter = begin;
        cout << *iter; // Print the first element
        iter++;        // Move to next element
        while (iter != end) { // Print the rest
            cout << ', ' << *iter;
            iter++;        // Move to next element
        }
    }
    cout << ' ';
}

/*
 * less_than(a, b)
 * Returns true if a < b; otherwise, returns
 * false.
 */
template <typename T>
bool less_than(const T& a, const T& b) {
    return a < b;
}

/*
 * greater_than(a, b)
 * Returns true if a > b; otherwise, returns
 * false.
 */
template <typename T>
bool greater_than(const T& a, const T& b) {
    return a > b;
}

```

```

/*
 * Comparer objects manage the comparisons and element
 * interchanges on the selection sort function below.
 */
template <typename T>
class LogComparer: public Comparer<T> {
    // Output stream to which logging messages are directed
    std::ofstream fout;

protected:
    // Method that actually performs the comparison
    bool compare_impl(const T& m, const T& n) override;

    // Method that actually performs the swap
    void swap_impl(T& m, T& n) override;

public:
    // The client must initialize a LogComparer object with a
    // suitable comparison function and the file name of a text
    // file to which the object will direct logging messages
    LogComparer(bool (*f)(const T&, const T&),
                const std::string& filename);

    // The destructor must close the log file
    ~LogComparer();
};

// Method that actually performs the comparison
// Derived classes may override this method
template <typename T>
bool LogComparer<T>::compare_impl(const T& m, const T& n) {
    fout << "Comparing " << m << " to " << n << std::endl;
    // Base class method does the comparison
    return Comparer<T>::compare_impl(m, n);
}

// Method that actually performs the swap
// Derived classes may override this method
template <typename T>
void LogComparer<T>::swap_impl(T& m, T& n) {
    fout << "Swapping " << m << " and " << n << endl;
    //T temp = m;
    //m = n;
    //n = temp;
    // Base class method does the swap
    Comparer<T>::swap_impl(m, n);
}

// The client must initialize a LogComparer object with a
// suitable comparison function and the file name of the
// text file to receive logging messages.
template <typename T>
LogComparer<T>::LogComparer(bool (*f)(const T&, const T&),

```

```

        const string& filename): Comparer<T>(f) {
    fout.open(filename);
    if (!fout.good()) {
        fout.close();
        cout << "Could not open log file " << filename
            << " for writing" << endl;
        exit(1); // Terminate the program
    }
}

// Destructor closes the log file
template <typename T>
LogComparer<T>::~LogComparer() {
    fout.close();
}

int main() {
    // Make a vector of integers
    vector<int> vec = { 23, -3, 10, 4, 215, 0, -3, 2 };

    cout << "Before: ";
    print(begin(vec), end(vec));
    cout << endl;
    LogComparer<int> lt(less_than<int>, "upsort.log");
    selection_sort(begin(vec), end(vec), lt);
    cout << "Ascending: ";
    print(begin(vec), end(vec));
    cout << " (" << lt.comparisons() << " comparisons, "
        << lt.swaps() << " swaps)" << endl;
    LogComparer<int> gt(greater_than<int>, "downsort.log");
    selection_sort(begin(vec), end(vec), gt);
    cout << "Descending: ";
    print(begin(vec), end(vec));
    cout << " (" << gt.comparisons() << " comparisons, "
        << gt.swaps() << " swaps)" << endl;

    cout << "-----" << endl;

    // Make a vector of string objects
    vector<string> words{"tree", "girl", "boy", "dog",
        "cat", "girl", "bird"};

    // Make a working copy of the original vector
    cout << "Before: ";
    print(begin(words), end(words));
    cout << endl;
    LogComparer<string> wlt(less_than<string>, "upwords.log");
    selection_sort(begin(words), end(words), wlt);
    cout << "Ascending: ";
    print(begin(words), end(words));
    cout << " (" << wlt.comparisons() << " comparisons, "
        << wlt.swaps() << " swaps)" << endl;
    LogComparer<string> wgt(greater_than<string>, "downwords.log");
    selection_sort(begin(words), end(words), wgt);

```

```

cout << "Descending: ";
print(begin(words), end(words));
cout << " (" << wgt.comparisons() << " comparisons, "
    << wgt.swaps() << " swaps)" << endl;

cout << "-----" << endl;

// Make an array of integers
int arr[] = { 23, -3, 10, 4, 215, 0, -3, 2 };

cout << "Before: ";
print(arr, arr + 8);
cout << endl;
LogComparer<int> lt2(less_than<int>, "upsort2.log");
selection_sort(arr, arr + 8, lt2);
cout << "Ascending: ";
print(arr, arr + 8);
cout << " (" << lt2.comparisons() << " comparisons, "
    << lt2.swaps() << " swaps)" << endl;
LogComparer<int> gt2(greater_than<int>, "downsort2.log");
selection_sort(arr, arr + 8, gt2);
cout << "Descending: ";
print(arr, arr + 8);
cout << " (" << gt2.comparisons() << " comparisons, "
    << gt2.swaps() << " swaps)" << endl;
}

```

Observe that the `selection_sort` function in Listing 18.15 (`genericloggingflexiblesort.cpp`) does not restrict the sequence it can process to be a vector or an array. The first two parameters of `selection_sort` are generic, and the code within the function treats these two parameters as if they are pointers. Since iterator objects behave exactly like pointers to the extent exercised within `selection_sort`, the compiler can instantiate the template to accept pointer or iterator arguments. Similarly, the `print` function works equally well with both pointer ranges for arrays and iterators for vectors. The `main` function demonstrates the flexibility of `print` and `selection_sort` functions by sending them both array pointer ranges and vector iterators. Behind the scenes the compiler will automatically instantiate two overloaded `print` functions and two overloaded `selection_sort` functions.

During its execution Listing 18.15 (`genericloggingflexiblesort.cpp`) produces the following output:

```

Before:  {23,-3,10,4,215,0,-3,2}
Ascending:  {-3,-3,0,2,4,10,23,215} (28 comparisons, 5 swaps)
Descending:  {215,23,10,4,2,0,-3,-3} (28 comparisons, 4 swaps)
-----
Before:  {tree,girl,boy,dog,cat,girl,bird}
Ascending:  {bird,boy,cat,dog,girl,girl,tree} (21 comparisons, 3 swaps)
Descending:  {tree,girl,girl,dog,cat,boy,bird} (21 comparisons, 4 swaps)
-----
Before:  {23,-3,10,4,215,0,-3,2}
Ascending:  {-3,-3,0,2,4,10,23,215} (28 comparisons, 5 swaps)
Descending:  {215,23,10,4,2,0,-3,-3} (28 comparisons, 4 swaps)

```


The code in Listing 18.15 (`genericloggingflexiblesort.cpp`) truly is generic:

- Clients may use vectors, arrays, and any sequence type that overloads the **begin** and **end** functions.
- Clients may work with sequences that contain elements of any type that supports the **<** and **=** operators.
- Clients may customize the element ordering.
- Clients may customize the behavior of the comparison and swapping procedures.

18.5 The Standard Template Library

CAUTION! SECTION UNDER CONSTRUCTION

The C++ Standard Template Library, or STL, contains a number of generic functions and classes. We have seen the `std::vector` class and `std::swap` function. Our `LList` class from Listing 18.9 (`llist.h`) is really re-inventing the wheel because `std::List` implements a doubly-linked list with pointers connecting a node to its successor and pointers connecting the node to its predecessor.

18.6 Summary

CAUTION! SECTION UNDER CONSTRUCTION

- Add item here.

18.7 Exercises

CAUTION! SECTION UNDER CONSTRUCTION

1. Add item here.

Chapter 19

Handling Exceptions

CAUTION! CHAPTER UNDER CONSTRUCTION

In our programming experience so far we have encountered several kinds of run-time errors. To this point, all of our run-time errors have resulted in the executing program's termination. C++ provides an *exception handling* framework that allows programmers to deal with certain kinds of run-time errors. Rather than always terminating the program's execution, the exception handling infrastructure enables programmers to detect a problem and execute code to correct the issue or manage it in other ways. This chapter explores C++'s exception handling mechanism.

19.1 Motivation

CAUTION! SECTION UNDER CONSTRUCTION

Algorithm design can be tricky because the details are crucial. It may be straightforward to write an algorithm to solve a problem in the general case, but there may be a number of special cases that must all be addressed within the algorithm for the algorithm to be correct. Some of these special cases might occur rarely under the most extraordinary circumstances. For the code implementing the algorithm to be robust, these exceptional cases must be handled properly; however, adding the necessary details to the algorithm may render it overly complex and difficult to construct correctly. Such an overly complex algorithm would be difficult for others to read and understand, and it would be harder to debug and extend.

Ideally, a developer would express the algorithm in its general form including any common special cases. Exceptional situations that should arise rarely, along with a strategy to handle them, could appear elsewhere, perhaps as an annotation to the algorithm. Thus, the algorithm is kept focused on solving the problem at hand, and measures to deal with exceptional cases are handled elsewhere.

C++'s exception handling infrastructure allows programmers to cleanly separate the code that implements the focused algorithm from the code that deals with exceptional situations that the algorithm may face. This approach is more modular and encourages the development of code that is cleaner and easier to maintain and debug.

An *exception* is an exceptional event that occurs during a program's execution. An exception always is possible, but it should be a relatively rare event. If it were not rare, it would be a customary or expected

event, and the program should handle it as part of its normal processing.

An exception almost always represents a problem, usually some sort of run-time error. Suppose we have a vector named `v` and an integer `i`. Unless $0 \leq i < v.size()$, the expression `v[i]` attempts to access an element outside the vector's bounds. The `operator[]` method performs no bounds checking; thus, an executing program using the expression `v[i]` could produce undefined behavior. Consider instead the expression `v.at(i)`. The `at` method works just like `operator[]`, except `at` does check the vector's bounds. If $i \geq v.size()$ or $i < 0$, the expression `v.at(i)` represents an exceptional situation, and we say the vector's `at` method *throws*, or *raises*, an exception.

Without taking advantage of C++'s exception handling infrastructure, the following statement:

```
v.at(i) = 4;
```

could terminate the program with an error message. While this is better than undefined behavior, it is not ideal. The program crashes. When a real application crashes the user may lose unsaved data. Badly behaving programs terminate unexpectedly. It would be better to defend against the out-of-bounds index and keep the program alive.

An algorithm could handle many potential problems itself. For example, a programmer can use an `if` statement to test to see if a vector's index is within the proper bounds:

```
if (0 <= i && i < v.size()) // Ensure i is in range
    v.at(i) = 4;
```

However, if the programmer accesses a vector in several different locations within a given function or method and the index variable can vary from place to place, the number of `if` statements necessary to ensure safe vector access might obscure the overall logic of the function or method.

19.2 Exception Examples

CAUTION! SECTION UNDER CONSTRUCTION

Consider Listing 19.1 (vectorboundscrash.cpp). The program crashes whenever the user enters a value equal to an index outside the range of vector `nums`.

Listing 19.1: vectorboundscrash.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<double> nums { 1.0, 2.0, 3.0 };
    int input;
    cout << "Enter an index: ";
    cin >> input;
    cout << nums.at(input) << endl;
}
```

The `at` method guarantees that an illegal access will not go undetected. To intercept the problem at run time and prevent the program from terminating due to an error, we use a `try/catch` block, which consists of two parts: a `try` block and a `catch` block. To form a `try/catch` block we

1. wrap the code that has the potential to throw an exception in a **try** block, and
2. provide code to execute only in the event of an exception in a **catch** block.

Listing 19.2 (vectorboundsexcept.cpp) adds the exception handling code to Listing 19.1 (vectorboundscrash.cpp).

Listing 19.2: vectorboundsexcept.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<double> nums { 1.0, 2.0, 3.0 };
    int input;
    cout << "Enter an index: ";
    cin >> input;
    try {
        cout << nums.at(input) << endl;
    }
    catch (exception& e) {
        cout << e.what() << endl;
    }
}
```

If a user enters a value less than zero or greater than two when executing Listing 19.2 (vectorboundsexcept.cpp), the program will print an error message and terminate successfully instead of crashing.

Both **try** and **catch** are keywords in C++. Unlike the bodies of the structured statements such as **if** and **while**, the statements within a **try** block and statements within a **catch** block must appear within curly braces, even if only one statement appears in the section.



Every **try** block or **catch** block must surround its statements with curly braces—even if it contains just one statement.

The variable **e** within the **catch** block of Listing 19.2 (vectorboundsexcept.cpp) is a reference to an **exception** object. The **exception** class is part of the standard C++ library and is the base class for all the standard exceptions. The **exception** class provides a **what** method that returns a string. The **exception::what** method's string is a message containing information about the exception. The C++ standard does not specify the exact message, but at the very least the **exception::what** method indicates the kind of exception caught. The following shows what the program reports under Visual C++ when the user enters the value 3:

```
Enter an index: 3
invalid vector<T> subscript
```

What makes the exception code

```
try {
    cout << nums.at(input) << endl;
}
catch (exception& e) {
    cout << e.what() << endl;
}
```

better than the conditional code

```
if (0 <= input && input < nums.size()) {
    cout << nums.at(input) << endl;
}
else {
    cout << "Vector out of bounds exception" << endl;
}
```

is not immediately obvious from this simple example. To see the real capability of exceptions, consider Listing 19.3 (callchainexception.cpp).

Listing 19.3: callchainexception.cpp

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// Used to keep track of object creation and destruction
class Tracker {
    string name;
public:
    Tracker(const string& s): name(s) {
        cout << "Creating Tracker (" << name << ")" << endl;
    }
    ~Tracker() {
        cout << "Destroying Tracker (" << name << ")" << endl;
    }
};

bool find(const vector<int>& v, int lower, int upper) {
    cout << "-----[Entering find]-----" << endl;
    Tracker obj("find"); // Local tracking object
    bool result = false; // Not there by default
    int seek;           // What to look for
    cout << "Enter item to locate: ";
    cin >> seek;
    for (int i = lower; i < upper; i++)
        if (v.at(i) == seek) {
            result = true; // Found it
            break;
        }
    cout << "-----[Leaving find]-----" << endl;
```

```

        return result;
    }

    void process(const vector<int>& v) {
        cout << "-----[Entering process]-----" << endl;
        Tracker obj("process"); // Local tracking object
        int low, high;
        cout << "Enter a range: ";
        cin >> low >> high;
        if (find(v, low, high))
            cout << "Found it" << endl;
        else
            cout << "Not there" << endl;
        cout << "-----[Leaving process]-----" << endl;
    }

    // Global tracking object
    Tracker obj("global");

    int main() {
        cout << "-----[Entering main]-----" << endl;
        Tracker obj("main"); // Local tracking object
        vector<int> nums { 11, 42, 23 };
        try {
            process(nums);
        }
        catch (exception& e) {
            cout << "nums vector bounds exceeded" << endl;
        }
        cout << "-----[Leaving main]-----" << endl;
    }

```

Listing 19.3 (callchainexception.cpp) is specially crafted to provide insight into the program's exact execution path. Each function prints a message at its beginning and end so we can see when the code within a function is active. Each function also declares a local **Tracker** object which is identified by the name of that function. An executing function constructs a local object at its point of declaration and automatically destroys the object when the function returns to its caller. These local objects, therefore, have a lifetime that mirrors the lifetime of the function that manages them. Finally, Listing 19.3 (callchainexception.cpp) declares a global **Tracker** object. An executing program creates global objects before it calls **main** and automatically destroys them after **main** completes.

In Listing 19.3 (callchainexception.cpp) the vector **nums**, declared in **main**, holds three elements. The **main** function passes its **nums** vector to **process**. During its call to **process**, the **main** function has no control over the **process** function's attempts to access an element outside the range of **nums**. The **process** function, however, does not itself attempt to access any elements within the vector that **main** supplies. Instead, **process** passes the vector on to the **find** function. It is in **find** where problems may arise because **find** calls the vector's **at** method. If the **find** function calls the vector's **at** method with an out-of-bounds index, **at** will throw an exception. Observe that unlike in Listing 19.2 (vectorboundsexcept.cpp), the method that calls **vector<int>::at** does *not* do so within a **try/catch** block. This means **find** cannot catch any exceptions that **vector<int>::at** might throw.

The following shows a sample run of Listing 19.3 (callchainexception.cpp):

```
Creating Tracker (global)
-----[Entering main]-----
Creating Tracker (main)
-----[Entering process]-----
Creating Tracker (process)
Enter a range: 0 2
-----[Entering find]-----
Creating Tracker (find)
Enter item to locate: 4
-----[Leaving find]-----
Destroying Tracker (find)
Not there
-----[Leaving process]-----
Destroying Tracker (process)
-----[Leaving main]-----
Destroying Tracker (main)
Destroying Tracker (global)
```

In this output of Listing 19.3 (callchainexception.cpp) we see the following sequence of events:

1. The program creates the global **Tracker** object.
2. The program calls **main**.
3. The **main** function creates its **Tracker** object.
4. **main** calls **process**.
5. The **process** function creates its **Tracker** object.
6. The **process** function obtains the user's input for range values. Here the users entered the range 0...2.
7. The **process** function calls **find**.
8. The **find** function creates its **Tracker** object.
9. The **find** function obtains the user's input for the element to seek. Here the user enters 4 (which is not present in the vector).
10. The **find** function executes its last statement.
11. When **find** returns to **process**, the program destroys **find**'s local **Tracker** object.
12. **process** uses **find**'s result to print the message that the element sought is not present in the vector.
13. The **process** function executes its last statement.
14. When **process** returns to **main**, the program destroys **process**'s local **Tracker** object.
15. The **main** function executes its last statement.
16. When **main** returns, the program destroys **main**'s local **Tracker** object.
17. When the program terminates it destroys the global **Tracker** object.

Figure 19.1 provides a more graphical representation of this process. In this case the user did not enter a range in the **process** function that would cause a problem in **find**, so no exceptions arise. Since the program's execution produced no exceptions, it did not execute the printing statement in the **catch** block.

To see what happens during an exception consider the following sample run of Listing 19.3 (callchainexception.cpp):

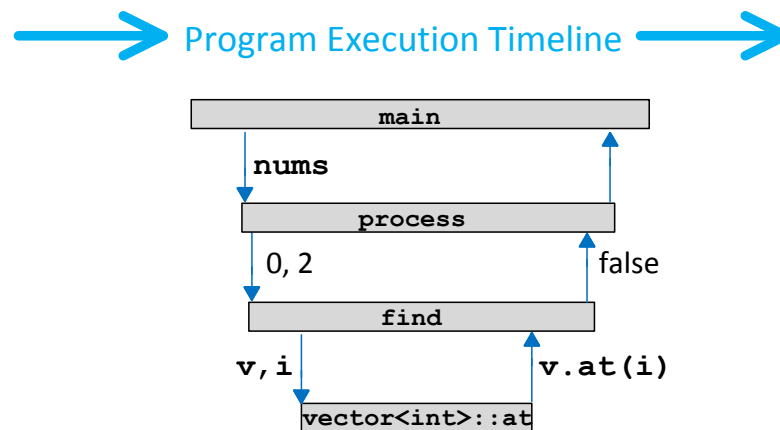


Figure 19.1: Function call chain without an exception. The `vector<int>::at` method, the `find` function, and the `process` function all complete their work and return as usual to their callers.

```

Creating Tracker (global)
-----[Entering main]-----
Creating Tracker (main)
-----[Entering process]-----
Creating Tracker (process)
Enter a range: 10 20
-----[Entering find]-----
Creating Tracker (find)
Enter item to locate: 4
Destroying Tracker (find)
Destroying Tracker (process)
nums vector bounds exceeded
-----[Leaving main]-----
Destroying Tracker (main)
Destroying Tracker (global)

```

The user supplied range of 10...20 definitely is outside the bounds of `main`'s `nums` vector. Because the program does not print `find`'s exit message we know the `find` function does not complete its execution as usual. This is because the `vector<int>::at` method called by `find` throws an exception. Furthermore, it does not appear that `find` returns normally to `process`, since `process` does not print its exit message either. We do see, though, that the program does properly destroy the the local **Tracker** objects owned by `find` and `process`. The program's execution path lands immediately back in `main`, which we see does exit normally after executing the statement in its `catch` block. The program finishes by destroying `main`'s **Tracker** object followed by destroying the global **Tracker** object. Figure 19.2

better illustrates the program's execution.

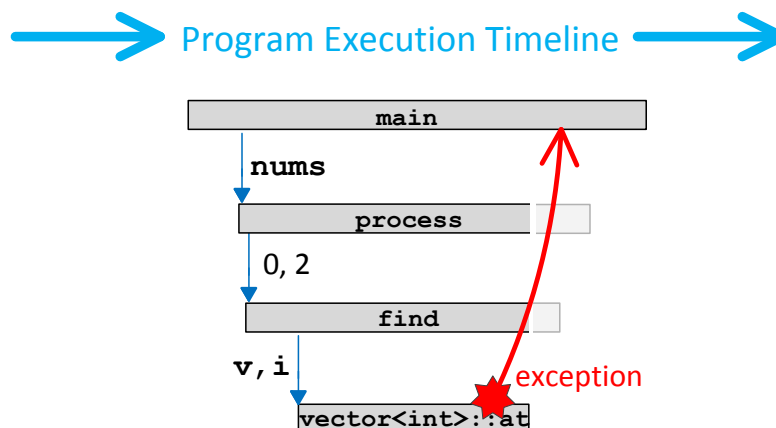


Figure 19.2: Function call chain interrupted by an exception. Neither the `vector<int>::at` method, the `find` function, nor the `process` function completes its work and returns as usual to its caller. Control passes back to the `catch` block associated with the nearest `try` block in the call chain.

Listing 19.3 (`callchainexception.cpp`) demonstrates that exceptions have the potential to radically alter the normal function call and return pathways. In one sense an exception works like a “super `goto`” statement. The program's execution jumps back to a previous point in its execution sequence, bypassing all functions in the call chain in between. Recall Section 6.3.2 discouraged the use of the `goto` statement except for its use to exit from within a nested loop. This is because the logic in code that uses indiscriminate `gotos` instead of the structured branching and looping constructs such as `if/else`, `switch`, `while`, and `for` tends to be obscure and difficult to extend and maintain. C++'s exception mechanism is different than a “super `goto`” in several important ways:

- A `goto` statement cannot jump to code outside of the function in which it appears. Exceptions have no such limitation.
- The programmer must specify a destination (label) for a `goto` statement. In the case of Listing 19.3 (`callchainexception.cpp`), the `vector<int>::at` method throws the exception. We can call `at` from many different functions within a single program and use `at` in many different programs. The code within the `at` method cannot possibly know where to go if it encounters an exceptional situation. An exception simply redirects the program's execution to the most recent `try/catch` block in the call chain that can handle the exception.

Listing 19.4 (`strboundsexcept.cpp`) follows.

Listing 19.4: `strboundsexcept.cpp`

```
#include <iostream>
```

```
#include <string>

using namespace std;

int main() {
    string s = "Wow";
    cout << s[3] << endl;
    try {
        cout << s.at(3) << endl;
    }
    catch (exception& e) {
        cout << e.what() << endl;
    }
}
```

Listing 19.5 (filereadexcept.cpp) follows.

Listing 19.5: filereadexcept.cpp

```
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

int main() {
    try {
        ifstream fin("data.dat");
        if (fin.good()) {
            int n;
            fin >> n;    // Size of dataset
            vector<int> data(n); // Allocate vector
            for (int i = 0; i <= n; i++) { // Error: should be <
                int value;
                fin >> value;
                data.at(i) = value;
            }
            fin.close();
            // Print the values
            for (auto value : data)
                cout << value << ' ';
            cout << endl;
        }
        else
            cout << "File does not exist" << endl;
    }
    catch (exception& e) {
        cout << e.what() << endl;
    }
}
```

19.3 Custom Exceptions

CAUTION! SECTION UNDER CONSTRUCTION

The standard C++ library has a limited number of standard exceptions. We can create our own custom exceptions for specialized error handling that our applications may require. Listing 19.6 (customfilereadexcept.cpp) defines a custom exception class, `FileNotFoundException`, derived from the standard `exception` class.

Listing 19.6: customfilereadexcept.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// Exception object to throw when a client attempts to
// open a text file via a name that does correspond to a
// file in the current working directory.
class FileNotFoundException : public exception {
    string message; // Identifies the exception and filename
public:
    // Constructor establishes the exception object's message
    FileNotFoundException(const string& fname):
        message("File \"" + fname + "\" not found") {}

    // Reveal message to clients
    const char *what() const {
        return message.c_str();
    }
};

// Creates and returns a vector of integers from data stored
// in a text file.
// filename: the name of the text file containing the data
// Returns a vector containing the data in the file, if possible
vector<int> load_vector(const string& filename) {
    ifstream fin(filename); // Open the text file for reading
    if (fin.good()) { // Did the file open successfully?
        vector<int> result; // Initially empty vector
        int n;
        fin >> n; // Size of data set
        for (int i = 0; i < n; i++) {
            int value;
            fin >> value; // Read in a data value
            result.push_back(value); // Append it to the vector
        }
        fin.close(); // Close the file
        return result; // Return the populated vector
    }
    else // Could not open the text file
        throw FileNotFoundException(filename);
}
```

```

}

int main() {
    try {
        vector<int> numbers = load_vector("values.data");
        for (int value : numbers)
            cout << value << ' ';
        cout << endl;
    }
    catch (exception& e) {
        cout << e.what() << endl;
    }
}

```

If we execute Listing 19.6 (customfilereadexcept.cpp) in a directory that does not contain a text file named values.data, it displays

```
File "values.data" not found
```

Since we derived the `FileNotFoundException` class from `exception`, any `FileNotFoundException` object also *is a* `exception` object (see Section 17.1). This enables the `main` function to catch a `FileNotFoundException` object in its `catch` body even though it is declared to catch `exception` objects.

The C++ standard library provides a number of exception classes, all derived from `exception`. The vector out-of-bounds exception objects thrown by the `at` method are instances of the standard `out_of_range` class. The `out_of_range` class is derived from the standard `logic_error` class which itself is derived from the standard `exception` class. Figure 19.3 shows the relationship among these exception classes.

19.4 Catching Multiple Exceptions

CAUTION! SECTION UNDER CONSTRUCTION

Listing 19.7 (filereadmultiexcept1.cpp) has the potential to throw more than one type of exception.

Listing 19.7: filereadmultiexcept1.cpp

```

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// Exception object to throw when a client attempts to
// open a text file via a name that does correspond to a
// file in the current working directory.
class FileNotFoundException : public exception {

```

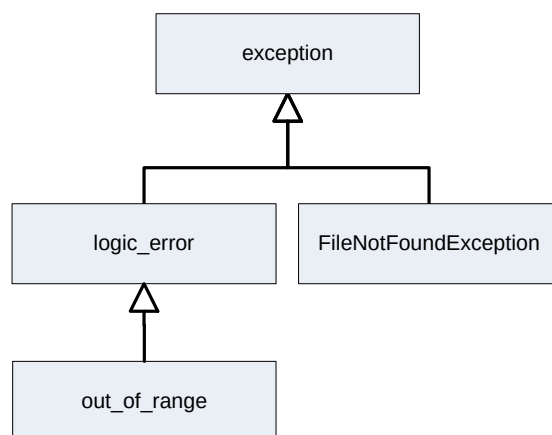


Figure 19.3: The relationship among the exception classes involved in Listing 19.6 (customfilereadexcept.cpp). The classes `exception`, `logic_error`, and `out_of_range` are standard exception classes, `FileNotFoundException` is our custom exception class.

```

    string message; // Identifies the exception and filename
public:
    // Constructor establishes the exception object's message
    FileNotFoundException(const string& fname):
        message("File \"" + fname + "\" not found") {}

    // Reveal message to clients
    const char *what() const {
        return message.c_str();
    }
};

// Creates and returns a vector of integers from data stored
// in a text file.
// filename: the name of the text file containing the data
// Returns a vector containing the data in the file, if possible
vector<int> load_vector(const string& filename) {
    ifstream fin(filename); // Open the text file for reading
    if (fin.good()) { // Did the file open successfully?
        int n;
        fin >> n; // Size of data set
        vector<int> result(n); // Allocate space for the vector
        int value, i = 0;
        while (fin >> value)
            result.at(i++) = value; // Insert it into the vector
        fin.close(); // Close the file
        return result; // Return the populated vector
    }
    else // Could not open the text file

```

```

        throw FileNotFoundException(filename);
    }

    int main() {
        try {
            vector<int> numbers = load_vector("values.data");
            for (int value : numbers)
                cout << value << ' ';
            cout << endl;
        }
        catch (exception& e) {
            cout << e.what() << endl;
        }
    }
}

```

The text file containing the integers to load into the vector has a specific format. The first integer read from the file is not a data element; instead, it specifies the number of integers that follow. This gives us two options for creating and populating the vector:

1. Create the vector with sufficient size up front and then use a loop to place to individual elements in their proper places.
2. Create an empty vector and then use a loop to **push_back** the individual elements.

Option 1 is more efficient than Option 2. Successive calls to `vector<int>::push_back` must reallocate and copy elements multiple times. Option 2 is safer than Option 1 because the number of elements specified at the start may not agree with the actual number of data elements in the file. If the specified number is too small, the function will attempt to overrun the bounds of the vector.

Listing 19.7 (filereadmultiexcept1.cpp) opts for efficiency at the expense of safety. This means it depends on a correctly created data file. If the file does not exist, `load_vector` will throw our **FileNotFoundException** custom exception:

```
File "values.data" not found
```

If the file exists and contains

```

3
10
20
30
40
50

```

the `load_vector` function will throw the standard `out_of_range` exception.

Listing 19.8 (filereadmultiexcept2.cpp) uses two `catch` blocks to print different messages depending on the exact type of exception object thrown by a `load_vector`.

Listing 19.8: filereadmultiexcept2.cpp

```

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// Exception object to throw when a client attempts to
// open a text file via a name that does correspond to a
// file in the current working directory.
class FileNotFoundException : public exception {
    string message; // Identifies the exception and filename
public:
    // Constructor establishes the exception object's message
    FileNotFoundException(const string& fname):
        message("File \"" + fname + "\" not found") {}

    // Reveal message to clients
    const char *what() const {
        return message.c_str();
    }
};

// Creates and returns a vector of integers from data stored
// in a text file.
// filename: the name of the text file containing the data
// Returns a vector containing the data in the file, if possible
vector<int> load_vector(const string& filename) {
    ifstream fin(filename); // Open the text file for reading
    if (fin.good()) { // Did the file open successfully?
        int n;
        fin >> n; // Size of data set
        vector<int> result(n); // Allocate space for the vector
        int value, i = 0;
        while (fin >> value)
            result.at(i++) = value; // Append it to the vector
        fin.close(); // Close the file
        return result; // Return the populated vector
    }
    else // Could not open the text file
        throw FileNotFoundException(filename);
}

int main() {
    try {
        vector<int> numbers = load_vector("values.data");
        for (int value : numbers)
            cout << value << ' ';
        cout << endl;
    }
    catch (out_of_range& e) {
        cout << "Error: vector bounds exceeded" << endl;
    }
}

```

```

        cout << e.what() << endl;
    }
    catch (FileNotFoundException& e) {
        cout << "Error: cannot open file" << endl;
        cout << e.what() << endl;
    }
}

```

19.5 Exception Mechanics

CAUTION! SECTION UNDER CONSTRUCTION

An exception handler within a `catch` block may take a few steps to handle the exception and then *re-throw* the exception or throw a completely different exception.

Listing 19.9: rethrow.cpp

```

#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

void filter(vector<int>& v, int i) {
    v.at(i)++;
}

void compute(vector<int>& a) {
    for (int i = 0; i < 6; i++) {
        try {
            filter(a, i);
        }
        catch (exception& ex) {
            cout << "*****" << endl;
            cout << "* For loop terminated prematurely" << endl;
            cout << "* when i = " << i << endl;
            cout << "*****" << endl;
            throw ex; // Rethrow the same exception
        }
    }
}

int main() {
    vector<int> list { 10, 20, 30, 40, 50 };
    try {
        compute(list);
    }
    catch (exception& e) {
        cout << "Caught an exception: " << e.what() << endl;
    }
    cout << "Program finished" << endl;
}

```


Listing 19.9 (rethrow.cpp) produces the following output:

```
*****
* For loop terminated prematurely
* when i = 5
*****
Caught an exception: invalid vector<T> subscript
Program finished
```

Notice that in Listing 19.9 (rethrow.cpp) the `compute` function does not create a new exception object; it simply re-throws the same exception object that it caught. This concept of intermediate exception handlers is important since a function invocation has complete knowledge of its own local context such as its local variables. This means that an intermediate function (or method) in the call chain has access to information that may be unavailable to all other functions (or methods) within that call chain. The `catch` block in the `compute` function of Listing 19.9 (rethrow.cpp) involves only local variables. The `catch` block within the `main` function cannot know the subscript that caused `compute`'s failure since `i` is local to `compute`. Some general exception guidelines include:

- Handle exceptions as close (up the call chain) as possible to the code that throws the exception.
- Use local information where applicable to correct as much of the problem as possible within that local context:
 - Local information which may be important for recovering from the exceptional situation is available; this local information is unavailable to functions or methods up the call chain.
 - The lower-level functions/methods that are prone to exceptions may be called by many different higher-level functions/methods. If the lower-level functions/methods are responsible for their own exception handling, then the higher-level functions/methods may be able to dispense with exception handling altogether making the lower-level methods more robust units of reuse.
 - The lower-level functions/methods may handle many different kinds of exceptions, do any local clean up they can based on the type of exception thrown, and then throw a different type of exception up to the higher-level methods. The higher-level method may just need to know there was a problem without needing to know exactly what the problem was. For example, in Listing 19.9 (rethrow.cpp), the `compute` function could throw a custom `ComputeException` object whenever `filter` throws its `out_of_range` instead of just passing up the call chain the `out_of_range` object it receives.

If total recovery is not possible or desirable at the lower levels of the call chain, re-throw the exception so an exception handler up the call chain can take steps to recover from the problem in a manner that makes sense in its context.

When a section of code throws an exception, the `catch` blocks within the nearest exception handler are checked in the order they appear in the C++ source code. The first `catch` block that matches the type of the exception thrown is the one executed. For example, Listing 19.10 (multipleexceptions.cpp) expects two types of exceptions, and it uses the catch-all exception in case any unexpected exceptions arise.

Listing 19.10: multipleexceptions.cpp

```
#include <iostream>
```

```

#include <fstream>
#include <vector>

using namespace std;

int main() {
    vector<int> a { 1, 2, 3 };
    try { // What possibly could go wrong?
        int n = a.size();
        cout << "Enter index in the range 0..." << n - 1 << ": ";
        // Will user enter a valid integer value?
        int i;
        cin >> i;
        // Will the user's index be in the range of valid indices?
        cout << "a[" << i << "] = " << a.at(i) << endl;
    }
    catch (out_of_range&) {
        cout << "Index provided is out of range" << endl;
    }
    catch (FormatException&) {
        cout << "Index provided is not an integer" << endl;
    }
    catch (...) { // What have we forgotten?
        cout << "Unknown error" << endl;
    }
}

```

When an exception is caught, its type is first compared to `out_of_range`. If it matches, the `out_of_range` `catch` block is executed. If not, the exception's type is then compared to `FormatException`. The `FormatException` `catch` block is executed in the case of a match. If neither of the two types match, the catch-all code is executed.

When inheritance is involved, the situation becomes more interesting. The comparison performed in each `catch` block is a test for *assignment compatibility*. A reference or a pointer to an instance of a derived class can be assigned to a variable declared to be a reference a pointer to a base class. This upcasting operation is always legal because of the *is a* relationship between a derived class and its base class. This means an exception object of type `out_of_range` is assignment compatible with both `out_of_range` variables and `exception` variables. Since the `catch` blocks are checked in the order they appear in the source code, the following code would have a problem:

```

try {
    // Execute some code that can throw an
    // out_of_range exception
    . . .
}
catch (exception& e) {
    // Do something here
}
catch (out_of_range& e) { // Error, this can never be reached
    // Do something else here
}

```

The `out_of_range` `catch` block will never execute since an `out_of_range` object is an `exception` instance also. For this reason it is illegal for a more specific type to follow a more general type in a sequence

of `catch` blocks. The above code fragment will not compile in any context. Thus, in a sequence of `catch` blocks, list the more specific exceptions first followed by the more general exceptions.

All C++ standard exception classes have `std::exception` as their direct or indirect base class. C++ does not require exception objects to be derived from `exception`; in fact, primitive types such as `int` and `char` can be thrown and caught as well. Because of this, the “catch all” exception clause is

```
catch (...) {
    // Catches any exceptions not caught by more specific
    // catch blocks in this try/catch block
}
```

The ellipses `(...)` stand for an exception of any type.

Any function or method can throw an exception object. Given an exception of type `E`, the statement to do so is:

```
throw E();
```

19.6 Using Exceptions

CAUTION! SECTION UNDER CONSTRUCTION

Exceptions should be reserved for uncommon errors. For example, the following code adds up all the elements in an integer vector named `vec`:

```
int sum = 0, n = vec.size();
for (int i = 0; i < n; i++) {
    sum += vec[i];
}
cout << "Sum = " << sum << endl;
```

This loop is fairly typical. Another approach uses exceptions:

```
sum = 0;
int i = 0;
try {
    while (true)
        sum += vec.at(i++);
}
catch (out_of_range&) {}
cout << "Sum = " << sum << endl;
```

Both approaches compute the same result. The second approach terminates the loop when the array access is out of bounds. It interrupts the statement

```
sum += vec.at(i++);
```

in midstream, so it does not incorrectly modify `sum`'s value. The second approach, however, *always* throws and catches an exception. The exception definitely is **not** an uncommon occurrence.

You should not use exceptions to dictate normal logical flow. While very useful for its intended purpose, the exception mechanism adds some overhead to program execution, especially when an exception

is thrown. This overhead is reasonable when exceptions are rare but not when exceptions are part of the program's normal execution.

Sometimes it is not clear when an exception is appropriate. Consider a function that returns the position of an element within a vector. The straightforward approach that does not use exceptions could be written:

```
int find(const vector<int>& a, int elem) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        if (list[i] == elem)
            return i; // Found it at position i
    return -1; // Element not present
}
```

Here a return value of -1 indicates that the element sought is not present in the array. Should an exception be thrown if the element is not present? The following code illustrates:

```
int find(const vector<int>& a, int elem) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        if (list[i] == elem)
            return i; // Found it at position i
    // Element not there; throw an exception
    throw ElementNotPresentException();
}
```

In the first approach, an unwary programmer may not check the result and blindly use -1 as a valid position. The exception code would not allow this to happen. However, the first approach is useful for determining *if* an element is present in the vector. If `find(x)` returns -1 , then `x` is not in the vector; otherwise, it is in the vector. If the exception approach is used, a client programmer cannot determine if an element is present without the risk of throwing an exception. Since exceptions should be rare, the second approach appears to be less than ideal. In sum,

- The first approach is more useful, but clients need to remember to properly check the result.
- The second approach provides an exception safety net, but an exception always will be thrown when searching for missing elements.

Which approach is ultimately better? The first version uses a common programming idiom and is the better approach for most programming situations. The exception version is a poorer choice since it is not uncommon to attempt to look for an element missing from a vector; exceptions should be reserved for uncommon error situations.

19.7 Summary

CAUTION! SECTION UNDER CONSTRUCTION

- Add item here.

19.8 Exercises

CAUTION! SECTION UNDER CONSTRUCTION

1. Add item here.

Chapter 20

Concurrent Programming

CAUTION! CHAPTER UNDER CONSTRUCTION

[Add content here]

20.1 Motivation

CAUTION! SECTION UNDER CONSTRUCTION

[Add content here]

20.2 Summary

CAUTION! SECTION UNDER CONSTRUCTION

- Add item here.

20.3 Exercises

CAUTION! SECTION UNDER CONSTRUCTION

1. Add item here.

Appendices

Appendix A

Using Visual Studio 2013 to Develop C++ Programs

This appendix describes the task of C++ software development under Visual Studio:

- To begin creating a C++ program, you must first launch Visual Studio 2013 from the Windows 8 start screen or other relevant shortcut. Figure A.1 shows the appropriate application tile to activate. You

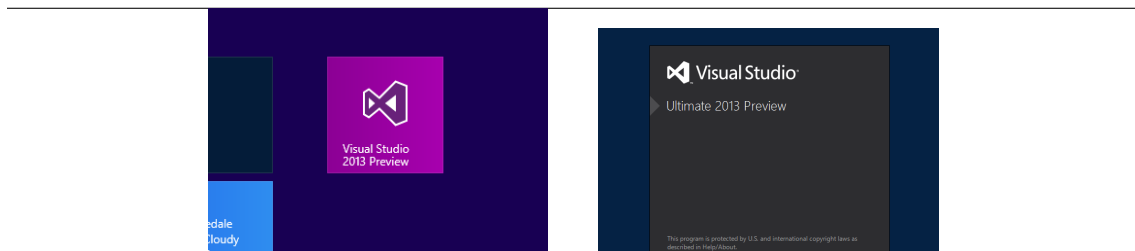


Figure A.1: Launching Visual Studio from the Windows 8 start screen and the ensuing splash screen that Visual Studio displays while it is launching.

soon should see a splash screen similar to the one shown on the right of Figure A.1.

If you have never before used the Visual Studio application, you must wait a few moments while it configures the development environment for you. At this point you will indicate that Visual C++ is your preferred development language (not necessary if you are using the Visual C++ Express Edition). You also may select a color scheme. The figures shown here reflect the *blue* color scheme.

Figure A.2 shows what Visual Studio looks like when it is fully loaded and ready to use.

- After Visual Studio has started, you begin the process of developing a C++ program by creating a new project. As Figure A.3 shows, you can create a new project by following the menu sequence: *File*→*New*→*Project*
- In the dialog that results, shown on the right of Figure A.3, you should choose the project type to be *Visual C++* in the left pane, and use the *Win32 Console Application* option in the center pane. In the name field near the bottom of the dialog, enter a name for the project; we will use the name *simple*. You may change the location to a different folder if you like, or even a different drive (such as a USB pen drive). In this example, we chose to not change the default location provided by Visual Studio.

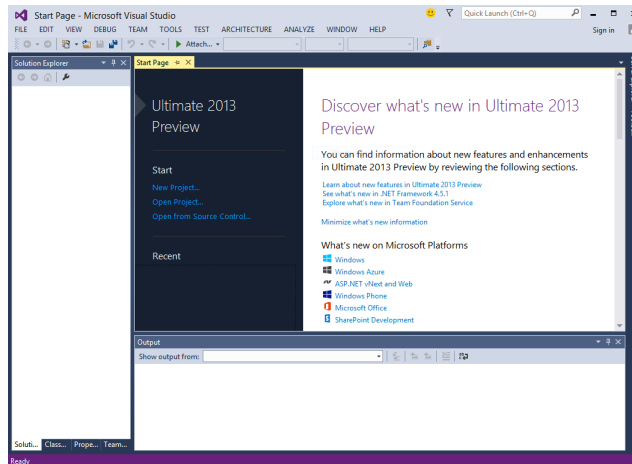


Figure A.2: Visual Studio Ready for Use

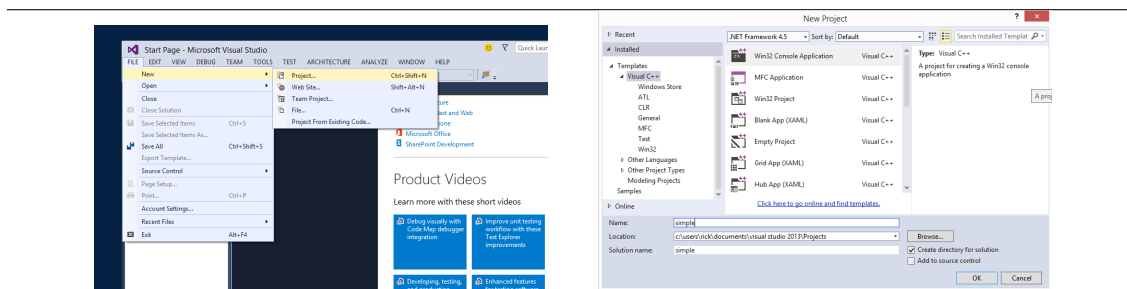


Figure A.3: Creating a New Project

- When you select **OK** on the project creation dialog, a *Win32 Application Wizard* as shown on the left of Figure A.4 appears. At this point, the instructions in the dialog say “Click **Finish** from any window to accept the current settings.” **Do not select Finish**; instead, select *Next* to continue. We have one more key step to complete so our project is set up correctly.
- In the subsequent *Applications Settings* dialog (see the right image in Figure A.4), select the *Empty project* checkbox. The dialog should look like the right image in Figure A.4 before you proceed. Choose *Finish* when you are ready to continue.
- At this point, the *Solution Explorer* panel shows the structure of the newly created, albeit empty, project. The left image in Figure A.5 shows the newly populated *Solution Explorer* pane.
- Right click on the *simple* element in the *Solution Explorer*. As shown on the right of in Figure A.5, select *Add* and *New Item* from the resulting pop-up menu.
- In the *Add New Item* dialog box, shown on the left in Figure A.6, select *C++ File (.cpp)* and enter a name for the C++ file in the text field at the bottom. You need not add **.cpp** to the end of the name, as Visual Studio will do that for you. The file here will be named *simple.cpp*. Press *Add* when done.
- As shown on the right in Figure A.6, the *Solution Explorer* pane now shows the file **simple.cpp**, and the large editor pane is ready for you to type in the source code for the program. The new source file is initially empty.

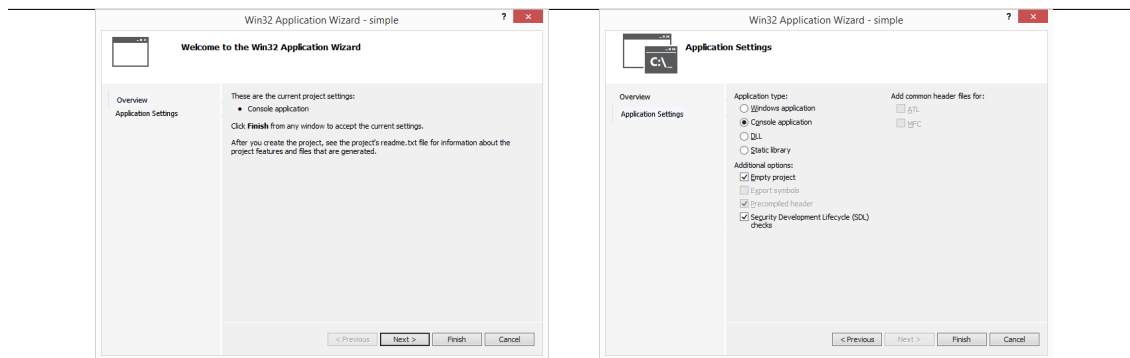


Figure A.4: Configuring the New Project

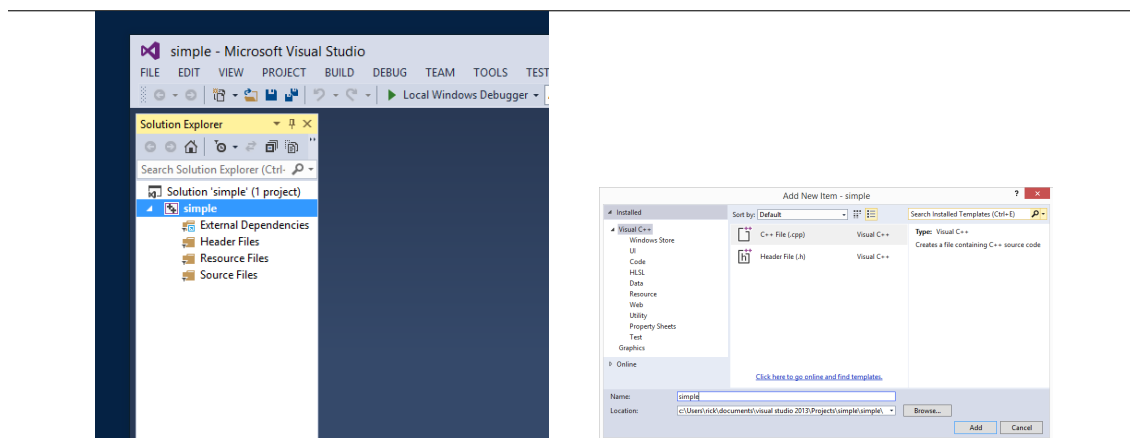


Figure A.5: Adding a C++ Source File

- In the editor pane with the *simple.cpp* tab at the top, type in the source code for our simple C++ program. Figure A.7 shows the completed code.
- You may save the source file at this point by selecting *Save* from the *File* menu or by pressing pressing at the same time the **Ctrl S** keys. If you do not save your program, Visual Studio will prompt you to do so before it builds and runs your program.

To run the program, select *Debug*→*Start Without Debugging*, as shown on the left of Figure A.8. (You also may use the **Ctrl F5** key sequence to build and run the program.) Visual Studio will attempt to build the executable program. It will prompt you to save your file if you have not saved it or have made changes since the last time you saved it. The progress of the build process is displayed in the *Output* panel in the bottom window. One of the lines has

```
1>Compiling. . .
```

and another later has

```
1>Linking. . .
```

The last few lines are very important:

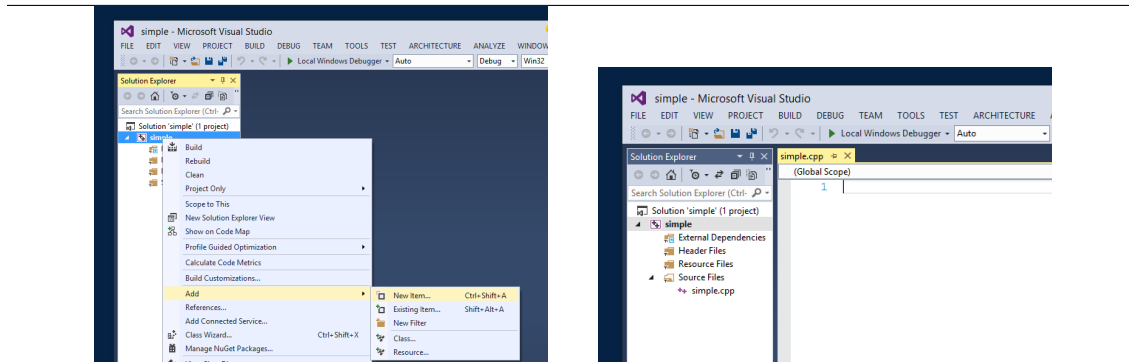


Figure A.6: Ready to Edit the C++ Source File

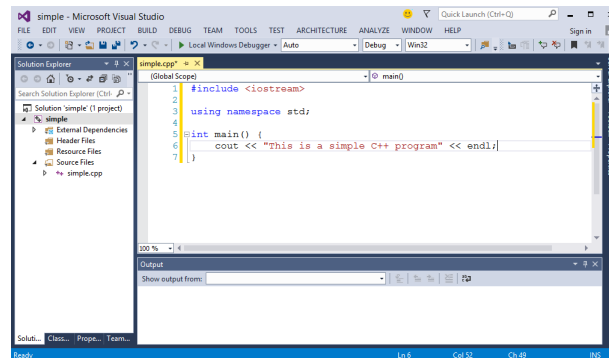


Figure A.7: Editor Pane

```
1>Build succeeded.
1>
1>Time Elapsed 00:00:02.98
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

These lines indicate the build was successful.

- A console window appears with the output of your program. The right image in Figure A.8 shows this text window. You can press any key on the keyboard to close the window. If the console window does not appear, you have typographical errors in your program; return to the editor, fix the errors, and try to run the program again.
- When you are finished with Visual Studio, select the *File*→*Exit* menu items as shown in Figure A.9.

These are the steps for writing a basic C++ program in Visual Studio 2013. While the steps initially may seem complex and tedious, they will become natural after you have written a few C++ programs.

When the program was run, the Visual Studio environment created a console window in which to run the program.

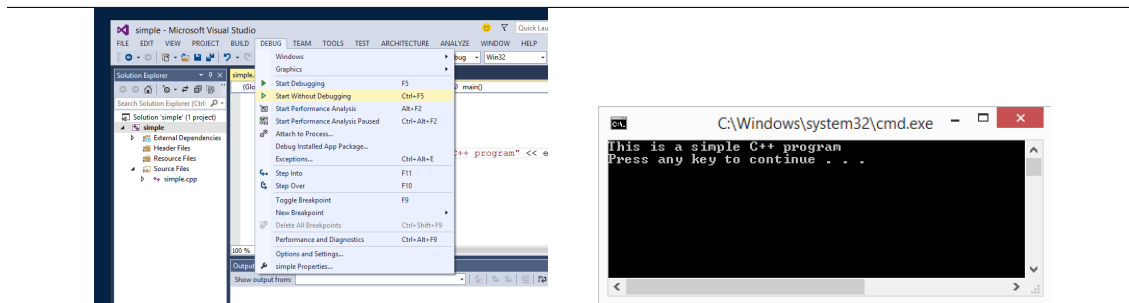


Figure A.8: Building and Running the Program

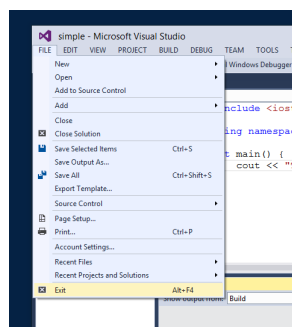


Figure A.9: Exiting Visual Studio

```
This is a simple C++ program!
Press any key to continue . . .
```

The first line of the output was printed by the program. The second line prompting the user to press any key to continue is added by the Visual C++ run-time environment so the console window stays visible long enough for the user to see the output. If you run the program from a the standard Windows command shell (CMD.EXE, usually found in the *Start* menu under *Accessories* and named *Command Prompt*), only the program's output will appear and the "Press any key to continue . . ." message will not appear.

The following summarizes the steps you should follow when writing a C++ program using the Visual Studio IDE:

1. In the main menu:
File→New→Project (or **Ctrl Shift N**)
2. Select "Win32 Console Project" and click "Next" to set the "Empty Project" option
3. In the Solution Explorer pane right click on "Source Files" and select
Add→New Item...
4. Select C++ File (.cpp) and enter the name of your file
5. Type in the source code for your program in the editor panel

6. Save your file:

File→Save *filename* (or **Ctrl S**)

7. In the main menu:

Debug→Start Without Debugging (or **Ctrl F5**)

It is possible to develop C++ programs without using Visual Studio's integrated development environment. Visual Studio comes with some additional tools for command-line development. Appendix B describes how you can edit the C++ source code with a standalone text editor and compile and run the program from the Windows command prompt (CMD.EXE). Command-line development under Linux and Mac OS X is covered in C. Some programmers prefer the freedom of using their favorite editor, a standalone compiler, and scripts to automate the build process.

Appendix B

Visual C++ Command Line Development

Visual Studio provides command line tools as an alternative to its integrated development environment. Some developers prefer the command line tools over the graphical IDE due to their simplicity, especially when developing simple console applications. The command line tools can be scripted to automate building and testing programs.

To use the command line tools, select the *Visual Studio Command Prompt* from the *Start* menu. A command console appears similar to the console that the IDE provides when executing a program (see Section 2.2). Instead of selecting menu items and interacting with dialog boxes, you type commands into the console window to build and run your program. To edit your program you can use the editor from the IDE, or you can use a standalone editor such as Notepad++ (<http://www.notepad-plus.sourceforge.net>) or Vim (<http://www.vim.org>). Both Notepad++ and Vim provide color syntax highlighting and folding like the built-in Visual Studio editor.

In order to use the command prompt version of Visual Studio, it is convenient to first create a folder in which you will place all your development files; for example, you could create a folder named `Code` under your Documents folder. Suppose the full path to this folder is

```
C:\Users\rick\Documents\Code
```

Once your development folder is created, you can launch the Visual Studio Command Prompt.

The first command you should issue in the console window is

```
C:
```

to ensure you are working on the correct drive. Next issue

```
cd \Users\rick\Documents\Code
```


The **cd** command means “change directory.” This command sets the console’s working directory to the development folder you previously created. If you are working in another folder, you can adjust the path given in the **cd** command to your work area.

You are ready to edit your code. If your environment is set up so that your editor program is in the system path, you can type

```
notepad++ simple.cpp
```

If you are using Vim, type

```
gvim simple.cpp
```

If the file `simple.cpp` already exists, you can type

```
devenv simple.cpp
```

to launch the Visual Studio editor.

Within the editor type in your program and then save it to a file.

When you have finished creating the source file, we will need to build the executable program. Switch back to the *Visual Studio Command Prompt* window. and issue the command

```
cl /W3 /EHsc /Za simple.cpp
```

The **cl** (“cee elle”, not “cee one”) command, which stands for *compile* and *link*, preprocesses, compiles, and links the program. The **/W3** switch uses the Level 3 warning level. Level 3 warnings provide more thorough analysis of your source code and can detect more programming mistakes. If you omit the **/W3** switch, by default the compiler operates at warning level 1. Warning level 1 is insufficient for catching many mistakes made by both beginning and experienced C++ programmers. By default the IDE compiler uses warning level 3, and the warning level can be changed by adjusting the project’s properties. On the command line, specifying **/W4** (the highest warning level) is just as easy as specifying **/W3**. It is better to use **/W4** so the compiler can do a more thorough check for potential programmer mistakes. The **/EHsc** is required for all the C++ programs we write. The **/Za** switch makes sure the compiler complies with standard C++ language rules and does not allow any Microsoft-specific language extensions.

If want to reduce the level of detail in the output produced by the compiler and linker, you can add the additional switch **/nologo**:

```
cl /W4 /EHsc /Za /nologo simple.cpp
```

Notice that we used the enhanced warning level (**/W4**) here. To see the complete list of compiler options, many more than you probably will ever need, type

```
cl /?
```

In C++ programs consisting of a single source file, the compiler produces an executable file with the same base name as the source file (simple, in our example), with an exe extension.

Once the program has been compiled successfully, you can run the program by entering the name of the executable produced. In our example, we would issue the command

```
simple
```

The program runs in the same window; no new console window is produced. When the program is finished executing, no “Press any key to continue” message is printed; you get simply a command prompt ready for you to enter your next command.

If you modify your source code in your editor, you must remember to recompile your code before you run it again. In the IDE, if you modify your code, the environment will suggest that you rebuild the project before running the program.



If the program consists of multiple source files, list all the required files on the command line. The exe file will be named after the first file listed; for example, the command

```
cl /W4 /EHsc /Za /nologo app.cpp list.cpp util.cpp
```

compiles the three C++ sources files and links the compiled code together to produce app.exe.

In the Visual Studio command prompt program you can enter the Visual Studio IDE at any time with the command

```
devenv
```

(**devenv** stands for “development environment.”) If you started Visual Studio in the IDE, not the command prompt, you can open a command prompt console window at any time by selecting the *Command Prompt* item under the *Tools* menu.

Appendix C

Developing C++ Programs with the GNU Tools

The tools provided in the GNU Compiler Collection (<http://gcc.gnu.org>), or GCC for short, include one of the most up-to-date standards compliant C++ compilers available. The GNU tools are free and are available for the major computing platforms:

- **Microsoft Windows XP–Windows 8.1**

Windows-based GCC tools can be found at the MinGW website <http://www.mingw.org>. The Nuwen site (www.nuwen.org) offers an easy to install custom distribution of MinGW with some additional useful C++ libraries.

- **Apple Macintosh OS X**

The MacPorts website (<http://www.macports.org/ports.php?by=variant&substr=gcc48>) brings the GCC toolset to Mac developers.

- **Linux**

Linux-based GCC tools are readily available for all the major Linux distributions. To get them, simply install the g++ development packages.

In this appendix we focus on building C++ programs with the GCC C++ compiler. While we will concentrate on the GCC tools, the code in this text is not GCC specific. The code herein aims to be C++ standards compliant and platform independent.

The name of the GNU C++ compiler is g++. It is a command-line tool, which means it is launched from a command shell—CMD.EXE in Windows or bash in the Mac and Linux environments.

There are a few commands that are essential for using the command-line environment. These center around manipulating and using files and folders. Our discussion assumes the Windows CMD.EXE shell, and we will note differences with Mac and Linux as needed. Mac OS X and Linux are both *Unix-like operating systems*.

When you run CMD.EXE in Windows, the Terminal application, or a bash terminal in Linux, the operating system presents a text-based console window. You type commands into this window and receive text feedback. Most of the commands are programs that you launch by typing their name. The following lists common commands useful for using the command-line GCC C++ tools:

- `dir`

When the command line interface starts it locates you in a particular folder, or directory, in the file system managed by the operating system. Your location is known as the *current working directory*. The command `dir` prints a list of the files in your current working directory. On Unix-like machine the `ls -l` command performs similar work. The command `dir /w` displays a wide listing of the files with fewer details; the `ls` command is equivalent on Unix-like systems.

- `mkdir name`

Creates a new subdirectory (subfolder) named *name* in the current directory. Subdirectories allow you to better organize your files.

- `cd name`

Changes you current directory to be that specified by *name*. Some examples of its use include

- `cd Code`

Changes the current working directory to be the subdirectory named *Code*. Here the name is relative to the current directory. It is an error to provide a name that does not correspond to a subdirectory of the current directory'

- `cd ..`

Changes the current working directory to its parent directory. Said another way, the current working directory is itself a subdirectory of some other directory. This command makes this other (parent) directory the current working directory. The `..` means “parent directory” or “parent folder.”

- `cd C:\Users\rick\Documents\Code`

Changes the current working directory to the completely specified one. The name here is known as a *full-path name*. The name is an absolute location; it is not relative to the current working directory. Unix-like systems do not have a drive letter (C:), and the backslashes (\) would be forward slashes (/).

- `cd .`

Changes the current working directory to itself, which effectively does nothing! This command does no useful work. The dot (.) in this context means “current working directory.”

- `del name`

Deletes the file named *name*. Use this command carefully. Unix-like systems use the `rm` command in its place.

- `ren name1 name2`

Renames the file *name1* to *name2*. The `move` command does the same thing, and Unix-like systems use the `mv` command to rename files.

In preparation for our programming, we can create a new directory for our code and make that new directory our current working directory with the commands:

```
mkdir Code
cd Code
```

We can edit our source code with a simple text editor such as Notepad or Wordpad for Windows, TextEdit on the Mac, or gEdit on Linux systems. More powerful programming editors such as Notepad++ (<http://>

[//notepad-plus.sourceforge.net](http://notepad-plus.sourceforge.net)) (Windows) and Vim (<http://www.vim.org>) (Windows, Mac, and Linux) make editing source code more convenient.

To see how the GNU C++ build system works, we will consider Listing 2.1 (`simple.cpp`) from Chapter 2. Use an editor to create a text file containing the code for Listing 2.1 (`simple.cpp`), and save the file with the name `simple.cpp`. In Windows, if `notepad++` is installed and set up properly, in the command shell you can type

```
notepad++ simple.cpp
```

This command brings up a separate window with an editor. We can type the source code for Listing 2.1 (`simple.cpp`) and save the file.

To create the executable program with GNU C++, back in our command-line window we issue the command

```
g++ -Wall -std=c++11 -o simple simple.cpp
```

This command has five parts:

- `g++ -Wall -std=c++11 -o simple simple.cpp`
`g++` invokes the GCC C++ build process. This one command runs the preprocessor, followed by the compiler, and finally the linker to produce the executable program. Even though the preprocessor and linker are also involved here, we generally simply say we are “compiling the program.”
- `g++ -Wall -std=c++11 -o simple simple.cpp`
`-Wall` stands for “warn all” and directs the compiler to be very strict when it is compiling the source code. It goes beyond checking that the program is well-formed C++ it also warns about code constructs that many programmers consider questionable because code of that nature often contains errors. The `-Wall` component can be interpreted to mean “warn about as many possible problems as possible.” You should compile all C++ programs with this directive because it enables the compiler to catch many careless programmer mistakes that otherwise might go unnoticed. Use of the `-Wall` directive is desirable for all C++ programmers from novice to expert.



The C++ standards committee is responsible for defining the C++ language. The committee refines the language specification over time. In addition to adding new features to the language, the committee also addresses shortcomings in existing language features. As a result of the committee’s work newer compilers can check more thoroughly the validity of C++ programs. Modern compilers do a better job of catching common programming errors.

The `-Wall` directive is optional because sometimes developers must work with older C++ source code. The code was written originally under more relaxed C++ language rules, but it may not be worthwhile to rewrite the code to bring it up to modern standards. If this older code has proven to be reliable, it may be compiled without the more stringent checks. All new code, however, should take advantage of the newer, better checking capabilities of the most up-to-date C++ compiler; therefore, always use `-Wall` on the compiler command line for code that you write.

- `g++ -Wall -std=c++11 -o simple simple.cpp`

The `-std=c++11` option directs the compiler to process the C++ source code according to the rules of the C++11 standard which the International Standards Organization (ISO) adopted in 2011. Since this standard is relatively new, if you omit this part of the command, `g++` assumes the programmer is writing the source code under the rules of the older, 1998 ISO C++ standard, usually referred to as C++98.

The code in this book is based on the C++11 standard.

- `g++ -Wall -std=c++11 -o simple simple.cpp`

The `-o simple` part of the command specifies the name of the executable program. The desired name appears after the `-o` part, so our executable program will be named `simple.exe` on Microsoft Windows systems. On Mac and Linux platforms the name of the executable file produced will be just `simple` (no `exe` name extension). If the C++ source code contains no errors, the compiler will produce the executable file named `simple.exe` (or `simple`). On a Windows Machine we can run the compiled program from the command line by typing the command

```
simple
```

(We also can use the full name, `simple.exe`.) On a Unix-like system (OS X or Linux), we instead would type

```
./simple
```

to execute the program.

- `g++ -Wall -std=c++11 -o simple simple.cpp`

`simple.cpp` specifies the name of the C++ source file we wish to compile. Here the file `simple.cpp` must reside in our current working directory for the compiler to process it.

The general form of the command to compile a C++ source file, therefore, is

```
g++ -Wall -std=c++11 -o executable source
```

where *source* is the name of the file containing the C++ source code, and *executable* is the name of the executable program the build process generates.

When you are finished with the executable file `simple.exe`, you can delete it with the following command:

```
del simple.exe
```

We can reconstruct the executable file whenever we like by simply recompiling the source file. Be careful not to delete accidentally the source file, or you have no choice but to type it in again.

Appendix D

Introduction to Binary Numbers and Computer Arithmetic

This appendix provides some additional details about the internal representation of C++'s numeric types.

D.1 The Integer Types

All data are stored internally in binary form. The binary number system is much simpler than the familiar decimal (base 10) number system because it uses only two digits: 0 and 1. The decimal system uses 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. Despite the lack of digits, every decimal integer has an equivalent binary representation. Binary numbers use a place value system not unlike the decimal system. Figure D.1 shows how the familiar base 10 place value system works.

...	4	7	3	4	0	6
...	10^5	10^4	10^3	10^2	10^1	10^0
...	100,000	10,000	1,000	100	10	1

$$\begin{aligned}
 473,406 &= 4 \times 10^5 + 7 \times 10^4 + 3 \times 10^3 + 4 \times 10^2 + 0 \times 10^1 + 6 \times 10^0 \\
 &= 400,000 + 70,000 + 3,000 + 400 + 0 + 6 \\
 &= 473,406
 \end{aligned}$$

Figure D.1: The base 10 place value system

With 10 digits to work with, the decimal number system distinguishes place values with powers of 10. Compare the base 10 system to the base 2 place value system shown in Figure D.2.

Since both binary and decimal numbers share the digits 0 and 1, we will use a subscript of 2 to represent binary numbers; therefore, 100 represents the decimal value *one hundred*, while 100_2 is the binary number *four*. Sometimes to be very clear we will attach a subscript of 10 to a decimal number, as in 100_{10} .

$$\begin{array}{rcccccc}
 \dots & \boxed{1} & \boxed{0} & \boxed{0} & \boxed{1} & \boxed{1} & \boxed{1} \\
 \dots & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 \dots & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}$$

$$\begin{aligned}
 100111_2 &= 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
 &= 32 + 0 + 0 + 4 + 2 + 1 \\
 &= 39
 \end{aligned}$$

Figure D.2: The base 2 place value system

With only two digits to work with, the binary number system distinguishes place values by powers of two.

In the decimal system, it is easy to add $3 + 8$:

$$\begin{array}{r}
 3 \\
 + 5 \\
 \hline
 8
 \end{array}$$

The sum $3 + 9$ is a little more complicated, as early elementary students soon discover:

$$\begin{array}{r}
 3 \\
 + 9 \\
 \hline
 \end{array}$$

The answer, of course, is 12, but there is no single *digit* that means 12—it takes two digits, 1 and 2. The sum is

$$\begin{array}{r}
 1 \\
 03 \\
 + 09 \\
 \hline
 12
 \end{array}$$

We can say $3 + 9$ is 2, *carry the 1*. The rules for adding binary numbers are shorter and simpler than decimal numnbers:

$$\begin{aligned}
 0_2 + 0_2 &= 0_2 \\
 0_2 + 1_2 &= 1_2 \\
 1_2 + 0_2 &= 1_2 \\
 1_2 + 1_2 &= 10_2
 \end{aligned}$$

Within the sum $1_2 + 1_2$ is 0_2 , *carry the 1*₂. A typical larger sum would be

$$\begin{array}{r}
 11 \\
 9_{10} = 1001_2 \\
 + 3_{10} = 11_2 \\
 \hline
 12_{10} = 1100_2
 \end{array}$$

Integers are whole numbers (no fractional parts), both positive and negative. The standard C++ integer types include **int**, **long**, **long long**, and **short**. These are distinguished by the number of bits required to store the type, and, consequently, the range of values that can be represented. Mathematical integers are infinite, but all of C++'s integer types correspond to finite subsets of mathematical integers. The

most commonly used integer type in C++ is `int`. All `ints`, regardless of their values, occupy the same amount of memory and, therefore use the same number of bits. The exact number of bits in an `int` is processor specific. A 32-bit processor, for example, is built to manipulate 32-bit integers very efficiently. A C++ compiler for such a system most likely would use 32-bit `ints`, while a compiler for a 64-bit machine would represent `ints` with 64 bits. On a 32-bit computer, the numbers 4 and 1,320,002,912 both occupy 32 bits of memory.

At the lower level computers deal exclusively with bits, so the sign of an integer must somehow be encoded as a bit. *Two's complement binary* is a specialized binary encoding used by most computer systems today. This format uses the leftmost bit to indicate the sign of the value: 0 for non-negative and 1 for negative. To better understand how the integer types work, consider the mythical type `nibble` (a *nibble* is defined formally as four bits, or one-half byte). C++ has no such `nibble` type, but a `nibble`'s smaller range of values makes it easier to explore the properties of integers in a computer system. A `nibble` uses four bits of storage, and Table D.1 shows the values that can be represented using only four bits in the two's complement binary representation.

Decimal Value	Binary Bit String
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Table D.1: The `nibble` values

The negative values have a one in the leftmost bit position. Notice that there are eight negative numbers but only seven positive numbers. Using this two's complement representation, there always will be one more negative value, regardless of the number of bits used. This is true for all of C++'s integer types since all commonly used computing systems today use two's complement binary integer representations.

The bit patterns for the positive numbers make sense; for example:

$$\begin{array}{r} 0\ 1\ 1\ 0 \\ \hline 8\ 4\ 2\ 1 \end{array} \implies 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 6$$

The bit patterns for negative numbers appear to be incorrect, but this is because they are represented in a special way that enables arithmetic to work out correctly. The following steps determine the value of a two's complement binary integer:

1. Once you note that the leftmost bit is a one, forget it. It is used to indicate a negative number, but it is not used to encode its value. For example, consider $-5 = 1011$. We consider only the bits **011**.

2. Invert all the remaining bits. This means change all the zeros to ones and all the ones to zeros. **011** becomes **100**.

3. Add one to the result:

$$\begin{array}{r} 100 \\ + \quad 1 \\ \hline 101 \end{array}$$

Since $101_2 = 5_{10}$, the original two's complement bit pattern is interpreted as -5 .

To see that arithmetic works, try adding $4 + (-7)$:

$$\begin{array}{rcl} 4_{10} & = & 0100_2 \\ + \quad -7_{10} & = & 1001_2 \\ \hline -3_{10} & = & 1101_2 \end{array}$$

or $-3 + 6$:

$$\begin{array}{rcl} -3_{10} & = & 1101_2 \\ + \quad 6_{10} & = & 0110_2 \\ \hline 3_{10} & = & 1\,0011_2 \end{array}$$

The extra one on the left results from a carry ($1_2 + 1_2 = 10_2$), but five digits cannot be stored in a **nibble**. Only the rightmost four digits remain, and they hold the correct answer.

Two important observations:

1. The fact that there are eight 4-bit values that begin with a zero also means that there are eight 4-bit values that begin with a one (negative values). Since zero begins with a zero (**0000**) and there are seven values left that are positive, this explains why there is always one more negative value than positive values in all the integer types (for example, 32-bit **ints** range from $-2,147,483,648 \dots +2,147,483,647$).
2. Seven is the largest **nibble**. The addition $7 + 1$:

$$\begin{array}{r} 0111 \\ + \quad 0001 \\ \hline 1000 \end{array}$$

results in a **nibble** that has a one in the first bit position—the value -8 !. This explains why increasing the value of an integer variable can cause it to have a negative value. The largest possible 32-bit **int** is $+2,147,483,647$. Add one to $+2,147,483,647$ and the result is $-2,147,483,648$, the smallest possible 32-bit integer. Thus, the smallest value for a given integer type immediately follows the largest value for that type. In fact, 32-bit two's complement **ints** follow the cyclic pattern shown in Figure D.3. In the figure, an addition moves a value clockwise around the circle, while a subtraction moves a value counterclockwise around the circle.

Item 1 is more of an intellectual curiosity, but Item 2 has significant consequences for unwary programmers. Attempting to exceed the maximum limit of a numeric type results in *overflow*, and attempting to exceed the minimum limit is called *underflow*. Integer arithmetic that overflows or underflows produces a valid, yet incorrect integer result. The compiler does not check that a computation will result in exceeding the limit of a type because it is impossible to do so in general (consider adding two integer variables whose values are determined at run time). Also significantly, an overflow or underflow situation does not generate a run-time error. It is therefore a logic error if a program performs an integral computation that, either as a final result or an intermediate value, is outside the range of the integer type being used.

When literal numeric values are used in a C++ program, as in

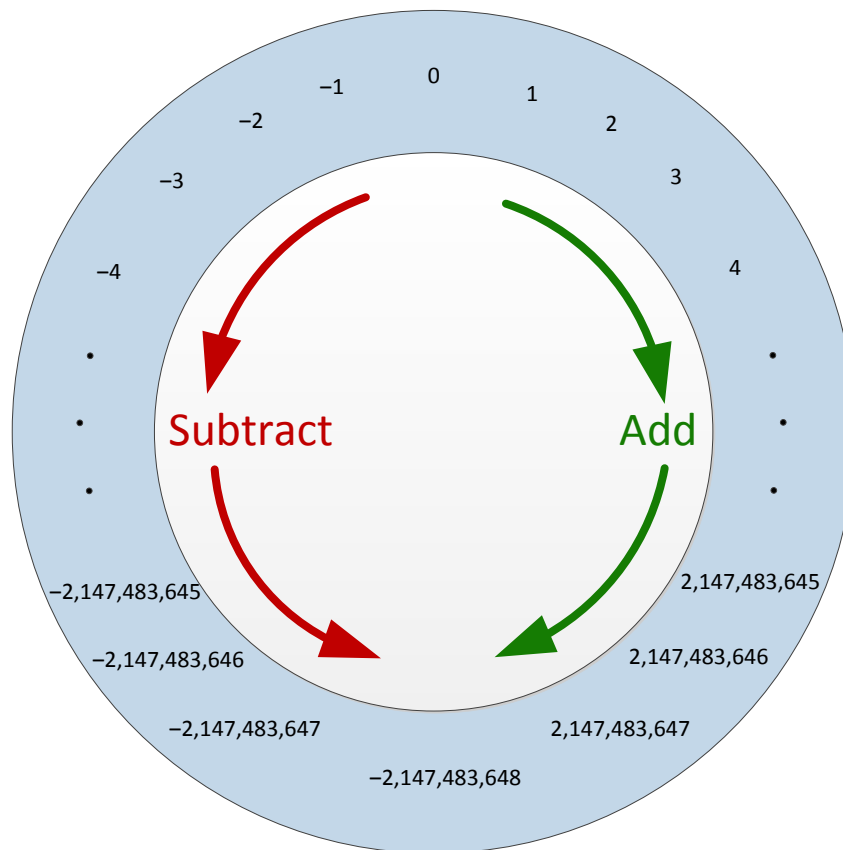


Figure D.3: The cyclic nature of two's complement integers

```
x = 2;
```

the number 2 has type `int`. To treat it as a `long`, append the letter `L` suffix:

```
x = 2;    // 2 is an int
x = 2L;   // 2 is a long
```

Lower case elle letter (`l`) also can be used, but it looks too much like the digit `1` (one) on many displays and printer fonts, so `L` is preferable.



All integer literals are `int` values unless an `l` or `L` is appended to the end of the number; for example, `2` is an `int` literal, while `2L` is a `long` literal.

D.2 The Floating-Point Types

Floating point types consist of **float** and **double**. Floating point numbers can have fractional parts (decimal places), and the term floating point refers to the fact the decimal point in a number can float left or right as necessary as the result of a calculation (for example, $2.5 \times 3.3 = 8.25$, two one-decimal place values produce a two-decimal place result). As with the integer types, the two floating-point types are distinguished by the number of bits of storage required and corresponding range of values. The type **float** stands for *single-precision floating-point*, and **double** stands for *double-precision floating-point*. Floating point numbers serve as rough approximations of mathematical *real numbers*, but as we shall see, they have some severe limitations compared to actual real numbers.

On most modern computer systems floating-point numbers are stored internally in exponential form according to the standard adopted by the Institute for Electrical and Electronic Engineers (IEEE 754). In the decimal system, *scientific notation* is the most familiar form of exponential notation:

One mole contains 6.023×10^{23} molecules.

Here 6.023 is called the mantissa, and 23 is the exponent.

The IEEE 754 standard uses binary exponential notation; that is, the mantissa and exponent are binary numbers. Single-precision floating-point numbers (type **float**) occupy 32 bits, distributed as follows:

Mantissa	24 bits
Exponent	7 bits
Sign	1 bit
Total	32 bits

Double-precision floating-point numbers (type **double**) require 64 bits:

Mantissa	52 bits
Exponent	11 bits
Sign	1 bit
Total	64 bits

The details of the IEEE 754 implementation are beyond the scope of this book, but a simplified example serves to highlight the limitations of floating-point types in general. Recall the fractional place values in the decimal system. The place values, from left to right, are

...	10,000	1,000	100	10	1	•	$\frac{1}{10}$	$\frac{1}{100}$	$\frac{1}{1000}$	$\frac{1}{10,000}$	...
...	10^4	10^3	10^2	10^1	10^0	•	10^{-1}	10^{-2}	10^{-3}	10^{-4}	...

Each place value is one-tenth the place value to its left. Move to the right, divide by ten; move to the right, multiply by ten. In the binary system, the factor is two instead of ten:

...	16	8	4	2	1	•	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	...
...	2^4	2^3	2^2	2^1	2^0	•	2^{-1}	2^{-2}	2^{-3}	2^{-4}	...

As in our **nibble** example (see Section D.1), consider a binary exponential number that consists of fewer bits than either **float**s or **double**s in C++. The mantissa consists of just three bits, all right of the

binary point. The base of the exponent is, of course, two, and a 2-bit exponent is used. Figure D.4 illustrates such a value. For simplicity, neither the mantissa nor the exponent is negative. Thus, the mantissa ranges

$$\begin{array}{c} \bullet \quad \boxed{1} \boxed{0} \boxed{1} \times 2^{\boxed{1} \boxed{0}} \\ \quad \quad \quad 2^{-1} 2^{-2} 2^{-3} \quad \quad \quad 2^1 2^0 \end{array}$$

Figure D.4: Simplified binary exponential value

from $.000_2$ (0_10) to $.111_2$, and the exponent ranges from 00_2 (0_10) to 11_2 (3_10). The number shown in Figure D.4 is thus

$$\begin{aligned} (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{(1 \times 2^1 + 0 \times 2^0)} &= \left(\frac{1}{2} + \frac{1}{8} \right) \times 2^2 \\ &= \frac{5}{8} \times 4 \\ &= 2.5 \end{aligned}$$

The range of these abbreviated floating-point numbers is $0 \dots 7$. This range is misleading, however, since it leaves the impression that any value in between could be represented. *2.5 can* be represented in this scheme, but *2.4 cannot*. This floating-point scheme is so simple that we easily can enumerate all the possible values, as Table D.2 does.

Mantissa Values	Exponent Values			
	2^{00_2} (1)	2^{01_2} (2)	2^{10_2} (4)	2^{11_2} (8)
$.000_2$ (0)	0	0	0	0
$.001_2$ (0.125)	0.125	0.25	0.5	1
$.010_2$ (0.25)	0.25	0.5	1	2
$.011_2$ (0.375)	0.375	0.75	1.5	3
$.100_2$ (0.5)	0.5	1	2	4
$.101_2$ (0.625)	0.625	1.25	2.5	5
$.110_2$ (0.75)	0.75	1.5	3	6
$.111_2$ (0.875)	0.875	1.75	3.5	7

Table D.2: A complete enumeration of our simplified floating-point numbers. Decimal equivalents appear in parentheses.

Figure D.5 plots these values on the real number line.

Table D.2 and Figure D.5 reveal several troubling issues:

1. There are many gaps; for example, the value 2.4 is missing and thus cannot be represented exactly (2.5 is the closest approximation).

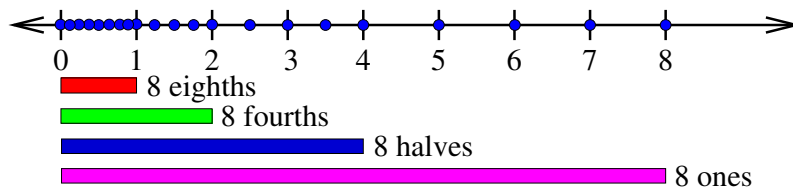


Figure D.5: A plot of the simplified floating-point numbers on the real number line.

2. The scheme duplicates some numbers; for example, three different bit patterns represent the decimal value 0.5:

$$0.100 \times 2^{00} = 0.010 \times 2^{01} = 0.001 \times 2^{10} = 0.5_{10}$$

This duplication limits the number of different values that can be represented by a given number of bits.

3. They are not uniformly dense. There are more values nearer to zero, and the numbers become sparse farther away from zero.

The integer types discussed in Section D.1 exhibit none of these weaknesses. All integers in a given range are present, no two bitstrings represent the same value, and the integers are uniformly distributed across their specified range. The standard floating-point types provided by C++ use many more bits than this simplified example, but the same problems shown here still exist to a much smaller degree. This is not solely a problem of C++'s implementation of floating-point numbers; all computer languages and hardware that adhere to the IEEE 754 standard exhibit these problems. Mathematical real numbers require an infinite amount of storage to be represented exactly, but any computer's memory is finite.

Listing D.1 (`imprecisedifference.cpp`) demonstrates the inexactness of floating-point arithmetic.

Listing D.1: `imprecisedifference.cpp`

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double d1 = 2000.5;
    double d2 = 2000.0;
    cout << setprecision(16) << (d1 - d2) << endl;
    double d3 = 2000.58;
    double d4 = 2000.0;
    cout << setprecision(16) << (d3 - d4) << endl;
}
```

The output of Listing D.1 (`imprecisedifference.cpp`) is:

```
0.5
0.57999999999999272
```

The first subtraction yields the correct answer. We now know that some floating-point numbers (like 0.5) have exact internal representations while others are only approximations. The exact answer for the second subtraction should be 0.58, and if we round the reported result to 12 decimal places, the answer matches. Floating-point arithmetic often produces results that are close approximations of the true answer. What are the ramifications for programmers of this inexactness of floating-point numbers? Section 9.4.6 shows how the misuse of floating-point values can lead to logic errors in programs.

In C++ source code, literal numbers that contain decimal places are by default type `double`. A `float` literals have an `f` or `F` suffix.



All floating-point literals are `double` values unless an `f` or `F` is appended to the end of the number; for example, `2.0` is a `double` literal, while `2.0f` is a `float` literal.

In C++ we may assign a `double` literal to a `float` variable. The compiler automatically narrows the `double` to a `float` as Listing D.2 (assignfloat.cpp) shows:

Listing D.2: assignfloat.cpp

```
#include <iostream>

using namespace std;

int main() {
    float number;
    number = 10.0; // OK, double literal assignable to a float
    cout << "number = " << number << endl;
}
```

The statement

```
number = 10.0;
```

assigns a `double` literal (10.0) to a `float` variable. You instead may explicitly use a `float` literal as:

```
number = 10.0f;
```


Bitwise Operators

Assignment	Short Cut
x = x & y;	x &= y;
x = x y;	x = y;
x = x ^ y;	x ^= y;
x = x << y;	x <<= y;
x = x >> y;	x >>= y;

Table E.1: The bitwise assignment operators

Appendix F

Arrays

An array in C++ is a variable that refers to a block of memory that can hold multiple values simultaneously; an array, therefore, represents a collection of values. An array has a name, and the values it contains are accessed via their position within the block of memory designated for the array. An array stores a sequence of values, and the values must all be of the same type. A collection of values all of the same type is said to be *homogeneous*.

F.1 Declaring and Using Arrays

Like any other variable, an array can be local or global, and it must be declared before it is used. The following code fragment declares an array variable named **list** that is assigned to a block of memory that can hold 25 integer values:

```
int list[25];
```

At the point of an array's declaration the square brackets indicate the amount of space reserved for the array's elements. The compiler generates machine language code that ensures space for 25 integers will be reserved at run time. On many systems integers are four bytes each, in which case an array of 25 integers would occupy 100 bytes of memory when the program executes. The exact amount of memory required depends on the size of an integer for a particular system.

If **list** is declared within a function, the array's contents are undetermined. This means that even though space is set aside for 25 elements, the values of those individual elements are unspecified. If instead the array is declared as a global variable, **list** will contain initially 25 zeros.

Once we declare an array, we can access its elements using the square bracket operator:

```
int list[3];      // Declare list to be an array of three ints
list[0] = 5 ;     // Make the first element 5
list[1] = -3 ;    // Make the second element -3
list[2] = 12 ;    // Make the last element 12
```

This code fragment shows that the square brackets used *after* the point of declaration allow us to access an individual element based on that element's position within the block of memory assigned to the array. The number within the square brackets indicates the distance from the beginning of the array. The expression

`list[0]` therefore indicates the element at the very beginning (a distance of zero from the beginning), and `list[1]` is the second element (a distance of one away from the beginning).

After executing these assignment statements, the `list` array conceptually looks like Figure F.1.

```
int list[3];  
list[0] = 5;  
list[1] = -3;  
list[2] = 12;
```

list		
5	-3	12
0	1	2

Figure F.1: A simple array with three elements

Once an array has been properly declared it can be used within a program. The square brackets, `[]`, are classified within C++ as a binary operator, since it requires two operands: an array name and an index. An array's elements can be accessed with the `[]` operator, as shown in the following code fragment:

```
int list[3];           // Declare list to be an array of three ints  
list[0] = 5 ;          // Make the first element 5  
list[1] = -3 ;         // Make the second element -3  
list[2] = 12 ;         // Make the last element 12  
cout << list[1];       // Print element in position 1  
list[1] = 7;          // Assign 7 to element in position 1  
cout << list[1];       // Print element in position 1
```

The `[]` operator is used to specify a position within an array. The array's name in conjunction with `[]` operator represents an element in the array.

In Section 10.6 we saw an assignment statement in which the left operand of the assignment operator involved more than just a single, simple variable. Observe that a statement such as

```
list[1] = -3 ;         // Make the second element -3
```

is a similar situation. The expression to the left of the assignment operator is not a simple, single variable name. This is legal because the expression to the left of the assignment operator refers to a memory location that can store a value. The memory location it identifies is a part of the memory reserved for the `list` array.

The array square brackets operator has two distinct meanings depending on the context:

1. At the array's declaration, for example,

```
double nums[10];
```



the number within the square brackets specifies the number of elements that the array can hold. The compiler uses this number along with the type of the array to determine how much memory to allocate for the array.

2. After the array's declaration, for example,

```
nums[3] = 10.45;
```

the number within the square brackets represents an index locating a specific element within the memory allocated to the array.

Declarations of arrays and non-array values can be interspersed, as in:

```
int list[5], x, s[3], *p, reg;
```

Here, **list** and **s** are integer arrays, but **x** and **reg** are simple integer variables, and **p** is a pointer to an integer (see Section 10.6, for a discussion of pointers).

The contents of an array may be initialized when it is declared. The following code fragment declares three arrays and specifies their initial contents:

```
int list[] = { 2, 4, 6, 8 };  
double collection[10] = { 1.0, 3.5, 0.5, 7.2 };  
char letters[] = { 'a', 'b', 'c' };
```

The elements of the array are provided by a comma-separated list within curly braces. This comma separated list of initial values is called an *array initialization list*. When an array initialization list is provided, the square brackets can be left empty as the compiler can calculate the array's size based on the number of elements in the list. If a number is provided within the square brackets, the array is allocated with the specified size, and the first positions within the array are filled with the elements specified in the initialization list.

The code fragment above declares three arrays: **list**, **collection**, and **letters**. The **list** array is created to hold four integer values consisting of 2, 4, 6, and 8. The **collection** array is created to hold 10 double-precision floating-point values, and the first four elements consist of 1.0, 3.5, 0.5, and 7.2. The **letters** array holds the three characters shown. Figure F.2 visualizes the contents of the three arrays.

We can observe three key points from Figure F.2:

- An array name represents the location of the first element in an array. For an array named **list**, the expressions **list** and **&list[0]** within C++ source code are equivalent.
- Elements in an array are stored in a contiguous block of memory. This means, for example, the memory occupied by the element at index 3 follows immediately after the memory occupied by the element at index 2 and immediately before the element at index 4. The size specified in the array's declaration determines the size this block of contiguous memory.

```
int list[] = { 2, 4, 6, 8 };
```

list

2	4	6	8
0	1	2	3

```
double collection[10] = { 1.0, 3.5, 0.5, 7.2 };
```

collection

1.0	3.5	0.5	7.2	?	?	?	?	?	?
0	1	2	3	4	5	6	7	8	9

```
char letters[] = { 'a', 'b', 'c' };
```

letters

'a'	'b'	'c'
0	1	2

Figure F.2: Array declarations and memory layout

- Elements in an array are located by a numeric index. The first element is at index *zero*, not one.

The array initialization list may be used only when an array is declared. It cannot be used to assign or reassign an array after the array has been declared; for example, the code

```
int nums[10];
nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }; // Illegal, will not compile
```

is illegal, and the compiler will report an error. The second statement is a simple assignment statement, not a declaration and initialization statement. An array variable may not appear all by itself on the left side of the assignment operator. An array, therefore, may be modified after its point of declaration only by reassigning individual elements using the `[]` operator.

In an expression such as

```
list[3]
```

the expression within the square brackets is called an *index* or *subscript*. The subscript terminology is borrowed from mathematicians who use subscripts to reference elements in a mathematical vector (for example, V_2 represents the second element in vector V). Unlike the convention often used in mathematics, however, the first element in an array is at position *zero*, not one. The expression `list[2]` can be read aloud as “**list** sub two.” The index indicates the distance from the beginning; thus, the very first element is at a distance of zero from the beginning of the array. The first element of array **a** is **a[0]**. As a consequence of a zero beginning index, if array **a** holds n elements, the last element in **a** is **a[n − 1]**, not **a[n]**.

To assign a specific element in an array you must specify the element's position. The following code fragment

```
double nums[10];
nums[0] = 0.5;
nums[3] = 5.3;
nums[9] = 0.2;
```

declares an array of **doubles** that initially holds 10 elements. The first element is set to 0.5, the item at index 3 (that is, the *fourth* item) is assigned 5.3, and the last element in the array is 0.2. The values of the other elements in the array **nums** are unspecified. Figure F.3 illustrates this code sequence.

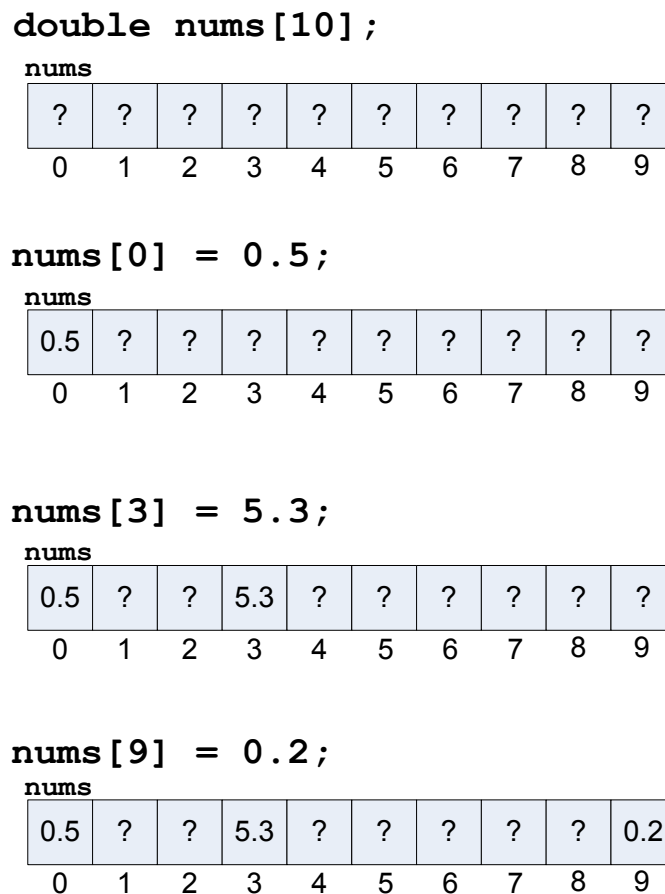


Figure F.3: Array element assignment

The elements of an array can be treated as any other variable; for example,

```
// Print the fourth element
cout << nums[3] << endl;
// The third element is the average of two other elements
```

```

nums[2] = (nums[0] + nums[9])/2;
// Get elements at indices 1 and 4 from the user
cin >> nums[1] >> nums[4];

```

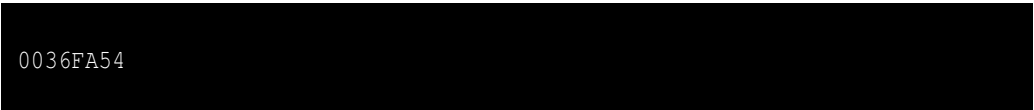
In C++ source code an array name references a block of memory, not any individual element. The following code fragment:

```

int nums[] = { 2, 4, 6, 8, 10 };
cout << nums << endl;

```

is unable to print the elements of the array. Instead, on most systems it prints a cryptic string like



```
0036FA54
```

This is a hexadecimal (base 16) number representing a memory address—the actual beginning address of the block of memory assigned to the array.

The expression within the `[]` operator must evaluate to an integer; some examples include

- an integer literal: `a[34]`
- an integer variable: `a[x]`
- an integer arithmetic expression: `a[x + 3]`
- an integer result of a function call that returns an `int`: `a[max(x, y)]`
- an element of an integer array: `a[b[3]]`

The action of moving through an array visiting each element is known as *traversal*. `for` loops are ideal for array traversals. If `a` is an integer array containing 10 elements, the following loop prints each element in `a`:

```

for (int i = 0; i < 10; i++)
    cout << a[i] << endl;

```

The loop control variable, `i`, steps through each valid index of array `a`. `i` starts at 0 and ends at 9, the last valid position in array `a`.

The following loop prints contents of array `a` in reverse order:

```

for (int i = 9; i >= 0; i--)
    cout << a[i] << endl;

```

The following code produces an array named `set` containing the integer sequence 0, 1, 2, 3, 4, 5, 6, 7, 8, 9:

```

int set[10];
for (int i = 0; i < 10; i++)
    set[i] = i;

```

The compiler will insist that the programmer use an integral value for an index, but the programmer must ensure that the index used is within the bounds of the array. Since the index may consist of an arbitrary integer expression whose value cannot be determined until run time, the compiler cannot check for out-of-bound array accesses; for example, in the code

```

int x;
int v[10]; // Make an array with 10 spaces available
cin >> x; // User enters x at run time
v[x] = 1; // Is this OK? What is x?

```

the compiler cannot predict what number the user will enter. This means that misuse of an array index can lead to run-time errors. To illustrate the problem, consider Listing F.1 (arrayoutofbounds.cpp).

Listing F.1: arrayoutofbounds.cpp

```

#include <iostream>
using namespace std;

int main() {
    const int SIZE = 3;
    int a[SIZE] = { 5, 5, 5 };
    // Print the contents of the array
    cout << "a contains ";
    for (int i = 0; i < SIZE; i++)
        cout << a[i] << " ";
    cout << endl;
    // Change all the 5s in array a to 8s
    for (int i = 0; i <= SIZE; i++) // Bug: <= should be <
        a[i] = 8;
    // Reprint the contents of the array
    cout << "a contains ";
    for (int i = 0; i < SIZE; i++)
        cout << a[i] << " ";
    cout << endl;
}

```

Listing F.1 (arrayoutofbounds.cpp) contains a logic error; the reassignment loop goes one past the end of array **a**. Attempting to access elements outside the bounds of an array produces what is known as *undefined behavior*. The C++ language standard uses this term to indicate a program's behavior is not specified, and compiler writers are free to do whatever they want. Some possible behaviors include:

- The compiled code could check to ensure the index is within range and produce an intelligible run-time error message,
- the program could simply crash mysteriously,
- the program could continue with no indication of a problem and appear to behave correctly most of the time,

or any number of other possible outcomes. In other words, the actual program behavior is system dependent and compiler dependent. The code within a program that exhibits undefined behavior is considered a logic error, since its action is inconsistent across platforms and compilers. Simply said, the program's behavior is unpredictable. Unpredictable behavior is incorrect behavior.

In most cases, an out-of-bounds access simply accesses memory outside the array. If this includes memory that does not belong to the executing program, modern operating systems will kill the program and produce some error message. Under Visual C++ when the program is built as a debug version, the program prints the contents of the array the first time but crashes before it can print it out a second time. Microsoft Windows then displays the dialog box shown in Figure 11.4. The program running under Linux or OS X may simply print in the console:

```
Segmentation fault
```

If your program is using an array, and it terminates with such a message, you should check its source code carefully for places where out-of-bounds array accesses are possible.

The following code fragment shows some proper and improper array accesses:

```
int numbers[10]; // Declare the array
numbers[0] = 5;  // Put value 5 first
numbers[9] = 2;  // Put value 2 last
numbers[-1] = 5; // Out of bounds; first valid index is 0
numbers[10] = 5; // Out of bounds; last valid index is 9
numbers[1.3] = 5; // Compile error, 1.3 is not an int
```

Notice that the compiler does check that the type of the index is a valid integer expression.

In an array traversal with a **for** loop such as

```
for (int i = 0; i < SIZE; i++)
    cout << a[i] << endl;
```

it is easy to ensure that **i** cannot fall outside the bounds of array **a**, but arbitrary index values should be checked before their use:

```
int x;
int a[10]; // Make an array with 10 spaces available
cin >> x;  // User enters x at run time
// Ensure x is within the bounds of the array
if (0 <= x && x < 10)
    a[x] = 1; // This is safe now
else
    cout << "Index provided is out of range" << endl;
```

Here, the **if** statement ensures the array access is within the array's bounds.

F.2 Arrays and Functions

An array can be passed to a function, as shown in Listing F.2 (arraytofunc.cpp)

Listing F.2: arraytofunc.cpp

```
#include <iostream>

using namespace std;

/*
 * print(a, n)
 * Prints the contents of an int array
 * a is the array to print
 * n is the size of the array
 */
```

```

void print(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}

/*
 * sum(a, n)
 *     Adds up the contents of an int array
 *     a is the array to sum
 *     n is the size of the array
 */
int sum(int a[], int n) {
    int result = 0;
    for (int i = 0; i < n; i++)
        result += a[i];
    return result;
}

int main() {
    int list[] = { 2, 4, 6, 8 };
    // Print the contents of the array
    print(list, 4);
    // Compute and display sum
    cout << sum(list, 4) << endl;
    // Zero out all the elements of list
    for (int i = 0; i < 4; i++)
        list[i] = 0;
    // Reprint the contents of the array
    print(list, 4);
    // Compute and display sum
    cout << sum(list, 4) << endl;
}

```

As shown in the **print** function's definition:

```
void print(int a[], int n)
```

when an array formal parameter is declared, its square brackets are left empty. The programmer can supply a number within the square brackets, but it is ignored by the compiler. When calling a function that accepts an array parameter, as in

```
print(list, 4);
```

the programmer must pass the array's size along with the array name. This is because an array simply references a block of memory and has no notion of its size.

An undecorated array name by itself in C++ source code behaves like a constant pointer to the beginning element of the array. Consequently, when an array is passed as an actual parameter during a function call, as in Listing F.2 (arraytofunc.cpp):

```
print(list, 4);
```

the function is passed the address of the array—the array's address is bound to the formal parameter. So, while the function cannot affect the address of the array itself, the function has total access to the array's

contents. In Listing F.3 (cleararray.cpp), the function **clear** modifies the contents of any array sent to it, making all the elements zero.

Listing F.3: cleararray.cpp

```
#include <iostream>

using namespace std;

/*
 * print(a, n)
 * Prints the contents of an int array
 * a is the array to print
 * n is the size of the array
 */
void print(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}

/*
 * clear(a, n)
 * Makes all the elements of array a zero
 * a is the array to zero out
 * n is the size of the array
 */
void clear(int a[], int n) {
    for (int i = 0; i < n; i++)
        a[i] = 0;
}

int main() {
    int list[] = { 2, 4, 6, 8 };
    // Print the contents of the array
    print(list, 4);
    // Zero out the array
    clear(list, 4);
    // Reprint the contents of the array
    print(list, 4);
}
```

The output of Listing F.3 (cleararray.cpp) is



```
2 4 6 8
0 0 0 0
```

The **clear** function actually modifies the contents of **main**'s **list** array. The function works on the array's actual elements, not copies of its elements.

Arrays, therefore, by default are passed by reference with respect to the elements they contain. By default, an array's contents are open to corruption by errant functions. In order to protect an array so that a function may read its contents but not modify its contents, the parameter must be declared **const**, as in

```
int sum(const int a[], int n)
```

In Listing F.2 (arraytofunc.cpp) there is no reason why the **print** and **sum** functions should be able to modify the contents of their array parameters, so the **const** specifier should be added to their definitions.

As a general rule, all functions that accept array parameters should declare the array contents as **const** unless they need to modify the elements of the array.



As a general rule, all functions that accept array parameters should declare the array contents as **const** unless they need to modify the elements of the array. For example, use

```
int sum(const int a[], int n)
```

rather than

```
int sum(int a[], int n)
```

if the **sum** function must be able to examine the contents of the array but is not intended to modify its contents.

C++ does not directly support empty arrays. A physical array must contain at least one element. Sometimes it is appropriate to consider an array that is conceptually empty; for example, what if we wish to pass an “empty” array to the **sum** function? We can pass 0 as the second parameter since an empty array contains no elements, but what should we pass as the first parameter? Any array will do, but there is no need create an real array when none truly is needed. Instead, we may use **nullptr**:

```
quantity = sum(nullptr, 0);
```

If you scrutinize the body of the **sum** function, you will see that this call will assign to **quantity** the correct answer, namely zero.

F.3 Prime Generation with an Array

Listing F.4 (fasterprimesarray.cpp) uses an algorithm developed by the Greek mathematician Eratosthenes who lived from 274 B.C. to 195 B.C. The principle behind the algorithm is simple: Make a list of all the integers two and larger. Two is a prime number, but any multiple of two cannot be a prime number (since a multiple of two has two as a factor). Go through the rest of the list and mark out all multiples of two (4, 6, 8, ...). Move to the next number in the list (in this case, three). If it is not marked out, it must be prime, so go through the rest of the list and mark out all multiples of that number (6, 9, 12, ...). Continue this process until you have listed all the primes you want.

Listing F.4: fasterprimesarray.cpp

```
// File primesieve.cpp

#include <iostream>
#include <ctime>

using namespace std;

// Display the prime numbers between 2 and 500,000 and
```

```

// time how long it takes

// Largest potential prime considered
//const int MAX = 500000;
const int MAX = 500;

// Each position in the Boolean array indicates
// if the number of that position is not prime:
// false means "prime," and true means "composite."
// Initially all numbers are prime until proven otherwise
bool nonprimes[MAX]; // Global array initialized to all zeros

int main() {
    clock_t start_time = clock(); // Record start time

    // First prime number is 2; 0 and 1 are not prime
    nonprimes[0] = nonprimes[1] = true;

    // Start at the first prime number, 2.
    for (int i = 2; i < MAX; i++) {
        // See if i is prime
        if (!nonprimes[i]) {
            cout << i << " ";
            // It is prime, so eliminate all of its
            // multiples that cannot be prime
            for (int j = 2*i; j < MAX; j += i)
                nonprimes[j] = true;
        }
    }
    cout << endl; // Move cursor down to next line
    // Print the elapsed time
    cout << "Elapsed time: "
         << static_cast<double>(clock() - start_time)/CLOCKS_PER_SEC
         << " seconds" << endl;
}

```

Recall Listing 8.8 (measureprimespeed.cpp), which also prints all the prime numbers up to 500,000. Using redirection (see Section 8.4), Listing 8.8 (measureprimespeed.cpp) takes 77 seconds. In comparison, Listing F.4 (fasterprimesarray.cpp) takes about one second to perform the same task on the system. This is comparable to the square root version, Listing 8.5 (moreefficientprimes.cpp), which takes two seconds to run. If the goal is to print the prime numbers up to 1,000,000, the original version averages 271 seconds, or about four and one-half minutes. The square root version averages 4.5 seconds. The new, array-based version averages 2 seconds. For 5,000,000 values the unoptimized original version takes a little over an hour and 33 minutes, the square root version takes a respectable 39 seconds, but array version averages only 9 seconds to run.

F.4 Multidimensional Arrays

The arrays we have seen thus far have been one dimensional—simple sequences of values. C++ supports higher-dimensional arrays. A *two-dimensional* array is a block of memory that is best visualized as a table with rows and columns. The statement

```
int a[2][3];
```

declares **a** to be a two-dimensional (2D) array of integers. A 2D array is sometimes called a *matrix*. In this case, the declaration specifies that array **a** contains two rows and three columns. Figure F.4 shows the logical structure of the array created by the following sequence of code:

```
int a[2][3]; // a is a 2D array
a[0][0] = 5;
a[0][1] = 19;
a[0][2] = 3;
a[1][0] = 22;
a[1][1] = -8;
a[1][2] = 10;
```

a

0	5	19	3
1	22	-8	10
	0	1	2

Figure F.4: A 2×3 two-dimensional array

The two-dimensional array **a** is said to be a 2×3 array, meaning it has two rows and three columns (as shown in Figure F.4). Rows are arranged horizontally, and the values in columns are arranged vertically. In each of the assignment statements above, for example

```
a[1][0] = 22;
```

the first index (here 1) signifies the row and the second index (here 0) denotes the column of the element within the array.

Using a syntax similar to the initialization lists of one-dimensional arrays, the 2D array **a** from above could have been declared and created as:

```
int a[2][3] = { { 5, 19, 3 },
                { 22, -8, 10 } };
```

Note that each row appears within its own set of curly braces, and each row looks like 1D array initialization list. You may omit the first index, as shown here:

```
int a[][3] = { { 5, 19, 3 },
               { 22, -8, 10 } };
```

For 2D arrays initialized in this manner the first subscript is optional, but the second subscript (that is, the size of each column) is required.

To access an element of a 2D array, use two subscripts:

```
a[r][c] = 4;           // Assign element at row r, column c
cout << a[m][n] << endl; // Display element at row m, column n
```

The following function prints the contents of a **ROWS** × **COLUMNS** 2D array of **doubles**:

```
void print(const double m[ROWS][COLUMNS]) {
    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLUMNS; col++)
            cout << setw(5) << m[row][col];
        cout << endl;
    }
}
```

The declaration of the parameter **m** is quite complicated. We can simplify the syntax by using a C++ **typedef** statement. The statement

```
typedef double Matrix[ROWS][COLUMNS];
```

defines a new type named **Matrix**. If you omit the word **typedef** from the above statement, it becomes a declaration of a 2D matrix named **Matrix**. The **typedef** keyword changes the meaning of the statement so that instead of declaring a variable it defines a new type. **typedef** statements usually appear near the top of a source file and most often have global scope. It is uncommon to see a local **typedef** statement within a function body.

Given the **typedef** statement defining the new type name **Matrix**, the parameter for the **print** function may be expressed more simply:

```
void print(const Matrix m) {
    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLUMNS; col++)
            cout << setw(5) << m[row][col];
        cout << endl;
    }
}
```

Listing F.5 (twodimarray.cpp) experiments with 2D arrays and takes advantage of the **typedef** statement to simplify the code.

Listing F.5: twodimarray.cpp

```
#include <iostream>
#include <iomanip>

using namespace std;

const int ROWS = 3,
        COLUMNS = 5;

// The name Matrix represents a new type
// that means a ROWS x COLUMNS
```

```

// two-dimensional array of double-precision
// floating-point numbers.
typedef double Matrix[ROWS][COLUMNS];

// Allow the user to enter the elements of a matrix
void populate_matrix(Matrix m) {
    cout << "Enter the " << ROWS << " rows of the matrix. " << endl;
    for (int row = 0; row < ROWS; row++) {
        cout << "Row #" << row << " (enter " << COLUMNS << " values):";
        for (int col = 0; col < COLUMNS; col++)
            cin >> m[row][col];
    }
}

void print_matrix(const Matrix m) {
    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLUMNS; col++)
            cout << setw(5) << m[row][col];
        cout << endl;
    }
}

int main() {
    // Declare the 2D array
    Matrix mat;
    // Populate the array
    populate_matrix(mat);
    // Print the array
    print_matrix(mat);
}

```

An expression that uses just one index with a 2D array represents a single row within the 2D array. This row is itself a 1D array. Thus, if **a** is a 2D array and **i** is an integer, then the expression

a[i]

is a 1D array representing row **i**.

C++ allows arrays with dimensions higher than two. Each “slice” of a 3D array is simply a 2D array, a 4D array is an array of 3D arrays, etc. For example, the statement

matrix[x][y][z][t] = 1.0034;

assigns 1.0034 to an element in a 4D array of **doubles**. In practice, arrays with more than two dimensions are rare, but advanced scientific and engineering applications sometimes require higher-dimensional arrays.

F.5 Pointers and Arrays

An array name used in C++ source code references a location in memory, the address of the first element (element at index 0) in the array. In this way an array name is similar to a constant pointer (see Section 10.6 for more information about C++ pointers). Because of this, an array identifier may be treated in some ways like a pointer. Similarly and more commonly, a pointer can be directed to point to an array, and then the pointer itself can be treated as if it were an array.

Listing F.6 (pointerprint.cpp) uses a pointer to traverse an array.

Listing F.6: pointerprint.cpp

```
#include <iostream>

using namespace std;

int main() {
    int a[] = { 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 },
        *p;

    p = &a[0]; // p points to first element of array a

    // Print out the contents of the array
    for (int i = 0; i < 10; i++) {
        cout << *p << ' '; // Print the element p points to
        p++; // Increment p so it points to the next element
    }
    cout << endl;
}
```

The statement

```
p = &a[0];
```

sets **p** to point to the first element of array **a**. A shorter way to accomplish the same thing is

```
p = a;
```

since **a** is itself a reference to the array's location in memory. This assignment statement clearly demonstrates the association between array variables and pointer variables. Note that the opposite assignment (**a = p**) is impossible, because array **a** declared as above may not appear by itself on the left side of the assignment operator.

Pointer variables can participate in addition and subtraction expressions. The statement

```
p++;
```

changes the address stored in **p** so subsequently the pointer will point to the next integer position in memory. If **p** is assigned to array **a**, incrementing **p** redirects it to point to **a**'s next element.¹ In Figure F.5, pointer **p** is assigned to array **a** and then incremented to refer to various elements within **a**.

The expression

```
p[0]
```

is another way to write

```
*p
```

¹ Each byte in memory has a unique numeric address. Since most C++ types require more than one byte of storage, incrementing a pointer by 1 does not simply add 1 to the address it holds. The amount added depends on the type of the pointer; for example, on systems using 32-bit (4 byte) integers, adding 1 to an integer pointer variable increments its address by 4, not 1. The compiler knows the type of the pointer variable because programmers must declare all variables. The compiler, therefore, can automatically adjust the arithmetic to work properly. If a pointer points to memory within an array and is of the same type as the array, incrementing the pointer correctly repositions the pointer to point to the next element in the array.

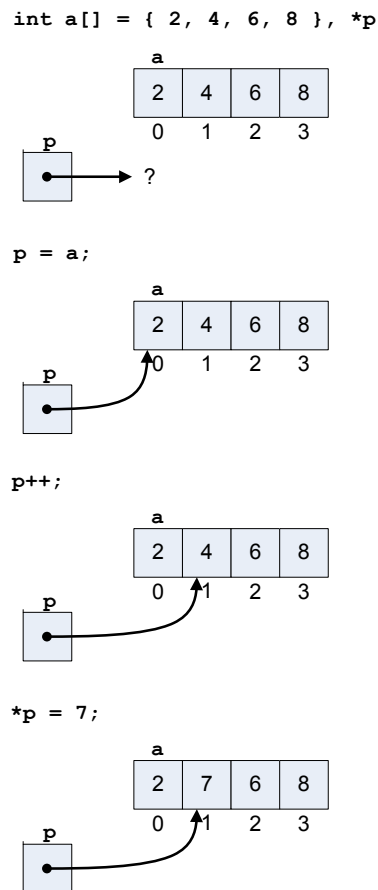


Figure F.5: Incrementing a pointer to an array

so the array indexing operator (`[]`) can be used with pointers. The expression

`p[5]`

represents the element at index 5 within the array pointed to by `p`.

Listing F.7 (pointerprint2.cpp) uses pointers in a different way to traverse an array.

Listing F.7: pointerprint2.cpp

```
#include <iostream>

using namespace std;

int main() {
    int a[] = { 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 },
        *begin, *end, *cursor;

    begin = a;          // begin points to the first element of array a
```



```

    end = a + 10;    // end points to just after the last element

    // Print out the contents of the array
    cursor = begin;
    while (cursor != end) {
        cout << *cursor << ' '; // Print the element
        cursor++; // Increment cursor so it points to the next element
    }
    cout << endl;
}

```

In Listing F.7 (pointerprint2.cpp), the code

```

cursor = begin;
while (cursor != end) {
    cout << *cursor << ' '; // Print the element
    cursor++; // Increment cursor so it points to the next element
}

```

can be expressed more succinctly as a **for** loop:

```

for (cursor = begin; cursor != end; cursor++)
    cout << *cursor << ' '; // Print the element

```

If pointer **p** points to an array, the element at index **i** can be accessed as either **p[i]** or ***(p + i)**. The expression ***(p + i)** dereferences the address that is **i** positions away from the address referenced by **p**.

Sometimes pointer notation is used to represent an array parameter to a function. The array print function that begins

```
void print(const int a[], int n)
```

could instead be written

```
void print(const int *a, int n)
```

where **a** is a pointer to an array. The compiler treats the two forms identically in the machine language it produces.

Listing F.8 (recursivearrayprint.cpp) uses two array printing functions, **iterative_print** and **recursive_print**. Both use the pointer notation when declaring their array parameters.

Listing F.8: recursivearrayprint.cpp

```

#include <iostream>

using namespace std;

/*
 *  iterative_print(a,n)
 *      Prints the contents of array a
 *      a is the array to print; a is not modified
 *      n is number of elements in a
 */
void iterative_print(const int *a, int n) {

```

```

        for (int i = 0; i < n; i++)
            cout << a[i] << ' ';
    }

    /*
     * recursive_print(a,n)
     *     Prints the contents of array a
     *     a is the array to print; a is not modified
     *     n is number of elements in a
     */
    void recursive_print(const int *a, int n) {
        if (n > 0) {
            cout << *a << ' '; // Print the first element of the array
            recursive_print(a + 1, n - 1); // Print rest of the array
        }
    }

    int main() {
        int list[] = { 23, -3, 4, 215, 0, -3, 2, 23, 100, 88, -10 };
        iterative_print(list, 11);
        recursive_print(list, 11);
    }

```

The function `iterative_print` uses a loop, while `recursive_print` uses recursion instead. Inside the `recursive_print` function, the expression `a + 1` points to the second element in array `a`. `a + 1` essentially represents the rest of the array—everything except for the first element. The statement

```
recursive_print(a + 1, n - 1);
```

calls the function recursively, passing the remainder of the array (`a + 1`) with a length that is one less (`n - 1`) than before. The recursion terminates when the array's length is zero.

F.6 Array Ranges

In Section F.2 we saw one way that we can pass an array to a function. We noted that since an array parameter is just the address of the array's first element, callers must include additional information so that the function knows the size of the array. Another approach passes the address of the array's first element and the address of the imaginary element that is just past the end of the array. This is known as the *range-based* approach to passing array parameters.

Consider the array `print` function in Listing F.2 (`arraytofunc.cpp`) in Section F.2:

```

void print(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}

```

This `print` function requires the caller to provide the array and the array's size, as in

```

int my_list[] = { 20, 30, 40, 50 };
print(my_list, 4);

```

A range-based version looks like

```
void print(int *begin, int *end) {
    for (int *cursor = begin; cursor != end; cursor++)
        cout << *cursor << " ";
    cout << endl;
}
```

In the range-based approach the caller passes the array (that is, the address of the element at index zero) and the address of the imaginary element *just past the end* of the array:

```
int my_list[] = { 20, 30, 40, 50 };
print(my_list, my_list + 4);
```

Here we see the caller passes two pointers to this range-based **print** function:

- **my_list** points to the first element (at index 0) of the array, and
- the expression **my_list + 4**, because of pointer arithmetic (see Section F.5), points to the address just past the element at the back of the array.

The local variable in the range-based **print** function, **cursor**, is a pointer that the loop directs to each element in the array. When **cursor** passes the last element in the array, the iterator ceases. This is because the loop can continue only while the expression **cursor != end** is true.

The C++ standard library generalizes this range-based approach to algorithms that process more sophisticated kinds of data structures.

F.7 C Strings

A string is a sequence of characters. C and C++ implement strings as arrays of **char**. The C++ language additionally supports string objects (see Section 13.1). In the C language, the only option is a **char** array. We use the term *C string* to refer to an array of characters as used in the C language. In this section, any mention of the term *string* refers to a C string.

A string is an array of characters. A string literal is a sequence of characters enclosed within quotation marks, as in

```
cout << "Howdy!" << endl;
```

All proper C strings are *null terminated*. This means the last character in the array is ASCII zero, which in C++ is represented by the character literal **'\0'**. Figure F.6 shows the physical layout of the string **"Howdy!"** in memory.

Since strings are actually arrays, care must be taken when using string variables:

- Enough space must be reserved for number of characters in the string, including the null terminating character.
- The array of characters must be properly null terminated.

The following code fragment is safe and acceptable:



Figure F.6: Physical layout of a C string

```
char *word = "Howdy!";
cout << word << endl;
```

The variable **word** is declared to be a pointer to a character, and it is initialized to point to a string literal.

The following code fragment is less safe:

```
char word[256];
cin >> word;
```

word can hold 255 viable characters plus the null terminator. If the user types in relatively short words (length less than 255 characters), there is no problem. If at any time the user types in more characters than will fit in the **word** array, the executing program will have a problem. The problem is known as a *buffer overrun*. In the best case, buffer overruns lead to buggy programs. In the worst case, clever users can exploit buffer overruns to compromise software systems. Buffer overruns are always logic errors and you should take great care to avoid them.

The following code provides a safe way to get user input:

```
char word[10];
fgets(word, 10, stdin);
cout << word << endl;
```

The **fgets** function is a standard C function. The second parameter specifies the maximum length of the string, including the terminating null character, that will be placed in the string **word**. The last argument, **stdin** is a C construct related to the C++ object **cin**. In order to use **fgets** within a program you must include the **<cstdio>** header.

The following code begs for disaster:

```
char *word;
cin >> word;
```

In this case **word** points to a random location in memory, and no buffer has been allocated to receive the input characters from **cin**. The program's behavior executing this code is undefined, but it likely will lead to the program crashing. Insidiously, depending on how the operating system manages memory, the program may run fine much of the time and crash only rarely. Regardless, the program contains a serious bug.

When passing an array to a function a caller must provide the size of the array so that the function may process the array properly. Since C strings are null terminated, such size information is not necessary. The **find_char** function in Listing F.9 (**findchar.cpp**) determines if a particular character is present in a string:

Listing F.9: **findchar.cpp**

```
#include <iostream>
```

```
using namespace std;

bool find_char(const char *s, char ch) {
    while (*s != '\0') { // Scan until we see the null character
        if (*s == ch)
            return true; // Found the matching character
        s++; // Advance to the next position within the string
    }
    return false; // Not found
}

int main() {
    const char *phrase = "this is a phrase";
    // Try all the characters a through z
    for (char ch = 'a'; ch <= 'z'; ch++) {
        cout << '\'' << ch << '\'' << " is ";
        if (!find_char(phrase, ch))
            cout << "NOT ";
        cout << "in " << '\'' << phrase << '\'' << endl;
    }
}
```

The output of Listing F.9 (findchar.cpp) is

```
'a' is in "this is a phrase"
'b' is NOT in "this is a phrase"
'c' is NOT in "this is a phrase"
'd' is NOT in "this is a phrase"
'e' is in "this is a phrase"
'f' is NOT in "this is a phrase"
'g' is NOT in "this is a phrase"
'h' is in "this is a phrase"
'i' is in "this is a phrase"
'j' is NOT in "this is a phrase"
'k' is NOT in "this is a phrase"
'l' is NOT in "this is a phrase"
'm' is NOT in "this is a phrase"
'n' is NOT in "this is a phrase"
'o' is NOT in "this is a phrase"
'p' is in "this is a phrase"
'q' is NOT in "this is a phrase"
'r' is in "this is a phrase"
's' is in "this is a phrase"
't' is in "this is a phrase"
'u' is NOT in "this is a phrase"
'v' is NOT in "this is a phrase"
'w' is NOT in "this is a phrase"
'x' is NOT in "this is a phrase"
'y' is NOT in "this is a phrase"
'z' is NOT in "this is a phrase"
```

The **find_char** function in Listing F.9 (findchar.cpp) uses pointer notation to traverse the string. It does not need to know in advance the number of characters in the string because it starts at the beginning and keeps scanning each character in turn until it finds the character it is looking for or encounters the null terminating character.

Recall from Section 5.1 that for Boolean conditions C++ treats a zero value as false and any non-zero value as true. Because of such a loose interpretation of Boolean expressions, the **find_char** function above may be written more compactly as

```
char find_char(const char *s, char ch) {
    // Scan until we see the null character or the character
    // we seek
    while (*s != '\0' && *s != ch)
        s++; // Advance to the next position within the string
    return *s; // Null character = false, any other is true
}
```

The only way out of the loop is to scan the null terminating character or the character sought. Here, if the loop encounters the null terminating character, it exits and returns that null character. The null character is simply ASCII zero—Boolean false. If the loop locates the sought character, it exits and returns that character which will not be ASCII zero. Any character except the null character has an ASCII value greater than zero; therefore, Boolean true.

Most routines that process strings depend on the strings to be null terminated in order to work properly. Some standard C string functions include

- `int strlen(const char *s)` returns the number of characters in string `s`, not including the null terminator.
- `char *strcpy(char *s, const char *t)` copies the contents of string `t` into string `s` up to and including the null terminator; `s` must point to a buffer large enough to hold all the characters of C string `t`.
- `char *strncpy(char *s, const char *t, unsigned n)` works like `strcpy` but copies a maximum of `n` characters; `s` must point to a buffer that can hold at least `n` characters.
- `int strcmp(const char *s, const char *t)` compares two strings for lexicographic (dictionary) ordering. The function returns an integer less than zero if `s` appears lexicographically before `t`. The function returns an integer greater than zero if `s` appears lexicographically after `t`. The function returns zero if the two strings are identical.

The following code fragment

```
cout << strcmp("ABC", "XYZ") << endl;
cout << strcmp("XYZ", "ABC") << endl;
cout << strcmp("ABC", "ABC") << endl;
```

prints

```
-1
1
0
```

- `int strncmp(const char *s, const char *t, int n)` compares the first `n` characters of two strings for lexicographic (dictionary) ordering. The function returns an integer less than zero if the first `n` characters of `s` appear lexicographically before the first `n` characters of `t`; that is, `s` would appear before `t` in a dictionary. The function returns an integer greater than zero if the first `n` characters of `s` appear lexicographically after the first `n` characters of `t`. The function returns zero if the first `n` characters of the two strings are identical.

You should be familiar with C strings because C++ code can use C libraries, and C libraries often use C strings. Since C strings tend to be problematic, however, in most cases you should use the newer `string` objects (see Section 13.1) whenever possible.

F.8 Dynamic Arrays

In all of the programs we have examined so far in this chapter the size of the arrays were determined at compile time. The programmer may change the size of the array in the source code, but the program then must be recompiled. We would like a program that truly is flexible and is built to handle any number of entries the user may need to process.

One approach to the flexibly-sized array is to define the array to hold as many items as conceivably will be needed. Listing F.10 (`largearrayaverage.cpp`) uses this approach.

Listing F.10: `largearrayaverage.cpp`

```

#include <iostream>

using namespace std;

// Maximum number of expected values is 1,000,000
const int MAX_NUMBER_OF_ENTRIES = 1000000;
double numbers[MAX_NUMBER_OF_ENTRIES];

int main() {
    double sum = 0.0;
    int size; // Actual number of entries

    // Get effective size of the array
    cout << "Please enter number of values to process: ";
    cin >> size;

    if (size > 0) { // Nothing to do with no entries
        cout << "Please enter " << size << " numbers: ";
        // Allow the user to enter in the values.
        for (int i = 0; i < size; i++) {
            cin >> numbers[i];
            sum += numbers[i];
        }
        cout << "The average of ";
        for (int i = 0; i < size - 1; i++)
            cout << numbers[i] << ", ";
        // No comma following last element
        cout << numbers[size - 1] << " is "
            << sum/size << endl;
    }
}

```

Listing F.10 (`largearrayaverage.cpp`) creates an array that can hold one million entries. The variable **size** keeps track of the actual size needed by the user. While the array's maximum size is one million, the part of the array actually used during a given execution may be much smaller.

Notice that the array **numbers** is a global variable and is not local to **main**. This is because most systems limit the amount of storage available to local variables within functions. Local variables are stored in an area of memory known as the *stack*. Local variables exist in memory only when the function that uses them is invoked; therefore, the stack grows and shrinks as functions execute and return. Global variables, on the other hand, exist in the computer's memory for the life of the program's execution. Global variables are stored in what is known as *static memory*. There is a limit to the amount of static memory available for global variables (the amount of memory in RAM or virtual memory on disk, if nothing else), but the global limit usually is much higher than the local limit. One million double-precision floating-point numbers consumes 8,000,000 bytes (8 megabytes) in Visual C++ and on many other systems. The default stack size for local variables under Visual C++ is only one megabyte, although the stack size can be increased by adjusting the compiler and linker build options.

While the approach taken in Listing F.10 (`largearrayaverage.cpp`) works, it wastes memory resources. In modern computing, a user may have multiple programs open at the same time. If each program is tying up the maximum amount of memory it may ever need, there may not be enough real memory (RAM) to go around, and the computer will be forced to use more virtual memory (shuttling portions of the running

program's memory to and from the disk drive). Virtual memory access greatly degrades the speed of a program, and so the user's experience suffers. It is important that each program uses its resources wisely.

Fortunately, there is a way for a programmer to create an array of the exact size needed, even if that size is not known until run time. Listing F.11 (flexiblearrayaverage.cpp) shows how.

Listing F.11: flexiblearrayaverage.cpp

```
#include <iostream>

using namespace std;

int main() {
    double sum = 0.0;
    double *numbers; // Note: numbers is a pointer, not an array
    int size; // Actual number of entries

    // Get effective size of the array
    cout << "Please enter number of values to process: ";
    cin >> size;

    if (size > 0) { // Nothing to do with no entries
        cout << "Please enter " << size << " numbers: ";
        // Allocate the exact size needed
        numbers = new double[size]; // Dynamically allocated array
        // Allow the user to enter in the values.
        for (int i = 0; i < size; i++) {
            cin >> numbers[i];
            sum += numbers[i];
        }
        cout << "The average of ";
        for (int i = 0; i < size - 1; i++)
            cout << numbers[i] << ", ";
        // No comma following last element
        cout << numbers[size - 1] << " is "
            << sum/size << endl;
        delete [] numbers; // Free up the space held by numbers
    }
}
```

Notice that **numbers** is not declared to be an array; it is a pointer:

```
double *numbers; // Note: numbers is a pointer, not an array
```

This statement makes space for a single pointer, merely four bytes on a 32-bit system or eight bytes on a 64-bit system. The expression

```
new double[size]
```

allocates at run time a block of memory for an array that can hold exactly **size** double-precision floating-point elements. The value of this expression is the starting address of the newly allocated memory block.

The statement

```
numbers = new double[size]; // Dynamically allocated array
```

therefore assigns to **numbers** the address of the allocated memory.

The reserved word **new** allocates memory for the array's elements. **new** technically is classified as an operator. The area of memory used for dynamic allocation via **new** is different from both the area used for local variables (the stack) and global variables (static memory). Dynamic memory comes from the *heap*. Section F.11 provides more details about the memory available to an executing C++ program.

The variable **numbers** is local to **main**, so it lives on the stack. It is a pointer, so it stores a memory address. The **new** expression returns an address to memory in the heap. So, even though the **numbers** variable itself is stored on the stack, the memory it references is located in the heap. We do not need to worry about our array being too big to be local to **main**, since its contents do not consume stack space.

Local variables are automatically allocated on the stack when a function is invoked and automatically deallocated when the function returns. The programmer does not need to explicitly manage local variables. Dynamically allocated memory, however, requires more attention on the part of the programmer. The statement

```
delete [] numbers; // Free up the space held by numbers
```

uses the **delete** operator to free up the memory held by the **numbers** array. The programmer is responsible for deallocating memory that was allocated with **new**. Every use of the **new** operator should have a corresponding use of **delete** somewhere later in the program's execution. Notice that the square brackets ([]) are empty. The run-time environment keeps track of the amount of memory to free up.

delete is a C++ reserved word.

You should develop the habit of ensuring that every use of the **new** operator has an associated call of **delete**. If the **delete** statement is omitted from Listing F.11 (`flexiblearrayaverage.cpp`), the program in this case likely will behave no differently since most operating systems reclaim all the memory held a programs when the program finishes executing. Serious problems arise in longer running programs that must allocate and deallocate dynamic memory over time. Programs that run for extended periods of time, like web servers and operating system kernels, can crash due to failures to properly deallocate memory. The condition is known a *memory leak*—the program overtime allocates more and more memory via **new** but never releases the memory back with **delete**. Eventually the program uses up all of the available heap space and crashes.

Listing F.11 (`flexiblearrayaverage.cpp`) requires the user to enter up front the number of values to average. This is inconvenient, and people are notoriously poor counters. One solution is to allocate a minimal size array, and then resize it as necessary when it fills up. Listing F.12 (`resizearray.cpp`) uses this approach.

Listing F.12: `resizearray.cpp`

```
#include <iostream>

using namespace std;

int main() {
    double sum = 0.0, // Sum of the elements in the list
           *numbers, // Dynamic array of numbers
           input;     // Current user entry

    // Initial size of array and amount to expand full array
    const int CHUNK = 3;
    int size = 0, // Current number of entries
        capacity = CHUNK; // Initial size of array
```

```

// Allocate a modest-sized array to begin with
numbers = new double[capacity];

cout << "Please enter any number of non-negative values "
      << "(negative value ends the list): ";
cin >> input;

while (input >= 0) { // Continue until negative number entered
    if (size >= capacity) { // Room left to add an element?
        capacity += CHUNK; // Expand array
        double *temp = new double[capacity]; // Allocate space
        for (int i = 0; i < size; i++)
            temp[i] = numbers[i]; // Copy existing values
        delete [] numbers; // Free up old space
        numbers = temp; // Update numbers to new location
        cout << "Expanding by " << CHUNK << endl;
    }
    numbers[size] = input; // Add number to array at last position
    size++; // Update last position
    sum += input; // Add to running sum
    cin >> input; // Get next number
}
if (size > 0) { // Can't average less than one number
    cout << "The average of ";
    for (int i = 0; i < size - 1; i++)
        cout << numbers[i] << ", ";
    // No comma following last element
    cout << numbers[size - 1] << " is "
         << sum/size << endl;
}
else
    cout << "No numbers to average" << endl;
delete [] numbers; // Free up the space held by numbers
}

```

F.9 The sizeof Operator

The size of a static array is fixed at compile time. The C++ `sizeof` operator computes the amount of memory in bytes that a static array consumes. The code

```

int list[100],
    set[] = { 1, 2, 3, 4, 5 };
cout << "list: " << (sizeof list) << ", set: " << (sizeof set) << endl;

```

prints

```
list: 400, set: 20
```

on a system in which integers use four bytes of storage. This is because `list` holds 100 integers at four bytes each, and `set` holds five integers at four bytes each. The `sizeof` operator computes the storage required for any type; for example,

```
cout << "int: " << (sizeof int) << ", double: " << (sizeof double) << endl;
```

prints

```
int: 4, double: 8
```

on systems that store integers in four bytes and double-precision floating-point numbers in eight bytes.

The number of elements in a statically allocated array can be obtained from a simple formula: divide the memory occupied by the array by the number of bytes required for each element. The following code fragment

```
int list[100];
double collection[] = { 2.8, 3.33, 0.4, 0.0, 10.4 };
cout << "list: " << (sizeof list)/(sizeof list[0])
    << ", collection: " << (sizeof collection)/(sizeof collection[0]) << endl;
```

prints

```
list: 100, collection: 5
```

on all systems regardless of how much memory integers and floating-point numbers consume.

The code in the `main` function of Listing F.2 (`arraytofunc.cpp`)

```
int list[] = { 2, 4, 6, 8 };
// Print the contents of the array
print(list, 4);
```

therefore could be written

```
int list[] = { 2, 4, 6, 8 };
// Print the contents of the array
print(list, (sizeof list)/(sizeof list[0]));
```

to eliminate the literal value 4 as the second parameter. The programmer then may add extra elements to, or remove elements from, the initialization list without changing the call to `print`.

The `sizeof` operator can be used only to compute sizes known at compile time and is not helpful for dynamically allocated arrays. The code

```
int *list;
list = new int[100];
cout << "list: " << (sizeof list)/(sizeof list[0]) << endl;
```

on most systems prints

```
list: 1
```

because pointers and integers often require the same amount of memory. Since `list`'s declared type is `int *`, the `sizeof` operator returns the size of a pointer, not the number of elements in the dynamically-allocated array `list` points to.

Because of the way arrays are passed to functions in C++, we must always pass the size of the array along with the array itself. The function declaration

```
void print(int a[]) {
    cout << (sizeof a) << endl; // Prints 4
    // Other details omitted . . .
}
```

is equivalent to

```
void print(int a[100]) {
    cout << (sizeof a) << endl; // Prints 4
    // Other details omitted . . .
    // Details omitted . . .
}
```

because the compiler ignores any numbers within the square brackets of an array parameter's declaration. Arrays are always passed as pointers to functions. That is why a declaration of the form

```
void print(int *a) {
    cout << (sizeof a) << endl; // Prints 4
    // Other details omitted . . .
    // Details omitted . . .
}
```

is so common—it most accurately reflects how arrays are passed to functions.

F.10 Copying an Array

It is important to remember that within C++ source code an array variable behaves similarly to a constant pointer. At first it may seem plausible to make a copy of an array as follows:

```
int a[10], b[10]; // Declare two arrays
for (int i = 0; i < 10; i++) // Populate one of them
    a[i] = i; // a is filled with increasing values
b = a; // Make a copy of array a?
```

Since `b` behaves like a constant pointer, it cannot be reassigned; that is, the name `b` cannot appear on the left side of the assignment operator all by itself. Wherever `b` points, it must continue to point there during its lifetime.

Perhaps the solution is to use a dynamic array (see Section F.8 for information about dynamic arrays). If `b` is not `const`, can the array be copied through a simple assignment? The following code is legal:

```

int a[10], *b;    // Declare two arrays, one dynamic
for (int i = 0; i < 10; i++) // Populate one of them
    a[i] = i;     // a is filled with increasing values
b = a;           // Make a copy of array a?

```

but **b** is *not* a copy of **a**. **b** aliases **a**, so changing the contents of **a** will also change the contents of **b** identically, since **b** refers to the exact memory to which **a** refers. Figure F.7 illustrates the aliasing.

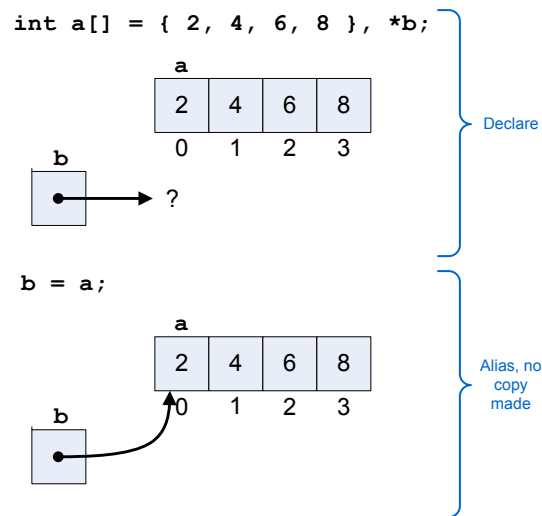


Figure F.7: Faulty array copy

The following code shows the proper way to make a copy of **a**:

```

int a[10], *b;    // Declare two arrays, one dynamic
for (int i = 0; i < 10; i++) // Populate one of them
    a[i] = i;     // a is filled with increasing values
// Really make a copy of array a
b = new int[10];  // Allocate b
for (int i = 0; i < 10; i++)
    b[i] = a[i];

```

Separate space for the dynamic array must be allocated, and then each element from the original array must be copied into the new array. Figure F.8 shows how this process works.

It is important to note that since the code above allocates **b** dynamically with **new**, code elsewhere within the program should eventually use **delete** to free up **b**'s space when it is no longer used. Failure to properly deallocate **b** constitutes a memory leak.

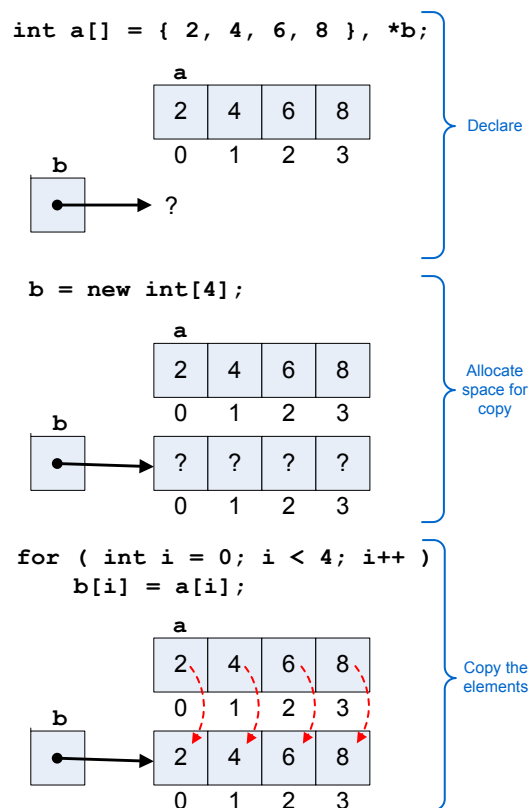


Figure F.8: Correct array copy

F.11 Memory Management

Memory management issues with `new` and `delete` frequently are the source of difficult to find and fix logic errors. Programmers must adhere to the following tenets:

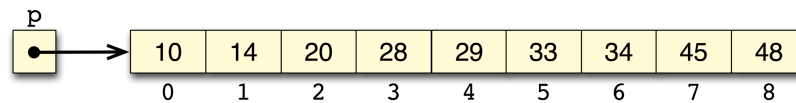
- Every call to `new` should have an associated call to `delete` when the allocated memory is no longer needed. It sounds simple enough, but it is not always clear when `delete` should be used. The following function exhibits a memory leak:

```
void calc(int n) {
    // ...
    // Do some stuff
    // ...
    int *p = new int[n];
    // ...
    // Do some stuff with p
    // ...
    // Exit function without deleting p's memory
}
```

If a program calls function `calc` enough times, the program will eventually run out of memory. Figure F.9 shows how memory is stranded when local pointers go out of scope at the end of a function's execution. In the `calc` function `p` is a local variable. As such, `p` lives on the stack (see Section F.8).

Before call to `calc`, `p` does not exist

During execution of `calc`, `p` is alive and points to dynamically allocated memory



After the call to `calc`, `p` goes away, but the memory allocated on its behalf is not deallocated

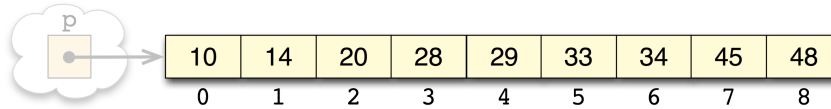


Figure F.9: Memory leak

When a particular call to `calc` completes, the function's clean up code automatically releases the space help by *the variable* `p`. This is because all functions automatically manage the memory for their parameters and local variables. The problem is `p` is assigned via `new` to point to memory allocated from the heap, not the stack. Function executions manage the stack memory only. When the function's execution completes, `p` is gone, and the memory to which `p` pointed is not deallocated automatically. Worse yet, the heap memory formerly referenced by the local variable `p` now is unreachable because no other variables that remain viable know anything about it.

- The `delete` operator never should be used to free up memory that was not allocated by a previous call to `new`. This code fragment illustrates one such example:

```
int list[10], *p = list; // p points to list
// ...
// Do some stuff
// ...
delete [] p; // Logic error, attempt to deallocate p's memory
```

The space referenced by pointer `p` was not allocated by `new`, so `delete` should not be used to attempt to free up its memory.

- `delete` must not be used to deallocate the same memory more than once. This can happen when two pointers refer to the same memory. Such pointers are called *aliases*. The following code fragment illustrates the situation:

```
int *p = new int[10], *q = p; // q aliases p
// ...
// Do some stuff with p and/or q
// ...
```



```

delete [] p;    // Free up p's memory
// ...
// Do some other stuff
// ...
delete [] q;    // Logic error, q's memory already freed!

```

Since pointer `p` and pointer `q` point to the same memory, they are aliases of each other. Deallocate the memory referenced by one of them, and the other's memory is also deallocated, since it is the same memory.

- **Memory previously deallocated via `delete` should never be accessed.** It is a logic error to access memory previously deallocated by `delete`. The code fragment

```

int *list = new int[10];
// ...
// Use list, then
// ...
delete [] list; // Deallocate list's memory
// ...
// Sometime later
// ...
int x = list[2]; // Logic error, but sometimes works!

```

illustrates how such a situation can arise. For efficiency reasons the `delete` operator generally marks heap space as “available” without modifying the contents of that memory. Careless programmers can accidentally use that memory obviously as if it were still live. The problem manifests itself when the freed up memory eventually gets reallocated elsewhere via a call to `new`. The result is that programs seem to “work” for a while and mysteriously fail at unpredictable times. Debugging such situations can be very difficult. The problem often happens because of aliasing:

```

int *list = new int[10];
// ...
int *arr = list; // arr aliases list
// ...
delete [] list; // Deallocate list's memory
// ...
// Sometime later
// ...
int x = arr[2]; // Same problem!

```

We `deleted` `list`'s memory, not `arr`'s memory, didn't we? No, since `arr` is an alias for `list`, `arr` references memory previously deallocated with `delete`.

The space for non-`static` local variables is automatically allocated when a function begins executing, and that space is automatically deallocated when the function returns. A programmer need not explicitly use something like `new` and `delete` to manage memory for locals. The space for function parameters is handled automatically also.

The space for global variables and `static` local variables is automatically allocated before the program begins executing its `main` function.

Figure F.10 illustrates the typical organization of the memory of an executing C++ program. Memory for an executing program is divided into four sections:

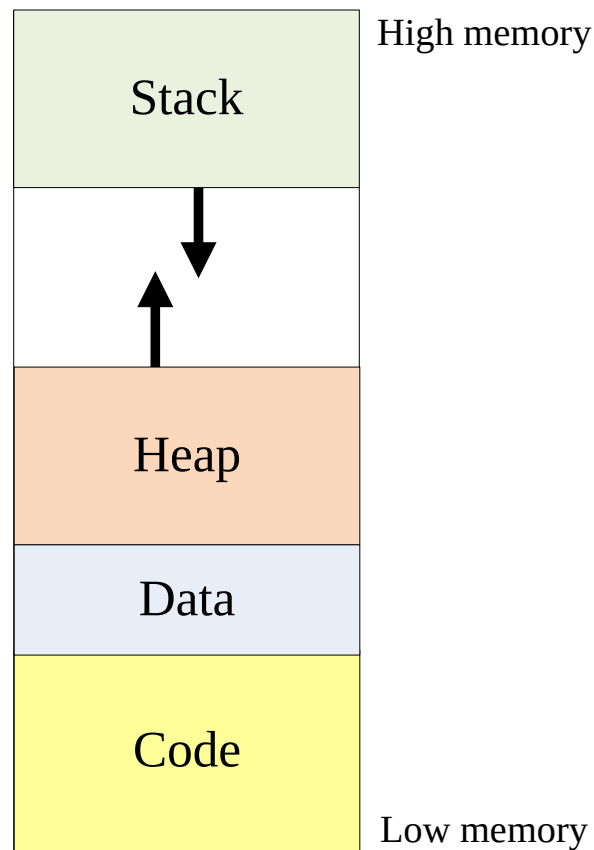


Figure F.10: Typical memory organization in an executing program

- **Code.** The code section of memory holds the program's compiled executable instructions. The code section should never change while the program executes, and some operating systems enforce this restriction. The size of the code segment does not change while the program executes.
- **Data.** The data section of memory contains global variables (see Section 10.1) and persistent local variables (**static** locals, see Section 10.2). The variables in the data section exist for the life of the executing program, but, unless they are constants (see Section 3.5), the executing program may freely change their values. The size of the data segment does not change while the program executes.
- **Heap.** The heap is where an executing program obtains dynamic memory. The **new** operator gets its memory from the heap, and **delete** returns previously allocated memory back to the heap. The size of the heap grows and shrinks during the program's execution as the program allocates and deallocates dynamic memory using **new** and **delete**.
- **Stack.** The stack is where local variables and function parameters live. Space for local variables and parameters appears when a function is called and disappears when the function returns. The size of the stack grows and shrinks during the program's execution as various functions execute.

Notice that the stack and heap segments grow toward each other. Deep recursion can consume a considerable amount of stack space. An improperly written recursive function, for example, one that omits the base case and thus exhibits “infinite” recursion, will cause the stack to collide with the heap. Modern operating systems will terminate a process that consumes all of its stack space, but on some embedded systems this stack overflow may go undetected. Similarly, a program that allocates a lot of dynamic memory, especially a program with memory leaks, may cause the heap to collide with the stack. In either case the executing program does not have enough memory available to satisfy its needs, so the operating system will terminate the program’s execution.

F.12 Summary

- An array represents a collection of homogeneous data values.
- An array variable references the starting address of a chunk of contiguous memory that holds the array’s elements.
- The type of values contained by an array and the number values the array can hold are specified when the array is declared; for example,

```
double list[20];
```

- An element of an array is accessed via a non-negative integer index using the `[]` operator; for example,

```
v[3] = 4.1;
```

The index of the desired element is placed within the square brackets.

- The first element in an array is at index zero. If the array contains n elements, the last element is at index $n - 1$.
- For non-dynamic arrays, an array’s size must be specified when it is declared; for example,

```
int numbers[10]; // numbers holds 10 integers
```

- An array’s contents may be initialized when the array is declared, as in

```
int arr[] = { 2, 4, 6, 8 };
```

- Any expression compatible with an integer can be placed within the `[]` operator. Literals, variables, and expressions can be used. The compiler will issue an error if a floating-point expression is used within the square brackets.
- A **for** loop is a convenient way to traverse the contents of an array.
- Like other variables, an array variable can be local, **static** local, or global.
- An array can be passed to a function. The actual parameter passed to the function is a pointer to the first element of the array. Ordinarily, the caller must pass an additional parameter specifying the number of elements within the array, since an array does not keep track of its own size.
- If a function is not supposed to modify the contents of an array, best programming practices insist the array parameter should be declared **const**.

- It is the programmer's responsibility to stay within the bounds of an array. Venturing outside the bounds of an array results in undefined behavior that introduces a logic error and often leads to a program crash.
- One array may not be assigned to another through simple assignment; a loop must be used to copy each element individually.
- Two-dimensional arrays conceptually store their elements in rows and columns. C++ supports arrays of arbitrary dimensions.
- In a 2D array expression like `a[i][j]` the first index (here `i`) represents a row, and the second index (here `j`) specifies a column.
- The space for an array's elements may be allocated dynamically at run time using `new`.
- During a program's execution, the elements of statically allocated local arrays live on the stack, the elements of statically allocated global arrays live in static memory, and the elements of dynamically allocated arrays (local or global) live on the heap.
- The use of the `new` operator allocates resources that can be reclaimed only through the use of `delete`. Failure to use a matching `delete` with each use of `new` results in a memory leak.

F.13 Exercises

1. Can you declare an array to hold a mixture of `int`s and `doubles`?
2. What happens if you attempt to access an element of an array using a negative index?
3. Given the declaration

```
int list[100];
```

- (a) What expression represents the very first element of `list`?
 - (b) What expression represents the very last element of `list`?
 - (c) Write the code fragment that prints out the contents of `list`.
 - (d) Is the expression `list[3.0]` legal or illegal?
4. Is the following code fragment legal or illegal?

```
int list1[5], list2[5] = { 3, 3, 3, 3, 3 };
list1 = list2;
```
 5. Provide a single declaration statement that declares an integer array named `list` that contains the values 45, -3, 16 and 8?
 6. Does an array keep track of the number of elements it contains?
 7. Complete the following function that adds up all the *positive* values in an array of integers. For example, if array `arr` contains the elements 3, -3, 5, 2, -1, and 2, the call `sum_positive(arr)` would evaluate to 12, since $3 + 5 + 2 + 2 = 12$. The function returns zero if the array is empty (that is, `n < 1`).

```

// Array a with length n
int sum_positive(const int *a, int n) {
    // Add your code...
}

```

8. Complete the following function that counts the even numbers in an array of integers. For example, if array **arr** contains the elements 3, 5, 2, -1, and 2, the call **count_evens(arr)** would evaluate to 4, since $2 + 2 = 4$. The function returns zero if the array is empty (that is, $n \leq 1$). The function does not affect the contents of the array.

```

// Array a with length n
int count_evens(const int *a, int n) {
    // Add your code...
}

```

9. Complete the following function that counts the even numbers in a 2D array of integers.

```

// 2D array v with r rows and c columns
int count_evens(const int **v, int r, int c) {
    // Add your code...
}

```

10. Suppose your task is to implement the function with the prototype

```
void proc(int a[]);
```

When you implement the body of **proc**, how can you determine the size of array **a**?

11. Consider the declaration

```
int collection[100][200];
```

What does the expression **collection[15][29]** represent?

12. Consider the declaration

```
int collection[100][200];
```

How many elements does **collection** hold?

13. Consider the declaration

```
int collection[100][200];
```

Write the C++ code that prints all the elements in **collection**. All the elements in the same row should appear on the same line, and but each successive row should appear on its own line.

14. Consider the declaration

```
int collection[100][200];
```

What does the expression **collection[15]** represent?

15. Consider the declaration

```
int mesh[100][200][100][50];
```

How many elements does **mesh** hold?

16. Rewrite the following expressions using pointer notation instead of array notation.

- (a) `a[4]`
- (b) `a[1]`
- (c) `a[0]`

17. Rewrite the following expressions using array notation instead of pointer notation.

- (a) `*(a + 3)`
- (b) `*a`
- (c) `*(a + 0)`

18. Rewrite the following code fragment using array notation instead of pointer notation:

```
void display(int *a, int n) {  
    while (n) {  
        cout << *a << " ";  
        a++;  
        n--;  
    }  
    cout << endl;  
}
```

19. Rewrite the following code fragment using pointer notation instead of array notation:

```
int sum(int *a, int n) {  
    int s = 0;  
    for (int i = 0; i < n; i++)  
        s += a[i];  
    return s;  
}
```

20. Consider the following declaration:

```
char *word = "abcde";
```

Evaluate each of the following expressions.

- (a) `word[1]`
- (b) `*word`
- (c) `word[5]`

21. Suppose your task is to implement the function with the prototype

```
void proc(char *s);
```

where `s` is a C string. When you implement the body of `proc`, how can you determine the length of string `s`?

22. Given the following declarations which appear in a function body:

```
double nums[100], *grid = new double[100];
```

- (a) Where will the `nums` array live, static memory, the stack, or the heap?

- (b) Where will the **grid** array live, static memory, the stack, or the heap?
23. What operator should eventually be used when the **new** operator is used to allocate memory? What is the consequence of its omission?
24. List some common errors programmers make when dealing with dynamic memory.
25. Complete the following function that counts the number of negative values in a 10×10 integer 2D array.

```
int count_negatives(int a[10][10]) {  
    // Add your code...  
}
```

Index

- abstract class, 472
- actual parameter, 196, 197
- alias, 611
- and, bitwise, 575
- and, logical, 82
- argument, 170
- array, 292
- array initialization list, 583
- array slice, 297
- assignment, 17
- associativity, operator, 44
- auxiliary methods, 384

- bias in pseudorandom number generation, 357
- binary search, 315
- bitwise and, 575
- bitwise exclusive or, 576
- bitwise left shift, 577
- bitwise negation, 576
- bitwise operators, 575
- bitwise or, 576
- bitwise right shift, 577
- bitwise shift operators, 577
- bitwise xor, 576
- block, 77
- bottom-checking loop, 153
- break statement, 127
- buffer overrun, 601

- call by reference, 256
- call by value, 203
- caller, 170
- case label in a switch statement, 150
- catch block, 529
- class, 364
- class derivation, 454
- class specialization, 454
- class templates, 504
- client, 342
- coincidental correctness, 447
- command line, 9
- compiler, 2

- concrete class, 473
- constructor, 377
- constructor initialization list, 379
- current working directory, 562

- data members, 365
- default constructor, 379
- delegation, method, 460
- delimit, 11
- derivation, class, 454
- destructor, 380
- dot operator (.), 367
- dynamic binding, 466

- early binding, 465
- encapsulation, 386
- enumerated type, 28
- enumeration type, 28
- evaluation, 36
- exception, 527
- exception handling, 527
- exclusive or, bitwise, 576
- explicit template instantiation, 497, 502
- expressions, 35

- fields, 365
- floating-point types, 570
- for statement, 156
- foreach, 280
- full-path name, 562
- function call, 170
- function definition, 191
- function invocation, 170
- function overloading, 238
- function pointers, 261

- generic function, 496
- goto statement, 129

- helper methods, 384
- higher-order function, 261
- homogeneous, 273, 581

- if statement, 74
- inheritance, 454
- initialization, 19
- instance, 365
- integer types, 565
- is-a relationship, 455
- iteration, 109
- iterators, 514

- late binding, 466
- left shift, 577
- linear search, 315
- linker, 4
- Liskov Substitution Principle, 455
- logical and, 82
- logical negation, 83
- logical not, 83
- logical or, 83
- loop, 109
- loop unrolling, 328

- matrix, 289, 593
- member function, 375
- member functions, 281
- memory leak, 396, 607
- Mersenne Twister pseudorandom number generator, 359
- method delegation, 460
- methods, 281
- multiple inheritance, 457

- negation, bitwise, 576
- negation, logical, 83
- nested loop, 119
- nibble, 567
- non-type parameters, template, 503
- non-type template parameters, 503
- not, logical, 83

- object slicing, 462
- operator associativity, 44
- operator precedence, 44
- or, logical, 83
- overflow, 568
- overloaded function, 238
- override, 460

- parameter, 170
- parameter, actual, 196, 197
- pass by reference, 256
- pass by value, 203

- period of a pseudorandom number generator, 357
- pointer arithmetic, 296
- polymorphism, 469
- precedence, operator, 44
- pretty printer, 50
- private inheritance, 457
- private members, 374
- profiler, 5
- protected member, 471
- public members, 365
- pure virtual methods, 472

- range-based for, 280
- range-based array parameter passing, 599
- ranges, 518
- rational number, 381
- re-throw, 541
- re-throwing an exception, 542
- real number, 570
- reference variable, 254
- relational operators, 72
- return statement, 195
- return value optimization, 288
- right shift, 577

- shift, bitwise, 577
- short-circuit evaluation, 85
- slice, array, 297
- source code formatting, 47
- specialization, class, 454
- static binding, 465
- steam manipulator, 116
- string concatenation, 345
- string objects, 342
- struct, 418
- subclassing, 454
- switch statement, 147

- template instantiation, 497
- template type parameter, 496
- template value parameters, 503
- templates, 495
- this, 396
- throws, 528
- try block, 529
- try/catch block, 528
- Two's complement binary, 567
- two-dimensional array, 592, 593
- two-dimensional vector, 289
- type parameter, 496
- typedef, 290, 594

undefined behavior, 279
underflow, 568

value parameters, template, 503
variable aliasing with references, 254
vector, 273
vector initializer list, 273
vtable, 467

while statement, 110

xor, bitwise, 576